

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

Database Question Bank With Answers

CSE 303 · CSE 215 · Database

Year Coverage: 2010–11 to 2024–25

Topics: Database Fundamentals, Relational Model, ER Model,
SQL & Query Languages, Normalization, Storage & Indexing,
Query Processing, Transaction Management, NoSQL & Big Data

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting

NotebookLM — Research & Extraction

Claude AI — $\text{\LaTeX}$ Typesetting

Gemini — Answer & Diagram Generation

Table of Contents

Part I — Database Fundamentals & Database Design ▷ S1. Database Fundamentals · Concepts of Database Systems · Relational Model ▷ S2. Database Design · Entity-Relationship (ER) Model · ER Diagrams · Database Schema Design
Part II — Query Languages ▷ S3. Query Languages · SQL · Relational Algebra · Joins · Constraints, Assertions & Triggers
Part III — Normalization ▷ S4. Normalization · Functional Dependencies · Normal Forms (1NF, 2NF, 3NF, BCNF, 4NF) · Anomalies & Decomposition
Part IV — Storage, Indexing & Query Processing ▷ S5. Storage and Indexing · Data Storage Architecture & RAID · Primary Index · Secondary & Bitmap Index · B⁺ Tree Index · Hash Index ▷ S6. Query Processing · Query Processing · Query Optimization
Part V — Transaction Management & Advanced Topics ▷ S7. Transaction Management · Transactions & ACID Properties · Concurrency Control · Deadlock · Recovery Systems ▷ S8. Advanced Topics

· [Distributed & Parallel Databases](#)

BUET · CSE 215 · Database

Part I

Database Fundamentals & Database Design

Database Fundamentals & Database Design

S1 — Database Fundamentals

Concepts of Database Systems

Q1. Draw the diagram of 2-tier and 3-tier database architectures. (5 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Two-Tier Database Architecture

In a **2-tier architecture**, the system is divided into two distinct layers: the client and the database server. The client machine contains both the user interface and the application business logic, while the database system runs on the server.

Client Tier (Tier 1)
GUI & Application Business Logic
(Fat Client)

↕
Direct Network Connection
(e.g., ODBC / JDBC API)

Database Tier (Tier 2)
Database Management System (DBMS)
(Data Storage & Access)

Properties & Characteristics:

- Working Principle:** The client application directly establishes a connection to the database server using APIs like ODBC or JDBC. Queries are generated by the client and sent directly to the database.
- Advantages:**
 - Simple to design, develop, and deploy.
 - Faster response times for small user groups due to the lack of an intermediate layer.
- Disadvantages:**
 - Scalability Issue:** Performance degrades significantly as the number of concurrent users increases (each requires a dedicated database connection).
 - Security Risk:** The database is directly exposed to client applications.
 - Maintenance:** Application logic changes require updates on every individual client machine.

2. Three-Tier Database Architecture

A **3-tier architecture** decouples the user interface, business logic, and database management into three separate layers. This is the standard architecture for modern web applications.

Presentation Tier (Tier 1)
User Interface (Web Browser / App)
(Thin Client)

↕
Network Protocol
(e.g., HTTP / HTTPS)

Application Tier (Tier 2)
Business Logic & Query Generation
(Web / App Server)

↕
Database Protocol
(e.g., TCP/IP, SQL Access)

Database Tier (Tier 3)
Database Management System (DBMS)
(Data Storage)

Properties & Characteristics:

- Working Principle:** The client sends a request to the application server. The application server evaluates the business rules, forms the necessary database queries, executes them against the database server, and finally sends the processed result back to the client.

5

- **Advantages:**
 - **High Scalability:** Connection pooling at the application layer allows thousands of clients to share a few database connections.
 - **Improved Security:** The database is shielded from direct client access. The application layer acts as a secure gateway.
 - **Maintainability:** Business logic updates are made centrally on the application server without altering client software.
- **Disadvantages:**
 - More complex to design, configure, and maintain.
 - Slight network overhead due to the intermediate routing through the application layer.

3. Comparison Summary

Feature	2-Tier Architecture	3-Tier Architecture
Client Type	Fat client (contains application logic)	Thin client (primarily presentation)
Scalability	Poor (difficult to scale beyond 100 users)	Excellent (handles thousands of users)
Security	Low (direct database access)	High (database hidden behind app server)
Maintenance	High effort (update every client device)	Low effort (centralized updates)
Use Case	Legacy desktop software, internal enterprise tools	Web applications, cloud services, e-commerce

Q2. Why do we use data abstraction? Describe the different data abstraction levels. (8 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Definition and Purpose of Data Abstraction

Data Abstraction is the process of hiding the complex, underlying physical storage details of a database from the end-users and applications, presenting them only with the relevant structural view of the data.

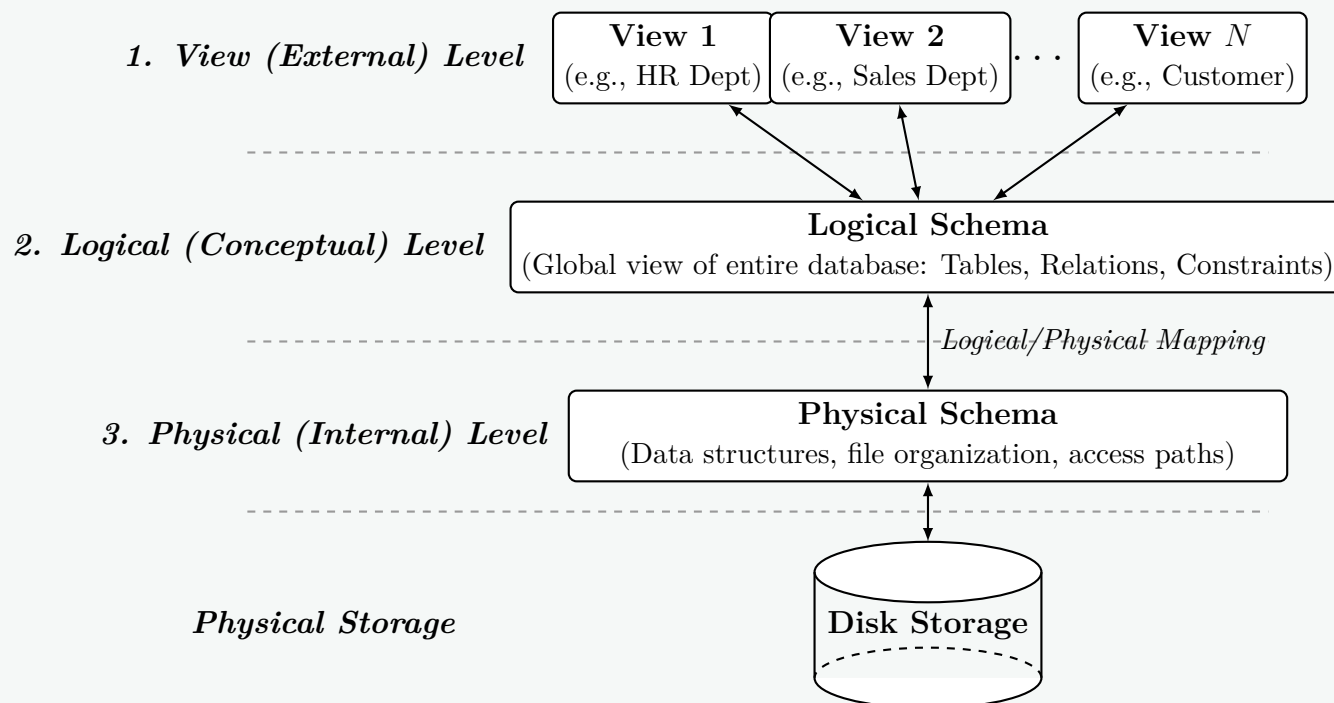
In a Database Management System (DBMS), highly complex data structures (like B+ trees, hash tables, and linked lists) are used to store and retrieve data efficiently. Because most database users are not trained to interact with such structures, the DBMS implements data abstraction.

Why do we use Data Abstraction?

- **Complexity Hiding:** End-users and application developers can interact with the database without needing to understand disk blocks, byte offsets, or indexing mechanisms.
- **Security & Authorization:** By restricting what specific users can see (via views), the DBMS prevents unauthorized access to sensitive information.
- **Data Independence:** The ability to change the schema at one level of the database system without having to change the schema at the next higher level.
- **Ease of Access:** It simplifies querying processes by allowing users to use high-level declarative languages like SQL.

2. Architecture of Data Abstraction

Data abstraction is typically implemented using the **Three-Schema Architecture** (also known as the ANSI/SPARC architecture), which divides the database into three distinct levels.



3. Detailed Description of Abstraction Levels

(a) Physical (Internal) Level:

- **Definition:** The lowest level of abstraction. It describes *how* the data is actually stored in the underlying storage mediums (HDDs/SSDs).
- **Working Principle:** It defines data structures (like B+ trees, Hash maps), file organizations (sequential, indexed), compression techniques, and encryption. This level is managed by the Database Administrator (DBA).
- **Example:** Storing a 'Customer' record as a contiguous block of 256 bytes at a specific memory address, utilizing an ISAM (Indexed Sequential Access Method) index.

(b) Logical (Conceptual) Level:

- **Definition:** The middle level of abstraction. It describes *what* data is stored in the database and the relationships existing among those data.
- **Working Principle:** It represents the entire database logically. Programmers and DBAs work at this level using Data Definition Language (DDL). There is no hardware awareness at this level.
- **Example:** Defining a relational table 'Customer(ID, Name, Account_Balance)' with a primary key on 'ID' and a foreign key linking to a 'Transactions' table.

(c) View (External) Level:

- **Definition:** The highest level of abstraction. It describes only *part* of the entire database.
- **Working Principle:** A database can have multiple views tailored for different user groups. It simplifies interaction by hiding data that is not relevant to the user and masking sensitive information.
- **Example:** A bank teller viewing a customer's 'Name' and 'Account_Balance', while the conceptual level also holds the customer's 'Encrypted_Password' and 'SSN', which the teller is completely unaware of.

Q3. Why does it yield current values when derived attributes are not stored in the database? Discuss with an example. **(10/7 marks)**
 [CSE 215 | L-2/T-1 | 2023–24] [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Definition and Concept

A **Derived Attribute** is an attribute in a database whose value is not permanently stored in the physical storage. Instead, its value is dynamically calculated (or derived) from one or more related **base attributes** (which are physically stored) or system functions (such as the current system time) whenever the attribute is queried.

2. Why Derived Attributes Yield Current Values

Derived attributes yield the most current and accurate values because of their **on-the-fly computation** mechanism. The underlying reasons include:

- **Runtime Calculation:** The DBMS evaluates the derived value at the exact moment the query is executed. It does not read a stale value from disk.
- **Dependency on Current State:** The derivation formula relies on the present state of the base attributes. If a base attribute changes, the derived attribute instantly reflects this change.

- **Integration with System State:** Many derived attributes depend on the system clock (e.g., `CURRENT_DATE`). As time progresses, the calculation naturally factors in the new time, ensuring the result is always up to date.
- **Elimination of Update Anomalies:** If a dynamic value (like age or years of service) were stored statically, a background job would be required to update every record periodically. Skipping this update would lead to stale, incorrect data. Derivation completely avoids this synchronization issue.

3. Illustrative Example

Consider a university database tracking students.

- **Base Attribute (Stored):** `Date_of_Birth` (e.g., '2002-05-15').
- **Derived Attribute (Not Stored):** `Age`.

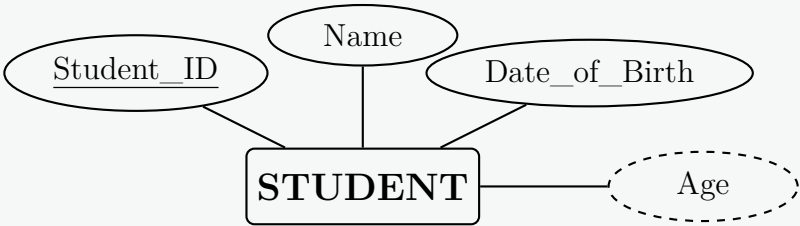
If `Age` were physically stored as '20' in the year 2022, it would become incorrect (stale) by the year 2024. However, by treating `Age` as a derived attribute, the DBMS computes it dynamically using the formula:

$$\text{Age} = \text{Current_Date} - \text{Date_of_Birth}$$

Because `Current_Date` advances automatically, executing this query in 2026 will accurately and mathematically yield an age of 24, without a single write operation being made to the database since the student's insertion.

4. ER Diagram Representation

In an Entity-Relationship (ER) model, derived attributes are denoted by a **dashed ellipse**. The base attributes are denoted by a standard solid ellipse.



5. Implementation (SQL View or Query)

In standard relational databases, derived attributes are typically implemented using expressions in a `SELECT` statement or encapsulated within a **View**.

```
-- Creating a view to continuously expose the derived 'Age' attribute
CREATE VIEW Student_Current_Profile AS
SELECT
    Student_ID,
    Name,
    Date_of_Birth,
    -- Derivation calculation happens here on the fly
    FLOOR(DATEDIFF(DAY, Date_of_Birth, CURRENT_DATE) / 365.25) AS Age
FROM
    STUDENT;
```

6. Comparison Summary

Property	Stored Attribute	Derived Attribute
Storage	Occupies physical disk space.	Does not occupy disk space.
Accuracy over time	Becomes stale if not updated continuously.	Always accurate; dynamically computes current state.
Performance	Faster retrieval (simple disk read).	Slower retrieval (requires CPU computation time).
Redundancy	Introduces redundancy if logically dependent on other attributes.	Completely eliminates logical redundancy.

Q4. What are the requirements of an ideal DBMS? Define entity and attribute in the context of database with suitable example(s). (10 marks)

CSE 215 | L-2/T-2 | 2018–19

Answer

1. Requirements of an Ideal DBMS

An ideal Database Management System (DBMS) must provide a robust, secure, and efficient environment for storing and retrieving data. The primary requirements include:

- **Data Integrity and Consistency:** The system must enforce constraints (e.g., primary keys, foreign keys, domain constraints) to ensure data remains accurate and valid across all tables, even after multiple transactions.
- **Concurrency Control:** It must allow multiple users to access and modify data simultaneously without causing data corruption or inconsistencies, typically achieved by enforcing ACID (Atomicity, Consistency, Isolation, Durability) properties.
- **Data Security and Authorization:** The DBMS must restrict unauthorized access, ensuring that only authenticated users with specific privileges can view or manipulate sensitive data.
- **Crash Recovery and Resilience:** In the event of a hardware or software failure, the system must be able to restore the database to a consistent state without losing any committed data.
- **Data Independence:** The architecture should allow changes in the physical storage layer without affecting the logical schema (Physical Data Independence), and changes in the logical schema without rewriting application programs (Logical Data Independence).
- **Performance and Scalability:** It must efficiently process complex queries over massive datasets and scale seamlessly as the volume of data and the number of concurrent users increase.
- **High-Level Query Language:** It should provide a declarative interface (like SQL) allowing users to retrieve data easily without needing to understand complex physical access paths.

2. Definition of Entity and Attribute

In the context of database design and the Entity-Relationship (ER) model:

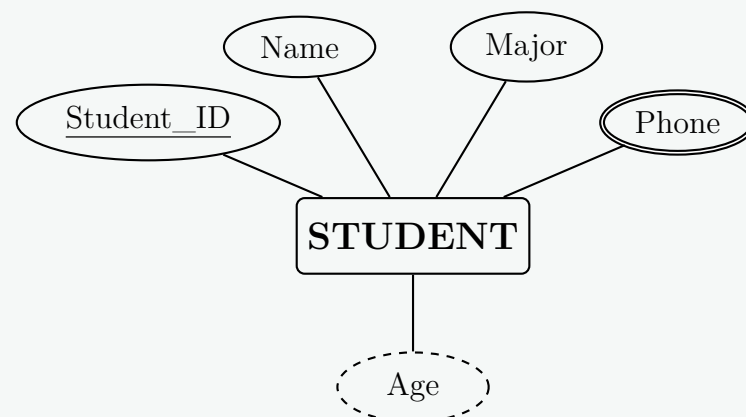
- **Entity:** An entity is a distinguishable real-world object, concept, or event that exists independently and about which data needs to be collected and stored. Entities are mapped to **Tables** (or relations) in a relational database.
- **Attribute:** An attribute is a characteristic, property, or trait that describes an entity. It holds specific pieces of information about the entity. Attributes are mapped to **Columns** in a relational database.

3. Suitable Example and ER Diagram

Let us consider a university database environment:

- **Entity:** STUDENT (represents a physical person).
- **Attributes of STUDENT:**
 - **Student_ID:** A *key attribute* (uniquely identifies the student).
 - **Name:** A *simple attribute*.
 - **Phone:** A *multivalued attribute* (a student can have multiple phone numbers).
 - **Age:** A *derived attribute* (calculated from Date of Birth).

The following ER Diagram visually represents the **STUDENT** entity and its associated attributes based on standard ER notation:



4. Comparison Table (Entity vs. Attribute)		
Feature	Entity	Attribute
Definition	A distinct real-world object or concept.	A property or characteristic describing an object.
Relational Mapping	Becomes a Table .	Becomes a Column in that table.
ER Diagram Notation	Represented by a Rectangle .	Represented by an Ellipse .
Example	‘Car’, ‘Employee’, ‘Order’	‘Color’, ‘Salary’, ‘Order_Date’

Q5. What are bags in DBMS? How are they different from sets? Give examples of union, intersection and minus operation on bags to highlight the differences. (15 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Definition of Bags (Multisets) in DBMS

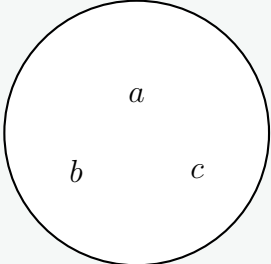
In the context of a Database Management System (DBMS), a **bag** (also known as a **multiset**) is an unordered collection of elements where **duplicate elements are permitted**.

While pure relational algebra is mathematically based on set theory (where duplicates are strictly prohibited), commercial database systems (like SQL) inherently operate on bags by default. This is because eliminating duplicates is a computationally expensive operation (requiring sorting or hashing). Therefore, unless a user explicitly requests duplicate removal (e.g., using the **DISTINCT** keyword), a DBMS will output a bag of tuples.

2. Difference Between Bags and Sets

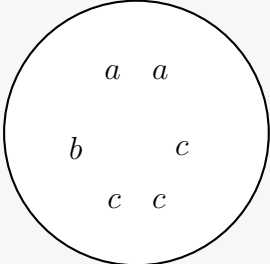
- Sets:** A set contains distinct elements. An element x either belongs to a set S or it does not. The cardinality (count) of any element in a set is at most 1.
- Bags:** A bag can contain multiple identical copies of the same element. An element x belongs to a bag B with a specific count or frequency, denoted as $C(x, B) \geq 0$.

Set



Unique elements only

Bag (Multiset)



Duplicates are permitted

3. Operations on Bags vs. Sets

Let $C(x, A)$ represent the number of occurrences of an element x in a bag A . Consider two bags:

- Bag $A = \{a, a, a, b, b, c\}$
- Bag $B = \{a, a, b, b, b, d\}$

Therefore, element counts are: $C(a, A) = 3$, $C(b, A) = 2$, $C(c, A) = 1$; and $C(a, B) = 2$, $C(b, B) = 3$, $C(d, B) = 1$.

A. Bag Union ($\cup$)

In a bag union, the count of an element in the result is the **sum** of its counts in the two bags.

$$C(x, A \cup B) = C(x, A) + C(x, B)$$

- Bag Result:** $A \cup B = \{a, a, a, a, a, b, b, b, b, b, c, d\}$
- Comparison to Set Union:** A strict set union of A and B would simply be $\{a, b, c, d\}$, completely ignoring frequencies.

B. Bag Intersection ($\cap$)

In a bag intersection, the count of an element in the result is the **minimum** of its counts in the two bags.

$$C(x, A \cap B) = \min(C(x, A), C(x, B))$$

- Bag Result:** $A \cap B = \{a, a, b, b\}$
- Explanation:** a appears three times in A and two times in B ; the minimum is two. b appears twice in A and three times in B ; the minimum is two. c and d do not appear in both, so their minimum count is zero.

10

C. Bag Difference / Minus (−)

In a bag difference, the count of an element in the result is the count in the first bag **minus** the count in the second bag, floored at zero (an element cannot have a negative count).

$$C(x, A - B) = \max(0, C(x, A) - C(x, B))$$

- **Bag Result:** $A - B = \{a, c\}$
- *Explanation:* For a : $\max(0, 3 - 2) = 1$. For b : $\max(0, 2 - 3) = 0$. For c : $\max(0, 1 - 0) = 1$.

4. Comparison Summary Table

Feature	Set Semantics	Bag (Multiset) Semantics
Duplicates	Strictly not allowed.	Allowed and maintained.
Union ($A \cup B$)	Element exists if it is in A or B .	Sum of occurrences: $C(A) + C(B)$.
Intersection ($A \cap B$)	Element exists if it is in both A and B .	Minimum of occurrences: $\min(C(A), C(B))$.
Difference ($A - B$)	Element exists if it is in A but not B .	Remaining occurrences: $\max(0, C(A) - C(B))$.
Performance	Slower (requires duplicate elimination).	Faster (simply streams tuples).

Q6. What are the key properties of a DBMS? (8 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Introduction

A Database Management System (DBMS) is designed to manage large bodies of information efficiently and securely. To achieve this, it possesses several key properties that distinguish it from traditional file-processing systems, making it the standard for modern data management.

2. Fundamental Properties of a DBMS

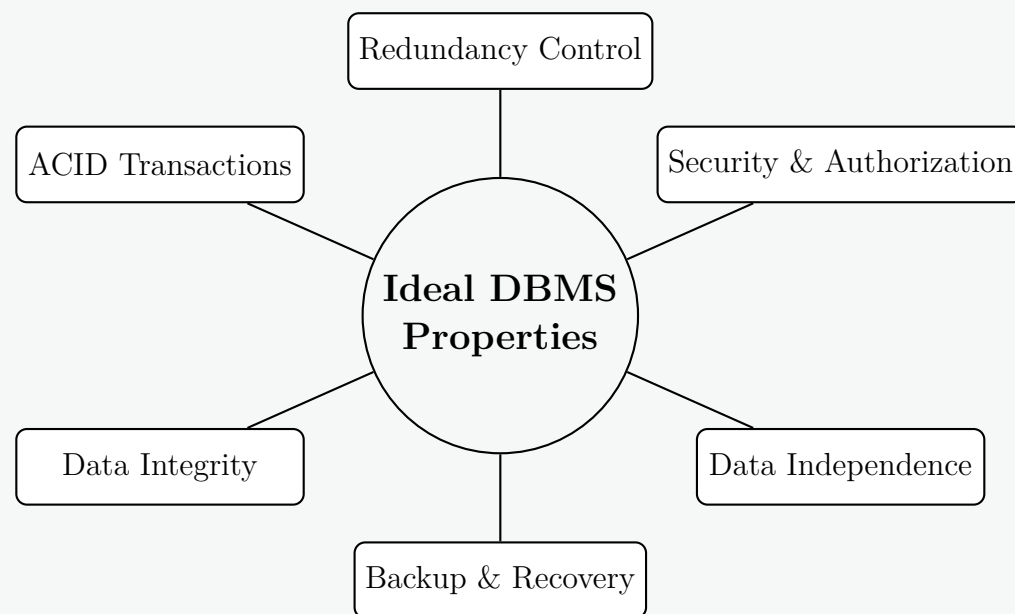
- **Control of Data Redundancy:** Unlike flat-file systems where duplicate data is scattered across multiple locations, a DBMS integrates data into a central repository, minimizing duplication and preventing data inconsistency.
- **Data Security and Authorization:** A DBMS enforces security by ensuring that only authorized users can access or modify the data. It supports the creation of user accounts, roles, and granular access privilege configurations.
- **Data Independence:** The architecture provides a strict separation between application programs and the physical data storage.
 - *Logical Data Independence:* The ability to change the conceptual schema without altering application programs.
 - *Physical Data Independence:* The ability to change the underlying storage structure without altering the conceptual schema.
- **Concurrency Control:** When multiple users or programs attempt to access the same data simultaneously, the DBMS systematically manages these interactions (usually via locking mechanisms) to prevent data corruption.
- **Backup and Recovery:** A DBMS provides robust, automatic mechanisms to back up data and recover it to a consistent state in the event of hardware crashes or software failures.
- **Data Integrity:** The system enforces structural rules and constraints (such as primary keys, foreign keys, and domain constraints) to ensure all data stored is structurally accurate and functionally valid.

3. Transaction Properties (ACID)

At the core of a robust relational DBMS is its ability to process database transactions reliably. These required transactional properties are summarized by the **ACID** acronym:

- (a) **Atomicity:** Often called the "all-or-nothing" rule. A transaction is treated as a single indivisible logical unit of work; either all its operations are executed successfully, or none are.
- (b) **Consistency:** A transaction must reliably transition the database from one valid, consistent state to another, strictly adhering to all defined database rules and constraints.
- (c) **Isolation:** Concurrent transactions execute completely independent of one another. Intermediate, uncommitted states of one transaction are invisible to other concurrent transactions.
- (d) **Durability:** Once a transaction is successfully committed, its changes are permanently recorded in non-volatile storage and will survive any subsequent system failures or power losses.

4. Visual Summary of Key Properties



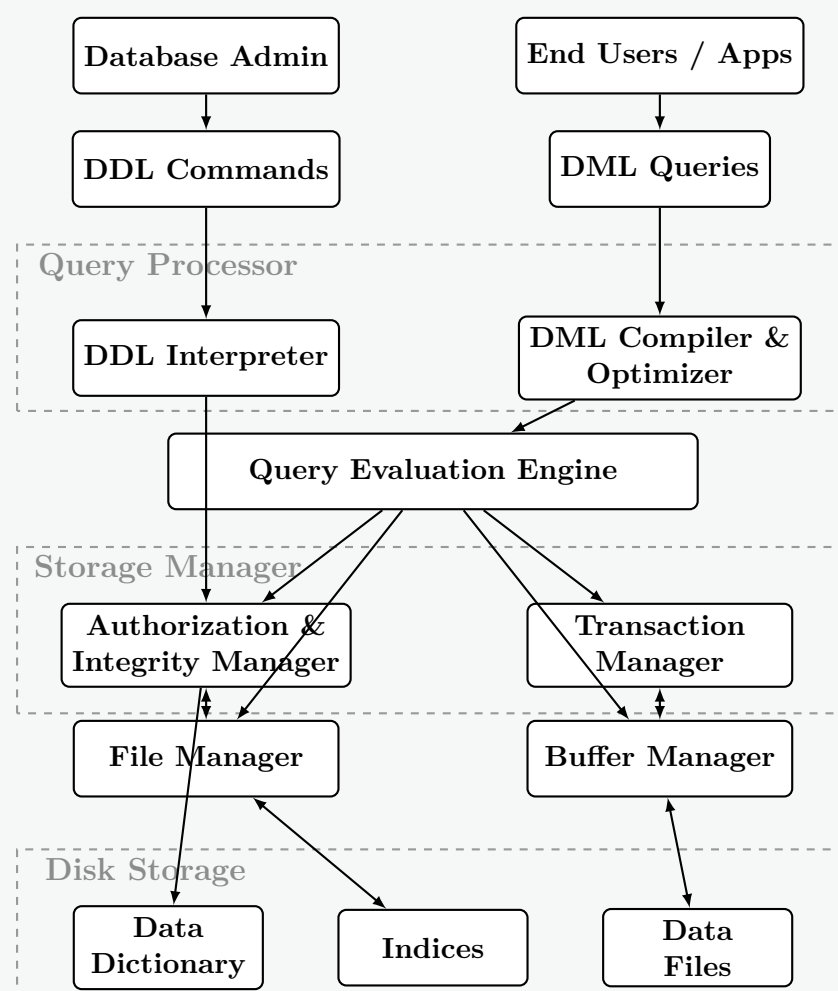
Q7. Show the major components and their interactions in a DBMS. Briefly explain the functionalities of major components. **(15 marks)**
 [CSE 303 | L-3/T-1 | 2011–12]

Answer

1. DBMS Architecture and Component Interactions

A Database Management System (DBMS) is highly complex and is divided into multiple modular components that interact to execute queries, manage storage, and ensure data integrity. The architecture is broadly categorized into the **Query Processor**, the **Storage Manager**, and the **Disk Storage**.

The diagram below illustrates the major components and the flow of instructions (interactions) from the user level down to the physical disk.



2. Functionalities of Major Components

The DBMS is divided into three primary functional blocks:

A. Query Processor

The query processor handles incoming queries, translates them into low-level instructions, and optimizes them for performance.

- **DDL Interpreter:** Parses Data Definition Language (DDL) statements (like `CREATE TABLE`) and records the definitions in the data dictionary.
- **DML Compiler and Query Optimizer:** Translates Data Manipulation Language (DML) statements (like `SELECT`, `UPDATE`) into an execution plan. The optimizer evaluates various execution strategies and selects the most efficient one based on cost (CPU, I/O).
- **Query Evaluation Engine:** Executes the low-level instructions generated by the DML compiler, coordinating with the storage manager to retrieve the required records.

B. Storage Manager

The storage manager acts as the interface between the low-level data on the disk and the application queries. It is responsible for efficient storage, retrieval, and updating of data.

- **Authorization and Integrity Manager:** Verifies that the user issuing the query has the correct permissions and ensures that the operation does not violate any database constraints (e.g., primary/foreign keys).
- **Transaction Manager:** Ensures that the database remains in a consistent state despite system failures. It controls concurrent access, ensuring transactions adhere to ACID properties.
- **File Manager:** Manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager:** Responsible for fetching data from disk storage into main memory (RAM) and deciding what data to cache to minimize expensive disk I/O operations.

C. Disk Storage (Data Structures)

This represents the physical layer where the information is permanently written.

- **Data Files:** The actual files residing on the disk that store the database records.
- **Data Dictionary (Metadata):** Stores data about the data, including table schemas, constraints, user permissions, and statistical data used by the query optimizer.
- **Indices:** Auxiliary data structures (such as B+ Trees or Hash tables) that provide fast access to data items based on key values, bypassing sequential scans.

Q8. Describe the key properties that give a DBMS edge over a file system. (14 marks)

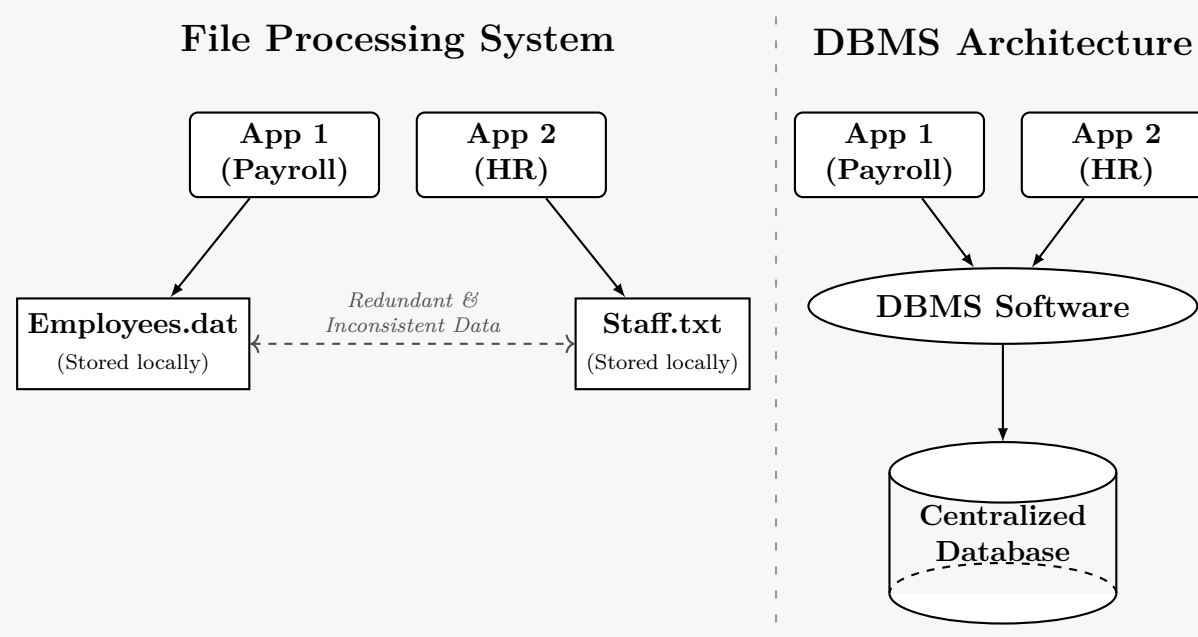
[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Introduction

Before the advent of Database Management Systems (DBMS), data was stored using traditional file-processing systems where each application maintained its own set of files. This approach led to severe limitations as data grew in size and complexity. A DBMS provides a centralized, robust architecture that overcomes these limitations, offering significant advantages in data handling, security, and performance.

2. Visual Comparison: Architecture



3. Key Properties That Give DBMS an Edge

The decisive edge of a DBMS over file-based systems stems from the following core properties:

(a) Data Abstraction and Independence:

- *File System:* Application code is tightly coupled to the physical structure of the files. Adding a new field to a text file requires rewriting every application that reads it.
- *DBMS:* Provides logical and physical **data independence**. Users interact with a logical view of the data. The underlying storage mechanisms (like switching from HDDs to SSDs or altering indexing methods) can change without impacting the application code.

(b) Control of Data Redundancy and Inconsistency:

- *File System:* Data is fragmented across multiple departments, leading to massive duplication (redundancy). If an employee’s address changes, it might be updated in the Payroll file but forgotten in the HR file (inconsistency).
- *DBMS:* Integrates and centralizes data via **normalization**. Duplication is minimized, ensuring that a single update reflects universally, maintaining global data consistency.

(c) High-Level Data Access (Query Languages):

- *File System:* Extracting specific data requires writing custom procedural scripts (e.g., writing a Python script to parse a CSV and filter records).
- *DBMS:* Offers declarative query languages like **SQL**. A user merely describes *what* data is needed (e.g., `SELECT * FROM Employees WHERE Salary > 50000`), and the DBMS optimizer dynamically determines the most efficient way to fetch it using built-in access paths and indices.

(d) Enforcement of Data Integrity:

- *File System:* Integrity constraints (e.g., "Age must be > 18" or "Department ID must exist") must be manually coded into every single application.
- *DBMS:* Constraints (Primary Keys, Foreign Keys, Domain constraints) are centralized within the database schema. The DBMS strictly rejects any transaction that violates these rules, guaranteeing valid data.

(e) Concurrency Control and ACID Transactions:

- *File System:* Employs coarse-grained locking. If one user opens a file to edit a single line, the entire file is often locked, preventing concurrent access by others.
- *DBMS:* Provides fine-grained (row-level) locking and guarantees **ACID** (Atomicity, Consistency, Isolation, Durability) properties. Thousands of users can read and write simultaneously without interfering with each other (Isolation), and transactions are treated as indivisible units of work (Atomicity).

(f) Crash Recovery (Durability):

- *File System:* A power failure mid-write can corrupt the file, requiring manual restoration from a backup.
- *DBMS:* Uses Write-Ahead Logging (WAL) and undo/redo mechanisms. If a crash occurs, the DBMS automatically recovers, rolling back incomplete transactions to ensure the database remains in a consistent state.

4. Comparison Summary

Feature	Traditional File System	DBMS
Data Redundancy	Very high (scattered duplicate data).	Minimized (centralized and normalized).
Data Independence	Low (applications depend on file structures).	High (abstracts physical storage).
Querying	Difficult (requires custom procedural code).	Easy (declarative languages like SQL).
Concurrency	Poor (file-level locking limits multi-user access).	Excellent (row-level locking and isolation protocols).
Data Integrity	Hard-coded in external applications.	Centralized enforcement via schema constraints.
Security	Coarse OS-level permissions (Read/Write file).	Granular role-based access control (RBAC).

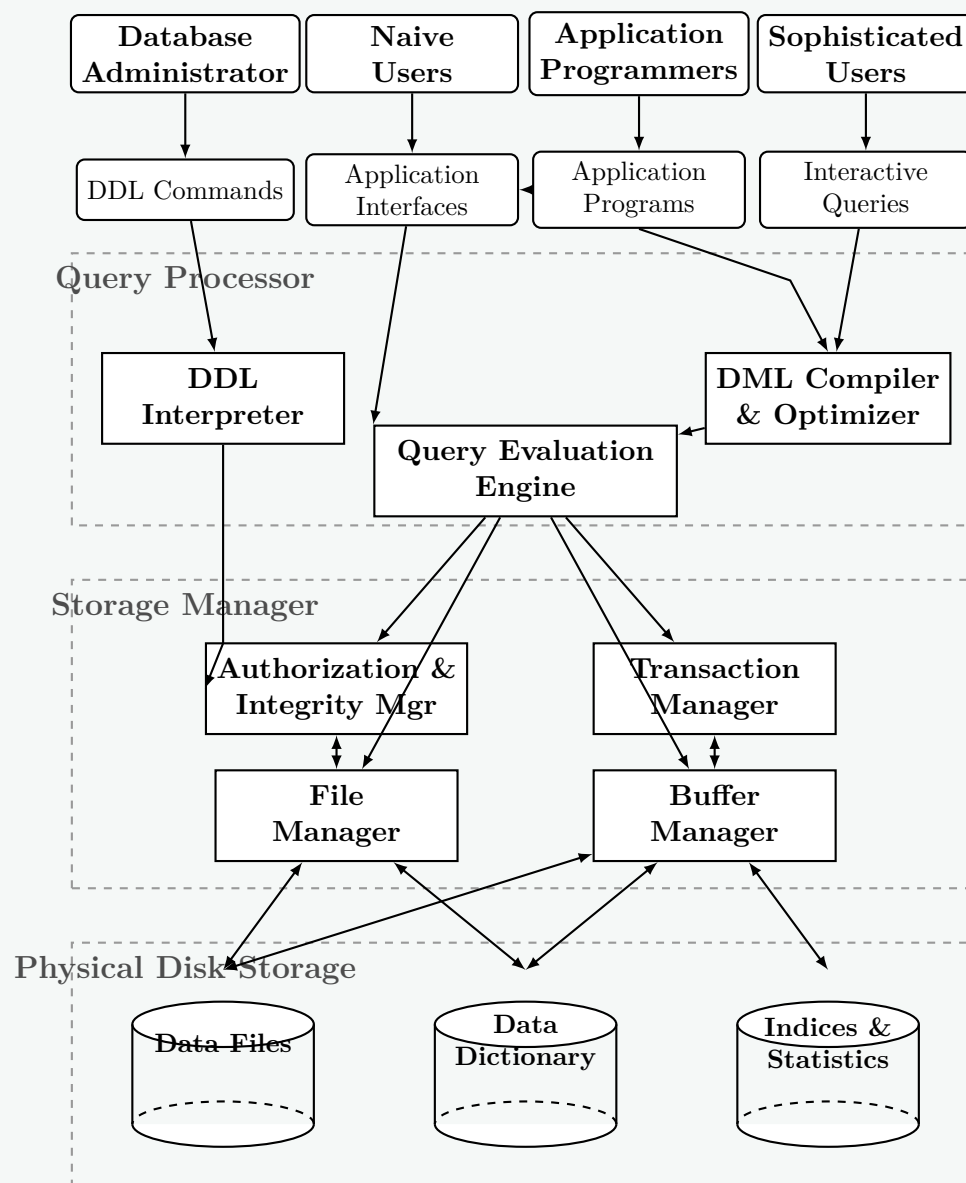
Answer

1. Overview of DBMS Architecture

A Database Management System (DBMS) is a complex software system tasked with managing, storing, and retrieving data efficiently and securely. To handle these responsibilities, the architecture of a DBMS is partitioned into distinct functional modules. The overall block diagram is primarily divided into three major layers:

- **Users and Interfaces:** The top layer where various users interact with the system.
- **Query Processor:** The brain of the DBMS that translates high-level user queries into low-level execution plans.
- **Storage Manager:** The interface between the query processor and the physical disk storage, responsible for efficient data handling and concurrency.

2. DBMS Block Diagram



3. Description of Major Components

A. Users and Interfaces

- **Database Administrator (DBA):** Interacts using DDL (Data Definition Language) to define the database schema, grant permissions, and configure storage.
- **Naive Users:** Interact through graphical application interfaces (e.g., banking tellers) without knowing underlying queries.
- **Application Programmers:** Write application code interacting with the database using embedded DML (Data Manipulation Language).
- **Sophisticated Users:** Use interactive query tools (like SQL Plus or pgAdmin) to directly request data.

B. Query Processor

- **DDL Interpreter:** Parses DDL statements and updates the system metadata (Data Dictionary) with schema definitions.
- **DML Compiler & Optimizer:** Translates DML queries into an evaluation plan. The **optimizer** determines the most efficient way to execute a query based on disk access costs and available indices.
- **Query Evaluation Engine:** Executes the low-level instructions generated by the compiler against the storage manager.

C. Storage Manager

- **Authorization & Integrity Manager:** Ensures the user has the required privileges to execute the query and checks that no integrity constraints (like primary/foreign keys) are violated.
- **Transaction Manager:** Ensures the database remains in a consistent state (ACID properties) despite system failures and manages concurrent transaction executions.
- **File Manager:** Manages the allocation of space on the disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager:** Responsible for caching data in main memory (RAM). It decides what disk pages to bring into memory and what pages to evict, minimizing slow disk I/O.

D. Disk Storage (Data Structures)

- **Data Files:** Store the actual database records.
- **Data Dictionary (Metadata):** Stores data about the database structure, schemas, user permissions, and constraints.
- **Indices:** Specialized data structures (e.g., B+ Trees) that speed up the retrieval of specific records without sequentially scanning the entire data file.

Q10. Discuss ‘Schema versus Data’ and ‘DDL versus DML’. (7 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Schema versus Data

In database design, it is critical to distinguish between the logical description of the database and the actual information it contains. This distinction is broadly referred to as **Schema** versus **Data** (or Instance).

- **Database Schema (Intension):** The overall design, structure, and logical blueprint of the database. It is defined during the database design phase and is expected to remain relatively static. It specifies the tables, attributes, data types, constraints, and relationships.
- **Database Data / Instance (Extension):** The actual collection of data stored in the database at a specific moment in time. The instance changes continuously as records are inserted, updated, or deleted during routine operations.

Analogy: A schema is like the architectural blueprint of a house (showing where the rooms and doors are), while the data/instance represents the actual furniture and people residing in the house at any given time.

Schema (Structure / Intension)

EMPLOYEE (Emp_ID INT, Name VARCHAR, Salary REAL)

Populates over time

Data / Instance (Records / Extension)

Emp_ID	Name	Salary
101	Alice	75000
102	Bob	68000
103	Carol	82000

Feature	Database Schema	Database Data (Instance)
Definition	The logical structure and rules of the database.	The actual content stored within the structure.
Frequency of Change	Rarely changes (static).	Changes constantly (dynamic).
Metadata	Stored in the database catalog (Data Dictionary).	Stored in the data files on the disk.
Terminology	Intension.	Extension.

2. DDL versus DML

To interact with both the schema and the data, a DBMS provides specific categories of SQL languages: **Data Definition Language (DDL)** and **Data Manipulation Language (DML)**.

16

- **DDL (Data Definition Language):** Used by database designers and administrators to specify, modify, or drop the **Database Schema**. Execution of DDL statements directly affects the Data Dictionary and is typically auto-committed (cannot be rolled back in standard relational systems).
 - *Common Commands:* CREATE, ALTER, DROP, TRUNCATE.
- **DML (Data Manipulation Language):** Used by users and applications to retrieve, insert, delete, and modify the actual **Data (Instance)** stored within the tables. DML commands form the basis of transactional processing.
 - *Common Commands:* SELECT, INSERT, UPDATE, DELETE.

Example Code Demonstration

```
-- === DDL: Defining the Schema ===
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

-- === DML: Manipulating the Data / Instance ===
INSERT INTO Employee (Emp_ID, Name, Salary) VALUES (101, 'Alice', 75000);
UPDATE Employee SET Salary = 80000 WHERE Emp_ID = 101;
SELECT * FROM Employee;
```

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Target	Operates on the Schema (metadata).	Operates on the Data (instances/records).
Primary Users	Database Administrators (DBA), Database Designers.	End Users, Application Programs.
Transaction Control	Usually auto-committed (cannot be rolled back).	Controlled by transaction blocks (can be rolled back).
Effect	Alters the structure (e.g., adding a column).	Alters the values (e.g., changing a salary).

Q11. Explain the data dictionary schema for DBMS. List the advantages and disadvantages for storing a relational database as follows:

- (a) Store each relation in one file.
- (b) Store multiple relations in one file.

(10 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Data Dictionary Schema for DBMS

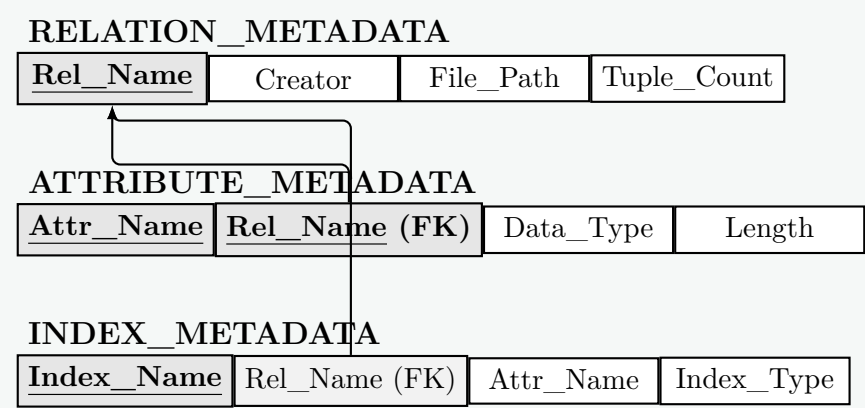
Definition: The data dictionary (also known as the system catalog) is an internal database maintained by the DBMS to store **metadata** that is, "data about the data." It acts as a comprehensive directory detailing the logical and physical structure of the entire database system.

Working Principle: The DBMS itself accesses the data dictionary constantly. Before executing any query (DML or DDL), the Query Processor consults the data dictionary to verify that relations exist, attributes match, users have appropriate permissions, and to identify available indices for query optimization.

Contents of a Data Dictionary: To manage this metadata, the DBMS stores the data dictionary as a set of relational tables. The schema of these system tables typically includes:

- **Relation Information:** Names of all tables, views, their creators, and physical storage paths.
- **Attribute Information:** Names of attributes for each table, their data types, lengths, and constraints (e.g., NOT NULL).
- **Index Information:** Index names, the attributes they index, and the type of index (e.g., B+ tree, Hash).
- **Integrity Constraints:** Primary keys, foreign key references, and check constraints.
- **User and Authorization Data:** Usernames, passwords, and access privileges (Grants).
- **Statistical Data:** Number of tuples in a relation, distribution of values, used heavily by the Query Optimizer.

Visual Representation of a Simplified Data Dictionary Schema



2. Storing the Relational Database: File Strategies

A DBMS needs a mapping between logical relations (tables) and physical operating system files. Two primary strategies exist, each with specific trade-offs.

Strategy 1: Store each relation in one separate file

In this approach, every table in the database maps directly to its own isolated file on the disk (e.g., `Employee` maps to `employee.dat`, `Department` to `department.dat`).

Each Relation in One File	
Advantages	• Simplicity: The mapping between the database conceptual schema and physical files is extremely straight-forward. • OS-Level Security: The DBMS can leverage underlying Operating System file-level permissions to restrict access to specific tables. • Easy Maintenance: Backing up, copying, or moving individual tables is very simple (just copy the specific file). • No Mixed-Record Overhead: Because a file contains only one type of record (all the same length or structure), calculating record offsets and managing free space is mathematically simpler.
Disadvantages	• OS Bottlenecks: If a database contains thousands of tables, it results in thousands of files. Operating systems impose hard limits on the maximum number of simultaneously open file descriptors. • Join Performance Penalty: Executing a JOIN operation between two relations requires the OS to open, read, and buffer multiple separate files concurrently, increasing disk I/O seek times and overhead. • Internal Fragmentation: Small tables might waste disk space because the minimum allocation unit is a full OS disk block.

Strategy 2: Store multiple relations in one file (Clustering)

In this approach, the DBMS consolidates multiple relationsespecially those that are frequently accessed togetherinto a single large, contiguous physical file (often referred to as a multitable clustering file organization).

Multiple Relations in One File	
Advantages	• Optimized Joins (Clustering): Related records from different tables can be stored physically adjacent on the same disk block. For example, a Department record and all its associated Employee records can be read in a single disk I/O fetch, dramatically speeding up JOIN queries. • Overcoming OS Limits: The DBMS uses very few OS files (sometimes just one massive file), completely avoiding OS limitations on the number of open file descriptors. • Space Efficiency: Better packing of blocks reduces internal fragmentation, as small tables share blocks with other tables.
Disadvantages	• High Complexity: The DBMS must build and maintain its own internal file system and buffer manager logic to track where different table records reside within the single file. • Complex Space Reclamation: When records of varying sizes from different tables are deleted, reclaiming the fragmented free space within the file is algorithmically difficult. • Sequential Scan Penalty: Performing a sequential scan on a single table (<code>SELECT * FROM Employee</code>) becomes much slower. The DBMS must read the entire massive file and explicitly filter out the intermingled records belonging to other tables.

Relational Model

Q1. Define super key, candidate key, and primary key with examples. (9 marks)

[CSE 215 | L-2/T-1 | 2024–25]

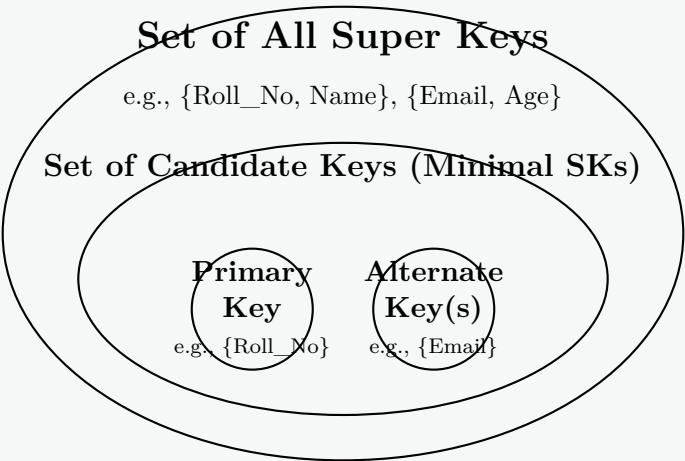
Answer
1. Introduction and Running Example In the relational data model, keys are fundamental constraints used to uniquely identify tuples (rows) within a relation (table) and to establish relationships between different relations. To illustrate these concepts, consider the following relation schema for a university system: STUDENT(<u>Roll_No</u>, Reg_No, Email, Name, Age, Branch) <i>Assumption:</i> Roll_No, Reg_No (Registration Number), and Email are strictly unique for every student. Two students may share the same Name, Age, or Branch. 2. Definitions and Mathematical Properties A. Super Key • Definition: A Super Key is a set of one or more attributes whose combined values uniquely identify a tuple within a relation. • Mathematical Property (Uniqueness): Let R be a relation schema. A subset of attributes $K \subseteq R$ is a super key if the functional dependency $K \rightarrow R$ holds. • Examples: – {Roll_No} – {Reg_No, Name} – {Email, Age, Branch} – {Roll_No, Reg_No, Email, Name, Age, Branch} (The trivial super key comprising all attributes). • Note: Adding any arbitrary attribute to a super key results in another valid super key. B. Candidate Key • Definition: A Candidate Key is a minimal super key. It is a super key such that no proper subset of its attributes also constitutes a super key. • Mathematical Property (Minimality): K is a candidate key if K is a super key, and for any proper subset $K' \subset K$, K' is <i>not</i> a super key. • Examples: – {Roll_No} (Minimal, removing it leaves the empty set). – {Reg_No} – {Email} • <i>Why is {Reg_No, Name} NOT a Candidate Key?</i> Because its proper subset {Reg_No} is already a super key. Thus, it violates the minimality condition.

C. Primary Key

- **Definition:** The Primary Key is the specific candidate key selected by the database designer to principally identify tuples in the relation.
- **Properties:**
 - A relation can have **only one** primary key.
 - Its attributes cannot contain NULL values (Entity Integrity Constraint).
 - Its values should ideally remain immutable (unchanging) over time.
- **Example:** From the candidate keys {Roll_No}, {Reg_No}, and {Email}, the designer selects {**Roll_No**} as the Primary Key.
- **Note:** The unselected candidate keys ({Reg_No} and {Email}) are then referred to as **Alternate Keys**.

3. Visualizing the Relationship

The relationship between these keys can be accurately represented using set theory. All candidate keys are super keys, and the primary key is exactly one of the candidate keys.



4. Comparison Summary

Feature	Super Key	Candidate Key	Primary Key
Definition	Uniquely identifies a tuple.	A minimal super key.	The chosen candidate key.
Quantity	Many per relation.	One or more per relation.	Exactly one per relation.
Minimality	Not required.	Strictly required.	Strictly required.
Null Values	May contain NULLs (if other attributes ensure uniqueness).	Cannot contain NULLs (in practice).	Strictly prohibits NULLs.
Subset Relation	The superset of all keys.	A subset of Super Keys.	A subset of Candidate Keys.

Q2. Suppose we have two relations A and B with the same set of attributes. Relation A has n_a tuples and relation B has n_b tuples. For each of the following relational algebra expressions, give the minimum and maximum number of tuples that can appear in the result. Assume that the produced results are bags, not sets.

- (a) $A \cap B$
- (b) $\sigma_c(A) - B$
- (c) $A \cup B$
- (d) $A \bowtie_L B$, where $\bowtie_L$ is the left outer join

(8 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Bag Semantics Overview

In relational algebra, standard operations often assume **set semantics** (where duplicates are eliminated). However, real-world Database Management Systems (DBMS) typically use **bag (multiset) semantics**, where duplicates are retained unless explicitly removed.

Let a tuple t appear x times in relation A (where $\sum x = n_a$) and y times in relation B (where $\sum y = n_b$). The output cardinality depends on how the specific bag operation computes the new frequency for each tuple.

—

1. Intersection ($A \cap B$)

- **Working Principle:** Under bag semantics, the intersection retains a tuple t exactly $\min(x, y)$ times.
- **Minimum Tuples: 0.** This occurs when relations A and B are completely disjoint (they share no tuples in common).
- **Maximum Tuples: $\min(\mathbf{n}_a, \mathbf{n}_b)$.** This occurs when all tuples of the smaller relation are entirely contained within the larger relation.

2. Selection and Difference ($\sigma_c(A) - B$)

- **Working Principle:** First, the selection $\sigma_c(A)$ filters A . The number of tuples generated by $\sigma_c(A)$ can range from 0 to n_a . Next, the bag difference $X - B$ computes the frequency of tuple t as $\max(0, \text{count}(t \in X) - y)$.
- **Minimum Tuples: 0.** This occurs if the selection condition c matches no tuples in A (yielding 0 tuples), OR if all tuples generated by $\sigma_c(A)$ also exist in B in equal or greater quantities, causing them to be entirely subtracted out.
- **Maximum Tuples: $\mathbf{n}_a$.** This occurs if the selection condition c is true for all tuples in A (yielding n_a tuples), AND relations A and B are completely disjoint, meaning no tuples are subtracted by B .

3. Union ($A \cup B$)

- **Working Principle:** Under bag semantics, union does **not** eliminate duplicates (equivalent to UNION ALL in SQL). The frequency of a tuple t in the result is simply $x + y$.
- **Minimum Tuples: $\mathbf{n}_a + \mathbf{n}_b$.**
- **Maximum Tuples: $\mathbf{n}_a + \mathbf{n}_b$.**
- **Note:** Unlike set union (which can range from $\max(n_a, n_b)$ to $n_a + n_b$), bag union always yields exactly the sum of the cardinalities of the two input relations.

4. Left Outer Join ($A \bowtie_L B$)

- **Working Principle:** Because A and B have the *same set of attributes*, the join condition evaluates all attributes for equality (Natural Left Outer Join).
 - If a tuple in A finds matching tuples in B ($y > 0$), it produces $x \times y$ copies.
 - If it finds no match ($y = 0$), the Left Outer Join ensures the tuple from A is preserved exactly x times, padded with NULLs (though here, all attributes overlap).

Mathematically, the total tuples are $\sum(x \cdot \max(y, 1))$.

- **Minimum Tuples: $\mathbf{n}_a$.** This occurs either when A and B are completely disjoint (every tuple in A gets preserved exactly once with no matches), or when every tuple in A matches exactly one tuple in B .
- **Maximum Tuples: $\max(\mathbf{n}_a, \mathbf{n}_a \times \mathbf{n}_b)$.** If $n_b > 0$, the maximum is $n_a \times n_b$. This explosion occurs when every tuple in A is identical, and every tuple in B is also identical to the tuples in A , resulting in a full Cartesian product. If B is empty ($n_b = 0$), the maximum is simply n_a .

Summary Table

Relational Algebra Expression	Minimum Tuples	Maximum Tuples
$A \cap B$	0	$\min(n_a, n_b)$
$\sigma_c(A) - B$	0	n_a
$A \cup B$	$n_a + n_b$	$n_a + n_b$
$A \bowtie_L B$	n_a	$\max(n_a, n_a \cdot n_b)$

Q3. Should a primary key have any semantic meaning? Explain your answer. **(6 marks)**

[CSE 303 | L-3/T-1 | 2014–15]

Answer**1. Direct Answer**

Ideally, **no**. A primary key should generally **not** have any semantic (real-world business) meaning.

In database design, keys are divided into two categories based on meaning:

- **Natural Key (Semantic):** An attribute that has a logical relationship to the business data (e.g., Social Security Number, Email Address, Vehicle Identification Number).

- **Surrogate Key (Meaningless):** An artificially generated attribute with no real-world meaning, created solely to identify a tuple uniquely (e.g., an Auto-Incrementing Integer, UUID/GUID).

Best practices in modern database design strongly favor the use of **Surrogate Keys** for primary keys, while applying **UNIQUE** constraints to Natural Keys.

2. Why Semantic Primary Keys are Problematic

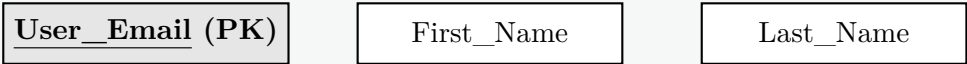
Using a semantic attribute as a primary key introduces several risks to the stability and performance of a relational database:

- (a) **Mutability (Data Changes):** Semantic data is tied to business rules and human behavior, meaning it is subject to change. For example, if an **Email** is used as a primary key and the user changes their email address, the database must perform a cascading update to every foreign key in every related table (e.g., Orders, Logs, Profiles). A surrogate key, being meaningless, never needs to change.
- (b) **Format and Business Rule Changes:** Business rules evolve. A 9-digit alphanumeric product code might be expanded to 12 digits. If this code is a primary key, altering the data type or length across millions of foreign key references is an expensive and risky operation.
- (c) **Reusability:** Some semantic identifiers get recycled over time (e.g., phone numbers or invoice numbers). Reusing a primary key can lead to catastrophic data integrity issues where old, unrelated records accidentally link to new entities.
- (d) **Privacy and Security:** Primary keys are frequently exposed in URLs, log files, and API endpoints. Using sensitive semantic data (like an SSN or phone number) as a primary key leaks personally identifiable information (PII).
- (e) **Performance (Storage and Indexing):** Semantic keys are often long strings (like **VARCHAR(255)** for an email). String comparisons are significantly slower than integer comparisons. Furthermore, B+ Tree indexes built on large strings consume more memory and disk space. A surrogate key, typically a 4-byte or 8-byte integer, is highly optimized for fast **JOIN** operations.

3. Visualizing the Recommended Approach

Instead of forcing a semantic attribute to act as the primary key, database designers implement a surrogate primary key and designate the semantic attributes as **Alternate Keys** (using the **UNIQUE** constraint).

Suboptimal Design: Semantic Primary Key



Recommended Design: Meaningless Surrogate Key



4. Comparison: Natural (Semantic) vs. Surrogate (Meaningless) Keys

Feature	Natural Key (Semantic)	Surrogate Key (Meaningless)
Origin	Exists in the real world (e.g., SSN).	System-generated (e.g., Auto-increment).
Immutability	Prone to change if business rules change.	100% Immutable. Never changes.
Performance	Often slow (e.g., long strings).	Very fast (e.g., 32/64-bit integers).
Coupling	Tightly couples DB structure to business rules.	Decouples structure from business logic.
Use Case	Enforce uniqueness (Alternate Key).	Primary Key / Foreign Key relationships.

Notes: While surrogate keys are generally preferred, there is one major exception: **Link Tables** (Junction Tables) used to resolve Many-to-Many relationships. In a link table (e.g., **Enrollment(Student_ID, Course_ID)**), the primary key should be the composite of the two foreign keys, which intrinsically carries semantic meaning regarding the relationship.

Answer

1. Object Identity in SQL

In standard relational databases, tuples (rows) are identified purely by their values, specifically through a **Primary Key**. However, in **Object-Relational Database Management Systems (ORDBMS)** complying with the SQL:1999 standard and later, tuples can be treated as unique objects.

- Object Identity (OID):** When a row is inserted into a “typed table” (a table built upon a user-defined complex data type), the DBMS assigns it a unique, system-generated, and immutable Object Identifier (OID).
- Properties of OID:** Unlike primary keys, an OID has no real-world semantic meaning, does not depend on the object’s attribute values, and never changes even if all the data within the row is updated.

2. Reference Types (REF)

A **Reference Type** is a special data type in SQL that stores an OID. It acts essentially as a strongly-typed **pointer** to a specific object (row) in another typed table.

- Working Principle:** Instead of using standard Foreign Keys that store copies of Primary Key values (which requires traditional joins), a REF attribute directly points to the memory/disk location of the referenced object, enabling highly efficient object traversal.
- Scoping:** A reference type is usually constrained using a SCOPE clause to ensure it only points to objects within a specific target table.

3. Example Scenario: DDL for Objects and References

Consider a university system where an **Employee** works for a **Department**. First, we define the object types, and then we create the typed tables.

```
-- 1. Create the base User-Defined Types (UDT)
CREATE TYPE DepartmentType AS (
    Dept_No INT,
    Dept_Name VARCHAR(50)
);

CREATE TYPE EmployeeType AS (
    Emp_ID INT,
    Name VARCHAR(50),
    Works_For REF(DepartmentType) -- The Reference Type
);

-- 2. Create Typed Tables
CREATE TABLE Departments OF DepartmentType (
    REF IS Dept_OID SYSTEM GENERATED, -- Explicitly generating OIDs
    PRIMARY KEY (Dept_No)
);

CREATE TABLE Employees OF EmployeeType (
    PRIMARY KEY (Emp_ID),
    SCOPE FOR Works_For IS Departments -- Restricting pointer target
);
```

4. Visualizing Object References

Employees Table (Source)

Emp_ID	Name	Works_For
101	Alice	0x9A4F...

Departments Table (Target)

OID (System)	Dept_No	Dept_Name
0x9A4F...	10	Research

0x9A4F...

Dereference (Pointer Traversal)

0x9A4F...

5. Query Processing Using Reference Types

To process queries involving reference types, standard JOIN operations can be entirely bypassed. Instead, object-relational SQL introduces **Path Expressions** and **Dereferencing**.

A. Dereferencing Operator (->)

The -> operator takes a REF value and retrieves the object it points to, allowing direct access to the target object’s attributes.

23


```
-- Retrieve employee names and the name of the department they work for.
-- Notice the absence of a JOIN clause.
SELECT e.Name,
       e.Works_For->Dept_Name AS Department_Name
FROM Employees e;
```

B. The Deref() Function

If the goal is to return the entire referenced object (tuple) rather than just a specific attribute, the Deref() function is utilized.

```
-- Retrieve the full Department object for the employee named Alice
SELECT Deref(e.Works_For) AS Dept_Object
FROM Employees e
WHERE e.Name = 'Alice';
```

6. Advantages of Object References

- **Performance (Implicit Joins):** Traversing pointers is computationally faster than performing hash-joins or sort-merge joins on standard foreign keys.
- **Simpler Query Syntax:** Complex, multi-table relationships can be queried cleanly using cascading path expressions (e.g., a->b->c) without cluttering the FROM clause.
- **Polymorphism:** References can be designed to point to an object of a super-type or any of its sub-types, enabling polymorphic queries.

S2 — Database Design

Entity-Relationship (ER) Model

Q1. Differentiate between strong entity and weak entity by showing appropriate examples. How do you design them in a relational schema? (10 marks) [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Distinction Between Strong and Weak Entities

In Entity-Relationship (ER) modeling, entities are categorized based on their ability to be uniquely identified strictly by their own attributes.

- **Strong Entity:** An entity type that possesses a **Primary Key**. It has an independent existence in the database and does not rely on any other entity for its identification.
- **Weak Entity:** An entity type that does **not** have sufficient attributes to form a primary key. It depends on a strong entity (called the **Identifying Owner**) for its existence and identification. It uses a **Partial Key** (or Discriminator) combined with the primary key of its owner to uniquely identify its instances.

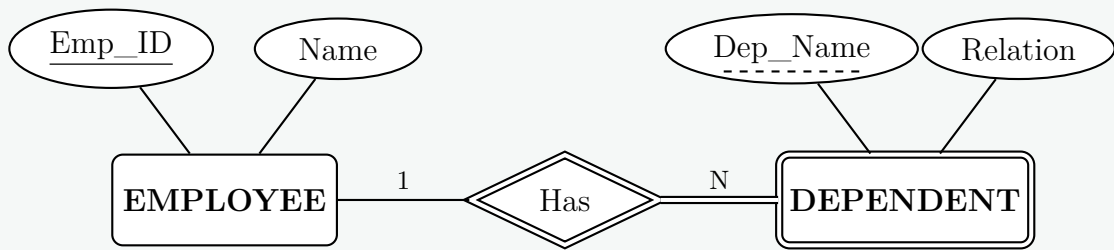
Comparison Table

Feature	Strong Entity	Weak Entity
Primary Key	Has its own primary key.	Does not have a primary key; has a partial key.
Existence	Independent existence.	Existence depends on the identifying strong entity.
Relationship	Associated via standard relationships.	Associated via an identifying relationship .
Participation	Can have total or partial participation.	Always has total participation in the identifying relationship.
ER Diagram	Represented by a single rectangle.	Represented by a double rectangle.

2. Example and ER Diagram Representation

Example Scenario: Consider a company database storing information about EMPLOYEES and their DEPENDENTS (spouses, children).

- EMPLOYEE is a **Strong Entity** uniquely identified by Emp_ID.
- DEPENDENT is a **Weak Entity**. A dependent’s name (Dep_Name) only identifies them uniquely *within* the context of a specific employee. It cannot uniquely identify a dependent across the entire company.



3. Design in a Relational Schema

To convert a strong and weak entity relationship into a relational database schema, apply the following mapping rules:

- (a) **Strong Entity Mapping:** Create a separate table for the strong entity. Its attributes become columns, and its key attribute becomes the **Primary Key (PK)**.
- (b) **Weak Entity Mapping:** Create a separate table for the weak entity. Include all of its own attributes. Then, add the Primary Key of the identifying strong entity as a **Foreign Key (FK)**.
- (c) **Weak Entity Primary Key:** The Primary Key of the weak entity table is a **Composite Key** formed by combining its Foreign Key and its Partial Key.

Relational Schema Diagram

EMPLOYEE	<u>Emp_ID</u>	Name
----------	---------------	------

DEPENDENT	<u>Emp_ID</u> (FK)	<u>Dep_Name</u>	Relation
-----------	-----------------------	-----------------	----------

* **Note:** The primary key of DEPENDENT is composite: (*Emp_ID*, *Dep_Name*).
** **Foreign Key Mapping:** DEPENDENT.*Emp_ID* → EMPLOYEE.*Emp_ID*

Q2. Define weak entity set. Also give an example of a weak entity set. (5 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Definition

A **weak entity set** is an entity type that does not contain sufficient attributes to uniquely identify its own entities (i.e., it lacks a primary key of its own). To be uniquely identified, a weak entity must be associated with another entity set, known as the **identifying entity set** (or strong entity set).

2. Key Characteristics

- **Existence Dependency:** A weak entity cannot exist in the database without its corresponding strong entity. If the strong entity is deleted, all its associated weak entities must also be deleted (cascade delete).
- **Identifying Relationship:** The relationship associating the weak entity set with the strong entity set is called the identifying relationship. In an Entity-Relationship (ER) diagram, it is denoted by a **double diamond**.
- **Total Participation:** The weak entity set always exhibits total participation (denoted by a **double line**) in the identifying relationship, meaning every weak entity must be related to an identifying entity.
- **Partial Key (Discriminator):** Instead of a primary key, a weak entity set has a **partial key** (or discriminator). This is an attribute (or set of attributes) that uniquely identifies weak entities related to the *same* strong entity. It is denoted by a **dashed underline** in an ER diagram.
- **Primary Key Formation:** The actual primary key of a weak entity is a composite key formed by combining the primary key of the strong entity set with the weak entity set's partial key.

3. Example Scenario

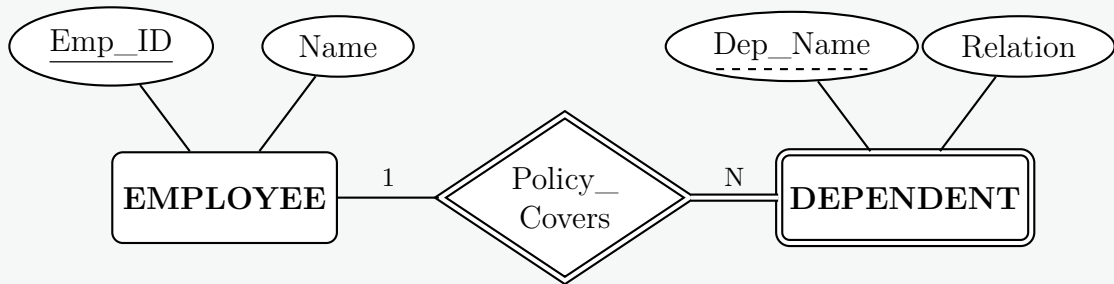
Consider a company database that stores information about employees and their dependents (e.g., spouses, children) for health insurance purposes.

- **Strong Entity Set:** EMPLOYEE (uniquely identified by Emp_ID).
- **Weak Entity Set:** DEPENDENT. A dependent's name (Dep_Name) is not globally unique across the whole company, but it is unique for a specific employee.

- Identifying Relationship: Policy_Covers.

In this context, a dependent only exists in the database if their associated employee exists.

4. ER Diagram Representation



Exam Tip (Relational Mapping): When mapping this ER model to a relational database schema, you do not create a separate table for the identifying relationship. Instead, the resulting **DEPENDENT** table will simply have the composite primary key (**Emp_ID**, **Dep_Name**), where **Emp_ID** acts as a foreign key referencing the **EMPLOYEE** table.

Q3. Show an example of specialization hierarchy in the context of ERD with disjoint and overlapping subtypes. (9 marks) [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Specialization Hierarchy and Constraints

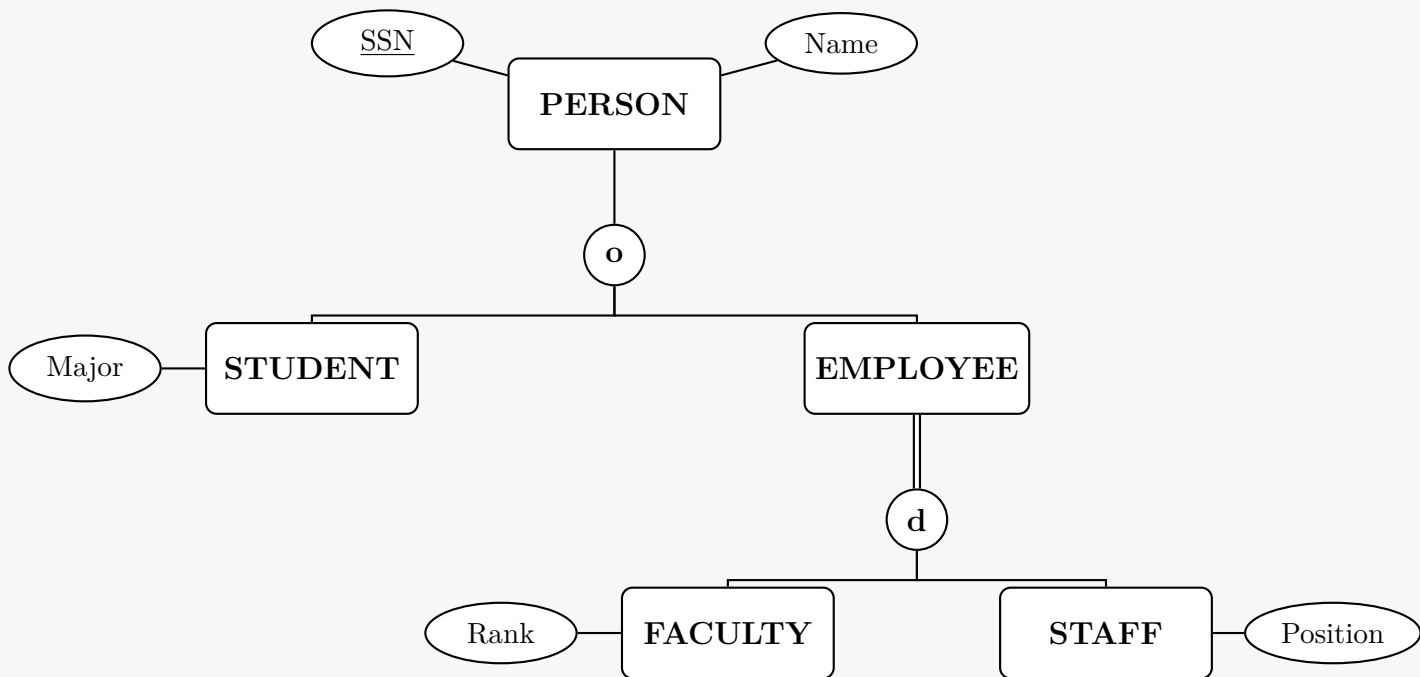
Specialization is the top-down design process of defining a set of subclasses (subtypes) from a single superclass (supertype) based on distinguishing characteristics. The entities in the subtypes inherit all attributes and relationships of the supertype. Specialization hierarchies are governed by **disjointness constraints**, which dictate whether an entity can belong to more than one subclass simultaneously:

- Disjoint Subtypes (denoted by ‘d’):** An entity instance of the supertype can belong to **at most one** subtype. The subtypes are mutually exclusive.
- Overlapping Subtypes (denoted by ‘o’):** An entity instance of the supertype can belong to **more than one** subtype simultaneously.

2. Example Scenario and ER Diagram

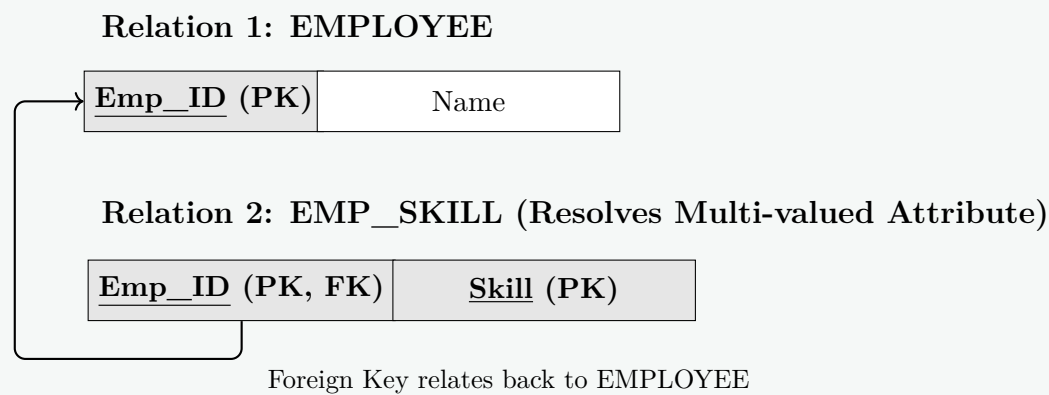
Consider a university database containing a general **PERSON** entity.

- A **PERSON** can be a **STUDENT**, an **EMPLOYEE**, or **both**. This is an **overlapping** specialization.
- An **EMPLOYEE** can be either a **FACULTY** member or a **STAFF** member, but **never both**. This is a **disjoint** specialization.
- Furthermore, every **EMPLOYEE** *must* be either **FACULTY** or **STAFF** (Total Specialization, denoted by a double line).



Relational Mapping (Decomposition)

To map this to a relational schema and conform to 1NF, we decompose the entity into two distinct tables: **EMPLOYEE** and **EMP_SKILL**.

**Explanation of the Mapping:**

- (a) **EMPLOYEE(Emp_ID, Name):** The original entity becomes a table, holding only the single-valued attributes. **Emp_ID** remains the Primary Key.
- (b) **EMP_SKILL(Emp_ID, Skill):** The multi-valued attribute **Skill** gets its own table. To link the skill to the correct employee, we import **Emp_ID** as a Foreign Key. Because one employee can have many skills, **Emp_ID** alone is not unique in this new table. Therefore, the Primary Key for **EMP_SKILL** is the composite key (**Emp_ID, Skill**).

Q5. What are the different elements of Entity-Relation Model? Briefly discuss them. **(15 marks)** [CSE 303 | L-3/T-1 | 2012–13]

Answer**Introduction to the ER Model**

The **Entity-Relationship (ER) Model** is a high-level conceptual data model used in database design to represent the logical structure of a database. It abstracts the real-world system into a collection of basic objects and the associations among them. The three fundamental elements of the ER model are **Entities**, **Attributes**, and **Relationships**.

1. Entities and Entity Sets

An **entity** is a real-world object or concept that exists independently and can be uniquely identified (e.g., a specific student, a specific car). An **entity set** is a collection of entities of the same type.

- **Strong Entity:** An entity set that contains sufficient attributes to uniquely identify all its entities. It has a primary key and is represented by a **single rounded rectangle**.
- **Weak Entity:** An entity set that does not have a primary key of its own and depends on another entity (the strong entity) for its existence. It is represented by a **double rounded rectangle**.

2. Attributes

Attributes are the properties or characteristics that describe an entity. They are represented by **ellipses**. Attributes can be classified into several types:

- **Simple Attribute:** Cannot be subdivided further (e.g., Gender, Salary).
- **Composite Attribute:** Can be divided into smaller subparts representing basic attributes (e.g., *Name* can be divided into *First Name*, *Middle Name*, and *Last Name*).
- **Single-valued Attribute:** Has only a single value for a particular entity (e.g., Date of Birth).
- **Multi-valued Attribute:** Can have multiple values for a single entity. Represented by a **double ellipse** (e.g., Phone Number, Skills).
- **Derived Attribute:** Its value is computed or derived from another related attribute or set of attributes. Represented by a **dashed ellipse** (e.g., *Age* derived from *Date of Birth*).
- **Key Attribute:** Uniquely identifies an entity within an entity set. Its name is **underlined**.

3. Relationships and Relationship Sets

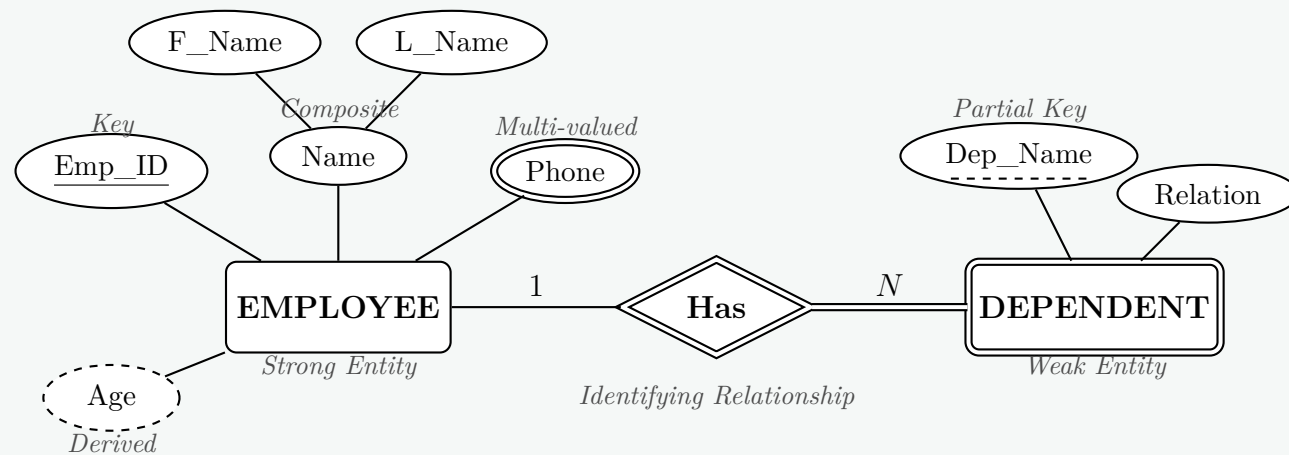
A **relationship** is an association among two or more entities. A **relationship set** is a collection of similar relationships. They are represented by **diamonds**.

- **Degree of Relationship:** The number of participating entity types (e.g., Unary, Binary, Ternary).
- **Cardinality Ratio:** Expresses the maximum number of relationship instances an entity can participate in (e.g., 1 : 1 (One-to-One), 1 : N (One-to-Many), M : N (Many-to-Many)).
- **Participation Constraint:**

- **Total Participation:** Every entity in the set must participate in at least one relationship instance. Represented by a **double line**.
- **Partial Participation:** Only some entities in the set participate in the relationship. Represented by a **single line**.
- **Identifying Relationship:** The relationship associating a weak entity to its strong entity. Represented by a **double diamond**.

4. Visual Summary of ER Elements

The following Entity-Relationship diagram illustrates all the fundamental elements working together in a unified database schema.



Q6. What is a weak entity set? What is the requirement of a weak entity set? (10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Definition of a Weak Entity Set

In the Entity-Relationship (ER) model, a **weak entity set** is an entity set that does not contain sufficient attributes to form a primary key of its own. Instead, it relies on another entity set called the **strong entity set** (or owner entity set) for its existence and identification.

The primary key of a weak entity set is formed by a combination of the primary key of its strong entity set and its own **discriminator** (or partial key).

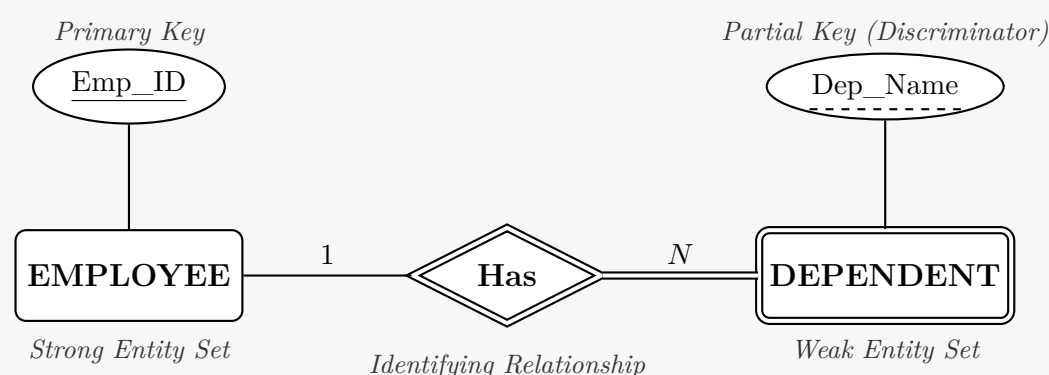
2. Requirements of a Weak Entity Set

For a weak entity set to exist and function correctly in a database schema, the following requirements must be met:

- Existence Dependency (Owner Entity):** A weak entity cannot exist in the database without its corresponding strong entity. If the strong entity is deleted, all its associated weak entities must also be deleted (cascading delete).
- Identifying Relationship:** The weak entity set must be associated with the strong entity set via a specific relationship known as the **identifying relationship**. In an ER diagram, this is represented by a double diamond.
- Total Participation:** The weak entity set must exhibit **total participation** in the identifying relationship. This means every instance of the weak entity must participate in the relationship. It is represented by a double line connecting the weak entity to the identifying relationship.
- Discriminator (Partial Key):** Although it lacks a full primary key, the weak entity set must possess a set of attributes that uniquely identifies its instances *among the instances related to a specific strong entity*. This attribute is called the partial key and is represented by a dashed underline in an ER diagram.

3. Visual Representation (ER Diagram)

Consider an organization where employees have dependents (e.g., children, spouse) for insurance purposes. A dependent cannot exist in the database without the employee, and two different employees might have dependents with the same name. Thus, **DEPENDENT** is a weak entity set owned by the **EMPLOYEE** strong entity set.



4. Comparison Summary			
	Feature	Strong Entity Set	Weak Entity Set
	Primary Key	Has its own primary key.	Lacks a primary key; uses a partial key.
	Existence	Exists independently.	Existence depends on a strong entity.
	Representation	Single rounded rectangle.	Double rounded rectangle.
	Relationship	Associated via regular relationships.	Associated via an identifying relationship (double diamond).

Q7. Consider the following scenario of a hospital:

- Patients are treated in a single ward by the doctors assigned to them. Usually each patient will be assigned a single doctor, but in rare cases he/she will be assigned two doctors.
- Healthcare assistants also attend to the patients; a number of these are associated with each ward.
- Initially the system will be concerned solely with drug treatment. Each patient is required to take a variety of drugs a certain number of times per day and for varying lengths of time.
- The system must record details concerning patient treatment and staff payment. Some staffs are paid on a part time basis and doctors and care assistants work varying amounts of overtime at varying rates (subject to grade).
- The system will also need to track what treatments are required for which patients and when, and it should be capable of calculating the cost of treatment per week for each patient.

Define entities and their attributes for the hospital management system described above. **(10 marks)** [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Entity Definitions and Attributes

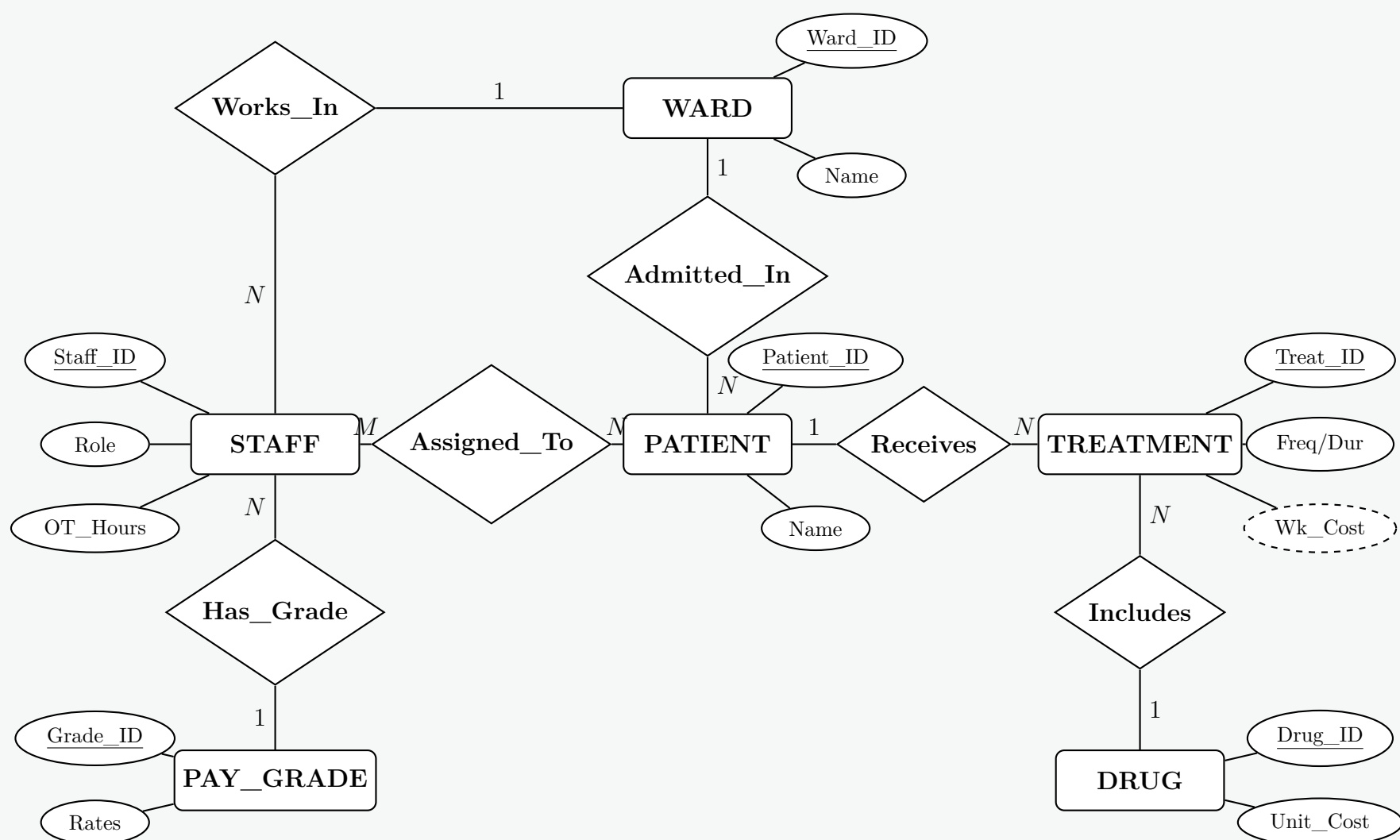
Based on the hospital scenario provided, the system can be modeled using the following core entities and their attributes. **Primary Keys (PK)** are underlined and used to uniquely identify records, while **Foreign Keys (FK)** establish relationships between the entities.

- **WARD:** Represents the physical wards where patients are admitted.
 - Ward_ID (PK): Unique identifier for the ward.
 - **Ward_Name:** Name or designation of the ward.
 - **Capacity:** Maximum number of patients the ward can hold.
- **PATIENT:** Stores information regarding the individuals receiving care.
 - Patient_ID (PK): Unique identifier for the patient.
 - **Name:** Full name of the patient.
 - **Admission_Date:** Date the patient was admitted.
 - **Ward_ID** (FK): The specific ward to which the patient is admitted.
- **STAFF:** A generalized entity representing both doctors and healthcare assistants to streamline the payment and shift tracking system.
 - Staff_ID (PK): Unique identifier for the staff member.
 - **Name:** Full name of the staff member.
 - **Role:** Distinguishes between 'Doctor' and 'Healthcare Assistant'.
 - **Employment_Type:** Indicates if the staff is 'Full-time' or 'Part-time'.
 - **Overtime_Hours:** Tracks varying amounts of overtime worked.
 - **Grade_ID** (FK): Links to the pay grade for rate calculations.
- **PAY_GRADE:** Manages the varying payment rates subject to a staff member's grade.
 - Grade_ID (PK): Unique identifier for the pay grade.
 - **Basic_Hourly_Rate:** Standard rate of pay.
 - **Overtime_Rate:** Multiplier or specific rate for overtime hours.
- **DRUG:** Stores the inventory and pricing of treatments.
 - Drug_ID (PK): Unique identifier for the medication.

- **Drug_Name**: Name of the drug.
- **Unit_Cost**: The cost per single administration/pill of the drug.
- **TREATMENT**: An associative entity (or relationship entity) tracking the specific drug regimens assigned to patients.
 - **Treatment_ID** (PK): Unique identifier for the specific treatment plan.
 - **Patient_ID** (FK): The patient receiving the treatment.
 - **Drug_ID** (FK): The specific drug prescribed.
 - **Doctor_ID** (FK): The staff ID of the doctor who prescribed it.
 - **Frequency_Per_Day**: Number of times the drug must be taken daily.
 - **Duration_Days**: Varying length of time the drug is required.
 - **Weekly_Cost** (Derived): A calculated attribute resolving the system's need to compute treatment cost per week ($\text{Frequency_Per_Day} \times 7 \times \text{Unit_Cost}$).
- **PATIENT_DOCTOR_ASSIGNMENT**: Resolves the many-to-many relationship mapping between patients and their assigned doctors (as a patient can have 1 or 2 doctors).
 - **Patient_ID** (PK, FK)
 - **Staff_ID** (PK, FK)

2. Conceptual Entity-Relationship (ER) Diagram

The following diagram illustrates the core entities, their key attributes, and the cardinalities of their relationships based on the constraints described in the scenario.



ER Diagrams

Q1. You are the manager of the website development department of your organisation. Each website developed by your organization can be uniquely identified by a url. It also has a public host-ip address and a date of going live. Each website is developed by a team of developers. Each developer can be uniquely identified by an employee id. The relevant information for a developer is his/her name, address, highest educational qualification, salary and hire date. Each website is developed by a web-based framework which is associated with a particular programming language. Each framework and its associated programming language have a specific name and the development year. The other information relevant to a web framework is its author name.

Design an Entity-Relationship Diagram for your website development department to manage it conveniently. **(10 marks)** [CSE 215 | L-2/T-2 | 2018-19]

Answer

1. Entity Identification and Attributes

Based on the provided scenario for the website development department, we can identify four primary entities along with their respective attributes. **Primary Keys (PK)** are used to uniquely identify each record.

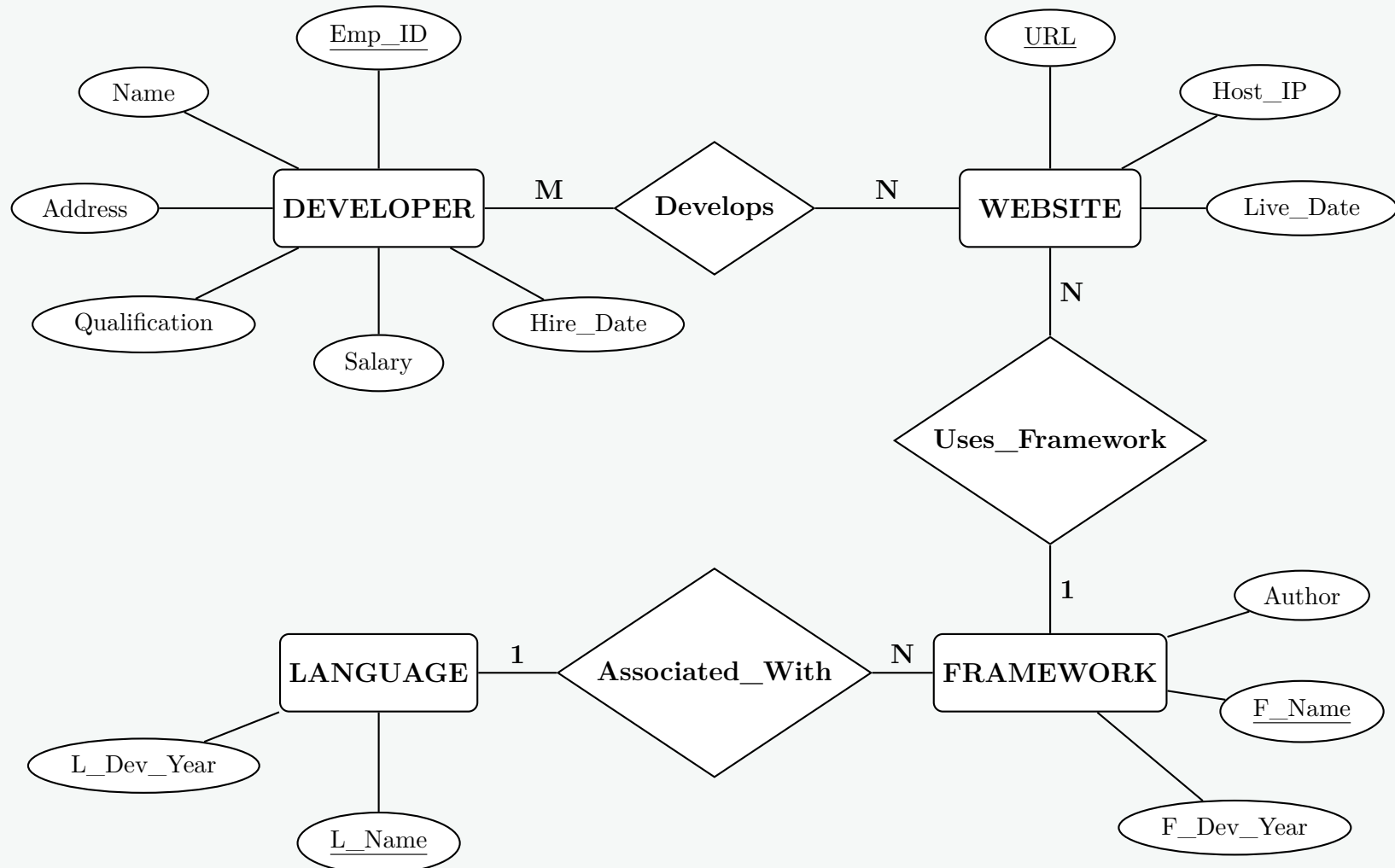
- **WEBSITE**: Stores details about the deployed projects.
 - **URL** (PK): The unique identifier for each website.
 - **Host_IP**: The public host-IP address.
 - **Live_Date**: The date the website went live.
- **DEVELOPER**: Stores professional and personal details of the staff.
 - **Emp_ID** (PK): The unique employee identifier.
 - **Name**: The full name of the developer.
 - **Address**: Residential address.
 - **Qualification**: Highest educational qualification.
 - **Salary**: Current compensation.
 - **Hire_Date**: Date of joining the organization.
- **FRAMEWORK**: Details the web-based frameworks utilized for development.
 - **F_Name** (PK): The specific name of the framework.
 - **F_Dev_Year**: The year the framework was developed.
 - **Author**: The author or creator of the framework.
- **LANGUAGE**: The programming language associated with the frameworks.
 - **L_Name** (PK): The specific name of the programming language.
 - **L_Dev_Year**: The development year of the language.

2. Relationships and Cardinalities

The entities interact through the following relationships:

- **Develops (DEVELOPER and WEBSITE)**: A **Many-to-Many (M:N)** relationship. The scenario states a website is developed by a "team of developers" (meaning multiple developers per website), and a developer can generally work on multiple websites over time.
- **Uses_Framework (WEBSITE and FRAMEWORK)**: A **Many-to-One (N:1)** relationship. Each website is developed using one specific web-based framework, but a single framework can be used to build many websites.
- **Associated_With (FRAMEWORK and LANGUAGE)**: A **Many-to-One (N:1)** relationship. Each framework is associated with a *particular* (single) programming language, while a programming language (like Python or JavaScript) can have multiple frameworks associated with it (e.g., Django, Flask).

3. Entity-Relationship (ER) Diagram



Q2. You are designing a database for a university with the following requirements: Each student has a student id, name, address and contact name. Each course has a course number, name, credit hours, and the minimum level and term. Each student enrolls in several courses. University employees are either faculty members or office staff. Faculty members have a name, designation, address, contact number and room number. Office staff have a name, contact, address and hourly payment rate. In each semester (identified by a label e.g. “January 2019”), every faculty member conducts several courses; a single course can have many teachers. If a course is conducted by several teachers, one is assigned as coordinator.

- Write down the list of entity sets with their attributes.
- Draw the E/R diagram using appropriate arrows to indicate multiplicity. Show attributes on relations where applicable.

(20 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Entity Sets and their Attributes

Based on the scenario, we can identify the following entity sets and their attributes. A generalized superclass **EMPLOYEE** is introduced to hold the shared attributes of **FACULTY** and **OFFICE_STAFF**. Primary keys are underlined.

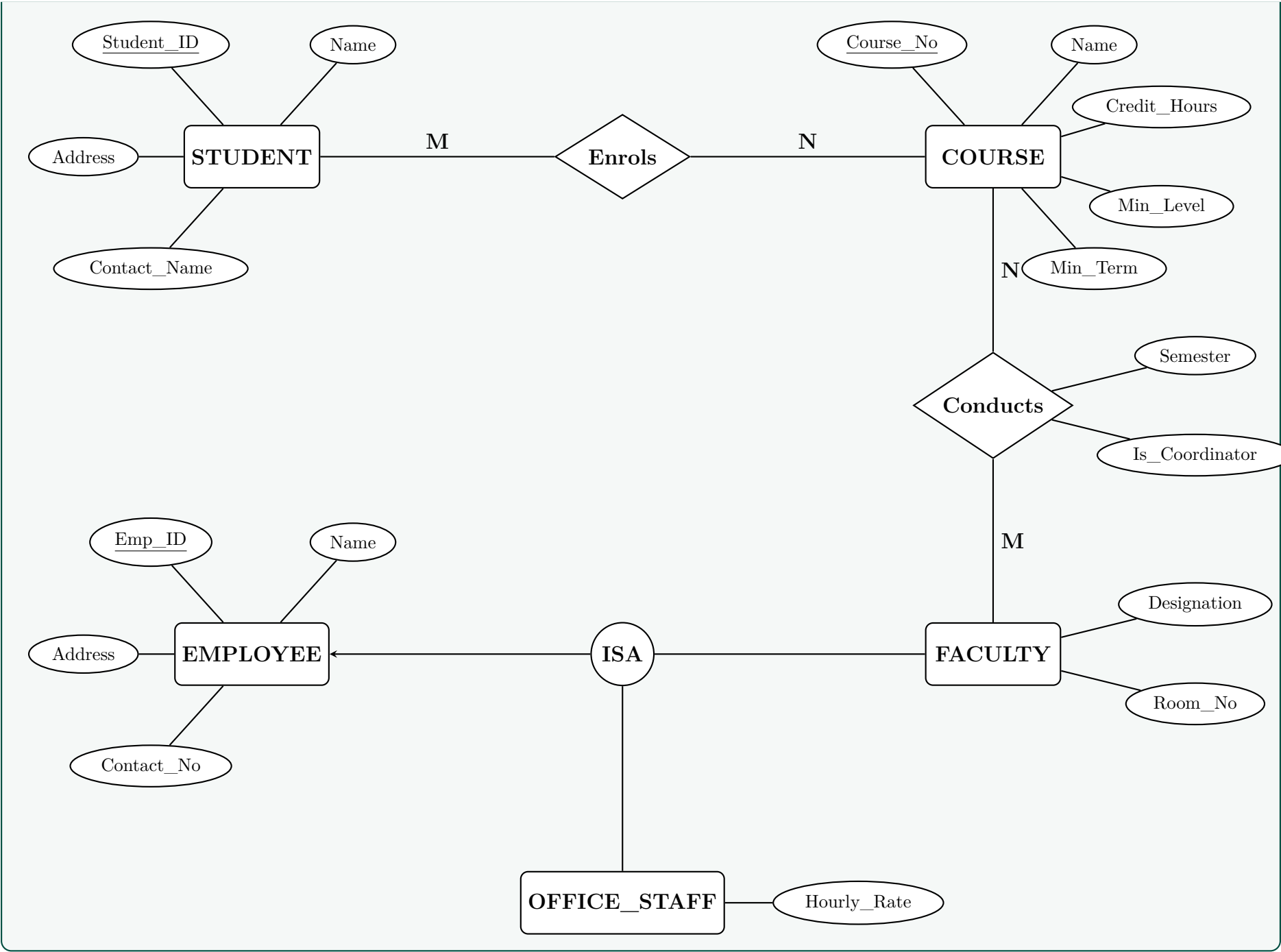
- STUDENT:** Student_ID, Name, Address, Contact_Name.
- COURSE:** Course_No, Name, Credit_Hours, Min_Level, Min_Term.
- EMPLOYEE** (Superclass): Emp_ID (Assumed as a unique identifier), Name, Address, Contact_Number.
- FACULTY** (Subclass of Employee): Designation, Room_Number.
- OFFICE_STAFF** (Subclass of Employee): Hourly_Rate.

Relationship Sets & their Attributes:

- Enrolls:** Associates **STUDENT** and **COURSE**. (Multiplicity: Many-to-Many).
- Conducts:** Associates **FACULTY** and **COURSE**. (Multiplicity: Many-to-Many).
 - Attributes: **Semester** (label indicating when it is taught), **Is_Coordinator** (Boolean flag indicating if the faculty is the assigned coordinator for that course).

2. Entity-Relationship (ER) Diagram

The following ER diagram uses standard Chen’s notation. The generalization (ISA) is represented by a circle linking the subclasses to the superclass.



- Q3.** There are many books in central library of BUET in different subjects. Each book must have a ISBN number, title, name of authors, subject and number of copies. A book can have multiple copies and each copy is identified by an accession number. The descriptive attributes of each copy of book are edition, price and data of purchase. A publisher publishes many books and a book is published by ofily one publisher. A publisher is identified by publisher id and described by name, country of origin, address and contact numbers. Book suppliers supply many copies of books to the library by tender in each year.- A tender has an id, name, total price and year. For each copy of books, you must store the record of which copy of book has been supplied by which supplier in which tender. A supplier is identified by supplier id and described by name, address and contact numbers. One copy of a book (book-copy) can be issued to multiple borrowers in different times and a borrower can borrow multiple number of book-copies. Borrowers can be teachers, students or staffs. Each borrower must have a borrower id, name, contact number and address. Teachers are described by joining date and designation. Students are described by session, level and term. Staffs are described by office name. When a book-copy is issued to borrower, some information such as issued data, accession number, borrower id, length of period for issue and return date are needed to enter into the database. At the same time, the status of the borrowed book should be labeled as issued and will be cleared after return by the borrower.
- (a) Draw an ERD for the above library management system.
- (b) Transform the ERD into tables showing all primary keys and foreign keys.

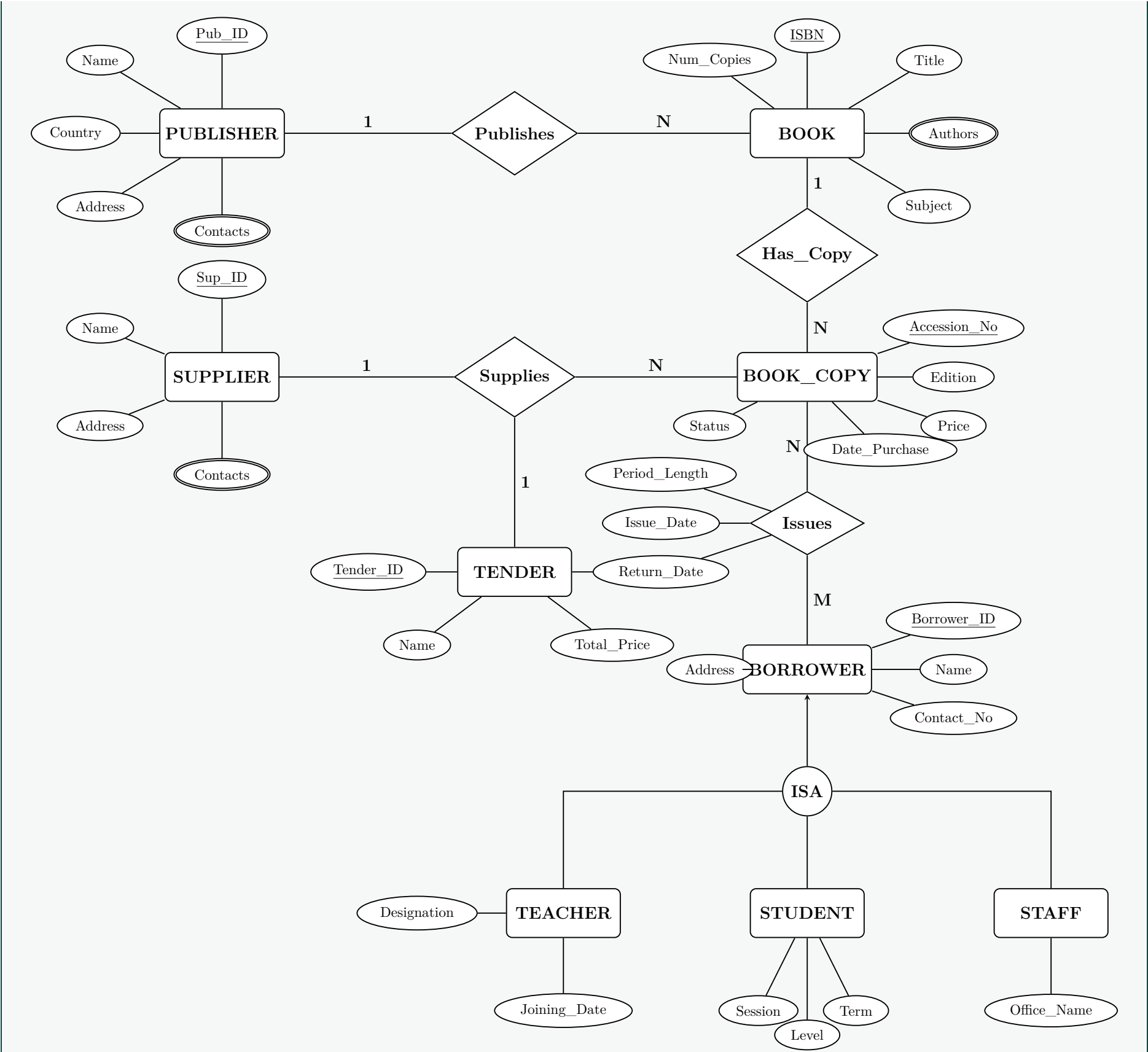
(35 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Entity-Relationship Diagram (ERD)

The following Entity-Relationship Diagram models the library management system. A generalized **BORROWER** superclass is used to represent **TEACHER**, **STUDENT**, and **STAFF**. A ternary relationship **Supplies** links **SUPPLIER**, **TENDER**, and **BOOK_COPY**.



2. Relational Schema (Tables)

The ER Diagram is mapped to the following normalized relational tables. **Primary Keys (PK)** are underlined, and **Foreign Keys (FK)** are listed below each relation.

- **PUBLISHER** (Publisher_ID, Name, Country_of_Origin, Address)
- **PUBLISHER_CONTACT** (Publisher_ID, Contact_Number)
 - FK: Publisher_ID references PUBLISHER(Publisher_ID)
- **BOOK** (ISBN, Title, Subject, Number_of_Copies, Publisher_ID)
 - FK: Publisher_ID references PUBLISHER(Publisher_ID)
- **BOOK_AUTHOR** (ISBN, Author_Name)
 - FK: ISBN references BOOK(ISBN)
- **SUPPLIER** (Supplier_ID, Name, Address)
- **SUPPLIER_CONTACT** (Supplier_ID, Contact_Number)
 - FK: Supplier_ID references SUPPLIER(Supplier_ID)

- **TENDER** (Tender_ID, Name, Total_Price, Year)
- **BOOK_COPY** (Accession_Number, Edition, Price, Date_of_Purchase, Status, ISBN, Supplier_ID, Tender_ID)
 - *FK: ISBN references BOOK(ISBN)*
 - *FK: Supplier_ID references SUPPLIER(Supplier_ID)*
 - *FK: Tender_ID references TENDER(Tender_ID)*
- **BORROWER** (Borrower_ID, Name, Contact_Number, Address)
- **TEACHER** (Borrower_ID, Joining_Date, Designation)
 - *FK: Borrower_ID references BORROWER(Borrower_ID)*
- **STUDENT** (Borrower_ID, Session, Level, Term)
 - *FK: Borrower_ID references BORROWER(Borrower_ID)*
- **STAFF** (Borrower_ID, Office_Name)
 - *FK: Borrower_ID references BORROWER(Borrower_ID)*
- **ISSUE** (Borrower_ID, Accession_Number, Issue_Date, Length_of_Period, Return_Date)
 - *FK: Borrower_ID references BORROWER(Borrower_ID)*
 - *FK: Accession_Number references BOOK_COPY(Accession_Number)*

Q4. East West Property Development is a construction company of Bashundhara Group with over 1000 employees. A customer can hire the company for more than one project, and employees sometimes work on more than one project at a time. Equipment is assigned to only one project. Draw an E-R diagram for this company indicating cardinality. **(7 marks)** [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Entity and Relationship Analysis

Based on the scenario provided for East West Property Development, we can identify the following entities, relationships, and their respective cardinalities. Since specific attributes were not detailed in the prompt, standard identifier (primary key) and descriptive attributes have been assumed for completeness.

- **Entities:**

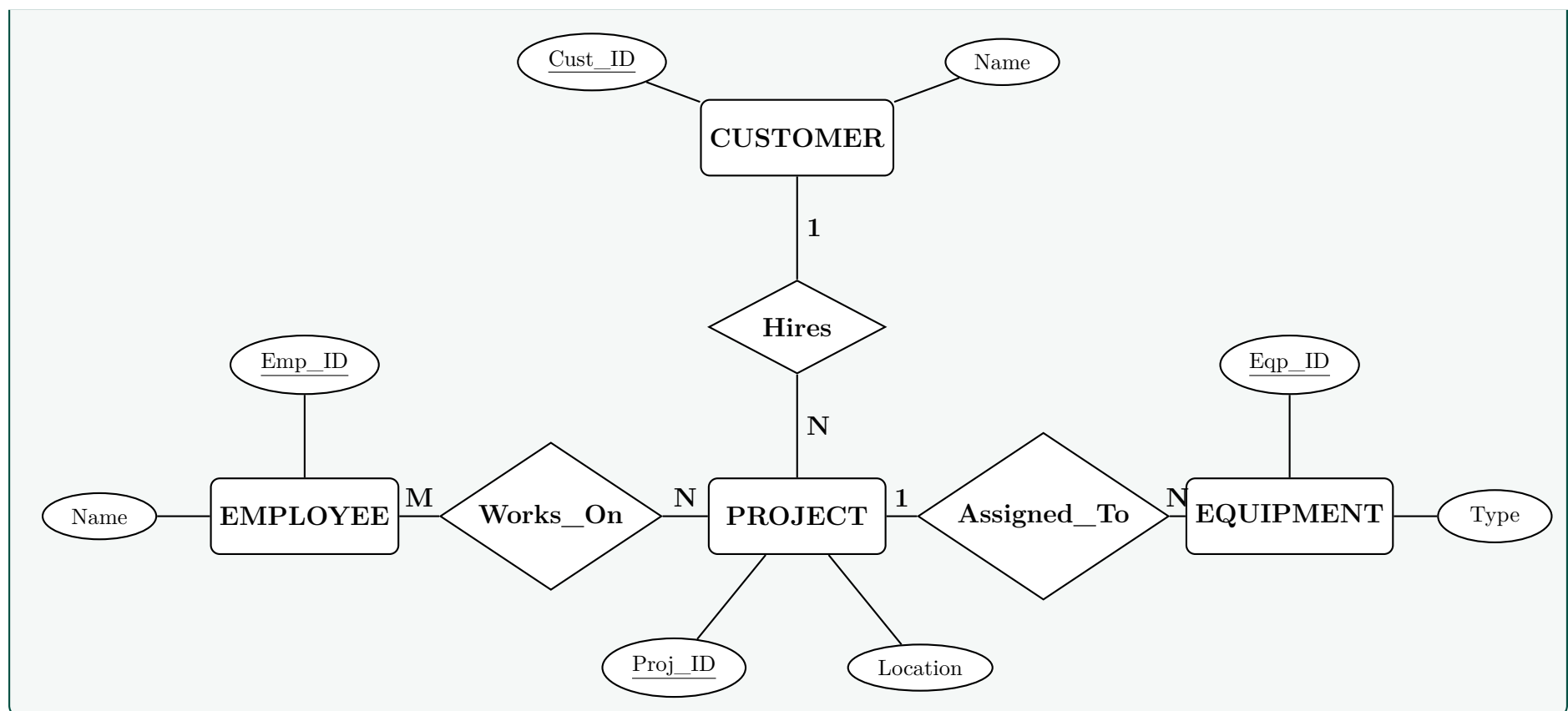
- **CUSTOMER:** Represents the clients who hire the company.
- **PROJECT:** Represents the construction projects undertaken by the company.
- **EMPLOYEE:** Represents the workforce of the company.
- **EQUIPMENT:** Represents the machinery/tools used in construction.

- **Relationships and Cardinalities:**

- **Hires (CUSTOMER and PROJECT):** A **One-to-Many (1:N)** relationship. A customer can hire the company for more than one project (N), but a specific project is typically commissioned by a single customer (1).
- **Works_On (EMPLOYEE and PROJECT):** A **Many-to-Many (M:N)** relationship. Employees can work on more than one project at a time (N), and a construction project naturally requires multiple employees (M).
- **Assigned_To (EQUIPMENT and PROJECT):** A **Many-to-One (N:1)** relationship. Equipment is assigned to only one project at a time (1), but a single project will require multiple pieces of equipment (N).

2. Entity-Relationship (ER) Diagram

The following ER diagram uses standard Chen's notation to visualize the organizational data requirements.



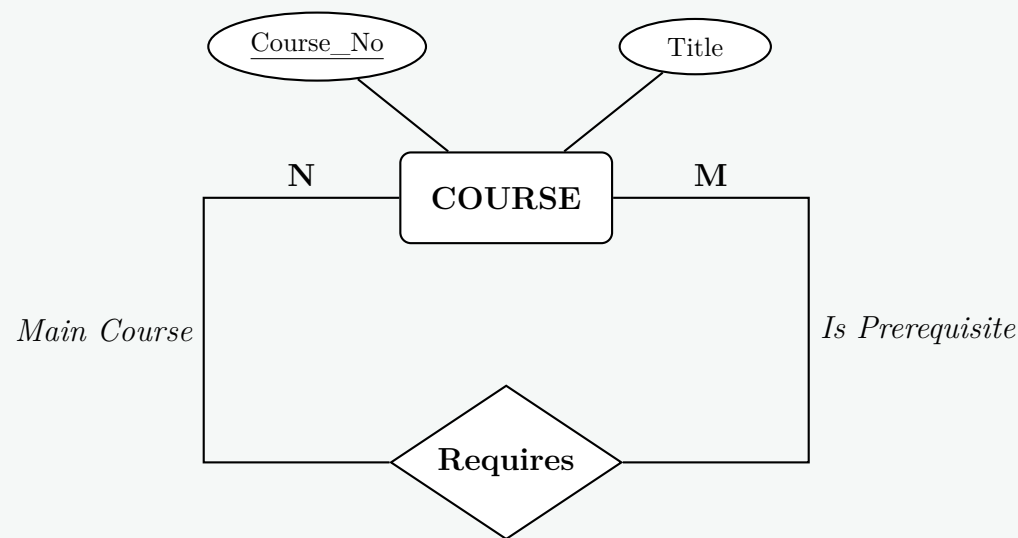
Q5. A university has a large number of courses. Each course may have one or more different courses as prerequisites, or may have no prerequisites. Similarly, a particular course may be a prerequisite of any number of courses or may not be a prerequisite of any other course. Sketch an E-R diagram that includes **COURSE** as the single entity, including cardinality. Each course has a course number and title. Convert the E-R diagram into a relational schema. (7 marks) [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Entity-Relationship (ER) Diagram

The scenario describes a **Unary (Recursive) Relationship** on a single entity **COURSE**. Since a course can have many prerequisites, and a course can be a prerequisite for many other courses, the relationship between **COURSE** and itself is **Many-to-Many (M:N)**.

To clarify the recursive relationship, *roles* (“Main Course” and “Prerequisite”) are indicated on the edges connecting the entity to the relationship diamond.



2. Relational Schema

To convert an $M : N$ recursive relationship into a relational schema, we must create two tables: one for the entity itself, and a junction (associative) table to represent the $M : N$ relationship. The primary keys (PK) are underlined.

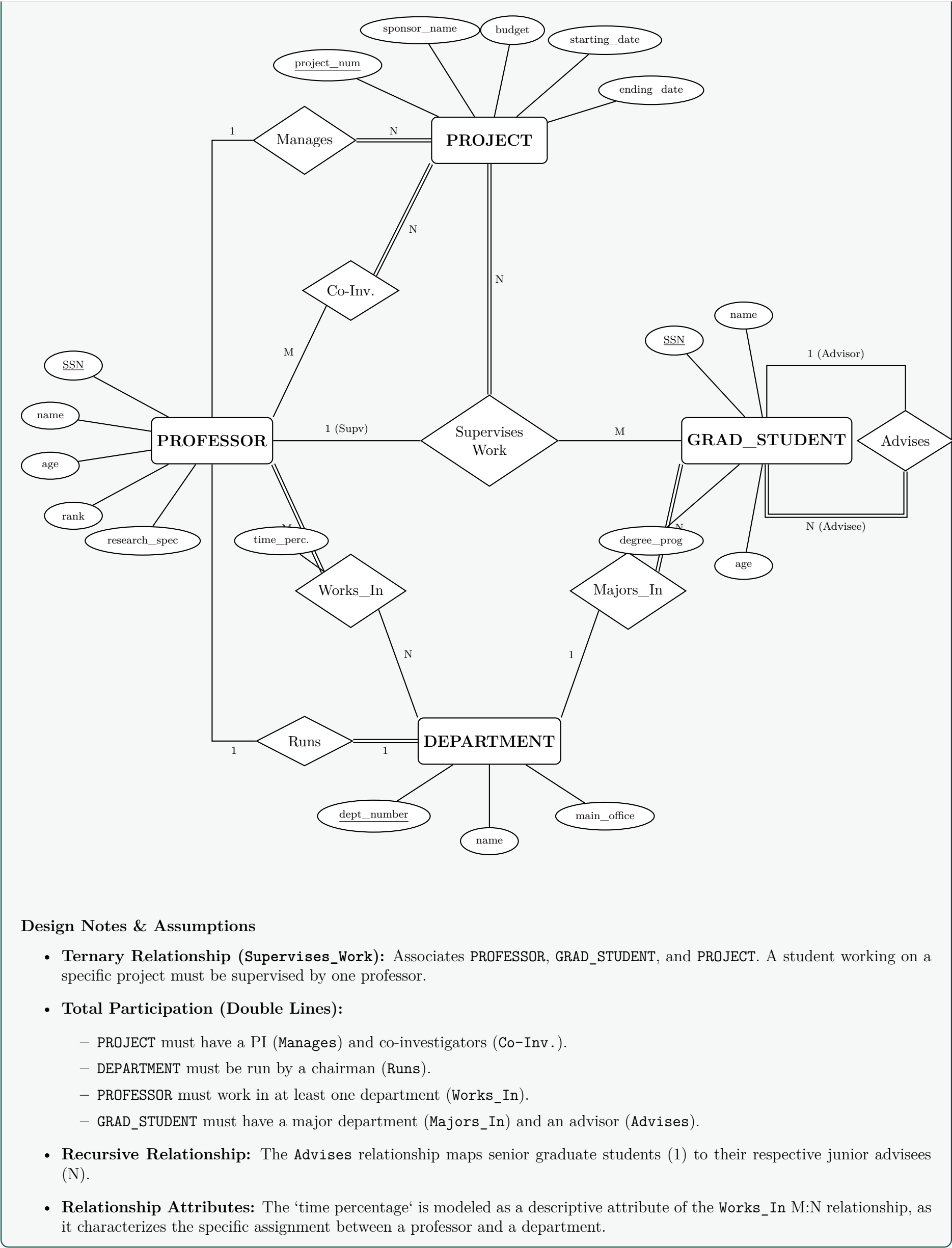
- **COURSE** (Course_No, Title)
- **PREREQUISITE** (Course_No, Prereq_No)

Foreign Key Constraints and Dependencies		
Relation	Foreign Key Attribute(s)	References
COURSE	None	N/A
PREREQUISITE	Course_No	COURSE (Course_No)
PREREQUISITE	Prereq_No	COURSE (Course_No)

Note: In the **PREREQUISITE** table, both **Course_No** and **Prereq_No** act together as a composite Primary Key to uniquely identify a prerequisite pairing. Simultaneously, each attribute independently acts as a Foreign Key referencing the **COURSE** table.

Q6. Consider the following information about a university database. Professors have an SSN, a name, an age, a rank, and a research speciality. Projects have a project number, a sponsor name (e.g., NSF), a starting date, an ending date, and a budget. Graduate students have an SSN, a name, an age, and a degree program (e.g., M.S. or Ph.D). Each project is managed by one professor (the principal investigator). Each project is worked on by one or more professors (co-investigators). Professors can manage and/or work on multiple projects. Each project is worked on by one or more graduate students (research assistants). When graduate students work on a project, a professor must supervise their work. Graduate students can work on multiple projects, having a (potentially different) supervisor for each. Departments have a department number, name, and main office. A professor (chairman) runs each department. Professors work in one or more departments with a time percentage per department. Graduate students have one major department. Each graduate student has a more senior graduate student (student advisor) who advises on course selection. Draw an Entity-Relationship (ER) diagram to represent the data requirements described above. **(18 marks)** [CSE 303 | L-3/T-1 | 2014–15]

Answer
Entity-Relationship (ER) Diagram The following ER diagram captures all the specified data requirements, including the ternary relationship for student project supervision, recursive relationship for student advising, and partial/total participation constraints.



Q7. A library keeps records of current loans of books to borrowers. Each borrower is identified by a borrower number and each copy of a book by an accession number (the library may have more than one copy of any given book). The name and address of each borrower is held so that overdue loan reminders can be sent. Information held about books includes the title, author’s name(s), publisher’s name, publication date, ISBN, purchase price, classification (reference or fiction), and number of pages. A given book may be written by a number of authors; the library regards a book as only being published by a single publisher. The library assigns its own unique in-house codes for authors and publishers.

A book may cover a number of different subjects, and borrowers can use an online catalogue to select texts by subject as well as title and author. There is a restriction on the number of books a borrower may have on loan at any time and the loan period — these limits depend on the borrower's classification (junior, adult, or organisation). When a book is borrowed, the return date is automatically recorded. Other borrowers may reserve books out on loan. A special borrower's status flag is maintained — borrowers who hold overdue books or have reached their loan limit are flagged to prevent further borrowings.

Draw an Entity-Relationship (ER) diagram to represent the Library data requirements described above. **(18 marks)** [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Entity-Relationship (ER) Diagram Analysis

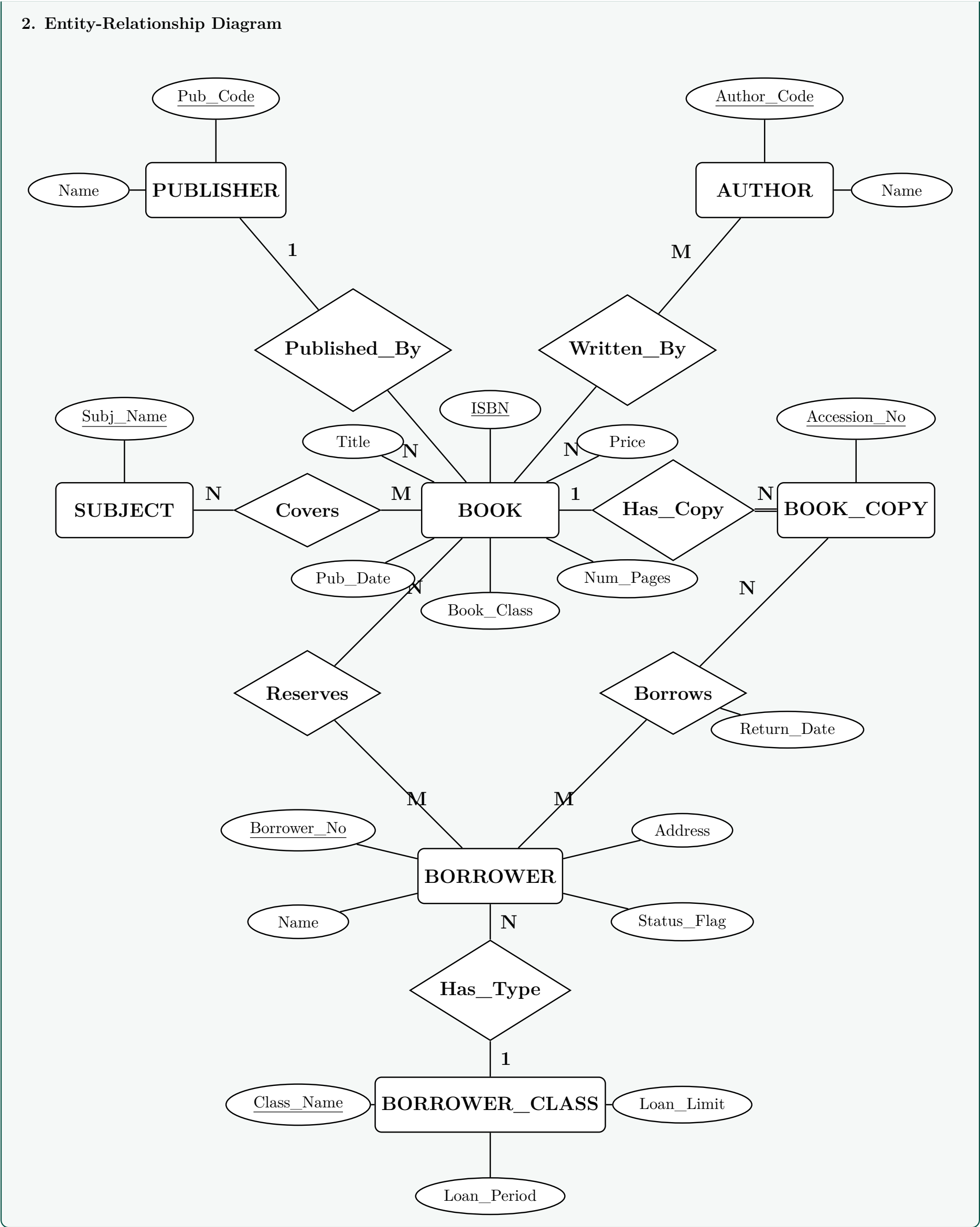
Based on the library data requirements provided, the following elements are identified to form the ER diagram:

- **Entities and Primary Keys:**

- **BOOK:** Identified by ISBN. Attributes include Title, Pub_Date, Price, Book_Class, and Num_Pages.
- **AUTHOR:** Identified by Author_Code. Attributes include Name.
- **PUBLISHER:** Identified by Pub_Code. Attributes include Name.
- **SUBJECT:** Identified by Subj_Name.
- **BOOK_COPY:** Identified by Accession_No. A strong entity as accession numbers are globally unique in a library system.
- **BORROWER:** Identified by Borrower_No. Attributes include Name, Address, and Status_Flag.
- **BORROWER_CLASS:** Identified by Class_Name (e.g., junior, adult, organisation). Attributes include Loan_Limit and Loan_Period. Extracted into a separate entity to enforce the business rule that limits depend strictly on classification.

- **Relationships and Cardinalities:**

- **Published_By:** N:1 between BOOK and PUBLISHER.
- **Written_By:** M:N between BOOK and AUTHOR.
- **Covers:** M:N between BOOK and SUBJECT.
- **Has_Copy:** 1:N between BOOK and BOOK_COPY. BOOK_COPY has total participation.
- **Borrows:** M:N between BORROWER and BOOK_COPY. Incorporates the automatically recorded attribute *Return_Date*.
- **Reserves:** M:N between BORROWER and BOOK.
- **Has_Type:** N:1 between BORROWER and BORROWER_CLASS.



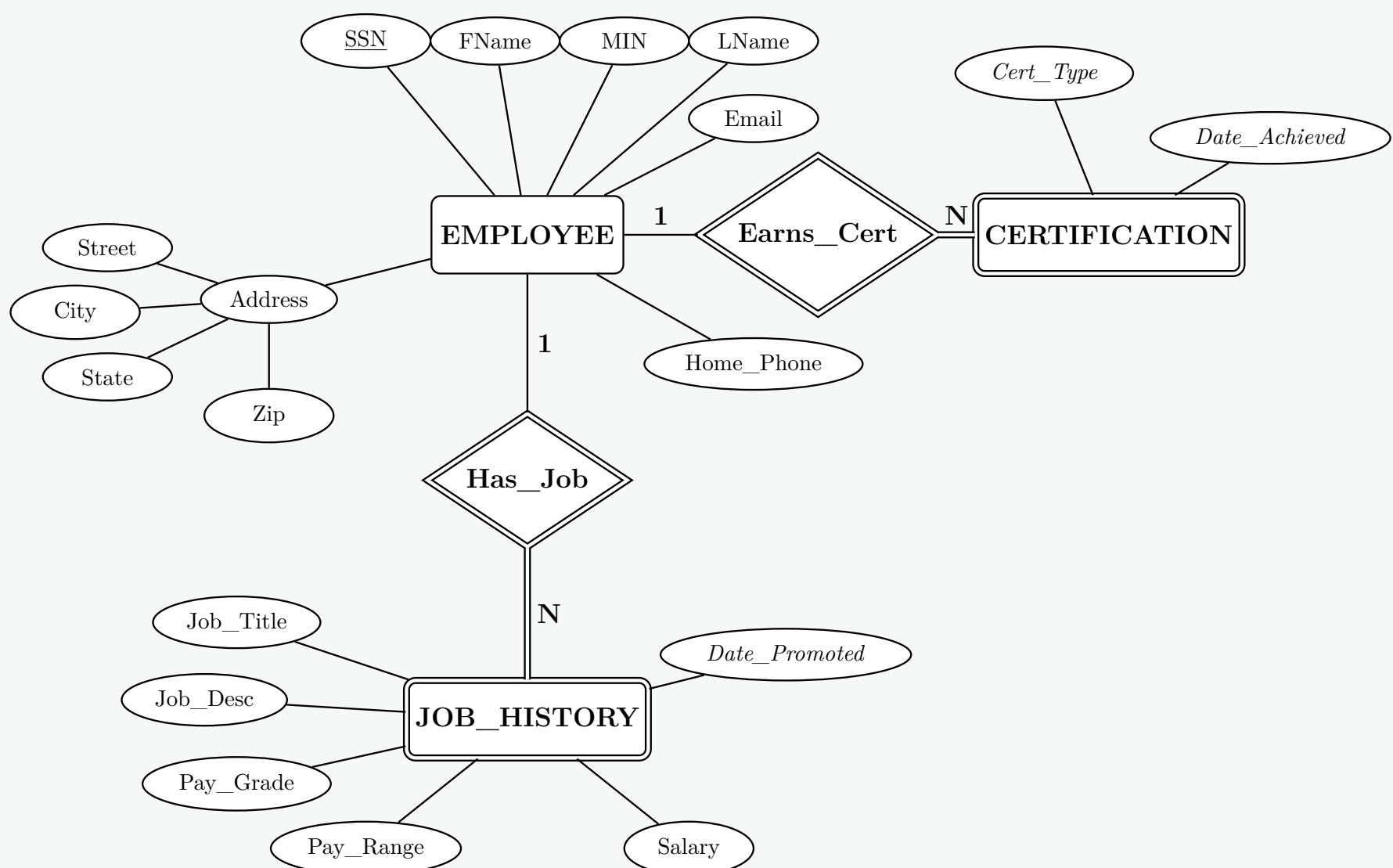
- Q8.** You have been asked to design an employee tracking database for a company. They want to track information about employees, the employee’s job history, and their certifications. Employee information includes first name, middle initials, last name, social security number, address, city, state, zip, home phone and email address. Job history would include job title, job description, pay grade, pay range, salary and date of promotion. For certifications, they want certification type and date achieved. An employee can have multiple jobs over time (i.e., Analyst, Sr. Analyst, QA Administrator). Employees can also earn certifications necessary for their jobs.
- (a) Draw an ER diagram for this application. Mark the multiplicity of each relationship. Identify key attributes. State all assumptions clearly. **(15 marks)**

- (b) Translate your ER diagram into a relational schema. Select approaches that yield the fewest number of relations; merge relations where appropriate. Specify the key of each relation in your schema. (10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer**Part (a): Entity-Relationship (ER) Diagram****Design Assumptions & Rules Applied:**

- **EMPLOYEE** is the core strong entity, uniquely identified by its Primary Key, **SSN**.
- **JOB_HISTORY** is modeled as a **Weak Entity** dependent on **EMPLOYEE**. An employee's job history instances cannot exist without the employee. The partial key is *Date_of_Promotion*.
- **CERTIFICATION** is also modeled as a **Weak Entity** dependent on **EMPLOYEE**. The partial key is a composite of *Cert_Type* and *Date_Achieved* to account for employees potentially renewing or re-earning the same certification over time.
- Identifying relationships (**Has_Job** and **Earns_Cert**) feature **total participation** (represented by double lines) from the weak entities.
- Partial keys of weak entities are denoted using *italics*.

**Part (b): Relational Schema**

To yield the fewest number of relations, we apply the standard mapping rule for weak entities: the weak entity relation absorbs the primary key of its identifying strong owner entity. This eliminates the need for separate tables to represent the identifying relationships. The result is exactly **three** optimally consolidated relations.

Relation	Attributes (Primary Key Underlined)	Foreign (FK)	Keys
EMPLOYEE	<u>SSN</u> , FName, MIN, LName, Street, City, State, Zip, Home_Phone, Email	None	
JOB_HISTORY	<u>SSN</u> , <u>Date_Promoted</u> , Job_Title, Job_Desc, Pay_Grade, Pay_Range, Salary	SSN references EMPLOYEE(SSN)	refer-EM- encesPLOYEE(SSN)
CERTIFICATION	<u>SSN</u> , <u>Cert_Type</u> , <u>Date_Achieved</u>	SSN references EMPLOYEE(SSN)	refer-EM- encesPLOYEE(SSN)

Schema Properties:

- **EMPLOYEE:** Contains only non-multivalued attributes belonging directly to the employee.
- **JOB_HISTORY:** The composite primary key (SSN, Date_Promoted) ensures that a single employee can have multiple unique job histories recorded over time without collision.
- **CERTIFICATION:** The composite primary key (SSN, Cert_Type, Date_Achieved) allows an employee to hold multiple certifications, as well as repeatedly earn the same certification on different dates.

Q9. Consider the following scenario of a hospital:

- Patients are treated in a single ward by the doctors assigned to them. Usually each patient will be assigned a single doctor, but in rare cases he/she will be assigned two doctors.
- Healthcare assistants also attend to the patients; a number of these are associated with each ward.
- Initially the system will be concerned solely with drug treatment. Each patient is required to take a variety of drugs a certain number of times per day and for varying lengths of time.
- The system must record details concerning patient treatment and staff payment. Some staffs are paid on a part time basis and doctors and care assistants work varying amounts of overtime at varying rates (subject to grade).
- The system will also need to track what treatments are required for which patients and when, and it should be capable of calculating the cost of treatment per week for each patient.

Draw ER diagrams to show the relationships among entities of the hospital management system. (25 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Entity-Relationship (ER) Conceptual Design

Based on the hospital scenario provided, the system can be modeled using the following entities, attributes, and relationships.

Entities and Attributes

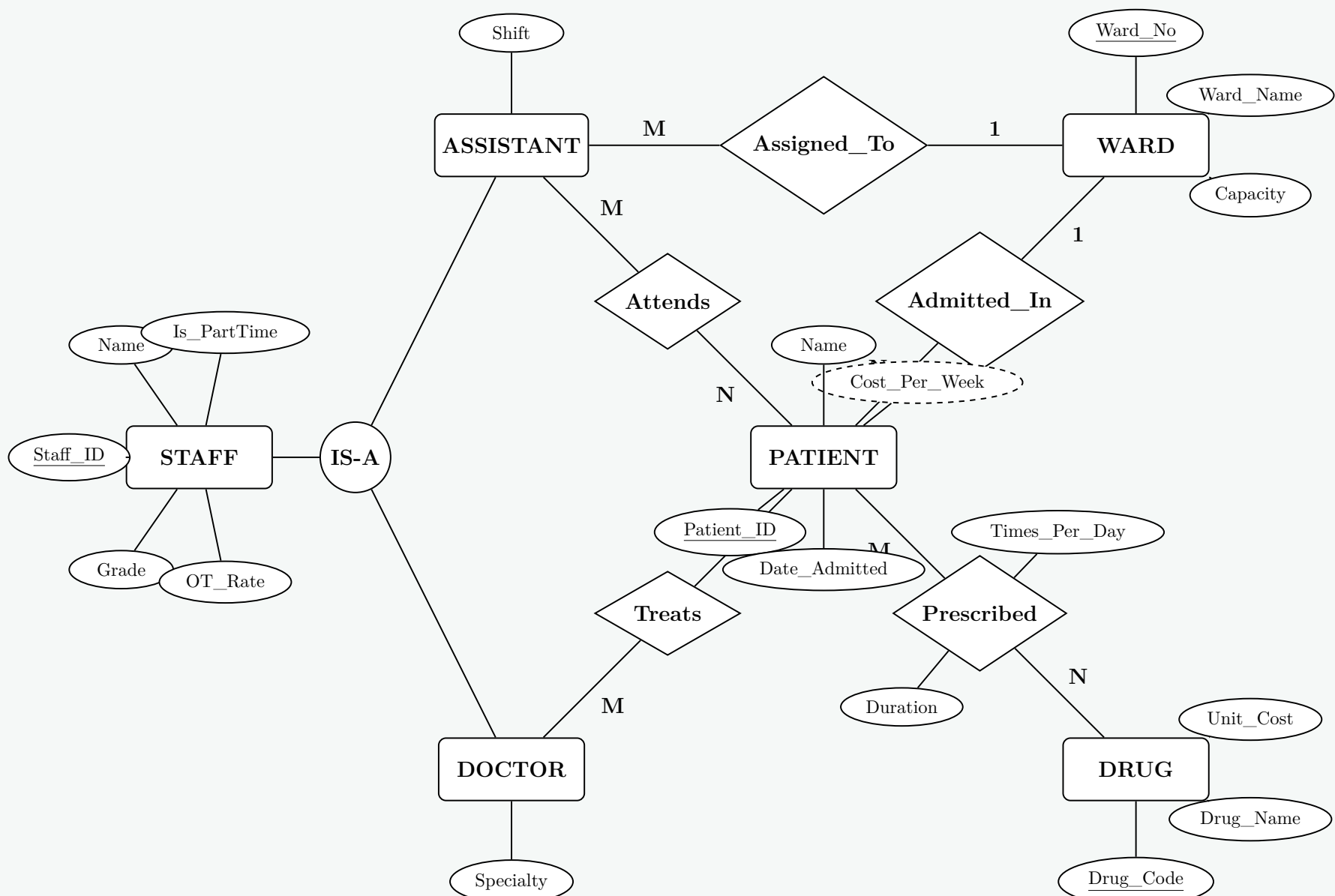
- **STAFF (Superclass):** Generalizes common attributes for all hospital workers.
 - *Attributes:* Staff_ID (Primary Key), Name, Grade, Is_PartTime, OT_Rate.
- **DOCTOR (Subclass of STAFF):** Inherits from STAFF.
 - *Attributes:* Specialty.
- **ASSISTANT (Subclass of STAFF):** Inherits from STAFF. Represents healthcare assistants.
 - *Attributes:* Shift.
- **PATIENT:** Represents individuals admitted for treatment.
 - *Attributes:* Patient_ID (Primary Key), Name, Date_Admitted, **Cost_Per_Week** (Derived Attribute: calculated from drug unit costs and dosage frequencies).
- **WARD:** The physical location where patients are treated.
 - *Attributes:* Ward_No (Primary Key), Ward_Name, Capacity.
- **DRUG:** Information regarding medicinal treatments.
 - *Attributes:* Drug_Code (Primary Key), Drug_Name, Unit_Cost.

Relationships and Cardinalities

- **Admitted_In** (PATIENT → WARD): N:1. Multiple patients are admitted to a single ward.
- **Assigned_To** (ASSISTANT → WARD): M:1. Multiple healthcare assistants are associated with one specific ward.
- **Treats** (DOCTOR → PATIENT): M:N. A doctor treats multiple patients. A patient is usually assigned one doctor, but can have two (hence, inherently M:N).
- **Attends** (ASSISTANT → PATIENT): M:N. Assistants attend to multiple patients, and a patient is attended by multiple assistants.
- **Prescribed** (PATIENT → DRUG): M:N. A patient takes multiple drugs, and a drug is given to multiple patients.

– *Relationship Attributes:* Times_Per_Day, Duration.

2. Entity-Relationship (ER) Diagram



Database Schema Design

Q1. Write the general procedure to convert a ternary relationship to binary relationships. (8 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

Conversion of a Ternary Relationship to Binary Relationships

In database design, a **ternary relationship** exists when three distinct entities participate in a single relationship. However, many implementation data models (and some design tools) restrict relationships to be strictly **binary** (degree 2). To resolve this, a ternary relationship must be decomposed into equivalent binary relationships using an **associative entity** (or weak entity).

General Procedure

The systematic procedure to convert a ternary relationship R connecting three entity types A , B , and C into binary relationships involves the following steps:

- Create an Associative Entity:** Create a new entity type, let's call it E_R , to represent the ternary relationship. This new entity is inherently existence-dependent on the original entities, often making it a weak entity.
- Establish Binary Relationships:** Remove the original ternary relationship R . Create three new binary relationships (e.g., R_A , R_B , and R_C) that connect the newly created associative entity E_R to each of the three original participating entities A , B ,

and C .

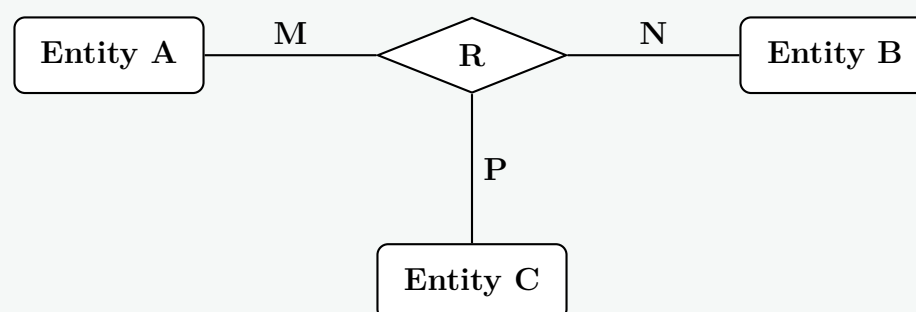
- (c) **Assign Cardinalities:** The relationship between the original entities and the new associative entity will always be **1:N** (One-to-Many).
- The original entities (A , B , and C) will have a cardinality of **1**.
 - The associative entity (E_R) will have a cardinality of **N** in all three binary relationships.
- (d) **Migrate Attributes:** Any descriptive attributes that originally belonged to the ternary relationship R must be moved to the new associative entity E_R .
- (e) **Define the Primary Key:** The primary key of the new associative entity E_R is formed by the combination (concatenation) of the primary keys of the participating entities A , B , and C . If R had its own partial key or discriminators, they are also included.

$$PK(E_R) = PK(A) \cup PK(B) \cup PK(C)$$

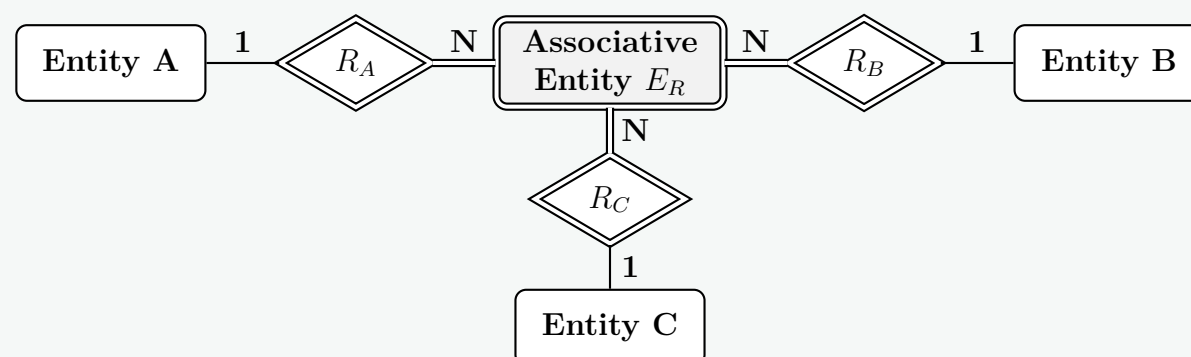
Visual Representation (ER Diagram Conversion)

The following diagram illustrates the transformation from a ternary relationship to three binary relationships linked by an associative entity.

Before: Ternary Relationship



After: Binary Equivalent



Relational Schema Mapping (Example)

If the entities A , B , and C have primary keys A_ID , B_ID , and C_ID respectively, and the ternary relationship R has an attribute $Date$, the resulting relational schema will be:

- Entity A:** ($\underline{A_ID}$, ...)
- Entity B:** ($\underline{B_ID}$, ...)
- Entity C:** ($\underline{C_ID}$, ...)
- Associative Entity E_R :** ($\underline{A_ID}$, $\underline{B_ID}$, $\underline{C_ID}$, $Date$)

Note: In the associative entity E_R , the combination of foreign keys (A_ID , B_ID , C_ID) serves as its composite primary key.

Q2. Explain the concept of total and partial participation of entities in a relationship set using an example ER diagram. **(10 marks)**

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Participation Constraints in Entity-Relationship (ER) Models

In an ER model, a **participation constraint** (also known as a minimum cardinality constraint) specifies whether the existence of an entity depends on its being related to another entity via a relationship type. It defines the minimum number of relationship instances that each entity must participate in.

There are two types of participation constraints: **Total** and **Partial**.

1. Total Participation (Existence Dependency)

- **Definition:** An entity set has total participation in a relationship set if **every** entity in the entity set is associated with at least one entity in the other entity set through that relationship.
- **Explanation:** The existence of the entity dictates that it must be related to something else. If an entity does not participate in the relationship, it cannot exist in the database.
- **Notation:** Represented by a **double line** connecting the entity rectangle to the relationship diamond.

2. Partial Participation

- **Definition:** An entity set has partial participation if **only some** entities in the entity set are associated with entities in the other entity set.
- **Explanation:** Participation is optional. An entity can exist in the database without participating in the relationship.
- **Notation:** Represented by a **single line** connecting the entity rectangle to the relationship diamond.

Example Scenario

Consider the entities **EMPLOYEE** and **DEPARTMENT**, and the relationship **Manages**:

- Not every employee is a manager. Therefore, the participation of **EMPLOYEE** in the **Manages** relationship is **partial**.
- However, every department must have a manager to function. Therefore, the participation of **DEPARTMENT** in the **Manages** relationship is **total**.

```
graph LR; EMPLOYEE[EMPLOYEE] ---|1 Partial| Manages{Manages}; Manages ---|1 Total| DEPARTMENT[DEPARTMENT]; EMPLOYEE --- Emp_ID([Emp_ID]); EMPLOYEE --- Name([Name]); DEPARTMENT --- Dept_No([Dept_No]); DEPARTMENT --- Dept_Name([Dept_Name]);
```

Comparison Table

Feature	Total Participation	Partial Participation
Mandatory/Optional	Mandatory (must participate)	Optional (may or may not participate)
Existence Dependency	Yes. Entity cannot exist without the relationship.	No. Entity can exist independently of the relationship.
ER Notation	Double continuous line	Single continuous line
Minimum Cardinality	Minimum constraint is ≥ 1	Minimum constraint is 0

Q3. How are composite attributes and multivalued attributes handled when an ER model is transformed into a relational database schema? Explain with examples. (10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Transformation of Composite and Multivalued Attributes

When mapping an Entity-Relationship (ER) model to a relational database schema, different types of attributes require distinct transformation rules to satisfy the principles of normalization (specifically First Normal Form, which requires atomic values).

1. Handling Composite Attributes

Rule: A composite attribute is **flattened**. Only its simple, component sub-attributes are included as columns in the resulting relation. The parent composite attribute itself is discarded and does not appear in the relational schema.

- Working Principle:** The relational model does not support nested structures or attributes containing sub-attributes. By extracting the leaf-level atomic attributes, the relation remains in First Normal Form (1NF).
- Example:** Consider an entity **EMPLOYEE** with a primary key *Emp_ID* and a composite attribute *Name* comprising *Fname*, *Minit*, and *Lname*.

ER Diagram

```
graph LR; Emp_ID([Emp_ID]) --- EMPLOYEE[EMPLOYEE]; EMPLOYEE --- Name([Name]); EMPLOYEE --- Lname([Lname]); Name --- Fname([Fname]); Name --- Minit([Minit]);
```

Relational Schema

<u>Emp_ID</u>	Fname	Minit	Lname
---------------	-------	-------	-------

2. Handling Multivalued Attributes

Rule: A multivalued attribute cannot be stored in a single column because it violates 1NF. It is transformed by creating a **new, separate relation** (table).

- Working Principle:**
 - Create a new relation to hold the multivalued data.
 - Include a column for the multivalued attribute itself.
 - Add the primary key of the parent entity as a **Foreign Key (FK)** in this new relation.
 - The **Primary Key (PK)** of the new relation is formed by the **combination** of the foreign key and the multivalued attribute.
- Example:** Consider an entity **EMPLOYEE** with a primary key *Emp_ID* and a multivalued attribute *Phone*.

ER Diagram

```
graph LR; Emp_ID([Emp_ID]) --- EMPLOYEE[EMPLOYEE]; EMPLOYEE --- Phone(((Phone)));
```

Relational Schema

EMPLOYEE	<u>Emp_ID</u>
----------	---------------

EMP_PHONE	<u>Emp_ID</u> (FK)	<u>Phone</u>
-----------	-----------------------	--------------

* **Composite Primary Key:** The primary key for EMP_PHONE is (*Emp_ID*, *Phone*).

** **Foreign Key Mapping:** EMP_PHONE.*Emp_ID* → EMPLOYEE.*Emp_ID*

47

Summary Comparison		
Attribute Type	Transformation Action	Result in Schema
Composite	Flatten the components. Drop the parent attribute.	Additional columns in the <i>same</i> original table.
Multivalued	Create a completely new relation.	A <i>new</i> table linked via a Foreign Key, forming a composite Primary Key.

- Q4.** A library keeps records of current loans of books to borrowers. Each borrower is identified by a borrower number; each copy of a book by an accession number. The library may hold more than one copy of any given book. Borrower name and address are stored for communications. Book information includes: title, author name(s), publisher name, publication date, ISBN, purchase price, classification (reference or fiction), and number of pages. A book may be written by multiple authors but published by a single publisher. The library uses unique in-house codes for authors and publishers. A book may cover multiple subjects; borrowers can use an online catalogue to select texts by subject, title, or author. There are restrictions on the number of books a borrower may have on loan and the loan period, depending on borrower classification (junior, adult, or organization). When a book is borrowed, the return date is automatically recorded. Other borrowers may reserve books currently on loan; the reservation date is recorded. A borrower status flag is maintained — borrowers who hold overdue books or have reached their loan limit are flagged to prevent further borrowings.
- (a) Draw an Entity-Relationship (ER) diagram to represent the library data requirements described above.
 - (b) Identify two attributes in the above scenario which can be derived using business rules. Write the schema description to store these business rules.

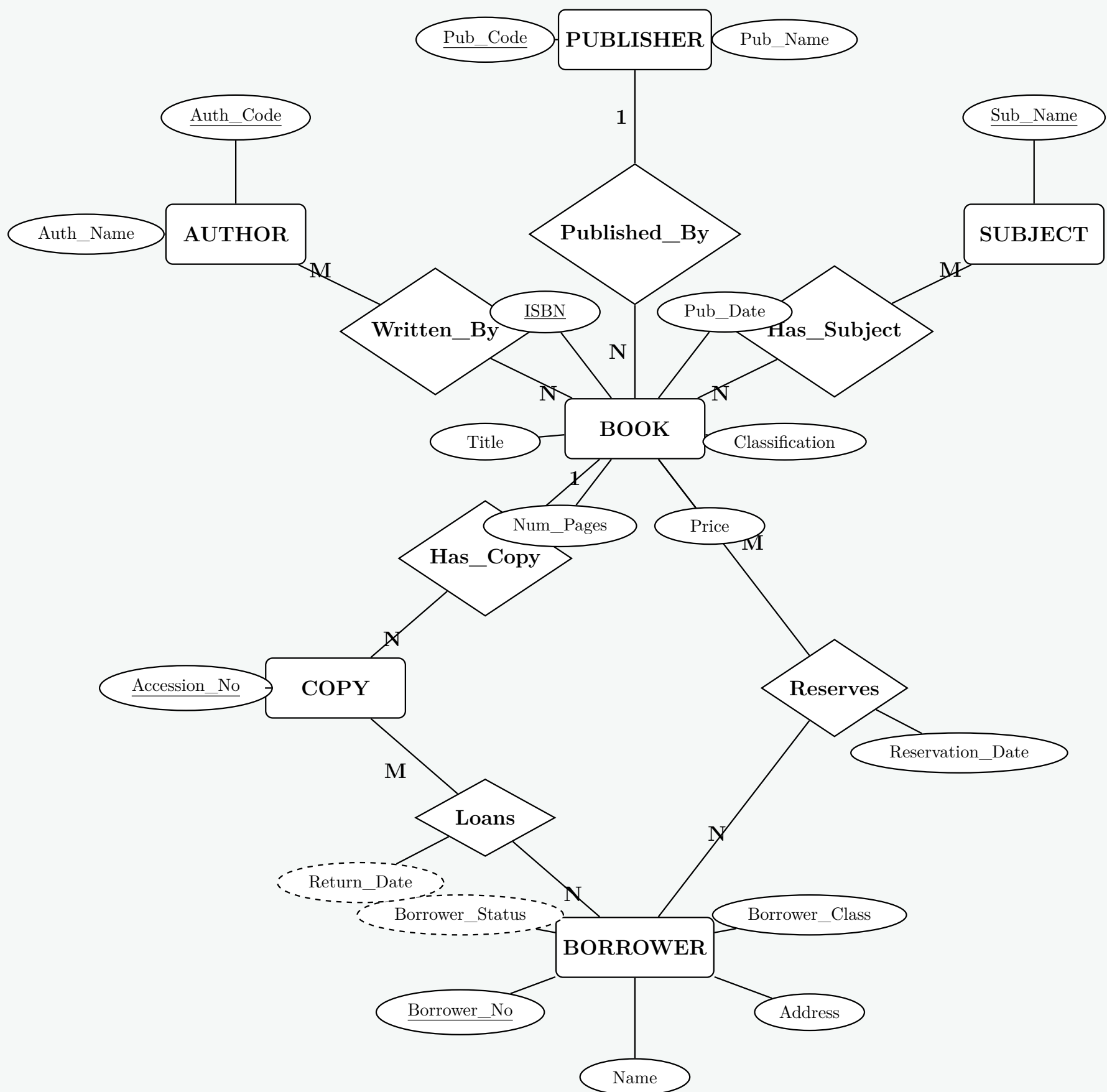
(26 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Entity-Relationship (ER) Diagram

Based on the library requirements, the conceptual data model involves central entities like **BOOK**, **COPY**, and **BORROWER**. Standard normalization rules are applied to extract **AUTHOR**, **PUBLISHER**, and **SUBJECT** into separate entities to accommodate their respective relationships (M:N for authors and subjects, 1:N for publishers).



2. Derived Attributes and Schema for Business Rules

In the given scenario, some data does not need to be hard-stored as static values but can be logically derived at runtime using established business rules.

Identified Derived Attributes

(a) **Return_Date** (on the *Loans* relationship):

When a book is borrowed, the return date is calculated by adding the designated *Loan_Period* (based on the borrower's classification) to the current checkout date.

(b) **Borrower_Status** (on the *BORROWER* entity):

This is a status flag derived by checking two conditions:

- Has the borrower exceeded their *Max_Loan_Limit*?
- Does the borrower currently hold any loans where the *Return_Date* is strictly less than the current date (overdue)?

Schema Description for Business Rules

To elegantly implement the limits based on the borrower classification (junior, adult, or organization), we create a **lookup schema** (reference table) that stores these constraints.

```
-- Relational Schema definition to store the business rules
CREATE TABLE Borrower_Class_Rules (
    Borrower_Class VARCHAR(20) PRIMARY KEY,
    Max_Loan_Limit INT NOT NULL,
    Loan_Period_Days INT NOT NULL
);
```

Relational Representation:

BORROWER_CLASS_RULES (Borrower_Class, Max_Loan_Limit, Loan_Period_Days)

Working Principle: When processing a loan, the DBMS retrieves the **Loan_Period_Days** for the user's **Borrower_Class** from this rule table to derive the **Return_Date**. Simultaneously, the system compares the user's current active loans against the **Max_Loan_Limit** to compute the **Borrower_Status** flag.

Q5. What is the difference between a view and a materialized view? Draw an example diagram of a disjoint specialization. **(10 marks)**
[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Difference Between a View and a Materialized View

In database management systems, both **Views** and **Materialized Views** are utilized to simplify complex queries, enhance security, and present a customized schema perspective to users. However, their underlying working principles and storage mechanisms differ significantly.

- View (Virtual View):** A view is a logical, virtual table defined by a SQL query. It does not store any data physically on the disk. Every time a view is queried, the DBMS dynamically executes the underlying SQL query against the base tables to fetch the current data.
- Materialized View:** A materialized view is a database object that physically stores the result set of its defining query on the disk. Because the data is pre-computed and stored, querying a materialized view is significantly faster. However, it requires periodic or triggered refreshing to synchronize its data with the underlying base tables.

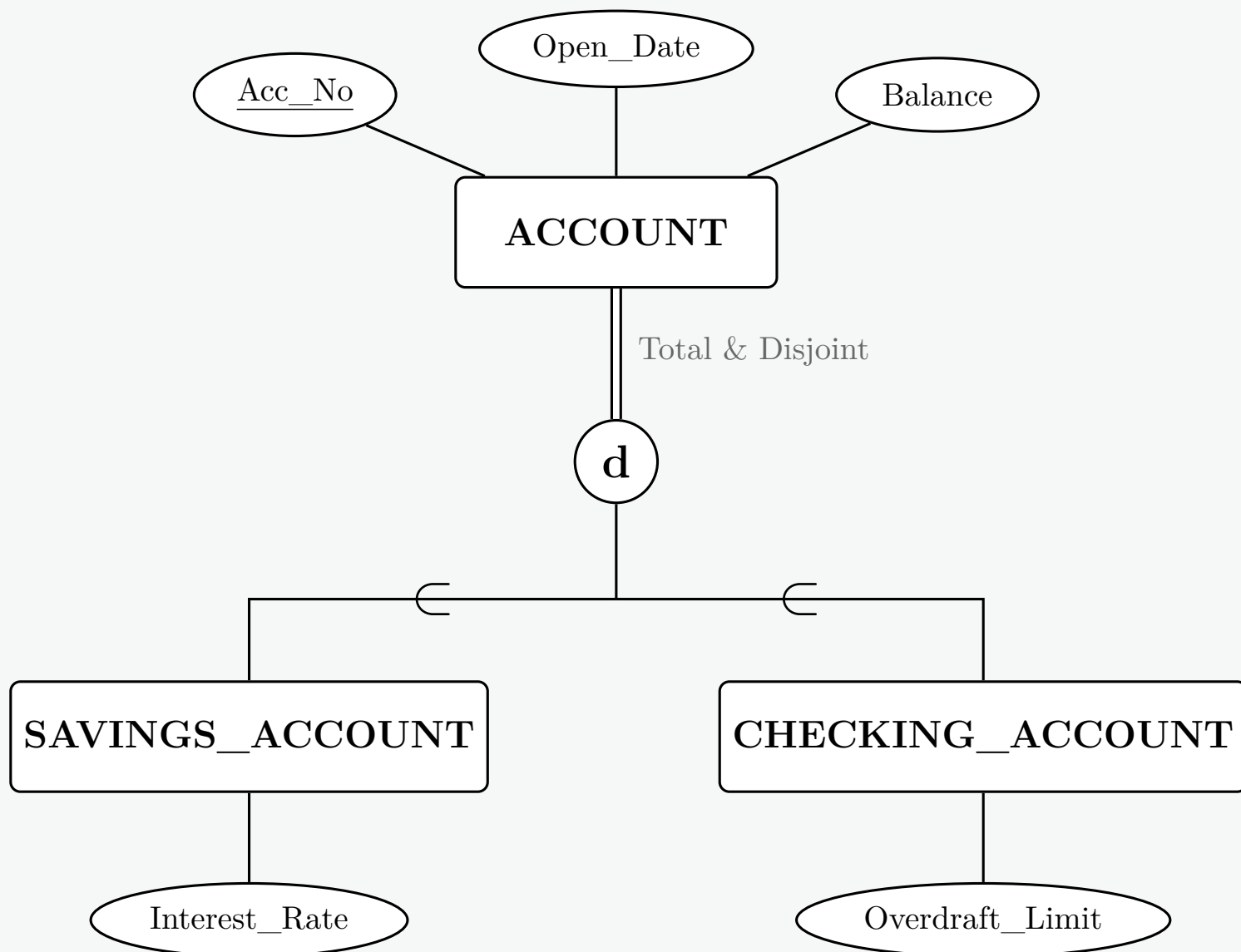
Comparison Table

Feature		Standard View	Materialized View
Storage		Virtual; stores only the query definition, not the data.	Physical; stores both the query definition and the actual data.
Performance		Slower for complex queries (computed on the fly at run-time).	Faster (data is pre-computed and retrieved directly from disk).
Data Freshness		Always up-to-date with the base tables.	May contain stale data unless it is actively refreshed.
Storage Cost		Negligible (only metadata is stored).	High (consumes disk space proportional to the result set).
Primary Case	Use	Security (hiding columns/rows), simplifying complex joins for users.	Data warehousing, caching expensive aggregations, business intelligence.

2. Disjoint Specialization in EER Modeling

In the Enhanced Entity-Relationship (EER) model, **specialization** is the process of defining a set of subclasses from a single superclass. A **disjoint specialization** specifies a constraint where an entity from the superclass can belong to **at most one** subclass. It is impossible for an entity to be a member of multiple subclasses simultaneously.

Example Scenario: Consider a banking system where an **ACCOUNT** can be either a **SAVINGS_ACCOUNT** or a **CHECKING_ACCOUNT**. An account cannot be both simultaneously. This requires a disjoint constraint (denoted by a circle containing a **d**). Additionally, if every account *must* be categorized as one of these two (Total Specialization), we use a double line connecting the superclass to the disjoint circle.



Note on the Diagram: The double line from **ACCOUNT** to the **d** circle denotes *total participation* (every account must be a specialized type). The **d** ensures *disjointness* (an account cannot be both). The $\subset$ symbols denote the subset relationship (subclasses inherit from the superclass).

Q6. YouTube is a very popular platform for hosting videos. People create channels and upload videos on that. Sometimes the creators can batch up the videos into playlists. Videos can be free to view or behind paywall which requires subscription to YouTube. Premium. A free video can be watched with or without signing into YouTube. However, only signed-in users' views are considered when counting the number of viewers for a video. A signed-in account may or may not have corresponding channels. Each video has a comment section where a signed-in user can comment; and comment to other peoples comments. Also, videos contain some tags (given by the uploader or YouTube itself) that help to filter videos. Signed-in users can subscribe to other channels. Now, for each of the mentioned features below, design the corresponding portion of ERD for YouTube only. Add attributes as you see fit and mark primary keys.

- User management:** Users should be able to create accounts, create channels on top of the accounts (one user can create at most one channel), and subscribe to different channels.
- Video hosting:** Creators should be able to upload videos with different tags, and create playlists by batching up multiple videos. Users should be able to view the videos, find them in their history when needed and, also, check the number of viewers to the videos.
- Commenting:** Users should be able to comment on any video any number of times, They should also be able to comment on other peoples comments.
- Subscription system:** Creators should be able to mark some videos as premium (i.e., only subscribed users can view). Users should be able to buy subscriptions up to some end date.

(e) Implement relational schemas for each ERD you designed in this question. **(15 marks)** [CSE 215 | L-2/T-1 | 2022–23]

(20 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

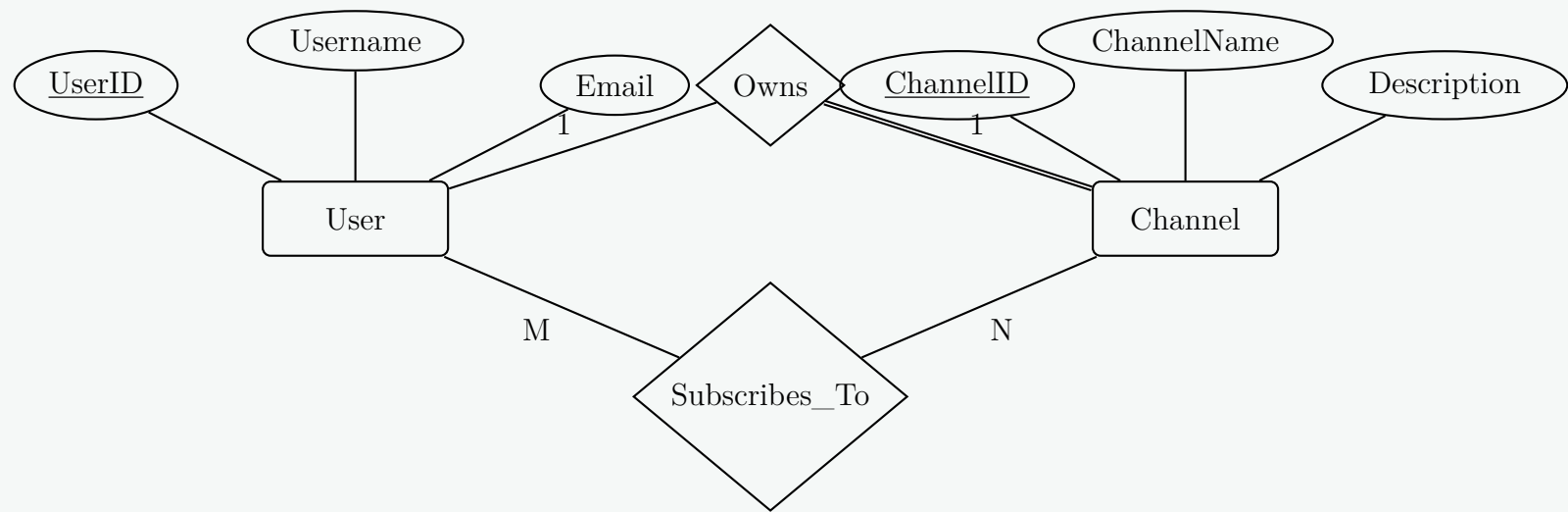
Entity-Relationship Diagrams (ERD) for YouTube Platform

In this section, we decompose the platform's requirements into four distinct, logical ERD components to represent User Management, Video Hosting, Commenting, and Subscription mechanisms. Standard ERD notation is used: rectangles for entities, diamonds for relationships, ellipses for standard attributes, double ellipses for multivalued attributes, and dashed ellipses for derived attributes.

1. User Management System

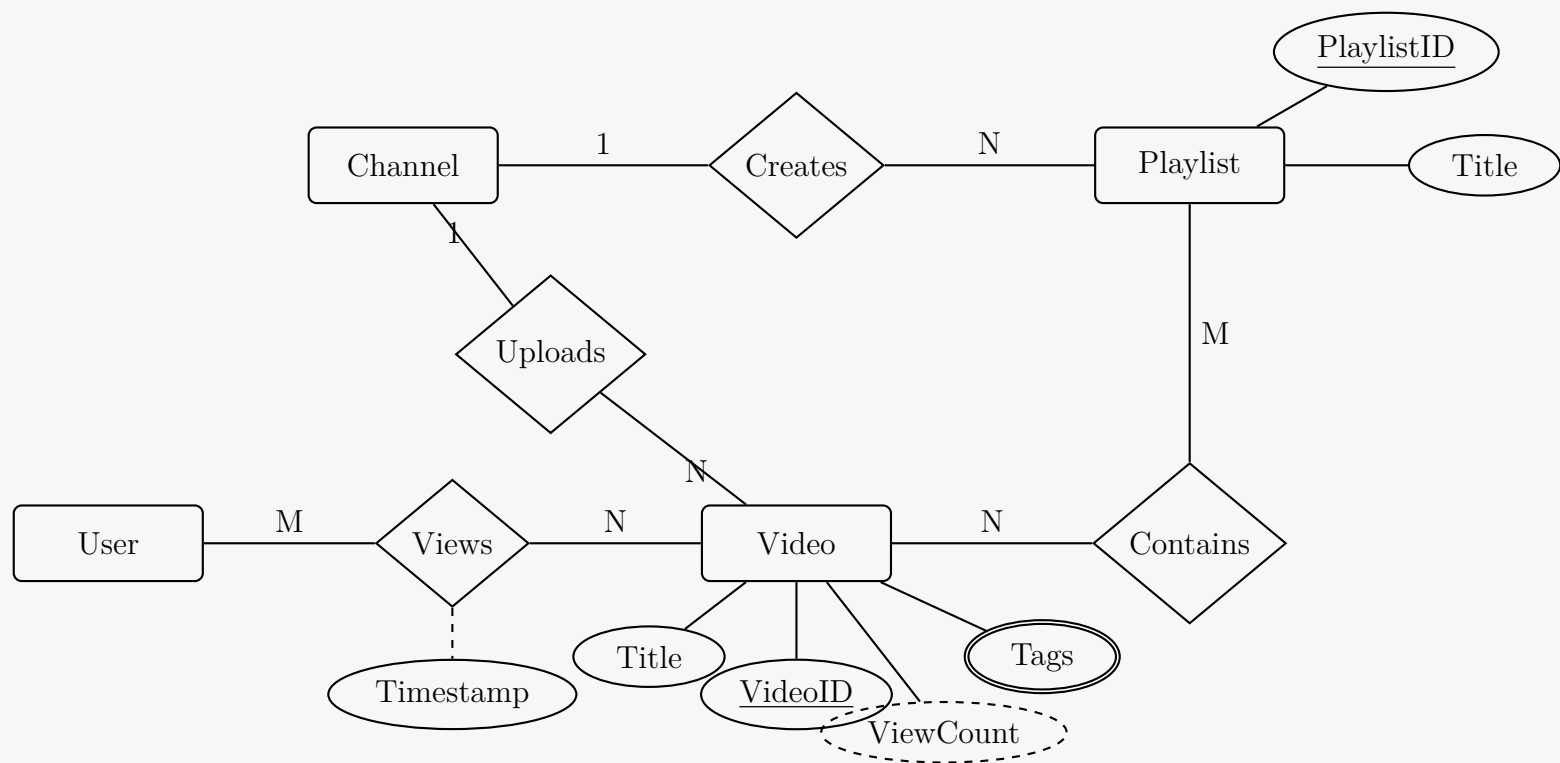
Working Principle: A **User** can create an account and optionally create at most one **Channel**. Therefore, the **Owns** relationship is 1:1, with partial participation from the User (not all users create channels) and total participation from the Channel (every channel must be owned by a user). The **Subscribes_To** relationship is M:N, as a user can subscribe to many channels, and a channel can

have many subscribers.



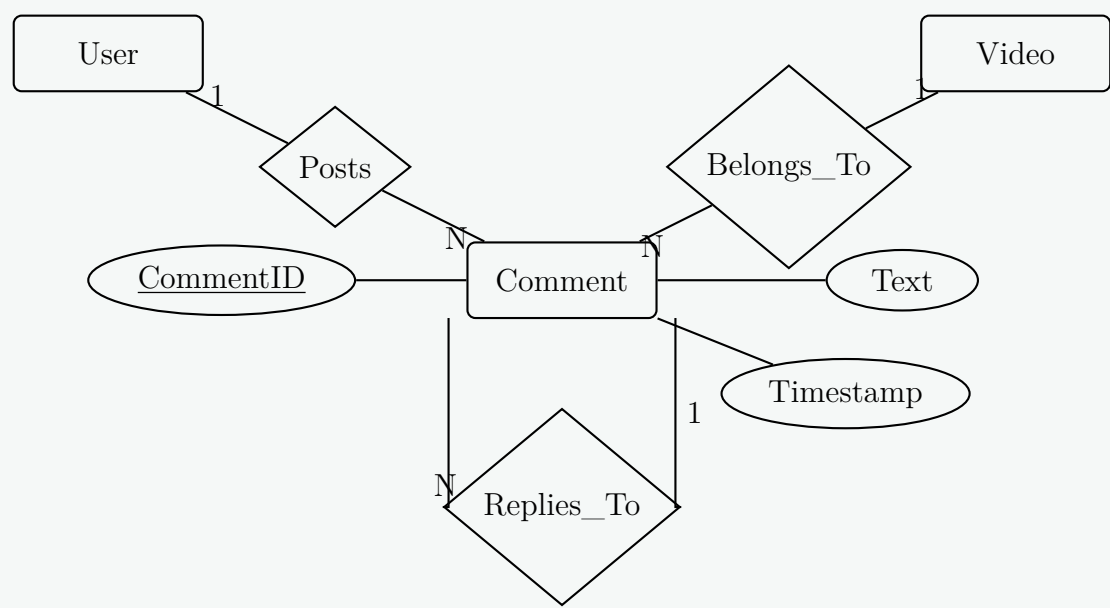
2. Video Hosting System

Working Principle: The **Channel** entity uploads **Video** entities (1:N) and creates **Playlist** entities (1:N). Playlists can contain multiple videos, and videos can belong to multiple playlists (M:N). Signed-in **Users** watch videos, which is modeled by the M:N **Views** relationship. **Views** includes a **Timestamp** attribute to construct watch history. **Tags** is a multivalued attribute, while **ViewCount** is a derived attribute calculated from the **Views** relationship.



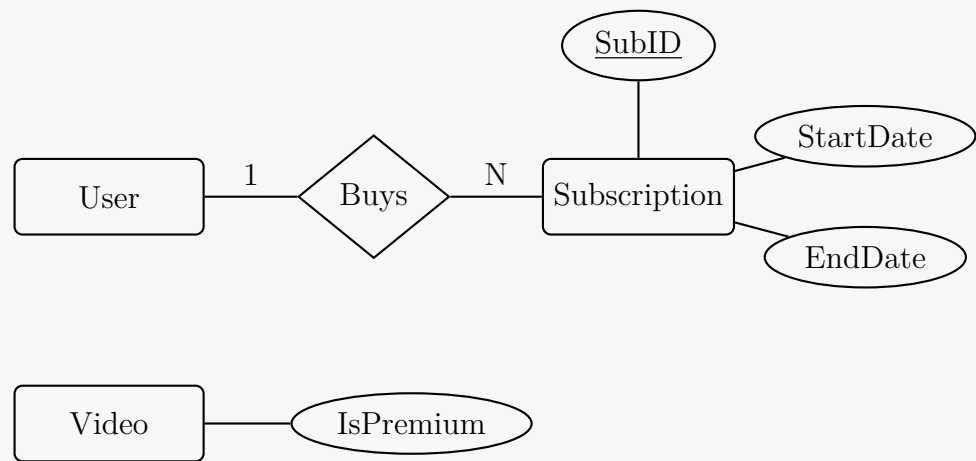
3. Commenting System

Working Principle: A **Comment** is generated by a single **User** (1:N) and belongs to a specific **Video** (1:N). To facilitate comment threading and replies, we model a unary relationship **Replies_To** on the **Comment** entity, establishing a recursive 1:N structural hierarchy where one parent comment can have multiple reply comments.



4. Subscription System

Working Principle: A **User** can buy a **Subscription** to gain premium privileges (1:N relationship, as a user may renew or buy different tiered subscriptions over time). The premium status of a **Video** is encapsulated as a simple boolean attribute **IsPremium**. Only subscribed users are granted application-layer access to view videos marked with this flag.



5. Relational Schemas

Below is the complete relational schema mapping for the provided ERDs. Primary keys (PK) are strictly underlined, and foreign keys (FK) refer to primary keys in their respective parent relations.

Relation	Attributes (Primary Keys Underlined)	Foreign Keys (FK)
User	<u>UserID</u> , Username, Email	None
Channel	<u>ChannelID</u> , ChannelName, Description, UserID	UserID → User(UserID)
Subscribes	<u>UserID</u> , <u>ChannelID</u>	UserID → User, ChannelID → Channel
Playlist	<u>PlaylistID</u> , Title, ChannelID	ChannelID → Channel(ChannelID)
Video	<u>VideoID</u> , Title, IsPremium, ChannelID	ChannelID → Channel(ChannelID)
Video_Tags	<u>VideoID</u> , <u>Tag</u>	VideoID → Video(VideoID)
Playlist_Video	<u>PlaylistID</u> , <u>VideoID</u>	PlaylistID → Playlist, VideoID → Video
Viewing_History	<u>UserID</u> , <u>VideoID</u> , <u>Timestamp</u>	UserID → User, VideoID → Video
		UserID → User(UserID)
Comment	<u>CommentID</u> , Text, Timestamp, UserID, VideoID, ParentID	VideoID → Video(VideoID)
		ParentID → Comment(CommentID)
Subscription	<u>SubID</u> , StartDate, EndDate, UserID	UserID → User(UserID)

Key Mapping Decisions & Notes

- **1:1 Relationship (Owns):** The 1:1 relationship between **User** and **Channel** is mapped by placing **UserID** as a foreign key in the **Channel** relation (as a user might not have a channel, but every channel must have a user, ensuring no nulls).
- **M:N Relationships:** **Subscribes**, **Playlist_Video**, and **Viewing_History** are intersection tables created from the many-to-many ERD relationships.
- **Multivalued Attribute:** The **Tags** attribute is extracted into its own table **Video_Tags** with a composite primary key.
- **Unary Relationship (Replies_To):** Modeled via **ParentID** within the **Comment** table, referencing the same table's **CommentID**. It is nullable to allow top-level comments.
- **Derived Attribute:** **ViewCount** is omitted from the base schema as it is dynamically computed using a **COUNT()** aggregate function query on the **Viewing_History** table.

Q7. Give an example of a Non-Binary Relationship. What issues arise for Non-Binary Relationships? (5 marks) [CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Non-Binary Relationship: Definition and Example

Definition

A **non-binary relationship** (also known as an n -ary relationship) is a relationship in an Entity-Relationship (ER) model that connects **three or more** entity sets simultaneously. Unlike a binary relationship, which relates exactly two entities, an n -ary relationship represents an atomic association among a combination of entities that cannot be naturally separated without losing contextual semantics. The most common form of a non-binary relationship is a **ternary relationship** ($n = 3$).

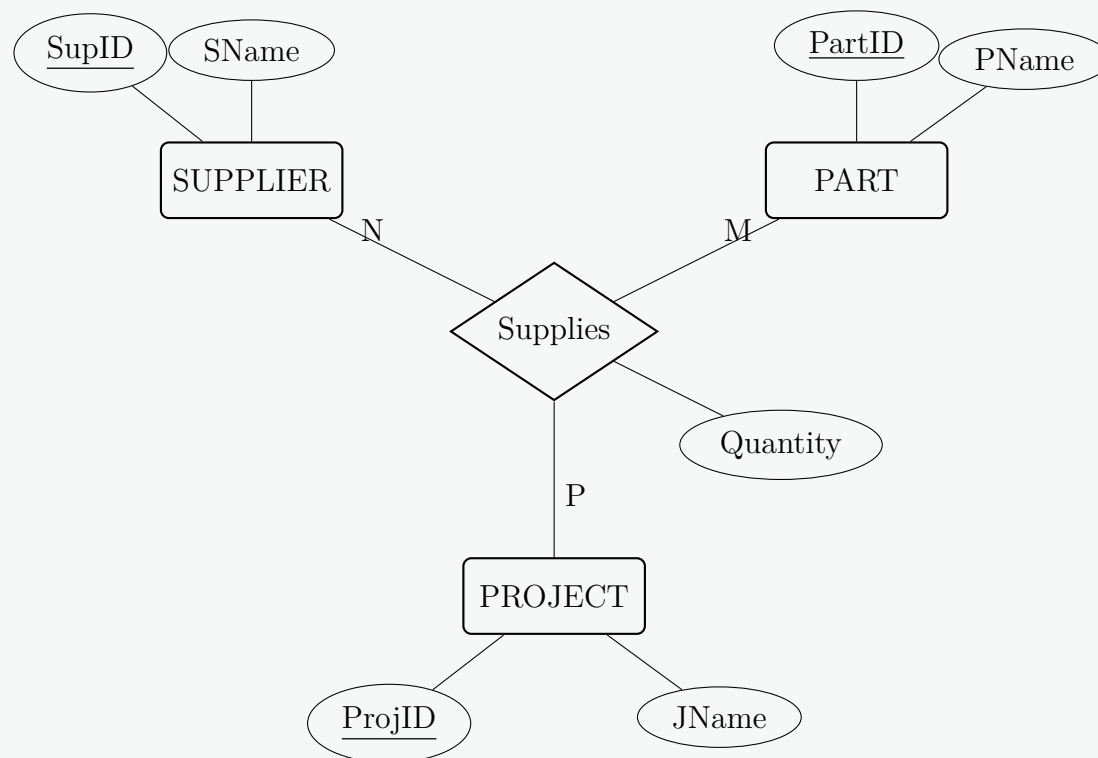
Real-World Example

Consider a manufacturing and supply chain system consisting of three entity sets:

- **Supplier:** The vendor providing components.
- **Part:** The specific item or component being supplied.
- **Project:** The construction project or department utilizing the parts.

A binary relationship between **Supplier** and **Part** would only indicate what parts a supplier can provide. A binary relationship between **Supplier** and **Project** would only specify which suppliers serve which projects. However, to record that "*Supplier S_1 supplies Part P_1 to Project J_1* " requires a **ternary relationship** named **Supplies**. This association is atomic because a supplier might supply a part to one specific project, but not to another.

Ternary Relationship Diagram (TikZ)



2. Issues and Challenges in Non-Binary Relationships

Designers face several structural, semantic, and performance complications when dealing with non-binary relationships:

(a) Ambiguity in Cardinality Constraints

- In a binary relationship, structural constraints (1:1, 1:N, M:N) have clear and standardized meanings. In an n -ary relationship, the interpretation of cardinalities becomes ambiguous and varies across design methodologies (e.g., Chen notation vs. Barker notation).
- For instance, a directed arrow or a "1" pointing from **Project** to the **Supplies** relationship can mean two completely different things depending on the conventions chosen:
 - *Interpretation A*: For a given combination of a Supplier and a Part, there is at most one unique Project.
 - *Interpretation B*: A single Project is constrained to only ever receive one unique combination of Part and Supplier.

(b) False Decomposition Traps (Loss of Semantics)

- Database designers often attempt to simplify their schema by splitting a ternary relationship into three distinct binary relationships: (Supplier-Part), (Part-Project), and (Supplier-Project).
- This often introduces a semantic flaw known as a **connection trap** or **fan trap**. Knowing that Supplier *A* makes Part *X*, Part *X* is used in Project *Y*, and Supplier *A* works on Project *Y* does *not* guarantee that Supplier *A* supplies Part *X* specifically to Project *Y*. The exact instance mapping is completely lost.

(c) Over-Structuring / Redundancy

- Conversely, using an n -ary relationship when separate binary relationships are sufficient introduces unnecessary constraints and data redundancies. For example, if a supplier always supplies a part to every project uniformly, treating this as an atomic ternary entity artificially pairs instances that do not depend on one another.

(d) Logical Representation and Null Value Pitfalls

- In a true ternary relationship, all participating entity instances must exist simultaneously to form a valid relationship tuple.
- If a supplier introduces a new part that has not yet been assigned to any project, this instance cannot be inserted into the ternary relation without introducing a NULL value for the ProjID attribute, which violates the integrity rules if ProjID forms part of the primary key.

(e) Physical Performance and Optimization Issues

- When converted into relational schemas, n -ary tables require composite primary keys composed of the primary keys of all participating entities: $PK = \{SupID, PartID, ProjID\}$.
- Queries tracking down basic relationships often degrade into expensive multi-way join operations across massive junction tables, leading to storage layout overhead and indexing complexities.

Comparison: Binary Decomposition vs. Ternary Representation		
Property	Decomposed Binary Setup	True Ternary Setup
Structural Complexity	Low (Simple 2-entity links)	High (n -way connection mapping)
Semantic Precision	Prone to connection traps	Completely accurate atomic tracking
Primary Key Composition	Simple keys	Large composite foreign keys
Instance Independence	Entities can pair independently	All n entities must coexist in the tuple

Q8. Suppose you are given the following requirements for a simple database for the Bangladesh Premier League (BPL) in a season:

- The BPL has many teams; each team has a name, a city, a coach, a captain, and a set of players.
- Each player belongs to only one team; a player may not be hired by any team.
- Each player has a name, a fielding position (such as slip, mid-wicket, long on, long off etc), a ranking, and a set of injury records.
- A team captain is also a player.
- Injury records must have an id, period (start and end), and a short description.
- A game is played between two teams (host_team and guest_team) with a date (such as May 11th, 2023) and scores (winning and losing team scores such as 143/10 and 148/6).

Construct a clean and concise E/R diagram for the BPL database. List your assumptions and clearly indicate cardinality mappings and any role indicators. **(10 marks)**

[CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Design Assumptions

To establish exact operational boundaries and clear cardinality mapping within the Entity-Relationship (ER) model, the following domain-specific design assumptions are made based on the provided requirements:

- **Team & Captaincy:** A team has exactly one captain at any time. The captain must be an active player on that team. This is modeled via a 1 : 1 relationship with a role indicator (**Captain**).
- **Player Assignment:** Each player belongs to **at most one** team. The constraint "a player may not be hired by any team" indicates partial participation (0 : 1) from the **PLAYER** side.
- **Injury Records:** Injury records cannot exist independently of a player. Thus, **INJURY_RECORD** is modeled as a **weak entity** set structurally dependent on the **PLAYER** entity set via an identifying relationship. Its unique descriptor within a player is an partial key (InjuryID).
- **Game Scheduling:** A game represents a head-to-head match between exactly two distinct teams: a **Host Team** and a **Guest Team**. It is modeled as a binary relationship between **TEAM** and itself (a recursive/unary relationship) with distinct role indicators, capturing the date and performance statistics (scores).

2. Entity-Relationship Diagram

3. Schema Map Summary				
Relation/Link	Type	Participation (Team)	Participation (Player)	Structural Meaning
BelongsTo	Binary	Total (1)	Partial (<i>N</i>)	A player is restricted to at most one team; a team possesses many players.
Captains	Binary	Partial (1)	Partial (1)	One specific player manages a specific team as a captain.
Plays	Unary	Partial (1)	—	Connects a host team and guest team together for an individual fixture.
Has	Weak	—	Total (1)	Structural dependency linking players tightly to their history of ailments.

- Q9.** Consider the following descriptions of an organization having many EMPLOYEES.
- (i) An EMPLOYEE has a name, nid number, data of birth, and address.
 - (ii) Based on the EMPLOYEE's Job, EMPLOYEE may be further grouped into: SECRETARY, ENGINEER, TECHNICIAN, and MANAGER
 - (iii) Based on the EMPLOYEE's method of pay, EMPLOYEE may also be grouped into: SALARIED_EMPLOYEE, and HOURLY_EMPLOYEE
 - (iv) A salaried employee who is also an engineer belongs to the two subclasses: ENGINEER, and SALARIED_EMPLOYEE
 - (v) A salaried employee who is also an engineering manager belongs to the three subclasses: MANAGER, ENGINEER, and SALARIED_EMPLOYEE.
 - (vi) We want to record typing speed of each SECRETARY.
 - (vii) Each technician has a technical grade (a number) which depends on his skill level
 - (viii) Each Engineer has a type (ME, CSE, EEE, etc ...)
 - (ix) Each MANAGER manages at least one project, each project has a project id, title, and a project value.
 - (x) Each HOURLY_EMPLOYEE is a member of some TRADE_UNION which has a name and member count.
- Construct an E/R diagram with the above description; clearly indicate the cardinality mappings as well as any role indicators in your E/R diagram.
 - Convert the E/R diagram into a set of relational database schema. You should avoid keeping null values in attributes.

(20 marks)

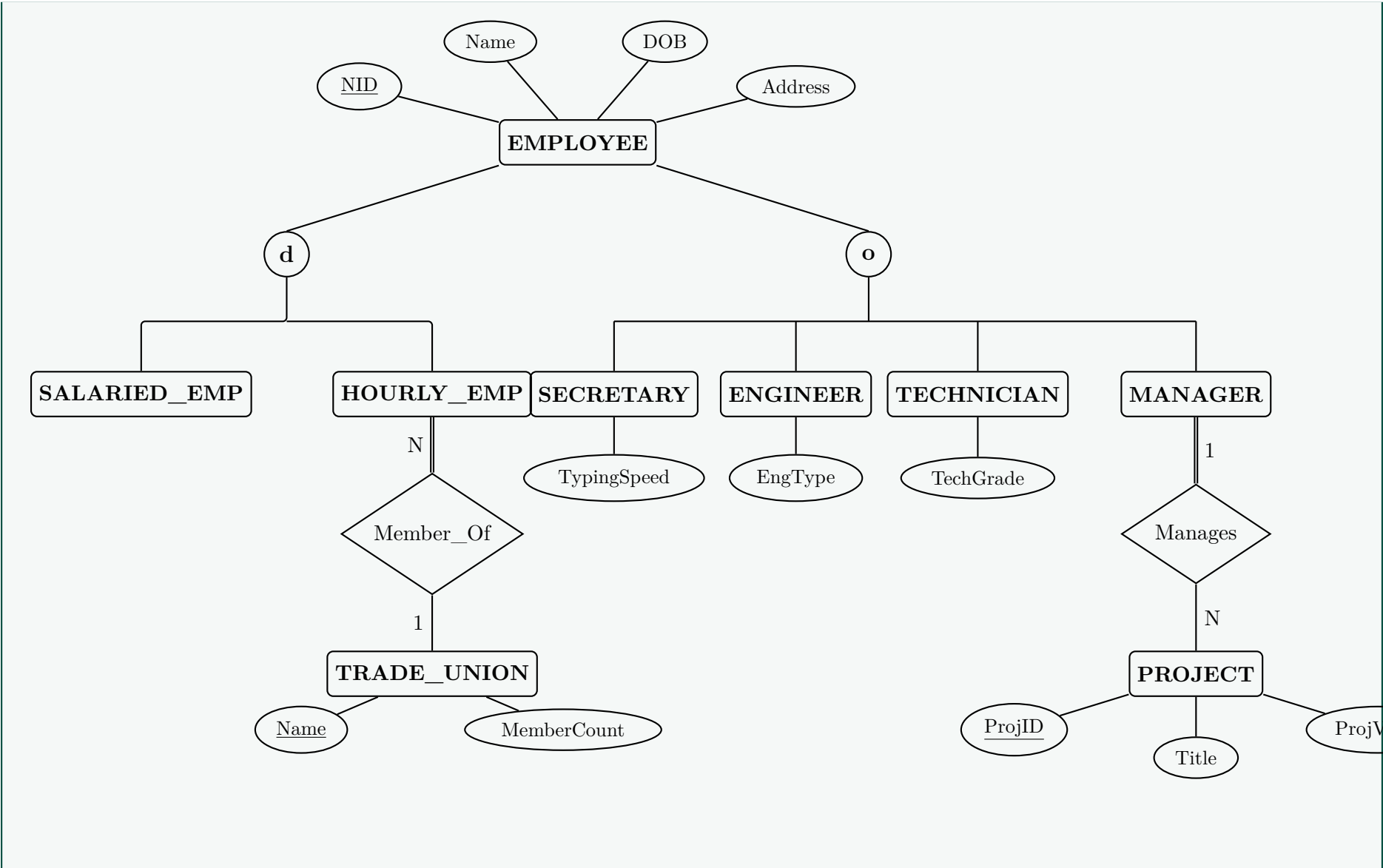
[CSE 215 | L-2/T-2 | 2021–22]

Answer

Part (i): Enhanced Entity-Relationship (EER) Diagram

Design Principles & Generalization Modeling:

- **Superclass:** EMPLOYEE acts as the root entity containing shared attributes (NID, Name, DOB, Address).
- **Generalization 1 (Pay Method):** Disjoint (**d**) specialization because an employee is either a SALARIED_EMPLOYEE or an HOURLY_EMPLOYEE, but not both.
- **Generalization 2 (Job Role):** Overlapping (**o**) specialization because an employee can simultaneously hold multiple roles (e.g., an ENGINEER who is also a MANAGER).
- **Relationships:** The MANAGER explicitly participates totally in the **Manages** relationship with PROJECT (1:N assumption). Similarly, HOURLY_EMPLOYEE totally participates in the **Member_Of** relationship with TRADE_UNION (N:1 assumption).



Part (ii): Relational Schema Mapping

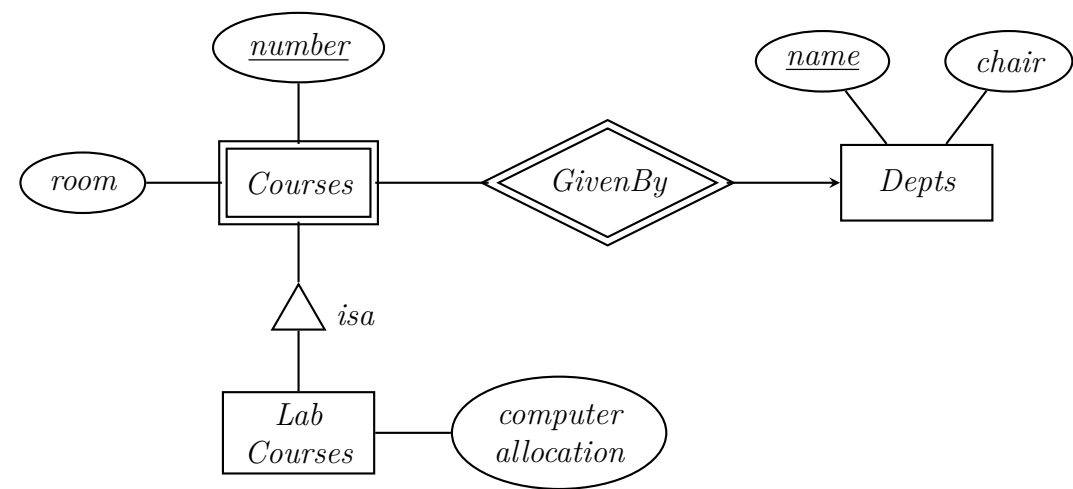
To **avoid null values**, we use the approach where multiple relations are created (one for the superclass and one for each subclass). Subclass tables only contain their specific attributes and the primary key of the superclass.

Relation	Attributes (Primary Keys Underlined)	Foreign Keys (FK)
EMPLOYEE	<u>NID</u> , Name, DOB, Address	None
SALARIED_EMP	<u>NID</u>	NID → EMPLOYEE(NID)
TRADE_UNION	<u>Name</u> , MemberCount	None
HOURLY_EMP	<u>NID</u> , UnionName	NID → EMPLOYEE(NID) UnionName → TRADE_UNION(Name)
SECRETARY	<u>NID</u> , TypingSpeed	NID → EMPLOYEE(NID)
ENGINEER	<u>NID</u> , EngType	NID → EMPLOYEE(NID)
TECHNICIAN	<u>NID</u> , TechGrade	NID → EMPLOYEE(NID)
MANAGER	<u>NID</u>	NID → EMPLOYEE(NID)
PROJECT	<u>ProjID</u> , Title, ProjValue, ManagerNID	ManagerNID → MANAGER(NID)

Design Notes on the Transformation:

- (a) **Generalization:** Subclasses inherit the primary key (NID) from EMPLOYEE. Since a single employee can be an engineer and a manager, their NID will simply appear in both the ENGINEER and MANAGER tables without issue. No NULL values are generated for attributes that don't apply to specific roles.
- (b) **1:N Relationship (Manages):** The 1:N relationship between MANAGER and PROJECT is handled by placing the manager's primary key (ManagerNID) into the PROJECT table as a foreign key.
- (c) **N:1 Relationship (Member_Of):** The relationship between HOURLY_EMPLOYEE and TRADE_UNION is handled by placing the union's primary key (UnionName) into the HOURLY_EMP table.

Q10. Convert the E/R diagram of Figure below into a relational database schema.



(5 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

Conversion of E/R Diagram to Relational Database Schema

The given Entity-Relationship diagram consists of a strong entity (**Depts**), an identifying relationship (**GivenBy**), a weak entity (**Courses**), and a subclass entity (**LabCourses**) structured under an **ISA** hierarchy. Below is the step-by-step systemic conversion of this E/R model into a set of normalized, professional relational database schemas.

—

1. Analysis of E/R Component Structural Constraints

- **Depts (Strong Entity):** Contains a primary key attribute **name** and a descriptive attribute **chair**.
- **Courses (Weak Entity):** Has a partial key (discriminator) **number** and an attribute **room**. Its existence and identity depend completely on the strong entity **Depts** via the identifying relationship **GivenBy**.
- **GivenBy (Identifying Relationship):** Represents a many-to-one ($\text{Many} \rightarrow 1$) structural constraint from **Courses** to **Depts**, indicated by the directed arrow towards **Depts**.
- **LabCourses (Subclass via ISA):** Inherits the primary identity of the superclass **Courses** and introduces its own specialized attribute **computer_allocation**.

—

2. Relational Schema Mapping

To translate these components seamlessly without introducing redundancy or illegal null states, the relational database tables are structured as follows:

Table 1: Depts (Strong Entity Relation)

This relation represents the department entity directly.

- **Primary Key (PK):** name

Depts(name, chair)

Table 2: Courses (Weak Entity Relation)

A weak entity does not possess a sufficient primary key on its own. Therefore, its schema must combine its partial key (**number**) with the primary key of its identifying owner department (**dept_name**) to create a composite primary key.

- **Primary Key (PK):** (number, dept_name)
- **Foreign Key (FK):** **dept_name** referencing Depts(name) with an **ON DELETE CASCADE** structural rule.

Courses(number, dept_name, room)

Table 3: LabCourses (Subclass Relation)

Using the standard textbook approach for ISA specialization hierarchies, a dedicated relation is created for the subclass. It imports the entire primary key configuration of its superclass (**Courses**) to establish its own primary identity and maintain referential integrity.

- **Primary Key (PK):** (number, dept_name)
- **Foreign Key (FK):** (**number**, **dept_name**) referencing Courses(number, dept_name).

LabCourses(number, dept_name, computer_allocation)

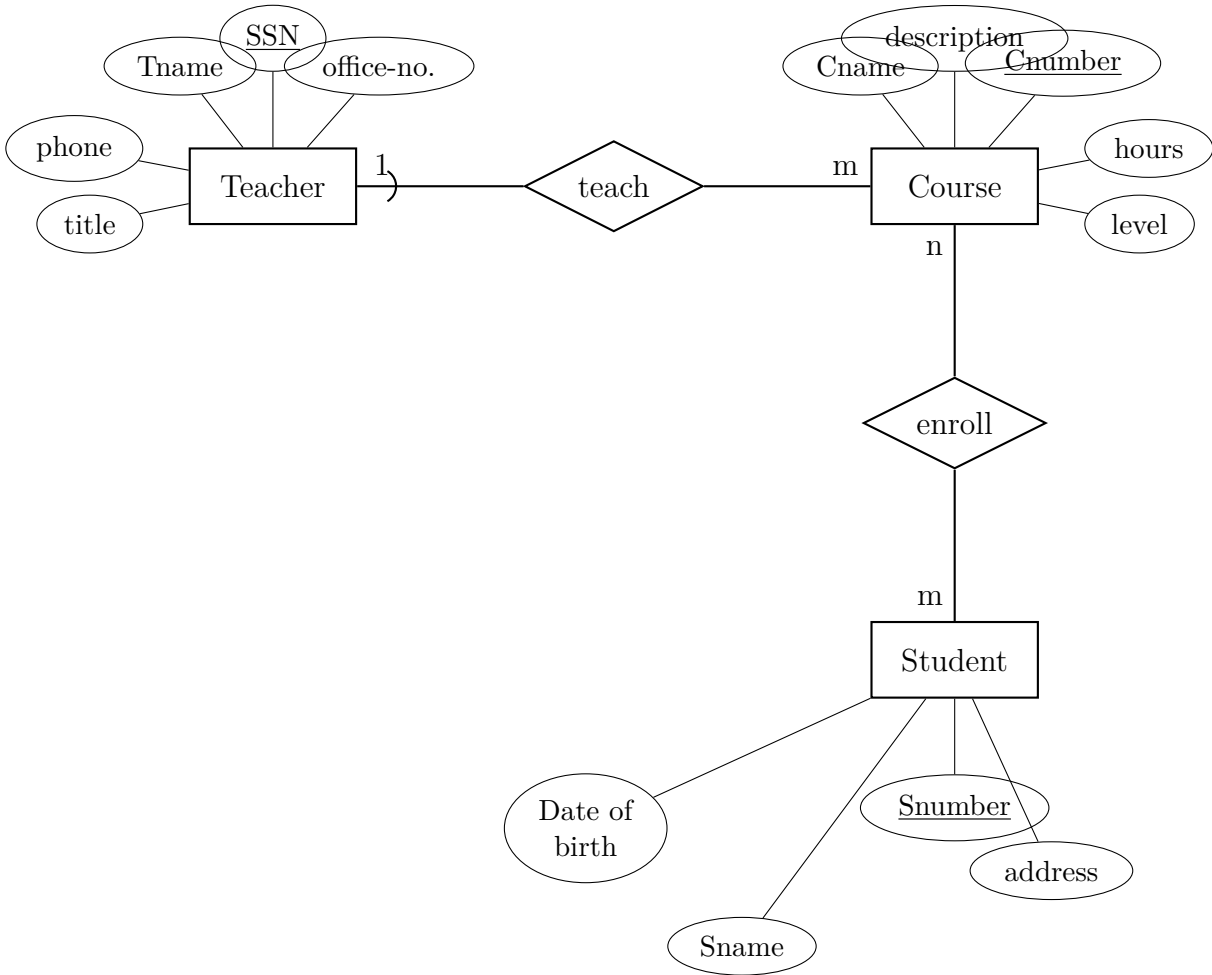
—

3. Summary Relational Schema Table

Relation Name	Attributes (Primary Key Underlined)	Foreign Keys & Constraints
Depts	<u>name</u> , chair	None
Courses	<u>number</u> , <u>dept_name</u> , room	dept_name → Depts(name)
LabCourses	<u>number</u> , <u>dept_name</u> , computer_allocation	(number, dept_name) → Courses(number, dept_name)

Design Note: The mapping rules state that an identifying relationship table for a weak entity (**GivenBy**) is completely redundant and absorbed into the weak entity relation itself (**Courses**). This occurs because the mapping inherently pairs every entry of **Courses** with its corresponding unique department owner via the composite foreign key attribute **dept_name**, preventing any loss of data while minimizing join overhead.

Q11. For the given E-R diagram with entities **Teacher**, **Student**, **Course**, and weak entity **Enroll**, determine which of the following statements hold:



- (a) Enroll is a strong entity set.
- (b) All teachers must teach courses.
- (c) All students must enroll in courses.
- (d) All courses must be enrolled by students.
- (e) All courses must be taught by teachers.
- (f) Each course is taught by only one teacher.
- (g) Each course is enrolled by only one student.
- (h) Each teacher may teach many courses.
- (i) Each student can enroll in many courses.
- (j) Teacher is a relationship set.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Evaluation of Statements Based on the E-R Diagram

Based on standard Entity-Relationship (E-R) modeling semantics depicted in the diagram, we analyze the structural constraints (cardinality and participation) and object types to determine the validity of each statement.

- **Rectangles** represent **Entity Sets** (Teacher, Course, Student).
- **Diamonds** represent **Relationship Sets** (teach, enroll).

- **Numbers/Letters on edges** (1, m, n) denote **Cardinality Ratios**.
- **Single lines** denote **Partial Participation** (optional), whereas double lines would denote total participation (mandatory).

Here is the evaluation of each statement:

- (a) **Enroll is a strong entity set.**
False. In the diagram, **enroll** is enclosed in a diamond, which dictates that it is a **relationship set**, not an entity set. Furthermore, it connects the **Course** and **Student** entity sets.
- (b) **All teachers must teach courses.**
False. The connection between the **Teacher** entity and the **teach** relationship is drawn with a single line. A single line indicates **partial (optional) participation**, meaning there can be teachers who do not teach any courses.
- (c) **All students must enroll in courses.**
False. The connection between **Student** and **enroll** is a single line, indicating partial participation.
- (d) **All courses must be enrolled by students.**
False. The connection between **Course** and **enroll** is a single line, indicating partial participation.
- (e) **All courses must be taught by teachers.**
False. The connection between **Course** and **teach** is a single line, indicating partial participation. (A course may exist in the catalog without an assigned teacher).
- (f) **Each course is taught by only one teacher.**
True. The cardinality ratio for the **teach** relationship is **1:m** (1 on the Teacher side, *m* on the Course side). This means a single teacher can teach many courses, but any given course maps back to at most **one** teacher.
- (g) **Each course is enrolled by only one student.**
False. The cardinality ratio for the **enroll** relationship is **n:m** (many-to-many). Therefore, a single course can have multiple students enrolled in it.
- (h) **Each teacher may teach many courses.**
True. The **m** on the **Course** side of the **1:m teach** relationship indicates that a single teacher entity can be associated with multiple course entities.
- (i) **Each student can enroll in many courses.**
True. The **n:m** cardinality of the **enroll** relationship dictates that a student can be associated with multiple courses, just as a course can have multiple students.
- (j) **Teacher is a relationship set.**
False. **Teacher** is enclosed in a rectangle, which defines it as an **entity set**, not a relationship set.

Summary of Valid Statements

Only the following statements hold true for the given E-R Diagram:

- **Statement 6:** Each course is taught by only one teacher.
- **Statement 8:** Each teacher may teach many courses.
- **Statement 9:** Each student can enroll in many courses.

- Q12.** You are running a startup company allowing users to post blogs and comment on other users’ blogs.
- (a) Design the E/R diagram for your company’s data. Store information about users, bloggers, and blogs. Every user has a user ID and a name. Every blog has a blog ID, text content, and one author (a blogger). Every blogger is a user with a rating attribute. Users may comment on blogs. Every comment has optional text content and/or an optional rating.
 - (b) Write a sequence of SQL queries that create the tables for this E/R diagram; **uid**, **bid**, and **rating** are integers, all other attributes are text. Include key, foreign key, and not null declarations.

(20 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Part (a): Enhanced Entity-Relationship (EER) Diagram Design

1. Analysis of Entities, Attributes, and Structural Constraints

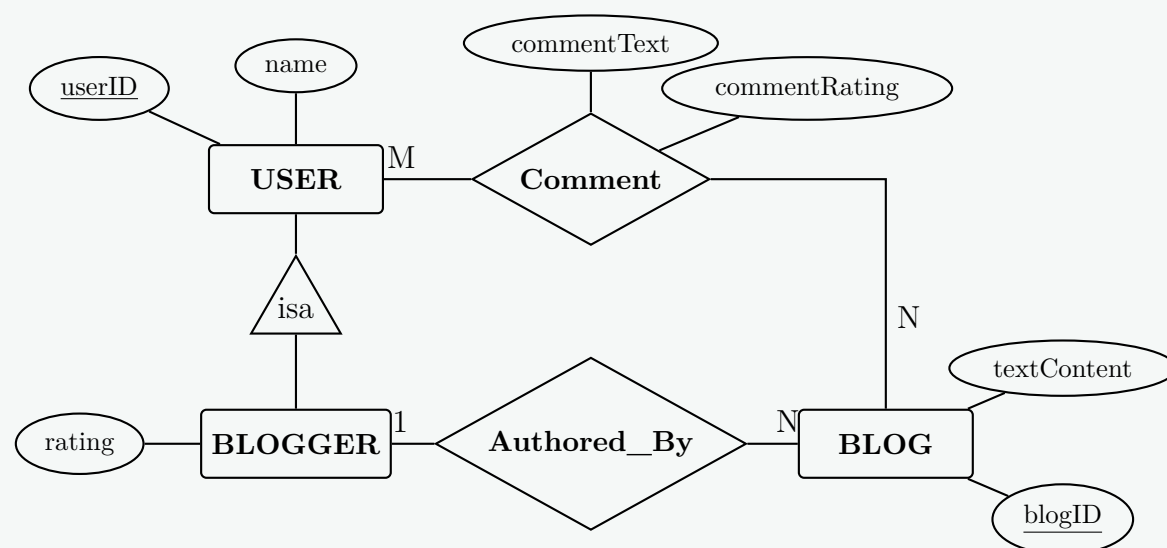
- **USER (Superclass Entity):** Contains primary key attribute **userID** and descriptive attribute **name**.
- **BLOGGER (Subclass Entity via ISA Hierarchy):** Represents a specialized role of a **USER**. It inherits all attributes of **USER** and contains a unique specific attribute **rating**. The specialization is **total** (every blogger must be a user) and **disjoint/single-subclass** in this context.

- **BLOG (Strong Entity):** Contains primary key attribute `blogID` and descriptive attribute `textContent`.
- **Authored_By (Relationship):** A many-to-one ($N : 1$) relationship connecting **BLOG** to **BLOGGER**. Each blog must have exactly one author (**total participation** on the **BLOG** side).
- **Comment (Relationship Set with Attributes):** A many-to-many ($M : N$) relationship linking **USER** and **BLOG**. It contains descriptive attributes: `commentText` (optional) and `commentRating` (optional).

2. Diagram Layout Planning

To maximize symmetry and minimize line crossings, a hierarchical vertical layout is used:

- The **USER** entity is placed at the top center.
- An **ISA** triangle points from the subclass **BLOGGER** upward to its superclass **USER**.
- The **BLOG** entity is positioned on the right side.
- The $M : N$ relationship **Comment** sits symmetrically between **USER** and **BLOG**.
- The $N : 1$ relationship **Authored_By** is positioned clearly between **BLOG** and **BLOGGER**.



Part (b): Relational Schema & Data Definition Language (DDL)

1. Mapping Strategy

- **Avoidance of Nulls via Subclass Table Mapping:** To implement the **USER-BLOGGER** hierarchy cleanly without causing structural NULL entries for regular users who are not bloggers, a separate **Blogger** table is defined. The primary key `uid` references `Users(uid)`.
- **Many-to-One Linkage:** The `Blogs` table integrates its structural author dependency directly via a NOT NULL foreign key reference `author_id` mapping to `Blogger(uid)`.
- **Many-to-Many Linkage with Attributes:** The **Comment** relationship translates into a dedicated intersection table `Comments` utilizing a composite key setup (`uid`, `bid`, `comment_id` to support multiple comments by a single user on a single blog).

2. SQL Execution Script

```

-- Create table for Superclass USER
CREATE TABLE Users (
    uid INTEGER,
    name TEXT NOT NULL,
    PRIMARY KEY (uid)
);

-- Create table for Subclass BLOGGER
CREATE TABLE Blogger (
    uid INTEGER,
    rating INTEGER NOT NULL,
    PRIMARY KEY (uid),
    FOREIGN KEY (uid) REFERENCES Users(uid) ON DELETE CASCADE
);

-- Create table for Entity BLOG
CREATE TABLE Blogs (

```

```

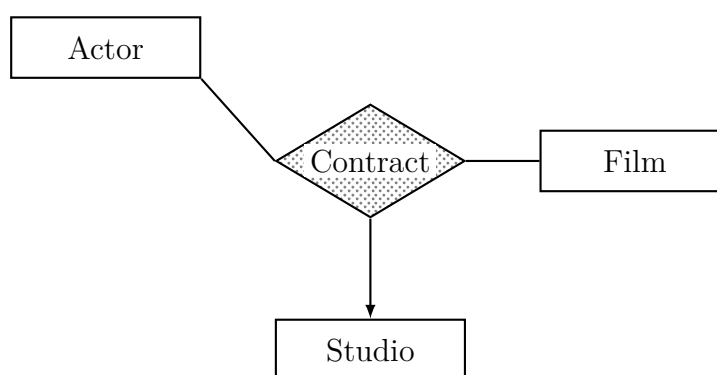
    bid INTEGER,
    textContent TEXT NOT NULL,
    author_id INTEGER NOT NULL,
    PRIMARY KEY (bid),
    FOREIGN KEY (author_id) REFERENCES Blogger(uid)
);

-- Create table for Relationship Set Comment (M:N Linkage)
CREATE TABLE Comments (
    comment_id INTEGER,
    uid INTEGER NOT NULL,
    bid INTEGER NOT NULL,
    commentText TEXT,
    commentRating INTEGER,
    PRIMARY KEY (comment_id),
    FOREIGN KEY (uid) REFERENCES Users(uid) ON DELETE CASCADE,
    FOREIGN KEY (bid) REFERENCES Blogs(bid) ON DELETE CASCADE
);

```

Q13. Consider a multi-way relationship between **Actor**, **Film** and **Studio**.

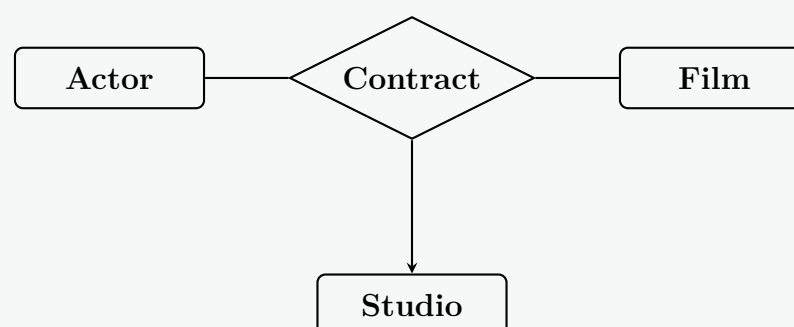
- There is an arrow pointing to entity set Studios. What is its interpretation?
- There are no arrows pointing to entity sets Actor or Film. What are their interpretations?



(5 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer



Interpretation of the Multi-way E/R Diagram

In Entity-Relationship (E/R) modeling, a ternary (3-way) relationship connects three entity sets. The presence or absence of arrows on the edges dictates the **cardinality constraints** (or key constraints) of the relationship.

(a) Interpretation of the arrow pointing to Studios

- Definition:** An arrow pointing from a multi-way relationship to an entity set indicates a **functional dependency** (a “many-to-one” constraint from the perspective of the other combined entities).
- Interpretation:** For any given **Actor** and **Film** combination, there is **at most one** associated **Studio**.
- Explanation:** If we know the actor and the film they are starring in, that specific contract uniquely identifies exactly one studio. An actor cannot be contracted by two different studios for the exact same film.

(b) Interpretation of no arrows pointing to Actor or Film

- Definition:** A standard undirected line (no arrow) in a multi-way relationship indicates a “many” cardinality constraint for that entity set relative to the others.
- Interpretation for Actor:** For a specific **Film** and **Studio** combination, there can be **many Actors**.
 - Explanation:* A studio producing a specific film will logically sign contracts with multiple actors for that single movie.

- **Interpretation for Film:** For a specific **Actor** and **Studio** combination, there can be **many Films**.
 - *Explanation:* A single actor can sign contracts with the same studio to appear in multiple different films over time.

Note on Relational Translation: If we were to convert this ternary relationship into a relational database schema, the primary key for the **Contract** table would be the composite of the primary keys of the entities without arrows: (**Actor_ID**, **Film_ID**). The **Studio_ID** would simply be a non-key foreign key attribute in this table, perfectly reflecting the rule that (Actor, Film) → Studio.

Q14. Consider the following information about a university database:

- Professors have an SSN, a name, an age, a rank, and a research specialty.
 - Projects have a project number, a sponsor name, a starting date, an ending date, and a budget.
 - Graduate students have an SSN, a name, an age, and a degree program (e.g., M.S. or Ph.D.).
 - Each project is managed by one professor (principal investigator) and worked on by one or more professors (co-investigators).
 - Each project is worked on by one or more graduate students (research assistants); a professor must supervise their work. Graduate students can work on multiple projects with a (potentially different) supervisor for each.
 - Departments have a department number, name, and main office; a professor (chairman) runs the department.
 - Professors work in one or more departments with an associated time percentage.
 - Graduate students have one major department and a more senior graduate student advisor.
 - Both professors and graduate students have multiple phone numbers and email addresses.
- (a) Draw an ER diagram that captures the above information. Indicate all key and cardinality constraints and any assumptions you make.
- (b) Convert the above ER diagram into relational schema.

(35 marks)

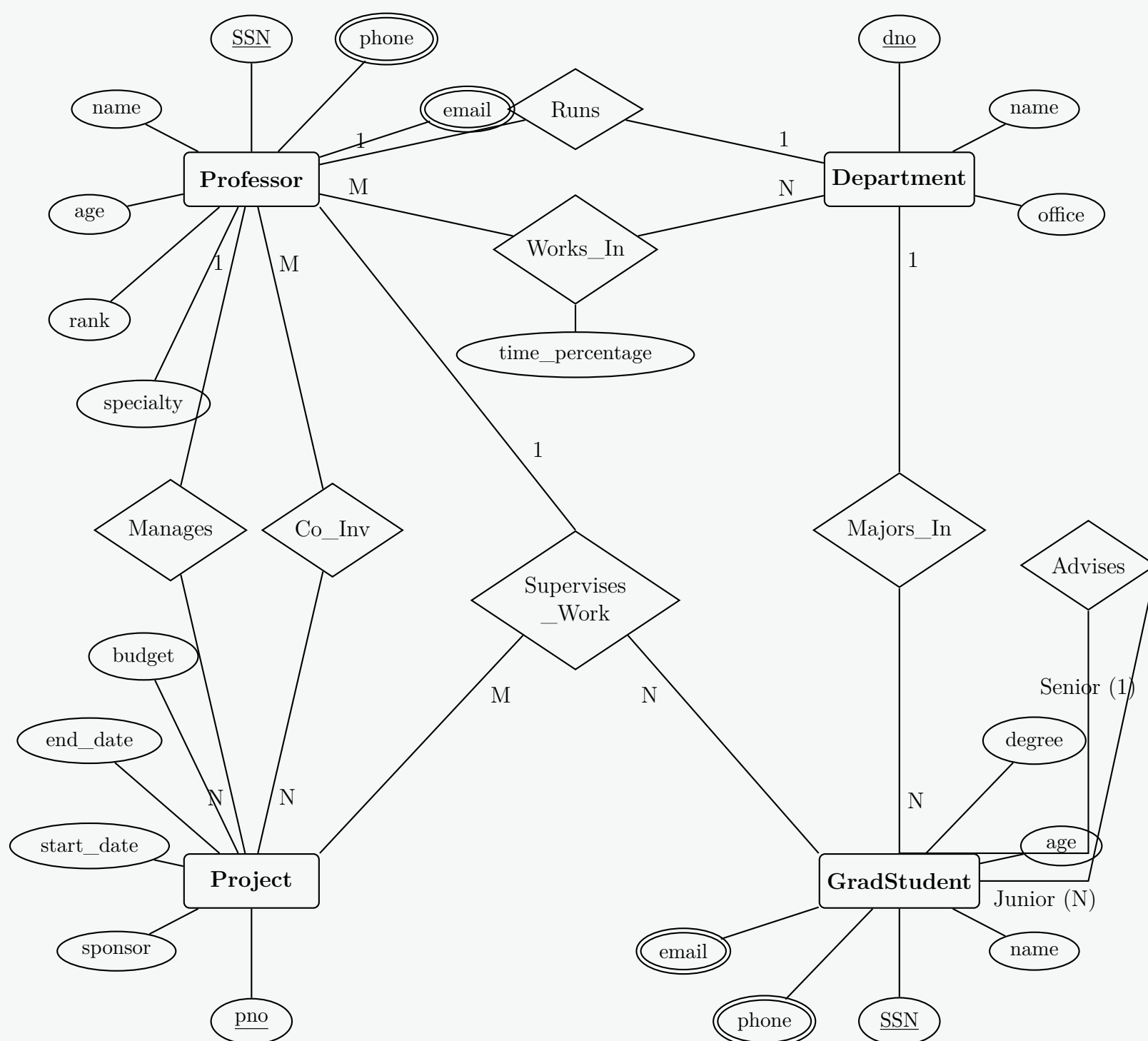
[CSE 215 | L-2/T-2 | 2019–20]

Answer

Part 1: Entity-Relationship (ER) Diagram

Design Assumptions & Modeling Choices:

- **Ternary Relationship:** The requirement stating "Each project is worked on by one or more graduate students; a professor must supervise their work... with a (potentially different) supervisor for each" is modeled as a ternary relationship **Supervises_Work** between **Professor**, **Project**, and **GradStudent**.
- **Multivalued Attributes:** Both **Professor** and **GradStudent** have multiple phone numbers and email addresses. These are represented as double ellipses.
- **Recursive Relationship:** **GradStudent** has a recursive relationship **Advises** to represent the senior advisor structure.
- **1:1 and M:N Roles:** **Runs** is assumed to be 1:1 (one chairman per department). **Works_In** is an M:N relationship with a descriptive attribute **time_percentage**.



Part 2: Relational Database Schema

The ER diagram is mapped to the following normalized relational schemas. Primary keys are underlined, and Foreign Keys are explicitly noted.

- **Department**(dno, name, office, chair_ssn)
 - *FK: chair_ssn references Professor(ssn)*
- **Professor**(ssn, name, age, rank, specialty)
- **Prof_Phone**(ssn, phone)
 - *FK: ssn references Professor(ssn)*
- **Prof_Email**(ssn, email)
 - *FK: ssn references Professor(ssn)*
- **Project**(pno, sponsor, start_date, end_date, budget, pi_ssn)
 - *FK: pi_ssn references Professor(ssn)*
- **GradStudent**(ssn, name, age, degree_program, major_dno, advisor_ssn)
 - *FK: major_dno references Department(dno)*
 - *FK: advisor_ssn references GradStudent(ssn)*
- **Grad_Phone**(ssn, phone)
 - *FK: ssn references GradStudent(ssn)*
- **Grad_Email**(ssn, email)
 - *FK: ssn references GradStudent(ssn)*

- **Works_In**(prof_ssn, dno, time_percentage)
 - FK: *prof_ssn* references **Professor**(*ssn*)
 - FK: *dno* references **Department**(*dno*)
- **Co_Investigates**(prof_ssn, pno)
 - FK: *prof_ssn* references **Professor**(*ssn*)
 - FK: *pno* references **Project**(*pno*)
- **Supervises_Work**(grad_ssn, pno, prof_ssn)
 - FK: *grad_ssn* references **GradStudent**(*ssn*)
 - FK: *pno* references **Project**(*pno*)
 - FK: *prof_ssn* references **Professor**(*ssn*)
 - **Note:** The primary key is (*grad_ssn*, *pno*) because a given grad student is supervised by exactly one professor for any specific project.

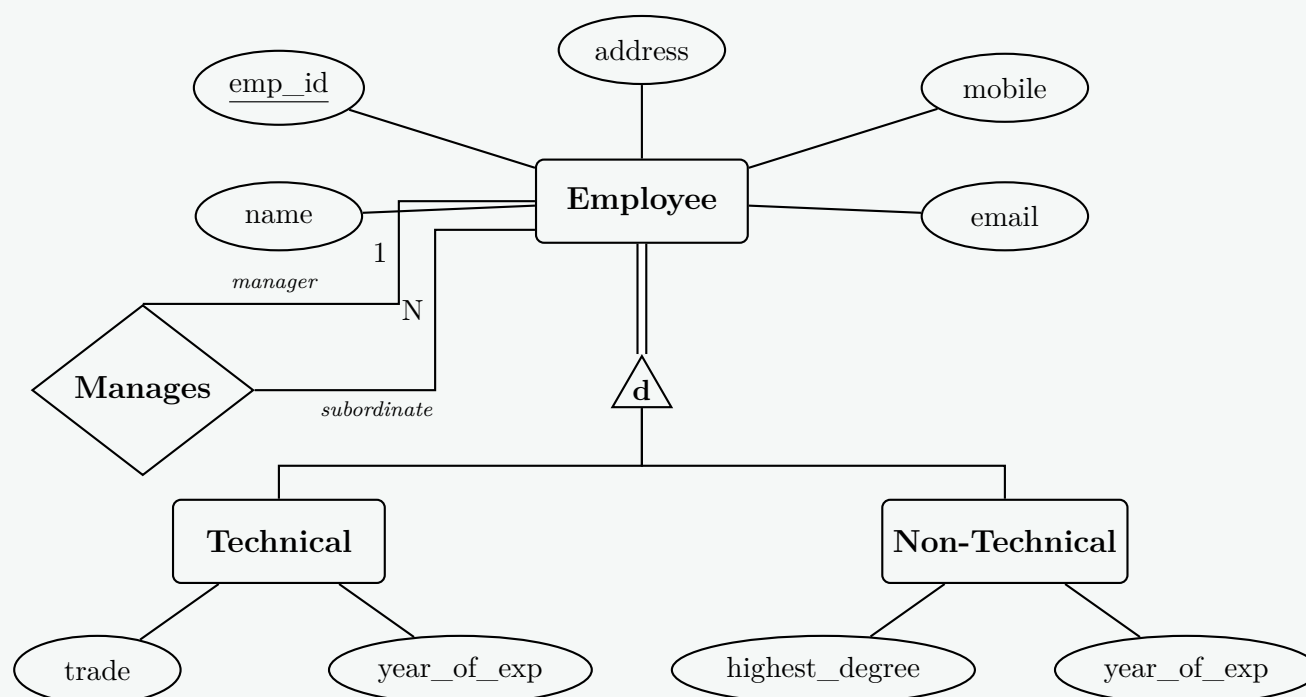
Q15. All employees in an organization are identified by employee id. All employees must have name, address, mobile and email. An employee must be one of the following: technical or non-technical. Technical employees have the trade and year of experience. Non-technical employees have the highest degree and year of experience. Each employee is managed by a manager who is also an employee. Draw the complete ERD for this scenario. Convert the ER diagram into two different possible relational schemas. (15 marks) [CSE 215 | L-2/T-2 | 2019–20]

Answer

Part 1: Entity-Relationship Diagram (ERD)

Design Considerations & Semantic Constraints:

- **Total and Disjoint Specialization:** The prompt explicitly states an employee *"must be one of the following"*. This translates to a **Total Specialization** (denoted by a double line from the superclass to the ISA triangle) and **Disjoint Specialization** (denoted by a 'd' inside the triangle).
- **Recursive Relationship:** The managerial structure is represented by a unary (recursive) relationship **Manages** on the **Employee** entity. An employee can manage many subordinates (N), but each employee is managed by exactly one manager (1).



Part 2: Relational Schema Conversion

An ER Diagram with a Superclass/Subclass hierarchy can be mapped into a relational schema in several standard ways. Because this is a **Total and Disjoint** specialization, two highly effective and structurally distinct schemas can be produced.

Option 1: Superclass/Subclass Multiple Relations (ER-to-Relational Standard)

In this approach, the superclass and all subclasses have their own separate tables. The subclass tables inherit the primary key from the superclass, which serves as both their primary key and a foreign key.

- **Employee** (emp_id, name, address, mobile, email, manager_id)

- *Foreign Key:* `manager_id` references **Employee**(`emp_id`)
- **Technical** (`emp_id`, `trade`, `year_of_exp`)
 - *Foreign Key:* `emp_id` references **Employee**(`emp_id`)
- **NonTechnical** (`emp_id`, `highest_degree`, `year_of_exp`)
 - *Foreign Key:* `emp_id` references **Employee**(`emp_id`)

Advantage: Highly normalized. Eliminates ‘NULL’ values entirely for the subclass-specific attributes.

Option 2: Single Universal Relation (Flattened Structure)

Because the specialization is disjoint and total, and there are relatively few subclass-specific attributes, we can collapse the hierarchy into a single table. A discriminator attribute (`emp_type`) is introduced, and shared attributes (like `year_of_exp`) are merged.

- **Employee_Universal** (`emp_id`, `emp_type`, `name`, `address`, `mobile`, `email`, `manager_id`, `trade`, `highest_degree`, `year_of_exp`)
 - *Foreign Key:* `manager_id` references **Employee_Universal**(`emp_id`)
 - *Note:* `emp_type` will store values like ‘Technical’ or ‘Non-Technical’. If ‘Technical’, `highest_degree` will be NULL. If ‘Non-Technical’, `trade` will be NULL.

Advantage: Improves read performance since no expensive ‘JOIN’ operations are required to assemble an employee’s full profile.

Q16. What do you mean by materialized view? Explain with examples the problems associated with materialized view. What restrictions do most database implementations have to allow updates on materialized views? **(10 marks)** [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Materialized View: Definition and Explanation

Definition: A **materialized view** is a database object that contains the precomputed results of a query. Unlike a standard (virtual) view, which dynamically computes the result set every time it is queried by executing the underlying SQL, a materialized view permanently stores (materializes) the data on physical storage (disk).

Working Principle: When a materialized view is created, the database evaluates the defining query and saves the output into a physical table structure. Subsequent queries against the materialized view read directly from this physical structure, drastically reducing execution time for complex joins, aggregations, and large data scans. They are widely used in **Data Warehousing** and **Business Intelligence (BI)** systems.

Feature	Standard (Virtual) View	Materialized View
Storage	Stores only the SQL query definition.	Stores both the query definition and the actual data.
Performance	Slower for complex queries (re-calculated every time).	Extremely fast (data is pre-computed and fetched from disk).
Data Freshness	Always up-to-date with base tables.	Can be stale; requires periodic or triggered refreshing.

2. Problems Associated with Materialized Views

While materialized views provide significant performance benefits for read-heavy workloads, they introduce several major challenges, primarily concerning **View Maintenance** (keeping the materialized data synchronized with the base tables).

A. The View Maintenance Problem (Data Staleness)

When the base tables are modified (INSERT, UPDATE, DELETE), the materialized view becomes out of sync (stale). The database must update the materialized view, which can be computationally expensive.

Example: Consider an e-commerce database with a materialized view that calculates total daily sales.

```
CREATE MATERIALIZED VIEW Daily_Sales_MV AS
SELECT store_id, transaction_date, SUM(amount) as total_sales
FROM Sales
GROUP BY store_id, transaction_date;
```

- **Problem Scenario:** If a customer makes a new purchase, a new row is inserted into the `Sales` table. The `Daily_Sales_MV` is now outdated.

- If the system is configured for **Immediate Refresh** (synchronous update), the insertion into the **Sales** table will be delayed because the database must simultaneously recalculate and update the materialized view. This degrades Online Transaction Processing (OLTP) performance.
- If configured for **Deferred Refresh** (asynchronous update, e.g., nightly), business analysts querying the view during the day will see inaccurate, stale data.

B. High Storage Overhead

Because materialized views store the actual result sets, they consume significant physical disk space. A poorly designed materialized view that joins multiple massive tables without strong filtering can duplicate terabytes of data.

C. Write Amplification

A single update to a base table might trigger cascading updates across multiple materialized views that depend on it, consuming excessive CPU, memory, and I/O bandwidth.

3. Restrictions on Updating Materialized Views

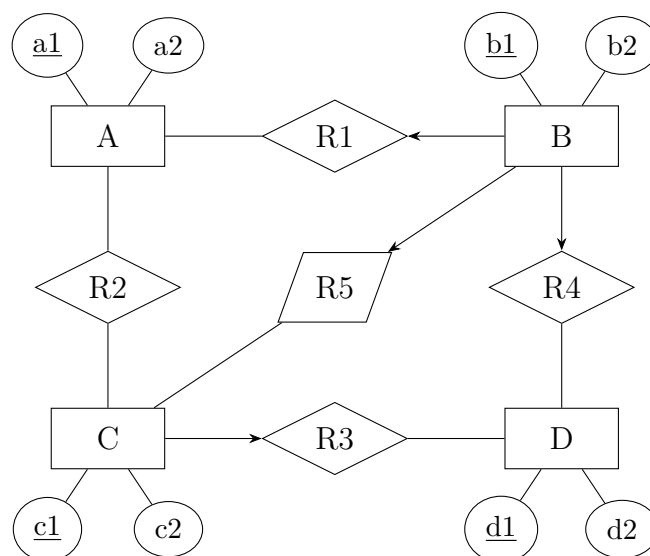
To mitigate the massive cost of completely recomputing a materialized view every time a base table changes, modern DBMS implementations (like Oracle, PostgreSQL, and SQL Server) use **Incremental Maintenance** (often called **Fast Refresh**). Instead of recalculating the whole query, the DBMS only calculates the delta (the changes).

However, to allow Fast Refresh (or to allow users to issue direct **UPDATE/INSERT** statements on the materialized view in updatable MV scenarios), the database enforces strict structural restrictions on the defining query:

- Primary Key / ROWID Requirement:** The defining query must include the primary keys or internal row identifiers (ROWID) of all underlying base tables. This allows the database to map a changed row in the base table directly to the corresponding row in the materialized view.
- Aggregate Function Limitations:**
 - If the view uses **GROUP BY**, it must usually include the **COUNT(*)** or **COUNT(column)** aggregate function. This is necessary so the database knows when to delete a row from the materialized view (i.e., when the count drops to zero after a base table deletion).
 - Complex aggregates like **MIN**, **MAX**, or statistical functions often prevent incremental updates because removing the current "maximum" value requires scanning the entire base table to find the new maximum.
- No Non-Deterministic Functions:** The query cannot contain volatile or time-dependent functions such as **SYSDATE**, **RAND()**, or **CURRENT_TIMESTAMP**.
- Join Restrictions:** While simple equi-joins are typically supported, complex outer joins, self-joins, or joins combined heavily with aggregations often disqualify a materialized view from fast refresh.
- Set Operator Restrictions:** Queries utilizing **UNION**, **INTERSECT**, or **EXCEPT (MINUS)** usually cannot be incrementally maintained.
- Subquery Limitations:** Correlated subqueries or subqueries in the **SELECT** clause generally prevent fast updates.

Note on Direct Updates: If referring to *Updatable Materialized Views* (where a user explicitly executes **UPDATE MV_Name SET ...**), most DBMSs forbid this entirely. Those that do support it (e.g., Oracle Advanced Replication) require strict primary key linkages and bidirectional replication logs to push the changes back to the base tables without conflict.

Q17. Reduce the given ERD to the minimum relational schemas (mention schema names and their attributes).



(10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Cardinality Analysis

To reduce the given Entity-Relationship Diagram (ERD) to its minimum relational schemas, we first establish the relationship cardinalities based on standard ER conventions. In standard notations (like Korth), a directed edge (arrow) pointing to/from an entity signifies the “1” (One) side of a relationship (often representing a key constraint), whereas an undirected edge (line) signifies the “N” (Many) side.

Based on the structure provided in the diagram:

- **R1 (between A and B):** The directed arrow on B’s side indicates a **N:1** relationship from A to B (A is Many, B is 1).
- **R2 (between A and C):** Undirected on both sides, indicating a **M:N** (Many-to-Many) relationship.
- **R3 (between C and D):** The directed arrow on C’s side indicates a **N:1** relationship from D to C (D is Many, C is 1).
- **R4 (between B and D):** The directed arrow on B’s side indicates a **N:1** relationship from D to B (D is Many, B is 1).
- **R5 (between B and C):** The directed arrow on B’s side indicates a **N:1** relationship from C to B (C is Many, B is 1). *Note: Though R5 is drawn as a parallelogram in the visual, it functions as a standard mapping in this reduction.*

2. Relational Schema Reduction

We apply the standard algorithm for ERD-to-Relational mapping:

- (a) **Strong Entities:** Each entity receives its own table.
- (b) **1:N Relationships:** The primary key of the “1” side is embedded as a **Foreign Key (FK)** in the table of the “N” side.
- (c) **M:N Relationships:** A separate table is created containing the primary keys of both participating entities.

This yields a minimum of **5 relational schemas**. Primary keys are underlined, and foreign keys are *italicized*.

- **B** (b1, b2)
- **A** (a1, a2, *b1*) – b1 references B(b1)
- **C** (c1, c2, *b1*) – b1 references B(b1)
- **D** (d1, d2, *c1*, *b1*) – c1 references C(c1), b1 references B(b1)
- **R2** (a1, c1) – a1 references A(a1), c1 references C(c1)

3. Relational Schema Diagram

```
graph TD; A["A ( a1, a2, b1 )"] --> B["B ( b1, b2 )"]; C["C ( c1, c2, b1 )"] --> B; D["D ( d1, d2, c1, b1 )"] --> C; D --> B; R2["R2 ( a1, c1 )"] --> A; R2 --> C;
```

Q18. How can redundancy and incompleteness lead to a bad database design? Explain with appropriate examples. **(10 marks)** [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Introduction to Bad Database Design

A poor database design typically suffers from two major structural deficiencies: **redundancy** (storing the same data multiple times unnecessarily) and **incompleteness** (failing to capture all required data or data relationships). In a relational database management system (RDBMS), these design flaws compromise data integrity, waste storage resources, and lead to programmatic anomalies during data manipulation.

2. Redundancy in Database Design

A. Definition and Explanation

Data redundancy occurs when the same piece of data is duplicated across multiple rows or tables due to an unnormalized schema. This usually happens when an entity’s operational details are blended with another independent entity’s attributes in a single relation.

B. Problems Caused by Redundancy: Data Anomalies

Redundancy directly causes three critical data anomalies during routine database updates:

- (a) **Insertion Anomaly:** The inability to insert critical facts about one entity without possessing completely unrelated data belonging to another entity.
- (b) **Deletion Anomaly:** The unintended loss of critical independent data because a related record containing that data is deleted.
- (c) **Update (Modification) Anomaly:** The requirement to modify multiple duplicate data rows when a single fact changes. Failure to update every row results in data inconsistency.

C. Concrete Example of Redundancy

Consider an unnormalized relation `Emp_Dept` that stores both employee and department information in a single table:

Emp_ID (PK)	Emp_Name	Role	Dept_ID	Dept_Name
E101	Alice	Developer	D01	Engineering
E102	Bob	Analyst	D02	Marketing
E103	Charlie	Developer	D01	Engineering

Analysis of Anomalies:

- **Update Anomaly:** If the `Dept_Name` for D01 changes from “Engineering” to “Research & Development”, the system must find and update both records for Alice (E101) and Charlie (E103). If one row fails to update, the database falls into an inconsistent state.
- **Insertion Anomaly:** Suppose the organization establishes a new department named “Finance” (D03), but has not hired any employees for it yet. Because `Emp_ID` is the primary key and cannot accept a NULL value, the data for the new department cannot be recorded.
- **Deletion Anomaly:** If Bob (E102) resigns from the company and his record is deleted from the database, all knowledge that department D02 is called “Marketing” is irreversibly wiped out.

D. Resolution via Normalization

To correct this, the database designer decomposes the table into two separate, normalized relations connected via a foreign key relationship:

Department Table	
Dept_ID	Dept_Name
D01	Engineering
D02	Marketing

Employee Table			
Emp_ID	Emp_Name	Role	Dept_ID
E101	Alice	Developer	D01
E102	Bob	Analyst	D02
E103	Charlie	Developer	D01

3. Incompleteness in Database Design

A. Definition and Explanation

Incompleteness refers to a database design that fails to fully capture all business rules, requirements, historical constraints, or entity attributes. It manifests as a lack of required fields, absence of necessary relationships, or structural inability to hold multivalued or temporal states.

B. Problems Caused by Incompleteness

- **Loss of Information:** Inability to capture vital business transaction metrics.
- **Misuse of NULL Values:** Forcing attributes to hold NULL values when a separate structure is actually needed, degrading query optimization and clarity.
- **Inability to Enforce Integrity Constraints:** Critical validation checks cannot be established if key supporting attributes are absent.

C. Concrete Example of Incompleteness

Consider a hospital database that tracks patients and their prescribed medications. An incomplete design places the prescribed drug directly in the `Patient` table:

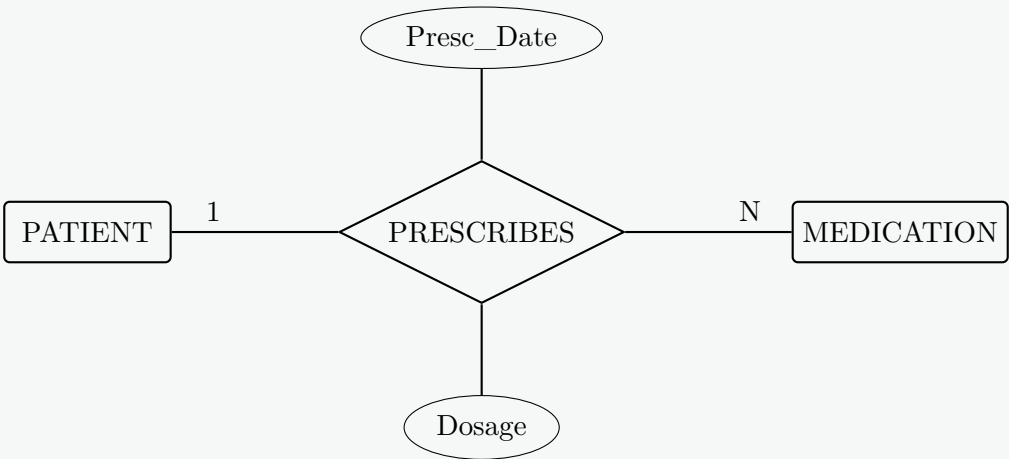
Patient (Incomplete Schema)			
Patient_ID	Patient_Name	Diagnosis	Prescribed_Medication
P501	John Doe	Hypertension	Lisinopril
P502	Jane Smith	Diabetes	Metformin

Analysis of Flaws:

- **Multivalued Attribute Issue:** If John Doe is diagnosed with multiple conditions and requires three different medications simultaneously, this single atomic field design breaks. Attempting to store comma-separated strings (e.g., “Lisinopril, Amlodipine”) violates First Normal Form (1NF), complicating string parsing and index usage.
- **Loss of History:** This design lacks data fields for prescription date, structural dosage instructions (e.g., “twice a day”), and prescribing doctor. Overwriting `Prescribed_Medication` when a drug is changed obliterates vital historical patient medical records.

D. Resolution via Proper Structural Modeling

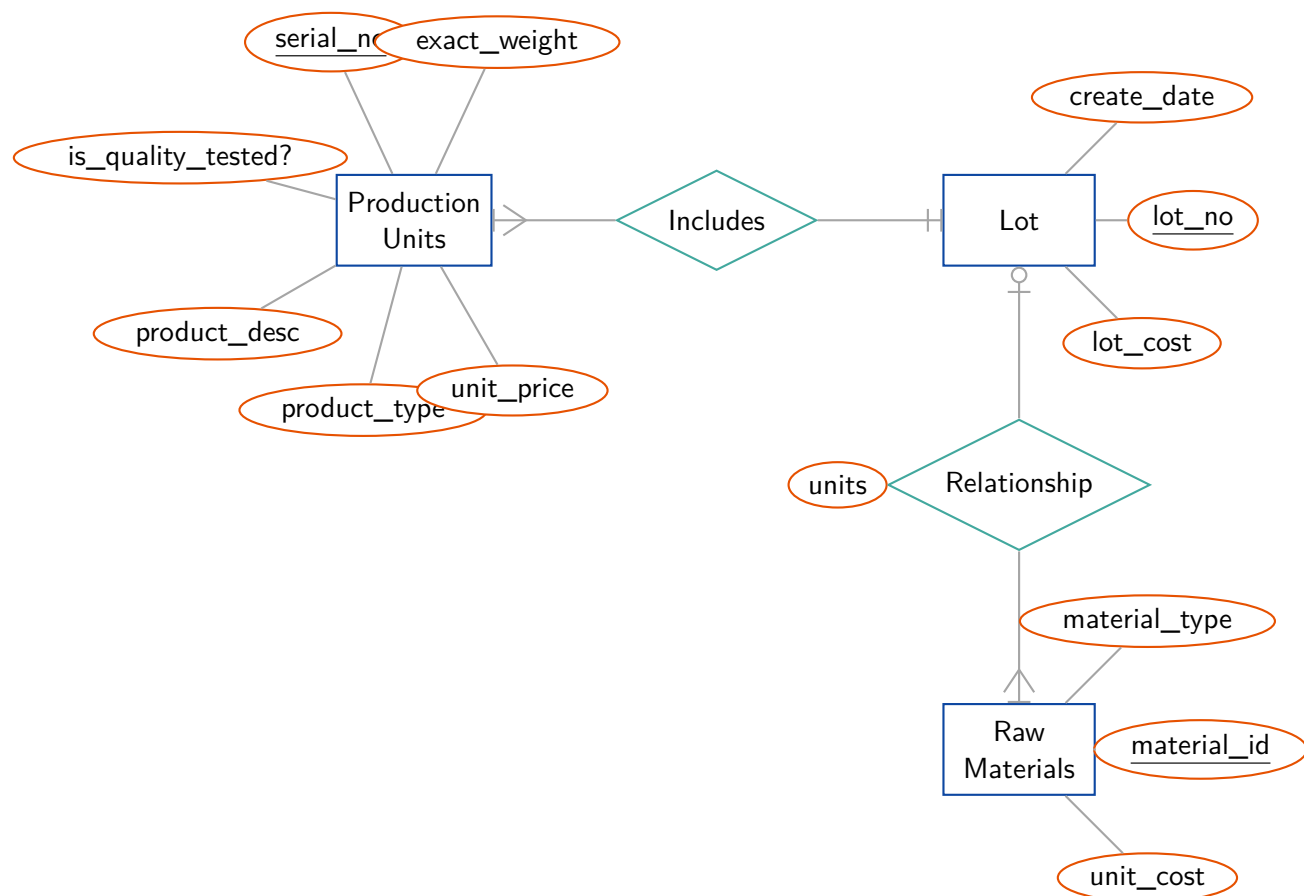
A robust design isolates multi-valued interactions into a distinct transaction table:



4. Summary Comparison

Design Flaw	Primary Root Cause	Typical Engineering Solution
Redundancy	Bad decomposition / Combining independent facts	Normalization (2NF, 3NF, BCNF)
Incompleteness	Poor requirement gathering / Violating cardinality	Comprehensive ER modeling & schema review

Q19. Production tracking is important in many manufacturing environments (e.g., the pharmaceuticals industry, children’s toys, etc.). The ER diagram of Figure captures important information of production tracking. Specifically, the ER diagram captures the relationships among production lots (or batches), individual production units, and raw materials.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

Based on the provided Entity-Relationship (ER) diagram, the standard database design task is to analyze the conceptual model and map it to a logical relational schema, followed by its physical implementation in SQL.

1. Conceptual Analysis (ER Model)

- **Entities and Primary Keys:**

- **Production Units:** Identified by the primary key serial_no. Additional attributes include exact_weight, is_quality_tested?, product_desc, product_type, and unit_price.
- **Lot:** Identified by the primary key lot_no. Additional attributes include create_date and lot_cost.
- **Raw Materials:** Identified by the primary key material_id. Additional attributes include material_type and unit_cost.

- **Relationships and Cardinalities:**

- **Includes (1:N):** A single **Lot** can include many **Production Units**, but a specific **Production Unit** is associated with exactly one **Lot**. This is represented by the mandatory one (||) at the Lot side and the mandatory many (crow's foot) at the Production Units side.
- **Relationship (M:N):** The diamond labeled “Relationship” connects **Lot** and **Raw Materials**. It contains its own descriptive attribute, units. This typically represents a Many-to-Many association (e.g., a Bill of Materials usage), indicating how many units of a specific raw material are consumed by a specific lot.

2. Relational Schema Mapping

To convert the conceptual ER model into a relational database schema, we apply standard mapping rules. Primary keys are underlined, and Foreign keys are *italicized*.

(a) **Lot** (lot_no, create_date, lot_cost)

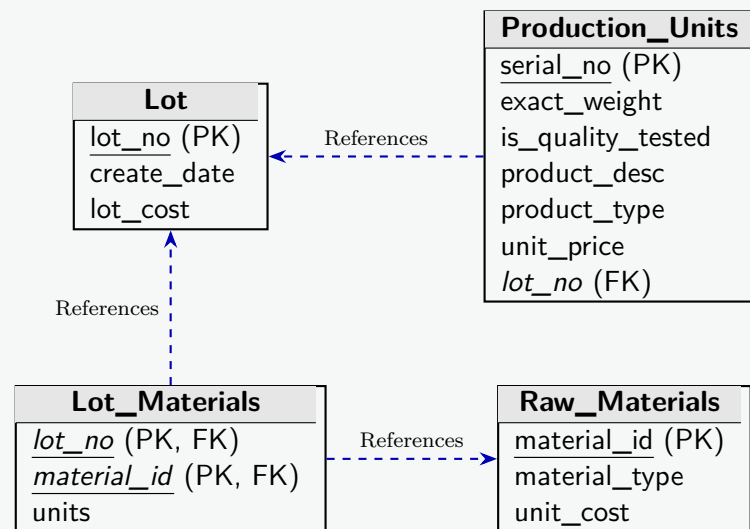
(b) **Production_Units** (serial_no, exact_weight, is_quality_tested, product_desc, product_type, unit_price, *lot_no*)
Note: lot_no is added as a foreign key to represent the 1:N “Includes” relationship.

(c) **Raw_Materials** (material_id, material_type, unit_cost)

(d) **Lot_Materials** (*lot_no*, *material_id*, units)

Note: A new associative table is created for the M:N “Relationship”. The primary key is composite, and both attributes also serve as foreign keys.

3. Schema Diagram



4. SQL Data Definition Language (DDL)

The logical schema above translates into the following formal SQL constraints:

```

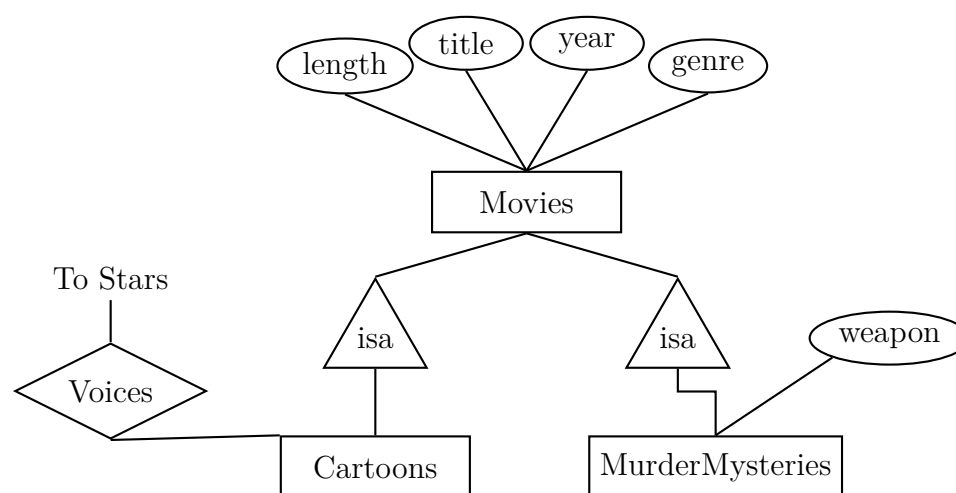
-- 1. Create independent entities first
CREATE TABLE Lot (
    lot_no VARCHAR(50) PRIMARY KEY,
    create_date DATE,
    lot_cost DECIMAL(10, 2)
);

CREATE TABLE Raw_Materials (
    material_id VARCHAR(50) PRIMARY KEY,
    material_type VARCHAR(100),
    unit_cost DECIMAL(10, 2)
);

-- 2. Create tables with foreign key dependencies
CREATE TABLE Production_Units (
    serial_no VARCHAR(50) PRIMARY KEY,
    exact_weight DECIMAL(10, 2),
    is_quality_tested BOOLEAN,
    product_desc VARCHAR(255),
    product_type VARCHAR(100),
    unit_price DECIMAL(10, 2),
    lot_no VARCHAR(50),
    CONSTRAINT fk_pu_lot FOREIGN KEY (lot_no)
        REFERENCES Lot(lot_no) ON DELETE CASCADE
);

CREATE TABLE Lot_Materials (
    lot_no VARCHAR(50),
    material_id VARCHAR(50),
    units INT,
    PRIMARY KEY (lot_no, material_id),
    CONSTRAINT fk_lm_lot FOREIGN KEY (lot_no)
        REFERENCES Lot(lot_no) ON DELETE CASCADE,
    CONSTRAINT fk_lm_mat FOREIGN KEY (material_id)
        REFERENCES Raw_Materials(material_id) ON DELETE CASCADE
);
  
```

Q20. Convert the ISA-hierarchies of the following ER diagram into schemas using three principal conversion strategies.



(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer**Overview of the ER Diagram**

- **Superclass: Movies** with attributes *title*, *year*, *length*, and *genre*. We assume the composite attribute (**title, year**) forms the primary key.
- **Subclasses: Cartoons and MurderMysteries.**
- **Subclass-specific attributes/relationships:**
 - **Cartoons** participates in a relationship **Voices** with an external entity **Stars** (assumed primary key: *star_id*).
 - **MurderMysteries** has a specific attribute *weapon*.

Below are the three principal strategies for converting this ISA (Inheritance) hierarchy into relational schemas. Primary keys are underlined, and Foreign keys are denoted with *italics* or explicitly stated.

Strategy 1: ER Approach (Superclass and Subclass Relations)

Working Principle: Create one relation for the superclass to hold all common attributes, and create separate relations for each subclass. The subclass relations contain the primary key of the superclass (which acts as both a primary key and a foreign key) and any subclass-specific attributes.

Relational Schemas:

- **Movies** (title, year, length, genre)
- **Cartoons** (title, year)
Foreign Key: (title, year) references Movies(title, year)
- **MurderMysteries** (title, year, weapon)
Foreign Key: (title, year) references Movies(title, year)
- **Voices** (title, year, star_id)
Foreign Key: (title, year) references Cartoons(title, year)

Advantages:

- Avoids NULL values for subclass-specific attributes.
- Works perfectly for both overlapping and disjoint subclasses.

Disadvantages:

- Querying a complete subclass (e.g., getting the length of a Cartoon) requires a JOIN operation between the superclass and subclass relations.

Strategy 2: Object-Oriented Approach (Multiple Concrete Relations)

Working Principle: Create a distinct relation for each concrete subclass. Each relation includes all the attributes of the superclass (inherited attributes) plus the specific attributes of that subclass. If a movie can exist without being a cartoon or murder mystery, a base **Movies** relation is also required.

Relational Schemas:

- **Movies** (title, year, length, genre) – *For general movies only*
- **Cartoons** (title, year, length, genre)

- **MurderMysteries** (title, year, length, genre, weapon)
- **Voices** (title, year, star_id)
Foreign Key: (title, year) references *Cartoons*(title, year)

Advantages:

- No JOIN operations are needed to retrieve all information about a specific subclass entity.

Disadvantages:

- If subclasses are overlapping (e.g., a movie is both a Cartoon and a Murder Mystery), its inherited attributes (length, genre) are redundantly stored in multiple tables.
- Global queries (e.g., finding the titles of all movies regardless of type) require UNION operations.

Strategy 3: Single Relation Approach (Null-Value Method)

Working Principle: Create one universal relation that collapses the entire hierarchy into a single table. The relation contains all attributes from the superclass and all subclasses. A discriminator (type) attribute is typically added to identify the specific subclass.

Relational Schemas:

- **Movies** (title, year, length, genre, movie_type, weapon)
- **Voices** (title, year, star_id)
Foreign Key: (title, year) references *Movies*(title, year)

Working Note for Strategy 3:

- If a tuple represents a **Cartoon**, *movie_type* = 'Cartoon', and *weapon* is set to NULL.
- If a tuple represents a **MurderMystery**, *movie_type* = 'MurderMystery', and *weapon* contains a value.
- If subclasses overlap, *movie_type* can be replaced by boolean flags (e.g., *is_cartoon*, *is_murder_mystery*).

Advantages:

- Extremely fast for global queries. No JOIN or UNION operations are required.

Disadvantages:

- Introduces many NULL values if the subclasses have many specific attributes, wasting storage space and potentially complicating application logic.
- Subclass-specific constraints (e.g., "weapon cannot be NULL for Murder Mysteries") must be enforced via complex CHECK constraints or application logic.

Q21. A car rental company maintains a database for all vehicles. For all vehicles: vehicle identification number, license number, manufacturer, size (height, width, length) and color. Special data for certain types:

- **Trucks:** load capacity, type
- **Sports car:** power, renter age
- **Bus:** number of passengers, seat-type (array of 40 seats of different types)

- Construct an SQL:99 schema definition for this database using appropriate inheritance.
- Write constructor functions for the above schema and an insert statement to insert data.

(20 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer**1. SQL:99 Schema Definition with Inheritance**

The SQL:99 standard introduced Object-Relational Database (ORDBMS) features, including User-Defined Types (UDTs), type inheritance, and collection types (like arrays).

To model the car rental database, we define a composite type for **Size**, a base type for **Vehicle**, and specific subtypes (**Truck**, **SportsCar**, and **Bus**) that inherit from the base vehicle type using the **UNDER** keyword.

```
-- 1. Create a composite type for the vehicle's size
CREATE TYPE Size_Type AS (
    height DECIMAL(5,2),
    width  DECIMAL(5,2),
    length DECIMAL(5,2)
) INSTANTIABLE;
```

```

-- 2. Create the base supertype for all vehicles
-- Marked as NOT FINAL to allow inheritance
CREATE TYPE Vehicle_Type AS (
    vin VARCHAR(20),
    license_no VARCHAR(15),
    manufacturer VARCHAR(50),
    v_size Size_Type,
    color VARCHAR(20)
) NOT FINAL;

-- 3. Create the Truck subtype
CREATE TYPE Truck_Type UNDER Vehicle_Type AS (
    load_capacity DECIMAL(8,2),
    truck_type VARCHAR(30)
) INSTANTIABLE;

-- 4. Create the Sports Car subtype
CREATE TYPE SportsCar_Type UNDER Vehicle_Type AS (
    power DECIMAL(6,2),
    renter_age INTEGER
) INSTANTIABLE;

-- 5. Create a collection (Array) type for bus seats (40 seats max)
CREATE TYPE Seat_Array_Type AS VARCHAR(20) ARRAY[40];

-- 6. Create the Bus subtype
CREATE TYPE Bus_Type UNDER Vehicle_Type AS (
    num_passengers INTEGER,
    seat_type Seat_Array_Type
) INSTANTIABLE;

-- 7. Create Typed Tables to store the actual data
CREATE TABLE Vehicles OF Vehicle_Type (
    PRIMARY KEY (vin)
);

CREATE TABLE Trucks OF Truck_Type UNDER Vehicles;
CREATE TABLE SportsCars OF SportsCar_Type UNDER Vehicles;
CREATE TABLE Buses OF Bus_Type UNDER Vehicles;

```

2. Constructor Functions and Insert Statements

In SQL:99, when a User-Defined Type (UDT) is declared as **INSTANTIABLE**, the database management system implicitly generates a **constructor function** with the exact same name as the type. This constructor accepts arguments corresponding to the type's attributes in the order they were defined.

Working Principle:

- To insert a composite attribute, we invoke its constructor: `Size_Type(h, w, l)`.
- To insert an array attribute, we invoke the array constructor: `Seat_Array_Type('val1', 'val2', ...)`.

```

-- -----
-- INSERTING A TRUCK
-- Uses the implicit Size_Type constructor for the size field
-- -----
INSERT INTO Trucks (
    vin, license_no, manufacturer, v_size, color,
    load_capacity, truck_type
) VALUES (
    'TRK-987654321',
    'NY-5544',
    'Volvo',
    Size_Type(3.8, 2.5, 12.0), -- Constructor for Size
    'White',
    15000.00,
    'Heavy-Duty_Box'
);

-- -----
-- INSERTING A SPORTS CAR
-- -----
INSERT INTO SportsCars (
    vin, license_no, manufacturer, v_size, color,

```

```

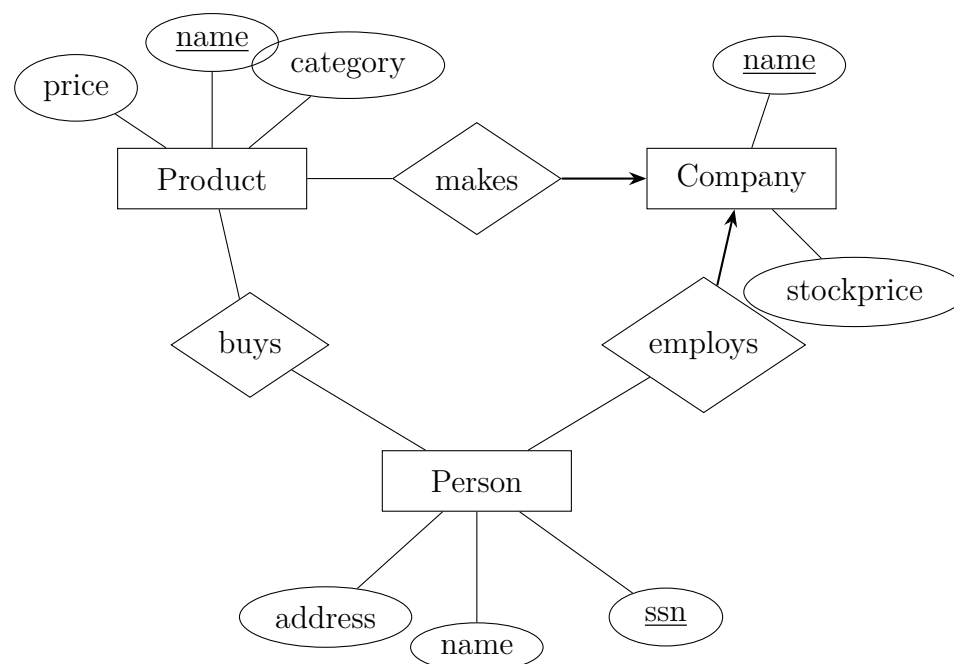
    power, renter_age
) VALUES (
    'SPC-123456789',
    'CA-7788',
    'Porsche',
    Size_Type(1.3, 1.9, 4.5),
    'Red',
    450.00,
    25
);

-----
-- INSERTING A BUS
-- Uses both Size_Type and Seat_Array_Type constructors
-----

INSERT INTO Buses (
    vin, license_no, manufacturer, v_size, color,
    num_passengers, seat_type
) VALUES (
    'BUS-456789123',
    'TX-1122',
    'Mercedes-Benz',
    Size_Type(3.5, 2.6, 14.0),
    'Silver',
    40,
    -- Constructor for the Array of 40 seats
    Seat_Array_Type(
        'Window-Recliner', 'Aisle-Recliner', 'Window-Standard', 'Aisle-Standard',
        'Window-Standard', 'Aisle-Standard', 'Window-Standard', 'Aisle-Standard',
        /* ... remaining 32 seats omitted for brevity ... */
    )
);

```

Q22. Convert the following E-R diagram into a relational schema:



(6 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Analysis of the ER Diagram

Before mapping the Entity-Relationship (ER) diagram to a relational schema, we analyze the semantic structural components, key attributes, and relationship constraints:

- **Entities and their Keys:**

- **Product:** Primary Key is name. Other attributes are *price* and *category*.
- **Company:** Primary Key is name. Other attribute is *stockprice*.
- **Person:** Primary Key is ssn. Other attributes are *name* and *address*.

- **Relationships and Mapping Cardinalities:**

- **makes:** Binary relationship between **Product** and **Company**. The presence of the directed arrow pointing to **Company** indicates a **Many-to-One (N : 1)** mapping cardinality. This means each product is made by at most one company, but a company can make multiple products.

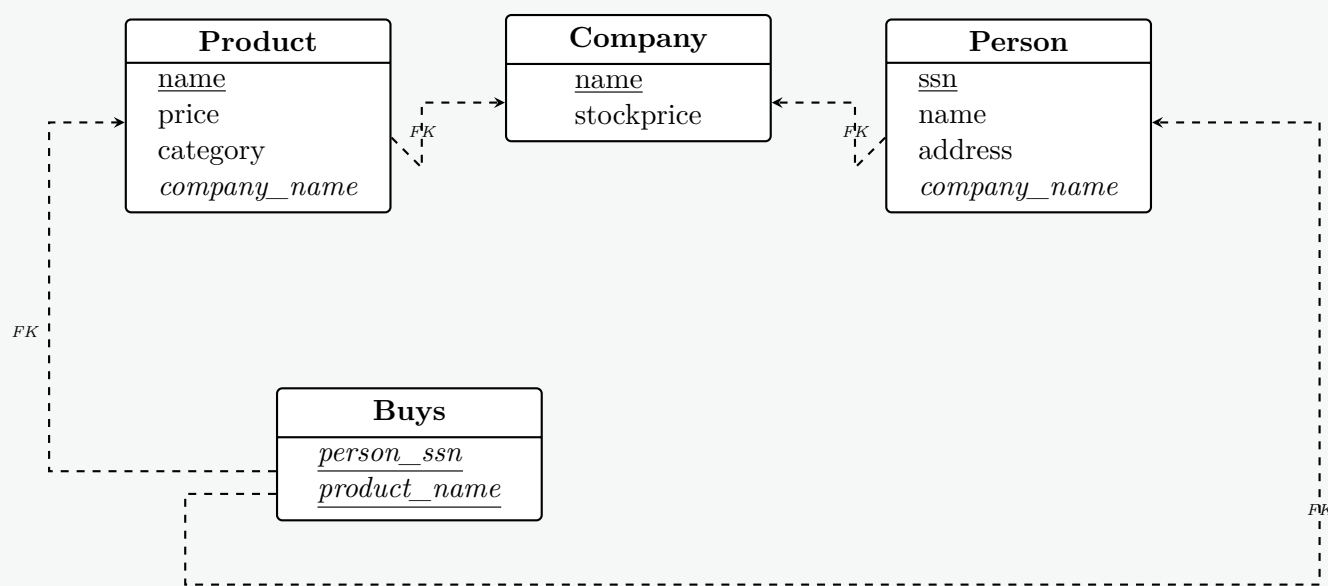
- **employs**: Binary relationship between **Person** and **Company**. The directed arrow pointing to **Company** indicates a **Many-to-One** (N : 1) mapping cardinality. Each person is employed by at most one company, while a company can employ multiple people.
- **buys**: Binary relationship between **Person** and **Product** with straight undirected edges, denoting a standard **Many-to-Many** (M : N) relationship. A person can buy multiple products, and a product can be bought by multiple people.

2. Mapping Process & Rules Applied

- Strong Entities**: Each strong entity type maps directly to a distinct table containing all its basic atomic attributes.
- Many-to-One (N : 1) Relationships (*makes*, *employs*)**: Instead of generating a separate table, the mapping is optimized by placing the primary key of the “One” side (**Company**) as a **Foreign Key** inside the table on the “Many” side.
 - For **makes**: Add *company_name* to the **Product** relation.
 - For **employs**: Add *company_name* to the **Person** relation.
- Many-to-Many (M : N) Relationships (*buys*)**: This must map to a separate relationship relation. The table contains the combination of primary keys from both participating entities, which forms a composite primary key.

3. Relational Schema Diagram

The relational schema design layout below shows tables, underlined primary keys, and clear foreign key directional lines mapping dependencies explicitly:



4. Relational Schema Definitions

Below are the normalized relational schema schemas written in standard textbook notation. Primary keys are **underlined and bolded**, while foreign keys refer back to their origin explicitly.

- Company**(**name**, stockprice)
- Product**(**name**, price, category, *company_name*)
 - Foreign Key [*company_name*] → **Company**(name)
- Person**(**ssn**, name, address, *company_name*)
 - Foreign Key [*company_name*] → **Company**(name)
- Buys**(**person_ssn**, **product_name**)
 - Foreign Key [*person_ssn*] → **Person**(ssn)
 - Foreign Key [*product_name*] → **Product**(name)

Q23. Discuss with example the difference between strong and weak relationships in the context of relational database modelling. How does security concern arise by the choice of primary key? (10 marks) [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Strong vs. Weak Relationships in Database Modeling

In relational database modeling, the terms **strong relationship** (identifying) and **weak relationship** (non-identifying) define how the primary key of a parent entity propagates to a child entity, and whether the child entity can exist independently.

Definitions and Working Principles

- Strong Relationship (Identifying Relationship):** A relationship where the existence of a child entity depends entirely on a parent entity. In the relational schema, the primary key of the parent entity is inherited by the child entity and becomes **part of the child's primary key**. This typically connects a strong entity to a weak entity.
- Weak Relationship (Non-Identifying Relationship):** A relationship between two independent strong entities. The child entity can exist independently of the parent. In the relational schema, the primary key of the parent entity is included in the child entity only as a **foreign key**, not as part of its primary key.

Example ER Diagram

Below is an Entity-Relationship diagram demonstrating both relationship types.

- Weak Relationship (Works_In):** An Employee works in a Department. An Employee has their own unique identifier (EmpID).
- Strong Relationship (Has_Dep):** An Employee has Dependents. A Dependent cannot exist without the Employee and is identified by combining the Employee's EmpID with their own Name.

The diagram shows two entities, Department and Employee, connected by a relationship Works_In. Department has attributes DeptID (primary key) and DName. Employee has attributes EmpID (primary key) and EName. Works_In is a weak relationship, indicated by a single diamond. The cardinality is 1 to N. A second entity, Dependent, is connected to Employee by a relationship Has_Dep. Dependent has attributes Name (Partial) and Relation. Has_Dep is a strong relationship, indicated by a double diamond. The cardinality is 1 to N. The primary key of Employee (EmpID) is inherited by Dependent as part of its primary key.

Comparison Table

Feature	Strong Relationship (Identifying)	Weak Relationship (Non-Identifying)
Entity Dependency	Child (weak entity) is existence-dependent on the parent.	Child (strong entity) is independent of the parent.
Primary Key	Parent's PK becomes part of the Child's PK.	Parent's PK becomes a standard Foreign Key in the Child.
ER Notation	Double diamond connecting to a double rectangle.	Single diamond connecting two single rectangles.
Schema Example	Dependent(<u>EmpID</u> , <u>Name</u> , Relation)	Employee(<u>EmpID</u> , Name, DeptID)

2. Security Concerns Arising from the Choice of Primary Key

The choice of a primary key has profound implications for database security, privacy, and data exposure. Security concerns primarily arise when **Natural Keys** containing sensitive or Personally Identifiable Information (PII) are chosen over arbitrary **Surrogate Keys**.

The Mechanism of Exposure

Because relational databases maintain referential integrity through foreign keys, a primary key from a parent table is replicated across all related child tables. If an employee's Social Security Number (SSN) or email address is used as the primary key in the **Employee** table, it must be copied into the **Payroll**, **Attendance**, and **Performance** tables as a foreign key.

Specific Security Vulnerabilities

- Data Leakage via Wide Distribution:** A natural primary key (e.g., SSN, Phone Number) spreads sensitive PII across the entire database. A user or application that only needs access to a peripheral table (e.g., a parking pass table) will inadvertently gain access to the employee's sensitive identifier.
- Exposure in URLs and API Endpoints:** Primary keys are frequently used in application routing (e.g., `https://app.com/users/987-65-4321`). Exposing a sensitive natural key in a URL makes it visible in browser histories, proxy logs, and HTTP referrers, creating a severe data breach risk.

- (c) **Inability to Comply with Privacy Regulations (GDPR / CCPA):** Privacy laws grant individuals the “Right to be Forgotten” or require masking of PII. If an email address is used as a primary key, changing or deleting it cascades across the entire database schema, which can be computationally expensive, prone to constraint violations, or outright impossible without destroying historical records.
- (d) **Enumeration Attacks:** If a predictable integer or a sequential natural key (like a sequential customer number) is used, malicious actors can easily guess valid keys and execute Insecure Direct Object Reference (IDOR) attacks by incrementing the key in API requests to scrape unauthorized data.

Mitigation

To secure the database, designers should adopt **Surrogate Keys** (e.g., UUIDs or auto-incrementing integers that hold no business meaning) as primary keys. Natural identifiers (like SSN or Email) should be stored as standard attributes with **UNIQUE** and **NOT NULL** constraints, and subjected to proper encryption or hashing without affecting the relational structure.

Q24. What are the advantages and disadvantages of storing derived attributes in tables? (8 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Definition and Core Concept

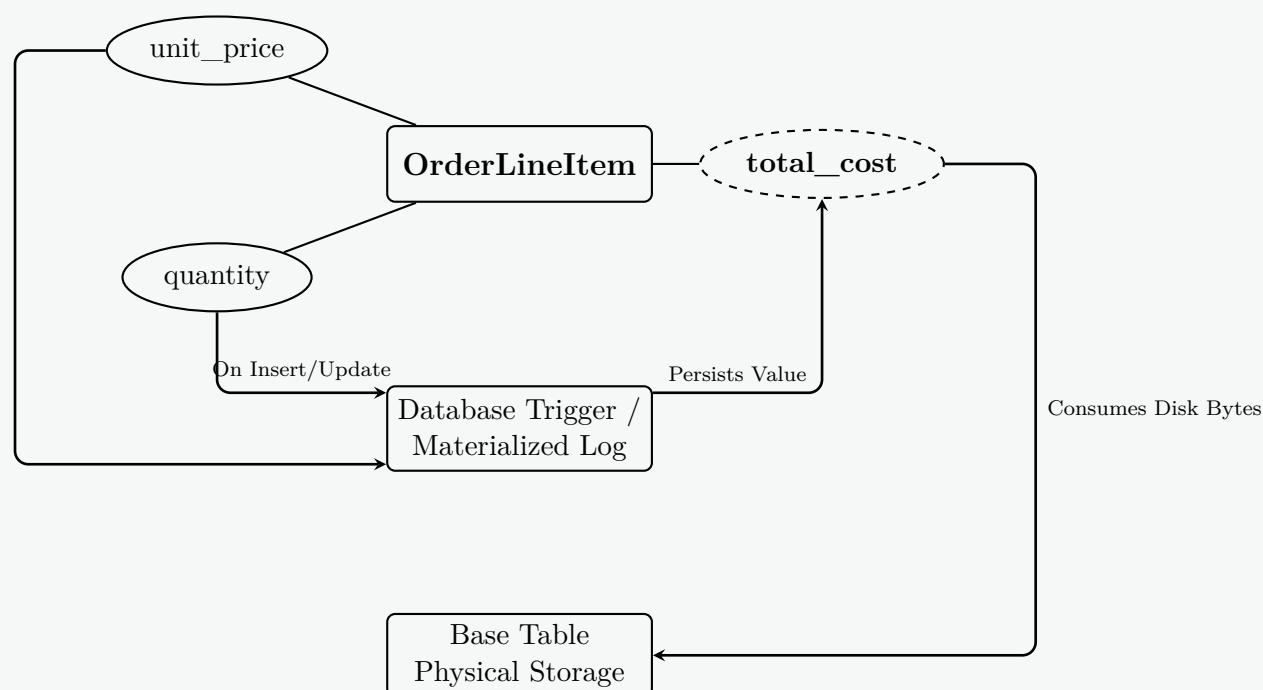
A **derived attribute** is an attribute whose value is not inherently independent but can be computed or calculated from other related attributes or entities already present in the database.

- **Example:** An `age` attribute derived from a `date_of_birth` attribute, or `total_amount` derived from a collection of order line items.

In physical database design, developers face a architectural choice: compute the value dynamically on-the-fly during runtime queries (non-stored approach), or physically write and maintain the calculated value within the base tables. Storing a derived attribute in a table is a core technique of **denormalization** and is often operationalized via **materialized views** or **database triggers**.

2. Diagrammatic Representation

Below is the conceptual mapping showing how a derived attribute (`total_cost`) is computed from stored base attributes (`quantity` and `unit_price`), along with the dual operational mechanics of keeping it physically persisted in a table.



3. Advantages of Storing Derived Attributes

- (a) **Significant Query Performance Enhancement:** Since values are pre-computed, retrieval requires basic reading operations. The Engine skips heavy mathematical computations, logical evaluations, or multi-table string transformations during read cycles.
- (b) **Elimination of Costly Aggregation Joins:** When an attribute relies on summarizing data across large child datasets (e.g., total sales across millions of rows), retrieving the pre-aggregated field avoids CPU-intensive `SUM()`, `COUNT()`, and `GROUP BY` queries accompanied by costly disk-based table joins.
- (c) **Indexability:** Physical columns can be explicitly indexed (B+ Tree or Hash). This enables extremely fast search filtration and ordering conditions within `WHERE` and `ORDER BY` clauses, which is generally restricted or non-performant with dynamic run-time calculations.
- (d) **Simplified Application Logic:** Data presentation logic becomes cleaner. Client applications can fetch ready-to-display computations directly from rows without carrying custom code layers to evaluate totals or periods.

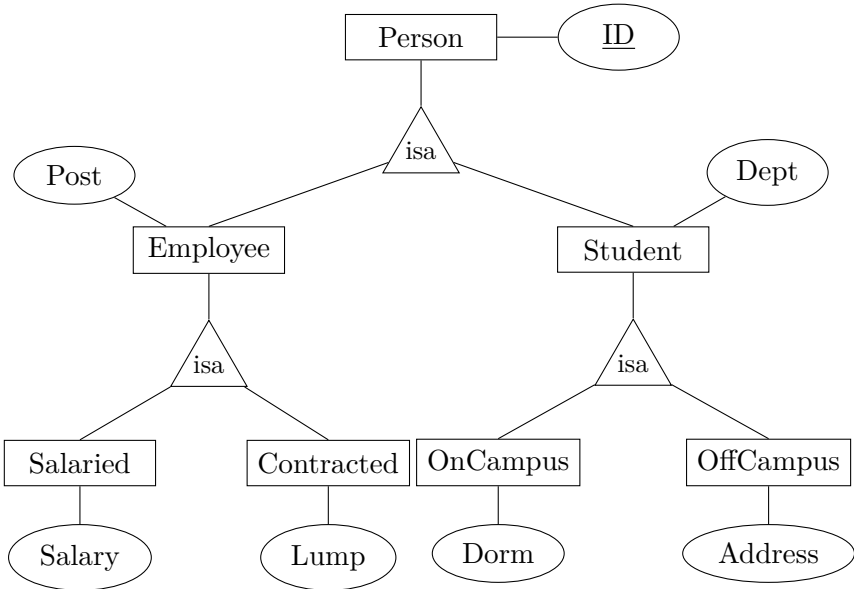
4. Disadvantages of Storing Derived Attributes

- (a) **Data Inconsistency Risks (Anomalies):** Storing derived values creates redundant state tracking. If a base column is updated (e.g., an order’s unit cost changes) and the derived column (*total*) fails to recalculate simultaneously, the database enters an inconsistent state, producing conflicting reporting answers.
- (b) **Write Performance and Transactional Overhead:** Every insertion, update, or deletion of a base component forces the engine to incur recalculation costs. Triggers or transactional loops must fire instantly to update the derived block, degrading data-modification throughput.
- (c) **Storage Space Overhead:** Physical storage requirements increase. While negligible for individual scalar metrics, storing multiple derived fields across hundreds of millions of relational tuples leads to noticeable tablespace inflation and memory buffer pool consumption.
- (d) **Architectural Maintenance Complexity:** Designers must engineer and validate strict maintenance routines, such as structural database constraints, stored procedures, or transaction locks, to safeguard synchronization.

5. Trade-off Comparison Summary

Metric	Persisted Method	/	Storing Dynamic Method	Computation
Primary Focus	Optimization of Read Workloads .		Optimization of Write Workloads .	
Read Performance	Ultra-fast ($O(1)$ or $O(\log N)$ via index).		Slower (Requires compute or table scans).	
Write Cost	High (Triggers/Recalculation overhead).		Low (Writes directly to base field).	
Storage Footprint	Increases proportional to row-cardinality.		Zero additional disk footprint.	
Data Integrity	High risk of anomalies if un-managed.		Always 100% accurate and fresh.	
Typical Use Case	Data Warehouses, OLAP, Reporting systems.		OLTP systems with volatile data streams.	

Q25. Convert the following ER-diagram into relations. Create three separate lists for the three ISA-hierarchy conversion approaches.



(15 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

Analysis of the Hierarchical ER Diagram

The given Entity-Relationship diagram exhibits a multi-level specialization/generalization hierarchy (ISA):

- **Level 0:** **Person** is the root superclass with the primary key ID.
- **Level 1:** **Employee** (attribute: *Post*) and **Student** (attribute: *Dept*) inherit from **Person**.
- **Level 2a:** **Salaried** (attribute: *Salary*) and **Contracted** (attribute: *Lump*) inherit from **Employee**.
- **Level 2b:** **OnCampus** (attribute: *Dorm*) and **OffCampus** (attribute: *Address*) inherit from **Student**.

Approach 1: The Multiple-Relation (ER/Superclass-Subclass) Mapping

Working Principle: A separate relation is created for each entity in the hierarchy. Subclass relations contain their specific local attributes along with the primary key inherited from the root entity, which acts as both the primary key and a foreign key.

- (a) **Person** (ID)
- (b) **Employee** (ID, Post)
Foreign Key: [ID] references Person(ID)
- (c) **Student** (ID, Dept)
Foreign Key: [ID] references Person(ID)
- (d) **Salaried** (ID, Salary)
Foreign Key: [ID] references Employee(ID)
- (e) **Contracted** (ID, Lump)
Foreign Key: [ID] references Employee(ID)
- (f) **OnCampus** (ID, Dorm)
Foreign Key: [ID] references Student(ID)
- (g) **OffCampus** (ID, Address)
Foreign Key: [ID] references Student(ID)

Approach 2: The Concrete-Class (Subclass-Only) Mapping

Working Principle: Relations are generated only for the distinct concrete entities (leaf nodes). Inherited attributes from parent classes are flattened down and explicitly duplicated into each subclass relation. This approach is optimal for total and disjoint specialization.

- (a) **Salaried** (ID, Post, Salary)
- (b) **Contracted** (ID, Post, Lump)
- (c) **OnCampus** (ID, Dept, Dorm)
- (d) **OffCampus** (ID, Dept, Address)

Note: If a **Person** can exist without belonging to any subclass, or an **Employee/Student** can exist without being further specialized, separate non-leaf relations like **Person**(ID), **Employee**(ID, Post), or **Student**(ID, Dept) must additionally be preserved to prevent data loss.

Approach 3: The Single-Relation (Universal/Flattened) Mapping

Working Principle: The entire multi-level hierarchy is collapsed into one massive single relation containing all attributes from every level. Type discriminator attributes are introduced to designate which subclass role a specific tuple represents, filling irrelevant specific attributes with NULL values.

- (a) **AllInOnePerson** (ID, Post, Dept, Salary, Lump, Dorm, Address, IsEmployee, IsStudent, EmployeeType, StudentType)

Discriminator Attribute Meanings:

- **IsEmployee, IsStudent:** Boolean flags handling potential overlapping roles (e.g., a student employee).
- **EmployeeType:** Domain variable containing values like {'Salaried', 'Contracted'} to interpret sub-level attributes.
- **StudentType:** Domain variable containing values like {'OnCampus', 'OffCampus'} to interpret sub-level attributes.

Q26. A database schema consists of four relations:

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

Suggest suitable keys and foreign keys of the above relations. **(5 marks)**

[CSE 303 | L-3/T-1 | 2011–12]

Answer**1. Schema Analysis and Key Correction**

The given database schema models a typical inheritance (or generalization/specialization) hierarchy where **Product** acts as the supertype (base relation) and **PC**, **Laptop**, and **Printer** act as subtypes (derived relations).

Although the problem statement underlines **maker** in the **Product** relation, **maker is not a suitable primary key**. A manufacturer

(maker) produces many different models of PCs, laptops, and printers; therefore, **maker** cannot uniquely identify a single product tuple.

The **natural identifier** across all these relations is the **model** number, which is unique for every hardware product.

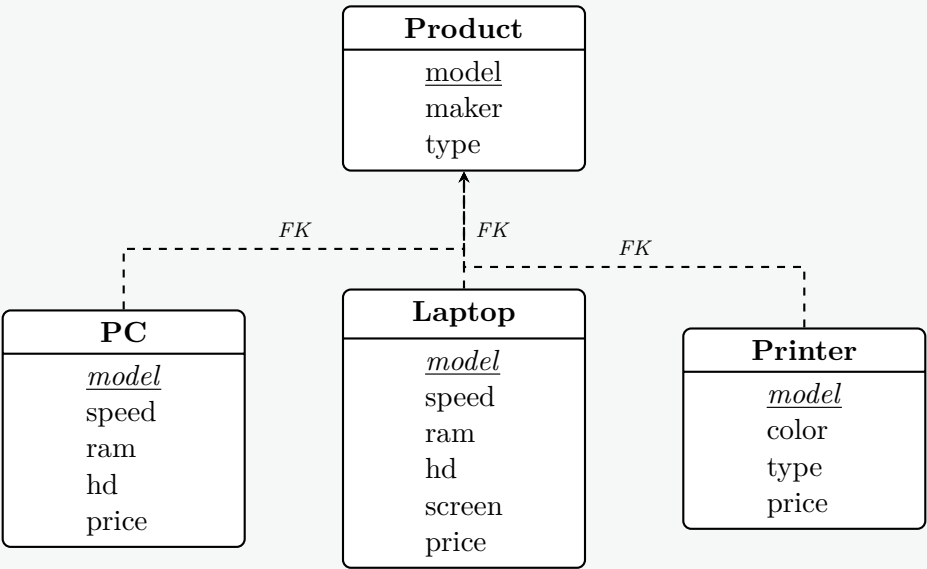
2. Suggested Primary and Foreign Keys

Based on standard database design principles for ISA hierarchies, the keys should be established as follows:

- **Product Relation:**
 - **Primary Key:** `model`. It uniquely identifies any product (PC, Laptop, or Printer) in the database.
 - **Foreign Keys:** None. This is the base table.
- **PC Relation:**
 - **Primary Key:** `model`. It uniquely identifies a specific PC.
 - **Foreign Key:** `model`. It references `Product(model)` to ensure that every PC exists as a product in the main catalog, enforcing referential integrity.
- **Laptop Relation:**
 - **Primary Key:** `model`. It uniquely identifies a specific laptop.
 - **Foreign Key:** `model`. It references `Product(model)`.
- **Printer Relation:**
 - **Primary Key:** `model`. It uniquely identifies a specific printer.
 - **Foreign Key:** `model`. It references `Product(model)`.

3. Relational Schema Diagram

The following diagram illustrates the corrected physical schema, explicitly showing the Primary Keys (underlined) and Foreign Keys (dashed arrows).



4. Summary Table of Schema Constraints

Relation	Primary Key (PK)	Foreign Key (FK)
Product	model	None
PC	model	model → Product(model)
Laptop	model	model → Product(model)
Printer	model	model → Product(model)

Q27. Consider the E-R diagram (Figure 2) which contains composite, multivalued and derived attributes.

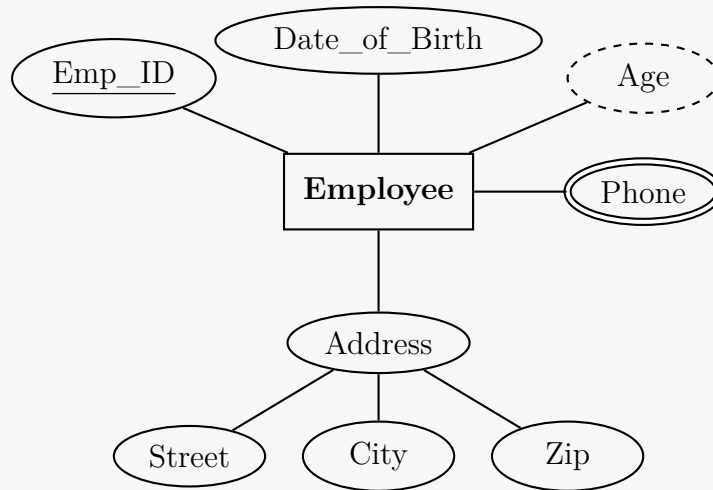
- (a) Give an SQL:2003 schema definition corresponding to the E-R diagram.
- (b) Give constructors for each of the structured types defined above.

Answer

1. Assumed E-R Diagram Structure

Since *Figure 2* was referenced but not provided, the following standard E-R diagram has been constructed to comprehensively demonstrate composite, multivalued, and derived attributes.

- **Entity:** Employee
- **Key Attribute:** Emp_ID
- **Composite Attribute:** Address (composed of Street, City, and Zip)
- **Multivalued Attribute:** Phone (an employee can have multiple phone numbers)
- **Derived Attribute:** Age (calculated from Date_of_Birth)



2. SQL:2003 Schema Definition

The SQL:2003 standard introduced advanced Object-Relational Database (ORDBMS) features. To accurately map the E-R diagram:

- **Composite Attributes** are modeled using User-Defined Types (UDTs).
- **Multivalued Attributes** are modeled using Collection Types (ARRAY or MULTISSET).
- **Derived Attributes** are modeled using the `GENERATED ALWAYS AS` clause, allowing the DBMS to compute the value dynamically.

```

-- 1. Create a composite User-Defined Type (UDT) for Address
CREATE TYPE Address_Type AS (
    Street VARCHAR(100),
    City VARCHAR(50),
    Zip VARCHAR(10)
) INSTANTIABLE;

-- 2. Create a collection type for the multivalued Phone attribute
CREATE TYPE Phone_Array_Type AS VARCHAR(15) ARRAY[5];

-- 3. Define the main Employee table incorporating all structures
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    -- Composite Attribute
    Emp_Address Address_Type,
    -- Multivalued Attribute
    Phones Phone_Array_Type,
    -- Base Attribute for Derivation
    Date_of_Birth DATE,
    -- Derived Attribute
    Age INT GENERATED ALWAYS AS (
        EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM Date_of_Birth)
    )
);
  
```

3. Constructors for Structured Types

In SQL:2003, when a UDT is defined as `INSTANTIABLE`, the system implicitly provides a **constructor function** matching the type's name. Arrays and Multisets also have standard constructor syntaxes.

Working Principle: To insert data into complex columns, you invoke these constructor functions, passing the internal scalar values in the exact order defined by the type schema.

```
-- Inserting a record using Object and Array Constructors
INSERT INTO Employee (
    Emp_ID,
    Emp_Address,
    Phones,
    Date_of_Birth
)
VALUES (
    101,
    -- Constructor for the composite Address_Type
    Address_Type('123_Silicon_Boulevard', 'San_Francisco', '94105'),

    -- Constructor for the multivalued Phone_Array_Type
    Phone_Array_Type('555-0101', '555-0199', '555-0255'),

    -- Standard Date literal (Age is automatically derived)
    DATE '1985-06-15'
);
```

BUET · CSE 215 · Database

Part II

Query Languages

Query Languages

S3 — Query Languages

SQL

Q1. Consider the following relational database schema:

Planet(planet_id, name, type, distance_sun_au), — planet-id is the primary key.
Moon(moon_id, name, planet_id, diameter_km), moon_id is the primary key, and planet_id is a foreign key referencing Planet(planet_id).

- (a) Write an SQL function total_moons() that returns a single integer representing the total number of moons possessed by all planets. Then write the SQL statement to call the function.
- (b) Find the names of all planets whose name contains a substring that starts with “Ma”, then has exactly two unknown characters, and ends with “s”.

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. SQL Function for Total Moons

To create a user-defined function in SQL that retrieves the total number of moons, we can use standard procedural SQL constructs (such as PL/pgSQL). The function executes an aggregate COUNT(*) query on the Moon table.

Function Definition:

```
CREATE OR REPLACE FUNCTION total_moons()  
RETURNS INTEGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    total INTEGER;  
BEGIN  
    -- Calculate the total number of records in the Moon table  
    SELECT COUNT(*) INTO total FROM Moon;  
  
    -- Return the calculated integer value  
    RETURN total;  
END;  
$$;
```

Function Call: Once the function is created, it can be invoked using a standard SELECT statement.

```
SELECT total_moons() AS total_moon_count;
```

2. SQL Pattern Matching using LIKE

To find planets whose name contains a specific substring, we must use the SQL LIKE operator combined with wildcard characters.

Working Principle of SQL Wildcards:

- **Percent Sign (%)**: Matches zero or more characters of any type.
- **Underscore (_)**: Matches exactly one single character.

Constructing the Pattern:

- The substring must start with "Ma".
- It must be followed by exactly two unknown characters, represented by "__".
- It must end with "s".
- Since this substring can appear anywhere in the planet's name (beginning, middle, or end), we wrap the entire pattern in % wildcards.
- Final Pattern: '%Ma__s%'

SQL Query:

```
SELECT name  
FROM Planet  
WHERE name LIKE '%Ma__s%';
```

Explanation of Results: This query will successfully match names such as:

- "Mars" (Matches Ma, zero unknown characters... Wait, Mars has 0 chars between a and s. Let's evaluate: M-a-r-s. The string "Mars" does **not** match Ma__s because it only has one character ('r') between 'a' and 's'.
- "Maaas" (Matches Ma, two characters aa, and s).

87

- "Old Massachusetts" (Matches because "Massa" fits the Ma__s pattern: Ma + ss + a... wait, ending in s is ss + a ≠ s. The substring "Massas" would match: Ma + ss + s).

Note: The exact string "Mars" fails this specific wildcard constraint since it requires exactly **two** characters between the 'a' and the 's'.

Q2. Consider the following relational database schema:

```
employee(employee_name, street, city),
works(employee_name, company_name, salary),
company(company_name, city),
manages(employee_name, manager_name)
```

Write SQL statements for the following:

- (a) Give a 20% salary rise to employees who are managers.
- (b) Delete all tuples in the **works** relation for employees of the company “Sonali Bank”. Note that works.employee_name is a foreign key referencing employee(employee_name), manages.employee_name is a foreign key referencing employee(employee_name), manages.manager_name is a foreign key referencing employee(employee_name) .
- (c) Design a trigger such that whenever a manager’s salary is updated, all employees managed by that manager receive a 10% salary increase.

(18 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Part 1: Update Statement (Salary Rise for Managers)

Explanation

To give a 20% salary rise to all employees who act as managers, we need to update the **works** relation. An employee is identified as a manager if their name appears in the **manager_name** column of the **manages** relation. We utilize a subquery with the **IN** operator to filter these employees.

SQL Implementation

```
UPDATE works
SET salary = salary * 1.20
WHERE employee_name IN (
    SELECT DISTINCT manager_name
    FROM manages
);
```

Part 2: Delete Statement (Employees of "Sonali Bank")

Explanation

To delete all tuples from the **works** relation for employees who work at "Sonali Bank", a straightforward **DELETE** statement filtered by the **company_name** attribute is used.

- **Note on Referential Integrity:** The question specifies that **works.employee_name**, **manages.employee_name**, and **manages.manager_name** are foreign keys referencing **employee(employee_name)**. Since we are only deleting records from the **works** relation (the child table in this relationship) and not from the primary **employee** relation, this deletion does not violate any foreign key constraints.

SQL Implementation

```
DELETE FROM works
WHERE company_name = 'Sonali Bank';
```

Part 3: Trigger Design (Cascading Salary Updates)

Definition & Working Principle

A **database trigger** is a stored procedural program that automatically executes (fires) when a specified event (such as **INSERT**, **UPDATE**, or **DELETE**) occurs on a particular table.

For this requirement:

- **Timing and Event:** **AFTER UPDATE OF salary ON works**
- **Granularity:** **FOR EACH ROW** (Row-level trigger) because we must look up and update the specific subordinates for every individual manager whose salary changes.
- **Conditioning:** The transition variable **NEW.employee_name** is evaluated against the **manages** table to identify subordinates.

SQL Implementation (Standard PostgreSQL / PL-pgSQL Dialect)

```
-- Step 1: Create the Trigger Function
CREATE OR REPLACE FUNCTION adjust_subordinate_salary()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if the employee whose salary was updated is a manager
    -- and update their subordinates' salaries by 10%
    UPDATE works
    SET salary = salary * 1.10
    WHERE employee_name IN (
        SELECT employee_name
        FROM manages
        WHERE manager_name = NEW.employee_name
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Step 2: Create the Trigger Event
CREATE TRIGGER trg_manager_salary_update
AFTER UPDATE OF salary ON works
FOR EACH ROW
EXECUTE FUNCTION adjust_subordinate_salary();
```

Alternative SQL Implementation (Standard SQL:1999)

If using standard ANSI SQL without distinct procedural function steps:

```
CREATE TRIGGER trg_manager_salary_update
AFTER UPDATE OF salary ON works
REFERENCING NEW ROW AS new_row
FOR EACH ROW
WHEN (new_row.employee_name IN (SELECT manager_name FROM manages))
BEGIN ATOMIC
    UPDATE works
    SET salary = salary * 1.10
    WHERE employee_name IN (
        SELECT employee_name
        FROM manages
        WHERE manager_name = new_row.employee_name
    );
END;
```

Important Notes & Potential Issues

- **Cascading/Recursion Risk:** If a manager reports to another manager who also gets a salary update, this trigger could cascade multiple times. Modern relational DBMSs limit the depth of recursive triggers to avoid infinite loops.
- **Performance Impact:** Row-level updates nested with secondary sub-updates inside triggers can cause significant overhead on batch UPDATE operations. Proper indexing on `manages(manager_name)` and `works(employee_name)` is highly recommended.

Q3. Create a table in SQL from a real-life scenario where you need to apply all types of integrity constraints. (10 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

Real-Life Scenario: Human Resources Management System

In a corporate Human Resources (HR) system, maintaining data accuracy and consistency is paramount. When recording employee information, the database must ensure that no two employees share the same corporate ID, salaries are logically valid (greater than zero), emails are distinct, and every employee belongs to a valid, existing department. To enforce these business rules at the database level, we apply **Integrity Constraints**.

Types of Integrity Constraints

Integrity constraints are rules applied to database columns to ensure data validity. They are broadly categorized into three types:

- (a) **Entity Integrity:** Ensures that each row in a table is uniquely identifiable (implemented via PRIMARY KEY and UNIQUE constraints).
- (b) **Referential Integrity:** Ensures the consistency of relationships between tables (implemented via FOREIGN KEY constraints).
- (c) **Domain Integrity:** Restricts the type, format, or range of data that can be inserted into a column (implemented via NOT NULL, CHECK, DEFAULT, and data types).

SQL Implementation

To demonstrate a complete set of constraints, we first define a referenced `departments` table, and then create the `employees` table which incorporates all types of integrity constraints.

```
-- Referenced Parent Table
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

-- Child Table demonstrating all integrity constraints
CREATE TABLE employees (
    -- 1. PRIMARY KEY (Entity Integrity)
    -- Ensures emp_id is unique and not null.
    emp_id INT PRIMARY KEY,

    -- 2. NOT NULL (Domain Integrity)
    -- Ensures an employee must have a registered first name.
    first_name VARCHAR(50) NOT NULL,

    last_name VARCHAR(50),

    -- 3. UNIQUE (Entity Integrity)
    -- Ensures no two employees can share the same email address.
    email VARCHAR(100) UNIQUE,

    -- 4. CHECK (Domain Integrity)
    -- Ensures the salary entered is a positive logical value.
    salary DECIMAL(10, 2) CHECK (salary > 0.00),

    -- 5. DEFAULT (Domain Integrity)
    -- Automatically assigns the current date if no hire date is provided.
    hire_date DATE DEFAULT CURRENT_DATE,

    dept_id INT,

    -- 6. FOREIGN KEY (Referential Integrity)
    -- Ensures an employee can only be assigned to a valid, existing department.
    CONSTRAINT fk_employee_department
        FOREIGN KEY (dept_id)
        REFERENCES departments(dept_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Constraint Analysis Summary

Constraint	Applied Column(s)	Integrity Type	Purpose / Business Rule
PRIMARY KEY	emp_id	Entity	Uniquely identifies each employee record.
UNIQUE	email	Entity	Prevents duplicate email registrations.
FOREIGN KEY	dept_id	Referential	Enforces that an assigned department actually exists.
NOT NULL	first_name	Domain	Prevents the entry of an employee without a name.
CHECK	salary	Domain	Prevents negative or zero values for salary.
DEFAULT	hire_date	Domain	Simplifies data entry by assuming the current date.

Q4. Why are roles used for assigning privileges to users? Create three users U1, U2, and U3 and assign them INSERT and UPDATE privileges on tables T1 and T2 by assigning them a role of “Admin”. Then transfer these privileges to another role named “Super-Admin”. Finally, revoke the UPDATE privilege from “Admin”. **(10 marks)** [CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Purpose of Roles in Privilege Management

A **role** is a named, logical collection of database privileges and permissions that can be granted to users or to other roles. Using roles instead of assigning privileges directly to individual users is a fundamental best practice in Database Management Systems (DBMS) for several reasons:

- **Simplified Administration:** Instead of executing hundreds of GRANT and REVOKE statements for each user, administrators can grant privileges to a single role and assign that role to many users.

- **Dynamic Maintenance:** If a business rule changes and a group of employees requires access to a new table, the administrator only needs to add the privilege to the role. All users possessing that role instantly inherit the new privilege.
- **Role-Based Access Control (RBAC):** Roles naturally map to real-world organizational job functions (e.g., HR_Manager, Data_Analyst). When an employee changes departments, their old role is simply revoked and a new one is granted, minimizing security risks.
- **Reduced Error Rate:** Managing fewer permission sets drastically reduces the likelihood of granting unintended privileges, thereby enforcing the *Principle of Least Privilege*.

2. SQL Implementation

The following SQL sequence performs the required operations. (Note: Exact syntax for creating users and roles may vary slightly depending on the specific DBMS dialect, such as PostgreSQL, Oracle, or MySQL. Standard SQL approaches are used below.)

```
-- 1. Create the three users
CREATE USER U1;
CREATE USER U2;
CREATE USER U3;

-- 2. Create the 'Admin' role
CREATE ROLE Admin;

-- 3. Assign INSERT and UPDATE privileges on tables T1 and T2 to 'Admin'
GRANT INSERT, UPDATE ON T1 TO Admin;
GRANT INSERT, UPDATE ON T2 TO Admin;

-- 4. Assign the 'Admin' role to the three users
GRANT Admin TO U1, U2, U3;

-- 5. Create the 'SuperAdmin' role
CREATE ROLE SuperAdmin;

-- 6. Transfer privileges to 'SuperAdmin'
-- Since SQL does not have a direct "TRANSFER" command, we explicitly
-- grant the target privileges to the new role.
-- (Alternatively, one could execute: GRANT Admin TO SuperAdmin;)
GRANT INSERT, UPDATE ON T1 TO SuperAdmin;
GRANT INSERT, UPDATE ON T2 TO SuperAdmin;

-- 7. Revoke the UPDATE privilege from 'Admin'
-- This leaves the 'Admin' role (and U1, U2, U3) with only the INSERT privilege,
-- while 'SuperAdmin' retains both INSERT and UPDATE.
REVOKE UPDATE ON T1 FROM Admin;
REVOKE UPDATE ON T2 FROM Admin;
```

Explanation of State Post-Execution

- **Users (U1, U2, U3):** Inherit privileges from the Admin role. They now only have INSERT privileges on tables T1 and T2.
- **Admin Role:** Holds the INSERT privilege on T1 and T2.
- **SuperAdmin Role:** Holds both INSERT and UPDATE privileges on T1 and T2.

Q5. Consider the following schema:

```
EMPLOYEE(EMPLOYEEID, LASTNAME, FIRSTNAME, PHONE, HIREDATE, JOBID, SALARY, DEPARTMENTID)
JOB(JOBID, JOBTITLE, MINSALARY, MAXSALARY)
DEPARTMENT(DEPARTMENTID, DEPARTMENTNAME, MANAGERID)
JOB_HISTORY(EMPLOYEEID, JOBID, DEPARTMENTID, STARTDATE, ENDDATE)
```

Write SQL to implement the following queries:

- For each employee, find the total number of employees hired before him/her and those hired after him/her. Print employee last name, total employees hired before, and total employees hired after.
- Print the last name of the employees who never changed their job or department.
- Print the last name of the employees and the difference between their salary and the maximum salary of their department, such that the difference is not more than 5000.
- For each department, print the department name, corresponding manager's last name, and the number of employees working under that manager in that department.
- Print the last name of employees who get a higher salary than the average salary of their department. Also print the difference between their salary and the department average.

(25 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

SQL Queries Implementation

(a) Employees hired before and after each employee

Explanation: To count the number of employees hired before and after a specific employee, we can use correlated subqueries in the SELECT clause. The subqueries compare the HIREDATE of the outer employee with the HIREDATE of all other employees.

```
SELECT
  E1.LASTNAME,
  (SELECT COUNT(*)
   FROM EMPLOYEE E2
   WHERE E2.HIREDATE < E1.HIREDATE) AS Total_Hired_Before,
  (SELECT COUNT(*)
   FROM EMPLOYEE E3
   WHERE E3.HIREDATE > E1.HIREDATE) AS Total_Hired_After
FROM
  EMPLOYEE E1;
```

(b) Employees who never changed job or department

Explanation: Employees who have changed their job or department will have an entry in the JOB_HISTORY table. To find those who have *never* changed, we can use the NOT IN (or NOT EXISTS) operator to filter out employees whose EMPLOYEEID appears in the JOB_HISTORY table.

```
SELECT LASTNAME
FROM EMPLOYEE
WHERE EMPLOYEEID NOT IN (
  SELECT EMPLOYEEID
  FROM JOB_HISTORY
);
```

(c) Salary difference from department maximum

Explanation: We first determine the maximum salary for each department using a subquery in the FROM clause (a derived table). We then join this derived table with the EMPLOYEE table to calculate the difference and apply the filter condition where the difference is ≤ 5000 .

```
SELECT
  E.LASTNAME,
  (D_MAX.Max_Salary - E.SALARY) AS Salary_Difference
FROM
  EMPLOYEE E
JOIN
  (SELECT DEPARTMENTID, MAX(SALARY) AS Max_Salary
   FROM EMPLOYEE
   GROUP BY DEPARTMENTID) D_MAX
ON
  E.DEPARTMENTID = D_MAX.DEPARTMENTID
WHERE
  (D_MAX.Max_Salary - E.SALARY) <= 5000;
```

(d) Department manager and employee count

Explanation: This requires joining three logical entities: the department, the manager of the department, and the employees working in that department. We use LEFT JOIN to ensure departments with no employees or no assigned managers are still included in the result.

```
SELECT
  D.DEPARTMENTNAME,
  M.LASTNAME AS Manager_LastName,
  COUNT(E.EMPLOYEEID) AS Number_Of_Employees
FROM
  DEPARTMENT D
LEFT JOIN
  EMPLOYEE M ON D.MANAGERID = M.EMPLOYEEID
LEFT JOIN
  EMPLOYEE E ON D.DEPARTMENTID = E.DEPARTMENTID
GROUP BY
  D.DEPARTMENTNAME,
  M.LASTNAME;
```

(e) **Employees earning more than department average**

Explanation: Similar to query 3, we calculate the average salary per department using a derived table. We then join this with the EMPLOYEE table, filter for those earning strictly more than the average, and calculate the difference.

```
SELECT
    E.LASTNAME ,
    (E.SALARY - D_AVG.Avg_Salary) AS Salary_Difference
FROM
    EMPLOYEE E
JOIN
    (SELECT DEPARTMENTID , AVG(SALARY) AS Avg_Salary
     FROM EMPLOYEE
     GROUP BY DEPARTMENTID) D_AVG
ON
    E.DEPARTMENTID = D_AVG.DEPARTMENTID
WHERE
    E.SALARY > D_AVG.Avg_Salary;
```

Q6. You have been tasked to prepare a database for a restaurant, The restaurant has provided the following report for you.

Food Item	Containing Items (For Packages)		Ingredients (For Simple Items)			Marketed Price	Sales		
	Simple Item	Qty	Ingredient	Unit Cost	Qty		Buyer	Date	Qty
Chicken Burger			Chicken Patty	60	1	200	C1	01/02/24	2
			Lettuce	30	0.02		C3	02/02/24	4
			Pickle	5	4		C8	02/02/24	1
			Mayonnaise	100	0.02		C1	03/02/24	3
			Sause	70	0.01				
Fried Rice			Raw Rice	50	0.1	80	C4	02/02/24	1
			Egg	7	1		C9	04/02/24	2
			Oil	500	0.005				
Set Menu 1	Fried Rice	1				500	C2	01/02/24	3
	Vegetable	1					C4	03/02/24	1
	Chicken Steak	1					C1	04/02/24	4
							C1	05/02/24	2

In the above table, Qty means quantity of the item. Now, create a relational schema, find its functional dependencies and normalize the schema. Write down all normalized forms up to BCNF, (1NF, 2NF, 3NF, BCNF).
Using the schema created for the restaurant database in:

- (a) Write a SQL query to create a view that contains the total cost of production of each food item. Cost of production for simple items depends on the total cost of ingredients (unit cost × quantity used); for packages, it depends on the total cost of the containing simple items.
- (b) Using the schema and view above, find the monthly profit for each food item where profit = (marketed price – cost of production) × quantity sold in the month.

(20 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

Part 1: Functional Dependencies and Normalization

1. Universal Relation and Functional Dependencies

From the provided report, we can define a universal unnormalized relation encompassing all attributes:
 $R(\text{FoodItem}, \text{ContainingItem}, \text{C_Qty}, \text{Ingredient}, \text{UnitCost}, \text{I_Qty}, \text{MarketedPrice}, \text{Buyer}, \text{Date}, \text{S_Qty})$

Based on the semantics of the restaurant data, the following **Functional Dependencies (FDs)** exist:

- $FD_1: \text{FoodItem} \rightarrow \text{MarketedPrice}$
- $FD_2: \text{Ingredient} \rightarrow \text{UnitCost}$
- $FD_3: \text{FoodItem}, \text{ContainingItem} \rightarrow \text{C_Qty}$
- $FD_4: \text{FoodItem}, \text{Ingredient} \rightarrow \text{I_Qty}$
- $FD_5: \text{FoodItem}, \text{Buyer}, \text{Date} \rightarrow \text{S_Qty}$

2. First Normal Form (1NF)

A table is in 1NF if it contains no repeating groups and all attributes are atomic. The original report contains multiple independent multi-valued facts (ingredients for simple items, containing items for packages, and sales records). Flattening this into a single 1NF table requires a composite primary key made of all determinant attributes to uniquely identify a row: **Primary Key:** {FoodItem, ContainingItem, Ingredient, Buyer, Date}

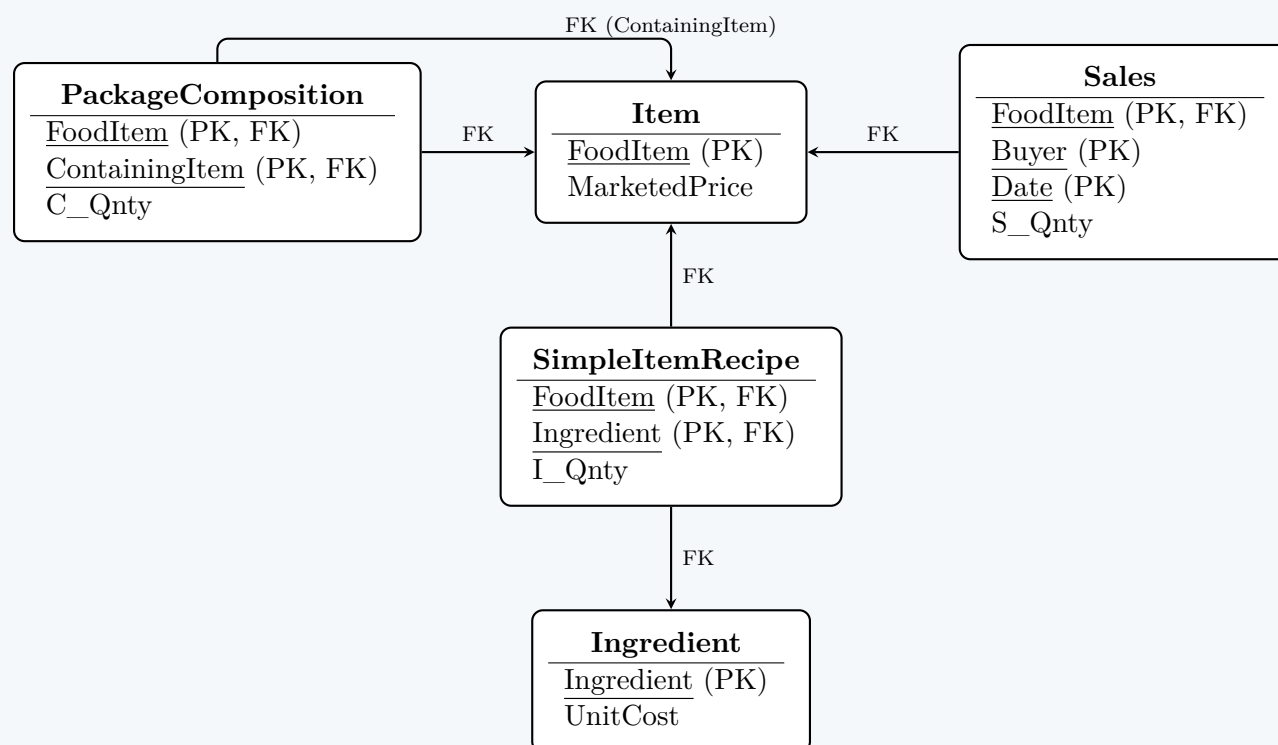
3. Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and all non-prime attributes are fully functionally dependent on the entire primary key (no partial dependencies). Looking at our FDs, every non-prime attribute depends only on a *part* of the composite key. We decompose the universal relation into five distinct tables to eliminate these partial dependencies:

- **Item**(FoodItem, MarketedPrice)
- **Ingredient**(Ingredient, UnitCost)
- **PackageComposition**(FoodItem, ContainingItem, C_Qnty)
- **SimpleItemRecipe**(FoodItem, Ingredient, I_Qnty)
- **Sales**(FoodItem, Buyer, Date, S_Qnty)

4. Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF)

- **3NF:** A relation is in 3NF if it is in 2NF and has no transitive dependencies (no non-prime attribute depends on another non-prime attribute). In all five 2NF tables, non-prime attributes depend *only* on their respective primary keys. Thus, the schema is **already in 3NF**.
- **BCNF:** A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. In our schema, the determinants for all FDs are exactly the primary keys of their respective tables. Thus, the schema is **already in BCNF**.



Part 2: SQL Implementations

1. View for Total Cost of Production

To calculate the total cost, we must account for both simple items (derived from ingredients) and packages (derived from simple items). We can achieve this cleanly using a Common Table Expression (CTE) integrated into a View.

```

CREATE VIEW v_ProductionCost AS
-- Step 1: Calculate production cost for Simple Items
WITH SimpleItemCost AS (
    SELECT
        r.FoodItem,
        SUM(r.I_Qnty * i.UnitCost) AS Cost
    FROM
        SimpleItemRecipe r
    JOIN
        Ingredient i ON r.Ingredient = i.Ingredient

```

```

        GROUP BY
            r.FoodItem
    )
    -- Combine Simple Items and Packages into the final view
    SELECT
        FoodItem,
        Cost AS TotalCost
    FROM
        SimpleItemCost

    UNION ALL

    -- Step 2: Calculate production cost for Packages
    SELECT
        p.FoodItem,
        SUM(p.C_Qnty * s.Cost) AS TotalCost
    FROM
        PackageComposition p
    JOIN
        SimpleItemCost s ON p.ContainingItem = s.FoodItem
    GROUP BY
        p.FoodItem;

```

2. Query for Monthly Profit

Using the generated schema and the newly created `v_ProductionCost` view, we calculate the profit aggregated by Food Item, Year, and Month.

```

    SELECT
        s.FoodItem,
        EXTRACT(YEAR FROM s.Date) AS SaleYear,
        EXTRACT(MONTH FROM s.Date) AS SaleMonth,
        SUM((i.MarkedPrice - v.TotalCost) * s.S_Qnty) AS MonthlyProfit
    FROM
        Sales s
    JOIN
        Item i ON s.FoodItem = i.FoodItem
    JOIN
        v_ProductionCost v ON s.FoodItem = v.FoodItem
    GROUP BY
        s.FoodItem,
        EXTRACT(YEAR FROM s.Date),
        EXTRACT(MONTH FROM s.Date)
    ORDER BY
        SaleYear DESC,
        SaleMonth DESC,
        MonthlyProfit DESC;

```

Q7. Consider the Toys database of a retail store. It has the following database schema:

Toys (id, name, color, min_age, weight, number_available)

Client (id, name, age, address, city)

Request (client id, toy id)

The underlined attributes are keys for each relation. The tables contain the following information:

- *Toys* records information about all of the toys in the store. Each Toy has a unique integer *id*. Attributes *name* and *color* are strings that describe the toy; *min_age*, *weight*, and *number_available* are integers giving the minimum age a child should be to use the toy, the weight of the toy in pounds, and how many are currently available in the store.
- *Client* stores information about each of the clients. Each client has a unique integer *id*, strings with the client's *name*, *address*, and *city*, and an integer *age*.
- *Request* has a pair of integers indicating that a particular client has requested a particular toy.

You should assume that all entries in all of the tables are not null. Answer the followings:

- Give the name of the clients who do not have any request at the moment for any toys.
- List the name of the toys that has weight more than the average weight of the toys with the same color.
- List the name of the requesting clients who have to wait because the toys are currently unavailable at the store.
- List all of the requests where *age* of the client is less than the *min-age* listed for the Toy. The answer should include the client id, client name, and the toy id. The results should be sorted by client name.

v. Give the id and name of the "most popular" toy. The most popular toy is the one that has more requests than all others. If there is a tie so there is more than one toy with the maximum number of requests, give the id's and names of all of these toys. They do not need to be in any particular order.

(25 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer**SQL Queries Implementation**

Note on Schema Attributes: For valid SQL syntax, spaces in attribute names provided in the problem description (e.g., client id, toy id) have been converted to standard underscore notation (client_id, toy_id) in the queries below.

i. Clients with no requests

Explanation: To find clients who have not requested any toys, we can use a subquery with the NOT IN operator (or NOT EXISTS) to filter out any client IDs that appear in the Request table.

```
SELECT name
FROM Client
WHERE id NOT IN (
    SELECT client_id
    FROM Request
);
```

ii. Toys weighing more than the average weight of their color

Explanation: This requires a **correlated subquery**. For each toy evaluated in the outer query, the inner query calculates the average weight of all toys sharing that specific color.

```
SELECT T1.name
FROM Toys T1
WHERE T1.weight > (
    SELECT AVG(T2.weight)
    FROM Toys T2
    WHERE T2.color = T1.color
);
```

iii. Clients waiting for unavailable toys

Explanation: We must join all three tables (Client, Request, and Toys) to connect clients to the toys they requested. We filter for records where the number_available is exactly 0. The DISTINCT keyword is used to avoid printing a client's name multiple times if they are waiting on more than one unavailable toy.

```
SELECT DISTINCT C.name
FROM Client C
JOIN Request R ON C.id = R.client_id
JOIN Toys T ON R.toy_id = T.id
WHERE T.number_available = 0;
```

iv. Requests violating the minimum age requirement

Explanation: Similar to the previous query, we join the relevant tables and apply a filtering condition comparing the client's age against the toy's min_age. Finally, the ORDER BY clause sorts the result set alphabetically by the client's name.

```
SELECT C.id AS client_id, C.name AS client_name, T.id AS toy_id
FROM Client C
JOIN Request R ON C.id = R.client_id
JOIN Toys T ON R.toy_id = T.id
WHERE C.age < T.min_age
ORDER BY C.name ASC;
```

v. The "most popular" toy(s)

Explanation: We first group the requests by toy_id to count how many times each toy was requested. To account for potential ties (multiple toys sharing the absolute maximum number of requests), we use the >= ALL condition against a subquery that lists all request counts.

```
SELECT T.id, T.name
FROM Toys T
JOIN Request R ON T.id = R.toy_id
GROUP BY T.id, T.name
HAVING COUNT(R.client_id) >= ALL (
    SELECT COUNT(client_id)
    FROM Request
    GROUP BY toy_id
);
```

Alternative Approach: In modern SQL (e.g., PostgreSQL, SQL Server), this can also be elegantly achieved using a Common Table

Expression (CTE) or the RANK() window function.

Q8. Consider the following relational database schema:

Product(maker, model, type), PC(model, speed, ram, hd, price), Laptop(model, speed, ram, hd, screen, price)

Write SQL DML queries to reflect the following:

- (a) Insert the fact that for every PC there is a laptop with the same manufacturer, speed, ram, and hard disk, a 17-inch screen, a model number 1100 greater and a price \$400 more.
- (b) Delete all laptops made by a manufacturer that does not make PCs.

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

Relational Schema Overview

Before writing the queries, it is important to visualize the schema and the referential integrity (foreign key) constraints. The `model` attribute in both PC and Laptop acts as a foreign key referencing the Product table.

PC

<u>model</u>	speed	ram	hd	price
--------------	-------	-----	----	-------

↓ FK

Product

maker	<u>model</u>	type
-------	--------------	------

↑ FK

Laptop

<u>model</u>	speed	ram	hd	screen	price
--------------	-------	-----	----	--------	-------

1. Inserting Corresponding Laptops for Every PC

To satisfy this requirement, data must be inserted into both the Product and Laptop relations. Because Laptop.model acts as a foreign key that references Product.model, the parent tuple must be created in the Product table before inserting the child tuple with the detailed specifications into the Laptop table.

```
-- Step 1: Insert the new models into the Product table
INSERT INTO Product (maker, model, type)
SELECT maker, model + 1100, 'laptop'
FROM Product
WHERE type = 'pc';

-- Step 2: Insert the detailed specifications into the Laptop table
INSERT INTO Laptop (model, speed, ram, hd, screen, price)
SELECT model + 1100, speed, ram, hd, 17, price + 400
FROM PC;
```

Working Principle:

- Query 1 (Product Table):** Retrieves all products designated as PCs. For each, it generates a new tuple inheriting the same maker, assigns the transformed primary key (model + 1100), and statically sets the type to 'laptop'.
- Query 2 (Laptop Table):** Iterates through the PC table to retrieve the exact hardware specifications. It applies the mathematical operations requested (model + 1100 and price + 400) and provides a constant scalar value (17) for the newly introduced screen attribute.

2. Deleting Laptops Based on Manufacturer Exclusivity

The objective is to eliminate records from the Laptop table if their manufacturer **does not** produce any PCs. This requires cross-referencing the Product table to determine the associated manufacturers for each laptop model and filtering out the ones who manufacture PCs.

```
DELETE FROM Laptop
WHERE model IN (
  SELECT model
  FROM Product
  WHERE maker NOT IN (
    SELECT maker
    FROM Product
    WHERE type = 'pc'
  )
);
```

Working Principle:

- Innermost Subquery:** SELECT maker FROM Product WHERE type = 'pc' generates a set of all manufacturers that produce at least one PC.

- **Middle Subquery:** Identifies all product `model` numbers whose `maker` is absent from the aforementioned set. Effectively, this isolates models built by manufacturers strictly dedicated to non-PC devices.
- **Outer Delete Statement:** Removes all tuples from the `Laptop` table whose `model` number falls within the restricted set identified by the subqueries.

Note on Referential Integrity: Assuming the database does not implement `ON DELETE CASCADE` constraints, executing the above query only clears the data in the `Laptop` table. If strict maintenance of the catalog is required, a symmetric `DELETE FROM Product` operation utilizing the identical `WHERE` clause must be dispatched subsequently to prune the orphaned generic items.

Q9. Show the SQL command to create a duplicate copy of a table named `Student`. What would the command be if we only want to copy the schema and not the data of the `Student` table? (5 marks) [CSE 215 | L-2/T-2 | 2021–22]

Answer

In SQL, duplicating a table can be achieved using the **CREATE TABLE AS SELECT (CTAS)** statement. This allows a new table to be created and populated based on the result set of a `SELECT` query.

1. Creating a Full Duplicate (Schema and Data)

To create a complete duplicate of the `Student` table, including both its structure (columns and data types) and all of its existing data, we select all records without any restrictive conditions.

SQL Command:

```
CREATE TABLE Student_Copy AS
SELECT * FROM Student;
```

Working Principle:

- The database engine first evaluates the `SELECT * FROM Student` query.
- It creates a new table named `Student_Copy` with the exact same column names and data types as the result set.
- It then inserts every row retrieved by the query into the newly created table.

2. Creating a Schema-Only Duplicate (No Data)

To copy only the table structure (schema) without transferring any records, we append a `WHERE` clause that evaluates to **FALSE** for every row. A universally accepted convention is to use `WHERE 1=0`.

SQL Command:

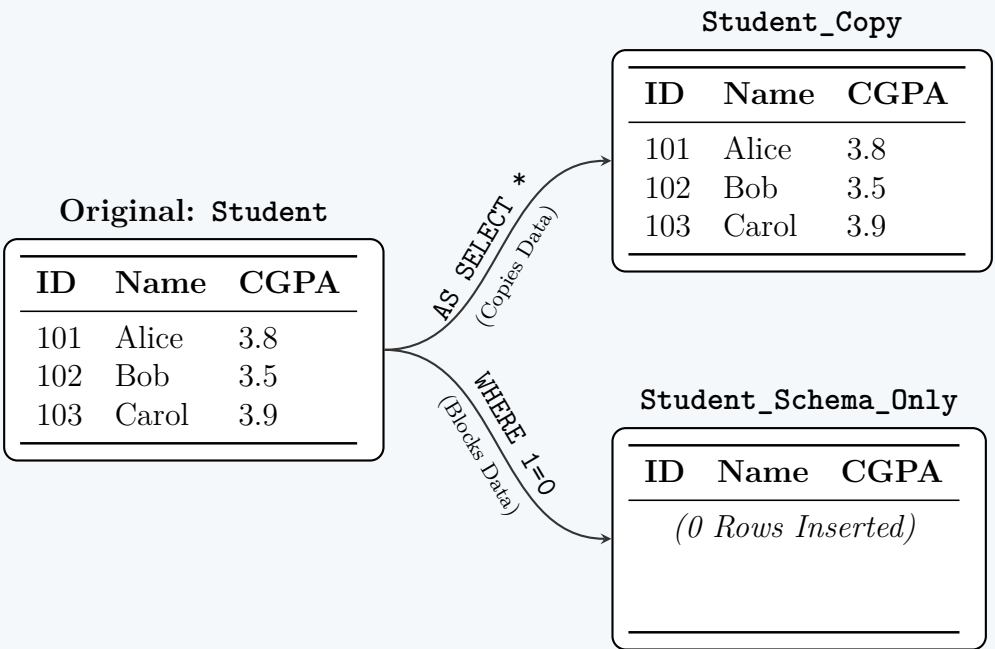
```
CREATE TABLE Student_Schema_Only AS
SELECT * FROM Student
WHERE 1=0;
```

Working Principle:

- The parser still analyzes the `SELECT *` statement to determine the metadata (column names and data types) required to construct the new table `Student_Schema_Only`.
- However, because the condition `1=0` is logically false, the query returns **zero rows**.
- As a result, the new table is created with the correct structure but remains completely empty.

Visual Representation of the Operations

The following diagram illustrates how the CTAS operation processes data based on the provided logical conditions.



Important Notes on Table Duplication

- **Loss of Constraints:** The standard `CREATE TABLE AS` command copies the column definitions and nullability, but it **does not** copy primary keys, foreign keys, default values, triggers, or indexes.
- **Dialect-Specific Alternatives:**
 - In **MySQL** and **PostgreSQL**, a perfect structural clone (including constraints and indexes) can be created using:
`CREATE TABLE Student_Schema_Only LIKE Student;`
 - In **SQL Server**, the equivalent approach uses the `INTO` clause:
`SELECT * INTO Student_Copy FROM Student;`

Q10. You have been analysing data from a social networking site and have derived the following relation which captures topics discussed by various users:

`Discussion(user1, user2, topic)`

The relation contains a tuple (u_1, u_2, t) every time a user u_1 discussed a topic t with user u_2 . To avoid duplicate entries, **user1** always precedes **user2** in alphabetical order.

- Write a SQL query that returns all topics discussed by Pori and Himu but not discussed by Pori and Misir Ali.
- Write a SQL query that returns the number of topics discussed by more than 10 pairs of users.

(24 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

To solve these queries, we must first analyze the structural constraint imposed by the relation: **user1** always precedes **user2** in alphabetical order.

- **Pair 1 (Pori and Himu):** Alphabetically, 'H' (Himu) comes before 'P' (Pori). Therefore, any discussion between them will be stored as `user1 = 'Himu'` and `user2 = 'Pori'`.
- **Pair 2 (Pori and Misir Ali):** Alphabetically, 'M' (Misir Ali) comes before 'P' (Pori). Therefore, this discussion will be stored as `user1 = 'Misir Ali'` and `user2 = 'Pori'`.

1. Topics discussed by Pori and Himu but not by Pori and Misir Ali

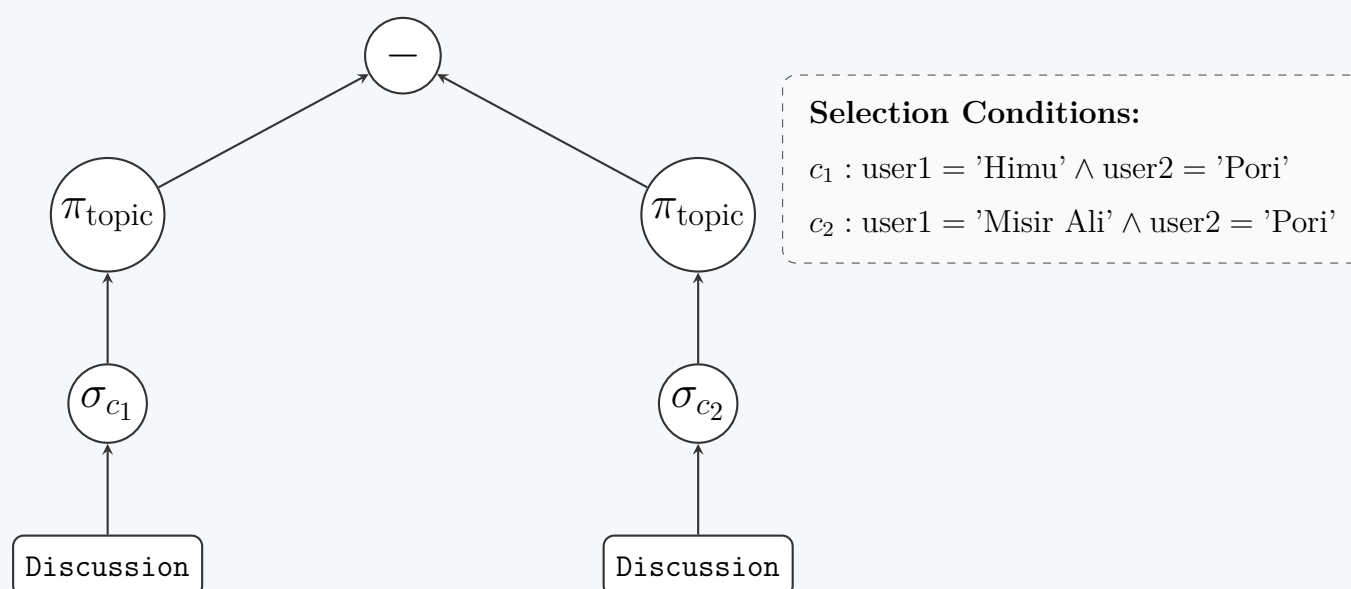
Explanation: This is a classic **set difference** problem. We need to find the set of topics discussed by the first pair and subtract the set of topics discussed by the second pair. In standard SQL, this is achieved using the `EXCEPT` (or `MINUS` in Oracle) operator.

SQL Query:

```
SELECT topic
FROM Discussion
WHERE user1 = 'Himu' AND user2 = 'Pori'
EXCEPT
SELECT topic
FROM Discussion
WHERE user1 = 'Misir Ali' AND user2 = 'Pori';
```

Note: An alternative valid approach is using a `NOT IN` or `NOT EXISTS` subquery.

Query Execution Tree (Relational Algebra):



2. Number of topics discussed by more than 10 pairs of users

Explanation: To answer this, we must first **group** the relation by `topic`. For each topic, we count the number of tuples (which correspond to unique pairs of users, given the schema's constraints). We then filter these groups using the `HAVING` clause to only

keep those with a count strictly greater than 10. Finally, because the question asks for the *number of topics* (a single scalar value) rather than the list of topics themselves, we must wrap this grouped result inside a subquery (or Common Table Expression) and apply the COUNT() aggregate function to the result set.

SQL Query:

```
SELECT COUNT(*) AS popular_topic_count
FROM (
    SELECT topic
    FROM Discussion
    GROUP BY topic
    HAVING COUNT(*) > 10
) AS PopularTopics;
```

Working Principle for Query 2:

- (a) **Internal Query (GROUP BY):** Aggregates the Discussion table into distinct topic buckets.
- (b) **Internal Query (HAVING):** Eliminates any topic bucket containing 10 or fewer user pairs.
- (c) **External Query (SELECT COUNT):** Counts the total number of rows returned by the internal query, yielding the final numerical answer.

Q11. You are in charge of managing the program committee for an important conference. The **conference database** stores information about papers submitted to the conference (table Paper), reviewers on the program committee (table Reviewer), and the assignment of reviewers to papers (table Reviews). Each reviewer on the program committee will have to review a set of papers. Each paper will be reviewed by a subset of reviewers.

Paper(pid, title)

Reviewer(rid, name)

Reviews(rid, pid)

- i. Declare the conference database using CREATE TABLE. Include the declaration of the keys and maintain the referential integrities as needed. The details are as follows:
 - pid is a unique paper identifier and the primary key of the Paper table.
 - rid is a unique reviewer identifier and the primary key of the Reviewer table.
 - rid in the Reviews table must be someone mentioned in the Reviewer table. Modifications to Reviewer table that violate this constraint result in:
 - * the rid in Reviews being set to NULL upon deletion.
 - * update of the rid in Reviews table.
 - pid in the Review table must be a paper mentioned in the Paper table. Modifications to Paper table that violate this constraint are **rejected**.
 - A reviewer is assigned zero or more papers.
 - A paper is assigned zero or more reviewers.
- ii. Write a **SQL query** that finds all papers with fewer than three reviewers assigned to them. The output of the query should be a list paper titles. The result should **include papers without any reviewers** assigned to them.
- iii. Write a SQL query that find the reviewers with the most papers assigned to them. There can be more than one such reviewer. The output of the query should be a list of reviewer names. A reviewer should be listed if no other reviewer has strictly more papers to review.

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Part i: Database Schema Declaration

Based on the rules provided for the program committee database, we must carefully define the primary keys (PK) and foreign keys (FK) along with their respective referential integrity constraints (ON DELETE and ON UPDATE clauses).

Because the Reviews table must allow rid to be set to NULL upon deletion of a reviewer, rid in the Reviews table must be a nullable column. Consequently, (rid, pid) cannot be a strict composite primary key (as PK columns cannot contain NULLs).

SQL DDL Commands:

```
CREATE TABLE Paper (
    pid INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL
);

CREATE TABLE Reviewer (
    rid INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE Reviews (
    review_id INT PRIMARY KEY, -- Surrogate key needed to allow NULLs in rid
```



```

rid INT,
pid INT NOT NULL,
CONSTRAINT fk_reviewer
    FOREIGN KEY (rid) REFERENCES Reviewer(rid)
    ON DELETE SET NULL
    ON UPDATE CASCADE,
CONSTRAINT fk_paper
    FOREIGN KEY (pid) REFERENCES Paper(pid)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT
);

```

Note: *RESTRICT* explicitly rejects modifications that violate referential integrity. In some DBMS implementations, *NO ACTION* achieves the same result.

Entity-Relationship (ER) Diagram:



Part ii: Papers with Fewer Than Three Reviewers

To include papers with **zero** reviewers, we must use a `LEFT JOIN` starting from the `Paper` table. We then group the results by paper and filter using the `HAVING` clause.

SQL Query:

```

SELECT p.title
FROM Paper p
LEFT JOIN Reviews r ON p.pid = r.pid
GROUP BY p.pid, p.title
HAVING COUNT(r.rid) < 3;

```

Working Principle: The `LEFT JOIN` ensures that a paper without any rows in the `Reviews` table still appears in the joined set with a `NULL` `rid`. `COUNT(r.rid)` ignores `NULL` values, yielding a count of 0 for such papers, correctly satisfying the `< 3` condition.

Part iii: Reviewers with the Most Papers Assigned

To find the reviewer(s) with the most assignments, we can utilize Common Table Expressions (CTEs) or nested subqueries. We must also account for reviewers who might have 0 papers if the maximum assigned to anyone happens to be 0 (though rare, it is logically rigorous).

SQL Query:

```

WITH ReviewerCounts AS (
    SELECT r.rid, r.name, COUNT(rv.pid) AS paper_count
    FROM Reviewer r
    LEFT JOIN Reviews rv ON r.rid = rv.rid
    GROUP BY r.rid, r.name
)
SELECT name
FROM ReviewerCounts
WHERE paper_count = (SELECT MAX(paper_count) FROM ReviewerCounts);

```

Working Principle:

- **ReviewerCounts (CTE):** Calculates the total number of assigned papers for every reviewer, using a `LEFT JOIN` to ensure those with 0 papers are recorded.
- **Subquery `MAX(paper_count)`:** Determines the highest assignment count among all reviewers.
- **Main Query:** Selects the names of all reviewers whose `paper_count` exactly matches the maximum value, gracefully handling any ties.

Q12. Consider the university schema: `Classroom`(building, room_number, capacity), `department`(dept_name, building, budget), `course`(course_id, title, dept_name, credits), `instructor`(ID, name, dept_name, salary), `section`(course_id, sec_id, semester, year, building, room_number, time_slot_id), `teaches`(ID, course_id, sec_id, semester, year), `student`(ID, name, dept_name, tot_cred), `takes`(ID, course_id, sec_id, semester, year, grade), `advisor`(s_ID, i_ID), `timeslot`(time_slot_id, day, start_time, end_time), `prereq`(course_id, prereq_id). Write SQL queries for the following:

- Find all instructors whose salary is above average for their department.
- Find all courses taught in both the Fall 2022 semester and in the Spring 2022 semester.
- Find all students who have taken all courses taught by instructor named 'John Doe'.

- (d) List all departments along with the number of students in each department.
- (e) Find the departments that have the lowest average salary.

(20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer**1. Instructors with Salary Above Department Average**

To find instructors whose salary exceeds the average salary of their own department, we use a **correlated subquery**. For each instructor in the outer query, the inner query calculates the average salary for that specific instructor's department.

```
SELECT i1.ID, i1.name, i1.dept_name, i1.salary
FROM instructor i1
WHERE i1.salary > (
    SELECT AVG(i2.salary)
    FROM instructor i2
    WHERE i2.dept_name = i1.dept_name
);
```

2. Courses Taught in Both Fall 2022 and Spring 2022

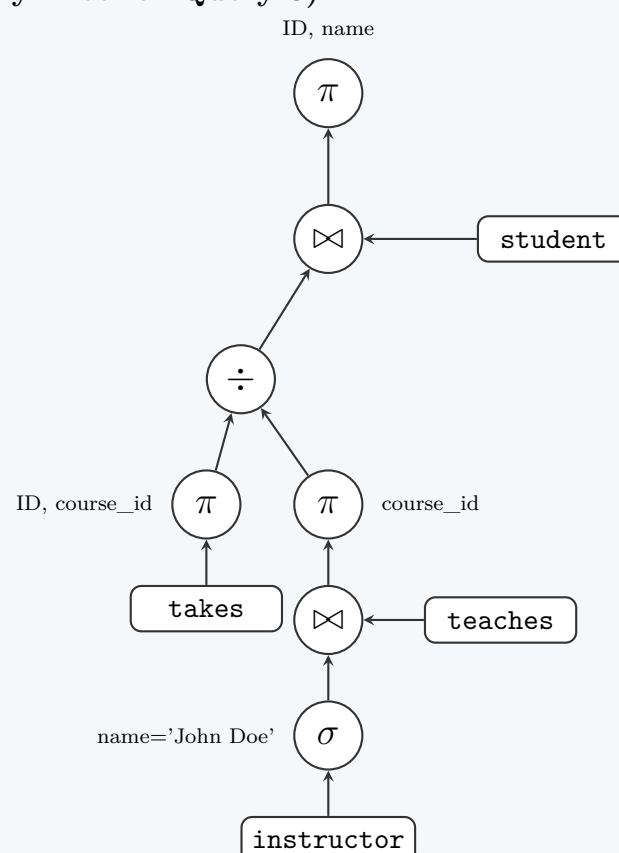
The **INTERSECT** operator is the most direct way to find the common elements between two sets. We query the courses for Fall 2022 and intersect them with the courses for Spring 2022.

```
SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2022
INTERSECT
SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2022;
```

3. Students Who Have Taken All Courses Taught by 'John Doe'

This query represents the relational algebra **division** operation. Since standard SQL does not have an explicit division operator, we implement it using double negation (**NOT EXISTS**). We look for a student for whom there does *not exist* a course taught by 'John Doe' that the student has *not* taken.

```
SELECT s.ID, s.name
FROM student s
WHERE NOT EXISTS (
    SELECT t.course_id
    FROM teaches t
    JOIN instructor i ON t.ID = i.ID
    WHERE i.name = 'John Doe'
    EXCEPT
    SELECT tk.course_id
    FROM takes tk
    WHERE tk.ID = s.ID
);
```

Visualizing Relational Division (Query Tree for Query 3):

4. List Departments with Number of Students

We use a LEFT JOIN to ensure that departments with zero students are still included in the output. The COUNT function targets s.ID so that missing students in a department evaluate to a count of 0 instead of 1.

```
SELECT d.dept_name, COUNT(s.ID) AS num_students
FROM department d
LEFT JOIN student s ON d.dept_name = s.dept_name
GROUP BY d.dept_name;
```

5. Departments with the Lowest Average Salary

This query groups the instructors by department to find the average salary. The HAVING clause uses the <= ALL operator to compare each department's average salary against all other departments' average salaries.

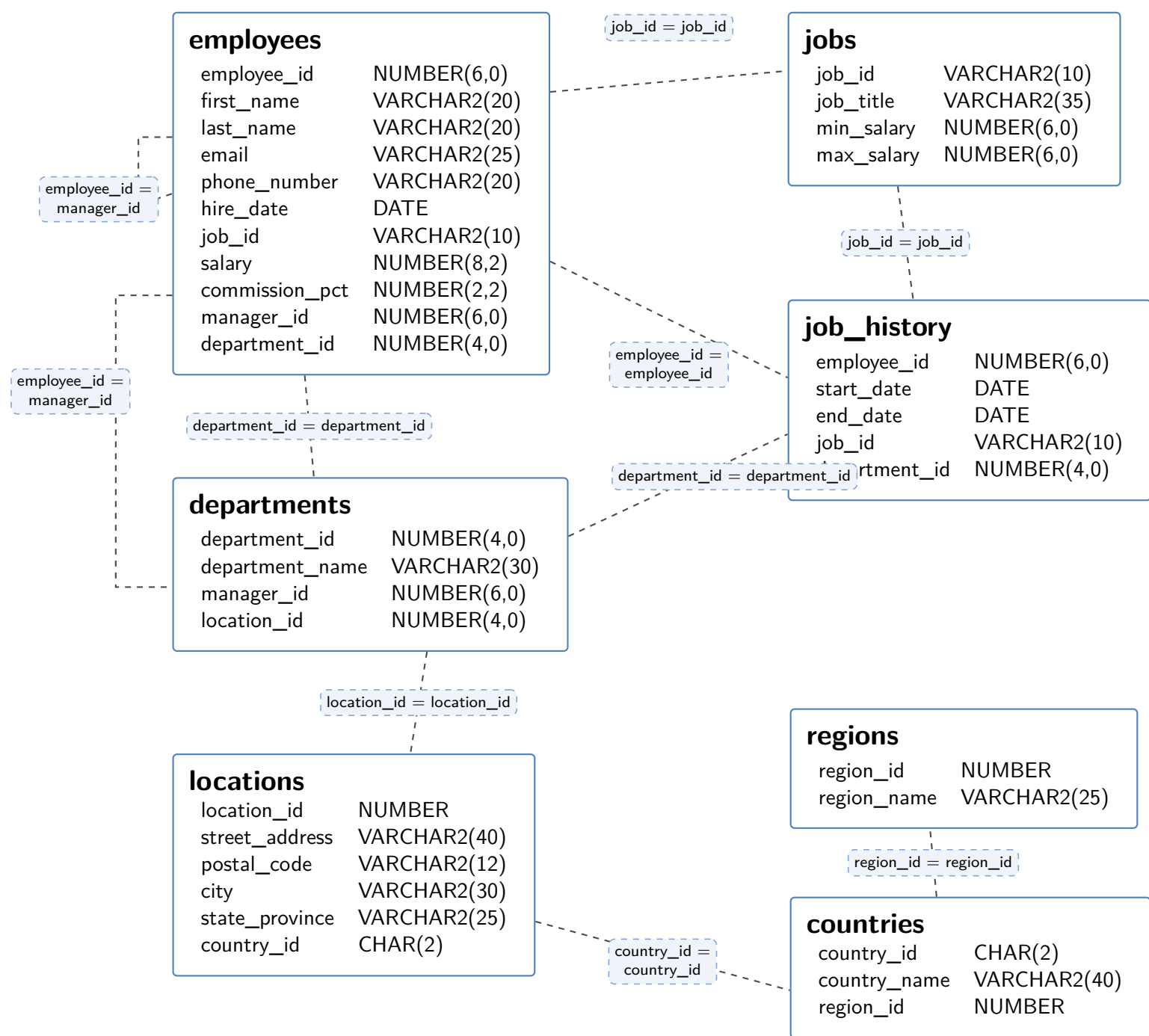
```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) <= ALL (
    SELECT AVG(salary)
    FROM instructor
    GROUP BY dept_name
);
```

Note: This approach correctly handles ties if multiple departments share the exact same minimum average salary.

- Q13.** (a) The following SQL query intended to show the last name of each manager along with the number of employees he is managing is not giving the correct result. Find the fault and rewrite it correctly:

```
SELECT E1.LAST_NAME, COUNT(*) AS "TOTAL_MANAGED_EMPLOYEES"
FROM EMPLOYEE E1 JOIN E2
ON (E1.MANAGER_ID = E2.EMPLOYEE_ID)
GROUP BY E1.MANAGER_ID
ORDER BY "TOTAL_MANAGED_EMPLOYEES" ASC;
```

- (b) Write an SQL query to show the department id, job id, first hiring date, last hiring date and average salary of employees for each combination of department id and job id, where the average salary is more than 3000. Ensure no null values are printed and order the result by department id.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Fault Identification and Query Correction

Faults in the Original Query:

(a) **Missing Table Alias Definition:** In the FROM clause, JOIN E2 is syntactically invalid because E2 is just an alias, but the actual table name it refers to is missing. It should be JOIN employees E2.

(b) **Logical Error in the Self-Join Roles:** Based on the condition E1.MANAGER_ID = E2.EMPLOYEE_ID, E1 acts as the *subordinate* and E2 acts as the *manager*. However, the SELECT statement requests E1.LAST_NAME, which incorrectly fetches the subordinate's name instead of the manager's.

(c) **Violation of SQL Grouping Rules:** The query selects E1.LAST_NAME but groups by E1.MANAGER_ID. In standard SQL, any unaggregated column present in the SELECT clause must be explicitly included in the GROUP BY clause.

Corrected SQL Query: By designating E1 as the Manager and E2 as the Employee, the relationship becomes clearer and the grouping complies with SQL standards.

```
SELECT E1.last_name AS "MANAGER_NAME",
       COUNT(E2.employee_id) AS "TOTAL MANAGED EMPLOYEES"
FROM employees E1
JOIN employees E2
  ON (E1.employee_id = E2.manager_id)
GROUP BY E1.employee_id, E1.last_name
ORDER BY "TOTAL MANAGED EMPLOYEES" ASC;
```

Working Principle of the Self-Join:

Manager (Alias: E1)

employee_id (e.g., 100)

last_name (e.g., 'King')

Matches

Employee (Alias: E2)

employee_id (e.g., 101)

manager_id (e.g., 100)

E1.employee_id = E2.manager_id

2. Aggregation and Filtering Query

Explanation of Operations:

- **Pre-Aggregation Filtering (WHERE):** The IS NOT NULL conditions ensure that records missing a department_id or job_id are excluded before any grouping occurs.
- **Grouping (GROUP BY):** The data is bucketed into unique permutations of department and job ID.
- **Aggregate Functions:** MIN() and MAX() evaluate the earliest and most recent hire_date respectively, while AVG() computes the arithmetic mean of the salaries within the group.
- **Post-Aggregation Filtering (HAVING):** Checks the computed average salary of each group and discards the entire group if the average is not strictly greater than 3000.

SQL Query:

```
SELECT department_id,
       job_id,
       MIN(hire_date) AS first_hiring_date,
       MAX(hire_date) AS last_hiring_date,
       AVG(salary) AS average_salary
FROM employees
WHERE department_id IS NOT NULL
  AND job_id IS NOT NULL
GROUP BY department_id, job_id
HAVING AVG(salary) > 3000
ORDER BY department_id ASC;
```

Q14. Consider the following schemas:

Books(isbn, title, publisher_id, publish_year, price_per_copy, copies_sold)
Author(author_id, name, contact)
Publisher(publisher_id, name, address)
AuthorOfBook(book_id, author_id)

Answer the following queries in SQL:

- (a) Find all the books authored by “Ernest Hemingway”.
- (b) Find the revenue earned by each publisher between 1980 and 1990 by selling books.
- (c) Find the names of authors who have published at least 5 books since 1995.
- (d) Find the publisher which published books of most of the authors.
- (e) Find the names of all books which are sold in more copies than any other book published before 1990 by the same publisher.

(25 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

Relational Schema Visualization

Before formulating the SQL queries, it is helpful to visualize the database schema and the relationships between the tables. We assume that book_id in the AuthorOfBook table is a foreign key referencing isbn in the Books table.

```
graph TD
    AuthorOfBook["AuthorOfBook  
book_id (FK)  
author_id (FK)"]
    Author["Author  
author_id  
name  
contact"]
    Books["Books  
isbn  
title  
publisher_id (FK)  
publish_year  
price_per_copy  
copies_sold"]
    Publisher["Publisher  
publisher_id  
name  
address"]

    AuthorOfBook -.->|author_id = author_id| Author
    AuthorOfBook -.->|book_id = isbn| Books
    Books -.->|publisher_id = publisher_id| Publisher
```

1. Books authored by “Ernest Hemingway”

We join the Books, AuthorOfBook, and Author tables to filter by the author’s name and retrieve the corresponding book titles.

```
SELECT B.title
FROM Books B
JOIN AuthorOfBook AB ON B.isbn = AB.book_id
JOIN Author A ON AB.author_id = A.author_id
WHERE A.name = 'Ernest Hemingway';
```

2. Revenue earned by each publisher between 1980 and 1990

Revenue for a single book is calculated as the product of `price_per_copy` and `copies_sold`. We aggregate this sum per publisher for books published within the specified decade.

```
SELECT P.name AS publisher_name,
       SUM(B.price_per_copy * B.copies_sold) AS total_revenue
FROM Publisher P
JOIN Books B ON P.publisher_id = B.publisher_id
WHERE B.publish_year BETWEEN 1980 AND 1990
GROUP BY P.publisher_id, P.name;
```

3. Authors who have published at least 5 books since 1995

We filter for books published in or after 1995, group the records by author, and apply a `HAVING` clause to restrict the result to those with a count of 5 or more distinct books.

```
SELECT A.name
FROM Author A
JOIN AuthorOfBook AB ON A.author_id = AB.author_id
JOIN Books B ON AB.book_id = B.isbn
WHERE B.publish_year >= 1995
GROUP BY A.author_id, A.name
HAVING COUNT(B.isbn) >= 5;
```

4. Publisher which published books of most of the authors

To solve this, we must count the **distinct** authors associated with each publisher. We use a Common Table Expression (CTE) to compute this count, then select the publisher(s) matching the maximum count.

```
WITH PublisherAuthorCounts AS (
  SELECT B.publisher_id, COUNT(DISTINCT AB.author_id) AS distinct_authors
  FROM Books B
  JOIN AuthorOfBook AB ON B.isbn = AB.book_id
  GROUP BY B.publisher_id
)
SELECT P.name
FROM Publisher P
JOIN PublisherAuthorCounts PAC ON P.publisher_id = PAC.publisher_id
WHERE PAC.distinct_authors = (
  SELECT MAX(distinct_authors)
  FROM PublisherAuthorCounts
);
```

Note: This approach correctly handles ties if multiple publishers share the highest count of distinct authors.

5. Books sold in more copies than any other book published before 1990 by the same publisher

This requires a correlated subquery. We compare a book’s `copies_sold` against **ALL** the copies sold of other books from the same publisher published before 1990.

```
SELECT B1.title
FROM Books B1
WHERE B1.copies_sold > ALL (
  SELECT B2.copies_sold
  FROM Books B2
  WHERE B2.publisher_id = B1.publisher_id
        AND B2.publish_year < 1990
        AND B2.isbn <> B1.isbn
);
```

Explanation: The `> ALL` operator evaluates to `TRUE` if the left operand is strictly greater than every value returned by the subquery. The `B2.isbn <> B1.isbn` condition ensures a book published before 1990 is not compared against itself.

Q15. Given the relation schema of CSE department course database:

```

course(course_number, title, credit_hour, level, term, pre_req_course_number)
takes(course_number, student_id)
student(student_id, name, CGPA)

```

Write SQL statements to:

- Find the list of only those students who have taken all the courses of level 3 and term 1.
- Find the list of courses that are not taken by any student.
- Find the student id and the number of courses taken for those students with CGPA higher than 3.5.

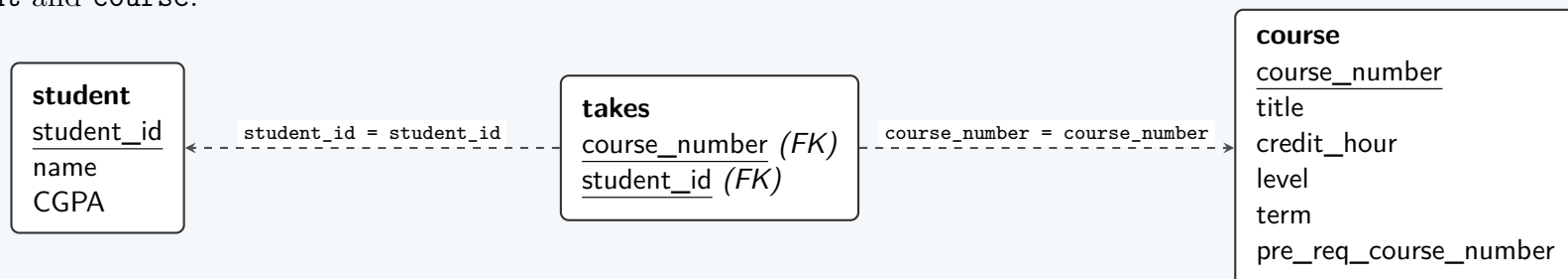
(15 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

Relational Schema Visualization

Before writing the queries, let us visualize the relationships between the tables. The **takes** relation acts as a junction table connecting student and course.



1. Students who have taken all the courses of level 3 and term 1

Explanation: This query requires a relational algebra **division** operation. Since standard SQL lacks a direct division operator, we achieve this using a double-nested NOT EXISTS pattern. The query logic translates to: “Find students for whom there does *not* exist a level 3, term 1 course that they have *not* taken.”

SQL Query:

```

SELECT S.student_id, S.name
FROM student S
WHERE NOT EXISTS (
    -- Set of all level 3, term 1 courses
    SELECT C.course_number
    FROM course C
    WHERE C.level = 3 AND C.term = 1

    EXCEPT

    -- Set of courses taken by the current student S
    SELECT T.course_number
    FROM takes T
    WHERE T.student_id = S.student_id
);

```

Note: MINUS can be used instead of EXCEPT if running on an Oracle database.

2. Courses that are not taken by any student

Explanation: This is an **anti-join** or set difference problem. We need to select courses from the **course** table that do not appear in the **course_number** column of the **takes** table. The NOT EXISTS clause provides an efficient and null-safe way to evaluate this condition.

SQL Query:

```

SELECT C.course_number, C.title
FROM course C
WHERE NOT EXISTS (
    SELECT 1
    FROM takes T
    WHERE T.course_number = C.course_number
);

```

3. Student ID and the number of courses taken for students with CGPA > 3.5

Explanation: To find the number of courses taken, we need to aggregate the records in the **takes** table. We apply a WHERE filter on the **student** table for the CGPA > 3.5 condition. We use a LEFT JOIN to ensure that if a student has a CGPA > 3.5 but has taken 0 courses, they will still appear in the result set with a count of 0 (rather than being excluded entirely).

SQL Query:


```
SELECT S.student_id, COUNT(T.course_number) AS number_of_courses_taken
FROM student S
LEFT JOIN takes T ON S.student_id = T.student_id
WHERE S.CGPA > 3.5
GROUP BY S.student_id;
```

Q16. Given the employee database:

```
Employee(employee_name, house_number, street, city)
Works(employee_name, company_name, salary)
Company(company_name, city)
Manages(employee_name, manager_name)
```

Write SQL DDL for the above employee database with the following constraints:

- A new employee name can be entered only in the employee table and cannot be duplicate.
- Manager name must be one of the employee names.
- City can only be Dhaka, Chittagong, Rajshahi or Khulna.
- No two employees can have the same house number, street and city.
- A company name cannot be duplicate and an employee cannot work in a company whose name is not present in the company table.

(15 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Mapping Constraints to SQL DDL

Before writing the SQL code, let us map the requested business rules to standard SQL constraints:

- Rule 1 (Unique Employee Names):** Addressed by setting `employee_name` as the **PRIMARY KEY** of the `Employee` table. Foreign keys in other tables ensure new employees can only originate here.
- Rule 2 (Manager is an Employee):** Addressed by a recursive **FOREIGN KEY** in the `Manages` table where `manager_name` references `Employee(employee_name)`.
- Rule 3 (Restricted Cities):** Addressed by a **CHECK** constraint on the `city` attribute restricting values to the four specified cities. We will apply this to both `Employee` and `Company` tables for consistency.
- Rule 4 (Unique Addresses):** Addressed by a composite **UNIQUE** constraint on (`house_number`, `street`, `city`) in the `Employee` table.
- Rule 5 (Valid Companies):** Addressed by setting `company_name` as the **PRIMARY KEY** of `Company` and creating a **FOREIGN KEY** in `Works` referencing it.

2. SQL Data Definition Language (DDL) Statements

To satisfy referential integrity (foreign keys must reference tables that already exist), the tables must be created in a specific order: parent tables first (`Employee`, `Company`), followed by dependent tables (`Works`, `Manages`).

```
-- 1. Create Employee Table
CREATE TABLE Employee (
    employee_name VARCHAR(100) PRIMARY KEY,
    house_number VARCHAR(50),
    street VARCHAR(100),
    city VARCHAR(50)
    CHECK (city IN ('Dhaka', 'Chittagong', 'Rajshahi', 'Khulna')),
    CONSTRAINT uk_employee_address UNIQUE (house_number, street, city)
);

-- 2. Create Company Table
CREATE TABLE Company (
    company_name VARCHAR(100) PRIMARY KEY,
    city VARCHAR(50)
    CHECK (city IN ('Dhaka', 'Chittagong', 'Rajshahi', 'Khulna'))
);

-- 3. Create Works Table
CREATE TABLE Works (
    employee_name VARCHAR(100),
    company_name VARCHAR(100),
    salary DECIMAL(10, 2),
    PRIMARY KEY (employee_name, company_name),
```

```

FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (company_name) REFERENCES Company(company_name)
ON DELETE CASCADE ON UPDATE CASCADE
);

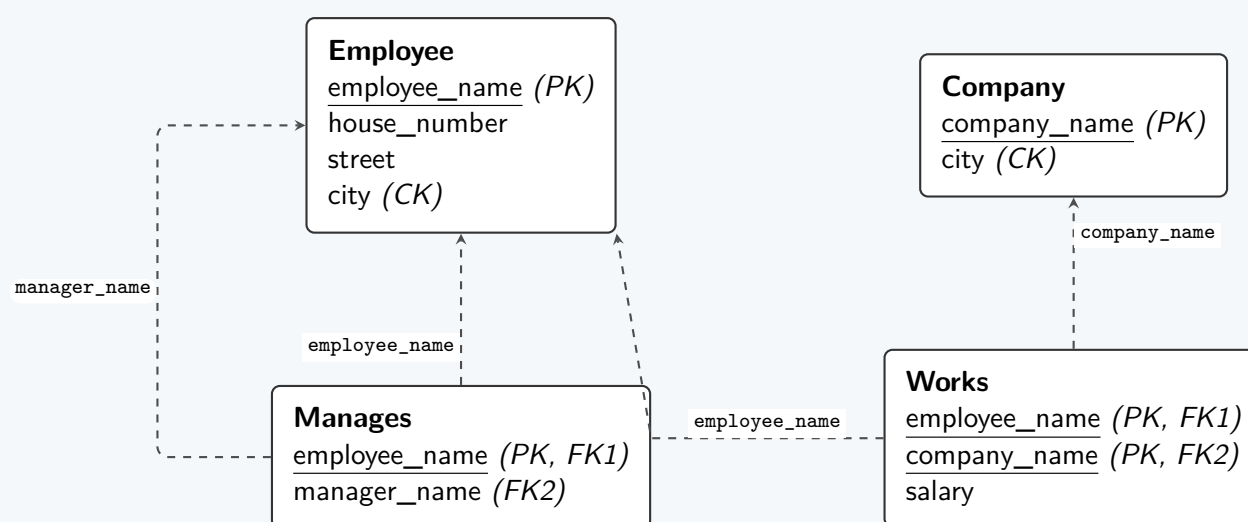
-- 4. Create Manages Table
CREATE TABLE Manages (
    employee_name VARCHAR(100) PRIMARY KEY,
    manager_name VARCHAR(100),
    FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (manager_name) REFERENCES Employee(employee_name)
ON DELETE SET NULL ON UPDATE CASCADE
);

```

Note: *ON DELETE* and *ON UPDATE* referential actions are standard practice to maintain integrity during modifications, though not strictly required by the prompt's basic constraints.

3. Relational Schema Visualization

The following diagram demonstrates the foreign key dependencies (referential integrity constraints) established by the DDL statements above.



Q17. Given the customer/account/transaction schema, a view is created:

```

customer(customer id, name, street, city)
account(account number, type, balance)
transaction(customer id, account number, date, amount)

```

```

CREATE VIEW cust_balance AS
SELECT a.customer_id, balance
FROM customer AS a, account AS b, transaction AS c
WHERE a.customer_id = c.customer_id
AND b.account_number = c.account_number;

```

Discuss the possible problems if insertion is allowed through the view `cust_balance`. (5 marks) [CSE 303 | L-3/T-1 | 2016–17]

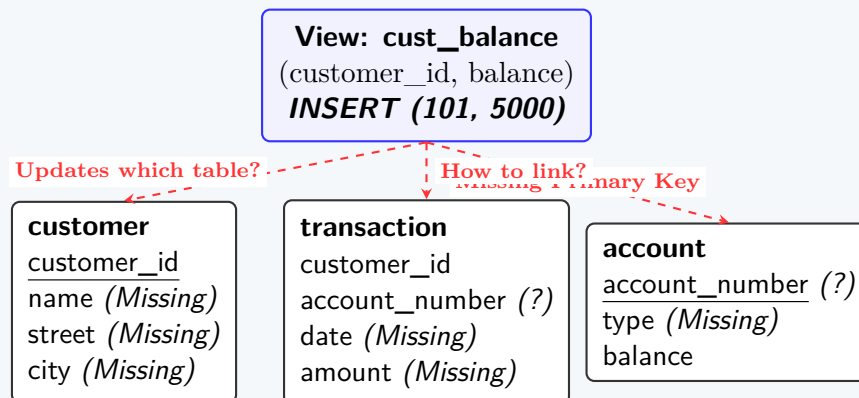
Answer

The View Update Problem

In relational database systems, a view is considered **updatable** (which includes insertions) only if the DBMS can unambiguously map the modification of the view back to the underlying base tables. The view `cust_balance` is a **complex view** because it is constructed using an equi-join across three base tables (`customer`, `account`, and `transaction`).

Allowing insertions through this specific view introduces several fundamental database anomalies and integrity violations, which are collectively known as the **View Update Problem**.

Diagram: Ambiguity in View Insertion



Specific Problems with Inserting via cust_balance

(a) **Ambiguity of Target Base Relations:**

An INSERT into cust_balance provides only two values: a customer_id and a balance. The DBMS cannot determine the user's true intent. Does the user want to:

- Create a new customer and a new account?
- Add a new account for an existing customer?
- Update the balance of an existing account?

Because the view joins three tables, the insertion mapping is fundamentally ambiguous.

(b) **Violation of Entity Integrity (Missing Primary Keys):**

To successfully insert a row that corresponds to this view, records must exist in customer, account, and transaction. However, the view **does not expose account_number**, which is the primary key for the account table and part of the required linking data in transaction. A primary key cannot be NULL. Therefore, the DBMS cannot create a new account record.

(c) **Violation of Domain Constraints (Missing NOT NULL Attributes):**

Even if the DBMS could generate surrogate keys, the customer table requires attributes like name, street, and city. The transaction table requires date and amount. Because these columns are not included in the view, they would have to be assigned NULL values during insertion. If any of these underlying columns have NOT NULL constraints (which is standard for basic entity properties), the insertion will inherently fail.

(d) **The Phantom Row (Visibility) Problem:**

For a row to appear in cust_balance, it must satisfy the join condition: a.customer_id = c.customer_id AND b.account_number = c.account_number. This means a linking record in the transaction table *must* exist. If the DBMS attempts to insert data only into customer and account, the inner join will fail to retrieve it. The user would perform an INSERT, but a subsequent SELECT * FROM cust_balance would not show the newly inserted data.

Conclusion

Because it violates key-preservation rules and lacks mandatory base-table columns, the SQL standard dictates that complex join views like cust_balance are strictly **read-only**. Any attempt to execute an INSERT statement against this view will be universally rejected by standard relational DBMS engines (e.g., PostgreSQL, MySQL, Oracle) with an "object not updatable" error.

Q18. Consider the following relational database schema:

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

The Product relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. We assume for convenience that model numbers are unique over all manufacturers and product types; that assumption is not realistic, and a real database would include a code for the manufacturer as part of the model number. The PC relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The Laptop relation is similar, except that the screen size (in inches) is also included. The Printer relation records for each printer model whether the printer produces color output ('T', if so; 'F' otherwise), the process type (laser or ink-jet, typically), and the price. Write the following SQL queries:

- Find those manufacturers that sell Laptops, but not PCs or Printers.
- Find those hard disk sizes that occur in two or more PCs.
- Find those pairs of PC models that have both the same speed and RAM. A pair should be listed only once.
- Find the manufacturers of PCs with at least three different speeds.
- Find the makers who produce only one type of product.
- Find the maker(s) of the cheapest color printer(s).
- Find the PC model with maximum processing speed without using any aggregate function.

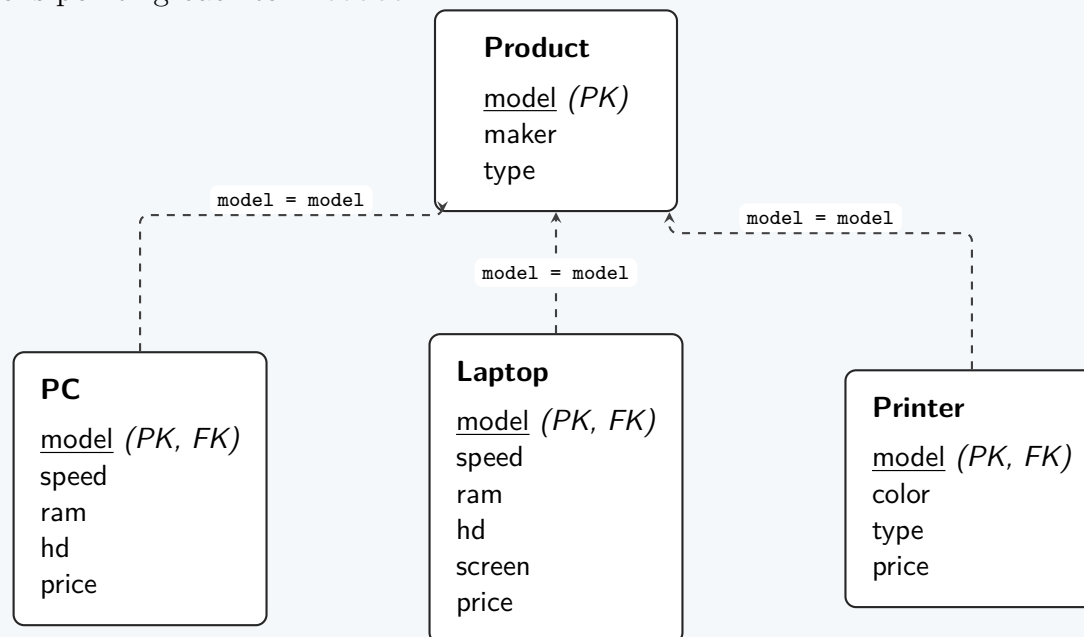
(28 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Relational Schema Visualization (IS-A Hierarchy)

The given schema models a textbook generalization/specialization hierarchy where **Product** acts as the superclass entity and **PC**, **Laptop**, and **Printer** act as subclass entities. The **model** attribute serves as the primary key across all relations and as a foreign key in the subclass relations pointing back to **Product**.



SQL Queries

1. Manufacturers selling Laptops, but not PCs or Printers

This is a set difference operation. We find all makers of laptops and subtract the set of makers who produce either PCs or Printers.

```

SELECT maker
FROM Product
WHERE type = 'laptop'
EXCEPT
SELECT maker
FROM Product
WHERE type IN ('PC', 'printer');
  
```

2. Hard disk sizes occurring in two or more PCs

We group the PC table by the hard disk size (**hd**) and apply a **HAVING** clause to filter groups that contain at least two records.

```

SELECT hd
FROM PC
GROUP BY hd
HAVING COUNT(model) >= 2;
  
```

3. Pairs of PC models with the same speed and RAM

A self-join is required to compare records within the same table. The condition **P1.model < P2.model** ensures that a pair is only listed once and prevents a PC from being paired with itself.

```

SELECT P1.model AS model_1, P2.model AS model_2
FROM PC P1
JOIN PC P2 ON P1.speed = P2.speed AND P1.ram = P2.ram
WHERE P1.model < P2.model;
  
```

4. Manufacturers of PCs with at least three different speeds

We join **Product** and **PC**, group by the manufacturer, and count the distinct speeds. **COUNT(DISTINCT speed)** is crucial here because a maker might sell multiple PCs with the *same* speed.

```

SELECT P.maker
FROM Product P
JOIN PC ON P.model = PC.model
GROUP BY P.maker
HAVING COUNT(DISTINCT PC.speed) >= 3;
  
```

5. Makers who produce only one type of product

By grouping the **Product** relation by **maker**, we can count the distinct product types associated with each manufacturer.

```
SELECT maker
FROM Product
GROUP BY maker
HAVING COUNT(DISTINCT type) = 1;
```

6. Maker(s) of the cheapest color printer(s)

We first find the minimum price among color printers using a subquery, and then join `Product` and `Printer` to retrieve the maker(s) matching that minimum price.

```
SELECT P.maker
FROM Product P
JOIN Printer Pr ON P.model = Pr.model
WHERE Pr.color = 'T'
      AND Pr.price = (
        SELECT MIN(price)
        FROM Printer
        WHERE color = 'T'
      );
```

7. PC model with maximum processing speed (without aggregate functions)

Since aggregate functions like `MAX()` are forbidden, we can use the `NOT EXISTS` operator. We select a PC model for which there does *not exist* any other PC model with a strictly greater speed.

```
SELECT P1.model
FROM PC P1
WHERE NOT EXISTS (
  SELECT 1
  FROM PC P2
  WHERE P2.speed > P1.speed
);
```

Note: An alternative valid approach is using the `>= ALL` operator: `SELECT model FROM PC WHERE speed >= ALL (SELECT speed FROM PC);`

Q19. Consider three tables *T1*, *T2* and *T3*, each with a single integer column:

T1	T2	T3
1,2,2,2,3,4,4	2,3,4,4,4,5	2,2,3,3,4,4,5,5

Show the result of the following SQL queries:

- (a) `SELECT * FROM T1 UNION SELECT * FROM T2`
- (b) `SELECT * FROM T1 MINUS SELECT * FROM T3`
- (c) `(SELECT * FROM T1 UNION ALL SELECT * FROM T2) INTERSECT ALL (SELECT * FROM T3)`

(7 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Initial Multiset Analysis

Before evaluating the SQL queries, it is useful to represent the tables as multisets (bags) with their respective element frequencies:

- T1:** {1 : 1, 2 : 3, 3 : 1, 4 : 2} (i.e., one 1, three 2s, one 3, two 4s)
- T2:** {2 : 1, 3 : 1, 4 : 3, 5 : 1}
- T3:** {2 : 2, 3 : 2, 4 : 2, 5 : 2}

1. `SELECT * FROM T1 UNION SELECT * FROM T2`

Working Principle: In SQL, the `UNION` operator (without the `ALL` keyword) performs a standard set union. It combines the result sets of both queries and **removes all duplicates**.

- Distinct elements of **T1** = {1, 2, 3, 4}
- Distinct elements of **T2** = {2, 3, 4, 5}
- {1, 2, 3, 4} $\cup$ {2, 3, 4, 5} = {1, 2, 3, 4, 5}

Result:

Result
1
2
3
4
5

2. SELECT * FROM T1 MINUS SELECT * FROM T3

Working Principle: The MINUS operator (equivalent to EXCEPT in standard SQL) performs a set difference. Like UNION, it implicitly removes duplicates from the first result set, and then removes any rows that are also present in the second result set.

- Distinct elements of **T1** = {1, 2, 3, 4}
- Distinct elements of **T3** = {2, 3, 4, 5}
- $\{1, 2, 3, 4\} \setminus \{2, 3, 4, 5\} = \{1\}$

Result:

Result
1

3. (SELECT * FROM T1 UNION ALL SELECT * FROM T2) INTERSECT ALL (SELECT * FROM T3)

Working Principle: This query uses multiset operations (ALL keyword preserves duplicates).

(a) Left Operand: T1 UNION ALL T2

UNION ALL adds the frequencies of identical elements from both multisets.

- Count of 1: $1 + 0 = 1$
- Count of 2: $3 + 1 = 4$
- Count of 3: $1 + 1 = 2$
- Count of 4: $2 + 3 = 5$
- Count of 5: $0 + 1 = 1$

Left Result (Multiset A): {1, 2, 2, 2, 2, 3, 3, 4, 4, 4, 4, 4, 5}

(b) Right Operand: T3

Multiset B: {2, 2, 3, 3, 4, 4, 5, 5} (Counts: 2 : 2, 3 : 2, 4 : 2, 5 : 2)

(c) Intersection: Multiset A INTERSECT ALL Multiset B

INTERSECT ALL takes the **minimum** frequency of each element present in both multisets.

- Count of 1: $\min(1, 0) = 0$
- Count of 2: $\min(4, 2) = 2$
- Count of 3: $\min(2, 2) = 2$
- Count of 4: $\min(5, 2) = 2$
- Count of 5: $\min(1, 2) = 1$

Result:

Result
2
2
3
3
4
4
5

Q20. Consider the following schema:

```
Department(name, headID, yearOfEstablishment)
Employee(id, name, address, designation, salary, gender)
```

- (a) Enforce the rule “Departmental head must be an employee of the university” — show two ways: one using a CHECK constraint

and one without any check constraint.

- (b) Write a **CHECK** constraint to make sure that no two departments have the same person as Head.
- (c) Write an assertion to check: if the Head of the Department's gender is male then his name must not begin with 'Ms'.
- (d) Write an assertion to check: the CSE Departmental head's salary must not be less than 1,50,000 Taka.

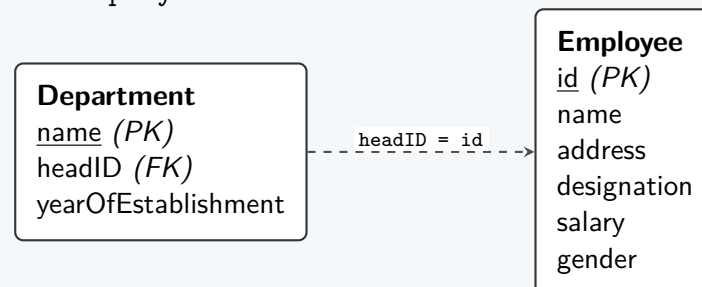
(16 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Relational Schema Mapping

Before defining the constraints, the following diagram illustrates the relationship between the **Department** and **Employee** tables, where the **headID** acts as a reference to the **Employee** table.



1. Enforcing “Departmental head must be an employee”

This rule mandates referential integrity between the **Department** and **Employee** tables.

Method A: Without using a **CHECK** constraint (Standard Practice)

The most efficient and standard way to enforce this is by defining **headID** as a **Foreign Key** that references the primary key of the **Employee** table.

```
ALTER TABLE Department
ADD CONSTRAINT fk_head_employee
FOREIGN KEY (headID) REFERENCES Employee(id);
```

Method B: Using a **CHECK** constraint

Standard SQL allows subqueries within **CHECK** constraints (though some specific RDBMS implementations do not support this). We can enforce the rule by checking if the **headID** exists in the **Employee** table.

```
ALTER TABLE Department
ADD CONSTRAINT chk_head_exists
CHECK (headID IN (SELECT id FROM Employee));
```

2. **CHECK** constraint for unique Department Heads

The requirement dictates that no two departments can share the same **headID**. While the standard practical approach is to use the **UNIQUE(headID)** constraint, the question explicitly requests a **CHECK** constraint. We achieve this using a table-level **CHECK** constraint containing a subquery to count occurrences.

```
ALTER TABLE Department
ADD CONSTRAINT chk_unique_head
CHECK (
    (SELECT COUNT(*)
     FROM Department D
     WHERE D.headID = headID) <= 1
);
```

Note: In standard relational database design, `ALTER TABLE Department ADD CONSTRAINT uq_head UNIQUE(headID);` is the preferred method for this requirement.

3. Assertion: Male Head of Department's name must not begin with 'Ms'

An **assertion** evaluates a condition across the entire database. If the condition evaluates to **FALSE**, the database rejects the modification. The best practice for assertions is to use **NOT EXISTS** to ensure violating rows cannot exist.

```
CREATE ASSERTION assert_male_head_name
CHECK (
    NOT EXISTS (
        SELECT 1
        FROM Department D
        JOIN Employee E ON D.headID = E.id
        WHERE E.gender = 'Male'
              AND E.name LIKE 'Ms%'
    )
);
```

*Explanation: The assertion searches for any male department head whose name starts with "Ms". If such a record is found, **EXISTS** evaluates to true, **NOT EXISTS** becomes false, and the transaction is aborted.*

4. Assertion: CSE Departmental head's salary must be $\geq 1,50,000$ Taka

Similar to the previous assertion, we combine the **Department** and **Employee** tables and check for any violation of the business rule (i.e., we check if there exists a CSE head earning *less* than 1,50,000).

```
CREATE ASSERTION assert_cse_head_salary
CHECK (
    NOT EXISTS (
        SELECT 1
        FROM Department D
        JOIN Employee E ON D.headID = E.id
        WHERE D.name = 'CSE'
            AND E.salary < 150000
    )
);
```

Explanation: The constraint enforces that no query will ever return a record where the CSE department head makes less than 1,50,000. If an update or insert attempts to create such a state, the assertion fails and prevents the change.

Q21. Answer the following:

- (a) Replace the NOT NULL constraint using a CHECK constraint:

```
CREATE TABLE MyTable (
    ....
    gender CHAR(1) NOT NULL,
    ....
);
```

- (b) Replace the UNIQUE constraint using a CHECK constraint:

```
CREATE TABLE MyTable (
    .....
    headID INT UNIQUE
    .....
);
```

- (c) What are the three different policies for UPDATE and DELETE for maintaining referential integrity? Which is more suitable for UPDATE and which for DELETE? Why?
- (d) Show the DDL to create a copy of a table *R* including the schema. What is the DDL if we just want to copy the schema and not the tuples?

(19 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Replacing NOT NULL with a CHECK Constraint

The NOT NULL constraint is essentially a shorthand for a CHECK constraint that explicitly evaluates whether the attribute IS NOT NULL.

```
CREATE TABLE MyTable (
    -- ...
    gender CHAR(1),
    CONSTRAINT chk_gender_not_null CHECK (gender IS NOT NULL)
    -- ...
);
```

2. Replacing UNIQUE with a CHECK Constraint

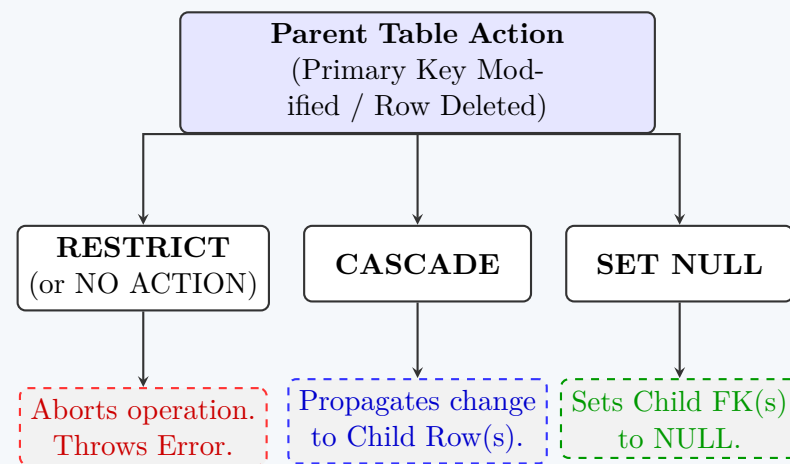
Standard SQL academically allows subqueries inside a CHECK constraint to enforce table-level conditions. To simulate a UNIQUE constraint, we can count the occurrences of the specific value; if the count exceeds 1, the constraint fails.

```
CREATE TABLE MyTable (
    -- ...
    headID INT,
    CONSTRAINT chk_headID_unique CHECK (
        (SELECT COUNT(*) FROM MyTable m WHERE m.headID = headID) <= 1
    )
    -- ...
);
```

*Note: While valid in academic SQL-92 standards, most modern relational DBMS implementations (like PostgreSQL or SQL Server) disable subqueries within **CHECK** constraints for performance reasons, preferring the standard **UNIQUE** index.*

3. Referential Integrity Policies for UPDATE and DELETE

When a referenced (parent) row is updated or deleted, the database must decide what to do with the referencing (child) rows to maintain referential integrity.



The Three Policies:

- **CASCADE:** Automatically updates or deletes the corresponding foreign key records in the child table to match the parent table.
- **SET NULL:** When the parent record is modified or deleted, the corresponding foreign key fields in the child table are set to NULL (provided the column allows NULLs).
- **RESTRICT / NO ACTION:** Prevents the UPDATE or DELETE from occurring on the parent table if any dependent child rows exist. The operation is aborted.

Suitability Analysis:

- **For UPDATE: CASCADE is highly suitable.** If a parent's identifier (Primary Key) simply changes form or gets corrected (e.g., updating a Department ID from 'CS' to 'CSE'), the conceptual entity remains the same. The child records should seamlessly follow this change to remain linked to the correct entity.
- **For DELETE: RESTRICT or SET NULL is more suitable.** Using CASCADE on DELETE is dangerous because deleting a single parent record (like a Customer) might accidentally wipe out thousands of child records (like Invoices or Orders), causing severe data loss. RESTRICT protects critical historical data, while SET NULL safely archives child data as an "orphan" without losing the record itself.

4. Copying a Table Schema (DDL)

We can create copies of an existing table *R* using the CREATE TABLE AS SELECT paradigm.

Copying the table schema INCLUDING the tuples (Data):

```
CREATE TABLE R_Copy AS
SELECT * FROM R;
```

Copying ONLY the schema (NO tuples):

To copy the structure without importing any data, we use a WHERE clause that universally evaluates to FALSE (such as $1 = 0$). This forces the database to return an empty set, but the structural metadata of the columns is still extracted and used to create the new table.

```
CREATE TABLE R_Schema_Only AS
SELECT * FROM R
WHERE 1 = 0;
```

Note: In some modern dialects like PostgreSQL, `CREATE TABLE R_Schema_Only (LIKE R INCLUDING ALL);` is also a highly effective alternative that copies constraints and indexes as well.

Q22. Consider the following schema:

```
Flights(flno: integer, from: string, to: string, distance: integer, departs: integer, arrives: time, price: real)
Aircraft(aid: integer, aname: string, cruisingrange: integer)
Certified(eid: integer, aid: integer)
Employees(eid: integer, ename: string, salary: integer)
```

Write SQL to implement the following queries:

- Find the names of aircraft such that all pilots certified to operate them have salaries more than \$80,000.
- For each pilot certified for more than three aircraft, find the eid and the maximum cruising range of the aircraft for which she

or he is certified.

- (c) Find the names of pilots whose salary is less than the price of the cheapest route from ‘Los Angeles’ to ‘Honolulu’.
- (d) Find the aids of all aircraft that can be used on routes from ‘Los Angeles’ to ‘Chicago’.
- (e) Identify the routes that can be piloted by every pilot who makes more than \$100,000.

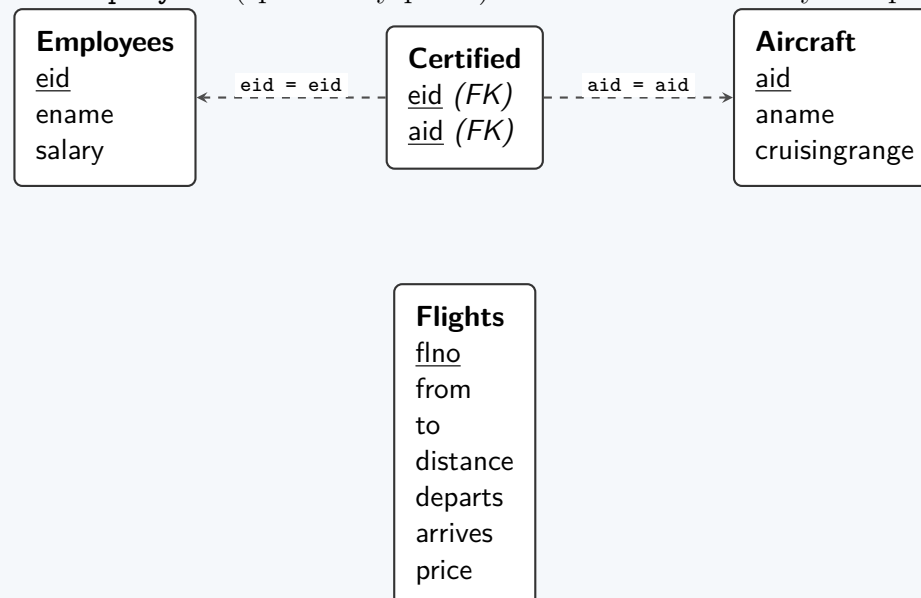
(25 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

Relational Schema Visualization

The diagram below maps out the foreign key connections across the airline database schema. The **Certified** relation serves as a bridge (junction table) between the **Employees** (specifically pilots) and the **Aircraft** they are qualified to fly.



SQL Queries

1. Aircraft where ALL certified pilots earn more than \$80,000

We can solve this by finding the aircraft for which there *does not exist* a certified pilot whose salary is less than or equal to \$80,000. We also must ensure that the aircraft has at least one certified pilot.

```

SELECT A.aname
FROM Aircraft A
JOIN Certified C ON A.aid = C.aid
WHERE NOT EXISTS (
    SELECT 1
    FROM Employees E
    JOIN Certified C2 ON E.eid = C2.eid
    WHERE C2.aid = A.aid AND E.salary <= 80000
)
GROUP BY A.aid, A.aname;
  
```

2. Max cruising range for pilots certified for more than 3 aircraft

We join **Certified** with **Aircraft**, group by the pilot's **eid**, filter using **HAVING** for the count constraint, and select the maximum range.

```

SELECT C.eid, MAX(A.cruisingrange) AS max_range
FROM Certified C
JOIN Aircraft A ON C.aid = A.aid
GROUP BY C.eid
HAVING COUNT(C.aid) > 3;
  
```

3. Pilots earning less than the cheapest route from Los Angeles to Honolulu

We use a scalar subquery to determine the minimum price of the specified flight path, and then extract the pilots whose salary falls below that benchmark.

```

SELECT E.ename
FROM Employees E
WHERE E.salary < (
    SELECT MIN(F.price)
    FROM Flights F
    WHERE F.from = 'Los Angeles' AND F.to = 'Honolulu'
);
  
```

4. Aircraft that can cover the distance from Los Angeles to Chicago

An aircraft can fulfill a route if its `cruisingrange` is greater than or equal to the route's travel distance.

```
SELECT DISTINCT A.aid
FROM Aircraft A
WHERE A.cruisingrange >= (
    SELECT MIN(F.distance)
    FROM Flights F
    WHERE F.from = 'Los Angeles' AND F.to = 'Chicago'
);
```

5. Routes that can be piloted by EVERY pilot making more than \$100,000

This requires a relational division strategy. We find flight routes where *there is no high-earning pilot* who lacks an aircraft certification capable of matching or exceeding that flight's distance requirement.

```
SELECT F.from, F.to
FROM Flights F
WHERE NOT EXISTS (
    -- Get all pilots making > $100,000
    SELECT E.eid
    FROM Employees E
    WHERE E.salary > 100000

    EXCEPT

    -- Get pilots capable of flying this specific flight's distance
    SELECT C.eid
    FROM Certified C
    JOIN Aircraft A ON C.aid = A.aid
    WHERE A.cruisingrange >= F.distance
);
```

Q23. Why are roles used for assigning privileges to users? Create three users U_1 , U_2 and U_3 and assign `INSERT` and `UPDATE` privileges to them on tables $T1$ and $T2$ respectively. Do this by assigning the users a role of “Admin”. Then transfer these privileges to another role named “SuperAdmin”. Finally, revoke `UPDATE` privilege from “Admin”. (13 marks) [CSE 303 | L-3/T-1 | 2014–15]

Answer

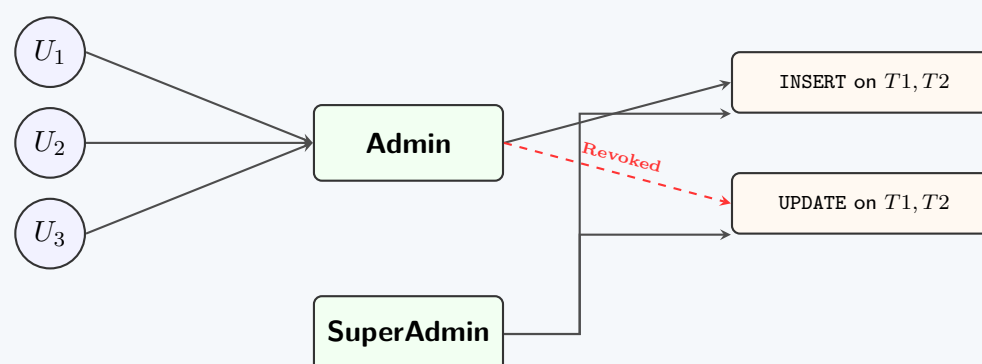
1. Why Roles are Used for Assigning Privileges

In Database Management Systems, a **Role** is a named collection of related privileges that can be granted to users or other roles. This paradigm is known as **Role-Based Access Control (RBAC)**. Roles are fundamentally used to simplify security administration for the following reasons:

- **Simplified Management:** Instead of granting individual privileges to each user (e.g., 100 users needing 10 identical privileges = 1000 operations), a DBA groups the privileges into a single role and assigns the role to the users.
- **Dynamic Updates:** If the access requirements for a job function change, the DBA only needs to modify the role's privileges. The changes automatically and immediately propagate to all users holding that role.
- **Logical Abstraction:** Roles map directly to real-world job functions or titles (e.g., `DataAnalyst`, `HR_Manager`, `Clerk`), making security policies much easier to understand and audit.
- **Reduced Errors:** Managing fewer security endpoints (roles instead of individual user lists) minimizes the risk of accidentally granting excessive permissions or omitting required ones.

2. Role-Based Access Control (RBAC) Visualization

The following diagram illustrates the relationship between the users, the defined roles, and the database privileges as requested in the scenario.



3. SQL Implementation

The following Data Control Language (DCL) statements implement the requested sequence of operations.

```
-- 1. Create the three users
CREATE USER U1;
CREATE USER U2;
CREATE USER U3;

-- 2. Create the 'Admin' role
CREATE ROLE Admin;

-- 3. Assign INSERT and UPDATE privileges on T1 and T2 to 'Admin'
GRANT INSERT, UPDATE ON T1 TO Admin;
GRANT INSERT, UPDATE ON T2 TO Admin;

-- 4. Assign the 'Admin' role to the users
GRANT Admin TO U1, U2, U3;

-- 5. Create the 'SuperAdmin' role
CREATE ROLE SuperAdmin;

-- 6. Transfer privileges to 'SuperAdmin'
-- (Explicitly granting the same privileges to the new role)
GRANT INSERT, UPDATE ON T1 TO SuperAdmin;
GRANT INSERT, UPDATE ON T2 TO SuperAdmin;

-- 7. Revoke the UPDATE privilege from 'Admin'
REVOKE UPDATE ON T1 FROM Admin;
REVOKE UPDATE ON T2 FROM Admin;
```

Note: Syntax for creating users (*CREATE USER*) may vary slightly depending on the specific DBMS (e.g., Oracle requires *IDENTIFIED BY password*), but the standard SQL ANSI approach is shown above. Step 6 demonstrates explicitly copying privileges; alternatively, some architectures allow roles to inherit other roles via *GRANT Admin TO SuperAdmin;*, though explicitly mirroring privileges ensures that revoking *UPDATE* from *Admin* in Step 7 does not accidentally cascade and strip it from *SuperAdmin*.

Q24. Consider the following schema:

Patient(ID, FIRSTNAME, SURNAME, ADMISSION_DATE, DR_ID, ward_no)
AILMENT(AILMENTID, NAME, PATIENTID)
Ward(ward_no, ward_name)
Doctor(ID, SURNAME, FIRSTNAME, no_of_patient, AILMENTID)

Write SQL to implement the following queries:

- (a) List the surnames of all patients of ‘Dr. Jones’.
- (b) List all the doctors who specialise in the ailment suffered by the patient whose surname is ‘Thomas’.
- (c) List the ward number and ward name of the ward(s) which have the most patients.
- (d) Provide a list of all patients who have not suffered any ailment.
- (e) Show the Doctor’s name who has the third highest no_of_patient.

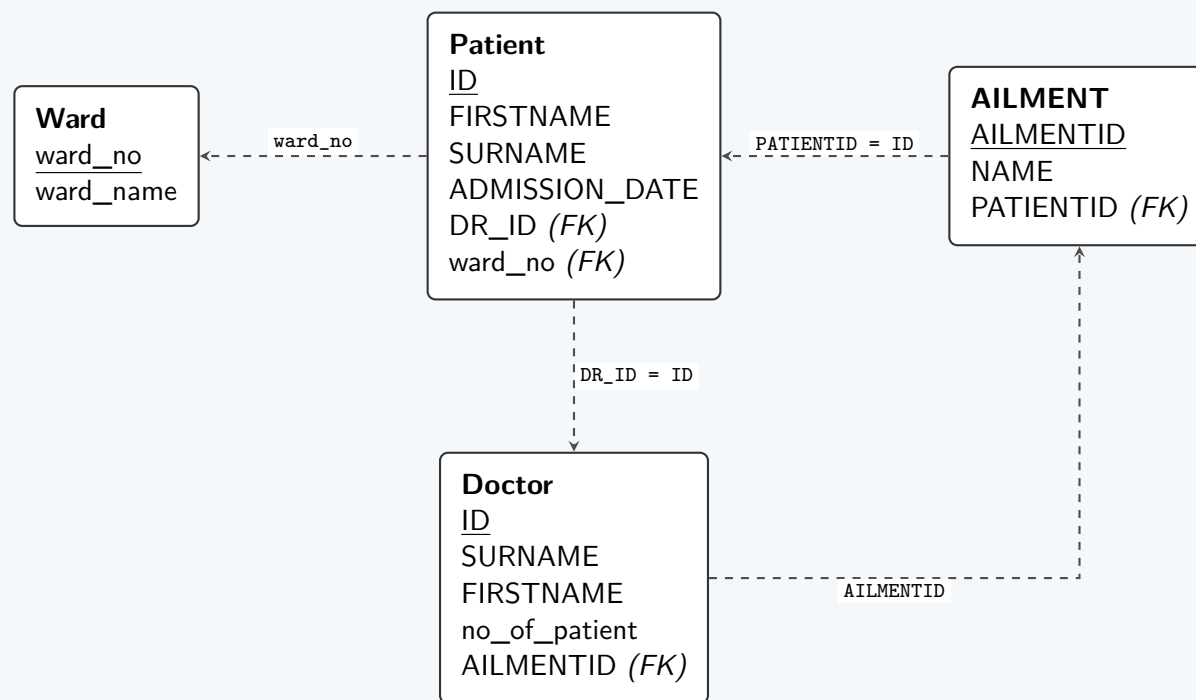
(25 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

Relational Schema Visualization

To accurately construct the SQL queries, we first map the foreign key dependencies between the relations. The arrows indicate the reference from a Foreign Key (FK) to a Primary Key (PK).



SQL Queries

1. List the surnames of all patients of 'Dr. Jones'

Explanation: This requires an equi-join between the **Patient** and **Doctor** tables using the doctor's identifier. We filter the result by the doctor's surname.

```
SELECT P.SURNAME
FROM Patient P
JOIN Doctor D ON P.DR_ID = D.ID
WHERE D.SURNAME = 'Jones';
```

2. List all the doctors who specialise in the ailment suffered by the patient whose surname is 'Thomas'

Explanation: This requires joining three tables. We start from the **Patient** table to find 'Thomas', link to the **AILMENT** table to find their specific ailment, and finally join with the **Doctor** table to find doctors specializing in that **AILMENTID**.

```
SELECT D.FIRSTNAME, D.SURNAME
FROM Doctor D
JOIN AILMENT A ON D.AILMENTID = A.AILMENTID
JOIN Patient P ON A.PATIENTID = P.ID
WHERE P.SURNAME = 'Thomas';
```

3. List the ward number and ward name of the ward(s) which have the most patients

Explanation: We calculate the count of patients per ward using `GROUP BY`. To identify the maximum count (handling potential ties for the top spot), we use a `HAVING` clause with the `>= ALL` operator comparing against all other ward counts.

```
SELECT W.ward_no, W.ward_name
FROM Ward W
JOIN Patient P ON W.ward_no = P.ward_no
GROUP BY W.ward_no, W.ward_name
HAVING COUNT(P.ID) >= ALL (
    SELECT COUNT(ID)
    FROM Patient
    GROUP BY ward_no
);
```

4. Provide a list of all patients who have not suffered any ailment

Explanation: This is a standard anti-join problem. The most robust and efficient way to express this in SQL is by using the `NOT EXISTS` operator, which searches for the absence of a related record in the **AILMENT** table.

```
SELECT P.ID, P.FIRSTNAME, P.SURNAME
FROM Patient P
WHERE NOT EXISTS (
    SELECT 1
    FROM AILMENT A
    WHERE A.PATIENTID = P.ID
);
```

5. Show the Doctor’s name who has the third highest no_of_patient

Explanation: To find the N -th highest value without relying on dialect-specific limits (like LIMIT or TOP), we use an ANSI-standard correlated subquery. The third highest count exists where exactly two distinct no_of_patient values are strictly greater than the current doctor’s value.

```
SELECT FIRSTNAME, SURNAME
FROM Doctor D1
WHERE 2 = (
    SELECT COUNT(DISTINCT D2.no_of_patient)
    FROM Doctor D2
    WHERE D2.no_of_patient > D1.no_of_patient
);
```

Note: In modern SQL implementations supporting the SQL:2008 standard, this can also be achieved efficiently using: ORDER BY no_of_patient DESC OFFSET 2 ROWS FETCH NEXT 1 ROWS ONLY;

Q25. Create three users U_1 , U_2 and U_3 and assign them SELECT and DELETE privileges respectively on $T1$ and $T2$ tables. Do this by assigning the users a role of “Admin”. Then transfer these privileges to another role named “SuperAdmin”. (8 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. SQL Implementation

The requested operations utilize **Role-Based Access Control (RBAC)** mechanisms standard in modern Relational Database Management Systems (such as Oracle, PostgreSQL, and MySQL).

Below is the complete sequence of SQL statements to achieve the requirements:

```
-- Step 1: Create the three users
CREATE USER U1 IDENTIFIED BY 'password_1';
CREATE USER U2 IDENTIFIED BY 'password_2';
CREATE USER U3 IDENTIFIED BY 'password_3';

-- Step 2: Create the 'Admin' role
CREATE ROLE Admin;

-- Step 3: Assign SELECT and DELETE privileges respectively on T1 and T2 to the Admin role
GRANT SELECT ON T1 TO Admin;
GRANT DELETE ON T2 TO Admin;

-- Step 4: Assign the 'Admin' role to the users
GRANT Admin TO U1, U2, U3;

-- Step 5: Create the 'SuperAdmin' role
CREATE ROLE SuperAdmin;

-- Step 6: Transfer/Delegate privileges to 'SuperAdmin'
-- Method A (Role Hierarchy): Grant the Admin role itself to SuperAdmin
GRANT Admin TO SuperAdmin;

-- Method B (Explicit Assignment): Alternatively, grant the privileges directly
-- GRANT SELECT ON T1 TO SuperAdmin;
-- GRANT DELETE ON T2 TO SuperAdmin;
```

2. Privilege Hierarchy Diagram

The following Entity-Relationship-style diagram illustrates the privilege flow and role hierarchy established by the SQL commands. It demonstrates how users inherit table-level privileges through their assigned roles.

The diagram shows three users (U_1 , U_2 , U_3) at the top, each with a circle containing its label. Arrows labeled "Assigned to" point from each user to a central role box labeled "Admin". To the left of the "Admin" box is another role box labeled "SuperAdmin", connected by a dashed arrow labeled "Inherits from". Below the "Admin" box, two arrows point to table boxes labeled "Table: T1" and "Table: T2". The arrow to T1 is labeled "GRANT SELECT" and the arrow to T2 is labeled "GRANT DELETE".

3. Conceptual Explanation

- **User Creation:** The `CREATE USER` command establishes new authentication profiles in the database catalog. Passwords (e.g., `IDENTIFIED BY`) are strictly required in most production DBMS environments.
- **Role Creation:** A **Role** (`Admin`) acts as a named container for privileges. Rather than granting privileges directly to U_1 , U_2 , and U_3 individually (which is redundant and hard to maintain), privileges are grouped into the role.
- **Privilege Assignment (Respectively):** Based on the problem statement, the `SELECT` privilege is mapped to table T_1 , and the `DELETE` privilege is mapped to table T_2 .
- **Role Delegation:** Executing `GRANT Admin TO U1, U2, U3`; binds the privilege container to the users. Whenever the `Admin` role is updated, the users automatically inherit the changes.
- **Privilege Transfer (Role Hierarchy):** To "transfer" or copy these privileges to `SuperAdmin`, the most efficient and scalable standard practice in RBAC is to grant the `Admin` role to the `SuperAdmin` role. This creates a **Role Hierarchy** where `SuperAdmin` implicitly inherits all privileges granted to `Admin`.

4. Advantages of Role-Based Access Control (RBAC)

Property	Description
Simplified Administration	Database Administrators (DBAs) can modify privileges for a single role instead of executing redundant <code>GRANT/REVOKE</code> statements for hundreds of users.
Hierarchical Modeling	Roles can be nested (e.g., <code>Admin</code> $\rightarrow$ <code>SuperAdmin</code>) to accurately reflect organizational charts and logical access tiers.
Reduced Error Surface	Centralizing privileges significantly prevents accidental over-provisioning of rights (the Principle of Least Privilege).
Dynamic Inheritance	If <code>Admin</code> is later granted <code>UPDATE</code> on T_3 , <code>SuperAdmin</code> , U_1 , U_2 , and U_3 instantly inherit this access without additional commands.

Q26. How do you insert a tuple without causing a constraint violation in the following table?
Person(id, name, father) where foreign key father references Person. (9 marks)

[CSE 303 | L-3/T-1 | 2013–14]

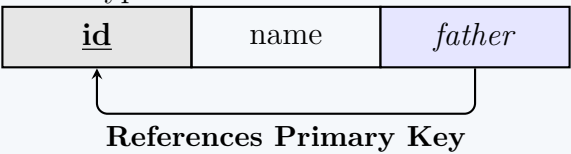
Answer

Handling Insertions with Recursive Foreign Keys

The given schema, `Person(id, name, father)`, contains a **recursive (or self-referencing) foreign key constraint**. The `father` attribute references the primary key `id` of the *same* table.

This creates an insertion anomaly: to insert a person, their father must already exist in the table. If the table is empty, or if there is a cyclic dependency, standard insertion will cause a **foreign key constraint violation**.

There are several standard database techniques to bypass this violation and successfully insert the tuples.



Method 1: Using NULL Values (Top-Down Approach)

If the `father` column allows NULL values (i.e., it is not constrained by `NOT NULL`), you can insert the "root" of the hierarchy by setting their `father` to NULL. Subsequent descendants can then safely reference this root node's `id`.

```
-- 1. Insert the root record (no father)
INSERT INTO Person (id, name, father)
VALUES (1, 'John_Sr.', NULL);

-- 2. Insert the child record referencing the existing father
INSERT INTO Person (id, name, father)
VALUES (2, 'John_Jr.', 1);
```

Advantage: Standard, ANSI SQL compliant, and universally supported.
Disadvantage: Requires the foreign key column to be nullable.

Method 2: Deferred Constraint Checking

In advanced DBMSs like PostgreSQL or Oracle, foreign key constraints can be declared as DEFERRABLE INITIALLY DEFERRED. This defers the constraint validation from the statement level to the **transaction commit level**.

```
-- Start a transaction block
BEGIN;

-- Both statements execute without immediate constraint checks
INSERT INTO Person (id, name, father) VALUES (2, 'John_Jr.', 1);
INSERT INTO Person (id, name, father) VALUES (1, 'John_Sr.', NULL);

-- Constraints are evaluated ONLY at commit time
COMMIT;
```

- Advantage: Allows arbitrary insertion order and handles cyclic references seamlessly.
- Disadvantage: Not supported by all database engines (e.g., standard SQL Server or older MySQL).

Method 3: Temporarily Disabling Foreign Key Checks

In some systems (like MySQL/MariaDB or SQL Server), you can forcefully disable foreign key checks at the session level before performing the bulk insert, and re-enable them afterward.

```
-- MySQL Specific Example
SET FOREIGN_KEY_CHECKS = 0;

INSERT INTO Person (id, name, father) VALUES (2, 'John_Jr.', 1);
INSERT INTO Person (id, name, father) VALUES (1, 'John_Sr.', NULL);

SET FOREIGN_KEY_CHECKS = 1;
```

- Advantage: Very fast for bulk data loading.
- Disadvantage: Bypasses database integrity rules temporarily; highly DBMS-specific.

Method 4: The Dummy/Self-Referencing Root

If NULL is strictly not allowed (NOT NULL constraint), you can initialize the table with a "dummy" root record that references itself. Once the root is established, other records can be inserted normally.

```
-- Insert a dummy root that references its own ID
INSERT INTO Person (id, name, father) VALUES (0, 'Unknown/Root', 0);

-- Insert normal records referencing the root where applicable
INSERT INTO Person (id, name, father) VALUES (1, 'John_Sr.', 0);
```

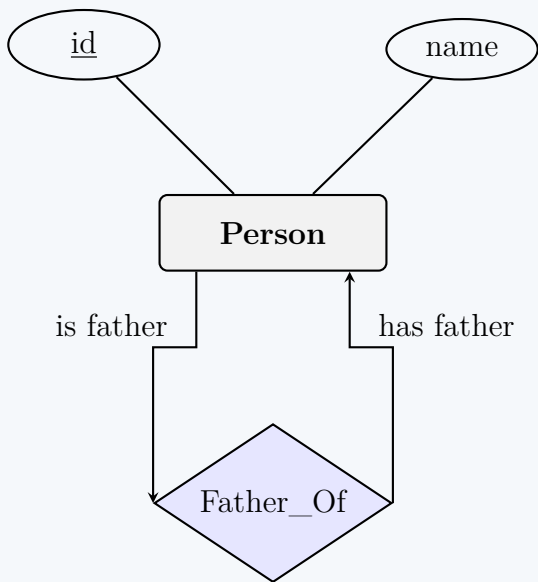


Figure 1: Entity-Relationship (ER) modeling of the recursive constraint.

Summary Comparison

Technique	Best Use Case	Limitation
1. Top-Down via NULL	Standard hierarchical structures	Requires father to be NULL-able
2. Deferred Constraints	Complex cyclic hierarchies	Not supported in all RDBMS
3. Disable Checks	Large bulk imports, migrations	Specific to DBMS; risky if not managed
4. Self-Referencing Dummy	Strict NOT NULL requirements	Litters database with non-real "dummy" data

Q27. What are the three types of relations in SQL? (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

The Three Types of Relations in SQL

In the context of the SQL standard and Relational Database Management Systems (RDBMS), a **relation** is broadly synonymous with a table (a mathematical set of tuples). However, relations manifest in different ways depending on their storage mechanism, lifecycle, and generation method.

The three fundamental types of relations in SQL are **Base Relations**, **Views (Virtual Relations)**, and **Temporary/Derived Relations**.

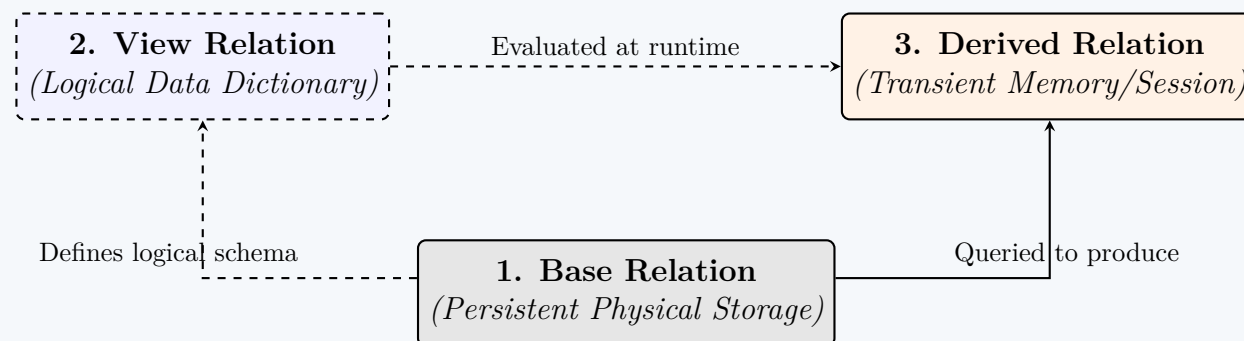


Figure : Architecture and lifecycle of the three SQL relation types.

1. Base Relations (Base Tables)

A base relation is an autonomous, named relation whose records are physically stored in the database's persistent storage (e.g., hard disks or SSDs).

- **Properties:** They define the primary conceptual schema of the database. They possess their own independent existence and persist until explicitly dropped.
- **Working Principle:** Data inserted into a base relation requires physical disk allocation.
- **SQL Creation:** Created using the `CREATE TABLE` Data Definition Language (DDL) command.

```
-- Creating a Base Relation
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50)
);
```

2. Views (Virtual Relations)

A view is a dynamically generated, virtual relation that does not physically store data itself (with the exception of *materialized views*). Instead, it stores a query definition in the database's system catalog.

- **Properties:** Acts as a "window" into one or more base tables. Modifying the base table automatically reflects in the view.
- **Advantages:** Provides logical data independence, enhances security by hiding sensitive columns, and simplifies complex queries.
- **SQL Creation:** Created using the `CREATE VIEW` DDL command.

```
-- Creating a Virtual Relation (View)
CREATE VIEW IT_Employees AS
SELECT EmpID, Name
FROM Employee
WHERE Department = 'IT';
```

3. Temporary / Derived Relations

A derived or temporary relation is transient. It is generated dynamically during the execution of a query and typically resides in the main memory (RAM) or a temporary workspace. Once the query execution or the user session ends, the relation is destroyed.

- **Properties:** Includes the final result set of a `SELECT` statement, Common Table Expressions (CTEs), inline views, and explicit temporary tables.
- **Applications:** Used to hold intermediate results for complex multi-step processing or to isolate data for a single user session without bloating the physical disk.
- **SQL Creation:** Implicitly created by queries or explicitly via `CREATE TEMPORARY TABLE` or `WITH` clauses.


```
-- Creating a Derived Relation using a CTE (WITH clause)
WITH DeptCounts AS (
    SELECT Department, COUNT(*) as Total
    FROM Employee
    GROUP BY Department
)
SELECT * FROM DeptCounts WHERE Total > 10;
```

Comparison of SQL Relations

Feature	Base Relation	View (Virtual)	Derived / Temp
Storage	Persistent Physical Disk	Query definition only (Data Dictionary)	Transient Memory / Temp DB space
Data Independence	Low (Tied to schema)	High (Abstracts schema)	High (Independent intermediate)
Lifespan	Permanent (Until dropped)	Permanent (Definition only)	Session or Query duration
Updateability	Directly Updatable	Updatable under strict rules	Read-Only / Session-specific
Main Use Case	Storing the actual application data	Security, simplification, logical abstraction	Intermediate processing, query results

Q28. Consider the following relations:

```
Emp(eno, ename, title, city)
Proj(pno, pname, budget, city)
Works(eno, pno, resp, dur)
Pay(title, salary)
```

where Emp.title is a foreign key to Pay.title, Works.eno is a foreign key to Emp.eno and Works.pno is a foreign key to Proj.pno. Write SQL statements to answer the following queries:

- (a) For each city, how many projects are located in that city and what is the total budget over all projects in the city?
- (b) For each project, what fraction of the budget is spent (in total) on salaries for the people working on that project? Sort your answer by the value of the budget.
- (c) Remove the work assignment of all persons to any projects for which more than 2 persons share the same responsibility.
- (d) List all projects located in Toronto city and include for each one the number of persons working on the project.
- (e) Find the names of projects whose budgets are greater than all projects located in Waterloo city.

(25 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

Relational Schema Visualization

Before writing the SQL queries, we establish the conceptual mappings of the given schema, identifying Primary Keys (underlined) and Foreign Keys (italicized and linked via arrows).

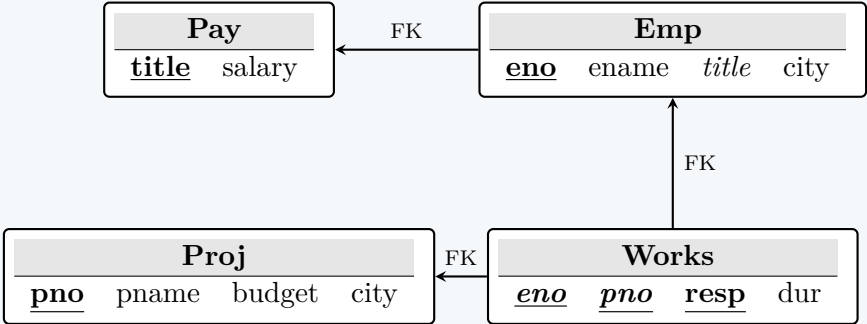


Figure 1: Relational Schema displaying tables, Primary Keys (bold/underlined), and Foreign Keys (linked).

SQL Query Solutions

1. Projects and Budget by City

Explanation: This query uses the GROUP BY clause to aggregate data on a per-city basis. We count the number of project IDs and sum the budget.

```
SELECT city,
       COUNT(pno) AS num_projects,
       SUM(budget) AS total_budget
FROM Proj
GROUP BY city;
```

2. Fraction of Budget Spent on Salaries

Explanation: We perform a four-table JOIN to link projects to the salaries of the employees working on them. The fraction is calculated by summing the salaries and dividing by the project's budget. The result is ordered by the budget.

```
SELECT P.pno,
       P.pname,
       (SUM(S.salary) / P.budget) AS salary_fraction
FROM Proj P
JOIN Works W ON P.pno = W.pno
JOIN Emp E ON W.eno = E.eno
JOIN Pay S ON E.title = S.title
GROUP BY P.pno, P.pname, P.budget
ORDER BY P.budget ASC;
```

3. Remove Congested Work Assignments

Explanation: We need to DELETE from the Works table. The subquery identifies the (pno, resp) pairs that have more than 2 employees assigned by using the HAVING clause.

```
DELETE FROM Works
WHERE (pno, resp) IN (
    SELECT pno, resp
    FROM Works
    GROUP BY pno, resp
    HAVING COUNT(eno) > 2
);
```

Note: In standard SQL, replacing IN with a tuple match is perfectly valid. Some specific RDBMS (like older MySQL versions) might require wrapping the subquery in an additional derived table to bypass mutating-table restrictions.

4. Toronto Projects and Worker Count

Explanation: A LEFT JOIN is strictly preferred here over an INNER JOIN. If a project in Toronto exists but currently has zero workers assigned to it, an INNER JOIN would filter it out. The LEFT JOIN ensures the project is listed with a count of 0.

```
SELECT P.pno,
       P.pname,
       COUNT(W.eno) AS num_workers
FROM Proj P
LEFT JOIN Works W ON P.pno = W.pno
WHERE P.city = 'Toronto'
GROUP BY P.pno, P.pname;
```

5. High-Budget Projects (Compared to Waterloo)

Explanation: The > ALL operator is utilized here. It compares the budget of each project against the set of all budgets from projects located in Waterloo.

```
SELECT pname
FROM Proj
WHERE budget > ALL (
    SELECT budget
    FROM Proj
    WHERE city = 'Waterloo'
);
```

Note: Alternatively, this can be written using > (SELECT MAX(budget) FROM Proj WHERE city = 'Waterloo'), which is logically equivalent and often yields a more optimal execution plan.

Q29. A database schema consists of four relations:

```

Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)

```

- (a) Delete all laptops made by a manufacturer that does not make printers. (5 marks) [CSE 303 | L-3/T-1 | 2011–12]
- (b) Assume that manufacturer A bought manufacturer B. Change all products made by B so that they are now made by A. (5 marks) [CSE 303 | L-3/T-1 | 2011–12]

Answer

Relational Schema Visualization

To better understand the constraints and relationships before performing Data Manipulation Language (DML) operations, we first visualize the database schema. The `model` attribute acts as the Primary Key for the `Product` table and as a Foreign Key in the `PC`, `Laptop`, and `Printer` tables, establishing an inheritance-like relationship (ISA hierarchy).

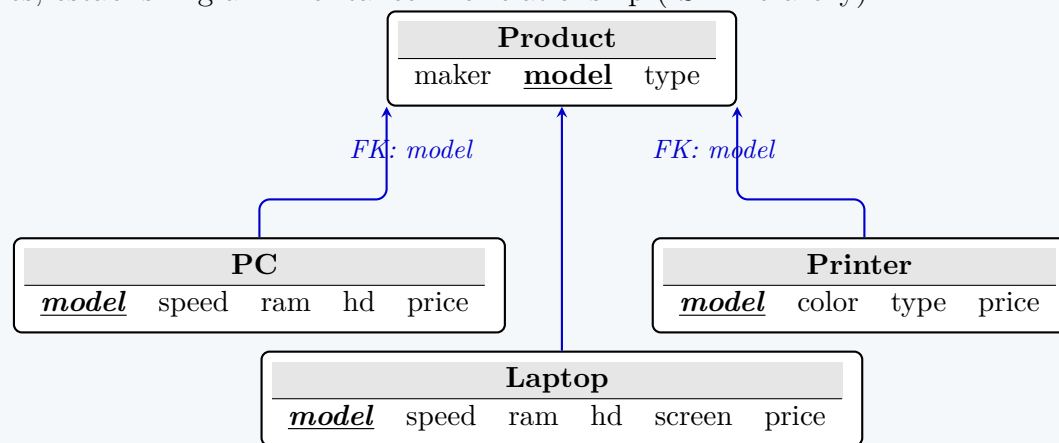


Figure 1: Relational Schema displaying the primary table (`Product`) and dependent entities. Primary Keys are bold and underlined; Foreign Keys are italicized.

SQL DML Solutions

1. Delete all laptops made by a manufacturer that does not make printers.

Working Principle: To accomplish this, we must delete records from the `Laptop` table. However, the manufacturer (`maker`) information resides in the `Product` table. We use a subquery to find all `model` numbers of laptops whose `maker` is not present in the set of makers who produce printers.

```

DELETE FROM Laptop
WHERE model IN (
    -- Find laptop models manufactured by the targeted makers
    SELECT model
    FROM Product
    WHERE type = 'laptop'
    AND maker NOT IN (
        -- Subquery to find makers who DO make printers
        SELECT maker
        FROM Product
        WHERE type = 'printer'
    )
);

```

Note: Depending on the database constraint configurations, deleting a row from `Laptop` may also require deleting the corresponding row in `Product`. If `ON DELETE CASCADE` is not enabled, the query above correctly cleans the `Laptop` relation.

2. Assume that manufacturer A bought manufacturer B. Change all products made by B so that they are now made by A.

Working Principle: This requires an `UPDATE` operation. Since the `maker` attribute is solely stored in the `Product` relation, we only need to update this single table. All specialized tables (`PC`, `Laptop`, `Printer`) reference `Product` via the `model` key, so updating the `maker` in `Product` implicitly applies to all specific product types.

```

UPDATE Product
SET maker = 'A'
WHERE maker = 'B';

```

Advantages of this schema design: Because `maker` is functionally dependent on `model` and stored exclusively in the `Product` table (3NF - Third Normal Form), updating the manufacturer requires modifying only one table, preventing update anomalies.

Q30. Consider the following schema:

```

Product(pid, name, price, category, year, maker_cid)
Purchase(buyer_ssn, seller_ssn, store, pid)
Company(cid, name, stock_price, country)
Person(ssn, name, phone_number, city)

```

Note: In Purchase: buyer_ssn, seller_ssn are foreign keys in Person, pid is foreign key in Product; in Product maker_cid is a foreign key in Company.

Write SQL statements to answer the following queries:

- Find names of people buying telephony products.
- Find products (and their manufacturers) that are more expensive than all products made by the same manufacturer before 1972.
- Find names of people who bought Bangladeshi products and did not buy Indian products.
- Find names of products (and their manufacturers) whose prices are less than the average product price of that company.

(20 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

Database Schema Visualization

To facilitate accurate SQL query construction, we first visualize the relational schema, identifying the primary keys (underlined) and the foreign key references (indicated by directed arrows).

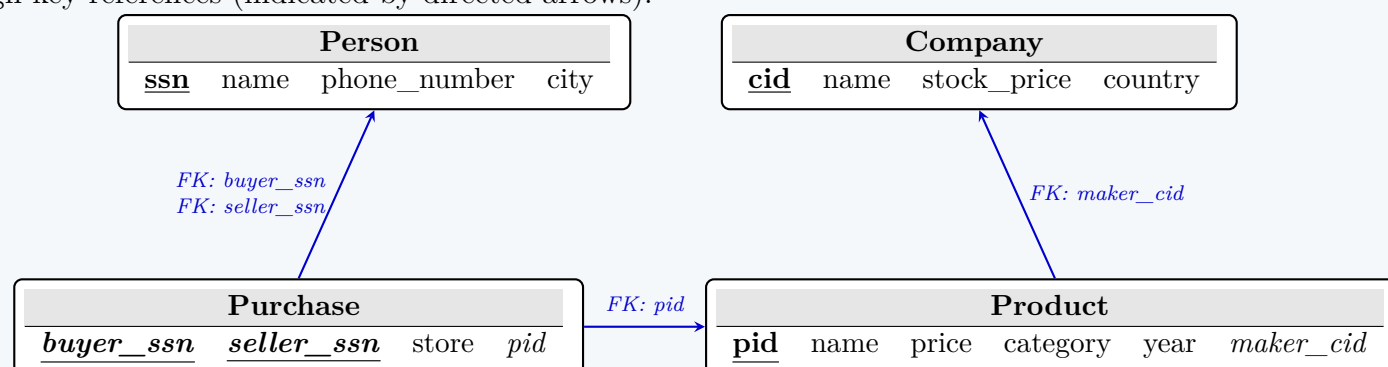


Figure 1: Relational Schema displaying tables, Primary Keys (bold/underlined), and Foreign Keys (italicized and linked).

SQL Query Solutions

1. Names of people buying telephony products

Working Principle: We require a three-table JOIN between Person (acting as the buyer), Purchase, and Product. We filter for the specific product category. The DISTINCT keyword ensures that if a person bought multiple telephony products, their name is only listed once.

```

SELECT DISTINCT Pe.name
FROM Person Pe
JOIN Purchase Pu ON Pe.ssn = Pu.buyer_ssn
JOIN Product Pr ON Pu.pid = Pr.pid
WHERE Pr.category = 'telephony';

```

2. Products and manufacturers more expensive than all products made by the same manufacturer before 1972

Working Principle: This is handled using the > ALL comparative operator coupled with a correlated subquery. The subquery fetches the prices of all products manufactured by the same company prior to 1972.

```

SELECT P1.name AS product_name, C.name AS manufacturer_name
FROM Product P1
JOIN Company C ON P1.maker_cid = C.cid
WHERE P1.price > ALL (
    SELECT P2.price
    FROM Product P2
    WHERE P2.maker_cid = P1.maker_cid
    AND P2.year < 1972
);

```

Note: According to relational algebra and standard SQL behavior, if a company has NO products made before 1972, the subquery returns an empty set, and the > ALL condition evaluates to TRUE. This satisfies strict mathematical logic.

3. Names of people who bought Bangladeshi products and did not buy Indian products

Working Principle: This query requires a set difference logic. We find the set of people who bought Bangladeshi products and exclude (NOT IN) those who exist in the set of people who bought Indian products.

```

SELECT DISTINCT Pe.name
FROM Person Pe

```

```
JOIN Purchase Pu ON Pe.ssn = Pu.buyer_ssn
JOIN Product Pr ON Pu.pid = Pr.pid
JOIN Company C ON Pr.maker_cid = C.cid
WHERE C.country = 'Bangladesh'
AND Pe.ssn NOT IN (
    -- Subquery: People who bought Indian products
    SELECT Pu2.buyer_ssn
    FROM Purchase Pu2
    JOIN Product Pr2 ON Pu2.pid = Pr2.pid
    JOIN Company C2 ON Pr2.maker_cid = C2.cid
    WHERE C2.country = 'India'
);
```

4. Names of products (and manufacturers) whose prices are less than the average product price of that company

Working Principle: A correlated subquery calculates the average price (AVG) of products for the *current* row’s manufacturer (maker_cid). The outer query then filters out products whose prices are equal to or greater than this calculated average.

```
SELECT P.name AS product_name, C.name AS manufacturer_name
FROM Product P
JOIN Company C ON P.maker_cid = C.cid
WHERE P.price < (
    -- Subquery: Calculate average price for the specific company
    SELECT AVG(P2.price)
    FROM Product P2
    WHERE P2.maker_cid = P.maker_cid
);
```

Relational Algebra

Q1. Consider the relations `Instructor` and `Teaches` below. Compute the output of the following:

- (a) $\Pi_{name}(\sigma_{salary < 900000}(\text{Instructor}))$
- (b) $\sigma_{year=2024}(\sigma_{\text{Instructor.id}=\text{Teaches.id}}(\text{Instructor} \times \text{Teaches}))$
- (c) $\Pi_{name}(\text{Instructor} \bowtie_{\text{Instructor.id}=\text{Teaches.id}} \text{Teaches})$

Instructor				Teaches			
id	name	dept_name	salary	id	course_id	semester	year
1	Alice	CSE	950000	1	CS101	Fall	2023
2	Bob	Physics	2300000	1	CS102	Spring	2024
3	Carol	CSE	720000	2	PH201	Fall	2023
4	David	Math	2200000	3	CS101	Fall	2023
5	Eve	CSE	2500000	5	CS201	Fall	2023

(9 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Relational Algebra Query Evaluation

To compute the outputs for the given relational algebra expressions, we break down the operations systematically. In relational algebra, operators are applied sequentially, often represented visually using **Query Trees**.

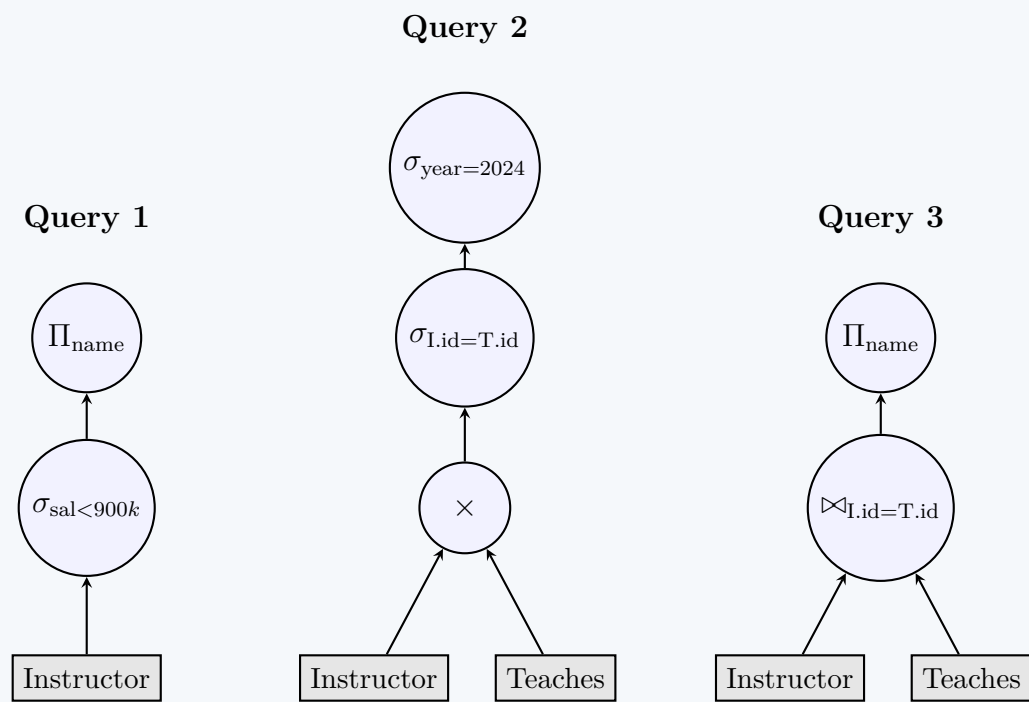


Figure 1: Query evaluation trees for the three relational algebra expressions. Nodes denote operations, and leaves denote base relations.

1. $\Pi_{name}(\sigma_{salary < 900000}(\mathbf{Instructor}))$

Working Principle:

- (a) **Selection (σ):** First, we filter the **Instructor** relation for tuples where the **salary** attribute is strictly less than 900,000. Looking at the data, only Carol has a salary (720,000) that satisfies this condition.
- (b) **Projection (Π):** Next, we extract only the **name** column from the resulting filtered tuples.

Output:

name
Carol

2. $\sigma_{year=2024}(\sigma_{\mathbf{Instructor.id=Teaches.id}}(\mathbf{Instructor} \times \mathbf{Teaches}))$

Working Principle:

- (a) **Cartesian Product ($\times$):** Combines every tuple in **Instructor** with every tuple in **Teaches**, yielding a table of $5 \times 5 = 25$ tuples. The schema includes all attributes from both relations. Conflicting attribute names (**id**) are fully qualified.
- (b) **Inner Selection ($\sigma_{\mathbf{Instructor.id=Teaches.id}}$):** Filters the 25 tuples down to those where the **id** matches. This is logically equivalent to an Equi-Join. There are 5 matching tuples.
- (c) **Outer Selection ($\sigma_{year=2024}$):** Filters the joined result, retaining only the tuples where **year** equals 2024. Only one tuple meets this criterion (Alice teaching CS102).

Output:

Instructor.id	name	dept_name	salary	Teaches.id	course_id	semester	year
1	Alice	CSE	950000	1	CS102	Spring	2024

3. $\Pi_{name}(\mathbf{Instructor} \bowtie_{\mathbf{Instructor.id=Teaches.id}} \mathbf{Teaches})$

Working Principle:

- (a) **Theta Join ($\bowtie$):** The relations are joined on the condition that their **id** attributes match. This identifies all instructors who are teaching at least one course.
- (b) **Projection (Π):** Extracts the **name** attribute from the joined tuples.
- (c) **Duplicate Elimination:** By strict definition of the mathematical set operations in Relational Algebra, a projection always returns a **set**. Duplicates (such as 'Alice', who appears twice because she teaches two courses) are implicitly removed.

Output:

name
Alice
Bob
Carol
Eve

*Note: David is excluded because his **id** (4) does not appear in the **Teaches** table, meaning he currently teaches no courses.*

Q2. Consider the following schema:

Suppliers(sid, sname, address), Parts(pid, pname, color), Catalog(sid, pid, cost)

Write relational algebraic expressions for the following:

- Find the names of suppliers who supply more than 10 parts.
- Find the sids of suppliers who supply some red or green part.
- Find the sids of suppliers who supply some red part or are located at 221 Packer Street.
- Find the sids of suppliers who supply some red part with each part's cost more than 100 taka.
- Find the sids of suppliers who supply every part.

(25 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

Relational Schema Visualization

Before constructing the relational algebra queries, it is crucial to map out the schema to identify relationships and foreign keys. The Catalog relation acts as a junction table connecting Suppliers and Parts.

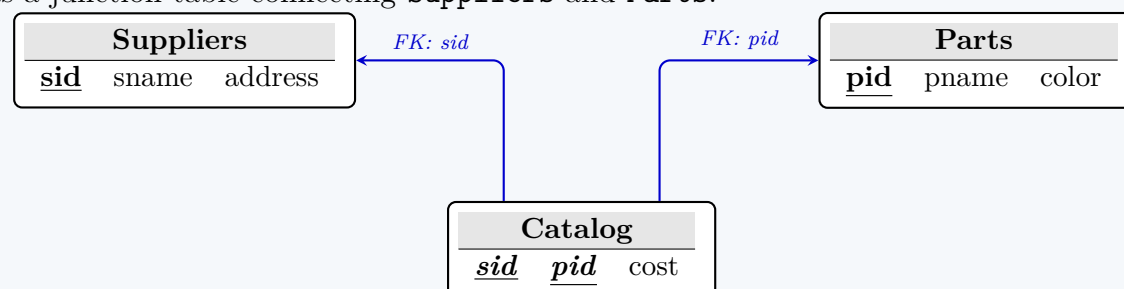


Figure 1: Schema diagram showing the junction table (Catalog) and its foreign key references.

Relational Algebra Expressions

1. Names of suppliers who supply more than 10 parts

Working Principle: Standard relational algebra does not include aggregation. We must use the extended relational algebra grouping operator ($\mathcal{G}$ or γ). We group by **sid**, count the **pid**, filter for counts greater than 10, and finally join with **Suppliers** to project the **sname**.

$$\Pi_{\text{sname}} \left(\text{Suppliers} \bowtie \sigma_{\text{count_pid} > 10} \left(\text{sid} \mathcal{G}_{\text{COUNT}(\text{pid}) \rightarrow \text{count_pid}} (\text{Catalog}) \right) \right)$$

2. Sids of suppliers who supply some red or green part

Working Principle: A natural join ($\bowtie$) between **Parts** and **Catalog** links the parts to their suppliers. A selection (σ) filters for the specified colors, followed by a projection (Π) to retrieve the supplier IDs.

$$\Pi_{\text{sid}} \left(\sigma_{\text{color} = \text{'red'} \vee \text{color} = \text{'green'}} (\text{Parts} \bowtie \text{Catalog}) \right)$$

3. Sids of suppliers who supply some red part or are located at 221 Packer Street

Working Principle: The word "or" between two distinct supplier conditions suggests the use of the Union ($\cup$) operator. We construct two separate queries and union their results.

$$\Pi_{\text{sid}} \left(\sigma_{\text{color} = \text{'red'}} (\text{Parts} \bowtie \text{Catalog}) \right) \cup \Pi_{\text{sid}} \left(\sigma_{\text{address} = \text{'221 Packer Street'}} (\text{Suppliers}) \right)$$

4. Sids of suppliers who supply some red part with each part's cost more than 100 taka

Working Principle: We apply a compound selection condition on the natural join of **Parts** and **Catalog**, ensuring that the part is red AND its cost exceeds 100.

$$\Pi_{\text{sid}} \left(\sigma_{\text{color} = \text{'red'} \wedge \text{cost} > 100} (\text{Parts} \bowtie \text{Catalog}) \right)$$

5. Sids of suppliers who supply every part

Working Principle: The phrase "every" or "all" strongly indicates the use of the Division ($\div$) operator. We take the set of all supplier-part combinations and divide it by the set of all possible parts.

$$(\Pi_{sid,pid}(Catalog)) \div (\Pi_{pid}(Parts))$$

Operator Summary Table

Symbol	Operator	Usage in Context
Π	Projection	Extracts specific columns (e.g., <code>sid</code> , <code>sname</code>).
σ	Selection	Filters rows based on a predicate (e.g., <code>color='red'</code>).
$\bowtie$	Natural Join	Combines tables based on shared attributes (like <code>pid</code>).
$\cup$	Union	Combines the results of two queries (logical OR).
$\div$	Division	Used for universal quantification queries (e.g., "every part").
$\mathcal{G}$	Grouping/Aggregation	Extended operator used for counting records.

Q3. A school has prepared a database to manage admission tests. The examinees are given roll numbers and assigned to specific rooms for the exam. The test contains 10 questions. Each question is marked by two examiners to ensure fairness and correctness. An absentee list is kept to ensure no script goes missing. The schema for the database is as: follows:

```
Room(Room_No, Floor, Capacity)
Examinee(Roll_No, Name, Address, Date_of_Birth, Room_No)
Evaluation(Roll_No, Question_No, Examiner, Marks)
Absentee(Roll_No)
```

Write queries using relational algebra to compute the following:

- (a) The students should be assigned to the rooms in a manner that all the Roll Nos in the room are sequential (e.g., if a room has 10 examinees and the lowest Roll No in that room is 501, then Roll Nos 502, 503, ...,510 should be assigned to that room). Find the rooms that violate this condition.
- (b) We need to check if all scripts have been correctly evaluated, i.e., all examinees' answers to all questions have been evaluated by exactly two teachers. Find out the Roll No and Question No of the students that have not got correctly evaluated. Please note that only the examinees that are present in the exam should be considered here.
- (c) The marks given by two examiners can only be at most 2 marks apart (i.e., for a student's answer to a specific question, the two marks by the two examiners should not differ by more than 2). Find the Roll No and Question No for which two examiners' marks differ by more than 2.

(30 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

Relational Schema Visualization

First, let's visualize the schema and the relationships between the relations to clearly understand the flow of primary and foreign keys.

Figure 1: Schema diagram showing tables, primary keys (underlined), and foreign keys (italicized and linked).

Relational Algebra Expressions

1. Rooms that violate the sequential roll number condition

Working Principle: A room violates the sequential condition if there are two examinees in the same room with roll numbers R_1 and R_2 ($R_1 < R_2$), but there exists some roll number R_3 (where $R_1 < R_3 < R_2$) that is assigned to a *different* room.

Let $E = \pi_{\text{Roll_No}, \text{Room_No}}(\text{Examinee})$. We rename E as E_1, E_2 , and E_3 to represent R_1, R_2 , and R_3 respectively.

$$\rho_{E_1}(E), \quad \rho_{E_2}(E), \quad \rho_{E_3}(E)$$

The violating rooms can be found using the following expression:

$$\Pi_{E_1.\text{Room_No}}\left(\sigma_C(E_1 \times E_2 \times E_3)\right)$$

Where the condition C is:

$$C \equiv (E_1.\text{Room_No} = E_2.\text{Room_No}) \wedge (E_1.\text{Roll_No} < E_3.\text{Roll_No}) \wedge (E_3.\text{Roll_No} < E_2.\text{Roll_No}) \wedge (E_1.\text{Room_No} \neq E_3.\text{Room_No})$$

2. Examinees whose scripts have not been correctly evaluated (exactly 2 evaluations)

Working Principle: We only consider present examinees. An error occurs if an examinee has **less than 2** or **more than 2** evaluations for any question.

Step 2.1: Find all valid present examinees and the universal set of required evaluations.

$$\text{Present} = \Pi_{\text{Roll_No}}(\text{Examinee}) - \Pi_{\text{Roll_No}}(\text{Absentee})$$

$$\text{Questions} = \Pi_{\text{Question_No}}(\text{Evaluation})$$

$$\text{Req} = \text{Present} \times \text{Questions}$$

Step 2.2: Find pairs with *at least 2* evaluations. Let $Ev_1 = \rho_{Ev_1}(\text{Evaluation})$ and $Ev_2 = \rho_{Ev_2}(\text{Evaluation})$.

$$\text{AtLeast2} = \Pi_{Ev_1.\text{Roll_No}, Ev_1.\text{Question_No}}\left(\sigma_{C_1}(Ev_1 \times Ev_2)\right)$$

$$C_1 \equiv (Ev_1.\text{Roll_No} = Ev_2.\text{Roll_No}) \wedge (Ev_1.\text{Question_No} = Ev_2.\text{Question_No}) \wedge (Ev_1.\text{Examiner} \neq Ev_2.\text{Examiner})$$

Step 2.3: Find pairs with *less than 2* evaluations.

$$\text{LessThan2} = \text{Req} - \text{AtLeast2}$$

Step 2.4: Find pairs with *more than 2* evaluations. Let $Ev_3 = \rho_{Ev_3}(\text{Evaluation})$.

$$\text{MoreThan2} = \Pi_{Ev_1.\text{Roll_No}, Ev_1.\text{Question_No}}\left(\sigma_{C_2}(Ev_1 \times Ev_2 \times Ev_3)\right)$$

$$C_2 \equiv C_1 \wedge (Ev_2.\text{Roll_No} = Ev_3.\text{Roll_No}) \wedge (Ev_2.\text{Question_No} = Ev_3.\text{Question_No}) \wedge (Ev_1.\text{Examiner} \neq Ev_3.\text{Examiner}) \wedge (Ev_2.\text{Examiner} \neq Ev_3.\text{Examiner})$$

Final Result:

$$\text{Incorrectly_Evaluated} = \text{LessThan2} \cup \text{MoreThan2}$$

3. Roll No and Question No for which two examiners' marks differ by more than 2

Working Principle: We perform a Cartesian product of the **Evaluation** relation with itself, find tuples matching the same roll number and question number but different examiners, and check if the absolute difference in their marks exceeds 2.

Let $E_1 = \rho_{E_1}(\text{Evaluation})$ and $E_2 = \rho_{E_2}(\text{Evaluation})$.

$$\Pi_{E_1.\text{Roll_No}, E_1.\text{Question_No}}\left(\sigma_{\text{Condition}}(E_1 \times E_2)\right)$$

Where the *Condition* is defined as:

$$(E_1.\text{Roll_No} = E_2.\text{Roll_No}) \wedge (E_1.\text{Question_No} = E_2.\text{Question_No}) \wedge (E_1.\text{Examiner} \neq E_2.\text{Examiner}) \wedge \left((E_1.\text{Marks} - E_2.\text{Marks} > 2) \vee (E_2.\text{Marks} - E_1.\text{Marks} > 2)\right)$$

Q4. Suppose we have two relations A and B with the same set of attributes. Relation A has n_a tuples and relation B has n_b tuples. For each of the following relational algebra expressions, give the minimum and maximum number of tuples that can appear in the result. Assume that the produced results are bags, not sets.

- (a) $A \cap B$
- (b) $\sigma_c(A) - B$
- (c) $A \cup B$
- (d) $A \bowtie_L B$, where $\bowtie_L$ is the left outer join

(8 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

Tuple Cardinality Bounds under Bag Semantics

In relational algebra with **bag (multiset) semantics**, duplicate tuples are retained. This fundamentally changes how operations like Union, Intersection, and Difference behave compared to strict set theory. Let $|A| = n_a$ and $|B| = n_b$.

Expression	Minimum	Maximum	Reasoning (Bag Semantics)
1. $A \cap B$	0	$\min(n_a, n_b)$	Min: Occurs when bags A and B are completely disjoint (share no tuples). Max: In bag intersection, a tuple appears $\min(x, y)$ times. The max size is bounded by the smaller relation if it is a complete sub-bag of the larger one.
2. $\sigma_c(A) - B$	0	n_a	Min: Occurs if the selection condition c rejects all tuples in A (yielding 0 tuples), or if all selected tuples are completely canceled out by identical tuples in B . Max: Occurs if the selection condition c accepts all n_a tuples, and B is completely disjoint from A , meaning no tuples are subtracted.
3. $A \cup B$	$n_a + n_b$	$n_a + n_b$	Min/Max: Bag union (equivalent to SQL's UNION ALL) simply appends the tuples of B to A without eliminating duplicates. Therefore, the result is always exactly $n_a + n_b$.
4. $A \bowtie_L B$	n_a	$n_a \times n_b$	Min: By the definition of a Left Outer Join, every tuple in the left relation (A) is preserved at least once. If no tuples in A match any in B , each of the n_a tuples is padded with NULLs, yielding n_a tuples. Max: Occurs if the join condition evaluates to true for every possible pair of tuples (acting like a Cartesian product), resulting in $n_a \times n_b$ tuples.

Q5. Using an example, show that $\pi_a(R \cap S) \neq \pi_a(R) \cap \pi_a(S)$. (5 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Proof by Counterexample

To prove that the projection operation (π) does not distribute over the intersection operation ($\cap$), we only need to provide a single valid instance of relations R and S where the Left-Hand Side (LHS) evaluates differently than the Right-Hand Side (RHS).

1. Define the Relations

Let R and S be two relations with the identical schema (a, b) . We populate them with the following tuples:

Relation R		Relation S	
a	b	a	b
1	2	1	3

2. Evaluate the Left-Hand Side (LHS): $\pi_a(R \cap S)$

First, we compute the intersection of the two relations, $R \cap S$. Since they do not share any identical entire tuples, their intersection is empty.

$$R \cap S = \emptyset$$

Next, we apply the projection operation on the empty set:

$$\pi_a(R \cap S) = \pi_a(\emptyset) = \emptyset$$

3. Evaluate the Right-Hand Side (RHS): $\pi_a(R) \cap \pi_a(S)$

First, we apply the projection operation on attribute a for both relations independently.

$$\pi_a(R) = \{1\}$$

$$\pi_a(S) = \{1\}$$

Next, we compute the intersection of these two projected sets:

$$\pi_a(R) \cap \pi_a(S) = \{1\} \cap \{1\} = \{1\}$$

4. Conclusion

Comparing the results of the LHS and RHS:

$$\text{LHS} = \emptyset$$

$$\text{RHS} = \{1\}$$

Since $\emptyset \neq \{1\}$, we have successfully proven that:

$$\pi_a(R \cap S) \neq \pi_a(R) \cap \pi_a(S)$$

Note: The logical reason for this inequality is that projecting before intersecting discards the contextual data (attribute b) that originally made the tuples distinct in relations R and S .

Q6. Given the university schema: Classroom(building, room_number, capacity), department(dept_name, building, budget), course(course_id, title, dept_name, credits), instructor(ID, name, dept_name, salary), section(course_id, sec_id, semester, year, building, room_number, time_slot_id), teaches(ID, course_id, sec_id, semester, year), student(ID, name, dept_name, tot_cred), takes(ID, course_id, sec_id, semester, year, grade), advisor(s_ID, i_ID), timeslot(time_slot_id, day, start_time, end_time), prereq(course_id, prereq_id). Write the following queries in relational algebra:

- Find the ID and name of each instructor in the Physics department.
- Find the ID and name of each instructor in a department located in the building 'Watson'.
- Find all the courses taught in the Fall 2022 semester but not in Spring 2022 semester.
- Find the ID and name of those instructors who earn more than the instructors in the Physics department.
- Find the ID and name of each student who has not taken any course section in the year 2022.

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer**Relational Algebra Expressions**

In relational algebra, queries are composed using fundamental operators such as Selection (σ), Projection (Π), Cartesian Product ($\times$), Set Difference ($-$), Union ($\cup$), and Rename (ρ). Derived operators like Natural Join ($\bowtie$) are also heavily utilized to simplify the expressions.

1. Instructors in the Physics department

Working Principle: We apply a **Selection** (σ) on the **instructor** relation to filter those whose **dept_name** is 'Physics'. Then, we apply a **Projection** (Π) to extract only the ID and **name** attributes.

$$\Pi_{\text{ID}, \text{name}} (\sigma_{\text{dept_name}='Physics'}(\text{instructor}))$$

2. Instructors in a department located in the building 'Watson'

Working Principle: The **building** attribute resides in the **department** relation, while ID and **name** reside in **instructor**. We perform a **Natural Join** ($\bowtie$) between them (they share the **dept_name** attribute). Finally, we select the required building and project the desired columns.

$$\Pi_{\text{ID}, \text{name}} (\sigma_{\text{building}='Watson'}(\text{instructor} \bowtie \text{department}))$$

3. Courses taught in Fall 2022 but not in Spring 2022

Working Principle: This requires the **Set Difference** ($-$) operator. We find the set of **course_ids** offered in Fall 2022 and subtract the set of **course_ids** offered in Spring 2022.

$$\begin{aligned} & \Pi_{\text{course_id}} (\sigma_{\text{semester}='Fall' \wedge \text{year}=2022}(\text{section})) \\ & - \\ & \Pi_{\text{course_id}} (\sigma_{\text{semester}='Spring' \wedge \text{year}=2022}(\text{section})) \end{aligned}$$

4. Instructors earning more than all instructors in the Physics department

Working Principle: Relational algebra lacks a direct aggregate function for "maximum". To find elements strictly greater than *all* elements in a target set, we subtract those that are less than or equal to *at least one* element in that set.

(a) Find all Physics instructors: $P = \rho_P(\sigma_{\text{dept_name}=\text{'Physics'}}(\text{instructor}))$

(b) Find instructors earning less than or equal to at least one Physics instructor:

$$L = \Pi_{\text{instructor.ID, instructor.name}}(\sigma_{\text{instructor.salary} \leq P.\text{salary}}(\text{instructor} \times P))$$

(c) Subtract L from the set of all instructors:

$$\Pi_{\text{ID, name}}(\text{instructor}) - L$$

5. Students who have not taken any course section in the year 2022

Working Principle: This also employs the **Set Difference** ($-$) operator. We take the set of all students and remove those who *did* take at least one course in 2022.

$$\Pi_{\text{ID, name}}(\text{student}) - \Pi_{\text{ID, name}}(\text{student} \bowtie \sigma_{\text{year}=2022}(\text{takes}))$$

Query Evaluation Tree (Query 5)

To visualize how the DBMS query optimizer might process Query 5, the following tree demonstrates the hierarchical execution of the relational algebra operations. Data flows from the leaf nodes (relations) upward to the root.

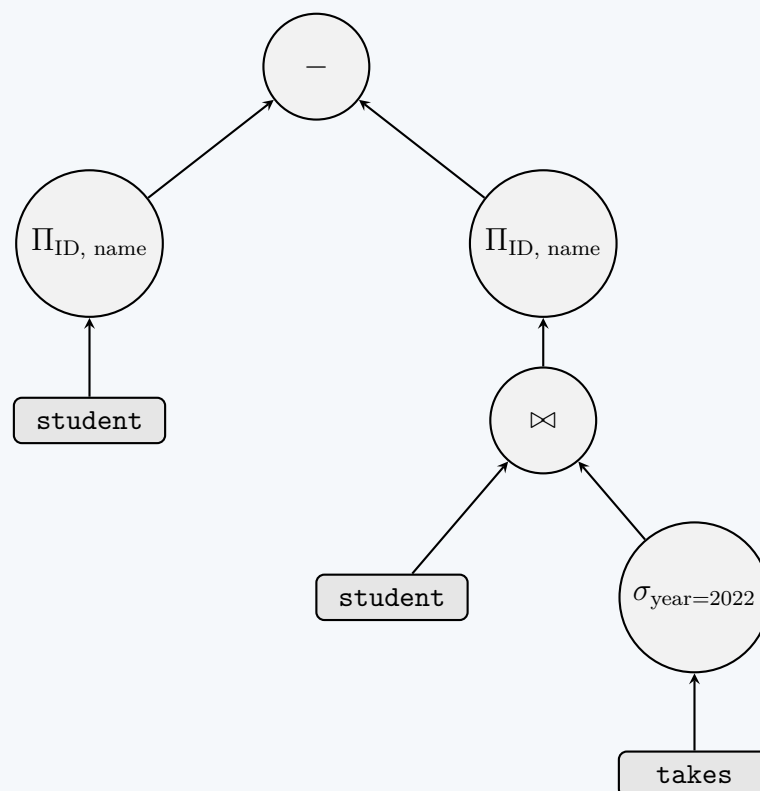


Figure 1: Query execution tree for retrieving students who did not take any course in 2022. Nodes evaluate from the bottom-up.

Q7. Consider the following schema of Sonali Bank Limited:

```

customer(customer_id, customer_name, customer_address)
account(account_id, account_type, account_branch, account_balance)
branch(branch_routing_number, branch_name)
transaction(account_id, transaction_amount, transaction_date)
loan(loan_id, customer_id, loan_amount)
  
```

Notice that the attribute `account_branch` of table `account` is a foreign key referencing the primary key `branch_routing_number` from table `branch`. On the other hand, the attribute `account_id` of table `account` is a foreign key referencing the primary key `customer_id` from table `customer`.

- Write a relational algebra expression using Cartesian product to find customers' names and their corresponding loan amounts who have a "savings" type account at the branch named "BUET Br."
- Write the equivalent relational algebra expression for the following SQL:

```

SELECT c.customer_name, a.account_id, t.transaction_amount
FROM customer c, account a, transaction t
WHERE c.customer_id = a.account_id
      AND a.account_id = t.account_id
      AND t.transaction_date = '01-JAN-2020'
      AND t.transaction_amount > 5000;
  
```

(20 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

Relational Schema Visualization

To construct the correct Relational Algebra expressions, we first visualize the database schema. This highlights the primary keys (underlined) and the specific foreign key references (indicated by directed arrows) as described in the problem statement.

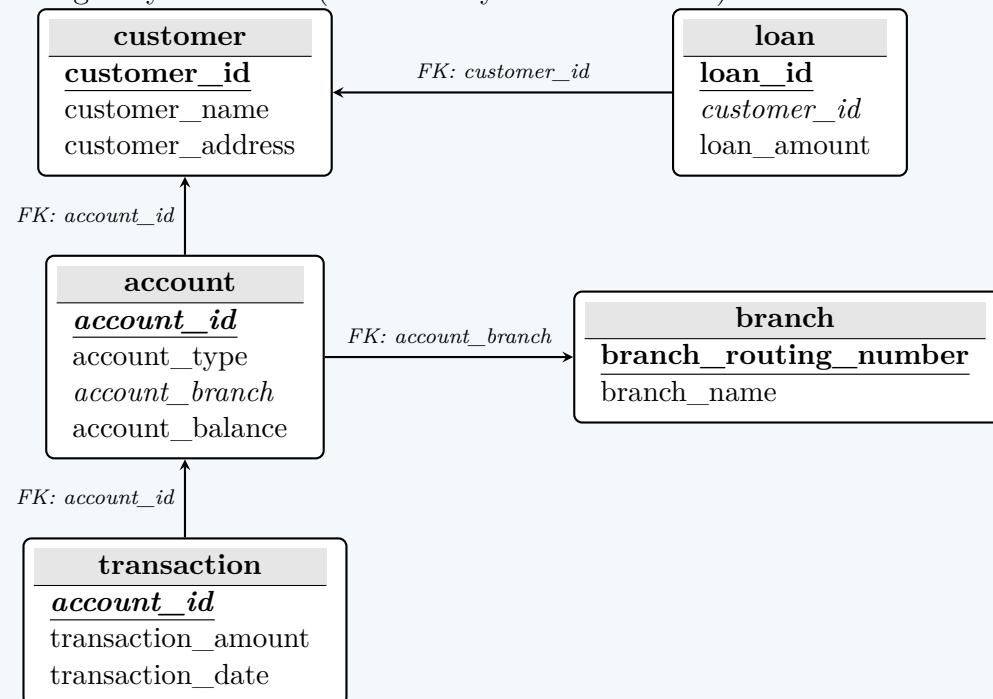


Figure 1: Relational Schema of Sonali Bank Limited showing Primary Keys (bold/underlined) and Foreign Keys (italicized and linked).

Relational Algebra Expressions

1. Customers' names and loan amounts with a "savings" account at "BUET Br."

Working Principle: The question strictly requires the use of the **Cartesian product** ($\times$) rather than a Natural Join ($\bowtie$). Therefore, we must explicitly define the join conditions inside the Selection (σ) operator. We calculate the Cartesian product of **customer**, **account**, **branch**, and **loan**, apply a complex selection predicate (θ) to map the foreign keys and filter the data, and finally project (Π) the desired attributes.

Relational Algebra Expression:

$$\Pi_{\text{customer_name, loan_amount}} \left(\sigma_{\theta} (\text{customer} \times \text{account} \times \text{branch} \times \text{loan}) \right)$$

Where the selection condition θ is defined as the conjunction of join conditions and filter predicates:

$$\begin{aligned} \theta \equiv & (\text{customer.customer_id} = \text{account.account_id}) \\ & \wedge (\text{account.account_branch} = \text{branch.branch_routing_number}) \\ & \wedge (\text{customer.customer_id} = \text{loan.customer_id}) \\ & \wedge (\text{account.account_type} = \text{'savings'}) \\ & \wedge (\text{branch.branch_name} = \text{'BUET Br.'}) \end{aligned}$$

2. Equivalent Relational Algebra for the given SQL Query

Working Principle: The provided SQL query uses implicit joins (listing tables separated by commas in the **FROM** clause) and specifies the join logic within the **WHERE** clause. This structure directly maps to a Cartesian product followed by a Selection in Relational Algebra. To maintain strict equivalence with the SQL aliasing (**c**, **a**, **t**), we use the rename operator (ρ) implicitly by qualifying attribute names with their source relation.

Relational Algebra Expression:

$$\Pi_{\text{customer.customer_name, account.account_id, transaction.transaction_amount}} \left(\sigma_{\phi} (\text{customer} \times \text{account} \times \text{transaction}) \right)$$

Where the selection condition ϕ directly translates the **WHERE** clause of the SQL query:

$$\begin{aligned} \phi \equiv & (\text{customer.customer_id} = \text{account.account_id}) \\ & \wedge (\text{account.account_id} = \text{transaction.account_id}) \\ & \wedge (\text{transaction.transaction_date} = \text{'01-JAN-2020'}) \\ & \wedge (\text{transaction.transaction_amount} > 5000) \end{aligned}$$

Note: While an equivalent query could be written using Natural Joins or Theta Joins, the Cartesian product formulation is the most direct literal translation of the provided implicit-join SQL syntax.

Q8. Consider the following schemas:

```

Manufacturer(man_id, name, address)
Laptop(model, man_id, speed, ram, hd, screen, price)
Printer(model, man_id, is_color, type, price)

```

Write relational algebra expressions for:

- Find the model and speeds of each laptop manufactured by “Dell”.
- Find the sorted list of names of the manufacturers that produce at least three different models of laptops.

(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

Relational Schema Visualization

Before writing the queries, it is useful to model the schema visually to understand the primary keys (underlined) and foreign key relationships (italicized and linked via arrows).

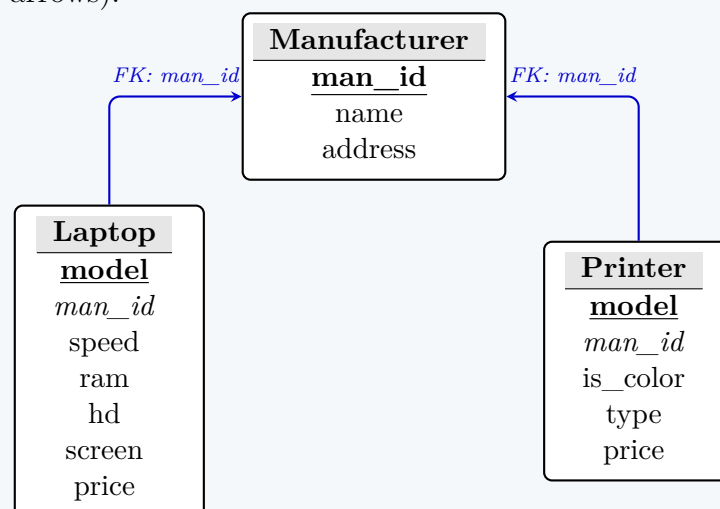


Figure 1: Schema diagram showing the base relations, primary keys, and foreign key dependencies.

Relational Algebra Expressions

1. Model and speeds of each laptop manufactured by “Dell”

Working Principle: To retrieve this data, we must link the **Laptop** relation to the **Manufacturer** relation. Since both relations share the identically named attribute **man_id**, we can safely use the **Natural Join** ($\bowtie$). We then apply a **Selection** (σ) to filter for 'Dell' and finally a **Projection** (Π) to isolate the desired columns.

Expression:

$$\Pi_{\text{model, speed}} (\sigma_{\text{name}='Dell'} (\text{Manufacturer} \bowtie \text{Laptop}))$$

2. Sorted list of names of the manufacturers that produce at least three different models of laptops

Working Principle: This query requires counting and sorting. Sorting is handled by the **Sort operator** (τ) from extended relational algebra. For the "at least three" condition, there are two standard approaches: one using purely fundamental Relational Algebra (via self-joins) and another using Extended Relational Algebra (via aggregation).

Method A: Pure Relational Algebra (Self-Join Approach) We create three distinct copies of the **Laptop** table using the **Rename operator** (ρ). We ensure they belong to the same manufacturer and represent distinct models. Using the strictly less-than operator ($<$) on the **model** attribute implicitly guarantees that all three models are distinct without explicitly writing three inequality ($\neq$) checks.

- Create renamed instances:

$$L_1 = \rho_{L_1}(\text{Laptop}), \quad L_2 = \rho_{L_2}(\text{Laptop}), \quad L_3 = \rho_{L_3}(\text{Laptop})$$

- Define the selection condition C for three distinct laptops from the same manufacturer:

$$C \equiv (L_1.\text{man_id} = L_2.\text{man_id}) \wedge (L_2.\text{man_id} = L_3.\text{man_id}) \wedge (L_1.\text{model} < L_2.\text{model}) \wedge (L_2.\text{model} < L_3.\text{model})$$

- Perform Cartesian product, select, project the **man_id**, join with **Manufacturer**, and sort by name:

$$\tau_{\text{name}} (\Pi_{\text{name}} (\text{Manufacturer} \bowtie \Pi_{L_1.\text{man_id}} (\sigma_C (L_1 \times L_2 \times L_3))))$$

Method B: Extended Relational Algebra (Aggregation Approach) Modern database systems handle this using the **Grouping/Aggregation operator** ($\mathcal{G}$). We group the **Laptop** relation by **man_id** and count the **models**, filter those ≥ 3 , join to get the name, and sort.

- (a) Group by `man_id` and count models:

$$\text{Counted} = \text{man_id} \mathcal{G}_{\text{COUNT(model)} \rightarrow \text{num_models}}(\text{Laptop})$$

- (b) Filter, join, and sort:

$$\tau_{\text{name}} (\Pi_{\text{name}} (\sigma_{\text{num_models} \geq 3}(\text{Counted}) \bowtie \text{Manufacturer}))$$

Q9. Given three relations:

```
customer(customer_id, name, street, city)
account(account_number, type, balance)
transaction(customer_id, account_number, date, amount)
```

- (a) Write a relational algebra expression using Cartesian product to find `customer_id`, `date` and `amount` of all transactions made by customer id “C005”.
- (b) Rewrite the expression of (i) using natural join.
- (c) Write the equivalent relational algebra expression for the following SQL:

```
SELECT a.customer_id, b.account_number, name, street, city
FROM customer AS a, account AS b, transaction AS c
WHERE a.customer_id = c.customer_id
      AND b.account_number = c.account_number
      AND date = '01-JAN-2017'
      AND amount > 10000;
```

(20 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

Relational Schema Visualization

To better understand the required relational algebra operations, we first visualize the schema. The `transaction` table acts as an associative entity (or junction table) mapping transactions to specific customers and accounts.

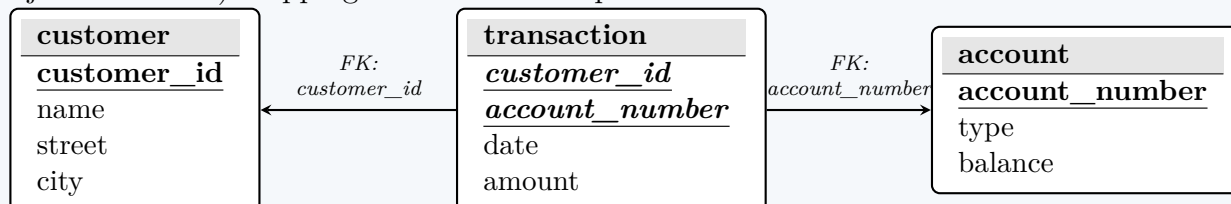


Figure 1: Schema diagram illustrating Primary Keys (underlined) and Foreign Keys (italicized and linked via arrows).

Relational Algebra Expressions

1. Transactions by customer “C005” using Cartesian Product

Working Principle: Although all the required attributes (`customer_id`, `date`, `amount`) exist purely within the `transaction` relation, the prompt explicitly requires the use of a **Cartesian product** ($\times$). In database theory, this typical exercise demonstrates how implicit joins are formed. We compute the Cartesian product of `customer` and `transaction`, apply a **Selection** (σ) to enforce the join condition and the specific customer ID, and finally apply a **Projection** (Π).

Expression:

$$\Pi_{\text{transaction.customer_id,date,amount}} \left(\sigma_{\theta} (\text{customer} \times \text{transaction}) \right)$$

Where the selection condition θ is:

$$\theta \equiv (\text{customer.customer_id} = \text{transaction.customer_id}) \wedge (\text{transaction.customer_id} = \text{'C005'})$$

2. Equivalent Expression using Natural Join

Working Principle: The **Natural Join** ($\bowtie$) operator automatically joins tables based on attributes with the exact same name (in this case, `customer_id`). It simplifies the query by removing the need for a Cartesian product and explicit equality checks for the join. Since `customer_id` appears only once in the resulting schema of a natural join, we no longer need to prefix it with the relation name.

Expression:

$$\Pi_{\text{customer_id,date,amount}} \left(\sigma_{\text{customer_id}=\text{'C005'}} (\text{customer} \bowtie \text{transaction}) \right)$$

3. Equivalent Relational Algebra for the given SQL

Working Principle: The provided SQL query utilizes a comma-separated FROM clause (FROM customer AS a, account AS b, transaction AS c). In relational algebra, this maps directly to an N -way **Cartesian product**. The WHERE clause filters the resulting combinations acting as the **Selection** (σ) predicate. Finally, the SELECT clause maps to the **Projection** (Π). To maintain logical equivalence with the SQL, we fully qualify the ambiguous attributes with their source relation names.

Expression:

$$\Pi_{\text{customer.customer_id, account.account_number, name, street, city}} \left(\sigma_{\phi} (\text{customer} \times \text{account} \times \text{transaction}) \right)$$

Where the combined filter and join predicate ϕ is:

$$\begin{aligned} \phi \equiv & (\text{customer.customer_id} = \text{transaction.customer_id}) \\ & \wedge (\text{account.account_number} = \text{transaction.account_number}) \\ & \wedge (\text{date} = \text{'01-JAN-2017'}) \\ & \wedge (\text{amount} > 10000) \end{aligned}$$

Note: While the SQL uses aliases (a, b, c), using the full relation names in relational algebra achieves the exact same logical resolution without requiring explicit rename (ρ) operators, keeping the expression clean and readable.

Q10. Consider the following schema:

```
Route(FN, DC, AC, DI, SD, SA, PM)
Flight(FN, DT, NP, AD, AA)
Plane(PM, CA, TP)
```

(FN=flight number, DC=departure city, AC=arrival city, DI=distance, SD=sched. depart, SA=sched. arrive, PM=plane model, DT=date, NP=no. of passengers, AD=actual depart, AA=actual arrive, CA=capacity, TP=type)

Translate the following queries into relational algebra:

- Which flights have at least 50 passengers on board and arrived more than 30 minutes late?
- What are the departure and arrival cities of all flights on October 10th that were completely full?
- What are the plane types that travel between BOSTON (BOS) and FRANCISCO (SFO)?
- Which flights departed late but arrived without any delay?

ROUTE	flight number (FN)	from city (FC)	to city (TC)	distance in miles (DI)	scheduled departure time (SD)	scheduled arrival time (SA)	plane (PM)
	1384	ROC	BOS	325	0650	0810	F101
	1205	BOS	ROC	325	1730	1900	F101
	110	BOS	LAX	3250	0900	1040	L1011
	398	LAX	SFO	340	1120	1210	757
	124	SFO	BOS	3180	1400	2310	L1011
	448	BOS	DCA	340	0700	0830	737
	540	DCA	BOS	340	1900	2030	737
FLIGHT	flight number (FN)	date (DT)	number of passengers (NP)	actual departure time (AD)	actual arrival time (AA)		
	1384	2000-10-09	40	0650	0800		
	1384	2000-10-10	12	0730	0842		
	1384	2000-10-11	30	0700	0815		
	1205	2000-10-09	42	1735	1900		
	1205	2000-10-10	8	1730	1855		
	1205	2000-10-11	20	1810	1932		
	110	2000-10-10	403	0940	1115		
	110	2000-10-11	385	0905	1035		
	398	2000-10-09	148	1150	1220		
	398	2000-10-10	95	1125	1210		
	124	2000-10-09	150	1500	2350		
	124	2000-10-10	414	1410	2310		
PLANE	model (PM)	capacity (CA)	type (TP)				
	737	120	jet				
	747	460	jet				
	757	200	jet				
	L1011	440	jet				
	F101	45	prop				

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Relational Schema Visualization

To accurately construct the relational algebra queries, it is essential to map the relationships between the relations. The schema consists of three tables where **Flight** is dependent on **Route**, and **Route** is dependent on **Plane**.

Flight

FN
DT
NP
AD
AA

FK: FN

Route

FN
DC
AC
DI
SD
SA
PM

FK: PM

Plane

PM
CA
TP

Figure 1: Database schema indicating Primary Keys (bold and underlined) and Foreign Keys (italicized and linked).

Relational Algebra Expressions

Note on semantics: A "flight" usually implies a specific instance occurring on a specific date, so we project both the flight number (FN) and the date (DT) to uniquely identify it.

1. Flights with at least 50 passengers on board and arrived more than 30 minutes late

Working Principle: We need the number of passengers (NP) and actual arrival (AA) from the **Flight** relation, and the scheduled arrival (SA) from the **Route** relation. We compute a **Natural Join** ($\bowtie$) on FN. Assuming the DBMS supports conceptual time arithmetic, we check if $AA - SA > 30$.

Expression:

$$\Pi_{\text{FN, DT}} \left(\sigma_{\text{NP} \geq 50 \wedge (\text{AA} - \text{SA}) > 30} (\text{Flight} \bowtie \text{Route}) \right)$$

2. Departure and arrival cities of all flights on October 10th that were completely full

Working Principle: "Completely full" means the number of passengers (NP) equals the plane's capacity (CA). This requires joining all three relations: *Route* (for cities), *Flight* (for date and passengers), and *Plane* (for capacity).

Expression:

$$\Pi_{\text{DC, AC}} \left(\sigma_{\text{DT} = '2000-10-10' \wedge \text{NP} = \text{CA}} (\text{Route} \bowtie \text{Flight} \bowtie \text{Plane}) \right)$$

3. Plane types that travel between BOSTON (BOS) and FRANCISCO (SFO)

Working Principle: "Between" implies the route can operate in either direction (BOS to SFO, or SFO to BOS). We join *Route* and *Plane* to get the plane type (TP) and apply a compound logical **OR** ($\vee$) condition on the departure (DC) and arrival (AC) cities.

Expression:

$$\Pi_{\text{TP}} \left(\sigma_{(\text{DC} = 'BOS' \wedge \text{AC} = 'SFO') \vee (\text{DC} = 'SFO' \wedge \text{AC} = 'BOS')} (\text{Route} \bowtie \text{Plane}) \right)$$

4. Flights that departed late but arrived without any delay

Working Principle: "Departed late" means actual departure (AD) is greater than scheduled departure (SD). "Arrived without delay" means actual arrival (AA) is less than or equal to scheduled arrival (SA). We join *Flight* and *Route* to evaluate these parallel conditions.

Expression:

$$\Pi_{\text{FN, DT}} \left(\sigma_{\text{AD} > \text{SD} \wedge \text{AA} \leq \text{SA}} (\text{Flight} \bowtie \text{Route}) \right)$$

Q11. Consider the following schema:

```
Suppliers(sid: integer, sname: string, address: string)
Parts(pid: integer, pname: string, color: string)
Catalog(sid: integer, pid: integer, cost: real)
```

Write relational algebra expressions for the following:

- Find the sids of suppliers who supply some red or green part.
- Find the sids of suppliers who supply some red parts and are at the address '221 Packer Street'.
- Find the sids of suppliers who supply some red parts and some green parts.
- Find the sids of suppliers who supply every part.

(20 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

Relational Schema Visualization

This is a classic database schema known as the Suppliers-Parts-Catalog database. The *Catalog* relation serves as a many-to-many junction table between *Suppliers* and *Parts*.

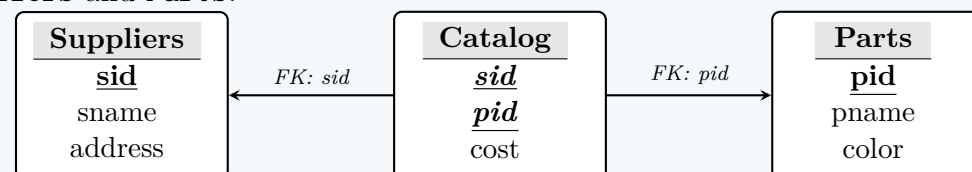


Figure 1: Schema diagram showing Primary Keys (bold and underlined) and Foreign Keys (italicized and linked).

Relational Algebra Expressions

1. Suppliers who supply some red or green part

Working Principle: We need information from both *Catalog* (which contains *sid*) and *Parts* (which contains the *color* attribute). We perform a **Natural Join** ($\bowtie$) between them. Then, we apply a **Selection** (σ) with a logical OR ($\vee$) condition to filter for the desired colors, and finally, a **Projection** (Π) to retrieve only the *sid*.

Expression:

$$\Pi_{\text{sid}} \left(\sigma_{\text{color} = 'red' \vee \text{color} = 'green'} (\text{Catalog} \bowtie \text{Parts}) \right)$$

2. Suppliers who supply some red parts and are at ‘221 Packer Street’

Working Principle: This query requires joining all three relations since we need supplier details (**address**), part details (**color**), and their linkage (**Catalog**). For a highly optimized relational algebra expression, it is best practice to push down the selections *before* the joins.

Expression:

$$\Pi_{\text{sid}} \left(\sigma_{\text{address}='221 \text{ Packer Street}'}(\text{Suppliers}) \bowtie \text{Catalog} \bowtie \sigma_{\text{color}='red'}(\text{Parts}) \right)$$

3. Suppliers who supply some red parts and some green parts

Working Principle: A common pitfall is to use a logical AND ($\wedge$) in a single selection (e.g., $\text{color} = 'red' \wedge \text{color} = 'green'$). This will return an empty set because a single part instance cannot be both red and green simultaneously. Instead, we must find the set of suppliers who supply red parts, find the set of suppliers who supply green parts, and compute the **Set Intersection** ($\cap$) of these two sets.

Expression:

$$\Pi_{\text{sid}} \left(\sigma_{\text{color}='red'}(\text{Catalog} \bowtie \text{Parts}) \right) \cap \Pi_{\text{sid}} \left(\sigma_{\text{color}='green'}(\text{Catalog} \bowtie \text{Parts}) \right)$$

4. Suppliers who supply every part

Working Principle: The keyword “every” (or “all”) in database queries is the classic indicator for the **Division** ($\div$) operator. The division operator $A \div B$ returns all entities in A that are associated with *every* entity in B . We divide the (sid, pid) pairs from the **Catalog** relation by the universe of all pids from the **Parts** relation.

Expression:

$$\Pi_{\text{sid, pid}}(\text{Catalog}) \div \Pi_{\text{pid}}(\text{Parts})$$

Q12. Consider the following schema:

```
EMPLOYEE(EMPLOYEEID, LASTNAME, FIRSTNAME, PHONE, HIREDATE, JOBID, SALARY, DEPARTMENT)
JOB(JOBID, JOBTITLE, MINSALARY, MAXSALARY)
DEPARTMENT(DEPARTMENTID, DEPARTMENTNAME, MANAGERID)
```

Write relational algebra expressions for the following:

- List the average salary of each department.
- List the FIRSTNAME and JOBTITLE of employee ‘John’.
- Increase all salaries more than 10000 by 5% and less than 10000 by 4%.
- Delete all employees from the EMPLOYEE table who earn less than 1000 as salary.

(20 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

Relational Schema Visualization

To better understand the relationships between the entities, we visualize the schema. We assume standard foreign key mappings based on the attribute names (e.g., **EMPLOYEE**.**JOBID** references **JOB**.**JOBID**, and **EMPLOYEE**.**DEPARTMENT** references **DEPARTMENT**.**DEPARTMENTID**).

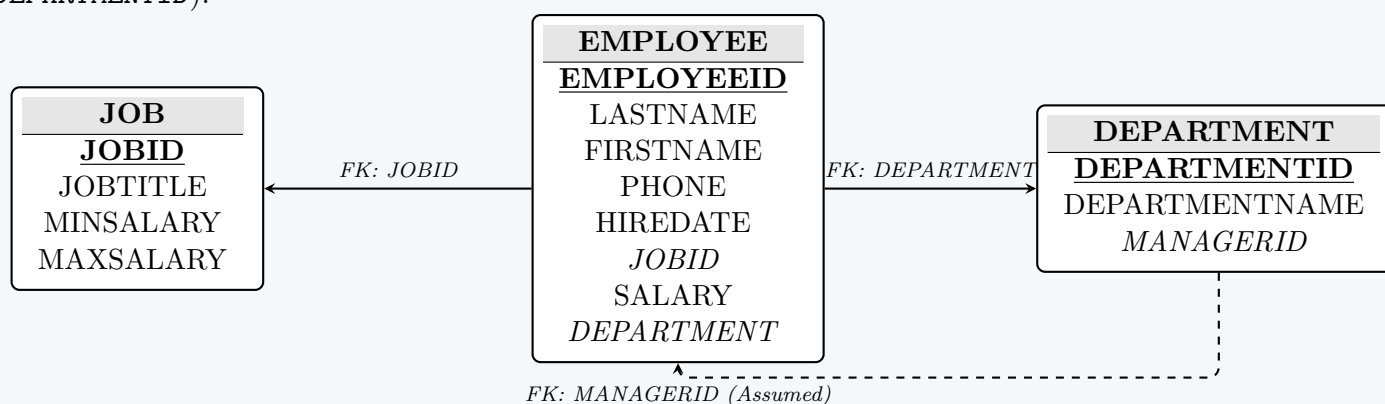


Figure 1: Schema diagram showing Primary Keys (bold and underlined) and Foreign Keys (italicized and linked via arrows).

Relational Algebra Expressions

While basic Relational Algebra (RA) is primarily a query language (for data retrieval), Data Manipulation Language (DML) operations such as **Update** and **Delete** can be represented theoretically using the **Assignment operator** ($\leftarrow$) combined with Extended Relational Algebra (like Generalized Projections and Aggregate Functions).

1. List the average salary of each department

Working Principle: To calculate aggregate values over groups, we must use the **Grouping and Aggregation operator** ($\mathcal{G}$) from Extended Relational Algebra. We group the **EMPLOYEE** relation by the **DEPARTMENT** attribute and apply the **AVG** aggregate function to the **SALARY** attribute.

Expression:

$$\text{DEPARTMENT} \mathcal{G}_{\text{AVG}(\text{SALARY}) \rightarrow \text{Avg_Salary}}(\text{EMPLOYEE})$$

2. List the FIRSTNAME and JOBTITLE of employee ‘John’

Working Principle: The **FIRSTNAME** is in the **EMPLOYEE** table, but the **JOBTITLE** is in the **JOB** table. Because both tables share the identically named attribute **JOBID**, we can use a **Natural Join** ($\bowtie$) to combine them. We then apply a **Selection** (σ) for the name ‘John’ and a **Projection** (Π) for the desired columns.

Expression:

$$\Pi_{\text{FIRSTNAME}, \text{JOBTITLE}} \left(\sigma_{\text{FIRSTNAME}='John'}(\text{EMPLOYEE} \bowtie \text{JOB}) \right)$$

3. Increase all salaries > 10000 by 5% and < 10000 by 4%

Working Principle: In relational algebra, an update is expressed by assigning ($\leftarrow$) a newly computed relation back to the original relation. To apply mathematical operations to existing attributes, we use a **Generalized Projection** (Π). Because there are two distinct conditions, we compute the two updated sets separately and combine them using a **Union** ($\cup$). (Note: We assume salaries exactly equal to 10000 receive the 4% increase to ensure no data is lost).

Expression:

$$\begin{aligned} \text{EMPLOYEE} \leftarrow & \Pi_{\text{Attrs}, (\text{SALARY} \times 1.05) \rightarrow \text{SALARY}} \left(\sigma_{\text{SALARY} > 10000}(\text{EMPLOYEE}) \right) \\ & \cup \\ & \Pi_{\text{Attrs}, (\text{SALARY} \times 1.04) \rightarrow \text{SALARY}} \left(\sigma_{\text{SALARY} \leq 10000}(\text{EMPLOYEE}) \right) \end{aligned}$$

Where *Attrs* represents all unmodified attributes of the **EMPLOYEE** relation: **EMPLOYEEID**, **LASTNAME**, **FIRSTNAME**, **PHONE**, **HIREDATE**, **JOBID**, **DEPARTMENT**.

4. Delete all employees who earn less than 1000 as salary

Working Principle: A deletion is logically equivalent to keeping only the tuples that *do not* meet the deletion criteria. Therefore, we redefine the **EMPLOYEE** relation by assigning it the result of a **Selection** (σ) that filters for employees earning 1000 or more. Alternatively, it can be written as a **Set Difference** ($-$).

Expression (Redefinition Method):

$$\text{EMPLOYEE} \leftarrow \sigma_{\text{SALARY} \geq 1000}(\text{EMPLOYEE})$$

Expression (Set Difference Method):

$$\text{EMPLOYEE} \leftarrow \text{EMPLOYEE} - \sigma_{\text{SALARY} < 1000}(\text{EMPLOYEE})$$

Q13. Consider the following relational schema:

```
employee(emp_no, name, office, age)
books(isbn, title, author, publisher)
loan(empno, isbn, date)
```

Answer the following queries in relational algebra:

- Find the names of employees who have borrowed a book published by McGraw-Hill.
- Find the names and ages of employees who have borrowed all books written by Carl Hamacher.
- Find the names of employees who have borrowed more than five different books published by Pearson Education.
- For each publisher, find the names of employees who have borrowed more than five books of that publisher.
- Find the titles and authors of the books that have been borrowed by Peter Hart.

(25 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

Relational Schema Visualization

Let’s visualize the schema first to clarify the relationships. Note that there is a slight naming discrepancy in the prompt (**emp_no** in **employee** vs. **empno** in **loan**). In our relational algebra expressions, we will handle this by explicitly defining the join condition using an Equi-Join ($\bowtie_{\text{emp_no}=\text{empno}}$).

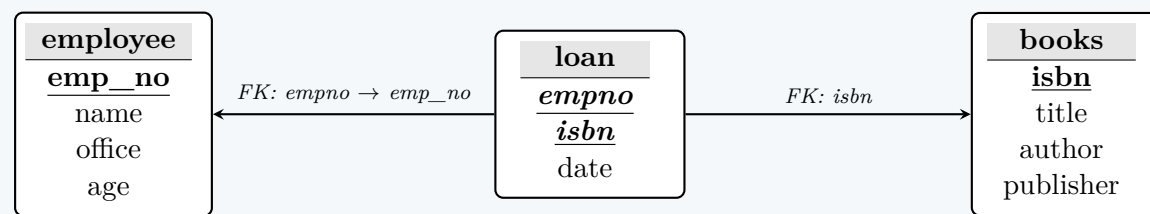


Figure 1: Schema diagram showing Primary Keys (bold and underlined) and Foreign Keys (italicized and linked via arrows).

Relational Algebra Expressions

1. Names of employees who borrowed a book published by McGraw-Hill

Working Principle: We need to join all three tables to connect the **publisher** attribute with the **name** attribute. We filter for books published by 'McGraw-Hill', join with the **loan** table using a Natural Join (on **isbn**), and then join with **employee** using an Equi-Join (to handle the **emp_no** / **empno** naming difference).

Expression:

$$\Pi_{\text{name}} \left(\text{employee} \bowtie_{\text{emp_no}=\text{empno}} \left(\text{loan} \bowtie_{\sigma_{\text{publisher}='McGraw-Hill'}} (\text{books}) \right) \right)$$

2. Names and ages of employees who borrowed all books by Carl Hamacher

Working Principle: The keyword “all” indicates the use of the **Division** ($\div$) operator. We divide the set of all (**empno**, **isbn**) pairs from the **loan** table by the set of **isbns** corresponding to books written by Carl Hamacher. This yields the **empnos** of those who borrowed every such book, which we then join back to the **employee** table.

Expression:

$$\Pi_{\text{name}, \text{age}} \left(\text{employee} \bowtie_{\text{emp_no}=\text{empno}} \left(\Pi_{\text{empno}, \text{isbn}} (\text{loan}) \div \Pi_{\text{isbn}} (\sigma_{\text{author}='Carl Hamacher'} (\text{books})) \right) \right)$$

3. Names of employees who borrowed > 5 different books published by Pearson Education

Working Principle: To count the number of books per employee, we use the **Grouping and Aggregation operator** ($\mathcal{G}$) from Extended Relational Algebra. We first filter for Pearson books, join them with the loans, group the results by **empno**, and count the **isbns**. Finally, we filter groups where the count is greater than 5 and project the name.

Expression:

$$\begin{aligned} L_p &\leftarrow \text{loan} \bowtie_{\sigma_{\text{publisher}='Pearson Education'}} (\text{books}) \\ \text{Result} &= \Pi_{\text{name}} \left(\text{employee} \bowtie_{\text{emp_no}=\text{empno}} \sigma_{\text{count}>5} \left(\text{empno} \mathcal{G}_{\text{COUNT}(\text{isbn}) \rightarrow \text{count}} (L_p) \right) \right) \end{aligned}$$

4. For each publisher, names of employees who borrowed > 5 books of that publisher

Working Principle: Similar to Query 3, but this time we must group by *both* **publisher** and **empno** before applying the aggregate count. This ensures we evaluate the count on a per-publisher, per-employee basis.

Expression:

$$\begin{aligned} G &\leftarrow \sigma_{\text{count}>5} \left(\text{publisher, empno} \mathcal{G}_{\text{COUNT}(\text{isbn}) \rightarrow \text{count}} (\text{loan} \bowtie \text{books}) \right) \\ \text{Result} &= \Pi_{\text{publisher}, \text{name}} \left(G \bowtie_{\text{empno}=\text{emp_no}} \text{employee} \right) \end{aligned}$$

5. Titles and authors of books borrowed by Peter Hart

Working Principle: We filter the **employee** relation for 'Peter Hart', use an Equi-Join to link his **emp_no** to the **loan** table, and then naturally join with **books** to access the **title** and **author** fields for the projection.

Expression:

$$\Pi_{\text{title}, \text{author}} \left(\text{books} \bowtie \left(\text{loan} \bowtie_{\text{empno}=\text{emp_no}} \sigma_{\text{name}='Peter Hart'} (\text{employee}) \right) \right)$$

Q14. Using the schema below, express the following queries using relational algebra:

Emp(eno, ename, title, city), Proj(pno, pname, budget, city)
Works(eno, pno, resp, dur), Pay(title, salary)

- Find names of employees whose salary is more than \$1000.
- List all projects where at least one person with 'Programmer' title is working on it. (A project name may appear more than once.)

(10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

Relational Schema Visualization

Before formulating the queries, it is crucial to map the foreign key dependencies. A subtle but important detail in this schema is that both **Emp** and **Proj** contain an attribute named **city**. We must be careful to avoid accidental joins on this attribute when combining these two tables.

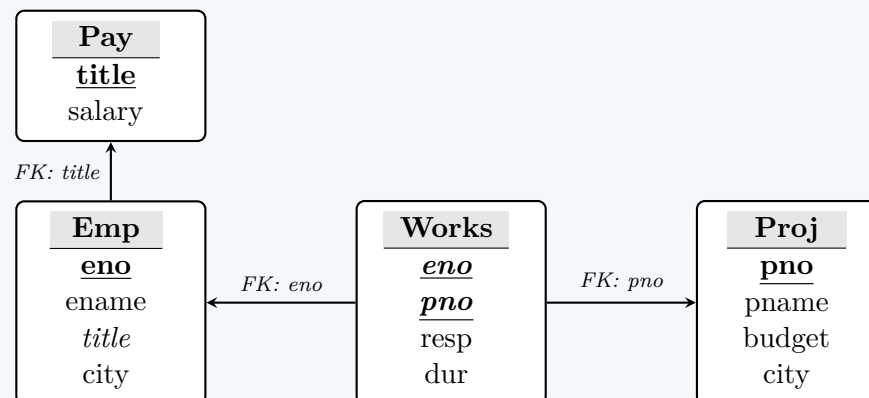


Figure 1: Schema diagram showing Primary Keys (bold and underlined) and Foreign Keys (italicized and linked via arrows).

Relational Algebra Expressions

1. Names of employees whose salary is more than \$1000

Working Principle: We need the **ename** from the **Emp** relation and the **salary** from the **Pay** relation. Because the only identically named attribute between these two relations is **title**, we can safely use a **Natural Join** ($\bowtie$). We then apply a **Selection** (σ) for the salary condition and a **Projection** (Π) for the employee name.

Expression:

$$\Pi_{\text{ename}} \left(\sigma_{\text{salary} > 1000} (\text{Emp} \bowtie \text{Pay}) \right)$$

2. Projects where at least one 'Programmer' is working

Working Principle: This query requires joining **Emp**, **Works**, and **Proj**. A major trap here is the **city** attribute, which exists in both **Emp** and **Proj**. If we simply natural join all three tables ($\text{Emp} \bowtie \text{Works} \bowtie \text{Proj}$), the system will inadvertently force $\text{Emp.city} = \text{Proj.city}$, returning only programmers who live in the same city where the project is located.

To prevent this, we proactively project out the unneeded **city** attributes *before* the join. We filter **Emp** for 'Programmer' and keep only **eno**. We keep only **pno** and **pname** from **Proj**. Then we safely natural join the results with **Works**.

Expression:

$$\Pi_{\text{pname}} \left(\Pi_{\text{pno}, \text{pname}} (\text{Proj}) \bowtie \text{Works} \bowtie \Pi_{\text{eno}} \left(\sigma_{\text{title} = \text{'Programmer'}} (\text{Emp}) \right) \right)$$

Note on duplicates: The prompt mentions "(A project name may appear more than once.)" Standard relational algebra operates on mathematical sets and automatically eliminates duplicate rows during projection. To explicitly retain duplicates, bag/multiset semantics (Extended Relational Algebra) would be required, though standard Π is universally accepted for logical representation.

Q15. Using the schema below, express the following queries using relational algebra:

```

Product(pid, name, price, category, year, maker_cid)
Purchase(buyer_ssn, seller_ssn, store, pid)
Company(cid, name, stock_price, country)
Person(ssn, name, phone_number, city)
  
```

- Find names of people who bought Indian products and live in Gulshan.
- Find people who bought stuff from Alam or bought products from a company whose stock price is more than BDT 2500.

(8 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

Relational Schema Visualization

To correctly formulate our queries, we first visualize the schema to identify primary keys, foreign keys, and potential attribute name collisions. Notably, the **name** attribute appears in **Person**, **Product**, and **Company**, which means we must avoid blind natural joins across these tables.

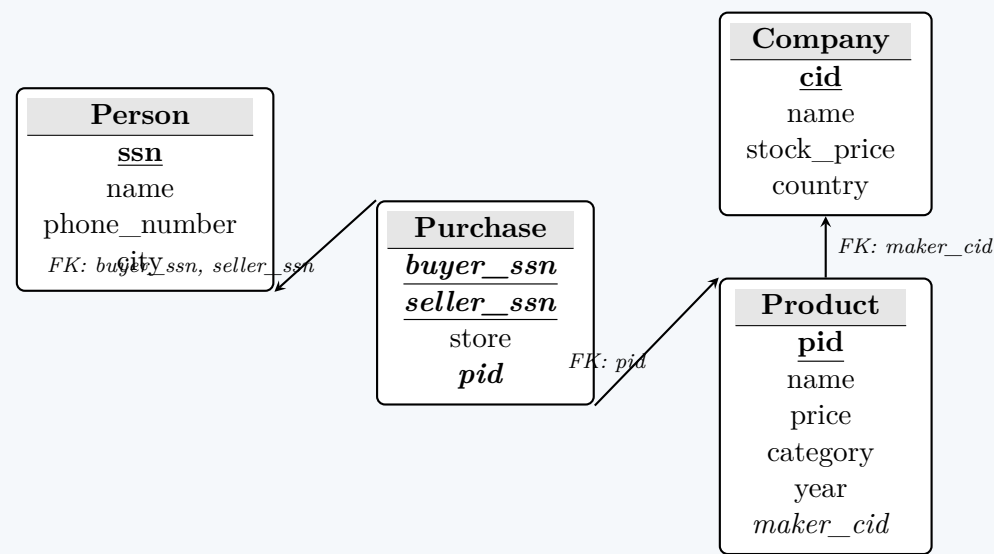


Figure 1: Schema diagram showing Primary Keys (bold and underlined) and Foreign Keys (italicized and linked via arrows).

Relational Algebra Expressions

1. Names of people who bought Indian products and live in Gulshan

Working Principle: We need to join **Person** (for the buyer's city and name), **Purchase** (to link buyer to product), **Product** (to link product to maker), and **Company** (for the country). Because **Person**, **Product**, and **Company** all have an attribute called **name**, we must use explicit **Equi-Joins** ($\bowtie_{\theta}$) instead of Natural Joins to prevent the database from mistakenly trying to match a person's name to a company's name.

Expression:

$$\Pi_{\text{Person.name}} \left(\sigma_{\text{city}='Gulshan'}(\text{Person}) \bowtie_{\text{ssn}=\text{buyer_ssn}} \left(\text{Purchase} \bowtie_{\text{Purchase.pid}=\text{Product.pid}} \left(\text{Product} \bowtie_{\text{maker_cid}=\text{cid}} \sigma_{\text{country}='India'}(\text{Company}) \right) \right) \right)$$

2. People who bought stuff from Alam or bought products from a company whose stock price is > 2500

Working Principle: The keyword “or” signals that we are looking for the combination of two distinct sets of people. The cleanest way to handle complex parallel conditions in relational algebra is to use the **Union** ($\cup$) operator. We will find the **buyer_ssn**s for the first condition, find the **buyer_ssn**s for the second condition, union them together, and finally join the result back to the **Person** table to retrieve the full details of those individuals.

(a) **Set 1 (Buyers from Alam):** Find the SSNs of buyers who purchased from a seller named Alam.

$$B_{\text{Alam}} = \Pi_{\text{buyer_ssn}} \left(\text{Purchase} \bowtie_{\text{seller_ssn}=\text{ssn}} \sigma_{\text{name}='Alam'}(\text{Person}) \right)$$

(b) **Set 2 (Buyers of high-stock products):** Find the SSNs of buyers who purchased a product made by a company with a stock price > 2500.

$$B_{\text{HighStock}} = \Pi_{\text{buyer_ssn}} \left(\text{Purchase} \bowtie_{\text{Purchase.pid}=\text{Product.pid}} \left(\text{Product} \bowtie_{\text{maker_cid}=\text{cid}} \sigma_{\text{stock_price}>2500}(\text{Company}) \right) \right)$$

(c) **Final Result:** Union the two sets of buyer SSNs and join with **Person**.

$$\text{Result} = \text{Person} \bowtie_{\text{ssn}=\text{buyer_ssn}} \left(B_{\text{Alam}} \cup B_{\text{HighStock}} \right)$$

Joins

Q1. Explain the application of left, right and full outer joins with examples. (15 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

An **Outer Join** is an extension of the inner join that retains un-matched rows from one or both participating relations. Outer joins are heavily used in data analysis, reporting, and data migration to ensure that no critical data is lost when cross-referencing tables. To demonstrate the application of Left, Right, and Full Outer Joins, we will use the following two sample relations: **Employee** (Table A) and **Department** (Table B).

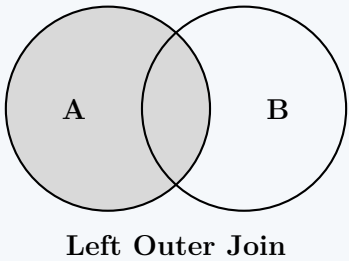
Employee (Table A)			Department (Table B)	
EmpID	EmpName	DeptID	DeptID	DeptName
1	Alice	10	10	HR
2	Bob	20	20	IT
3	Charlie	NULL	30	Sales

1. Left Outer Join (LEFT JOIN)

Definition: Returns all records from the left table (Table A), and the matched records from the right table (Table B). The result is NULL from the right side if there is no match.

Relational Algebra: $A \bowtie B$

Application: Used when we need a complete dataset from the primary (left) table, regardless of whether a related entity exists in the secondary table. For instance, generating a list of *all* employees, including those who have not yet been assigned a department (e.g., new hires).



```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employee E
LEFT JOIN Department D ON E.DeptID = D.DeptID;
```

Result:

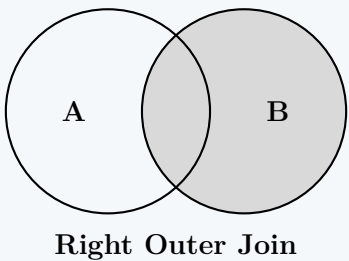
EmpID	EmpName	DeptName
1	Alice	HR
2	Bob	IT
3	Charlie	NULL

2. Right Outer Join (RIGHT JOIN)

Definition: Returns all records from the right table (Table B), and the matched records from the left table (Table A). The result is NULL from the left side when there is no match.

Relational Algebra: $A \subset \bowtie B$

Application: Used when the secondary (right) table acts as the authoritative master list for the query. For example, generating a report of *all* departments and identifying which employees work in them, specifically highlighting departments that currently have no employees (e.g., newly created departments).



```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employee E
RIGHT JOIN Department D ON E.DeptID = D.DeptID;
```

Result:

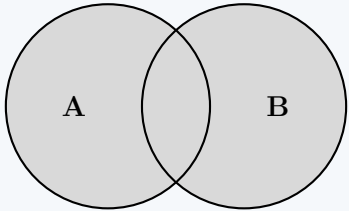
EmpID	EmpName	DeptName
1	Alice	HR
2	Bob	IT
NULL	NULL	Sales

3. Full Outer Join (FULL OUTER JOIN)

Definition: Returns all records when there is a match in either the left or right table. It combines the results of both Left and Right Outer Joins.

Relational Algebra: $A \sqcup B$

Application: Used in data auditing, reconciliation, and integration processes. For instance, comparing systems during a merger to find *every* employee and *every* department, identifying both unassigned employees and empty departments in a single query.



Full Outer Join

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employee E
FULL OUTER JOIN Department D ON E.DeptID = D.DeptID;
```

Result:

EmpID	EmpName	DeptName
1	Alice	HR
2	Bob	IT
3	Charlie	NULL
NULL	NULL	Sales

Summary Comparison

Join Type	Preserved Table	Unmatched Left Rows?	Unmatched Right Rows?
Inner Join	None (Intersection only)	No	No
Left Join	Left	Yes	No
Right Join	Right	No	Yes
Full Join	Both	Yes	Yes

Q2. Distinguish between ‘cross join’, ‘natural join’ and ‘outer joins’. When (left or right) outer joins are useful? Give an example. (10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Distinguishing Cross Join, Natural Join, and Outer Joins

To understand how relational tables are combined, we must distinguish between fundamental join types based on their matching logic and how they handle unmatched rows.

- Cross Join (Cartesian Product):** Combines every row of the first table with every row of the second table. It does not evaluate any matching condition. If Table *A* has *M* rows and Table *B* has *N* rows, the Cross Join results in exactly $M \times N$ rows. *Notation:* $A \times B$
- Natural Join:** Automatically matches rows between two tables based on the equality of *all* columns that share the same name. It strictly returns only the matching rows (intersection) and removes duplicate columns from the result. Unmatched rows are discarded. *Notation:* $A \bowtie B$
- Outer Joins:** An extension of inner/natural joins designed to prevent data loss. Outer joins return all matched rows between the two tables, but they also *preserve* unmatched rows from one or both tables by padding the missing attributes with **NULL** values.

Natural Join
(Intersection Only)

Left Outer Join
(Preserves A)

Right Outer Join
(Preserves B)

Comparison Table

Feature	Cross Join	Natural Join	Outer Joins
Matching Condition	None	Implicit (Shared columns)	Explicit (ON clause)
Preserves Unmatched?	N/A	No	Yes (Left, Right, or Both)
Result Set Size	$M \times N$	$\leq \min(M, N)$	$\geq \max(M, N)$
Duplicate Columns	Yes	No	Yes (Unless abstracted)

2. When are Outer Joins Useful?

Left and Right Outer Joins are fundamentally asymmetric. They are extremely useful in practical database applications for the following scenarios:

- (a) **Preservation of Primary Information:** When generating reports, you often need a complete list from a primary entity (e.g., all Students), along with related data (e.g., Course Enrollments). An outer join ensures that students who are not enrolled in any courses are not accidentally excluded from the report.
- (b) **Finding "Orphan" Records (Anomaly Detection):** Outer joins are the standard method for finding records in one table that have no corresponding record in another. By filtering for NULL values in the joined columns, you can quickly find anomalies, such as Customers with zero Orders, or Departments with zero Employees.
- (c) **Hierarchical Data Extraction:** When dealing with optional relationships (0..1 to N), outer joins safely retrieve parent records without mandating the existence of child records.

3. Example of a Left Outer Join

Consider a company database where we want to list **all** employees and the department they work in. Some new employees have not been assigned to a department yet.

Table A: Employee		
EmpID	Name	DeptID
101	Alice	1
102	Bob	2
103	Charlie	NULL

Table B: Department	
DeptID	DeptName
1	Engineering
2	Sales
3	HR

If we use a **Natural Join**, Charlie will be missing from the results. To preserve Charlie, we use a **Left Outer Join**:

```
SELECT Employee.EmpID, Employee.Name, Department.DeptName
FROM Employee
LEFT OUTER JOIN Department
  ON Employee.DeptID = Department.DeptID;
```

Query Result:

EmpID	Name	DeptName
101	Alice	Engineering
102	Bob	Sales
103	Charlie	NULL

Explanation: The Left Outer Join guarantees that every row from the **Employee** table (the "left" table) is included. Because Charlie has no matching **DeptID** in the **Department** table, the database simply inserts a NULL value for his **DeptName**, rather than discarding his record.

Q3. Define natural joins and outer joins. (7 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Natural Join

Definition: A **Natural Join** is a specific type of inner join that automatically evaluates the equality of *all* attributes that have the same name in both participating relations. It returns only the tuples that have matching values in these common attributes.

Relational Algebra Notation: $R \bowtie S$

Working Principle & Properties:

- **Implicit Condition:** Unlike a Theta Join ($\bowtie_\theta$) or Equi-Join, the join condition is not explicitly stated. The DBMS automatically identifies attributes with identical names in both schemas.
- **Duplicate Elimination:** The resulting relation contains only one copy of each common attribute. If relation R has attributes

(A, B, C) and S has (C, D, E), the schema of $R \bowtie S$ is (A, B, C, D, E).

- **Intersection:** It strictly acts as an intersection operation regarding the tuples; if a tuple in R does not find a match in S based on the common attributes, it is discarded from the result.
- **Degeneration:** If there are no common attributes between R and S , the natural join degrades into a **Cross Join (Cartesian Product)**, yielding $R \times S$.

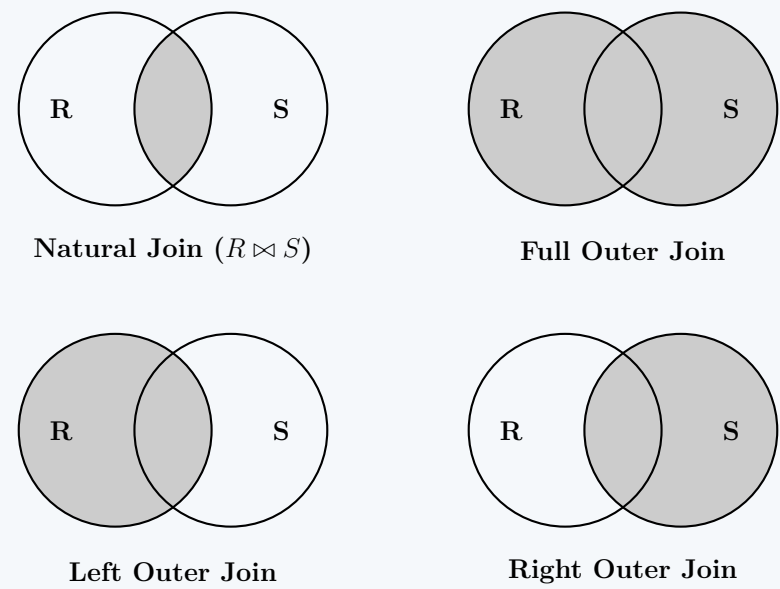
2. Outer Joins

Definition: An **Outer Join** is an extension of the inner join operation that not only returns the matching tuples but also **preserves unmatched tuples** from one or both of the participating relations. Attributes of the unmatched tuples that belong to the other relation are populated with **NULL** values.

Types of Outer Joins:

- (a) **Left Outer Join ($R \ltimes S$):** Preserves *all* tuples from the left relation (R). If a tuple in R has no match in S , the output row will contain the values from R and NULLs for all attributes of S .
- (b) **Right Outer Join ($R \Join S$):** Preserves *all* tuples from the right relation (S). If a tuple in S has no match in R , the output row will contain the values from S and NULLs for all attributes of R .
- (c) **Full Outer Join ($R \Join S$):** Preserves all tuples from *both* relations. It is essentially the union of a Left Outer Join and a Right Outer Join. Unmatched tuples from either side are padded with NULLs.

Visual Representation (Venn Diagrams)



Comparison Table

Property	Natural Join	Outer Joins
Handling of Unmatched Tuples	Discarded entirely from the result set.	Preserved and padded with NULL values.
Join Condition	Implicit (equality of common attributes).	Explicit (using ON clause) or Implicit.
Duplicate Columns	Removed automatically.	Retained (unless specified otherwise).
Primary Use Case	Finding strict intersections of entities (e.g., enrolled students).	Preserving parent records even if no child exists (e.g., all students, including un-enrolled).

Constraints, Assertions & Triggers

Q1. Consider the following schema:

```
Student(Ssn, Name), TakenCourses(Ssn, CourseNo)
Register(Ssn, CourseNo), Course(CourseNo, Name, Year, Term)
PreRequisite(CourseNo, ReqCourseNo)
```

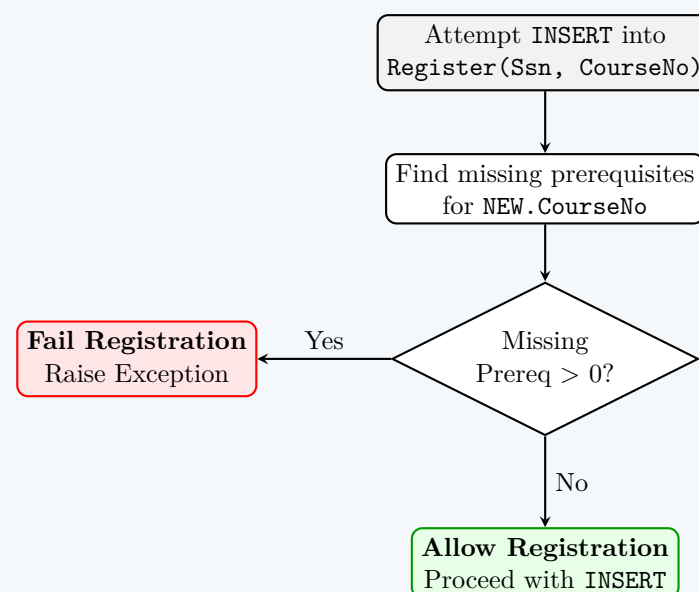
Table **TakenCourses** manages courses a student has previously taken. Write a trigger to check if a student has taken the prerequisite courses when he/she registers for a new course. If a prerequisite course has not been taken, the registration should fail and an appropriate message should be displayed. **(10 marks)** [CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Problem Analysis

To maintain database integrity, we must ensure that a student cannot register for a course without having completed its prerequisites. This requires an **Active Database Rule** (Trigger) that intercepts any attempt to insert a new tuple into the **Register** table. The trigger must execute **BEFORE** the **INSERT** operation. For each new registration attempt for a specific **CourseNo** and **Ssn**, the database must check if there exists any **ReqCourseNo** in the **PreRequisite** table that does *not* appear in the **TakenCourses** table for that same **Ssn**.

2. Trigger Execution Flow



3. SQL Implementation (Oracle PL/SQL)

The following trigger is written using Oracle PL/SQL, which is standard for academic Database Management Systems courses. It utilizes row-level tracking via the **:NEW** correlation name.

```

CREATE OR REPLACE TRIGGER Check_Prerequisites
BEFORE INSERT ON Register
FOR EACH ROW
DECLARE
    missing_prerequisites INT;
BEGIN
    -- Count how many prerequisite courses the student has NOT taken
    SELECT COUNT(*)
    INTO missing_prerequisites
    FROM PreRequisite P
    WHERE P.CourseNo = :NEW.CourseNo
    AND P.ReqCourseNo NOT IN (
        SELECT T.CourseNo
        FROM TakenCourses T
        WHERE T.Ssn = :NEW.Ssn
    );

    -- If the count is greater than 0, the student is missing prerequisites
    IF missing_prerequisites > 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Registration Failed: Student ' || :NEW.Ssn ||
            ' has not completed all prerequisites for course ' || :NEW.CourseNo
        );
    END IF;
END;
/
  
```

4. Explanation of Trigger Components

- **BEFORE INSERT ON Register:** The trigger fires before the database actually writes the new row to the **Register** table. This allows us to intercept and cancel the transaction if necessary.
- **FOR EACH ROW:** This specifies that the trigger is a row-level trigger. It will evaluate the condition for every individual registration record being inserted, allowing the use of the **:NEW** pseudo-record.

- **:NEW.CourseNo & :NEW.Ssn:** These refer to the values of the course and student ID being passed in the current INSERT statement.
- **Subquery Logic:** The query cross-references the `PreRequisite` table with the `TakenCourses` table. If a required course is not found in the student's completed courses, it contributes to the `missing_prerequisites` count.
- **RAISE_APPLICATION_ERROR:** This built-in procedure halts the execution of the trigger, rolls back the attempted INSERT, and returns a custom, user-friendly error message to the application or user.

Q2. What is the difference between a UNIQUE key and a PRIMARY KEY? (3 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

Both `UNIQUE` constraints and `PRIMARY KEY` constraints are used to enforce entity integrity and ensure data uniqueness within a database table. However, they serve fundamentally different roles in relational database design.

1. Definitions

- **PRIMARY KEY:** A column (or a combination of columns) that uniquely identifies each row in a table. It enforces entity integrity by requiring values to be both unique and non-null. A table can have **only one** primary key.
- **UNIQUE Key:** A constraint that ensures all values in a column (or a combination of columns) are distinct across the table. Unlike a primary key, it allows for `NULL` values (depending on the DBMS), and a single table can have **multiple** unique constraints.

2. Key Structural Differences

A. Handling of NULL Values

- **PRIMARY KEY:** Implicitly contains a `NOT NULL` constraint. It can never accept a `NULL` value because a `NULL` represents an unknown state, which violates the requirement to uniquely identify a specific tuple.
- **UNIQUE Key:** Allows `NULL` values. In most relational database management systems (such as PostgreSQL, Oracle, and MySQL), a `UNIQUE` column can accept multiple `NULL` values because `NULL` is treated as mathematically unequal to another `NULL` ($NULL \neq NULL$). However, some systems or configurations allow only a single `NULL` value.

B. Cardinality per Table

- **PRIMARY KEY:** A table is restricted to exactly **one** primary key. This primary key serves as the default target identifier for foreign key relationships from other tables.
- **UNIQUE Key:** A table can have **multiple** `UNIQUE` keys defined across different columns or composite column sets to enforce distinct alternative attributes (e.g., `Email`, `SocialSecurityNumber`, `PhoneNumber`).

C. Default Indexing Mechanism

- **PRIMARY KEY:** Automatically creates a **Clustered Index** (in systems like Microsoft SQL Server and MySQL InnoDB). This physically organizes the row data on disk according to the key sequence.
- **UNIQUE Key:** Automatically creates a **Non-Clustered Index** by default. The physical arrangement of data remains unaffected; instead, a separate pointer structure is built to enforce uniqueness efficiently.

3. Comparison Summary Table

Feature	PRIMARY KEY	UNIQUE Key
Purpose	Identifies a record uniquely.	Prevents duplicate values in a column.
Quantity per Table	Strictly one .	Multiple constraints allowed.
Nullability	Throws an error on <code>NULL</code> .	Allows <code>NULL</code> values by default.
Index Type (Default)	Clustered Index.	Non-Clustered Index.
Foreign Key Target	Standard choice for foreign keys.	Can be targeted by a foreign key.

4. Example Usage

Consider a `UserAccount` table design where a relational architect needs to track primary identity along with alternative unique attributes:

```
CREATE TABLE UserAccount (
  UserID INT PRIMARY KEY,           -- Cannot be NULL, only one per table
  GovernmentID VARCHAR(20) UNIQUE,  -- Can be NULL, must be unique if present
  EmailAddress VARCHAR(255) UNIQUE NOT NULL, -- Unique alternative, explicit NOT NULL
  FullName VARCHAR(100)
);
```

Q3. Consider the following relational database schema:

`Product(maker, model, type), PC(model, speed, ram, hd, price),`
`Laptop(model, speed, ram, hd, screen, price), Printer(model, color, type, price)`

The *Product* relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. model numbers are unique over all manufacturers and product types. The *PC* relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The *Laptop* relation is similar, except that the screen size (in inches) is also included. The *Printer* relation records for each printer model whether the printer produces color output ('T', if so; 'F' otherwise), the process type (laser or ink-jet, typically), and the price. Write necessary constraints to implement the following (answer each independently):

- speed, ram, hd, screen and price cannot be negative.
- A model of a product must also be the model of a PC, a laptop, or a printer.
- A laptop with a screen size less than 15 inches must have at least a 40 GB hard disk or sell for less than \$1000.
- No manufacturer of PCs may also make laptops.
- If a laptop has a larger main memory than a PC, then the laptop must also have a higher price than the PC.
- A manufacturer of a PC also makes a laptop with at least as great a processor speed.

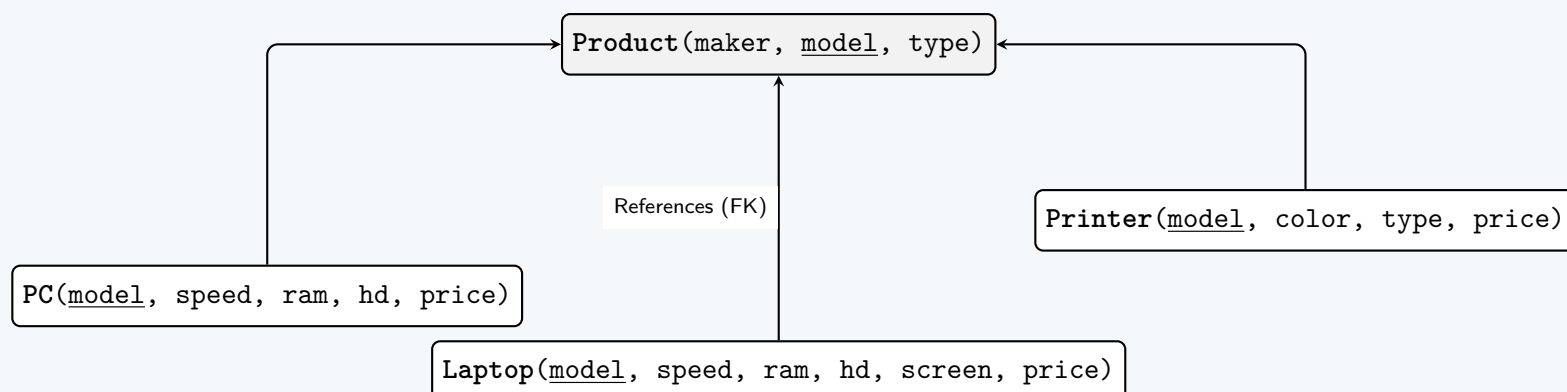
(24 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

Relational Schema Visualization

Before implementing the constraints, we can visualize the database schema. The relations follow a standard inheritance (IS-A) structure where PC, Laptop, and Printer act as specialized relations referencing the core Product relation via the `model` primary key.



Implementation of Integrity Constraints

In standard SQL, constraints can be formulated using either **Table-Level CHECK constraints** (for intra-table conditions) or **Database-Level ASSERTIONS** (for complex, cross-table rules).

(a) Constraint 1: Non-negative attributes

- Type:** Tuple-level (Table-level) Check Constraint.
- Explanation:** This requires simple CHECK clauses attached to the schema definition of the PC, Laptop, and Printer tables.

```
-- Applied to the PC table:
ALTER TABLE PC ADD CONSTRAINT chk_pc_positive
CHECK (speed >= 0 AND ram >= 0 AND hd >= 0 AND price >= 0);

-- Applied to the Laptop table:
ALTER TABLE Laptop ADD CONSTRAINT chk_laptop_positive
CHECK (speed >= 0 AND ram >= 0 AND hd >= 0 AND screen >= 0 AND price >= 0);

-- Applied to the Printer table:
```



```
ALTER TABLE Printer ADD CONSTRAINT chk_printer_positive
CHECK (price >= 0);
```

(b) **Constraint 2: Total Participation / Specialization**

- **Type:** Database-level ASSERTION.
- **Explanation:** A model found in the `Product` table must exist in at least one of the specialized tables. This ensures the generalization hierarchy is total. We assert that there exists no model in `Product` that is absent from all three sub-tables.

```
CREATE ASSERTION Product_Is_Specialized CHECK (
  NOT EXISTS (
    SELECT model FROM Product
    WHERE model NOT IN (SELECT model FROM PC)
      AND model NOT IN (SELECT model FROM Laptop)
      AND model NOT IN (SELECT model FROM Printer)
  )
);
```

(c) **Constraint 3: Conditional Laptop Specifications**

- **Type:** Tuple-level CHECK Constraint.
- **Explanation:** The requirement translates to the logical implication:
 $(\text{screen} < 15) \implies (\text{hd} \geq 40 \vee \text{price} < 1000)$.
 Using the logical equivalence $P \implies Q \equiv \neg P \vee Q$, this becomes:
 $(\text{screen} \geq 15) \vee (\text{hd} \geq 40) \vee (\text{price} < 1000)$.

```
ALTER TABLE Laptop ADD CONSTRAINT chk_laptop_specs
CHECK (screen >= 15 OR hd >= 40 OR price < 1000);
```

(d) **Constraint 4: Mutually Exclusive Manufacturers**

- **Type:** Database-level ASSERTION.
- **Explanation:** The set of makers who produce PCs must have a null intersection with the set of makers who produce laptops.

```
CREATE ASSERTION No_PC_Laptop_Maker CHECK (
  NOT EXISTS (
    SELECT maker FROM Product WHERE type = 'PC'
    INTERSECT
    SELECT maker FROM Product WHERE type = 'Laptop'
  )
);
```

(e) **Constraint 5: Cross-Table Dependency (RAM and Price)**

- **Type:** Database-level ASSERTION.
- **Explanation:** The implication is: If `Laptop.ram > PC.ram`, then `Laptop.price > PC.price`. We enforce this by ensuring that there is no combination of a Laptop and a PC where the Laptop has strictly more RAM but a price less than or equal to the PC.

```
CREATE ASSERTION Laptop_PC_RAM_Price CHECK (
  NOT EXISTS (
    SELECT * FROM Laptop L, PC P
    WHERE L.ram > P.ram AND L.price <= P.price
  )
);
```

(f) **Constraint 6: Manufacturer PC-Laptop Speed Dependency**

- **Type:** Database-level ASSERTION.
- **Explanation:** For every PC made by a manufacturer, there must exist a Laptop made by the *same* manufacturer such that the Laptop's speed is greater than or equal to the PC's speed. We trigger a violation if we can find a PC whose manufacturer does **not** also make a Laptop with an adequate speed.

```
CREATE ASSERTION PC_Maker_Laptop_Speed CHECK (
  NOT EXISTS (
    SELECT *
    FROM Product P1
```



```

        JOIN PC ON P1.model = PC.model
        WHERE NOT EXISTS (
            SELECT *
            FROM Product P2
            JOIN Laptop L ON P2.model = L.model
            WHERE P1.maker = P2.maker AND L.speed >= PC.speed
        )
    )
);

```

Q4. Consider the following table definition:

```

CREATE TABLE MyTable(
    ...
    head INT UNIQUE,
    ...
);

```

Replace the above UNIQUE constraint using a CHECK constraint. (15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Replacing a UNIQUE Constraint with a CHECK Constraint

In relational database theory and the SQL standard, a **UNIQUE** constraint enforces that no two tuples in a relation share the same non-null value for a specified attribute. To replicate this exact behavior using a **CHECK** constraint, we must utilize a **subquery** that counts the occurrences of the specific **head** value within the table and ensures the count never exceeds 1.

SQL Implementation

The equivalent table definition using a **CHECK** constraint is formulated as follows:

```

CREATE TABLE MyTable(
    -- ... other attributes ...
    head INT,
    -- ... other attributes ...

    CONSTRAINT chk_unique_head CHECK (
        1 >= (SELECT COUNT(*)
            FROM MyTable M
            WHERE M.head = head)
    )
);

```

Working Principle

- When a new tuple is inserted (or updated), the **CHECK** constraint evaluates the condition.
- The subquery (`SELECT COUNT(*) FROM MyTable M WHERE M.head = head`) counts how many rows currently exist in the table where the **head** value matches the **head** value of the row being processed.
- The constraint ensures this count is ≤ 1 . If a second row attempts to insert the same value, the count evaluates to 2, the condition `1 >= 2` becomes **FALSE**, and the transaction is aborted.

Important Notes on RDBMS Compatibility

While the above query is the correct theoretical equivalent according to the ANSI SQL standard, it is important to note the practical limitations in modern database management systems:

- **Standard SQL:** Fully supports subqueries inside **CHECK** constraints.
- **Commercial RDBMS:** Major systems like **PostgreSQL**, **MySQL**, **Oracle**, and **SQL Server** explicitly *forbid* the use of subqueries in **CHECK** constraints due to the massive performance overhead required to execute a table scan or index lookup on every single **INSERT** or **UPDATE**.
- **Practical Workaround:** In real-world applications where **UNIQUE** cannot be used, database administrators enforce this logic using **Triggers** (e.g., `BEFORE INSERT OR UPDATE`) or **Database-level Assertions** (if supported).

Q5. Consider the following two schemas:

Product(maker, model, type), Laptop(model, speed, ram, hd, screen, price)

Write necessary triggers to implement the following constraints:

- speed, ram, hd, screen and price cannot be negative.
- When a Laptop model is deleted, that model must also be deleted from the Product table.

(12 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Implementation of Constraints using Triggers

Triggers are procedural codes automatically executed in response to certain events on a particular table or view in a database. They are highly effective for enforcing complex business rules and data integrity constraints that cannot be managed by standard `CHECK` constraints or foreign keys alone.

Below are the SQL implementations (using standard SQL/MySQL compliant syntax) for the given constraints.

1. Enforcing Non-Negative Attributes

Requirement: The attributes `speed`, `ram`, `hd`, `screen`, and `price` in the `Laptop` table cannot be negative.

Trigger Design: We require a `BEFORE INSERT OR UPDATE` trigger. By evaluating the data *before* it is written to the disk, we can intercept negative values and abort the transaction.

```
DELIMITER //
```

```
CREATE TRIGGER trg_laptop_non_negative
BEFORE INSERT ON Laptop
FOR EACH ROW
BEGIN
    IF (NEW.speed < 0 OR NEW.ram < 0 OR NEW.hd < 0
        OR NEW.screen < 0 OR NEW.price < 0) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Violation: Laptop attributes cannot be negative.';
    END IF;
END //
```

```
CREATE TRIGGER trg_laptop_non_negative_update
BEFORE UPDATE ON Laptop
FOR EACH ROW
BEGIN
    IF (NEW.speed < 0 OR NEW.ram < 0 OR NEW.hd < 0
        OR NEW.screen < 0 OR NEW.price < 0) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Violation: Laptop attributes cannot be negative.';
    END IF;
END //
```

```
DELIMITER ;
```

Working Principle:

- **Timing (BEFORE):** Ensures the data is validated prior to modifying the table.
- **Transition Variable (NEW):** The `NEW` pseudo-record holds the incoming tuple data for inserts and updates.
- **Action (SIGNAL):** Raises a custom exception (`SQLSTATE '45000'` is designated for unhandled user-defined exceptions) if any condition is violated, which rolls back the triggering DML statement.

2. Cascading Deletion to Product Table

Requirement: When a `Laptop` model is deleted, that same model must also be deleted from the `Product` table.

Trigger Design: We require an `AFTER DELETE` trigger. Once a laptop record is successfully removed, the trigger fires to delete the corresponding tuple in the parent `Product` relation.

```
DELIMITER //
```

```
CREATE TRIGGER trg_laptop_delete_cascade
AFTER DELETE ON Laptop
FOR EACH ROW
BEGIN
    -- Delete the corresponding product using the OLD model reference
    DELETE FROM Product
    WHERE model = OLD.model;
END //
```

```
DELIMITER ;
```

Working Principle:

- **Timing (AFTER):** Executes immediately following the successful deletion of the row from the `Laptop` table.
- **Transition Variable (OLD):** The `OLD` pseudo-record contains the values of the tuple just before it was deleted.
- **Action (DELETE):** Executes a standard `DELETE` statement on the `Product` relation targeting the matching `model` identifier.

Notes

- In standard database normalization, the relationship between **Product** and **Laptop** typically utilizes a Foreign Key on **Laptop(model)** referencing **Product(model)** with an **ON DELETE CASCADE** constraint flowing from **Product** to **Laptop**. The scenario described here enforces an *inverse* dependency mapping via a trigger.
- Standard **CHECK** constraints are generally preferred over triggers for the non-negative conditions (Constraint 1) because they are computationally cheaper. Triggers are used here as explicitly requested by the problem constraints.

Q6. What is the difference between referential integrity constraint and foreign key constraint? What are the three different policies for UPDATE and DELETE that can be mentioned for maintaining referential integrity? Among these, which one is more suitable for UPDATE, and which one is more suitable for DELETE? Why? **(10 marks)** [CSE 215 | L-2/T-2 | 2019–20, 2015–16]

Answer

1. Referential Integrity vs. Foreign Key Constraint

While often used interchangeably in practice, there is a fundamental theoretical distinction between **Referential Integrity** and a **Foreign Key Constraint**.

- **Referential Integrity Constraint (The Principle):** This is a *relational database theory concept*. It dictates that any relational dependency must be valid. Specifically, if a tuple in one relation references a tuple in another relation, the referenced tuple must actually exist. It is a property of the database state.
- **Foreign Key Constraint (The Implementation):** This is the *physical SQL mechanism* used by a Database Management System (DBMS) to enforce referential integrity. It is an explicit rule defined in the schema (using the **FOREIGN KEY** clause) that binds a column (or set of columns) in a child table to a primary key or unique key in a parent table.

Feature	Referential Integrity	Foreign Key Constraint
Nature	Theoretical concept / Logical rule	Physical schema definition
Scope	Database state and data consistency	Table-level object
Purpose	To prevent orphaned records	The tool used to achieve the rule

Department(dept_id, dept_name)

Employee(emp_id, name, dept_id)

Maintains Referential Integrity

Foreign Key Constraint

2. Maintenance Policies for UPDATE and DELETE

When a referenced tuple (in the parent table) is updated or deleted, the DBMS must handle the dependent tuples (in the child table) to avoid violating referential integrity. The SQL standard defines three primary compensatory policies:

(a) **RESTRICT (or NO ACTION):**

- **Working Principle:** The DBMS rejects the UPDATE or DELETE operation on the parent table if there exists at least one dependent row in the child table.
- **Result:** The operation is aborted, and an error is raised.

(b) **CASCADE:**

- **Working Principle:** The DBMS automatically propagates the change. If a parent’s key is updated, the child’s foreign key is updated to the new value. If a parent is deleted, all dependent child rows are also deleted.
- **Result:** Cascading changes keep the relationship intact automatically or prune orphaned data.

(c) **SET NULL:**

- **Working Principle:** If a parent tuple is updated or deleted, the foreign key attributes in all dependent child tuples are explicitly set to **NULL** (provided the column allows **NULL** values).
- **Result:** The child rows are preserved but are no longer linked to a specific parent.

3. Suitability Analysis

Most Suitable for UPDATE: CASCADE

Why? Primary keys should ideally be immutable, but in real-world scenarios (e.g., merging systems, standardizing IDs), they occasionally need to be modified. When a parent record’s identifier changes, the actual real-world entity hasn’t been destroyed, it merely has a new label. Using **ON UPDATE CASCADE** ensures that all child records logically follow the parent to its new identifier without requiring manual, multi-step queries, thus preserving the semantic relationship seamlessly.

Most Suitable for DELETE: RESTRICT (or SET NULL)**Why?**

- **Use RESTRICT (Default Choice):** This is the safest and most generally applicable policy. Accidental deletion of a parent record (e.g., a `Department`) should not silently wipe out hundreds of dependent child records (e.g., `Employees`). `RESTRICT` forces the user to explicitly handle or reassign child records before deleting a parent, preventing catastrophic data loss.
- **Use SET NULL (Alternative):** If the child entity can exist independently of the parent (e.g., deleting a specific `Project` simply means an `Employee` is temporarily unassigned, not fired), `SET NULL` safely preserves the child data while breaking the link.
- **Avoid CASCADE:** `ON DELETE CASCADE` is highly dangerous and is typically strictly reserved for **Weak Entities** (e.g., deleting an `Order` cascades to `Order_Items`), where the child has absolutely no logical existence without the parent.

Q7. Consider the schema `MovieExec(name, address, cert#, netWorth)`. We want to prevent the average net worth of movie executives from dropping below \$50,000. Write a trigger that checks this constraint before each update to the `netWorth` column and, if violated, sets the value of `netWorth` to the minimum amount required to satisfy the constraint. **(10 marks)** [CSE 215 | L-2/T-2 | 2017–18]

Answer**Trigger Implementation for Average Constraint**

To enforce the business rule that the average `netWorth` must not fall below \$50,000, we require a `BEFORE UPDATE` trigger. If a proposed update reduces the average below the threshold, the trigger dynamically modifies the incoming `NEW.netWorth` value to the mathematical minimum required to maintain exactly a \$50,000 average.

1. Mathematical Formulation

Let the current state of the database be defined by:

- N : Total number of executives.
- S_{current} : Current sum of all `netWorth` values.
- v_{old} : The current net worth of the executive being updated (`OLD.netWorth`).
- v_{new} : The proposed new net worth (`NEW.netWorth`).

The sum of the net worth of all *other* executives is:

$$S_{\text{other}} = S_{\text{current}} - v_{\text{old}}$$

If the update proceeds, the new average will be:

$$\text{New Average} = \frac{S_{\text{other}} + v_{\text{new}}}{N}$$

We require the new average to be at least 50,000:

$$\frac{S_{\text{other}} + v_{\text{new}}}{N} \geq 50000$$

$$v_{\text{new}} \geq (50000 \times N) - S_{\text{other}}$$

If v_{new} is strictly less than this boundary, we must override it with the minimum required value:

$$v_{\text{required}} = (50000 \times N) - S_{\text{other}}$$

2. SQL Implementation

The following implementation uses a standard procedural SQL syntax (e.g., PostgreSQL PL/pgSQL or MySQL compliant logic).

```
DELIMITER //

CREATE TRIGGER trg_maintain_avg_networth
BEFORE UPDATE ON MovieExec
FOR EACH ROW
BEGIN
    DECLARE total_execs INT;
    DECLARE current_total_sum DECIMAL(15,2);
    DECLARE others_sum DECIMAL(15,2);
    DECLARE min_required DECIMAL(15,2);

    -- Only calculate if the net worth is actually decreasing
    IF NEW.netWorth < OLD.netWorth THEN

        -- Retrieve the aggregate data
        SELECT COUNT(*), SUM(netWorth)
        INTO total_execs, current_total_sum
        FROM MovieExec;
```

```
-- Calculate the sum of everyone except the row being updated
SET others_sum = current_total_sum - OLD.netWorth;

-- Calculate the mathematical minimum required
SET min_required = (50000 * total_execs) - others_sum;

-- Override NEW.netWorth if it falls below the minimum required
IF NEW.netWorth < min_required THEN
    SET NEW.netWorth = min_required;
END IF;

END IF;
END //
```

DELIMITER ;

3. Working Principle

- **Timing (BEFORE):** The trigger intercepts the UPDATE operation *before* it is committed to the disk, allowing us to inspect and modify the incoming values.
- **Optimization:** The IF NEW.netWorth < OLD.netWorth condition prevents the trigger from doing unnecessary table scans if the executive’s net worth is increasing (which could only raise the average, not lower it).
- **Assignment:** By setting NEW.netWorth = min_required, the DBMS silently changes the user’s requested update value to the mathematically safe value, ensuring the aggregate constraint holds true without aborting the transaction entirely.

4. Note on DBMS Specifics (The Mutating Table Problem)

In a strict textbook environment, the logic above perfectly satisfies the theoretical requirements. However, in practical DBMS environments like **Oracle** or **MySQL**, querying the exact same table that is currently firing a row-level trigger results in a **Mutating Table Error**. To implement this in those specific systems:

- **Oracle:** Would require a **Compound Trigger** or an Autonomous Transaction to track the sums before the row-level execution.
- **PostgreSQL:** Natively supports this logic without raising a mutating table error, making it the closest fit for the provided SQL standard syntax.

Q8. What is a Materialized View? Discuss four instances when a trigger can be applied. **(10 marks)** [CSE 303 | L-3/T-1 | 2013–14]
↪ Similar: 2014–15: “What is a Materialized View? Discuss the restrictions on triggers in brief.”

Answer

1. Materialized View

Definition

A **Materialized View** is a database object that contains the actual results of a query. Unlike a standard view, which is merely a saved SQL query (virtual) executed on-the-fly every time it is accessed, a materialized view executes the query and *physically stores the resulting data on disk* like a standard table.

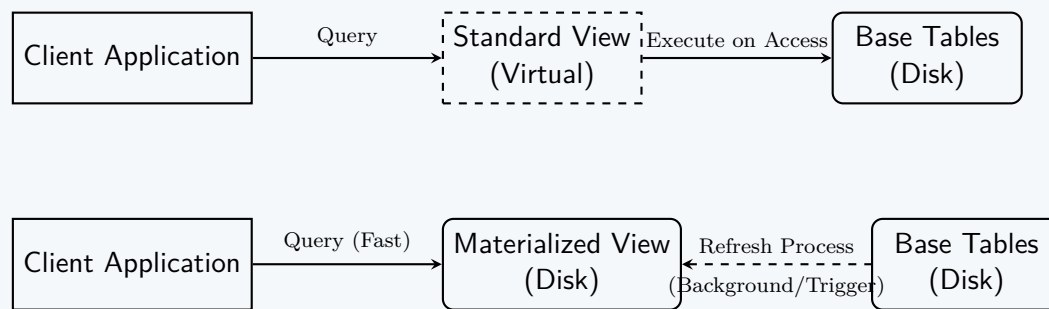
Working Principle

Because the data is pre-computed and stored, querying a materialized view is significantly faster than executing the complex joins and aggregations on the base tables. However, this introduces the problem of **staleness**. When the underlying base tables change, the materialized view must be updated using a process called **View Maintenance** or **Refresh**. This refresh can be:

- **Immediate (Synchronous):** Refreshed automatically as part of the transaction modifying the base table.
- **Deferred (Asynchronous):** Refreshed periodically (e.g., nightly) or manually on demand.

Comparison: Standard View vs. Materialized View

Feature	Standard View	Materialized View
Storage	Does not consume physical disk space (only query text is saved).	Consumes physical disk space to store the result set.
Performance	Slower (computes on the fly).	Very fast (data is pre-computed).
Data Freshness	Always strictly up-to-date.	May be stale depending on the refresh interval.
Primary Use Case	Security, simplifying complex queries, logical abstraction.	Data warehousing (OLAP), dashboards, high-read environments.



2. Four Instances When a Trigger Can Be Applied

Triggers are procedural objects fired automatically in response to specific database events. Four common real-world instances for their application include:

- Implementing Complex Business Rules and Integrity Constraints:** Standard `CHECK` constraints cannot reference other tables or complex aggregation states. A trigger can enforce cross-table integrity, such as preventing a withdrawal if an account balance (calculated from a ledger table) falls below zero, or ensuring a specific scheduling constraint is not violated.
- Maintaining Audit Trails (Logging):** Triggers are heavily used for security and compliance. An `AFTER UPDATE` or `AFTER DELETE` trigger can capture the old values, the new values, the user ID, and the timestamp, automatically inserting this record into a dedicated `Audit_Log` table without relying on the application code to do so.
- Synchronous Replication and Derived Data Maintenance:** If a database requires an instantaneous update to a summarized value, a trigger is applied. For example, maintaining a real-time `total_inventory` column in a `Products` table by firing triggers whenever a row in the `Shipments` or `Sales` tables is inserted or deleted.
- Enforcing Complex Security and Authorization Policies:** Triggers can restrict operations based on context rather than standard user privileges. For example, a `BEFORE INSERT` trigger can abort a transaction if a user attempts to insert payroll data outside of normal business hours (e.g., on a weekend) or from an unauthorized IP subnet.

3. Restrictions on Triggers (Variant Query)

While powerful, triggers operate within the transaction space of the triggering statement, necessitating several restrictions to maintain ACID properties:

- No Transaction Control Statements:** Inside a standard trigger, statements like `COMMIT`, `ROLLBACK`, or `SAVEPOINT` are strictly forbidden. The trigger executes as part of the statement that fired it; committing inside the trigger would prematurely commit the entire transaction. (*Note: Autonomous transactions are an exception in some DBMS*).
- The Mutating Table Restriction:** In row-level triggers (e.g., `FOR EACH ROW`), the trigger generally cannot read or modify the exact table that is currently being modified by the DML statement that fired it. Doing so raises a "Mutating Table Error" because the table is in an inconsistent, intermediate state.
- Cascading Limits:** To prevent infinite loops (e.g., Table A updates Table B, which triggers an update on Table A), the DBMS sets a strict cap on the depth of nested triggers (typically 16 to 32 levels).
- Data Definition Language (DDL) Limitations:** In many systems, triggers cannot execute structural schema changes (e.g., `CREATE TABLE`, `ALTER TABLE`) directly inside a standard DML row-level trigger.

Q9. What are the different policies of maintaining referential integrity? Discuss them with examples. **(10 marks)** [CSE 303 | L-3/T-1 | 2012–13]

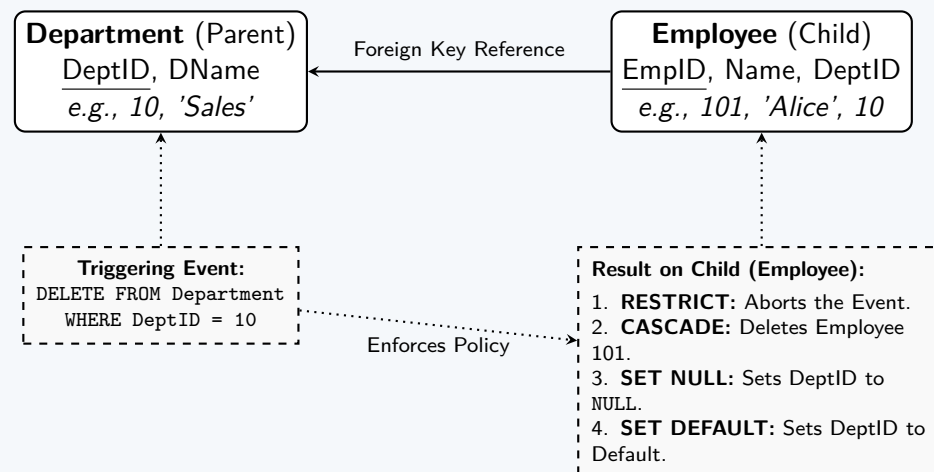
Answer

Policies for Maintaining Referential Integrity

Referential Integrity ensures that relationships between tables remain consistent. When a user attempts to `UPDATE` or `DELETE` a primary key in a parent table that is currently referenced by a foreign key in a child table, the Database Management System (DBMS) must enforce a **maintenance policy** to prevent orphaned records.

The SQL standard defines four primary policies for handling these operations: **RESTRICT**, **CASCADE**, **SET NULL**, and **SET DEFAULT**.

Visualizing Referential Integrity Policies



Detailed Discussion of Policies

(a) RESTRICT (or NO ACTION)

- **Working Principle:** The DBMS checks if there are any dependent child records. If at least one matching child record exists, the UPDATE or DELETE operation on the parent is **blocked/aborted**, and an error is raised.
- **Example:** If a user tries to delete the *Sales* department (DeptID = 10), but an employee named *Alice* is currently assigned to DeptID 10, the database rejects the deletion.
- **Application:** Used when child records must never be accidentally orphaned or deleted. It is the default policy in most relational databases.

(b) CASCADE

- **Working Principle:** The DBMS automatically propagates the change from the parent to the child.
 - ON UPDATE CASCADE: Updating the parent's primary key automatically updates the child's foreign key to the new value.
 - ON DELETE CASCADE: Deleting the parent record automatically deletes all referencing child records.
- **Example:** If the *Sales* department is deleted, the database automatically deletes *Alice's* employee record. If the DeptID is updated from 10 to 99, *Alice's* DeptID changes to 99.
- **Application:** Ideal for **weak entities** where the child cannot logically exist without the parent (e.g., deleting an *Order* should delete its *Order_Items*).

(c) SET NULL

- **Working Principle:** If the parent record is modified or deleted, the dependent foreign key fields in the child records are set to NULL.
- **Example:** If the *Sales* department is deleted, *Alice* is not deleted; instead, her DeptID becomes NULL (indicating she is currently unassigned).
- **Application:** Used when the child entity has an independent existence but the relationship is optional (e.g., an employee can temporarily be without a department). The foreign key column must be defined to allow NULL values.

(d) SET DEFAULT

- **Working Principle:** When the parent record is deleted or updated, the foreign key in the child record is set to a predefined default value specified during table creation.
- **Example:** If the default DeptID is 0 (an "Unassigned" pool) and the *Sales* department is deleted, *Alice's* DeptID automatically changes to 0.
- **Application:** Useful for maintaining historical data or assigning orphaned records to a default holding state.

SQL Implementation Example

The policies are implemented during table creation using the FOREIGN KEY constraint syntax:

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(100),
    DeptID INT DEFAULT 0,

    -- Defining the referential integrity policies
    CONSTRAINT fk_dept
        FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
        ON DELETE SET NULL      -- Policy for deletion
        ON UPDATE CASCADE      -- Policy for updates
);
```

Comparison Table		
Policy	Action on DELETE	Action on UPDATE
RESTRICT	Aborts parent deletion.	Aborts parent PK update.
CASCADE	Deletes matching child rows.	Updates child FK to match new PK.
SET NULL	Child FK becomes NULL.	Child FK becomes NULL.
SET DEFAULT	Child FK becomes default value.	Child FK becomes default value.

Q10. What are the differences between UNIQUE and PRIMARY KEY constraints in SQL? (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

Differences Between PRIMARY KEY and UNIQUE Constraints

Both PRIMARY KEY and UNIQUE constraints are mechanisms used in relational databases to enforce uniqueness across a column or a set of columns. However, they serve fundamentally different logical purposes in database design.

1. Core Conceptual Differences

- PRIMARY KEY (Entity Integrity):** Its primary purpose is to **uniquely identify** every tuple (row) within a relation. By definition, an identifier cannot be unknown or missing, hence it strictly forbids NULL values.
- UNIQUE Constraint (Alternate Keys):** Its purpose is to ensure that non-primary key attributes (candidate keys) do not contain duplicate data. Because it is not the primary identifier of the record, it generally permits NULL values.

2. Comparison Summary

Feature	PRIMARY KEY	UNIQUE Constraint
Nullability	Cannot accept NULL values.	Can accept NULL values.*
Quantity Limit	Maximum of one per table.	Multiple constraints allowed per table.
Index Creation	Typically creates a Clustered Index (DBMS dependent).	Typically creates a Non-Clustered Index .
Foreign Key Reference	Standard target for Foreign Keys.	Can also be referenced by Foreign Keys.
Logical Role	Primary identifier of the entity.	Ensures uniqueness of alternate properties.

*Note: In standard SQL (e.g., PostgreSQL, Oracle), multiple NULLs are allowed because $NULL \neq NULL$. However, some systems like SQL Server restrict a UNIQUE column to exactly one NULL value.

3. Visual Representation

Exactly ONE per table.
No NULLs allowed.

EmpID (PK)
101
102
103

Multiple per table allowed.
NULLs are permitted.

Email (UNIQUE)
alice@mail.com
bob@mail.com
NULL

Name
Alice
Bob
Charlie

4. SQL Implementation Example

The following SQL script demonstrates how both constraints are typically applied during schema definition.

```
CREATE TABLE Employee (  
    -- The PRIMARY KEY constraint uniquely identifies the row  
    EmpID INT PRIMARY KEY,  
  
    Name VARCHAR(100) NOT NULL,  
  
    -- The UNIQUE constraint ensures no two employees share an email  
    -- However, an employee without an email can have a NULL value  
    Email VARCHAR(150) UNIQUE,  
  
    -- A second UNIQUE constraint (allowed)  
    PassportNo VARCHAR(20) UNIQUE  
);
```

Q11. Give an example of a CHECK constraint and discuss why it is used. (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

The CHECK Constraint

Definition

A **CHECK** constraint is a declarative integrity constraint used in SQL to specify a logical Boolean condition that must be satisfied by all tuples (rows) in a relation. For a data modification operation (**INSERT** or **UPDATE**) to succeed, the condition defined in the **CHECK** clause must evaluate to **TRUE** or **UNKNOWN** (in the case of **NULLs**). If the condition evaluates to **FALSE**, the DBMS rejects the operation and aborts the transaction.

Example Implementation

Consider an **Employee** table where business rules dictate that employees must be adults ($\text{age} \geq 18$), their salaries must be strictly positive, and they must hold recognized roles.

```
CREATE TABLE Employee (  
  EmpID INT PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Age INT,  
  Salary DECIMAL(10,2),  
  Role VARCHAR(20),  
  
  -- 1. Numeric Range Check  
  CONSTRAINT chk_emp_age CHECK (Age >= 18),  
  
  -- 2. Positive Value Check  
  CONSTRAINT chk_emp_salary CHECK (Salary > 0),  
  
  -- 3. Set Membership (Domain) Check  
  CONSTRAINT chk_emp_role CHECK (Role IN ('Manager', 'Developer', 'Analyst'))  
);
```

Visualizing the Working Principle

```
graph LR  
  A["Input Tuple:  
(Bob, Age: 25)"] --> B{"CHECK  
(Age >= 18)"}  
  B -- TRUE --> C["Database Table  
(Row Inserted)"]  
  
  D["Input Tuple:  
(Alice, Age: 16)"] --> E{"CHECK  
(Age >= 18)"}  
  E -- FALSE --> F["Transaction  
Aborted (Error)"]
```

Why is it Used?

The **CHECK** constraint is primarily used to enforce **Domain Integrity** and basic business rules at the database level. Its core advantages include:

- **Data Quality Assurance:** It prevents invalid, out-of-range, or illogical data from entering the database (e.g., negative prices, invalid email formats, or future birthdates).
- **Application Independence:** By enforcing rules in the database schema, the integrity of the data is guaranteed regardless of which front-end application, API, or user is inserting the data.
- **Performance Optimization:** Database-level declarative checks are generally highly optimized and faster than writing complex programmatic triggers or enforcing the logic on the application server.
- **Self-Documenting Schema:** Explicitly defining allowed domains makes the database schema easier to read and understand for future developers and database administrators.

Important Note on NULL Values

In SQL, conditional operations involving **NULL** evaluate to **UNKNOWN** rather than **FALSE**. Because a **CHECK** constraint only blocks an operation if the condition evaluates to exactly **FALSE**, a **CHECK** constraint will implicitly allow **NULL** values to be inserted unless a **NOT NULL** constraint is also explicitly defined on the column.

Q12. A database schema consists of four relations:

```
Product(maker, model, type)  
PC(model, speed, ram, hd, price)  
Laptop(model, speed, ram, hd, screen, price)  
Printer(model, color, type, price)
```

(a) Write the following check constraints (using the PC/Laptop/Printer schema):

- (i) The model number of a product must be a 7-digit number.
- (ii) A PC with a processor speed less than 2.0 GHz must have at least 512 MB RAM or sell for less than \$800.

(5 marks)

[CSE 303 | L-3/T-1 | 2011–12]

(b) Write the following assertions:

- (i) The price of a laptop must be larger than the average price of PCs made by the same manufacturer.
- (ii) A manufacturer of a PC must also make Laptop(s).
- (iii) If a PC has a greater processor speed than a laptop then the PC must also have a higher price than the laptop.

(15 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

Part 1: Tuple-Level CHECK Constraints

Tuple-level CHECK constraints are applied directly within the table definition or via ALTER TABLE statements. They validate data within a single row at the time of insertion or update.

(a) **Model Number Length Constraint**

- **Table:** Product
- **Logic:** The model number must be exactly a 7-digit number. Depending on the data type, this can be enforced using mathematical bounds (for integers) or string pattern matching (for VARCHAR/CHAR). We provide the standard integer bound logic.

```
ALTER TABLE Product ADD CONSTRAINT chk_model_length
CHECK (model BETWEEN 1000000 AND 9999999);

-- Note: If model is a string, use pattern matching:
-- CHECK (model LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9]')
```

(b) **PC Specification Implication Constraint**

- **Table:** PC
- **Logic:** The requirement is an implication: $(\text{speed} < 2.0) \implies (\text{ram} \geq 512 \vee \text{price} < 800)$. Using the logical equivalence $P \implies Q \equiv \neg P \vee Q$, the constraint translates directly to: $(\text{speed} \geq 2.0) \vee (\text{ram} \geq 512) \vee (\text{price} < 800)$.

```
ALTER TABLE PC ADD CONSTRAINT chk_pc_specs
CHECK (speed >= 2.0 OR ram >= 512 OR price < 800);
```

Part 2: Database-Level ASSERTIONS

Assertions are independent database schema objects that evaluate to a boolean condition across multiple tables. If an assertion evaluates to FALSE, the DBMS blocks the modifying transaction. We implement these using standard SQL NOT EXISTS clauses to search for violations of the rule.

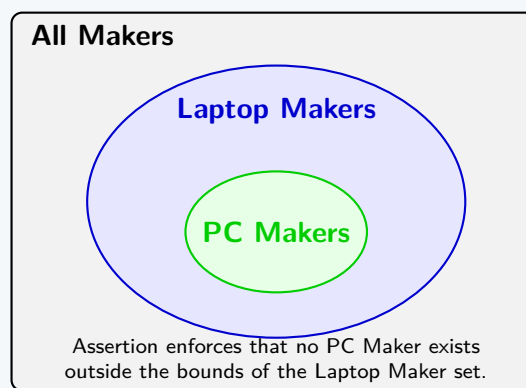
(a) **Laptop Price vs. Maker's Average PC Price**

- **Logic:** We must prevent the existence of any laptop whose price is less than or equal to the average price of all PCs manufactured by the same maker. This requires joining Laptop with Product to identify the maker, and calculating an aggregate subquery for the maker's PCs.

```
CREATE ASSERTION Assert_Laptop_Price_Vs_Avg_PC CHECK (
  NOT EXISTS (
    SELECT *
    FROM Laptop L
    JOIN Product PL ON L.model = PL.model
    WHERE L.price <= (
      -- Subquery to find the average PC price for this specific maker
      SELECT AVG(P.price)
      FROM PC P
      JOIN Product PP ON P.model = PP.model
      WHERE PP.maker = PL.maker
    )
  )
);
```

(b) **PC Maker must also make Laptops**

- **Logic:** Formally, the set of manufacturers who make PCs must be a **subset** of the set of manufacturers who make Laptops. We trigger a violation if the set difference (EXCEPT) between PC makers and Laptop makers is not empty.



```
CREATE ASSERTION Assert_PC_Maker_Makes_Laptop CHECK (
  NOT EXISTS (
    SELECT maker FROM Product WHERE type = 'PC'
    EXCEPT
    SELECT maker FROM Product WHERE type = 'Laptop'
  )
);
```

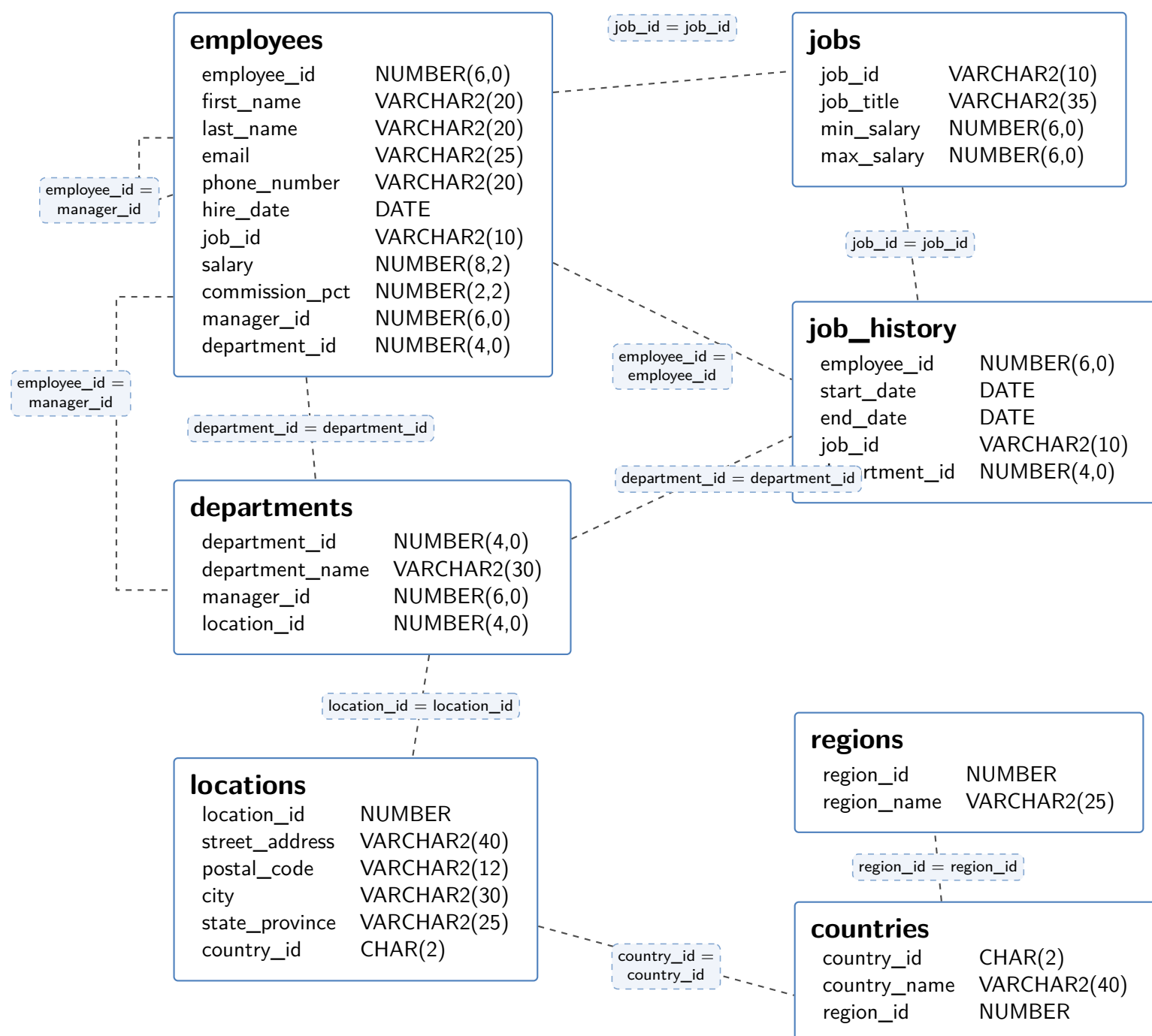
(c) **Speed-to-Price Consistency across PCs and Laptops**

- **Logic:** If any PC has a greater processor speed than any Laptop, the PC must have a higher price. The violation condition is finding a pairing where $(PC.speed > Laptop.speed) \wedge (PC.price \leq Laptop.price)$. We search the Cartesian product of PC and Laptop for this violation.

```
CREATE ASSERTION Assert_Speed_Price_Consistency CHECK (
  NOT EXISTS (
    SELECT *
    FROM PC, Laptop
    WHERE PC.speed > Laptop.speed
      AND PC.price <= Laptop.price
  )
);
```

Functions and Procedures

- Q1.** Write a procedure named `IncreaseOrDecrease` which takes an employee id as input. If the salary of the employee is greater than the average salaries of all departments except their own, but less than the average salary of their own department, the procedure increases the salary by 10%. Otherwise it decreases by 5%. During execution it prints “Increasing Salary” or “Decreasing Salary” accordingly.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

Procedure Implementation

Based on the provided schema diagram, the data types (such as `VARCHAR2` and `NUMBER`) strongly indicate an Oracle Database environment (commonly known as the Oracle HR schema). Therefore, the procedure is written using **PL/SQL**.

The logic requires us to:

- Fetch the target employee's `salary` and `department_id`.
- Calculate the average salary of their specific department.
- Calculate the average salary of all employees *not* in their department.
- Evaluate the condition and apply the appropriate `UPDATE` and `DBMS_OUTPUT` print statement.

PL/SQL Code

```

CREATE OR REPLACE PROCEDURE IncreaseOrDecrease (
    p_emp_id IN employees.employee_id%TYPE
)
IS
    v_salary      employees.salary%TYPE;
    v_dept_id     employees.department_id%TYPE;
    v_avg_own_dept NUMBER;
    v_avg_other_dept NUMBER;
BEGIN
    -- 1. Fetch the target employee's current salary and department
    SELECT salary, department_id
    INTO v_salary, v_dept_id
    FROM employees
    WHERE employee_id = p_emp_id;

    -- 2. Calculate the average salary of their own department
    SELECT AVG(salary)
    INTO v_avg_own_dept
  
```



```

FROM employees
WHERE department_id = v_dept_id;

-- 3. Calculate the average salary of all OTHER departments
SELECT AVG(salary)
INTO v_avg_other_dept
FROM employees
WHERE department_id != v_dept_id
  OR (v_dept_id IS NULL AND department_id IS NOT NULL);

-- 4. Evaluate the condition and update accordingly
IF (v_salary > v_avg_other_dept) AND (v_salary < v_avg_own_dept) THEN

    -- Increase by 10%
    UPDATE employees
    SET salary = salary * 1.10
    WHERE employee_id = p_emp_id;

    DBMS_OUTPUT.PUT_LINE('Increasing Salary');

ELSE

    -- Decrease by 5%
    UPDATE employees
    SET salary = salary * 0.95
    WHERE employee_id = p_emp_id;

    DBMS_OUTPUT.PUT_LINE('Decreasing Salary');

END IF;

-- Commit the transaction
COMMIT;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Employee ID not found. ');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
    ROLLBACK;
END IncreaseOrDecrease;
/

```

Key Implementation Details

- **%TYPE Attribute:** Used for variable declarations to anchor the data types dynamically to the table's schema, ensuring robust code even if the column types change in the future.
- **Handling the Nulls:** The WHERE clause in the third step accounts for potential NULL department IDs, ensuring the logic doesn't break if some employees are unassigned.
- **DBMS_OUTPUT:** Used to print the required text to the console during execution. (Note: SET SERVEROUTPUT ON must be executed in the SQL*Plus/SQL Developer environment to see these messages).
- **Exception Handling:** A basic NO_DATA_FOUND exception block is included to handle cases where an invalid employee_id is passed into the procedure.

Q2. Consider the following schema:

```

Movie(title, year, filmType, inColor, studioId)
Studio(studioId, name, address)

```

Write a stored procedure that takes as input the name of a studio and a year, then counts and prints the number of black and white movies produced by that studio in the specified year. **(10 marks)** [CSE 215 | L-2/T-2 | 2017–18]

Answer

Stored Procedure Implementation

Based on standard academic database environments, the following stored procedure is written using **Oracle PL/SQL**. It performs a standard JOIN between the **Movie** and **Studio** tables to filter by the provided studio name and year, counts the results, and prints the output.

Assumptions

Since the exact data type of the `inColor` column is not explicitly defined in the schema (e.g., Boolean, CHAR(1), or VARCHAR), the implementation assumes it stores a single character where 'N' stands for "No" (Black and White) and 'Y' stands for "Yes" (Color).

PL/SQL Code

```
CREATE OR REPLACE PROCEDURE CountBnWMovies (
    p_studio_name IN VARCHAR2,
    p_year        IN NUMBER
)
IS
    v_movie_count NUMBER;
BEGIN
    -- Query to count the specific movies
    SELECT COUNT(*)
    INTO v_movie_count
    FROM Movie m
    JOIN Studio s ON m.studioId = s.studioId
    WHERE s.name = p_studio_name
        AND m.year = p_year
        AND m.inColor = 'N'; -- Assuming 'N' represents False/Black & White

    -- Print the results to the console
    DBMS_OUTPUT.PUT_LINE('---Black and White Movie Report---');
    DBMS_OUTPUT.PUT_LINE('Studio: ' || p_studio_name);
    DBMS_OUTPUT.PUT_LINE('Year: ' || p_year);
    DBMS_OUTPUT.PUT_LINE('Total B&W Movies Produced: ' || v_movie_count);

EXCEPTION
    -- Catch any unexpected errors during execution
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END CountBnWMovies;
/
```

Execution Example

To execute this procedure and see the output in an Oracle environment (like SQL*Plus or SQL Developer), you would run:

```
-- Enable console output
SET SERVEROUTPUT ON;

-- Execute the procedure
EXECUTE CountBnWMovies('Warner Bros.', 1942);
```

Working Principle

- **Parameters:** The procedure takes `p_studio_name` and `p_year` as input arguments.
- **The Query:** It joins `Movie` and `Studio` on their common key (`studioId`). It filters the rows matching the exact input name, input year, and the condition that the movie is not in color.
- **INTO Clause:** The aggregate `COUNT(*)` result is stored directly into the local variable `v_movie_count`.
- **DBMS_OUTPUT:** This built-in Oracle package is used to print the formatted strings and the variable value to the standard output buffer.

Q3. Write constructor functions for the car rental SQL:99 schema (Trucks, Sports car, Bus) and insert statement to insert data into the tables. **(10 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

Object-Relational Schema in SQL:99

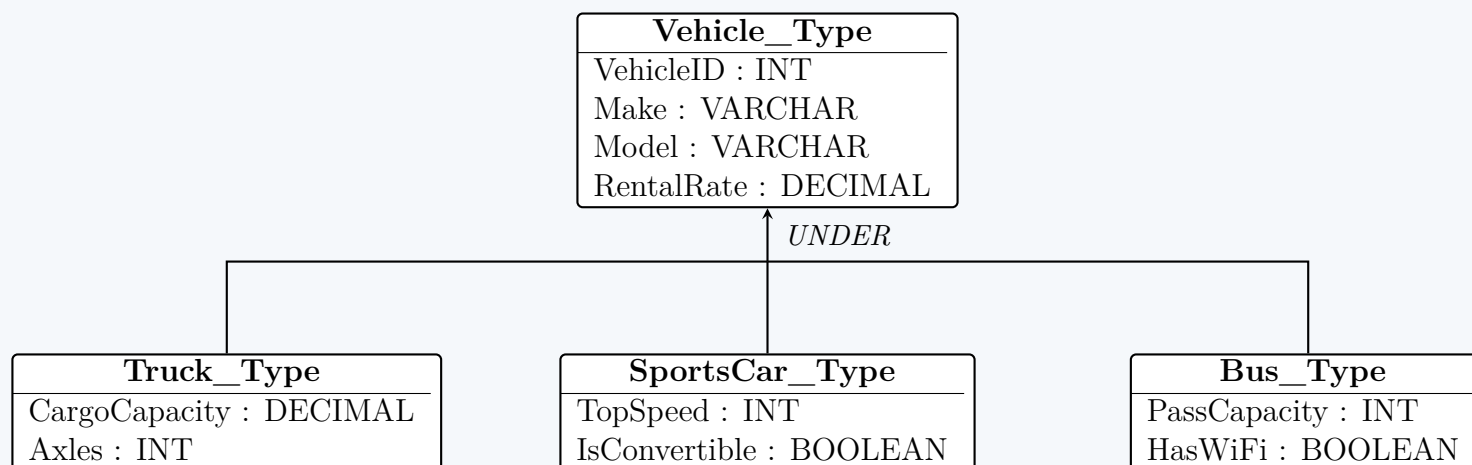
The SQL:99 standard introduced Object-Relational Database (ORDBMS) concepts, allowing for the creation of **User-Defined Types (UDTs)**, inheritance hierarchies, and structured objects. In an ORDBMS schema for a car rental system, vehicles such as Trucks, Sports Cars, and Buses share common attributes but also possess specific, specialized attributes.

1. Core Concepts

- **User-Defined Type (UDT):** A custom data type that encapsulates multiple attributes.
- **Type Inheritance:** Subtypes (e.g., `Truck_Type`) can inherit attributes from a supertype (e.g., `Vehicle_Type`) using the `UNDER` keyword.
- **Constructor Function:** A system-generated function bearing the exact name of the UDT. It is used to instantiate objects of that type by passing parameters corresponding to the type's attributes (including inherited ones).

2. Type Hierarchy and Schema Diagram

Before writing the constructor functions and schema definitions, we design the type hierarchy. The common attributes are encapsulated in the `Vehicle_Type` supertype, while specialized attributes are delegated to the respective subtypes.



3. SQL:99 Schema Definition

The following SQL queries define the types. By defining these UDTs, SQL:99 implicitly generates the constructor functions: $F_{construct}(a_1, a_2, \dots, a_n) \rightarrow Instance$

```

-- 1. Create the Base Supertype
CREATE TYPE Vehicle_Type AS (
    VehicleID INT,
    Make VARCHAR(50),
    Model VARCHAR(50),
    RentalRate DECIMAL(8,2)
) NOT FINAL;

-- 2. Create the Subtypes using UNDER
CREATE TYPE Truck_Type UNDER Vehicle_Type AS (
    CargoCapacity DECIMAL(10,2),
    Axles INT
);

CREATE TYPE SportsCar_Type UNDER Vehicle_Type AS (
    TopSpeed INT,
    IsConvertible BOOLEAN
);

CREATE TYPE Bus_Type UNDER Vehicle_Type AS (
    PassengerCapacity INT,
    HasWiFi BOOLEAN
);

-- 3. Create Typed Tables to hold the objects
CREATE TABLE Trucks OF Truck_Type;
CREATE TABLE SportsCars OF SportsCar_Type;
CREATE TABLE Buses OF Bus_Type;
  
```

4. Constructor Functions and Data Insertion

In SQL:99, an object is instantiated by calling its constructor. Because of inheritance, the constructor signature for a subtype includes the attributes of the supertype followed by its own specific attributes.

- **Truck_Type Constructor Signature:**
`Truck_Type(VehicleID, Make, Model, RentalRate, CargoCapacity, Axles)`
- **SportsCar_Type Constructor Signature:**
`SportsCar_Type(VehicleID, Make, Model, RentalRate, TopSpeed, IsConvertible)`
- **Bus_Type Constructor Signature:**
`Bus_Type(VehicleID, Make, Model, RentalRate, PassengerCapacity, HasWiFi)`

Insertion Statements

To populate the typed tables, we insert the instantiated objects generated by the constructor functions.

```

-- Inserting data into the Trucks table
INSERT INTO Trucks VALUES (
    Truck_Type(101, 'Volvo', 'VNL_860', 250.00, 45000.00, 5)
);
  
```

```

INSERT INTO Trucks VALUES (
    Truck_Type(102, 'Ford', 'F-150', 85.00, 3325.00, 2)
);

-- Inserting data into the SportsCars table
INSERT INTO SportsCars VALUES (
    SportsCar_Type(201, 'Porsche', '911_Turbo_S', 450.00, 205, FALSE)
);

INSERT INTO SportsCars VALUES (
    SportsCar_Type(202, 'Ferrari', 'Portofino_M', 600.00, 199, TRUE)
);

-- Inserting data into the Buses table
INSERT INTO Buses VALUES (
    Bus_Type(301, 'Mercedes-Benz', 'Tourismo', 350.00, 55, TRUE)
);

INSERT INTO Buses VALUES (
    Bus_Type(302, 'Volvo', '9700', 320.00, 48, FALSE)
);

```

5. Properties and Advantages

- **Polymorphism and Substitutability:** If we were to create a table `RentalFleet` with a column of type `Vehicle_Type`, it could accept instances of `Truck_Type`, `SportsCar_Type`, or `Bus_Type` due to substitutability.
- **Relational Mapping:** Using relational algebra, querying the base type table automatically maps to the subtypes. For instance, the relational operation $\Pi_{Make, Model}(Vehicles)$ implicitly spans across all subtype tables if standard inheritance semantics are fully enforced.
- **Encapsulation:** Custom methods (e.g., calculating insurance premiums based on `TopSpeed` or `Axles`) can be bound directly to the user-defined types.

BUET · CSE 215 · Database

Part III

Normalization

Normalization

S4 — Normalization

Functional Dependencies

Q1. Consider a relation with schema $R(A, B, C, D, E)$ and functional dependencies: $A \rightarrow C$, $BE \rightarrow D$, $CD \rightarrow A$, $BC \rightarrow E$, and $AD \rightarrow B$.

(a) Determine whether the following functional dependencies follow from the given FDs: $BC \rightarrow AD$, $AC \rightarrow BD$.

(b) Determine whether the following are keys of the given relation: $\{A, B\}$, $\{A, E\}$.

(10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Given:

- Relation Schema: $R(A, B, C, D, E)$
- Set of Functional Dependencies (FDs), F :

1. $A \rightarrow C$
2. $BE \rightarrow D$
3. $CD \rightarrow A$
4. $BC \rightarrow E$
5. $AD \rightarrow B$

To determine whether a specific functional dependency $X \rightarrow Y$ holds, or whether a set of attributes is a key, we compute the **attribute closure** of the left-hand side (LHS), denoted as X^+ . If $Y \subseteq X^+$, then $X \rightarrow Y$ logically follows from F . If X^+ contains all attributes of the relation schema, then X is a superkey.

(a) Determine whether the following functional dependencies follow from F

1. Does $BC \rightarrow AD$ hold?

We calculate the attribute closure of BC , denoted as $(BC)^+$:

- (a) **Initialization:** $(BC)^+ = \{B, C\}$
- (b) **Apply $BC \rightarrow E$:** Since $\{B, C\} \subseteq (BC)^+$, we add E .
 $(BC)^+ = \{B, C, E\}$
- (c) **Apply $BE \rightarrow D$:** Since $\{B, E\} \subseteq (BC)^+$, we add D .
 $(BC)^+ = \{B, C, D, E\}$
- (d) **Apply $CD \rightarrow A$:** Since $\{C, D\} \subseteq (BC)^+$, we add A .
 $(BC)^+ = \{A, B, C, D, E\}$

Conclusion: Since both A and D are present in $(BC)^+$, we have $\{A, D\} \subseteq (BC)^+$. Therefore, the functional dependency $BC \rightarrow AD$ logically follows from the given FDs.

2. Does $AC \rightarrow BD$ hold?

We calculate the attribute closure of AC , denoted as $(AC)^+$:

- (a) **Initialization:** $(AC)^+ = \{A, C\}$
- (b) **Apply $A \rightarrow C$:** The attribute C is already in the closure.
 $(AC)^+ = \{A, C\}$
- (c) **Check remaining FDs:**
 - $BE \rightarrow D$: LHS $\{B, E\} \not\subseteq \{A, C\}$
 - $CD \rightarrow A$: LHS $\{C, D\} \not\subseteq \{A, C\}$
 - $BC \rightarrow E$: LHS $\{B, C\} \not\subseteq \{A, C\}$
 - $AD \rightarrow B$: LHS $\{A, D\} \not\subseteq \{A, C\}$

No more FDs can be applied. The final closure is $(AC)^+ = \{A, C\}$.

Conclusion: Since $\{B, D\} \not\subseteq \{A, C\}$, Therefore, the functional dependency $AC \rightarrow BD$ does NOT follow from the given FDs.

(b) Determine whether the following are keys of the given relation

A set of attributes K is a **candidate key** for relation R if:

- (a) $K \rightarrow R$ (i.e., $K^+ = \text{All attributes of } R$, making it a superkey).
- (b) No proper subset of K is a superkey (minimality constraint).

1. Is $\{A, B\}$ a key?

First, we calculate the closure of $\{A, B\}$:

- (a) **Initialization:** $(AB)^+ = \{A, B\}$
- (b) **Apply $A \rightarrow C$:** $(AB)^+ = \{A, B, C\}$
- (c) **Apply $BC \rightarrow E$:** Since $\{B, C\} \subseteq (AB)^+$, add E . $(AB)^+ = \{A, B, C, E\}$
- (d) **Apply $BE \rightarrow D$:** Since $\{B, E\} \subseteq (AB)^+$, add D . $(AB)^+ = \{A, B, C, D, E\}$

Since $(AB)^+$ contains all attributes of R , $\{A, B\}$ is a **superkey**.

Now, we check for minimality by testing its proper subsets, $\{A\}$ and $\{B\}$:

- $(A)^+ = \{A, C\}$ (using $A \rightarrow C$). Does not contain all attributes.
- $(B)^+ = \{B\}$ (no FDs apply). Does not contain all attributes.

Conclusion: Since $\{A, B\}$ determines all attributes and none of its proper subsets do, **$\{A, B\}$ is a valid candidate key.**

2. Is $\{A, E\}$ a key?

We calculate the closure of $\{A, E\}$:

- (a) **Initialization:** $(AE)^+ = \{A, E\}$
- (b) **Apply $A \rightarrow C$:** $(AE)^+ = \{A, C, E\}$
- (c) **Check remaining FDs:**
 - $BE \rightarrow D$: $\{B, E\} \not\subseteq \{A, C, E\}$
 - $CD \rightarrow A$: $\{C, D\} \not\subseteq \{A, C, E\}$
 - $BC \rightarrow E$: $\{B, C\} \not\subseteq \{A, C, E\}$
 - $AD \rightarrow B$: $\{A, D\} \not\subseteq \{A, C, E\}$

No more FDs can be applied. The final closure is $(AE)^+ = \{A, C, E\}$.

Conclusion: Since $(AE)^+ \neq \{A, B, C, D, E\}$, the set $\{A, E\}$ cannot determine all attributes of the relation. Therefore, **$\{A, E\}$ is NOT a key.**

Q2. Consider a relation with schema $R(A, B, C, D, E)$ and FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.

- (a) What are all the nontrivial FDs that follow from the given FDs? Restrict yourself to FDs with single attributes on the right sides.
- (b) What are all the keys of R ?
- (c) What are all the super-keys that are not keys?

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer**Given:**

- Relation Schema: $R(A, B, C, D, E)$
- Functional Dependencies (FDs): $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.

(a) Nontrivial FDs with Single Attributes on the Right Side

First, we analyze the given FDs. The dependencies $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ form a cyclic dependency chain. This means that the attributes A, B, C , and D are functionally equivalent, and any one of them can determine the other three. Their attribute closures are:

$$A^+ = B^+ = C^+ = D^+ = \{A, B, C, D\}$$

The attribute E does not appear on the right-hand side of any FD, meaning $E^+ = \{E\}$, and it cannot be determined by any other attribute.

A functional dependency $X \rightarrow Y$ is **nontrivial** if $Y \notin X$. Given the restriction that the right-hand side (RHS) must be a single attribute, $Y \in \{A, B, C, D, E\}$. However, since nothing determines E except E itself (which would be trivial), Y can only be A, B, C , or D .

The set of all nontrivial FDs with a single attribute on the RHS that follow from the given FDs can be defined logically as:

Any FD $X \rightarrow Y$ such that: 1. $Y \in \{A, B, C, D\}$ 2. $X \subseteq \{A, B, C, D, E\}$ 3. $X \cap \{A, B, C, D\} \neq \emptyset$ (LHS contains at least one of A, B, C , or D) 4. $Y \notin X$ (to ensure the FD is nontrivial)

For clarity, the fundamental (irreducible) nontrivial FDs with a single attribute on both LHS and RHS are:

- $A \rightarrow B, \quad A \rightarrow C, \quad A \rightarrow D$
- $B \rightarrow A, \quad B \rightarrow C, \quad B \rightarrow D$
- $C \rightarrow A, \quad C \rightarrow B, \quad C \rightarrow D$
- $D \rightarrow A, \quad D \rightarrow B, \quad D \rightarrow C$

Any other valid FD can be formed by augmenting the left-hand sides of the above FDs with any subset of the remaining attributes (including E), provided the RHS attribute is not in the augmented LHS.

(b) All Keys of R

A set of attributes K is a **candidate key** (or simply a key) if $K^+ = \{A, B, C, D, E\}$ and no proper subset of K satisfies this property.

(a) **Identify Essential Attributes:** Since E never appears on the right side of any functional dependency, it cannot be functionally determined by any other attribute. Therefore, E **must be a part of every candidate key**.

(b) **Test Combinations with E :** We compute the closure of E paired with each of the equivalent attributes from the cycle:

- $\{A, E\}^+ = \{A, B, C, D, E\}$
- $\{B, E\}^+ = \{A, B, C, D, E\}$
- $\{C, E\}^+ = \{A, B, C, D, E\}$
- $\{D, E\}^+ = \{A, B, C, D, E\}$

Since each of these pairs determines all attributes in R , and neither single attribute ($\{A\}, \{B\}, \{C\}, \{D\}$, or $\{E\}$) can determine all attributes in R on its own, they are all minimally sufficient.

Conclusion: The keys of R are exactly:

$$\{A, E\}, \quad \{B, E\}, \quad \{C, E\}, \quad \{D, E\}$$

(c) All Superkeys that are not Keys

A **superkey** is any set of attributes that contains a key. To find the superkeys that are *not* candidate keys, we must find all attribute combinations that contain one of the keys identified in part (b) but have additional extraneous attributes (i.e., supersets of the candidate keys with a cardinality strictly greater than 2).

Since every key requires E plus at least one attribute from $\{A, B, C, D\}$, a strict superkey will consist of E along with 2, 3, or 4 attributes from $\{A, B, C, D\}$.

Here is the exhaustive list of the **11 superkeys that are not keys**:

- **Superkeys with 3 attributes (6 total):**

$$\{A, B, E\}, \quad \{A, C, E\}, \quad \{A, D, E\}, \quad \{B, C, E\}, \quad \{B, D, E\}, \quad \{C, D, E\}$$

- **Superkeys with 4 attributes (4 total):**

$$\{A, B, C, E\}, \quad \{A, B, D, E\}, \quad \{A, C, D, E\}, \quad \{B, C, D, E\}$$

- **Superkey with 5 attributes (1 total):**

$$\{A, B, C, D, E\}$$

Q3. Consider the following table:

FirstName	FirstInitial	LastName	LastInitial	FullName	FullAddress	ZipCode
Alice	A	Levy	L	Alice Levy	45h St, Seattle, 98185	98185
Bob	B	Levy	L	Bob Levy	45h St, Seattle, 98185	98185
Alice	A	Davis	D	Alice Davis	Oak Ave, Seattle, 98185	98185
Carol	C	Davis	D	Carol Davis	Sunnyvale, 94085	94085
Carol	C	Louis	L	Carol Louis	Oak Ave, Seattle, 98185	98185

- (a) Find all functional dependencies that hold on this table. You only need to write a minimal set of functional dependencies that logically imply all others.
- (b) Using the functional dependencies identified , decompose the relation into BCNF. Show your work and the final normalized schema including all keys.

(20 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Functional Dependencies

By analyzing the data provided in the table, we can identify the semantic relationships between the attributes. A functional dependency $X \rightarrow Y$ holds if every valid instance of the table ensures that two tuples agreeing on X also agree on Y . Based on the dataset, the minimal set of non-trivial functional dependencies (canonical cover) is:

- F_1 : **FirstName** $\rightarrow$ **FirstInitial**
(e.g., Every 'Alice' has the initial 'A').
- F_2 : **LastName** $\rightarrow$ **LastInitial**
(e.g., Every 'Levy' has the initial 'L').
- F_3 : **FirstName, LastName** $\rightarrow$ **FullName**
(The full name is a direct concatenation of the first and last names).
- F_4 : **FullName** $\rightarrow$ **FirstName**
(A full name uniquely identifies the first name component).
- F_5 : **FullName** $\rightarrow$ **LastName**
(A full name uniquely identifies the last name component).
- F_6 : **FullName** $\rightarrow$ **FullAddress**
(Each unique person in this dataset resides at exactly one address).
- F_7 : **FullAddress** $\rightarrow$ **ZipCode**
(A specific street address maps to exactly one ZIP code).

Candidate Keys for the original relation:
Given these dependencies, both **{FirstName, LastName}** and **{FullName}** can determine all other attributes in the table. Therefore, they are the **candidate keys**.

2. BCNF Decomposition

A relation is in **Boyce-Codd Normal Form (BCNF)** if, for every non-trivial functional dependency $X \rightarrow Y$ that holds in the relation, X is a superkey.

We start with the universal relation:
 $R(\text{FirstName, FirstInitial, LastName, LastInitial, FullName, FullAddress, ZipCode})$

Step 1: Address Decomposition

- Violation:** F_7 ($\text{FullAddress} \rightarrow \text{ZipCode}$) violates BCNF because FullAddress is not a superkey of R .
- Action:** Decompose R into two relations:
 - $R_1(\underline{\text{FullAddress}}, \text{ZipCode})$
 - $R_2(\text{FirstName, FirstInitial, LastName, LastInitial, FullName, FullAddress})$

Step 2: FirstName Decomposition

- Violation:** In R_2 , F_1 ($\text{FirstName} \rightarrow \text{FirstInitial}$) violates BCNF because FirstName is not a superkey.
- Action:** Decompose R_2 into:
 - $R_3(\underline{\text{FirstName}}, \text{FirstInitial})$
 - $R_4(\text{FirstName, LastName, LastInitial, FullName, FullAddress})$

Step 3: LastName Decomposition

- **Violation:** In R_4 , F_2 ($\text{LastName} \rightarrow \text{LastInitial}$) violates BCNF because LastName is not a superkey.
- **Action:** Decompose R_4 into:
 - $R_5(\underline{\text{LastName}}, \text{LastInitial})$
 - $R_6(\text{FirstName}, \text{LastName}, \text{FullName}, \text{FullAddress})$

Step 4: Final Validation

- In R_6 , the remaining functional dependencies are:

$$\text{FirstName, LastName} \rightarrow \text{FullName, FullAddress}$$

$$\text{FullName} \rightarrow \text{FirstName, LastName, FullAddress}$$

- **Verification:** Both $\{\text{FirstName, LastName}\}$ and $\{\text{FullName}\}$ are candidate keys for R_6 . Since the left-hand side of every dependency in R_6 is a superkey, R_6 is in BCNF.

Final Normalized Schema (BCNF)

The decomposition results in four BCNF tables. The primary keys are underlined.

(a) **Addresses**(FullAddress, ZipCode)

(b) **FirstNames**(FirstName, FirstInitial)

(c) **LastNames**(LastName, LastInitial)

(d) **Persons**(FullName, FirstName, LastName, FullAddress)

Note: FirstName and LastName serve as a composite alternate key in the Persons table. FullAddress, FirstName, and LastName are foreign keys referencing the other tables.

Q4. Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.

- What are all the nontrivial FDs that follow from the given FDs? Restrict yourself to FDs with single attributes on the right sides.
- What are all the keys of R ?
- What are all the superkeys that are not keys?

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer**Given:**

- Relation Schema: $R(A, B, C, D)$
- Set of Functional Dependencies (FDs): $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$

To answer the questions, we first determine the attribute closures for all single attributes and key combinations based on F :

- $A^+ = \{A\}$
- $B^+ = \{B\}$
- $C^+ = \{A, C, D\}$ (since $C \rightarrow D$ and $D \rightarrow A$)
- $D^+ = \{A, D\}$ (since $D \rightarrow A$)
- $(AB)^+ = \{A, B, C, D\}$ (since $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$)
- $(BD)^+ = \{A, B, C, D\}$ (since $D \rightarrow A$, so $BD \rightarrow AB \rightarrow C \rightarrow D$)
- $(BC)^+ = \{A, B, C, D\}$ (since $C \rightarrow D$, $D \rightarrow A$, giving ABC , and $AB \rightarrow C$)

(a) All Nontrivial FDs with Single Attributes on the Right Side

A functional dependency $X \rightarrow Y$ is **nontrivial** if the right-hand side (RHS) attribute is not present in the left-hand side (LHS) attribute set ($Y \notin X$). We evaluate all possible LHS subsets for each possible single RHS attribute (A, B, C , or D):

(a) FDs determining A (where $A \notin$ LHS):

- From single attributes: $C \rightarrow A$ (by transitivity $C \rightarrow D \rightarrow A$), $D \rightarrow A$
- From pairs: $BC \rightarrow A$, $BD \rightarrow A$, $CD \rightarrow A$
- From triples: $BCD \rightarrow A$

(b) **FDs determining B (where $B \notin \text{LHS}$):**

- **None.** The attribute B does not appear on the right side of any given FD, so no combination of other attributes can determine B .

(c) **FDs determining C (where $C \notin \text{LHS}$):**

- From pairs: $AB \rightarrow C$ (given), $BD \rightarrow C$ (since $BD \rightarrow AB \rightarrow C$)
- From triples: $ABD \rightarrow C$

(d) **FDs determining D (where $D \notin \text{LHS}$):**

- From single attributes: $C \rightarrow D$ (given)
- From pairs: $AB \rightarrow D$ (since $AB \rightarrow C \rightarrow D$), $AC \rightarrow D$, $BC \rightarrow D$
- From triples: $ABC \rightarrow D$

(b) **All Keys of R**

A set of attributes K is a **candidate key** if it functionally determines all attributes in the schema ($K^+ = \{A, B, C, D\}$) and no proper subset of K has this property (minimality).

- Since B does not appear on the right side of any given FD, **B must be part of every candidate key.**
- Let us evaluate combinations containing B :
 - $\{B\}^+ = \{B\}$ (Not a key)
 - $\{A, B\}^+ = \{A, B, C, D\} \implies \mathbf{AB \text{ is a key.}}$
 - $\{B, C\}^+ = \{A, B, C, D\} \implies \mathbf{BC \text{ is a key.}}$
 - $\{B, D\}^+ = \{A, B, C, D\} \implies \mathbf{BD \text{ is a key.}}$

Since AB , BC , and BD are minimal and determine all attributes, the candidate keys of R are:

$$\{\mathbf{A, B}\}, \quad \{\mathbf{B, C}\}, \quad \{\mathbf{B, D}\}$$

(c) **All Superkeys that are Not Keys**

A **superkey** is any set of attributes that contains a candidate key. To find the superkeys that are *not* candidate keys, we identify all sets that strictly contain at least one of the candidate keys ($\{A, B\}$, $\{B, C\}$, or $\{B, D\}$) but have additional extraneous attributes. Since the only candidate keys are pairs involving B , any superset of size 3 or 4 containing B is a strict superkey.

- **Superkeys with 3 attributes (all contain at least one key):**

$$\{A, B, C\}, \quad \{A, B, D\}, \quad \{B, C, D\}$$

- **Superkey with 4 attributes:**

$$\{A, B, C, D\}$$

Therefore, the superkeys that are not keys are exactly:

$$\{\mathbf{A, B, C}\}, \quad \{\mathbf{A, B, D}\}, \quad \{\mathbf{B, C, D}\}, \quad \{\mathbf{A, B, C, D}\}$$

Q5. Write down the six rules with examples to find out the closure of a set of functional dependencies. (5 marks) [CSE 215 | L-2/T-2 | 2019–20]

Answer

Closure of Functional Dependencies (F^+)

The **closure** of a set of functional dependencies (FDs), denoted as F^+ , is the complete set of all possible functional dependencies that can be logically derived from F . To compute F^+ , database theorists use a set of inference rules known as **Armstrong's Axioms**.

These rules are mathematically **sound** (they do not generate any incorrect dependencies) and **complete** (they can derive all valid dependencies). The first three rules are the fundamental axioms, while the remaining three are derived from the basic axioms for convenience.

Rule Type	Rule Name	Formal Definition
Fundamental	1. Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$
	2. Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$
	3. Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
Derived	4. Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
	5. Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
	6. Pseudotransitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$

The Six Inference Rules with Examples

In the examples below, let us assume a relation schema containing attributes such as EmpID, EmpName, DeptID, DeptName, Project, and Location.

1. Reflexivity Rule (Trivial Dependency)

- **Definition:** If a set of attributes Y is a subset of a set of attributes X , then X functionally determines Y .
- **Example:** Since $\{\text{EmpID}\} \subseteq \{\text{EmpID}, \text{EmpName}\}$, we can state:
 $\{\text{EmpID}, \text{EmpName}\} \rightarrow \text{EmpID}$

2. Augmentation Rule

- **Definition:** If $X \rightarrow Y$ holds, and Z is any set of attributes, then appending Z to both sides yields a valid dependency: $XZ \rightarrow YZ$.
- **Example:** Given that $\text{EmpID} \rightarrow \text{EmpName}$, we can augment both sides with DeptID to get:
 $\{\text{EmpID}, \text{DeptID}\} \rightarrow \{\text{EmpName}, \text{DeptID}\}$

3. Transitivity Rule

- **Definition:** If X determines Y , and Y determines Z , then X transitively determines Z .
- **Example:** If $\text{EmpID} \rightarrow \text{DeptID}$ (an employee belongs to a department) and $\text{DeptID} \rightarrow \text{DeptName}$ (a department ID determines its name), then:
 $\text{EmpID} \rightarrow \text{DeptName}$

4. Decomposition (Projective) Rule

- **Definition:** If X determines a combined set of attributes YZ , then X determines each of those attributes individually.
- **Example:** If $\text{EmpID} \rightarrow \{\text{EmpName}, \text{DeptID}\}$, then we can decompose this into:
 $\text{EmpID} \rightarrow \text{EmpName}$ and $\text{EmpID} \rightarrow \text{DeptID}$

5. Union (Additive) Rule

- **Definition:** If X determines Y and X also determines Z , then X determines the union of Y and Z .
- **Example:** If $\text{EmpID} \rightarrow \text{EmpName}$ and $\text{EmpID} \rightarrow \text{DeptID}$, we can combine them to state:
 $\text{EmpID} \rightarrow \{\text{EmpName}, \text{DeptID}\}$

6. Pseudotransitivity Rule

- **Definition:** If $X \rightarrow Y$ and $WY \rightarrow Z$ hold, then by replacing Y with X in the second dependency, we can derive $WX \rightarrow Z$.
- **Example:** Given $\text{EmpID} \rightarrow \text{DeptID}$ and $\{\text{DeptID}, \text{Project}\} \rightarrow \text{Location}$, we can substitute DeptID with EmpID to conclude:
 $\{\text{EmpID}, \text{Project}\} \rightarrow \text{Location}$

Q6. Write the steps to compute the closure of a set of attributes with respect to a set of functional dependencies (FD). Consider a relation with attributes A, B, C, D, E, F and the FDs:

$$AB \rightarrow C, \quad BC \rightarrow AD, \quad D \rightarrow E, \quad CF \rightarrow B$$

Find the closure of $\{A, B\}$, that is $\{A, B\}^+$. (10 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer**1. Steps to Compute the Attribute Closure (X^+)**

The **attribute closure** of a set of attributes X with respect to a set of functional dependencies F , denoted as X^+ , is the maximal set of attributes that can be functionally determined by X using the inference rules of functional dependencies.

Algorithm: Computing X^+

(a) **Initialization:** Start by setting the closure to the initial set of attributes.

$$X^+ := X$$

(b) **Iteration:** Iterate through the given set of functional dependencies F . For each dependency $Y \rightarrow Z \in F$:

- Check if the left-hand side (LHS) attributes are already in the closure (i.e., if $Y \subseteq X^+$).
- If true, add the right-hand side (RHS) attributes to the closure:

$$X^+ := X^+ \cup Z$$

(c) **Termination Condition:** Repeat Step 2 until either:

- X^+ includes all attributes of the relation schema R (meaning X is a superkey).
- A complete pass through F yields no new attributes being added to X^+ .

(d) **Result:** The final set X^+ is the attribute closure of X .

2. Computation of $\{A, B\}^+$ **Given:**

- Relation Schema Attributes: $\{A, B, C, D, E, F\}$
- Set of FDs (F):

1. $AB \rightarrow C$
2. $BC \rightarrow AD$
3. $D \rightarrow E$
4. $CF \rightarrow B$

- Target Attribute Set (X): $\{A, B\}$

Execution Steps:

(a) **Initialize:**

$$X^+ = \{A, B\}$$

(b) **Pass 1 over F :**

- Evaluate $AB \rightarrow C$: Since $\{A, B\} \subseteq \{A, B\}$, we add C .
 $X^+ = \{A, B, C\}$
- Evaluate $BC \rightarrow AD$: Since $\{B, C\} \subseteq \{A, B, C\}$, we add A and D .
 $X^+ = \{A, B, C, D\}$
- Evaluate $D \rightarrow E$: Since $\{D\} \subseteq \{A, B, C, D\}$, we add E .
 $X^+ = \{A, B, C, D, E\}$
- Evaluate $CF \rightarrow B$: Since $\{C, F\} \not\subseteq \{A, B, C, D, E\}$ (attribute F is missing), we cannot apply this FD.

(c) **Pass 2 over F :**

- We evaluate all FDs again.
- No new attributes can be generated because the only remaining attribute is F , and F never appears on the RHS of any functional dependency.
- The closure does not change during this pass, satisfying our termination condition.

Final Result: The closure of $\{A, B\}$ is:

$$\{A, B\}^+ = \{A, B, C, D, E\}$$

Q7. Given a set F of functional dependencies on $R(A, B, C)$:

$$A \rightarrow BC, \quad B \rightarrow C, \quad A \rightarrow B, \quad AB \rightarrow C$$

(a) Compute the canonical cover for F .

(b) Decompose R into R_1 and R_2 such that the decomposition is lossless. What are the normal forms of the decomposed relations?

(9 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

Given:

- Relation Schema: $R(A, B, C)$
- Set of Functional Dependencies (FDs): $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

(a) Compute the Canonical Cover (F_c)

The **canonical cover** (or minimal cover) is a simplified, minimal set of functional dependencies that is logically equivalent to F . We compute it using the following steps:

Step 1: Ensure single attributes on the right-hand side (RHS)

Apply the decomposition rule to FDs with multiple attributes on the RHS.

- $A \rightarrow BC$ decomposes into $A \rightarrow B$ and $A \rightarrow C$.

The initial revised set becomes:

$$F_1 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C\}$$

(Note: The explicitly given $A \rightarrow B$ is now a duplicate and merged.)

Step 2: Remove extraneous attributes from the left-hand side (LHS)

We check if any attributes can be removed from composite LHSs without changing the closure.

- Look at the FD $AB \rightarrow C$: Is A or B extraneous?
- **Check if A is extraneous:** We test if we can drop A and just use $B \rightarrow C$. Since $B \rightarrow C$ is already explicitly given in our set F_1 , A is definitively extraneous.
- Therefore, $AB \rightarrow C$ reduces to $B \rightarrow C$, which is already in the set.

Our set reduces to:

$$F_2 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C\}$$

Step 3: Remove redundant functional dependencies

We check if any complete FD is logically implied by the remaining FDs (by computing the attribute closure without that FD).

- **Check $A \rightarrow B$:** If removed, remaining FDs are $\{A \rightarrow C, B \rightarrow C\}$. The closure $(A)^+$ under these is $\{A, C\}$. It does not contain B . Thus, $A \rightarrow B$ is **not redundant**.
- **Check $A \rightarrow C$:** If removed, remaining FDs are $\{A \rightarrow B, B \rightarrow C\}$. The closure $(A)^+$ under these is $\{A, B, C\}$ (since $A \rightarrow B$ and $B \rightarrow C$). It contains C . Thus, $A \rightarrow C$ is **redundant** and can be removed.
- **Check $B \rightarrow C$:** If removed, remaining FDs are $\{A \rightarrow B\}$. The closure $(B)^+$ is just $\{B\}$. It does not contain C . Thus, $B \rightarrow C$ is **not redundant**.

Conclusion: The minimal canonical cover is:

$$F_c = \{A \rightarrow B, B \rightarrow C\}$$

(b) Lossless Decomposition and Normal Forms

We must decompose $R(A, B, C)$ into R_1 and R_2 such that the decomposition is lossless.

1. Decomposition Strategy

Using the canonical cover $F_c = \{A \rightarrow B, B \rightarrow C\}$, we note that the candidate key of R is A . The FD $B \rightarrow C$ violates BCNF (since B is not a superkey). We decompose R based on this violating FD:

- $R_1(B, C)$ with FD: $\{B \rightarrow C\}$
- $R_2(A, B)$ with FD: $\{A \rightarrow B\}$

2. Testing for Lossless Join

A decomposition of R into R_1 and R_2 is lossless if the intersection of their attributes is a superkey for at least one of the decomposed relations.

- **Intersection:** $R_1 \cap R_2 = \{B, C\} \cap \{A, B\} = \{B\}$
- **Superkey check:** In $R_1(B, C)$, the attribute B is the candidate key because $B \rightarrow C$.
- Since $R_1 \cap R_2 \rightarrow R_1$ holds, the decomposition satisfies the Lossless Join Theorem.

3. Normal Forms of the Decomposed Relations

To determine the highest normal form, we check the FDs valid in each decomposed relation:

(a) Relation $R_1(B, C)$:

- **Valid FD:** $B \rightarrow C$
- **Candidate Key:** B
- **Evaluation:** Since the left-hand side (B) of the only non-trivial FD is a superkey of R_1 , the relation is in **Boyce-Codd Normal Form (BCNF)**.

(b) Relation $R_2(A, B)$:

- **Valid FD:** $A \rightarrow B$
- **Candidate Key:** A
- **Evaluation:** Since the left-hand side (A) of the only non-trivial FD is a superkey of R_2 , the relation is in **Boyce-Codd Normal Form (BCNF)**.

Conclusion: The relations $R_1(B, C)$ and $R_2(A, B)$ provide a lossless decomposition, and **both decomposed relations are in BCNF** (and consequently in 1NF, 2NF, and 3NF).

Q8. Given a set F of functional dependencies on $r_1(A, B, C, G, H, I)$:

$$A \rightarrow B, \quad A \rightarrow C, \quad CG \rightarrow H, \quad CG \rightarrow I, \quad B \rightarrow H$$

Compute the attribute closure $(AG)^+$ and show that AG is a candidate key of r_1 . **(6 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

Given:

- Relation Schema: $r_1(A, B, C, G, H, I)$
- Set of Functional Dependencies (FDs), F :

1. $A \rightarrow B$
2. $A \rightarrow C$
3. $CG \rightarrow H$
4. $CG \rightarrow I$
5. $B \rightarrow H$

1. Computation of Attribute Closure $(AG)^+$

To find the attribute closure $(AG)^+$, we start with the initial set and repeatedly apply the functional dependencies from F until no more attributes can be added.

(a) **Initialization:** Start with the given attributes.

$$(AG)^+ = \{A, G\}$$

(b) **Iteration 1:** Evaluate the FDs against the current closure.

- Apply $A \rightarrow B$: Since $A \in (AG)^+$, we add B .
 $(AG)^+ = \{A, B, G\}$
- Apply $A \rightarrow C$: Since $A \in (AG)^+$, we add C .
 $(AG)^+ = \{A, B, C, G\}$
- Apply $CG \rightarrow H$: Since $\{C, G\} \subseteq (AG)^+$, we add H .
 $(AG)^+ = \{A, B, C, G, H\}$
- Apply $CG \rightarrow I$: Since $\{C, G\} \subseteq (AG)^+$, we add I .
 $(AG)^+ = \{A, B, C, G, H, I\}$

- Apply $B \rightarrow H$: Since $B \in (AG)^+$, we add H . (However, H is already in the closure, so no change).

(c) **Termination:** The closure now contains all attributes of the relation schema r_1 . No further additions are possible.

Result:

$$(AG)^+ = \{A, B, C, G, H, I\}$$

2. Proof that AG is a Candidate Key

A set of attributes K is a **candidate key** for a relation if it satisfies two conditions:

- Superkey Property:** K functionally determines all attributes of the relation ($K^+ = \text{All attributes}$).
- Minimality Property:** No proper subset of K is a superkey.

Step 1: Verify Superkey Property

From our computation in Part 1, we established that $(AG)^+ = \{A, B, C, G, H, I\}$. Because the closure of AG contains every attribute in the schema r_1 , AG is a **superkey**.

Step 2: Verify Minimality Property

To ensure AG is minimal, we must compute the closures of all its proper subsets (i.e., $\{A\}$ and $\{G\}$) to verify that neither can determine the entire relation on its own.

- **Closure of A (A^+):**
 - Initialize: $A^+ = \{A\}$
 - Apply $A \rightarrow B, A \rightarrow C$: $A^+ = \{A, B, C\}$
 - Apply $B \rightarrow H$: $A^+ = \{A, B, C, H\}$
 - No other FDs apply. Thus, $A^+ = \{A, B, C, H\} \neq \{A, B, C, G, H, I\}$.
 - *Conclusion:* A is not a superkey.
- **Closure of G (G^+):**
 - Initialize: $G^+ = \{G\}$
 - No FDs in F have only G (or a subset of G^+) on the left-hand side.
 - Thus, $G^+ = \{G\} \neq \{A, B, C, G, H, I\}$.
 - *Conclusion:* G is not a superkey.

Final Conclusion: Since AG is a superkey and none of its proper subsets ($\{A\}$ or $\{G\}$) are superkeys, AG is minimal. Therefore, **AG is mathematically proven to be a candidate key** of the relation r_1 .

Q9. Consider a relation $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $BC \rightarrow D$, $CD \rightarrow A$, and $AD \rightarrow B$.

- What are all nontrivial FDs that follow from the given FDs? (Restrict yourself to FDs with single attributes on the right side.)
- What are all the keys of R ?
- What are all the superkeys for R that are not keys?

(12 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Given:

- Relation Schema: $R(A, B, C, D)$
- Set of Functional Dependencies (FDs): $F = \{AB \rightarrow C, BC \rightarrow D, CD \rightarrow A, AD \rightarrow B\}$

Before addressing the specific questions, let's determine the attribute closures for single attributes and pairs, as this will guide the rest of the analysis:

- **Single Attributes:** $A^+ = \{A\}$, $B^+ = \{B\}$, $C^+ = \{C\}$, $D^+ = \{D\}$. (None determine any other attributes).
- **Pairs:**
 - $(AB)^+ = \{A, B, C, D\}$ (since $AB \rightarrow C \implies ABC$, and $BC \rightarrow D \implies ABCD$)
 - $(BC)^+ = \{A, B, C, D\}$ (since $BC \rightarrow D \implies BCD$, and $CD \rightarrow A \implies ABCD$)
 - $(CD)^+ = \{A, B, C, D\}$ (since $CD \rightarrow A \implies ACD$, and $AD \rightarrow B \implies ABCD$)
 - $(AD)^+ = \{A, B, C, D\}$ (since $AD \rightarrow B \implies ABD$, and $AB \rightarrow C \implies ABCD$)

- $(AC)^+ = \{A, C\}$ (no FDs apply)
- $(BD)^+ = \{B, D\}$ (no FDs apply)

(a) All Nontrivial FDs with Single Attributes on the Right Side

A functional dependency $X \rightarrow Y$ is **nontrivial** if $Y \notin X$. We are looking for all dependencies where Y is a single attribute (A, B, C , or D) and X is a subset of $\{A, B, C, D\}$.

Since no single attribute determines any other, and the pairs AC and BD do not determine anything outside themselves, the left-hand side (X) must be one of the superkeys evaluated above (which determine all attributes).

By extracting the non-trivial, single-attribute dependencies from these closures, we get exactly **12 FDs**:

- **From LHS of size 2:**

- $AB \rightarrow C$ (given), $AB \rightarrow D$
- $BC \rightarrow A$, $BC \rightarrow D$ (given)
- $CD \rightarrow A$ (given), $CD \rightarrow B$
- $AD \rightarrow B$ (given), $AD \rightarrow C$

- **From LHS of size 3:**

(Formed by taking sets of 3 attributes and mapping them to the only attribute not in the set)

- $ABC \rightarrow D$
- $ABD \rightarrow C$
- $ACD \rightarrow B$
- $BCD \rightarrow A$

(b) All Keys of R

A set of attributes K is a **candidate key** if it functionally determines all attributes in the schema ($K^+ = \{A, B, C, D\}$) and no proper subset of K has this property.

Based on our closure calculations at the beginning:

- (a) None of the single attributes ($\{A\}, \{B\}, \{C\}, \{D\}$) determine the whole relation.
- (b) The pairs $\{A, B\}$, $\{B, C\}$, $\{C, D\}$, and $\{A, D\}$ all determine the entire relation $\{A, B, C, D\}$. Since their subsets are just single attributes (which are not keys), they are minimal.

Therefore, the candidate keys of R are exactly:

$$\{A, B\}, \quad \{B, C\}, \quad \{C, D\}, \quad \{A, D\}$$

(c) All Superkeys that are Not Keys

A **superkey** is any set of attributes that contains a candidate key. To find the superkeys that are *not* keys, we identify all combinations that strictly contain at least one of the candidate keys identified in part (b) but have a cardinality greater than 2.

Since any 3-attribute subset of $\{A, B, C, D\}$ contains at least one of the adjacent pairs (AB, BC, CD , or AD), all triples are superkeys.

The complete list of **5 superkeys that are not keys** is:

- **Superkeys with 3 attributes (4 total):**

$$\{A, B, C\}, \quad \{A, B, D\}, \quad \{A, C, D\}, \quad \{B, C, D\}$$

- **Superkey with 4 attributes (1 total):**

$$\{A, B, C, D\}$$

Q10. Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $AD \rightarrow B$, $B \rightarrow D$. Find all non-trivial FDs that follow from the given FDs. **(10 marks)** [CSE 303 | L-3/T-1 | 2011–12]

Answer**Given:**

- Relation Schema: $R(A, B, C, D)$
- Set of Functional Dependencies (FDs): $F = \{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

To find all non-trivial functional dependencies that logically follow from F , we first systematically compute the attribute closures for all possible subsets of $\{A, B, C, D\}$.

1. Compute Attribute Closures

- **Closures of Single Attributes:**

- $A^+ = \{A\}$
- $B^+ = \{B, D\}$ (since $B \rightarrow D$)
- $C^+ = \{C\}$
- $D^+ = \{D\}$

- **Closures of Attribute Pairs:**

- $(AB)^+ = \{A, B, C, D\}$ (since $B \rightarrow D \implies ABD$, and $AB \rightarrow C \implies ABCD$)
- $(AC)^+ = \{A, C\}$
- $(AD)^+ = \{A, B, C, D\}$ (since $AD \rightarrow B \implies ABD$, and $AB \rightarrow C \implies ABCD$)
- $(BC)^+ = \{B, C, D\}$ (since $B \rightarrow D \implies BCD$)
- $(BD)^+ = \{B, D\}$ (no additional attributes)
- $(CD)^+ = \{C, D\}$

- **Closures of Attribute Triples:**

- $(ABC)^+ = \{A, B, C, D\}$
- $(ABD)^+ = \{A, B, C, D\}$
- $(ACD)^+ = \{A, B, C, D\}$
- $(BCD)^+ = \{B, C, D\}$

2. Extract All Completely Non-Trivial FDs

A functional dependency $X \rightarrow Y$ is **completely non-trivial** if the left-hand side and right-hand side share no attributes ($X \cap Y = \emptyset$). Using the closures computed above, we extract every valid FD by mapping each subset X to all possible non-empty subsets of $X^+ \setminus X$.

- **From LHS of size 1:**

- $B \rightarrow D$ (since $B^+ = \{B, D\}$)

- **From LHS of size 2:**

- **From $(AB)^+$:** $AB \rightarrow C, AB \rightarrow D, AB \rightarrow CD$
- **From $(AD)^+$:** $AD \rightarrow B, AD \rightarrow C, AD \rightarrow BC$
- **From $(BC)^+$:** $BC \rightarrow D$

- **From LHS of size 3:**

- **From $(ABC)^+$:** $ABC \rightarrow D$
- **From $(ABD)^+$:** $ABD \rightarrow C$
- **From $(ACD)^+$:** $ACD \rightarrow B$

Note: Semi-trivial dependencies (where $Y \not\subseteq X$ but $X \cap Y \neq \emptyset$, such as $AB \rightarrow BD$) can be formed by augmenting the right-hand sides of the above FDs with attributes from their respective left-hand sides.

Q11. Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$. What are the non-trivial FDs that follow from the given FDs? (8 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Core Concepts and Definitions

To systematically find all non-trivial functional dependencies (FDs) that follow from a given set of dependencies F , we utilize the concept of **attribute closure** and **Armstrong's Axioms** (Reflexivity, Augmentation, and Transitivity).

- **Functional Dependency (FD):** A constraint between two sets of attributes. Given a relation R , an FD $X \rightarrow Y$ holds if each X -value is associated with precisely one Y -value.
- **Closure of an Attribute Set (X^+):** The set of all attributes that are functionally determined by X under the given set of FDs F .
- **Non-Trivial FD:** An dependency $X \rightarrow Y$ is non-trivial if $Y \not\subseteq X$. It is considered **completely non-trivial** if $X \cap Y = \emptyset$.
- **Closure of a Set of FDs (F^+):** The set of all functional dependencies that can be logically inferred from F .

2. Computation of Attribute Closures

Given the schema $R(A, B, C, D)$ and the set of functional dependencies:

$$F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$$

We compute the attribute closure X^+ for every non-empty subset $X \subseteq \{A, B, C, D\}$.

2.1. Single-Attribute Closures

- $A^+ = \{A\}$
- $B^+ = \{B\}$
- $C^+ = \{C\} \xrightarrow{C \rightarrow D} \{C, D\} \xrightarrow{D \rightarrow A} \{A, C, D\}$
- $D^+ = \{D\} \xrightarrow{D \rightarrow A} \{A, D\}$

2.2. Two-Attribute Closures

- $(AB)^+ = \{A, B\} \xrightarrow{AB \rightarrow C} \{A, B, C\} \xrightarrow{C \rightarrow D} \{A, B, C, D\}$
- $(AC)^+ = \{A, C\} \cup C^+ = \{A, C, D\}$
- $(AD)^+ = \{A, D\} \cup D^+ = \{A, D\}$
- $(BC)^+ = \{B, C\} \cup C^+ = \{A, B, C, D\}$
- $(BD)^+ = \{B, D\} \cup D^+ = \{B, D, A\} \xrightarrow{AB \rightarrow C} \{A, B, C, D\}$
- $(CD)^+ = \{C, D\} \cup C^+ \cup D^+ = \{A, C, D\}$

2.3. Three-Attribute Closures

- $(ABC)^+ = \{A, B, C, D\}$
- $(ABD)^+ = \{A, B, C, D\}$
- $(ACD)^+ = \{A, C, D\}$
- $(BCD)^+ = \{A, B, C, D\}$

3. Derivation of Non-Trivial Functional Dependencies

An FD $X \rightarrow Y$ is valid if $Y \subseteq X^+$. To extract the non-trivial dependencies, we find all pairs (X, Y) such that $Y \subseteq X^+$ and $Y \not\subseteq X$. We restrict our standard presentation to single attributes on the right-hand side (RHS), as any compound RHS can be derived via the **Union Rule**.

Below is the complete textbook classification of all completely non-trivial dependencies ($X \cap Y = \emptyset$) that follow from F :

(a) From Single Attributes on the Left-Hand Side:

- From C^+ : $C \rightarrow A, C \rightarrow D$
- From D^+ : $D \rightarrow A$

(b) From Two Attributes on the Left-Hand Side:

- From $(AB)^+$: $AB \rightarrow C, AB \rightarrow D$
- From $(AC)^+$: $AC \rightarrow D$

- From $(BC)^+$: $BC \rightarrow A, BC \rightarrow D$
- From $(BD)^+$: $BD \rightarrow A, BD \rightarrow C$
- From $(CD)^+$: $CD \rightarrow A$

(c) From Three Attributes on the Left-Hand Side:

- From $(ABC)^+$: $ABC \rightarrow D$
- From $(ABD)^+$: $ABD \rightarrow C$
- From $(BCD)^+$: $BCD \rightarrow A$

4. Summary Table of Dependent Attribute Mappings

The following professional table synthesizes the non-trivial functional dependencies grouped by their respective left-hand side determinants:

Determinant (X)	Full Closure (X^+)	Completely Non-Trivial FDs	Candidate Key Status
C	$\{A, C, D\}$	$C \rightarrow A, C \rightarrow D$	No
D	$\{A, D\}$	$D \rightarrow A$	No
AB	$\{A, B, C, D\}$	$AB \rightarrow C, AB \rightarrow D$	Yes
AC	$\{A, C, D\}$	$AC \rightarrow D$	No
BC	$\{A, B, C, D\}$	$BC \rightarrow A, BC \rightarrow D$	Yes
BD	$\{A, B, C, D\}$	$BD \rightarrow A, BD \rightarrow C$	Yes
CD	$\{A, C, D\}$	$CD \rightarrow A$	No
ABC	$\{A, B, C, D\}$	$ABC \rightarrow D$	Superkey Only
ABD	$\{A, B, C, D\}$	$ABD \rightarrow C$	Superkey Only
BCD	$\{A, B, C, D\}$	$BCD \rightarrow A$	Superkey Only

Note on Composite Right-Hand Sides: By utilizing the Union Axiom, any combinations of the single-attribute RHS targets can be bound together to form larger valid non-trivial dependencies (e.g., $C \rightarrow AD$, $AB \rightarrow CD$, $BC \rightarrow AD$, and $BD \rightarrow AC$).

Normal Forms (BCNF, 3NF, 4NF)

- Q1.** Suppose R is a relation with attributes $A_1, A_2, \dots, A_n$. As a function of n , determine how many superkeys R has for each of the following cases:
- (a) The only key is A_1 .
 - (b) The only keys are $\{A_1, A_2\}$ and $\{A_1, A_3\}$.

(8 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Core Concepts and Definitions

To determine the number of superkeys, we rely on the fundamental definitions of keys in relational database design:

- **Superkey:** Any set of attributes $S \subseteq R$ that functionally determines all attributes in R . A set S is a superkey if and only if it contains at least one candidate key as a subset.
- **Candidate Key (or Key):** A minimal superkey. No proper subset of a candidate key can be a superkey.
- **Attribute Pool:** The relation schema contains n distinct attributes: $R = \{A_1, A_2, \dots, A_n\}$.

2. Case (a): The only candidate key is $\{A_1\}$

2.1. Analysis

For a subset of attributes to be a superkey in this scenario, it must contain the candidate key $\{A_1\}$.

* The attribute A_1 **must** be present in every superkey. * The remaining $n - 1$ attributes ($\{A_2, A_3, \dots, A_n\}$) are optional; each can either be included or excluded from the superkey independently.

2.2. Mathematical Derivation

The number of ways to choose a subset from the remaining $n - 1$ attributes is equal to the size of the power set of those $n - 1$ attributes:

Number of Superkeys = 2^{n-1}

3. Case (b): The only candidate keys are $\{A_1, A_2\}$ and $\{A_1, A_3\}$

3.1. Analysis

For a subset of attributes to be a superkey in this scenario, it must contain at least one of the candidate keys. This means the subset must contain either:

- (a) The attribute combination $\{A_1, A_2\}$
- (b) The attribute combination $\{A_1, A_3\}$

Notice that in both cases, the attribute A_1 **must** be present. Therefore, any valid superkey must take the form $\{A_1\} \cup Y$, where Y is a subset of the remaining $n - 1$ attributes ($\{A_2, A_3, \dots, A_n\}$) such that Y contains A_2 or A_3 (or both).

3.2. Mathematical Derivation using Inclusion-Exclusion

Let us focus on selecting the subset Y from the remaining $n - 1$ attributes: * Total number of possible subsets of the remaining $n - 1$ attributes is 2^{n-1} . * We must exclude any subset Y that contains **neither** A_2 nor A_3 . * If a subset Y contains neither A_2 nor A_3 , then Y must be chosen entirely from the remaining $(n - 1) - 2 = n - 3$ attributes ($\{A_4, A_5, \dots, A_n\}$). * The number of such invalid subsets is 2^{n-3} .

Subtracting the invalid combinations from the total possible combinations gives:

$$\begin{aligned} \text{Number of Superkeys} &= 2^{n-1} - 2^{n-3} \\ &= 4 \cdot 2^{n-3} - 1 \cdot 2^{n-3} \\ &= (4 - 1) \cdot 2^{n-3} \\ &= 3 \cdot 2^{n-3} \end{aligned}$$

Note: This formula holds for all $n \geq 3$. If $n = 2$, the key $\{A_1, A_3\}$ cannot exist, making the question contextually bound to $n \geq 3$.

4. Summary of Results

The following professional table summarizes the count of superkeys for both conditions as a function of the total number of attributes n :

Case	Candidate Keys	Total Number of Superkeys
(a)	$\{A_1\}$	2^{n-1}
(b)	$\{A_1, A_2\}$ and $\{A_1, A_3\}$	$3 \cdot 2^{n-3}$ (for $n \geq 3$)

Q2. For a relation schema $R(A, B, C, D, E)$, consider the following set of functional dependencies: $ABE \rightarrow C$, $BC \rightarrow D$, $BD \rightarrow E$, $DE \rightarrow A$. Decompose the relation into BCNF. Show the final schemas and the projected functional dependencies for each schema after decomposition. Is your final decomposition dependency-preserving? Justify your answer. **(16 marks)** [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Analysis of Candidate Keys

To decompose the relation $R(A, B, C, D, E)$ into Boyce-Codd Normal Form (BCNF), we first need to determine the candidate keys. We are given the set of functional dependencies (FDs):

$$F = \{ABE \rightarrow C, BC \rightarrow D, BD \rightarrow E, DE \rightarrow A\}$$

Attribute **B** does not appear on the right-hand side of any functional dependency, so it must be part of every candidate key. Let us compute the attribute closures for combinations involving B :

- $(BC)^+ = \{B, C, D, E, A\} = \{A, B, C, D, E\}$ (using $BC \rightarrow D$, then $BD \rightarrow E$, then $DE \rightarrow A$)
- $(BD)^+ = \{B, D, E, A, C\}$ (using $BD \rightarrow E$, then $DE \rightarrow A$, then $ABE \rightarrow C$)
- $(ABE)^+ = \{A, B, E, C, D\}$ (using $ABE \rightarrow C$, then $BC \rightarrow D$)

Since $(BC)^+$, $(BD)^+$, and $(ABE)^+$ all yield the set of all attributes in R , and no proper subset of these determines all attributes, the **Candidate Keys** for R are: **BC, BD, and ABE**.

2. Identification of BCNF Violations

A relation is in BCNF if and only if for every non-trivial functional dependency $X \rightarrow Y$, the left-hand side X is a superkey. Let us evaluate each FD in F :

Functional Dependency	Left-Hand Side (LHS)	Is LHS a Superkey?	BCNF Status
$ABE \rightarrow C$	ABE (Candidate Key)	Yes	Valid
$BC \rightarrow D$	BC (Candidate Key)	Yes	Valid
$BD \rightarrow E$	BD (Candidate Key)	Yes	Valid
$DE \rightarrow A$	DE ($(DE)^+ = \{A, D, E\}$)	No	Violation

The functional dependency **DE → A** violates BCNF because DE is not a superkey of R .

3. BCNF Decomposition

To remove the BCNF violation caused by $DE \rightarrow A$, we decompose R into two new relations:

- (a) $R_1 = X \cup Y = \{D, E\} \cup \{A\} = \{\mathbf{A, D, E}\}$
- (b) $R_2 = R - Y = \{A, B, C, D, E\} - \{A\} = \{\mathbf{B, C, D, E}\}$

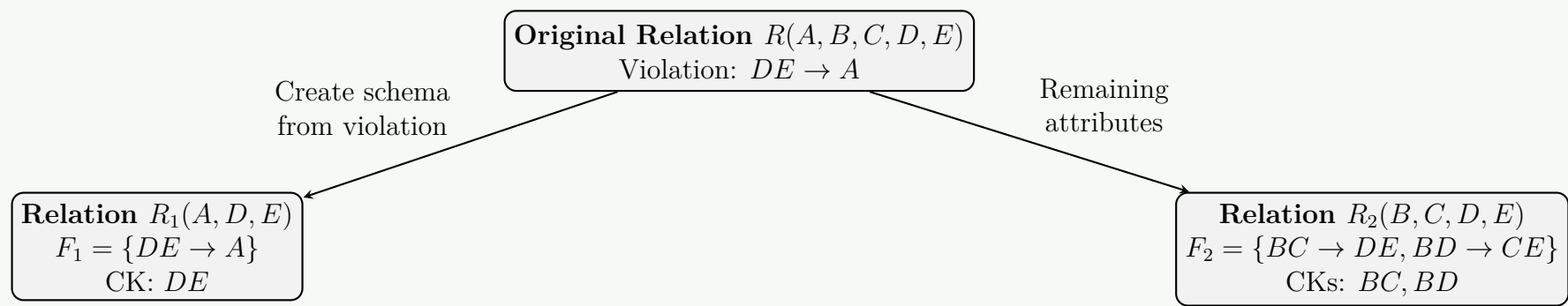
Evaluating Sub-schemas for BCNF:

For $R_1(A, D, E)$:

- **Projected FDs (F_1):** The only applicable FD is $DE \rightarrow A$.
- **Candidate Key:** DE .
- **BCNF Status:** Since DE is the only determinant and it is a candidate key, R_1 is in **BCNF**.

For $R_2(B, C, D, E)$:

- **Projected FDs (F_2):** We find the closure of subsets of $\{B, C, D, E\}$ under F :
 - $(BC)^+ \cap \{B, C, D, E\} = \{A, B, C, D, E\} \cap \{B, C, D, E\} = \{B, C, D, E\} \implies BC \rightarrow DE$
 - $(BD)^+ \cap \{B, C, D, E\} = \{A, B, C, D, E\} \cap \{B, C, D, E\} = \{B, C, D, E\} \implies BD \rightarrow CE$
- Thus, $F_2 = \{BC \rightarrow D, BC \rightarrow E, BD \rightarrow C, BD \rightarrow E\}$.
- **Candidate Keys:** BC and BD .
- **BCNF Status:** Every determinant (BC, BD) in F_2 is a candidate key. Therefore, R_2 is in **BCNF**.



4. Final Schemas and Projected Functional Dependencies

The decomposition process terminates as both resulting schemas are in BCNF.

- **Schema 1:** $R_1(A, D, E)$
- **Projected FDs for R_1 :** $\{DE \rightarrow A\}$
- **Schema 2:** $R_2(B, C, D, E)$
- **Projected FDs for R_2 :** $\{BC \rightarrow D, BC \rightarrow E, BD \rightarrow C, BD \rightarrow E\}$

5. Dependency Preservation Check

A decomposition is **dependency-preserving** if the union of the projected functional dependencies $(F_1 \cup F_2)$ logically implies all the functional dependencies in the original set F . That is, $(F_1 \cup F_2)^+ = F^+$.

Let us test if the original functional dependency $ABE \rightarrow C$ is preserved under $F' = F_1 \cup F_2$:

$$F' = \{DE \rightarrow A, BC \rightarrow D, BC \rightarrow E, BD \rightarrow C, BD \rightarrow E\}$$

We calculate the closure of the left-hand side $(ABE)^+$ using only the dependencies in F' :

(a) **Initial state:** $Closure = \{A, B, E\}$

(b) **Applying FDs from F' :**

- We check $DE \rightarrow A$: We have A and E , but not D . Cannot apply.
- We check $BC \rightarrow D, E$: We have B , but not C . Cannot apply.
- We check $BD \rightarrow C, E$: We have B , but not D . Cannot apply.

(c) **Final Closure:** $(ABE)_{F'}^+ = \{A, B, E\}$

Since $C \notin \{A, B, E\}$, the functional dependency $ABE \rightarrow C$ **cannot** be derived from the dependencies in the decomposed relations.

Conclusion: The final BCNF decomposition is **NOT dependency-preserving** because the dependency $ABE \rightarrow C$ is lost across the decomposed relations.

Q3. Discuss an example where a table is in 3NF but not in BCNF. Then, modify the design to satisfy BCNF. (9 marks) [CSE 215 | L-2/T-1 | 2023–24]

Answer

1. The Classic Example: Student-Course-Instructor

Consider a university database that tracks student enrollments in courses and the instructors teaching them. We define a relation schema R as:

Enrollment (Student, Course, Instructor)

Business Rules and Functional Dependencies

Assume the following constraints apply to our university:

- (a) A student can take many courses, but for a specific course, they are taught by exactly one instructor.
- (b) An instructor teaches exactly one course.
- (c) A course can be taught by multiple instructors.

These rules translate to the following set of functional dependencies F :

$$F = \{\text{Student, Course} \rightarrow \text{Instructor}, \\ \text{Instructor} \rightarrow \text{Course}\}$$

2. Candidate Keys and Prime Attributes

To evaluate the normal forms, we first determine the candidate keys by computing attribute closures.

- $\{\text{Student, Course}\}^+ = \{\text{Student, Course, Instructor}\} \implies$ **Candidate Key**
- $\{\text{Student, Instructor}\}^+ = \{\text{Student, Instructor, Course}\} \implies$ **Candidate Key**

Since an attribute is **prime** if it is a part of *any* candidate key, all three attributes in this relation (**Student, Course, Instructor**) are prime attributes. There are no non-prime attributes.

3. Normal Form Evaluation

Evaluating for Third Normal Form (3NF)

A relation is in 3NF if, for every non-trivial functional dependency $X \rightarrow Y$, at least one of the following holds:

- X is a superkey of the relation.
- Y is a prime attribute.

Let us check the dependencies in F :

Dependency ($X \rightarrow Y$)	Is X a Superkey?	Is Y a Prime Attribute?	3NF Status
Student, Course $\rightarrow$ Instructor	Yes (Candidate Key)	Yes	Valid
Instructor $\rightarrow$ Course	No	Yes (part of {Student, Course})	Valid

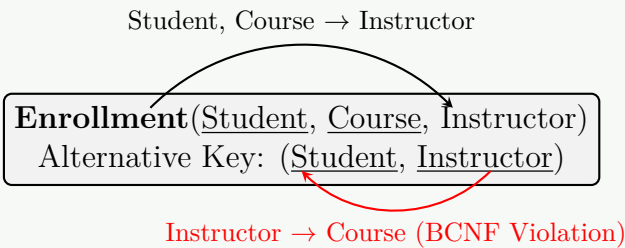
Conclusion: Because every right-hand side attribute is prime, the relation is trivially in **3NF**.

Evaluating for Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X must be a superkey.

- Instructor $\rightarrow$ Course: The determinant is **Instructor**. The closure $\{\text{Instructor}\}^+ = \{\text{Instructor}, \text{Course}\}$. It does not determine **Student**, therefore it is **not a superkey**.

Conclusion: The relation **Enrollment** violates BCNF and suffers from anomalies (e.g., insertion anomaly: we cannot record what course an instructor teaches until a student enrolls).



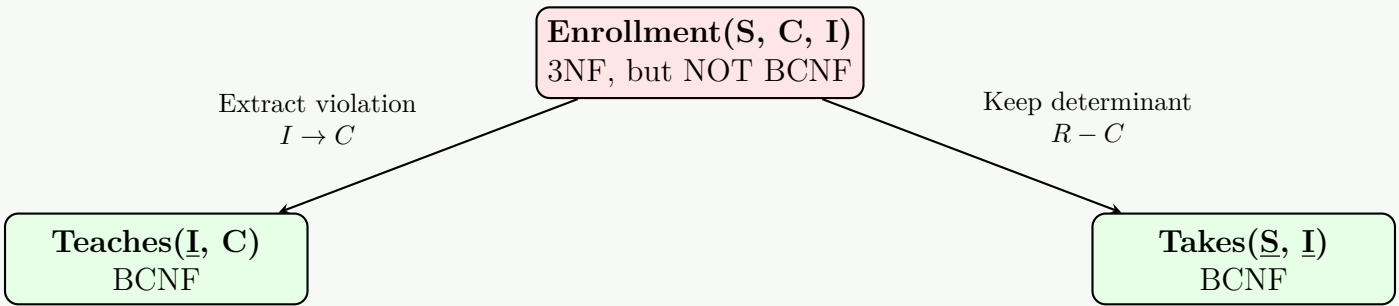
4. Modifying the Design to Satisfy BCNF

To resolve the BCNF violation caused by $\text{Instructor} \rightarrow \text{Course}$, we must decompose the relation using the violating functional dependency. We split R into two new relations:

- (a) **Create a new relation for the violating FD:** The determinant becomes the primary key.
 $R_1 = X \cup Y = \{\text{Instructor}, \text{Course}\}$
- (b) **Create a relation for the remaining attributes:** The determinant acts as a foreign key connecting back.
 $R_2 = R - Y = \{\text{Student}, \text{Instructor}\}$

The Decomposed Schemas

- **Teaches**(Instructor, Course)
 - **FD:** $\text{Instructor} \rightarrow \text{Course}$.
 - **Status:** BCNF (Instructor is the only key).
- **Takes**(Student, Instructor)
 - **FD:** None (all attributes form the key).
 - **Status:** BCNF.



5. Trade-off: Loss of Dependency Preservation

While the new design achieves BCNF and eliminates data anomalies, it introduces a significant trade-off: **Dependency Preservation is lost**.

- The original dependency $\text{Student, Course} \rightarrow \text{Instructor}$ spans across both new tables.
- In the decomposed schema, we cannot easily enforce the rule that "a student takes a specific course with only one instructor" without joining **Teaches** and **Takes** back together upon every insertion.

Note: This scenario highlights a fundamental theorem in database design: *It is not always possible to achieve a decomposition that is simultaneously BCNF, lossless-join, and dependency-preserving.* In such edge cases, database designers must choose between leaving the schema in 3NF (and handling anomalies using application-level triggers) or decomposing to BCNF (and writing complex queries to enforce the lost dependency).

Q4. Why is the normalization technique used in database design? Normalize the following table by drawing a dependency diagram and identifying all dependencies to derive up to 3NF. Show the schema in 3NF.

Attribute	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5
INV_NUM	211347	211347	211347	211348	211349
PROD_NUM	AA-E3422QW	QD-300932X	RU-995748G	AA-E3422QW	GH-778345P
SALE_DATE	15-Jan-2010	15-Jan-2010	15-Jan-2010	15-Jan-2010	16-Jan-2010
PROD_LABEL	Rotary sander	0.25-in. drill bit	Band saw	Rotary sander	Power drill
VEND_CODE	211	211	309	211	157
VEND_NAME	NeverFail Inc.	NeverFail Inc.	BeGood Inc.	NeverFail Inc.	ToughGo. Inc
QUANT_SOLD	1	8	1	2	1
PROD_PRICE	\$49.95	\$3.45	\$39.99	\$49.95	\$87.75

(15 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Purpose of Normalization

Normalization is a systematic process in database design used to organize tables to minimize redundancy and dependency. The primary reasons for applying normalization are:

- Eliminate Data Redundancy:** Prevents the same data from being stored in multiple places, which saves storage space and reduces inconsistency.
- Prevent Update Anomalies:** Ensures that updating a data point (e.g., a vendor’s name) only needs to happen in one place.
- Prevent Insertion Anomalies:** Allows the insertion of facts independently (e.g., adding a new product before it has been sold).
- Prevent Deletion Anomalies:** Ensures that deleting a record does not accidentally delete unrelated information (e.g., deleting the only invoice for a product shouldn’t delete the product’s details).
- Improve Data Integrity:** Enforces structural constraints to maintain accurate and reliable data.

2. Identification of Dependencies

Based on the provided dataset, the composite primary key for the unnormalized table is {INV_NUM, PROD_NUM}. We can identify the following functional dependencies:

(a) **Full Functional Dependency:** Attributes that depend on the *entire* primary key.

- {INV_NUM, PROD_NUM} → QUANT_SOLD

(b) **Partial Dependencies:** Attributes that depend on only a *part* of the composite primary key.

- INV_NUM → SALE_DATE
- PROD_NUM → PROD_LABEL, PROD_PRICE, VEND_CODE, VEND_NAME

(c) **Transitive Dependencies:** Non-key attributes that depend on other non-key attributes.

- VEND_CODE → VEND_NAME

3. Dependency Diagram (1NF)

The following diagram illustrates the initial state of the table (First Normal Form) containing partial and transitive dependencies.

```
graph TD
    subgraph Attributes
        INV_NUM
        PROD_NUM
        SALE_DATE
        QUANT_SOLD
        PROD_LABEL
        PROD_PRICE
        VEND_CODE
        VEND_NAME
    end
    INV_NUM
    PROD_NUM
    QUANT_SOLD
    SALE_DATE
    PROD_LABEL
    PROD_PRICE
    VEND_CODE
    VEND_NAME
    INV_NUM -- "Partial Dependency" --> SALE_DATE
    PROD_NUM -- "Partial Dependency" --> PROD_LABEL
    PROD_NUM -- "Partial Dependency" --> PROD_PRICE
    PROD_NUM -- "Partial Dependency" --> VEND_CODE
    PROD_NUM -- "Partial Dependency" --> VEND_NAME
    INV_NUM -- "Full Dependency" --> QUANT_SOLD
    PROD_NUM -- "Full Dependency" --> QUANT_SOLD
    VEND_CODE -- "Transitive Dependency" --> VEND_NAME
```

4. Derivation to Normal Forms

A. Second Normal Form (2NF)

To achieve 2NF, we must eliminate partial dependencies. We split the 1NF table into three distinct tables based on the determinants of the partial dependencies, creating tables for Invoice, Product, and the remaining intersection (Line Item).

- INVOICE** (INV_NUM, SALE_DATE)
- PRODUCT** (PROD_NUM, PROD_LABEL, PROD_PRICE, VEND_CODE, VEND_NAME)
- LINE_ITEM** (INV_NUM, PROD_NUM, QUANT_SOLD)

Note: The PRODUCT table still contains a transitive dependency ($VEND_CODE \rightarrow VEND_NAME$), so it is not yet in 3NF.

B. Third Normal Form (3NF)

To achieve 3NF, we must eliminate transitive dependencies. We extract the vendor information from the **PRODUCT** table into a new **VENDOR** table, leaving VEND_CODE behind as a foreign key.

5. Final Schema in 3NF

The final optimized and normalized database schema is presented below:

Table Name	Schema	Description
INVOICE	<u>INV_NUM</u> , SALE_DATE	Stores unique invoice header details.
VENDOR	<u>VEND_CODE</u> , VEND_NAME	Stores unique vendor details.
PRODUCT	<u>PROD_NUM</u> , PROD_LABEL, PROD_PRICE, <i>VEND_CODE</i> (FK)	Stores product details and links to its supplier.
LINE_ITEM	<u>INV_NUM</u> (FK), <u>PROD_NUM</u> (FK), QUANT_SOLD	Resolves the M:N relationship between invoices and products.

Q5. Consider a relation with schema $R(A, B, C, D, E)$ and FDs: $AB \rightarrow C$, $DE \rightarrow C$, and $B \rightarrow D$. Decompose the relations, as necessary, into collections of relations that are in BCNF. Show your work and the final normalized schema including all keys. **(20 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Analysis of Candidate Keys

To decompose the relation $R(A, B, C, D, E)$ into Boyce-Codd Normal Form (BCNF), we must first identify its candidate keys. We are given the following set of functional dependencies F :

$$F = \{AB \rightarrow C, DE \rightarrow C, B \rightarrow D\}$$

Let us analyze the attributes to find the core elements of any candidate key:

- Attributes **A** and **E** never appear on the right-hand side of any functional dependency. Therefore, $\{A, E\}$ must be a part of every candidate key.
- Let us compute the attribute closure of $\{A, B, E\}$:

$$\begin{aligned} (ABE)^+ &= \{A, B, E\} \\ &= \{A, B, E, D\} \quad (\text{since } B \rightarrow D) \\ &= \{A, B, E, D, C\} \quad (\text{since } AB \rightarrow C \text{ or } DE \rightarrow C) \end{aligned}$$

Since $(ABE)^+ = \{A, B, C, D, E\}$, the attribute combination **ABE** is a superkey. No proper subset of ABE contains both A and E while also including a determinant that can derive the missing attributes. Thus, the unique **Candidate Key** for R is **ABE**.

2. Evaluation of BCNF Violations

A relation is in BCNF if and only if for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey. Let us check the given dependencies:

FD	Left-Hand Side (LHS)	Is LHS a Superkey?	BCNF Status
$AB \rightarrow C$	$AB \ ((AB)^+ = \{A, B, C, D\})$	No	Violation
$DE \rightarrow C$	$DE \ ((DE)^+ = \{C, D, E\})$	No	Violation
$B \rightarrow D$	$B \ ((B)^+ = \{B, D\})$	No	Violation

Since all three functional dependencies have determinants that are not superkeys, the relation R is not in BCNF.

3. Step-by-Step BCNF Decomposition

We will decompose the relation step-by-step by selecting a violating functional dependency at each stage.

Step 3.1: Decompose R using $B \rightarrow D$

We pick $B \rightarrow D$ as the violation. This splits $R(A, B, C, D, E)$ into two sub-schemas:

- $R_1 = X \cup Y = \{B\} \cup \{D\} = \{\mathbf{B}, \mathbf{D}\}$
- $R_2 = R - Y = \{A, B, C, D, E\} - \{D\} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{E}\}$

Step 3.2: Check and Decompose the Sub-schemas

For $R_1(B, D)$:

- Projected FDs:** $\{B \rightarrow D\}$
- Candidate Key:** B
- BCNF Status:** Since the determinant B is a candidate key, R_1 is in **BCNF**.

For $R_2(A, B, C, E)$:

- Projected FDs:** The dependency $AB \rightarrow C$ applies entirely within this schema. (Note that $DE \rightarrow C$ does not apply because D is absent).
- Candidate Key:** ABE , since $(ABE)^+_{R_2} = \{A, B, C, E\}$.
- BCNF Status:** The dependency $AB \rightarrow C$ violates BCNF because AB is not a superkey of R_2 .

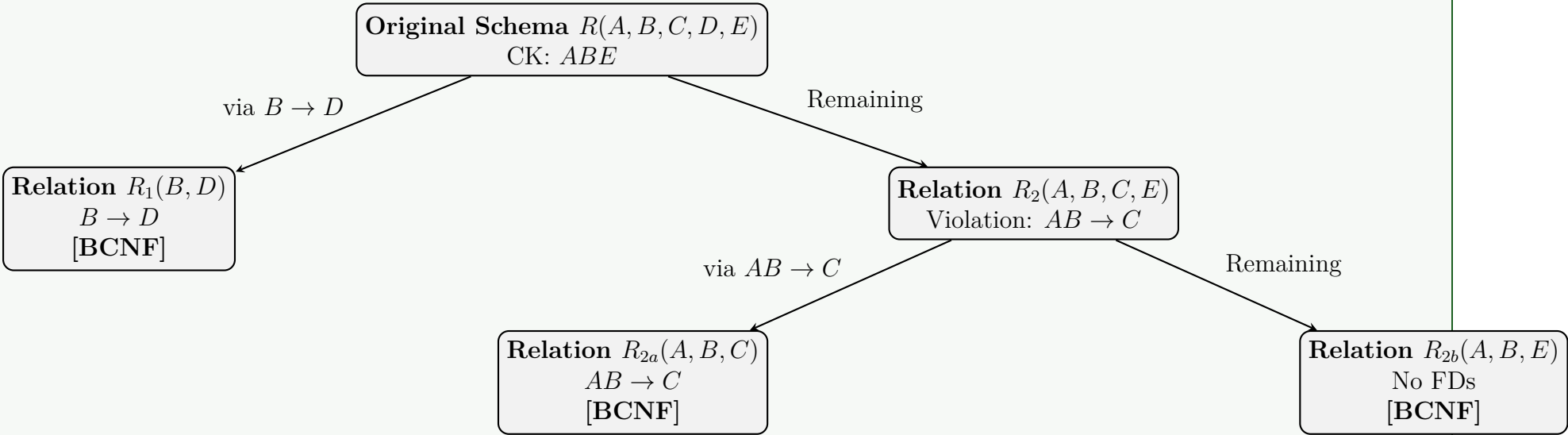
Step 3.3: Decompose R_2 using $AB \rightarrow C$

To fix the violation in R_2 , we split it using $AB \rightarrow C$:

- $R_{2a} = \{A, B\} \cup \{C\} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$
- $R_{2b} = R_2 - \{C\} = \{\mathbf{A}, \mathbf{B}, \mathbf{E}\}$

Step 3.4: Final Verification of the Leaf Schemas

- For $R_{2a}(A, B, C)$:** The projected dependency is $AB \rightarrow C$. The candidate key is AB . Since AB is a superkey, R_{2a} is in **BCNF**.
- For $R_{2b}(A, B, E)$:** No non-trivial functional dependencies are projected onto this schema. The candidate key is ABE . With no dependencies violating the rule, R_{2b} is in **BCNF**.



4. Final Normalized BCNF Schema

The final collection of relations, their primary keys (underlined>, and foreign keys are listed below:

(a) $R_1(\underline{B}, D)$

- **Candidate Key:** B
- **Projected FDs:** $B \rightarrow D$

(b) $R_{2a}(\underline{A}, \underline{B}, C)$

- **Candidate Key:** AB
- **Projected FDs:** $AB \rightarrow C$
- **Foreign Key:** B references $R_1(B)$

(c) $R_{2b}(\underline{A}, \underline{B}, E)$

- **Candidate Key:** ABE
- **Projected FDs:** None
- **Foreign Key:** (A, B) references $R_{2a}(A, B)$

Note on Dependency Preservation: This decomposition is lossless but **not dependency-preserving** because the original dependency $DE \rightarrow C$ cannot be verified within any single resulting schema without a join operation.

Q6. What do you know about BCNF and 3NF? What is the major difference between them? Let R be a schema that is not in BCNF and $\alpha \rightarrow \beta$ be the functional dependency that causes a violation of BCNF. How can R be decomposed such that R will be in BCNF? (10 marks) [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Third Normal Form (3NF)

Definition: A relation schema R is in Third Normal Form (3NF) if it is in Second Normal Form (2NF) and for every non-trivial functional dependency $X \rightarrow Y$ defined on R , at least one of the following conditions holds true:

- X is a **superkey** of R .
- Y is a **prime attribute** of R (i.e., Y is a part of some candidate key).

Properties:

- Ensures that no non-prime attribute is transitively dependent on the primary key.
- Always allows for a decomposition that is both **lossless-join** and **dependency-preserving**.
- May suffer from slight anomalies if overlapping candidate keys exist.

2. Boyce-Codd Normal Form (BCNF)

Definition: A relation schema R is in Boyce-Codd Normal Form (BCNF) if for every non-trivial functional dependency $X \rightarrow Y$ defined on R , the following condition holds true:

- X is a **superkey** of R .

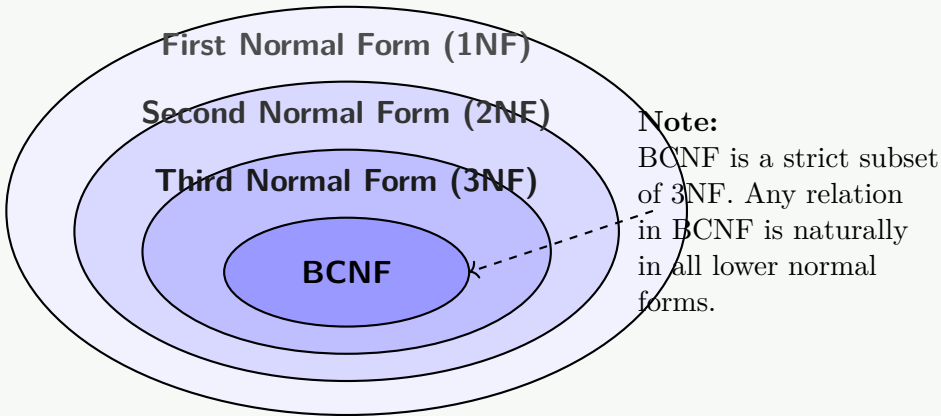
Properties:

- BCNF is a stronger, stricter version of 3NF. It eliminates the leniency allowed by the "prime attribute" condition in 3NF.
- Every relation in BCNF is mathematically guaranteed to also be in 3NF, but the reverse is not always true.
- BCNF decomposition is always **lossless**, but it is **not guaranteed to be dependency-preserving**.

3. Major Differences Between 3NF and BCNF

The fundamental distinction between 3NF and BCNF lies in how they handle dependencies where the determinant is not a superkey, but the dependent attribute is prime.

Feature	Third Normal Form (3NF)	Boyce-Codd Normal Form (BCNF)
Rule	$X \rightarrow Y$: X is superkey OR Y is prime.	$X \rightarrow Y$: X MUST be a superkey.
Strictness	Less strict.	More strict (eliminates more anomalies).
Dependency Preservation	Guaranteed to be dependency-preserving.	Not guaranteed to be dependency-preserving.
Overlapping Candidate Keys	Allows anomalies if a non-key attribute determines part of a candidate key.	Does not allow such anomalies.



4. Decomposition of Schema R into BCNF

Problem Statement: Let R be a schema that is not in BCNF, and let $\alpha \rightarrow \beta$ be the non-trivial functional dependency that causes the violation (meaning α is not a superkey of R , and $\alpha \cap \beta = \emptyset$).

Decomposition Algorithm: To eliminate the anomaly caused by the violating dependency $\alpha \rightarrow \beta$, we systematically decompose the original relation R into two smaller relations, R_1 and R_2 , defined as follows:

- (a) **First Relation (R_1):** Contains all the attributes in the violating functional dependency.

$$R_1 = \alpha \cup \beta$$

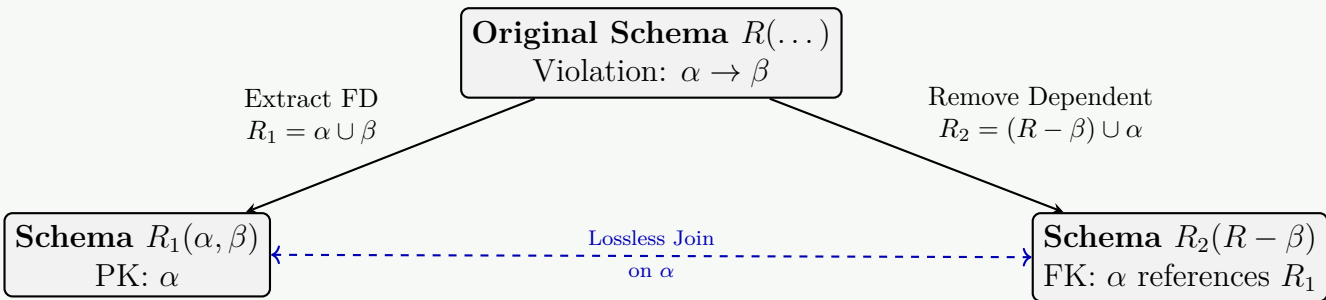
Primary Key of R_1 : α .

- (b) **Second Relation (R_2):** Contains the determinant of the violating dependency and all other attributes of R not involved in the dependent part.

$$R_2 = R - \beta$$

(Equivalently written as $R_2 = (R - \beta) \cup \alpha$ to emphasize that the determinant α is kept as the link).
Foreign Key of R_2 : α referencing $R_1(\alpha)$.

Working Principle: By extracting $\alpha \rightarrow \beta$ into its own table (R_1), the determinant α becomes the primary key of R_1 , thus becoming a superkey for that specific table. The remaining attributes stay in R_2 , where α acts as a foreign key to maintain a **lossless-join** property.



Termination Check: After performing this split, one must evaluate the projected functional dependencies for R_1 and R_2 . If either of these new schemas still contains a BCNF violation, the decomposition algorithm is applied recursively to the violating schema until all resulting sub-schemas are completely in BCNF.

Q7. Suppose that we decompose the schema $R = (A, B, C, D, E)$ into (A, B, C) and (A, D, E) . Show whether this decomposition is a

lossless decomposition if the following set F of functional dependencies holds:

$$A \rightarrow BC, \quad CD \rightarrow E, \quad B \rightarrow D, \quad E \rightarrow A$$

(10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Theorem for Lossless-Join Decomposition

A decomposition of a relation schema R into two schemas R_1 and R_2 is a **lossless-join decomposition** with respect to a set of functional dependencies F if and only if at least one of the following functional dependencies is in F^+ :

- (a) $(R_1 \cap R_2) \rightarrow R_1$ (which is equivalent to $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$)
- (b) $(R_1 \cap R_2) \rightarrow R_2$ (which is equivalent to $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$)

2. Evaluation of the Given Schema and Decomposition

We are given:

- Original Schema: $R = (A, B, C, D, E)$
- Decomposed Schemas: $R_1 = (A, B, C)$ and $R_2 = (A, D, E)$
- Set of Functional Dependencies F :

$$\begin{aligned} A &\rightarrow BC \\ CD &\rightarrow E \\ B &\rightarrow D \\ E &\rightarrow A \end{aligned}$$

Let us identify the intersection and differences between R_1 and R_2 :

- **Intersection:** $R_1 \cap R_2 = \{A\}$
- **Difference ($R_1 - R_2$):** $\{A, B, C\} - \{A, D, E\} = \{B, C\}$
- **Difference ($R_2 - R_1$):** $\{A, D, E\} - \{A, B, C\} = \{D, E\}$

3. Attribute Closure Analysis

To determine if the intersection $R_1 \cap R_2 = \{A\}$ functionally determines either $(R_1 - R_2)$ or $(R_2 - R_1)$, we compute the attribute closure of A (denoted as A^+) under the given set of FDs F :

(a) **Initialization:**

$$A^+ = \{A\}$$

(b) **Iteration 1:** Since $A \rightarrow BC$ is given and $A \subseteq A^+$, we add B and C to the closure:

$$A^+ = \{A, B, C\}$$

(c) **Iteration 2:** Since $B \rightarrow D$ is given and $B \subseteq A^+$, we add D to the closure:

$$A^+ = \{A, B, C, D\}$$

(d) **Iteration 3:** Since $CD \rightarrow E$ is given and $\{C, D\} \subseteq A^+$, we add E to the closure:

$$A^+ = \{A, B, C, D, E\}$$

Thus, the final attribute closure is:

$$A^+ = \{A, B, C, D, E\} = R$$

4. Verification of the Lossless-Join Condition

We check whether the intersection schema $\{A\}$ functionally determines either of the remaining subsets:

- Does $A \rightarrow R_1$ hold? Since $R_1 = \{A, B, C\}$ and $\{A, B, C\} \subseteq A^+$, the dependency $A \rightarrow R_1$ **holds**.
- Does $A \rightarrow R_2$ hold? Since $R_2 = \{A, D, E\}$ and $\{A, D, E\} \subseteq A^+$, the dependency $A \rightarrow R_2$ **holds**.

Alternative Verification: The Chase Matrix Method

We can visually verify this using a Chase matrix table. We construct rows for R_1 and R_2 , and columns for each attribute. If a row contains the attribute, we label it with a_i , otherwise b_{ij} .

Relation	A	B	C	D	E
$R_1(A, B, C)$	a_1	a_2	a_3	b_{14}	b_{15}
$R_2(A, D, E)$	a_1	b_{22}	b_{23}	a_4	a_5

Applying the functional dependencies to equate symbols:

- (a) Apply $A \rightarrow BC$: Both rows have a_1 in column A . Therefore, their values in columns B and C must match. Row 2 updates to match Row 1's a_2 and a_3 .
- (b) Apply $B \rightarrow D$: Both rows now have a_2 in column B . Therefore, their values in column D must match. Row 1 updates to a_4 .
- (c) Apply $CD \rightarrow E$: Both rows now have a_3 and a_4 in columns C and D . Therefore, column E values must match. Row 1 updates to a_5 .

The updated matrix becomes:

Relation	A	B	C	D	E
$R_1(A, B, C)$	a_1	a_2	a_3	a_4	a_5
$R_2(A, D, E)$	a_1	a_2	a_3	a_4	a_5

Since we obtained at least one row entirely composed of distinguished variables $(a_1, a_2, a_3, a_4, a_5)$, the decomposition is confirmed to be lossless.

5. Conclusion

Since $R_1 \cap R_2 = \{A\}$ is a superkey for both R_1 and R_2 , any natural join between the two relations will reconstruct the original table without introducing any spurious tuples.
Therefore, the decomposition of R into R_1 and R_2 is **guaranteed to be a lossless decomposition**.

Q8. “A table in 3NF may fail to meet the criteria of BCNF” — do you agree? Show an appropriate example to validate your claim. (10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Affirmation of the Claim

Yes, I completely **agree** with the statement. A relation that satisfies the requirements of the Third Normal Form (3NF) can still fail to meet the stricter criteria of the Boyce-Codd Normal Form (BCNF).
This occurs when a relation contains **overlapping candidate keys** (i.e., composite candidate keys that share at least one attribute) and a functional dependency exists where a part of one candidate key is determined by a non-trivial combination of attributes that is not itself a superkey.

2. The Key Theoretical Difference

- To understand why this happens, we must compare the formal definitions of both normal forms for any non-trivial functional dependency $X \rightarrow Y$:
- **Third Normal Form (3NF)**: Allows the dependency if X is a superkey **OR** if Y is a prime attribute (a member of any candidate key).
 - **Boyce-Codd Normal Form (BCNF)**: Strictly requires that X **MUST** be a superkey, completely removing the "prime attribute" loophole.

Therefore, any relation containing a dependency $X \rightarrow Y$ where X is *not* a superkey but Y *is* a prime attribute satisfies 3NF but **violates BCNF**.

3. Illustrative Example: The Consultant-Client Assignment

Consider a database schema designed for a consulting firm that tracks projects assigned to clients and the specialized consultants managing them:

Assignment (Client, Consultant, Specialty)

Domain Constraints and Business Rules

- (a) For any given **Client** and a specific **Specialty**, only one **Consultant** is assigned.

- (b) Each **Consultant** has exactly one unique **Specialty**.
- (c) A **Specialty** can have multiple consultants, and a **Consultant** can work with multiple clients.

Functional Dependencies (FDs)

The semantic business rules yield the following set of dependencies F :

FD1: $\{\text{Client}, \text{Specialty}\} \rightarrow \text{Consultant}$

FD2: $\text{Consultant} \rightarrow \text{Specialty}$

4. Validation of Candidate Keys

Let us compute the closures to determine the candidate keys for the **Assignment** table:

- $\{\text{Client}, \text{Specialty}\}^+ = \{\text{Client}, \text{Specialty}, \text{Consultant}\} \implies \text{Candidate Key 1}$
- $\{\text{Client}, \text{Consultant}\}^+ = \{\text{Client}, \text{Consultant}, \text{Specialty}\}$ (via FD2) $\implies \text{Candidate Key 2}$

The **prime attributes** (attributes belonging to any candidate key) are: **Client, Specialty, and Consultant**. Since every attribute in this relation is prime, there are no non-prime attributes.

5. Proof: Why it is in 3NF but NOT in BCNF

3NF Verification

We test both functional dependencies against the 3NF rule:

- (a) For **FD1** ($\{\text{Client}, \text{Specialty}\} \rightarrow \text{Consultant}$): The determinant is a candidate key (superkey). (**Valid under 3NF**)
- (b) For **FD2** ($\text{Consultant} \rightarrow \text{Specialty}$): The determinant is *not* a superkey. However, the dependent attribute **Specialty** is a prime attribute. (**Valid under 3NF**)

Since both dependencies pass the check, the table is fully in **3NF**.

BCNF Verification

We test the same dependencies against the stricter BCNF rule:

- (a) For **FD1**: The determinant is a superkey. (**Valid under BCNF**)
- (b) For **FD2** ($\text{Consultant} \rightarrow \text{Specialty}$): The determinant **Consultant** is **not a superkey** because its closure does not include **Client**.

Because of **FD2**, the table explicitly **violates BCNF**.

6. Resolving the Violation

To move this 3NF design into BCNF, we decompose the table using the violating dependency ($\text{Consultant} \rightarrow \text{Specialty}$):

- $R_1(\underline{\text{Consultant}}, \text{Specialty})$ with candidate key: **Consultant**
- $R_2(\text{Client}, \underline{\text{Consultant}})$ with candidate key: **{Client, Consultant}**

Both schemas R_1 and R_2 are now in BCNF. However, the original dependency $\{\text{Client}, \text{Specialty}\} \rightarrow \text{Consultant}$ can no longer be enforced locally without a join operation, showing the classic trade-off between BCNF and dependency preservation.

Q9. The following schema has been designed for an address book application:

$R(\underline{SSN}, \text{Name}, \text{PhoneType}, \text{PhoneNumber})$

The following FDs hold: $SSN \rightarrow \text{Name}$; $SSN, \text{PhoneType} \rightarrow \text{PhoneNumber}$; $SSN, \text{PhoneType} \rightarrow \text{Name}$; $\text{PhoneNumber} \rightarrow SSN, \text{Name}, \text{PhoneType}$.

- (a) What are the keys for R ?
- (b) Is R in BCNF? Why or why not?
- (c) Decompose R into BCNF (if not already in BCNF).

(15 marks)

[CSE 215 | L-2/T-2 | 2017–18]

$\hookrightarrow$ Similar: 2011–12: same question with additional subpart (iii) “Can you remove any dependency without violating equivalency of F ?”

Answer

1. Finding the Candidate Keys for R

To determine the candidate keys for the relation schema $R(SSN, Name, PhoneType, PhoneNumber)$, we look at the given set of functional dependencies F :

1. $SSN \rightarrow Name$

2. $SSN, PhoneType \rightarrow PhoneNumber$

3. $SSN, PhoneType \rightarrow Name$

4. $PhoneNumber \rightarrow SSN, Name, PhoneType$

Let us compute the attribute closures to find the minimal superkeys:

- Closure of $\{SSN, PhoneType\}$:**

$$\begin{aligned}(SSN, PhoneType)^+ &= \{SSN, PhoneType\} \\ &= \{SSN, PhoneType, PhoneNumber\} \quad (\text{using FD 2}) \\ &= \{SSN, PhoneType, PhoneNumber, Name\} \quad (\text{using FD 1 or 3})\end{aligned}$$
- Since $(SSN, PhoneType)^+$ contains all attributes of R and no proper subset of it determines all attributes, **$\{SSN, PhoneType\}$** is a candidate key.
- Closure of $\{PhoneNumber\}$:**

$$(PhoneNumber)^+ = \{PhoneNumber, SSN, Name, PhoneType\} \quad (\text{using FD 4})$$
- Since $(PhoneNumber)^+$ contains all attributes of R , **$\{PhoneNumber\}$** is a candidate key.

Conclusion: The relation R has two **Candidate Keys**:

(a) **$\{SSN, PhoneType\}$**

(b) **$\{PhoneNumber\}$**

2. BCNF Analysis

A relation schema is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey.

Let us evaluate the given dependencies:

Functional Dependency	Determinant	Is Determinant a Superkey?	BCNF Status
$SSN \rightarrow Name$	SSN	No ($(SSN)^+ = \{SSN, Name\}$)	Violation
$SSN, PhoneType \rightarrow PhoneNumber$	$\{SSN, PhoneType\}$	Yes (Candidate Key)	Valid
$SSN, PhoneType \rightarrow Name$	$\{SSN, PhoneType\}$	Yes (Candidate Key)	Valid
$PhoneNumber \rightarrow SSN, Name, PhoneType$	$PhoneNumber$	Yes (Candidate Key)	Valid

Conclusion: No, R is **NOT in BCNF** because the dependency $SSN \rightarrow Name$ violates the rule (SSN is a determinant but it is not a superkey).

3. BCNF Decomposition

To transform the schema into BCNF, we decompose R using the violating functional dependency $SSN \rightarrow Name$.

(a) **First Relation (R_1):** Formed by the attributes of the violating dependency.

$$R_1 = \{SSN\} \cup \{Name\} = \{SSN, Name\}$$

- Projected FD:** $SSN \rightarrow Name$
- Candidate Key:** SSN
- BCNF Status:** Valid (SSN is a superkey for R_1).

(b) **Second Relation (R_2):** Formed by keeping the determinant and removing the dependent attribute from the original schema.

$$R_2 = R - \{Name\} = \{SSN, PhoneType, PhoneNumber\}$$

- Projected FDs:**
 - $SSN, PhoneType \rightarrow PhoneNumber$
 - $PhoneNumber \rightarrow SSN, PhoneType$ (derived from FD 4)
- Candidate Keys:** $\{SSN, PhoneType\}$ and $\{PhoneNumber\}$
- BCNF Status:** Valid, because the determinants for all non-trivial dependencies in R_2 are superkeys.

Final Normalized Relational Schema

- $R_1(\underline{SSN}, Name)$
- $R_2(\underline{SSN}, PhoneType, PhoneNumber)$ or $R_2(SSN, PhoneType, \underline{PhoneNumber})$
- Note: SSN in R_2 acts as a **Foreign Key** referencing R_1 .

Original Schema R
 $(SSN, Name, PhoneType, PhoneNumber)$
Violation: $SSN \rightarrow Name$

Extract violation

Schema R_1
 $(\underline{SSN}, Name)$
[BCNF]

Foreign Key Relationship
 (SSN)

Remaining attributes

Schema R_2
 $(\underline{SSN}, PhoneType, PhoneNumber)$
[BCNF]

Q10. Give an example of a relation which is in 3NF but not in BCNF. Define Multivalued Dependency. (10 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Relation in 3NF but not in BCNF

Definition and Structural Criteria

A relation schema R is in **Third Normal Form (3NF)** if, for every non-trivial functional dependency $X \rightarrow Y$, either X is a superkey or Y is a prime attribute.

A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if, for every non-trivial functional dependency $X \rightarrow Y$, X is strictly a superkey. Thus, a BCNF violation occurs in 3NF when a non-superkey determines a prime attribute.

Example: The Movie Production Schema

Consider a relation that stores information about movie productions where each studio assigns specific producers to direct projects within a specific genre:

Production (Studio, Genre, Producer)

Domain Constraints

(a) For a given **Studio** and a specific **Genre**, there is exactly one designated **Producer**.

(b) Each **Producer** specializes in exactly one **Genre** and works exclusively within it.

(c) A single **Genre** can have multiple producers across different studios.

Functional Dependencies (FDs)

Based on these constraints, we establish the following functional dependencies:

FD1: $\{Studio, Genre\} \rightarrow Producer$

FD2: $Producer \rightarrow Genre$

Candidate Keys and Prime Attributes

Let us calculate the attribute closures:

- $\{Studio, Genre\}^+ = \{Studio, Genre, Producer\} \implies$ **Candidate Key 1**
- $\{Studio, Producer\}^+ = \{Studio, Producer, Genre\}$ (via FD2) $\implies$ **Candidate Key 2**

The **prime attributes** are **Studio**, **Genre**, and **Producer**.

Normal Form Evaluation

- **Why it is in 3NF:** In FD1, $\{Studio, Genre\}$ is a superkey. In FD2, the determinant **Producer** is not a superkey, but the dependent attribute **Genre** is a **prime attribute**. Therefore, it passes the 3NF requirements.
- **Why it is NOT in BCNF:** In FD2 ($Producer \rightarrow Genre$), the determinant **Producer** is **not a superkey**. Since BCNF does not allow the prime attribute loophole, the relation violates BCNF.

—

2. Multivalued Dependency (MVD)

Definition

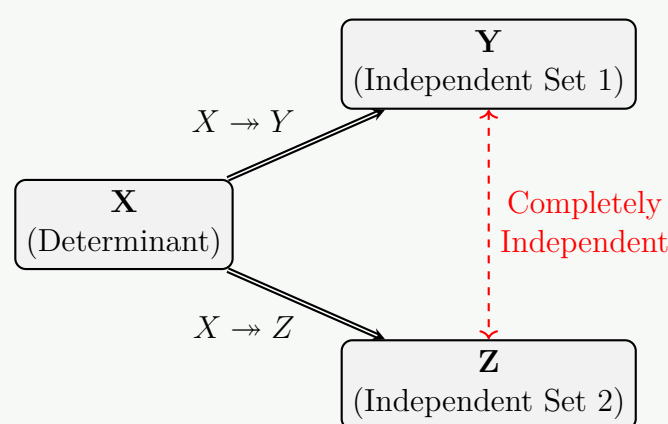
A **Multivalued Dependency** (denoted as $X \twoheadrightarrow Y$) exists on a relation schema R if, given values for a set of attributes X , the presence of a specific value for a set of attributes Y is completely independent of the values of the remaining attributes Z (where $Z = R - (X \cup Y)$).

Formally, $X \twoheadrightarrow Y$ holds on R if for all possible legal tuples t_1 and t_2 in R such that $t_1[X] = t_2[X]$, there must also exist tuples t_3 and t_4 in R such that:

$$\begin{aligned} t_3[X] &= t_4[X] = t_1[X] = t_2[X] \\ t_3[Y] &= t_1[Y] \quad \text{and} \quad t_3[Z] = t_2[Z] \\ t_4[Y] &= t_2[Y] \quad \text{and} \quad t_4[Z] = t_1[Z] \end{aligned}$$

Working Principle and Diagram

An MVD represents a **one-to-many** relationship that is independent of other attributes. When a single entity attribute X maps to multiple independent values in columns Y and Z , we are forced to repeat all combinations of Y and Z for every X value to maintain relational consistency.



Example of Multivalued Dependency

Consider a professor database tracking teaching specialties and extracurricular hobbies:

ProfessorData (Prof_ID, Course, Hobby)

If Professor P1 teaches "Databases" and "OS", and has hobbies "Chess" and "Music", the data must explicitly state all cross-product tuples to avoid structural distortion:

Prof_ID (X)	Course (Y)	Hobby (Z)
P1	Databases	Chess
P1	Databases	Music
P1	OS	Chess
P1	OS	Music

Here, $\text{Prof_ID} \twoheadrightarrow \text{Course}$ and $\text{Prof_ID} \twoheadrightarrow \text{Hobby}$ hold independently. This structural redundancy is resolved by decomposing the table into Fourth Normal Form (4NF).

Q11. Prove that any two-attribute relation is in BCNF. (7 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Mathematical Definitions and Preliminaries

To establish the proof that any two-attribute relation schema is guaranteed to be in **Boyce-Codd Normal Form (BCNF)**, we begin by stating the formal definitions of a relation schema and the requirements for BCNF.

Relation Schema

Let $R(A, B)$ be a relation schema consisting of exactly two distinct attributes, A and B . The set of all attributes in the schema is denoted by $\text{Attr}(R) = \{A, B\}$.

Boyce-Codd Normal Form (BCNF)

A relation schema R is in **BCNF** with respect to a set of functional dependencies F if, for every non-trivial functional dependency $X \rightarrow Y$ in F^+ , the determinant X is a **superkey** of R .

Trivial vs. Non-Trivial Dependencies

A functional dependency $X \rightarrow Y$ is defined as:

- **Trivial:** If $Y \subseteq X$. Trivial dependencies hold universally in all relations and can never violate any normal form.
- **Non-Trivial:** If $Y \not\subseteq X$. Only non-trivial dependencies can potentially cause normal form violations.

2. Exhaustive Case Analysis for $R(A, B)$

We evaluate every mathematically possible set of functional dependencies F that can exist over the schema $\text{Attr}(R) = \{A, B\}$. Since any functional dependency $X \rightarrow Y$ must draw its attributes from $\{A, B\}$, we can systematically analyze the cases based on the presence of non-trivial dependencies.

The only possible non-trivial functional dependencies that can exist in $R(A, B)$ are:

- (a) $A \rightarrow B$
- (b) $B \rightarrow A$

Case 1: No non-trivial functional dependencies exist ($F = \emptyset$)

If F contains no non-trivial dependencies, then F^+ contains only trivial dependencies of the form $A \rightarrow A$, $B \rightarrow B$, $AB \rightarrow A$, $AB \rightarrow B$, and $AB \rightarrow AB$.

- **Candidate Key:** Since no single attribute can determine the other, the only candidate key is the composite key $\{A, B\}$.
- **BCNF Evaluation:** Because there are no non-trivial functional dependencies, there are no determinants to evaluate.
- **Conclusion:** R satisfies BCNF vacuously.

Case 2: Only $A \rightarrow B$ exists ($F = \{A \rightarrow B\}$)

Assume the dependency $A \rightarrow B$ holds, but $B \rightarrow A$ does not hold.

- **Candidate Key Calculation:** We compute the attribute closure of A :

$$(A)^+ = \{A, B\} = \text{Attr}(R)$$

Since A determines all attributes of the relation and is minimal, $\{A\}$ is the unique candidate key of R .

- **BCNF Evaluation:** The only non-trivial functional dependency to evaluate is $A \rightarrow B$. The determinant is $X = \{A\}$. Since $\{A\}$ is a candidate key, it is by definition a superkey.
- **Conclusion:** R satisfies BCNF.

Case 3: Only $B \rightarrow A$ exists ($F = \{B \rightarrow A\}$)

Assume the dependency $B \rightarrow A$ holds, but $A \rightarrow B$ does not hold.

- **Candidate Key Calculation:** We compute the attribute closure of B :

$$(B)^+ = \{B, A\} = \text{Attr}(R)$$

Since B determines all attributes of the relation and is minimal, $\{B\}$ is the unique candidate key of R .

- **BCNF Evaluation:** The only non-trivial functional dependency to evaluate is $B \rightarrow A$. The determinant is $X = \{B\}$. Since $\{B\}$ is a candidate key, it is by definition a superkey.
- **Conclusion:** R satisfies BCNF.

Case 4: Both $A \rightarrow B$ and $B \rightarrow A$ exist ($F = \{A \rightarrow B, B \rightarrow A\}$)

Assume both dependencies hold simultaneously.

- **Candidate Key Calculation:** We compute the closures for both individual attributes:

$$(A)^+ = \{A, B\} = \text{Attr}(R) \implies \{A\} \text{ is a candidate key}$$

$$(B)^+ = \{B, A\} = \text{Attr}(R) \implies \{B\} \text{ is a candidate key}$$

Thus, R possesses two alternative, single-attribute candidate keys.

- **BCNF Evaluation:** We check both non-trivial functional dependencies:
 - (a) For $A \rightarrow B$: The determinant is $\{A\}$, which is a candidate key (superkey).
 - (b) For $B \rightarrow A$: The determinant is $\{B\}$, which is a candidate key (superkey).
- **Conclusion:** R satisfies BCNF because the determinants of all non-trivial dependencies are superkeys.

3. Formal Proof by Contradiction

To provide a rigorous mathematical validation, we can construct a proof by contradiction.

- (a) Suppose there exists a two-attribute relation schema $R(A, B)$ that is **NOT in BCNF**.
- (b) By the definition of BCNF violations, there must exist at least one non-trivial functional dependency $X \rightarrow Y$ in F^+ such that the determinant X is **not a superkey** of R .
- (c) Since the dependency $X \rightarrow Y$ is non-trivial, $Y \not\subseteq X$, which implies that the dependent set Y must contain at least one attribute that is not present in the determinant X .
- (d) Given that $\text{Attr}(R) = \{A, B\}$ has a cardinality of exactly 2 ($|\text{Attr}(R)| = 2$), let us analyze the size of the determinant set X :
 - If $|X| = 2$, then $X = \{A, B\}$. A set containing all attributes of a relation is always a superkey. This contradicts our assumption that X is not a superkey.
 - If $|X| = 0$, then $X = \emptyset$. A dependency of the form $\emptyset \rightarrow Y$ implies that Y consists of constant values. If Y is non-empty, the key of the relation would still be determined by the remaining attributes, which does not break BCNF logic.
 - Therefore, the determinant X must be a single-attribute set. Without loss of generality, let $X = \{A\}$.
- (e) Since $X = \{A\}$ and the dependency $X \rightarrow Y$ is non-trivial, the dependent set Y must contain the remaining attribute, meaning $Y = \{B\}$ (or $Y = \{A, B\}$, which collapses to the same non-trivial fact). Thus, the violating dependency is:

$$A \rightarrow B$$

- (f) Let us compute the attribute closure of the determinant under this assumption:

$$(A)^+ = \{A\} \cup \{B\} = \{A, B\} = \text{Attr}(R)$$

- (g) Because $(A)^+$ contains every attribute in the schema R , the set $X = \{A\}$ is, by definition, a **superkey** of R .
- (h) This directly contradicts our initial assumption in Step 2 that X is not a superkey.
- (i) Since our assumption leads to a direct contradiction in all structural configurations, the hypothesis must be false.

Hence, it is mathematically impossible for a two-attribute relation to violate BCNF.

■ Q.E.D.

4. Summary Matrix of Properties

The following summary table outlines the behavioral characteristics of binary relations under normalization theory:

Functional Dependencies (F)	Candidate Key(s)	Violating FDs	Highest Normal Form
$\emptyset$ (Only trivial FDs)	$\{A, B\}$	None	BCNF (Vacuous)
$\{A \rightarrow B\}$	$\{A\}$	None	BCNF
$\{B \rightarrow A\}$	$\{B\}$	None	BCNF
$\{A \rightarrow B, B \rightarrow A\}$	$\{A\}, \{B\}$	None	BCNF

Design Note: This theorem provides a powerful tool during database decomposition. When splitting higher-order relations to eliminate anomalies, decomposing down to two-attribute schemas (binary relations) guarantees that the leaf nodes of the decomposition tree are fully optimized and free from any functional dependency anomalies.

Q12. For the relation schema $R(A, B, C, D, E)$ with FDs $AB \rightarrow C$, $DE \rightarrow C$, and $B \rightarrow D$:

- (a) Indicate all the BCNF violations.
- (b) Decompose the relations, as necessary, into a collection of relations that are in BCNF.

(10 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Candidate Key Determination

Before evaluating Boyce-Codd Normal Form (BCNF) violations, we must first determine the candidate key(s) for the relation $R(A, B, C, D, E)$.

We are given the set of functional dependencies F :

$$\begin{aligned} AB &\rightarrow C \\ DE &\rightarrow C \\ B &\rightarrow D \end{aligned}$$

To find the candidate key, we identify the attributes that **never appear on the right-hand side** of any functional dependency. These attributes must be part of every candidate key:

- Attributes A , B , and E do not appear on the right-hand side of any given FD. Therefore, $\{A, B, E\}$ must be part of the key.
- We compute the attribute closure of $\{A, B, E\}$ under F :

$$\begin{aligned} (ABE)^+ &= \{A, B, E\} \\ &= \{A, B, D, E\} \quad (\text{using } B \rightarrow D) \\ &= \{A, B, C, D, E\} \quad (\text{using } AB \rightarrow C \text{ or } DE \rightarrow C) \end{aligned}$$

Since $(ABE)^+ = \{A, B, C, D, E\}$ (all attributes in R) and no proper subset of $\{A, B, E\}$ can determine all attributes, the unique **Candidate Key** for R is **ABE**.

Part 1: Indicate All BCNF Violations

A relation schema is in **BCNF** if and only if, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey of the relation.

We evaluate all the given non-trivial functional dependencies against this rule:

Functional Dependency	Determinant (X)	Closure of X (X^+)	Is X a Superkey?
$AB \rightarrow C$	AB	$\{A, B, C, D\}$	No
$DE \rightarrow C$	DE	$\{C, D, E\}$	No
$B \rightarrow D$	B	$\{B, D\}$	No

Conclusion for Part 1: Because **none** of the determinants (AB , DE , or B) are superkeys of R (none of their closures contain all attributes of R), **all three functional dependencies are BCNF violations**.

Part 2: Decompose into a Collection of BCNF Relations

To achieve BCNF, we systematically decompose the relation by selecting a violating functional dependency and splitting the schema into two relations: one containing the attributes of the violating FD, and the other containing the remaining attributes along with the determinant.

Step 1: Decompose R using the violation $B \rightarrow D$

We extract $B \rightarrow D$ from the original schema $R(A, B, C, D, E)$:

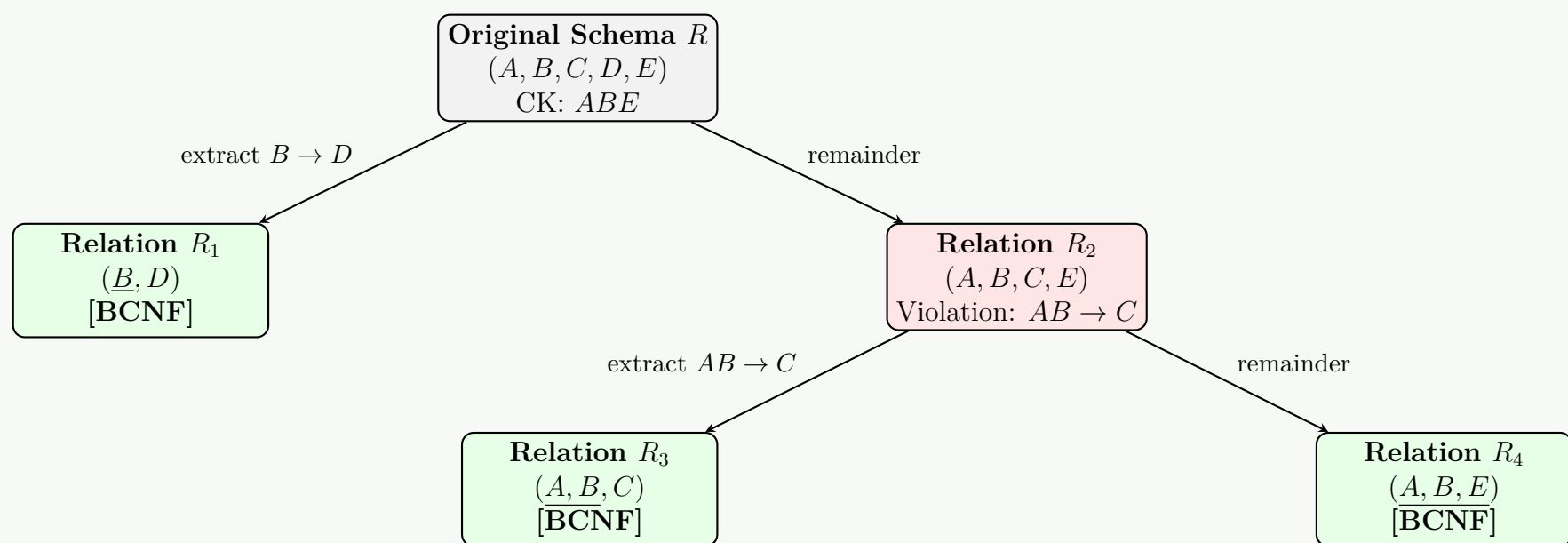
- Relation R_1 (Extract the FD):** $R_1 = X \cup Y = \{B, D\}$
 - Candidate Key: B
 - Projected FDs: $B \rightarrow D$
 - Status: **BCNF** (because B is a superkey of R_1).
- Relation R_2 (Remaining attributes + Determinant):** $R_2 = R - \{D\} = \{A, B, C, E\}$
 - Candidate Key: ABE
 - Projected FDs: $AB \rightarrow C$ applies. (Note: $DE \rightarrow C$ is lost because D is no longer in this schema).
 - Status: **Not in BCNF** (because $AB \rightarrow C$ holds in R_2 , but AB is not a superkey of R_2).

Step 2: Decompose R_2 using the violation $AB \rightarrow C$

We must further decompose $R_2(A, B, C, E)$ to resolve the $AB \rightarrow C$ violation:

- Relation R_3 (Extract the FD):** $R_3 = X \cup Y = \{A, B, C\}$
 - Candidate Key: AB
 - Projected FDs: $AB \rightarrow C$
 - Status: **BCNF** (because AB is a superkey of R_3).
- Relation R_4 (Remaining attributes + Determinant):** $R_4 = R_2 - \{C\} = \{A, B, E\}$
 - Candidate Key: ABE
 - Projected FDs: None (only trivial dependencies remain).
 - Status: **BCNF**.

Decomposition Tree Diagram



Final Normalized BCNF Schema Collection

The final decomposition results in the following three relations, which are all in BCNF:

- (a) $R_1(\underline{B}, D)$
- (b) $R_3(\underline{A}, \underline{B}, C)$ (with Foreign Key $\{B\}$ referencing R_1)
- (c) $R_4(\underline{A}, \underline{B}, E)$ (with Foreign Key $\{A, B\}$ referencing R_3)

Note on Dependency Preservation: This decomposition guarantees a **lossless join**. However, it is **not dependency-preserving** because the original functional dependency $DE \rightarrow C$ spans across the decomposed tables and cannot be locally verified within a single relation without performing a join.

Q13. Suppose R is a relation with attributes $A_1, A_2, \dots, A_n$. As a function of n , find how many superkeys R has if the only keys are $\{A_1\}$ and $\{A_2, A_3\}$. (6 marks) [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Definition and Core Concept

A **superkey** is defined as any set of attributes that uniquely identifies tuples in a relation. By definition, any superset of a candidate key is automatically a superkey.

Given the relation R with n attributes, $U = \{A_1, A_2, \dots, A_n\}$, we are provided with exactly two candidate keys:

- $K_1 = \{A_1\}$
- $K_2 = \{A_2, A_3\}$

Note: Since the key K_2 requires the attribute A_3 , we assume $n \geq 3$.

To find the total number of superkeys, we must count all subsets of U that contain either K_1 or K_2 (or both). We will use the **Principle of Inclusion-Exclusion** from set theory.

2. Step-by-Step Calculation

Step 2.1: Superkeys generated by K_1

Let S_1 be the set of all superkeys containing $K_1 = \{A_1\}$. Since A_1 must be present, we are left with $(n - 1)$ attributes: $\{A_2, A_3, \dots, A_n\}$. Each of these remaining attributes can either be included or excluded from the superkey.

$$|S_1| = 2^{n-1} \quad (1)$$

Step 2.2: Superkeys generated by K_2

Let S_2 be the set of all superkeys containing $K_2 = \{A_2, A_3\}$. Since both A_2 and A_3 must be present, we are left with $(n - 2)$ attributes: $\{A_1, A_4, \dots, A_n\}$.

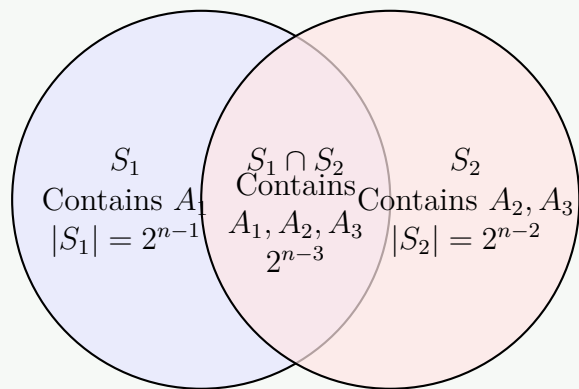
$$|S_2| = 2^{n-2} \quad (2)$$

Step 2.3: Intersection (Superkeys generated by both)

Some superkeys contain *both* candidate keys, meaning they contain the union of K_1 and K_2 , which is $\{A_1, A_2, A_3\}$. These superkeys have been counted twice (once in S_1 and once in S_2). Let $S_1 \cap S_2$ be the set of superkeys containing $\{A_1, A_2, A_3\}$. This leaves $(n - 3)$

attributes.

$$|S_1 \cap S_2| = 2^{n-3} \quad (3)$$



3. Final Computation using Inclusion-Exclusion

The total number of unique superkeys for relation R is the cardinality of the union of S_1 and S_2 :

$$\begin{aligned} |S_1 \cup S_2| &= |S_1| + |S_2| - |S_1 \cap S_2| \\ &= 2^{n-1} + 2^{n-2} - 2^{n-3} \end{aligned}$$

To simplify this expression, we factor out the common term 2^{n-3} :

$$\begin{aligned} |S_1 \cup S_2| &= (2^2 \cdot 2^{n-3}) + (2^1 \cdot 2^{n-3}) - (1 \cdot 2^{n-3}) \\ &= (4 + 2 - 1) \cdot 2^{n-3} \\ &= 5 \cdot 2^{n-3} \end{aligned}$$

4. Conclusion

The total number of superkeys for the relation R , as a function of n (where $n \geq 3$), is exactly:

$$5 \cdot 2^{n-3}$$

Q14. Show an example where a table is in 3NF, but not in BCNF. Then modify the design to satisfy BCNF. **(10 marks)** [CSE 303 | L-3/T-1 | 2014–15]

↪ *Similar: 2013–14: “Show an example where a table is in 3NF, but not in BCNF.” (5 marks)*

Answer

1. Example of a Table in 3NF but not in BCNF

A classic example of a relation that is in Third Normal Form (3NF) but violates Boyce-Codd Normal Form (BCNF) occurs when a relation contains overlapping candidate keys.

Consider a university database with a relation **Enrollment** that stores information about students, the courses they are taking, and the instructors teaching those courses.

Relational Schema

Enrollment(Student, Course, Instructor)

Business Rules and Assumptions

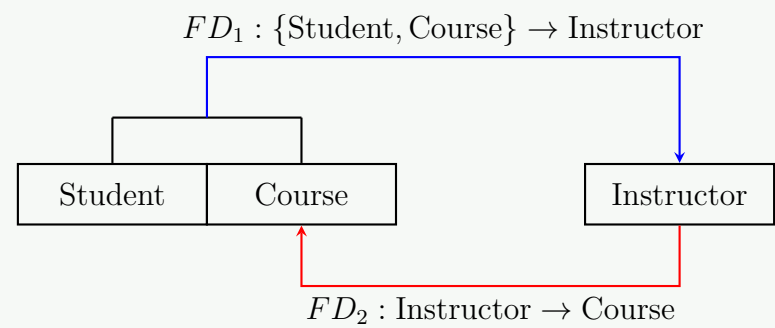
- A student can take multiple courses, but for each course, they are assigned exactly one instructor.
- A course can have multiple instructors.
- Every instructor teaches exactly one course.

Functional Dependencies (FDs)

Based on the rules above, we can identify the following functional dependencies:

$$\begin{aligned} FD_1 &: \{\text{Student, Course}\} \rightarrow \text{Instructor} \\ FD_2 &: \text{Instructor} \rightarrow \text{Course} \end{aligned}$$

Functional Dependency Diagram



Sample Data Table

Student	Course	Instructor
Alice	DBMS	Dr. Smith
Bob	DBMS	Dr. Smith
Charlie	DBMS	Dr. Jones
Alice	OS	Dr. White

2. Proving 3NF but NOT BCNF

To determine the normal form, we first find the candidate keys and prime attributes.

- **Candidate Keys:**
 - (a) {Student, Course} (Derived directly from FD_1)
 - (b) {Student, Instructor} (Since $Instructor \rightarrow Course$, adding Student to both sides yields $\{Student, Instructor\} \rightarrow \{Student, Course\}$, which determines all attributes).
- **Prime Attributes:** Student, Course, Instructor (All attributes are part of some candidate key).

Checking 3NF

A relation is in 3NF if, for every non-trivial functional dependency $X \rightarrow Y$, **either** X is a superkey **or** Y is a prime attribute.

- **For FD_1 ($\{Student, Course\} \rightarrow Instructor$):** The LHS ($\{Student, Course\}$) is a superkey. (Satisfies 3NF)
- **For FD_2 ($Instructor \rightarrow Course$):** The LHS (Instructor) is **not** a superkey. However, the RHS (Course) **is** a prime attribute. (Satisfies 3NF)

Conclusion: The relation **Enrollment** is in 3NF.

Checking BCNF

A relation is in BCNF if, for *every* non-trivial functional dependency $X \rightarrow Y$, X **must** be a superkey.

- **For FD_2 ($Instructor \rightarrow Course$):** The LHS (Instructor) is **not** a superkey, and BCNF does not allow the "prime attribute" exception.

Conclusion: The relation **Enrollment** violates BCNF. Because of this, it suffers from redundancy and update anomalies (e.g., if Dr. Smith changes the course he teaches, multiple rows must be updated).

3. Modifying the Design to Satisfy BCNF

To convert a relation to BCNF, we decompose it using the functional dependency that violates BCNF. In this case, FD_2 ($Instructor \rightarrow Course$) is the violator.

Decomposition Rule: If $X \rightarrow Y$ violates BCNF, we decompose the relation R into two relations:

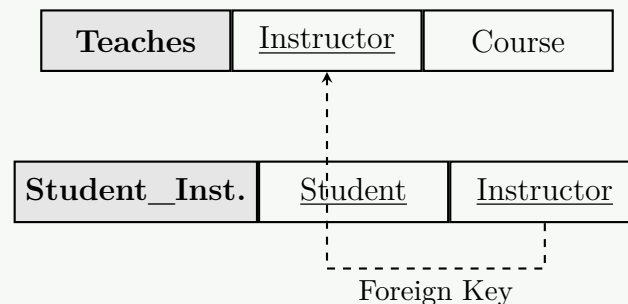
- (a) $R_1(X, Y)$ (The violating dependency becomes its own table)
- (b) $R_2(X, \text{Rest of the attributes})$

Applying this rule using $Instructor \rightarrow Course$:

- **Teaches**(Instructor, Course)
- **Student_Instructor**(Student, Instructor)

Validating the Decomposition

- **Teaches Table:** The only FD is $\text{Instructor} \rightarrow \text{Course}$. **Instructor** is the candidate key. Since the LHS is a superkey, it is in BCNF.
- **Student_Instructor Table:** The only candidate key is $\{\text{Student}, \text{Instructor}\}$. There are no non-trivial FDs. Thus, it is trivially in BCNF.
- **Lossless Join:** The common attribute is **Instructor**. Since **Instructor** is a superkey in the **Teaches** table, the decomposition is lossless.

Decomposed Relational Schema Diagram

Q15. Suppose that we have a schema $R = (A, B, C, D)$ with functional dependencies:

$$B \rightarrow C \quad D \rightarrow A$$

List the keys of R . Is R in BCNF? If not, convert R to BCNF. **(15 marks)**

[CSE 303 | L-3/T-1 | 2012–13]

Answer**1. Derivation of Candidate Keys**

To find the candidate keys of the relation $R(A, B, C, D)$, we must identify the minimal set of attributes whose closure determines all attributes in R .

Given the functional dependencies (FDs):

$$F = \{B \rightarrow C, \quad D \rightarrow A\}$$

First, we observe that attributes B and D do not appear on the right-hand side of any functional dependency. Therefore, they **must** be part of any candidate key. Let us compute the attribute closure of $\{B, D\}$:

$$\begin{aligned} (BD)^+ &= \{B, D\} \quad (\text{Reflexivity}) \\ (BD)^+ &= \{B, D, C\} \quad (\text{Using } B \rightarrow C) \\ (BD)^+ &= \{B, D, C, A\} \quad (\text{Using } D \rightarrow A) \end{aligned}$$

Since $(BD)^+$ contains all attributes in the schema $\{A, B, C, D\}$, the set $\{B, D\}$ is a superkey. Because no proper subset of $\{B, D\}$ can determine all attributes (i.e., $B^+ = \{B, C\}$ and $D^+ = \{A, D\}$), it is minimal.

Conclusion: The only candidate key of R is $\{B, D\}$.

2. BCNF Evaluation

Definition: A relation schema is in Boyce-Codd Normal Form (BCNF) if and only if, for every non-trivial functional dependency $X \rightarrow Y$ defined on it, X is a superkey.

Let us test the given FDs against the candidate key $\{B, D\}$:

- **FD 1 ($B \rightarrow C$):** The left-hand side is B . However, $(B)^+ = \{B, C\} \neq \{A, B, C, D\}$. Thus, B is not a superkey.
- **FD 2 ($D \rightarrow A$):** The left-hand side is D . However, $(D)^+ = \{A, D\} \neq \{A, B, C, D\}$. Thus, D is not a superkey.

Conclusion: Because both $B \rightarrow C$ and $D \rightarrow A$ violate the BCNF condition, the relation R is **not in BCNF**.

3. Conversion to BCNF

To convert the relation into BCNF, we decompose it using the violating functional dependencies.

Step 3.1: First Decomposition

Choose a violating FD, let's select $B \rightarrow C$. We decompose $R(A, B, C, D)$ into two sub-relations:

- R_1 containing the attributes of the violating FD: $R_1(\underline{B}, C)$
- R_{rest} containing the determinant of the violating FD and the remaining attributes: $R_{rest}(A, B, D)$

Check BCNF for R_1 : The only non-trivial FD is $B \rightarrow C$, and B is the candidate key. Thus, R_1 is in BCNF.

Check BCNF for R_{rest} : The relevant FD projected onto R_{rest} is $D \rightarrow A$. The candidate key for R_{rest} is $\{B, D\}$. Since D is not a superkey of R_{rest} , R_{rest} violates BCNF and requires further decomposition.

Step 3.2: Second Decomposition

Now, decompose $R_{rest}(A, B, D)$ using the violating FD $D \rightarrow A$:

- (a) R_2 containing the attributes of the violating FD: $R_2(\underline{D}, A)$
- (b) R_3 containing the determinant and remaining attributes: $R_3(\underline{B}, \underline{D})$

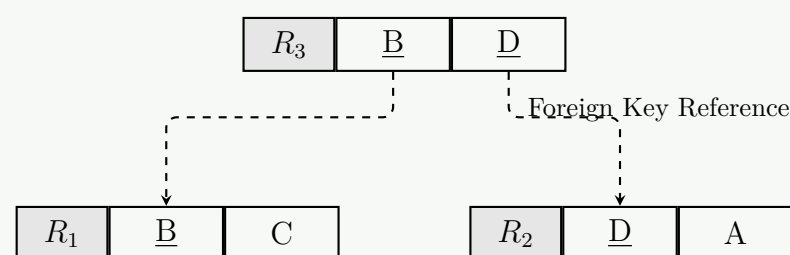
Check BCNF:

- For $R_2(\underline{D}, A)$, the only FD is $D \rightarrow A$. D is the candidate key, so R_2 is in BCNF.
- For $R_3(\underline{B}, \underline{D})$, there are no non-trivial FDs. The candidate key is $\{B, D\}$. It is trivially in BCNF.

Final BCNF Schema

The decomposition is lossless and dependency-preserving. The final relations in BCNF are:

- $R_1(\underline{B}, C)$
- $R_2(\underline{D}, A)$
- $R_3(\underline{B}, \underline{D})$ (This acts as a junction table connecting R_1 and R_2)



Q16. The following schema has been designed for an address book application:

$$R(SSN, Name, PhoneType, PhoneNumber)$$

A person can have different types of phone numbers (e.g. Mobile, Home or Office). The following set F of functional dependencies hold on R :

$$SSN \rightarrow Name$$

$$SSN, PhoneType \rightarrow PhoneNumber$$

$$SSN, PhoneType \rightarrow Name$$

$$PhoneNumber \rightarrow SSN, Name, PhoneType$$

- (a) What are the keys for R ?
- (b) Is R in BCNF? Why or why not?
- (c) Can you remove any dependency without violating equivalency of F ?
- (d) Decompose R into BCNF (if it is not already in BCNF).

(15 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer**1. Keys for R**

To find the keys of the relation $R(SSN, Name, PhoneType, PhoneNumber)$, we compute the attribute closures of potential key sets to see which ones determine all attributes in the schema.

- **Test $\{SSN, PhoneType\}$:**

$$\begin{aligned} \{SSN, PhoneType\}^+ &= \{SSN, PhoneType\} \quad (\text{Reflexivity}) \\ &= \{SSN, PhoneType, PhoneNumber\} \quad (\text{Using } SSN, PhoneType \rightarrow PhoneNumber) \\ &= \{SSN, PhoneType, PhoneNumber, Name\} \quad (\text{Using } SSN \rightarrow Name) \end{aligned}$$

Since it determines all attributes, $\{SSN, PhoneType\}$ is a superkey. No proper subset (SSN^+ or $PhoneType^+$) yields all attributes, so it is a **candidate key**.

- **Test $\{PhoneNumber\}$:**

$$\begin{aligned} \{PhoneNumber\}^+ &= \{PhoneNumber\} \\ &= \{PhoneNumber, SSN, Name, PhoneType\} \quad (\text{Using given FD}) \end{aligned}$$

Since it determines all attributes and consists of a single attribute, $\{PhoneNumber\}$ is also a **candidate key**.

Conclusion: The keys for R are $\{SSN, PhoneType\}$ and $\{PhoneNumber\}$.

2. Is R in BCNF?

Definition: A relation schema is in Boyce-Codd Normal Form (BCNF) if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey.

Let us evaluate the given FDs against the keys we found:

- (a) $SSN, PhoneType \rightarrow PhoneNumber$: LHS is a superkey. (Satisfies BCNF)
- (b) $SSN, PhoneType \rightarrow Name$: LHS is a superkey. (Satisfies BCNF)
- (c) $PhoneNumber \rightarrow SSN, Name, PhoneType$: LHS is a superkey. (Satisfies BCNF)
- (d) $SSN \rightarrow Name$: The LHS is SSN . However, $SSN^+ = \{SSN, Name\}$, which does not contain all attributes. Therefore, SSN is **not** a superkey.

Conclusion: Because the dependency $SSN \rightarrow Name$ exists where the determinant (SSN) is not a superkey, R is **not in BCNF**.

3. Removing Dependencies without Violating Equivalency

We check if any dependency in F is redundant (i.e., can be logically derived from the others using Armstrong's Axioms). Consider the dependency: $SSN, PhoneType \rightarrow Name$.

- We are already given that $SSN \rightarrow Name$.
- According to the **Augmentation Rule**, if $X \rightarrow Y$, then $XZ \rightarrow YZ$.
- Thus, we can augment $SSN \rightarrow Name$ with $PhoneType$ to get $SSN, PhoneType \rightarrow Name, PhoneType$.
- By the **Decomposition Rule**, this implies $SSN, PhoneType \rightarrow Name$.

Conclusion: The dependency $SSN, PhoneType \rightarrow Name$ is completely redundant. We can remove it from F without changing the closure of the set of functional dependencies.

4. Decomposition into BCNF

To convert R into BCNF, we decompose it using the violating functional dependency, $SSN \rightarrow Name$.

Decomposition Strategy: Create two new relations:

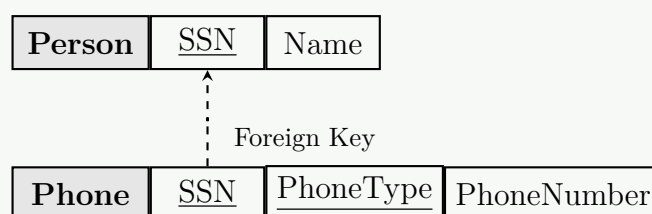
- (a) R_1 containing the attributes of the violating FD.
- (b) R_2 containing the determinant of the violating FD and the remaining attributes from R .

Resulting Relations:

- **Person**(SSN , $Name$)
- **Phone**(SSN , $PhoneType$, $PhoneNumber$) – Note: $\{PhoneNumber\}$ is also an alternate key here.

BCNF Verification:

- In **Person**, the only non-trivial FD is $SSN \rightarrow Name$. Since SSN is the key, it is in BCNF.
- In **Phone**, the FDs are $SSN, PhoneType \rightarrow PhoneNumber$ and $PhoneNumber \rightarrow SSN, PhoneType$. Both determinants are candidate keys. Thus, it is in BCNF.



Q17. Define BCNF, 3NF, and 4NF. Describe relationships among these normal forms. (17 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Definitions of Normal Forms

Let R be a relation schema, F be the set of functional dependencies, and $X \rightarrow Y$ denote a non-trivial functional dependency where $X \subset R$ and $Y \subset R$. Let $X \twoheadrightarrow Y$ denote a non-trivial multivalued dependency.

Third Normal Form (3NF)

A relation R is in **3NF** if it is in 2NF and there are no transitive dependencies of non-prime attributes on the primary key. Formally, R is in 3NF if, for every non-trivial functional dependency $X \rightarrow Y$, at least one of the following conditions holds:

- X is a **superkey**.
- Y is a **prime attribute** (an attribute that is part of some candidate key).

Note: 3NF guarantees a lossless join and dependency-preserving decomposition.

Boyce-Codd Normal Form (BCNF)

A relation R is in **BCNF** if it is a stronger, stricter version of 3NF. Formally, R is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, the following condition holds unconditionally:

- X is a **superkey**.

Note: BCNF removes the “prime attribute” exception present in 3NF. A BCNF decomposition always guarantees a lossless join, but it **cannot** always guarantee dependency preservation.

Fourth Normal Form (4NF)

A relation R is in **4NF** if it is in BCNF and contains no independent multi-valued dependencies (MVDs). Formally, R is in 4NF if, for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, the following condition holds:

- X is a **superkey**.

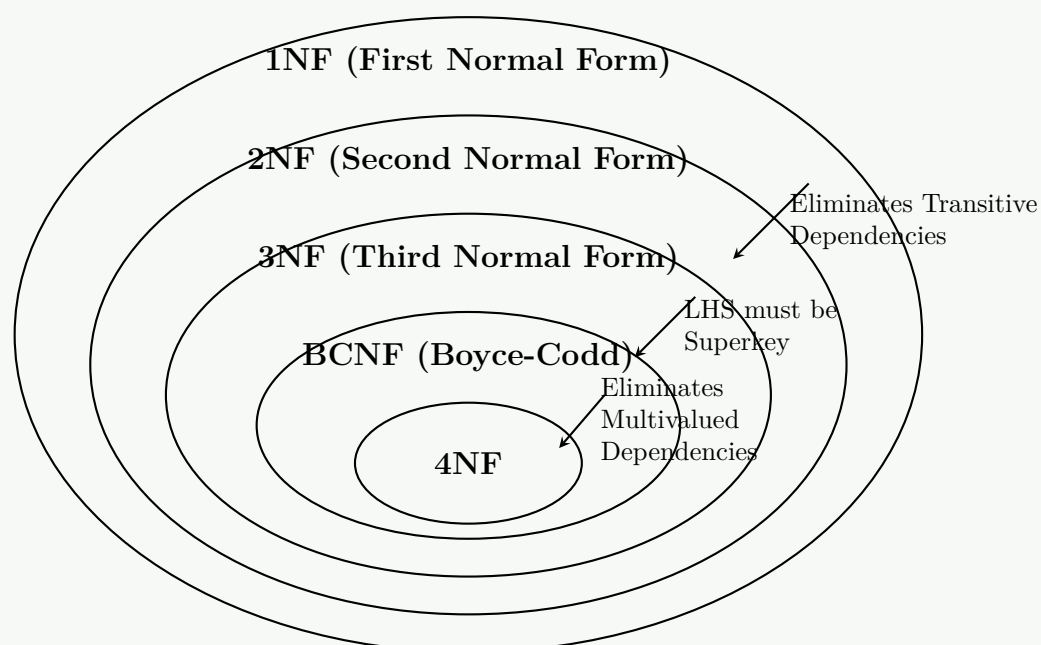
Note: 4NF addresses redundancy caused by independent, one-to-many relationships existing in the same relation, which BCNF and FDs cannot express.

2. Relationships Among the Normal Forms

The normal forms follow a strict hierarchical and nested relationship. Higher normal forms impose stricter constraints to eliminate different types of data anomalies.

- **Strict Inclusion (Hierarchy):** Every table in 4NF is automatically in BCNF. Every table in BCNF is automatically in 3NF. The reverse is **not** true.
- **Anomaly Coverage:**
 - **3NF** eliminates anomalies caused by transitive functional dependencies.
 - **BCNF** goes further by eliminating anomalies caused by overlapping candidate keys (which 3NF allows via the prime-attribute exception).
 - **4NF** goes further by eliminating anomalies caused by independent multivalued dependencies (MVDs), which do not violate BCNF because MVDs are not strictly functional dependencies.
- **Decomposition Trade-offs:**
 - Any schema can be decomposed into **3NF** while preserving both information (lossless join) and functional dependencies.
 - A schema can be decomposed into **BCNF** or **4NF** with a lossless join, but such decompositions may occasionally fail to preserve all functional dependencies.

Venn Diagram of Normal Form Hierarchy



Q18. Describe the BCNF decomposition algorithm. (10 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Definition and Goal

The **Boyce-Codd Normal Form (BCNF) Decomposition Algorithm** is a systematic, step-by-step procedure used to transform a relational schema R that violates BCNF into a collection of smaller schemas $\{R_1, R_2, \dots, R_n\}$ such that:

- Each sub-relation R_i satisfies the strict conditions of BCNF.
- The decomposition is guaranteed to be **lossless-join** (the original data can be perfectly reconstructed via natural joins).

2. The Algorithm Pseudocode

(a) **Initialize:** Set the set of decomposed relations $\mathcal{S} = \{R\}$.

(b) **Loop until convergence:** While there exists a relation schema $R_k \in \mathcal{S}$ that is not in BCNF:

(i) Find a non-trivial functional dependency $X \rightarrow Y$ holding on R_k such that:

- $X \cap Y = \emptyset$
- $X \rightarrow R_k$ is false (i.e., X is **not a superkey** for R_k).

(ii) Decompose R_k into two smaller schemas:

- $R_{k1} = X \cup Y$ (The violating dependency attributes form their own table).
- $R_{k2} = R_k - Y$ (The determined attributes are removed, leaving X behind as a connector).

(iii) Replace R_k in $\mathcal{S}$ with R_{k1} and R_{k2} :

$$\mathcal{S} = (\mathcal{S} - \{R_k\}) \cup \{R_{k1}, R_{k2}\}$$

(c) **Terminate:** Return $\mathcal{S}$ once no more BCNF violations can be found.

3. Core Properties and Design Trade-offs

Property	Status	Explanation / Working Principle
Lossless-Join	Guaranteed	At each step, $R_{k1} \cap R_{k2} = X$, and since $X \rightarrow Y$, X is a superkey of R_{k1} . This fulfills the lossless join condition.
Dependency	NOT	Some original functional dependencies may span across multiple final sub-relations,
Preservation	Guaranteed	making them impossible to enforce locally without a join operation.

4. Step-by-Step Execution Diagram

The working principle of the algorithm can be visualized as a hierarchical tree structure where a violating table is recursively split:

```
graph TD; A["Original Relation R  
Violates BCNF via X → Y"] -- "Extract X → Y" --> B["Relation Rk1  
Attributes: X ∪ Y  
(X is now a Superkey)"]; A -- "Remove Y" --> C["Relation Rk2  
Attributes: R − Y  
(Check for further violations)"]; B --> D["BCNF Satisfied (Stop)"]; C -.-> E["Repeat if needed..."];
```

Anomalies & Normalization

Q1. Suppose we are generating a **StaffPropertyInspection** table to include property inspection by staff members. When the staffs are required to undertake the inspections, they are allocated a company car for use on the day of the inspections. However, a car may be allocated to several staff members on the same day at different time slots. A member of staff may inspect several properties on a given date, but a property is only inspected once by a single staff member on a given date. Normalize the **StaffPropertyInspection** table up to 2NF. Show only the following steps:
Hint: you have to perform and only show the results of the following steps:

- (a) Find all the candidate keys.
- (b) Determine the primary key.
- (c) Find all the functional dependencies.
- (d) Get the first normal form (1NF).
- (e) Get the second normal form (2NF).

StaffPropertyInspection

propertyNo	iDate	iTime	pAddress	comments	staffNo	sName	carReg
PG4	18-Oct-00	10.00	6 Lawrence St, Glasgow	Need to replace crockery	SG37	Ann Beech	M231 JGR
PG4	22-Apr-01	09.00	6 Lawrence St, Glasgow	In good order	SG14	David Ford	M533 HDR
PG4	1-Oct-01	12.00	6 Lawrence St, Glasgow	Damp rot in bathroom	SG14	David Ford	N721 HFR
PG16	22-Apr-01	13.00	5 Novar Dr, Glasgow	Replace living room carpet	SG14	David Ford	M533 HDR
PG16	24-Oct-01	14.00	5 Novar Dr, Glasgow	Good condition	SG37	Ann Beech	N721 HFR

(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Find all the Candidate Keys

To identify the candidate keys, we analyze the given business rules:

- “A member of staff may inspect several properties on a given date, but a property is only inspected once by a single staff member on a given date.”
This implies that for a given **propertyNo** and **iDate**, there is a unique inspection time, a unique staff member, and a unique comment. Thus, {propertyNo, iDate} uniquely identifies all other attributes.
- “A car may be allocated to several staff members on the same day at different time slots.”
This means that a car registration combined with a date and time ({carReg, iDate, iTime}) or a staff member on a specific date and time ({staffNo, iDate, iTime}) can also uniquely pinpoint an inspection. However, looking at the structural constraints, the minimal unique combinations that determine the entire tuple are:

Candidate Keys:

(a) {propertyNo, iDate}

(b) {staffNo, iDate, iTime}

(c) {carReg, iDate, iTime}

2. Determine the Primary Key

The most appropriate and minimal key that directly represents the core entity of an inspection event is chosen as the primary key.

Primary Key: {propertyNo, iDate}

3. Find all the Functional Dependencies

Based on the business rules and the sample data instance, the full set of functional dependencies (*F*) is:

$FD_1 : \{propertyNo, iDate\} \rightarrow iTime, pAddress, comments, staffNo, sName, carReg$

(Primary Key Dependency)

$FD_2 : propertyNo \rightarrow pAddress$

(Partial Dependency on Primary Key)

$FD_3 : staffNo \rightarrow sName$

(Transitive Dependency via staffNo)

$FD_4 : \{staffNo, iDate\} \rightarrow carReg$

(Staff car allocation for the day)

$FD_5 : \{carReg, iDate, iTime\} \rightarrow propertyNo, iTime, pAddress, comments, staffNo, sName$

4. First Normal Form (1NF)

A relation is in **1NF** if all its attributes contain only atomic (indivisible) values, and there are no repeating groups.

The given **StaffPropertyInspection** table is already in **1NF** because every cell contains a single, atomic value, and each row is distinct.

1NF Relation Schema

StaffPropertyInspection(propertyNo, iDate, iTime, pAddress, comments, staffNo, sName, carReg)

—

5. Second Normal Form (2NF)

Definition:

A relation is in **2NF** if it is in 1NF and every non-prime attribute is **fully functionally dependent** on the primary key (i.e., there are no **partial dependencies**, where a non-prime attribute depends on only a part of a composite primary key).

Violation Analysis

Our chosen primary key is {propertyNo, iDate}.

•

From FD_2 (propertyNo $\rightarrow$ pAddress), the attribute **pAddress** depends strictly on **propertyNo** alone, which is a proper subset of the primary key. This is a clear **partial dependency** violation.

Decomposition into 2NF

To eliminate the partial dependency, we separate the violating attribute (pAddress) and its determinant (propertyNo) into their own relation, leaving the remaining attributes to be fully dependent on the complete primary key.

(a) **Property** (Eliminates partial dependency of address on property number):

Property(propertyNo, pAddress)

(b) **PropertyInspection** (Contains the remaining attributes fully dependent on the key):

PropertyInspection(propertyNo, iDate, iTime, comments, staffNo, sName, carReg)

Normalized 2NF Tables Overview

	Property	
	<u>propertyNo</u>	pAddress
	PG4	6 Lawrence St, Glasgow
	PG16	5 Novar Dr, Glasgow

PropertyInspection	<u>propertyNo</u>	<u>iDate</u>	iTime	comments	staffNo	sName	carReg
	PG4	18-Oct-00	10.00	Need to replace crockery	SG37	Ann Beech	M231 JGR
	PG4	22-Apr-01	09.00	In good order	SG14	David Ford	M533 HDR
	PG4	1-Oct-01	12.00	Damp rot in bathroom	SG14	David Ford	N721 HFR
	PG16	22-Apr-01	13.00	Replace living room carpet	SG14	David Ford	M533 HDR
	PG16	24-Oct-01	14.00	Good condition	SG37	Ann Beech	N721 HFR

Q2. Normalize the following booking table up to 3NF by drawing a dependency diagram and identifying all dependencies. Show the resulting 3NF schema.

Date	Time	Customer	Contact	Event	Person	Total	Price
12 Aug, 10	12.30 PM	Maria Indrawan	04222223333	Lunch	Dora Smith	6	60
12 Aug, 10	12.30 PM	John Smith	04113333333	Master Class	Simon Becket	1	120
12 Aug, 10	7.00 PM	Mary Joe	95671290	Dinner	Linda Dee	2	150
13 Aug, 10	12.30 PM	Lindsay Smith	99031001	Lunch	Dora Smith	4	60
13 Aug, 10	12.30 PM	Sunshine Dee	99021900	Lunch	Dora Smith	2	60

(13 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Initial Data Analysis & Functional Dependencies

To normalize the relation, we first determine the **Primary Key (PK)** and establish the **Functional Dependencies (FDs)**.

• **Primary Key Selection:** A customer can book an event on a specific date and time, but two customers can also book the same event concurrently (e.g., Lindsay and Sunshine both booked Lunch on 13 Aug at 12.30 PM). Therefore, the minimal composite key required to uniquely identify each booking record is **{Date, Time, Customer}**.

• **Full Functional Dependency:** The composite primary key uniquely determines the remaining attributes.

• **Partial Dependency:** The attribute **Contact** depends entirely on the **Customer** (which is only a part of the primary key).

• **Transitive Dependency:** The attributes **Person** and **Price** depend entirely on the **Event** attribute, which is not a primary key.

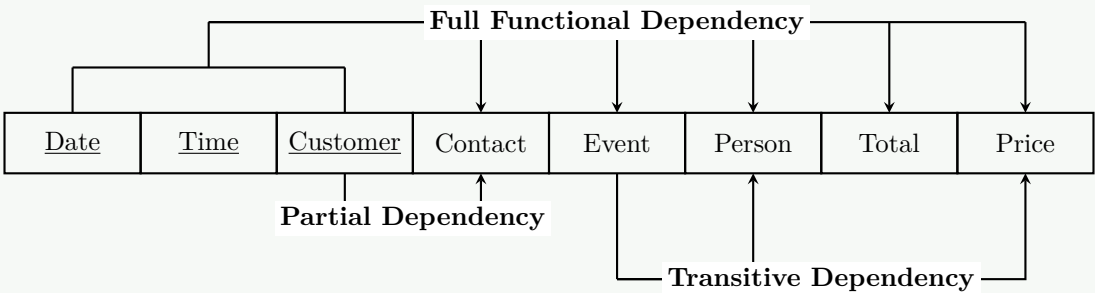
215

Mathematically, the dependencies are:

$$FD_1 : \{Date, Time, Customer\} \rightarrow \{Contact, Event, Person, Total, Price\}$$
$$FD_2 : Customer \rightarrow Contact$$
$$FD_3 : Event \rightarrow \{Person, Price\}$$

2. Unnormalized Form / First Normal Form (1NF) Dependency Diagram

The relation is inherently in **1NF** because all attributes contain atomic (indivisible) values. The following dependency diagram visualizes the dependencies identified above:



3. Second Normal Form (2NF)

A schema is in **2NF** if it is in 1NF and contains no partial dependencies. We resolve FD_2 by creating a separate **Customer** table.

- **Customer**(Customer, Contact)
- **Booking_2NF**(Date, Time, Customer, Event, Person, Total, Price)

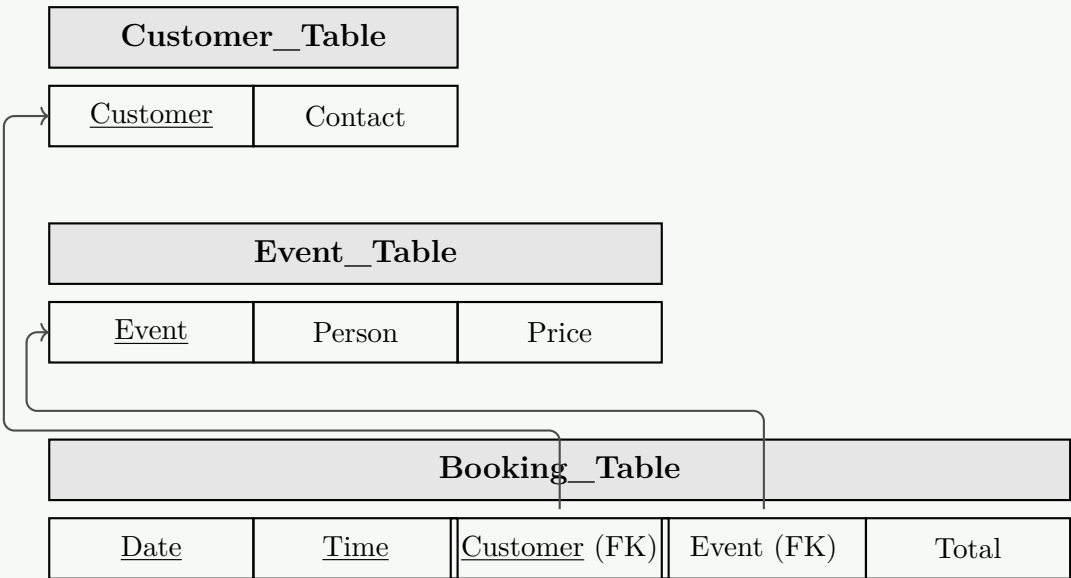
4. Third Normal Form (3NF)

A schema is in **3NF** if it is in 2NF and contains no transitive dependencies. We resolve FD_3 by removing **Person** and **Price** to a separate **Event** table.

- **Customer**(Customer, Contact)
- **Event**(Event, Person, Price)
- **Booking**(Date, Time, Customer, Event, Total)

5. Final 3NF Schema Diagrams

The finalized relational schema clearly exhibits the Primary Keys (PKs) and Foreign Keys (FKs) mapping between relations:



6. Populated 3NF Tables

Below is the structured data mapped cleanly into the final normalized tables:

Customer (PK)	Contact
Maria Indrawan	04222223333
John Smith	04113333333
Mary Joe	95671290
Lindsay Smith	99031001
Sunshine Dee	99021900

Event (PK)	Person	Price
Lunch	Dora Smith	60
Master Class	Simon Becket	120
Dinner	Linda Dee	150

Date (PK)	Time (PK)	Customer (PK/FK)	Event (FK)	Total
12 Aug, 10	12.30 PM	Maria Indrawan	Lunch	6
12 Aug, 10	12.30 PM	John Smith	Master Class	1
12 Aug, 10	7.00 PM	Mary Joe	Dinner	2
13 Aug, 10	12.30 PM	Lindsay Smith	Lunch	4
13 Aug, 10	12.30 PM	Sunshine Dee	Lunch	2

Q3. Normalize the following invoice table up to 3NF by drawing a dependency diagram and identifying all dependencies. Show the resulting 3NF schema.

INV_NUM	PROD_NUM	SALE_DATE	PROD_LABEL	VEND_CODE	VEND_NAME	QTY	PRICE
211347	AA-E3422QW	15-Jan-2010	Rotary Sander	211	NeverFail, Inc	1	\$49.95
211347	QD-300932X	15-Jan-2010	0.25-in drill bit	211	NeverFail, Inc	8	\$3.45
211347	RU-995748G	15-Jan-2010	Band Saw	309	BeGood, Inc.	1	\$39.99
211348	AA-E3422QW	15-Jan-2010	Rotary Sander	211	NeverFail, Inc.	2	\$49.95
211349	GH-7781	16-Jan-2010	Power drill	157	ToughGood	1	\$87.75

(15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Initial Data Analysis & Functional Dependencies

To begin the normalization process, we first identify the **Primary Key (PK)** of the relation and analyze the **Functional Dependencies (FDs)** among the attributes.

- Primary Key Selection:** A single invoice (**INV_NUM**) can contain multiple products. Conversely, a single product (**PROD_NUM**) can appear on many different invoices. Therefore, the minimal composite key required to uniquely identify each row (line item) is **{INV_NUM, PROD_NUM}**.
- Full Functional Dependency:** The composite primary key uniquely determines the quantity (**QTY**) of the product purchased on that specific invoice.
- Partial Dependencies:**
 - SALE_DATE** is determined entirely by **INV_NUM**. (All products on the same invoice share the same sale date).
 - PROD_LABEL**, **PRICE**, and **VEND_CODE** are determined entirely by **PROD_NUM**.
- Transitive Dependency:** The attribute **VEND_NAME** is determined by **VEND_CODE**, which is in turn determined by **PROD_NUM** (a non-key attribute functionally depending on another non-key attribute).

Mathematically, the functional dependencies are:

FD₁ : {INV_NUM, PROD_NUM} → {QTY}

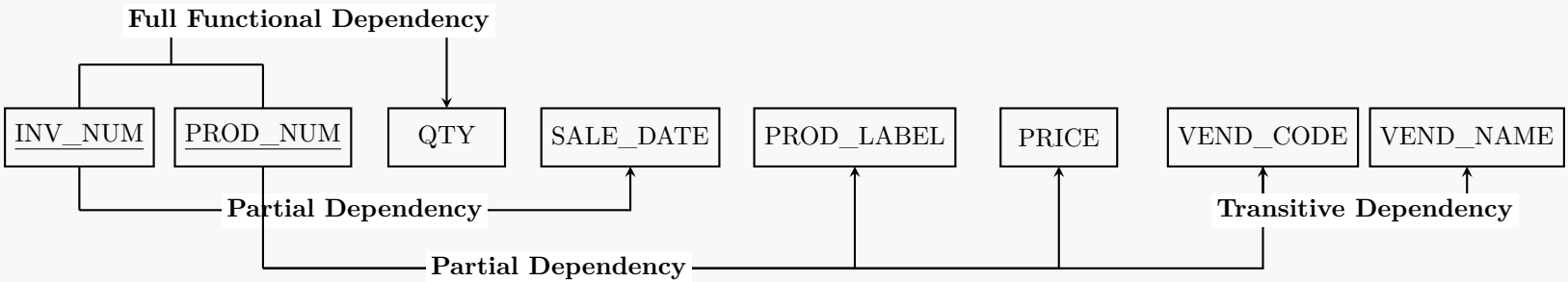
FD₂ : INV_NUM → SALE_DATE

FD₃ : PROD_NUM → {PROD_LABEL, PRICE, VEND_CODE}

FD₄ : VEND_CODE → VEND_NAME

2. First Normal Form (1NF) Dependency Diagram

The relation is in **1NF** because all attributes contain atomic values. The dependency diagram below illustrates the full, partial, and transitive functional dependencies present in the 1NF structure.



3. Second Normal Form (2NF)

A schema is in **2NF** if it is in 1NF and contains **no partial dependencies**. We resolve FD₂ and FD₃ by splitting the attributes into distinct tables.

- **INVOICE**(INV_NUM, SALE_DATE)
- **PRODUCT_2NF**(PROD_NUM, PROD_LABEL, PRICE, VEND_CODE, VEND_NAME)
- **LINE_ITEM**(INV_NUM, PROD_NUM, QTY)

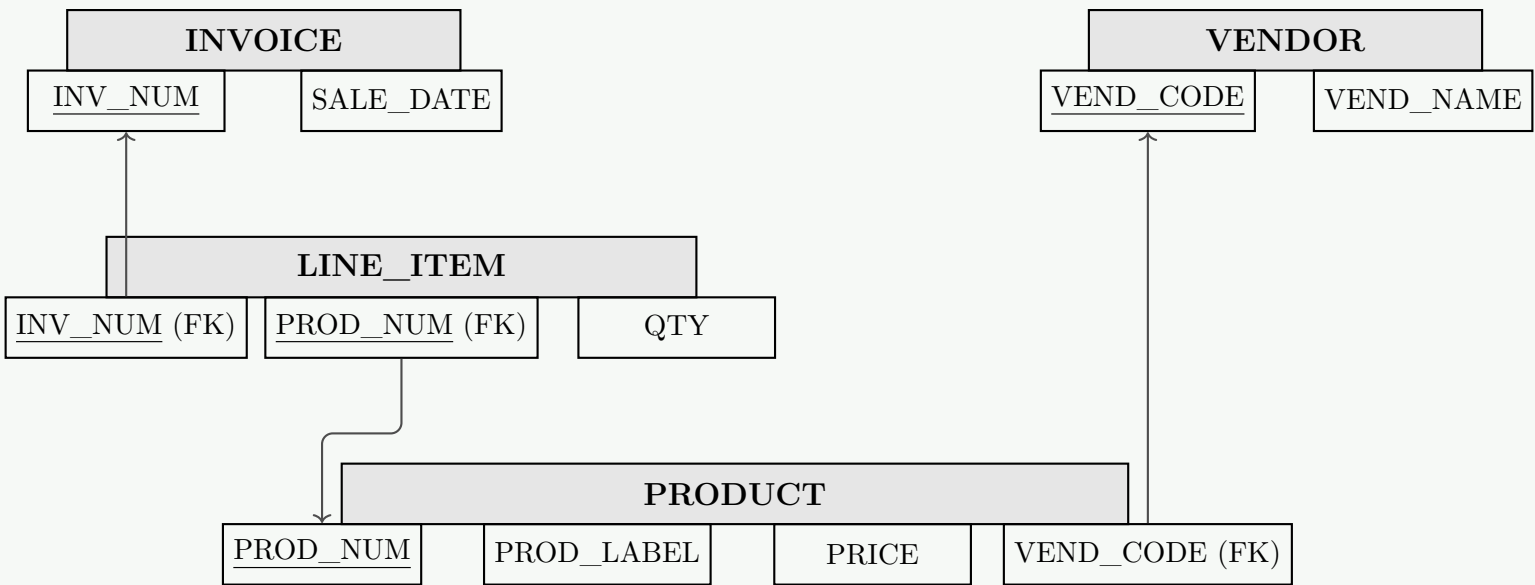
4. Third Normal Form (3NF)

A schema is in **3NF** if it is in 2NF and contains **no transitive dependencies**. We resolve FD₄ by extracting the vendor details into a separate table, linking it via a Foreign Key.

- **INVOICE**(INV_NUM, SALE_DATE)
- **VENDOR**(VEND_CODE, VEND_NAME)
- **PRODUCT**(PROD_NUM, PROD_LABEL, PRICE, VEND_CODE)
- **LINE_ITEM**(INV_NUM, PROD_NUM, QTY)

5. Final 3NF Schema Diagram

The schema diagram below demonstrates the normalized architecture, specifically highlighting Primary Keys (PK) and how Foreign Keys (FK) establish relationships between the relations.



6. Populated 3NF Tables

Breaking down the original 1NF dataset populates the normalized relations as follows, eliminating data anomalies and minimizing redundancy:

INVOICE		LINE_ITEM		
INV_NUM (PK)	SALE_DATE	INV_NUM (PK/FK)	PROD_NUM (PK/FK)	QTY
211347	15-Jan-2010	211347	AA-E3422QW	1
211348	15-Jan-2010	211347	QD-300932X	8
211349	16-Jan-2010	211347	RU-995748G	1
		211348	AA-E3422QW	2
		211349	GH-7781	1

VENDOR	
VEND_CODE (PK)	VEND_NAME
211	NeverFail, Inc.
309	BeGood, Inc.
157	ToughGood

PRODUCT			
PROD_NUM (PK)	PROD_LABEL	PRICE	VEND_CODE (FK)
AA-E3422QW	Rotary Sander	\$49.95	211
QD-300932X	0.25-in drill bit	\$3.45	211
RU-995748G	Band Saw	\$39.99	309
GH-7781	Power drill	\$87.75	157

Q4. What are the three anomalies in relational database schemas? Discuss them. (10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

↔ Similar: 2013–14: same question (10 marks)

Answer

Introduction to Data Anomalies

In Relational Database Management Systems (RDBMS), **data anomalies** are undesirable irregularities, inconsistencies, or errors that occur when manipulating data in poorly planned or unnormalized tables. They arise primarily due to **data redundancy** and the grouping of unrelated data entities into a single table.

There are three primary types of data anomalies: **Insertion Anomaly**, **Deletion Anomaly**, and **Update Anomaly**.

To illustrate these anomalies, consider the following unnormalized relation, **Course_Registration**, where the Primary Key is the composite of {**StudentID**, **CourseID**}:

StudentID	StudentName	CourseID	CourseName	CourseFee
101	Alice Smith	CS101	Database Systems	\$500
102	Bob Jones	CS101	Database Systems	\$500
103	Charlie Doe	MA201	Calculus	\$400

1. Insertion Anomaly

- Definition:** An insertion anomaly occurs when certain attributes or facts cannot be inserted into the database without simultaneously inserting information about a completely different entity.
- Explanation:** This typically happens when a record requires a composite primary key, but data for one part of the key is not yet available, violating the *Entity Integrity Rule* (primary keys cannot be NULL).
- Example in Schema:** Suppose the university creates a new course, **PH301 (Physics)**, with a fee of \$600. We cannot add this course to the **Course_Registration** table until at least one student registers for it, because **StudentID** is a mandatory part of the primary key.

2. Deletion Anomaly

- Definition:** A deletion anomaly occurs when the deletion of a record to eliminate specific facts inadvertently causes the unintended loss of other, unrelated facts.
- Explanation:** This happens when attributes belonging to different conceptual entities are stored in the same row. Deleting the row removes all facts contained within it.
- Example in Schema:** If the student **Charlie Doe (103)** drops out or cancels his registration, we must delete his row. However, deleting Charlie’s record completely removes all information about the **MA201 (Calculus)** course and its \$400 fee from the database.

3. Update (Modification) Anomaly

- Definition:** An update anomaly occurs when a data value that represents a single logical fact is stored redundantly in multiple rows, and updating this value requires modifying all those rows simultaneously.

- **Explanation:** If the redundant data is not updated comprehensively across every instance, the database falls into an inconsistent state where a single entity possesses conflicting values.
- **Example in Schema:** If the fee for **CS101 (Database Systems)** is increased from \$500 to \$550, the DBMS must locate and update multiple rows (for both Alice and Bob). If a system failure occurs after updating Alice’s row but before updating Bob’s row, the database is left in a contradictory state regarding the actual fee of CS101.

Resolution

Data anomalies are resolved through the mathematical process of **Normalization** (typically up to the Third Normal Form (3NF) or Boyce-Codd Normal Form (BCNF)). Normalization decomposes the flawed table into smaller, well-structured relations. For the example above, the anomalies are eliminated by decomposing the table into three distinct relations:

- (a) **Student**(StudentID, StudentName)
- (b) **Course**(CourseID, CourseName, CourseFee)
- (c) **Enrollment**(StudentID, CourseID) – *junction table linking Student and Course*

Q5. What are redundancy, deletion and update anomalies? (10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

Introduction to Schema Irregularities

In Relational Database Management Systems (RDBMS), poorly designed schemas often group attributes of distinct logical entities into a single relation. This architectural flaw leads to **data redundancy**, which is the root cause of **data anomalies**. To clearly illustrate these concepts, consider the following unnormalized **Employee_Project** relation where the composite Primary Key is {**EmpID**, **ProjectID**}:

EmpID	EmpName	ProjectID	ProjectName	ProjectBudget
E01	Alice Smith	P100	Cloud Migration	\$50,000
E02	Bob Jones	P100	Cloud Migration	\$50,000
E03	Carol White	P200	Security Audit	\$30,000

1. Data Redundancy

- **Definition:** Data redundancy is the unnecessary repetition or duplication of data within a database relation.
- **Explanation:** When multiple conceptual entities (like Employees and Projects) are forced into a single table without proper normalization, facts about one entity are repeated for every instance of the other entity it associates with.
- **Example in Schema:** In the table above, the facts that Project **P100** is named **Cloud Migration** and has a budget of **\$50,000** are recorded redundantly for both Alice and Bob.
- **Consequences:** Redundancy wastes storage space, degrades database performance during modification operations, and critically, creates the environment for update and deletion anomalies.

2. Update (Modification) Anomaly

- **Definition:** An update anomaly occurs when a change to a single logical fact requires modifying multiple rows, and a failure to successfully update all instances leaves the database in an inconsistent state.
- **Working Principle:** Because of data redundancy, a single piece of information is stored in multiple places. If a modification is applied partially (e.g., due to a system crash midway through an update transaction), the same entity will possess conflicting data values.
- **Example in Schema:** If the budget for the **Cloud Migration** project is increased to **\$60,000**, the DBMS must perform a multi-row update targeting both Alice’s row and Bob’s row. If only Alice’s row is updated, the database enters a contradictory state regarding the true budget of Project P100.

3. Deletion Anomaly

- **Definition:** A deletion anomaly occurs when deleting a row to remove facts about one specific entity unintentionally results in the permanent loss of facts about a completely different, independent entity.
- **Working Principle:** This occurs because attributes of independent conceptual entities are intrinsically linked within the same tuple. Deleting the tuple to erase one entity effectively wipes out the attributes of the other entity bundled with it.
- **Example in Schema:** If Carol White (E03) resigns from the company and her employee record is deleted, the database entirely loses the information that Project **P200** exists, is named **Security Audit**, and has a budget of **\$30,000**.

Resolution

Note: Redundancy and its associated anomalies are systematically resolved through the mathematical process of **Normalization**. By decomposing the flawed relation into smaller, functionally cohesive tables based on Functional Dependencies (FDs), redundancy is minimized.

For the example above, the anomalies are eliminated by decomposing the table into three BCNF/3NF relations:

- (a) **Employee**(EmpID, EmpName)
- (b) **Project**(ProjectID, ProjectName, ProjectBudget)
- (c) **Assignment**(EmpID, ProjectID) – *junction table*

BUET · CSE 215 · Database

Part IV

Storage, Indexing & Query Processing

Storage, Indexing & Query Processing

S5 — Storage and Indexing

Data Storage Architecture & RAID

Q1. Suppose we are using a RAID level 4 scheme with four data disks and one redundant disk, where each block is one byte. Calculate the block of the redundant disk if the corresponding blocks of data disks are: 01010110, 11000000, 00111011, and 11111011. **(10 marks)** [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Working Principle of RAID 4 Parity

In a **RAID 4** (Redundant Array of Independent Disks, Level 4) architecture, data is striped across multiple disks at the block level, while one dedicated disk is reserved exclusively for storing **parity information**. This parity block is used for error recovery; if any single data disk fails, its contents can be perfectly reconstructed by computing the bitwise XOR ($\oplus$) of the surviving data blocks and the parity block. The parity block (P) is generated by performing a logical bitwise **Exclusive-OR (XOR)** operation across the corresponding bits of all data blocks ($D_1, D_2, \dots, D_n$):

$$P = D_1 \oplus D_2 \oplus D_3 \oplus \dots \oplus D_n \tag{4}$$

The XOR operation follows these fundamental rules:

- **Even number of 1s** yields a **0**.
- **Odd number of 1s** yields a **1**.

2. Step-by-Step Calculation

Given the following four 1-byte data blocks from the corresponding stripes of the four data disks:

- Disk 1 (D_1): 01010110
- Disk 2 (D_2): 11000000
- Disk 3 (D_3): 00111011
- Disk 4 (D_4): 11111011

We align the bytes vertically and calculate the XOR sum for each bit column:

$D_1 :$	0	1	0	1	0	1	1	0	
$D_2 :$	1	1	0	0	0	0	0	0	
$D_3 :$	0	0	1	1	1	0	1	1	
$\oplus D_4 :$	1	1	1	1	1	0	1	1	
Parity (P) :	0	1	0	1	0	1	1	0	$\leftarrow$ Calculated Redundant Block

Explanation of the Column Calculation:

- **Bit 7 (MSB):** $0 \oplus 1 \oplus 0 \oplus 1 = 0$ (*Two 1s $\rightarrow$ Even $\rightarrow$ 0*)
- **Bit 6:** $1 \oplus 1 \oplus 0 \oplus 1 = 1$ (*Three 1s $\rightarrow$ Odd $\rightarrow$ 1*)
- **Bit 5:** $0 \oplus 0 \oplus 1 \oplus 1 = 0$ (*Two 1s $\rightarrow$ Even $\rightarrow$ 0*)
- **Bit 4:** $1 \oplus 0 \oplus 1 \oplus 1 = 1$ (*Three 1s $\rightarrow$ Odd $\rightarrow$ 1*)
- **Bit 3:** $0 \oplus 0 \oplus 1 \oplus 1 = 0$ (*Two 1s $\rightarrow$ Even $\rightarrow$ 0*)
- **Bit 2:** $1 \oplus 0 \oplus 0 \oplus 0 = 1$ (*One 1 $\rightarrow$ Odd $\rightarrow$ 1*)
- **Bit 1:** $1 \oplus 0 \oplus 1 \oplus 1 = 1$ (*Three 1s $\rightarrow$ Odd $\rightarrow$ 1*)
- **Bit 0 (LSB):** $0 \oplus 0 \oplus 1 \oplus 1 = 0$ (*Two 1s $\rightarrow$ Even $\rightarrow$ 0*)

Final Result: The corresponding block on the redundant parity disk is **01010110**.

3. RAID 4 Block Architecture Diagram

The following diagram illustrates the block-level striping and the relationship between the data disks and the dedicated parity disk for the given byte sequence.

Data Disk 1	Data Disk 2	Data Disk 3	Data Disk 4	Parity Disk
Block 0	Block 1	Block 2	Block 3	Parity 0-3
01010110	⊕ 11000000	⊕ 00111011	⊕ 11111011	= 01010110
Block 8	Block 9	Block 10	Block 11	Parity 8-11

Q2. What is the purpose of using multiple clustering file organization? Give an example of its usage. Mention the disadvantages of this organization. **(10 marks)** [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Purpose of Multitable Clustering File Organization

Multitable Clustering File Organization (often referred to simply as cluster file organization when involving multiple relations) is a storage architecture where records from two or more related tables are stored physically together in the same disk block. The primary **purpose** of this organization is to optimize the performance of frequent **join operations** (e.g., $R \bowtie S$). By storing related records adjacently based on a common join attribute (the *cluster key*), the Database Management System (DBMS) can retrieve a parent record and all of its associated child records in a single disk I/O operation. This drastically reduces the random disk accesses that would otherwise be required if the relations were stored in separate files.

2. Example of Usage

Consider a University database with two relations:

- Department** (Dept_ID, DName)
- Employee** (Emp_ID, Name, Dept_ID)

A frequent query in this system is to list a department along with all its employees:

$$\Pi_{DName, Name}(\text{Department} \bowtie_{\text{Dept_ID}} \text{Employee})$$

In a standard file organization, fetching the "IT" department requires one block read, and fetching its employees might require jumping to several different blocks scattered across the disk. Using multitable clustering on the **Dept_ID** attribute, the **Department** record for "IT" is immediately followed by the **Employee** records for "Alice" and "Bob" (who work in IT) within the exact same physical block.

Physical Disk Block (Block i)

Department: Dept_ID = 10 | DName = 'IT'

Employee: Emp_ID = 101 | Name = 'Alice' | Dept_ID = 10

Employee: Emp_ID = 102 | Name = 'Bob' | Dept_ID = 10

Department: Dept_ID = 20 | DName = 'HR'

Employee: Emp_ID = 103 | Name = 'Carol' | Dept_ID = 20

Employee: Emp_ID = 104 | Name = 'Dave' | Dept_ID = 20

Employee: Emp_ID = 105 | Name = 'Eve' | Dept_ID = 20

3. Disadvantages of Multitable Clustering

While highly efficient for specific joins, this organization introduces several significant drawbacks:

- (a) **Inefficient Single-Table Scans:** If a query requires scanning only the **Employee** table (e.g., finding all employees with a specific salary), the performance degrades severely. The system must read past the interleaved **Department** records, and the **Employee** records are now spread across a larger number of blocks, increasing total I/O.
- (b) **Variable Cluster Sizes and Block Overflow:** The number of child records per parent record is rarely uniform. If a department has 500 employees, the cluster will exceed the size of a single disk block. This forces the DBMS to use overflow blocks and pointer chains, nullifying the contiguous read advantage.
- (c) **Poor Performance on Non-Cluster Keys:** Queries that filter or join on attributes other than the clustering key do not benefit from this organization and suffer from the scattered layout of the base relations.
- (d) **Maintenance and Insertion Overhead:** Inserting a new **Employee** into a fully packed cluster block is complex. The DBMS must physically relocate records or create overflow chains, leading to data fragmentation and high maintenance costs during heavy **INSERT**, **UPDATE**, or **DELETE** operations.

4. Comparison Summary

Feature	Standard File Organization	Multitable Clustering
Join Performance	High I/O (random accesses)	Low I/O (sequential access)
Single-Table Scan	Very Fast (contiguous data)	Slow (interleaved data)
Data Fragmentation	Low (records append sequentially)	High (due to variable cluster sizes)
Ideal Use Case	Independent table queries	Heavy parent-child join queries

Q3. The following table stores information about courses offered by BUET:

```
CREATE TABLE course (  
  course_code    CHAR(10) PRIMARY KEY,  
  course_title   VARCHAR(255) NOT NULL,  
  credit_hours   NUMERIC NOT NULL,  
  old_course_code CHAR(10) REFERENCES course(course_code)  
);
```

Here VARCHAR is variable-length, all other data types are fixed length, size of NUMERIC is 8 bytes, and size of CHAR(n) is *n* bytes. Draw the structure of the records, and the structure of the page when the records of this table are stored using the Slotted Page Structure. (10 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Record Structure Analysis

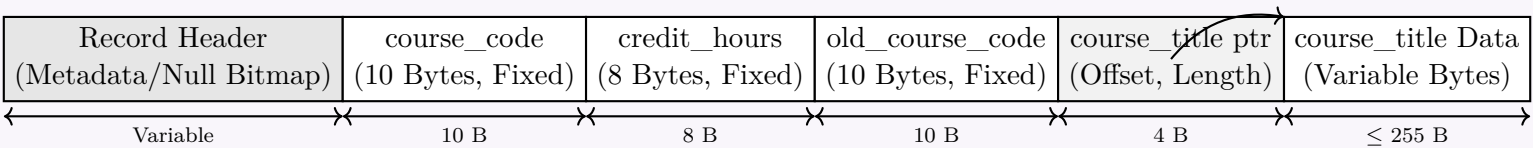
The table contains both fixed-length and variable-length attributes. To handle variable-length attributes (VARCHAR) efficiently within a record, the standard textbook design uses a record header followed by fixed-length data fields, and then pointers (offset, length pairs) that point to the variable-length data stored at the end of the record.

Attribute Types and Sizes

- **course_code:** CHAR(10) → Fixed-length (10 bytes)
- **course_title:** VARCHAR(255) → Variable-length (Stored at the end; referenced via a 4-byte pointer/offset-length pair in the fixed part)
- **credit_hours:** NUMERIC → Fixed-length (8 bytes)
- **old_course_code:** CHAR(10) → Fixed-length (10 bytes)

Record Layout Diagram

The record consists of a **Record Header** (containing metadata such as null-bitmap, attribute count, etc.), followed by the fixed-length parts and references, and finally the variable-length string values.



2. Slotted Page Structure

The **Slotted Page Structure** is used to organize variable-length records within a page.

- **Page Header:** Located at the beginning of the page, containing the number of record entries, the end of free space pointer, and an array of slots.
- **Slots (Slot Directory):** Each slot contains an (**Offset**, **Size**) pair pointing to the absolute location of the corresponding record on the page. The slot directory grows **downward** from the top.
- **Records:** Extracted and written sequentially starting from the bottom of the page growing **upward**.
- **Free Space:** Resides in the middle, dynamically shrinking as new slots and records are added.

3. Core Properties of this Setup

- **Indirection Through Slots:** Pointers outside this disk page point to the specific slot entry (e.g., `PageID:SlotNumber`), not the physical byte offset of the record. This allows records to be fragmented, compacted, or moved within the page without invalidating external references.
- **Deletion Handling:** When a record is deleted, its slot entry offset is set to -1 (or nullified), and the space occupied by the record is marked as free. Defragmentation (compaction) happens dynamically when the free space becomes highly fragmented.

Q4. A relational database table is stored in a database file using fixed-length records and a free list.

(a) Show the structure of the file after deleting Record 2 (Mohammad Naim) and inserting a new row (Tamim Iqbal, Batter, 34).

(b) Discuss how the table would be stored in a slotted page structure with a high-level block diagram showing only the records.

Header	Player Name	Role	Age	Free List
Record 0	Shakib Al Hasan	Allrounder	36	-
Record 1				←
Record 2	Mohammad Naim	Batter	24	←
Record 3	Taskin Ahmed	Bowler	28	←
Record 4				←
Record 5	Mushfiqur Rahim	Batter	36	←
Record 6				←
Record 7	Litton Das	Batter	28	
Record 8	Mustafizur Rahman	Bowler	28	
Figure				

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

1. File Structure After Deletion and Insertion

In a fixed-length record organization with a free list, deleted records are typically managed using a **LIFO (Last-In, First-Out) stack mechanism** to minimize pointer traversal overhead.

- Deletion of Record 2:** When Mohammad Naim is deleted, Record 2 is pushed to the head of the free list.
Free List becomes: Header → Record 2 → Record 1 → Record 4 → Record 6.
- Insertion of New Row:** When Tamim Iqbal is inserted, the DBMS pops the first available slot from the head of the free list, which is Record 2.
Free List reverts to: Header → Record 1 → Record 4 → Record 6.

The final structure is identical to the initial one regarding the free list pointers, but Record 2 now holds the new data.

Header	Player Name	Role	Age	Free List
Record 0	Shakib Al Hasan	Allrounder	36	-
Record 1				
Record 2	Tamim Iqbal	Batter	34	
Record 3	Taskin Ahmed	Bowler	28	
Record 4				
Record 5	Mushfiquur Rahim	Batter	36	
Record 6				
Record 7	Litton Das	Batter	28	
Record 8	Mustafizur Rahman	Bowler	28	

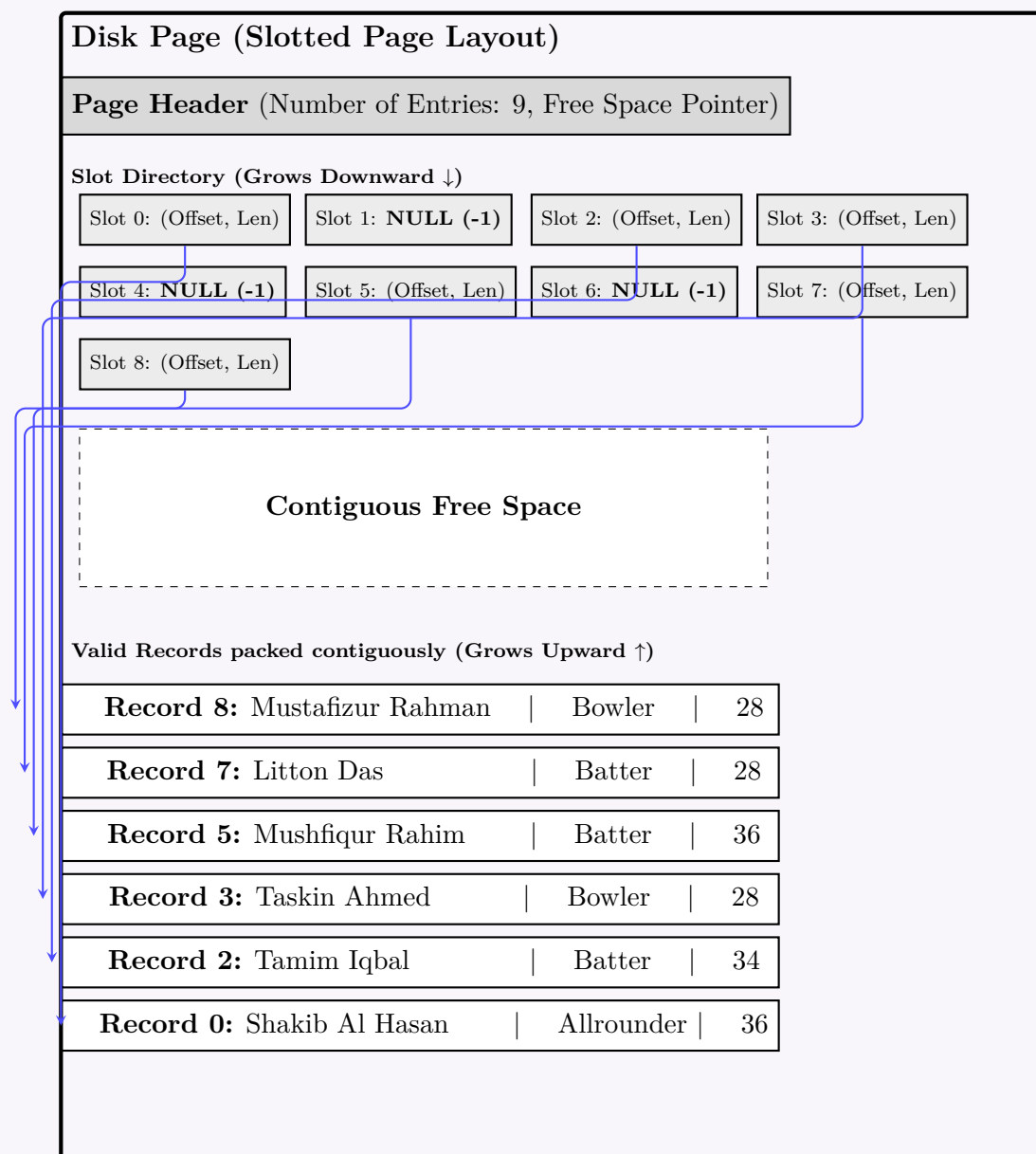
2. Storage in a Slotted Page Structure

If this table were stored using a **Slotted Page Structure**, the physical layout would decouple the logical record IDs from their physical placement. This is particularly advantageous for preventing internal fragmentation caused by empty records (like 1, 4, and 6).

Working Principle for this Table:

- Slot Directory:** The top of the page maintains a directory of slots. Each slot corresponds to a logical Record ID and contains an (Offset, Length) pointer.
- Empty Records:** Slots 1, 4, and 6 will have their offsets set to -1 (or NULL), indicating they are empty. No physical space is wasted at the bottom of the page for these deleted/empty records.
- Dense Packing:** The actual valid records (0, 2, 3, 5, 7, 8) are stored contiguously at the bottom of the page, growing upward.
- Free Space:** A contiguous block of free space remains in the middle, allowing efficient insertions without needing a linked free list.

227



Q5. Briefly describe the differences between RAID-5 and RAID-6. (5 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Overview and Core Differences

RAID (Redundant Array of Independent Disks) levels 5 and 6 both utilize **block-level striping with distributed parity** to provide a balance of performance, storage capacity, and fault tolerance. The fundamental distinction between the two lies in the **amount of parity data** calculated and stored per stripe, which directly dictates their fault tolerance and hardware requirements.

- **RAID-5 (Single Distributed Parity):** Calculates and writes one parity block per data stripe. The parity blocks are distributed across all drives in the array in a rotating manner (to avoid a single dedicated parity disk bottleneck).
- **RAID-6 (Double Distributed Parity):** Calculates and writes *two* independent parity blocks (typically using a Reed-Solomon polynomial for the second parity, often denoted as P and Q) per data stripe. These are also distributed across all drives.

2. Key Distinctions

- (a) **Fault Tolerance:** RAID-5 can survive the failure of **exactly one** drive without data loss. If a second drive fails during the rebuild process (which can be lengthy for large drives), all data is lost. RAID-6 can survive the simultaneous failure of **two** drives. This makes RAID-6 significantly safer for large arrays where Unrecoverable Read Errors (UREs) during a rebuild are a statistical risk.
- (b) **Minimum Disk Requirement:** Because RAID-5 uses one parity block per stripe, it requires a minimum of **3 disks**. RAID-6 requires two parity blocks per stripe, necessitating a minimum of **4 disks**.
- (c) **Storage Efficiency (Capacity):** For an array of N disks, each of capacity C :
 - RAID-5 Usable Capacity = $(N - 1) \times C$
 - RAID-6 Usable Capacity = $(N - 2) \times C$
- (d) **Write Penalty:** Every write operation in RAID requires reading the old data and parity, calculating the new parity, and writing the new data and parity. RAID-5 incurs a write penalty of **4 I/O operations** (2 reads, 2 writes). RAID-6 must calculate and write two parities, incurring a write penalty of **6 I/O operations** (3 reads, 3 writes), making random writes noticeably slower than RAID-5.

3. Disk Layout Visualization

RAID-5 (Single Parity)
Min: 3 Disks (4 Shown)

Disk 0	Disk 1	Disk 2	Disk 3
A1	A2	A3	P(A)
B1	B2	P(B)	B3
C1	P(C)	C2	C3
P(D)	D1	D2	D3

RAID-6 (Double Parity)
Min: 4 Disks (5 Shown)

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
A1	A2	A3	P(A)	Q(A)
B1	B2	P(B)	Q(B)	B3
C1	P(C)	Q(C)	C2	C3
P(D)	Q(D)	D1	D2	D3

4. Comparison Summary

Feature	RAID-5	RAID-6
Parity Method	Single Distributed	Double Distributed (P+Q)
Minimum Drives	3	4
Drive Failures Tolerated	1	2
Usable Capacity	$(N - 1) \times \text{Capacity}$	$(N - 2) \times \text{Capacity}$
Write Penalty	Medium (4 I/O operations)	High (6 I/O operations)
Primary Use Case	General purpose file storage, cost-effective storage	Archival, large capacity drives where rebuilds take days

Q6. A Megatron 800 Disk has the following pending disk requests (listed by track number). If the head is currently at track 71 and moving towards the higher track number, calculate average disk seek times for both the SCAN and C-SCAN algorithms.

Disk Requests: 21, 98, 12, 80, 85, 10

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Problem Analysis and Assumptions

To calculate the disk seek times using track numbers, we determine the **Seek Distance** (measured in tracks/cylinders). Since specific hardware timing parameters (e.g., seek overhead, track-to-track traverse time) are not provided, the total and average seek distances serve as the proportional metric for seek time.

- **Pending Requests (Sorted):** 10, 12, 21, 80, 85, 98
- **Total Number of Requests (N):** 6
- **Initial Head Position:** Track 71
- **Direction:** Moving towards higher track numbers ($\uparrow$)
- **Assumption on Disk Size:** Since the maximum requested track is 98, we assume a standard 100-track disk boundary with tracks numbered from 0 to 99 ($T_{min} = 0, T_{max} = 99$). *Note: If the algorithm does not go to the disk extremities, it is referred to as LOOK/C-LOOK. Strict SCAN/C-SCAN requires hitting the boundaries.*

2. SCAN Algorithm (Elevator Algorithm)

Working Principle: The disk arm starts at track 71, moves towards the highest track (99), servicing requests along the way. Upon reaching the end (T_{max}), it reverses direction and services the remaining requests while moving towards the lowest track.

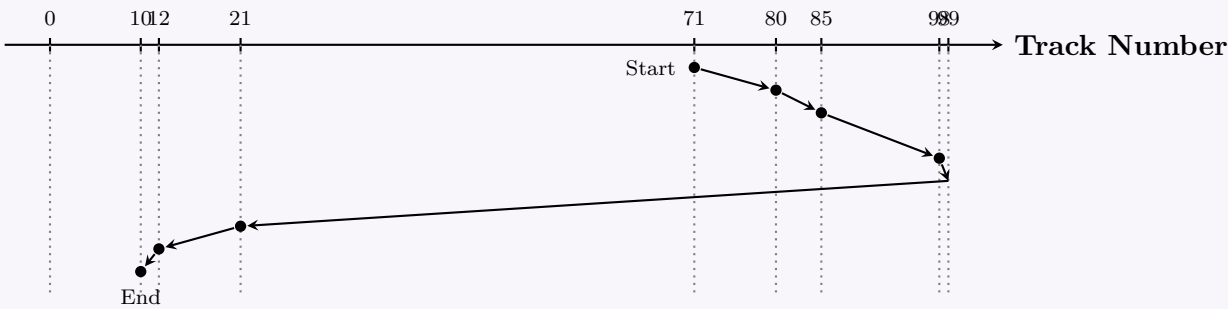
- **Path:** 71 $\rightarrow$ 80 $\rightarrow$ 85 $\rightarrow$ 98 $\rightarrow$ 99 (End of Disk) $\rightarrow$ 21 $\rightarrow$ 12 $\rightarrow$ 10

Calculation

Total Seek Distance = $|80 - 71| + |85 - 80| + |98 - 85| + |99 - 98|$
 $+ |21 - 99| + |12 - 21| + |10 - 12|$
 $= 9 + 5 + 13 + 1 + 78 + 9 + 2$
 $= \mathbf{117}$ tracks

Alternatively, calculating by turning points: $(99 - 71) + (99 - 10) = 28 + 89 = 117$.

Average Seek Distance = $\frac{\text{Total Seek Distance}}{\text{Number of Requests}} = \frac{117}{6} = \mathbf{19.5}$ tracks/request



3. C-SCAN Algorithm (Circular SCAN)

Working Principle: The disk arm starts at track 71, moves towards the highest track (99), servicing requests. Upon reaching the end, it makes a rapid return jump to the lowest track (0) *without servicing any requests on the way down*. It then resumes servicing requests in the upward direction.

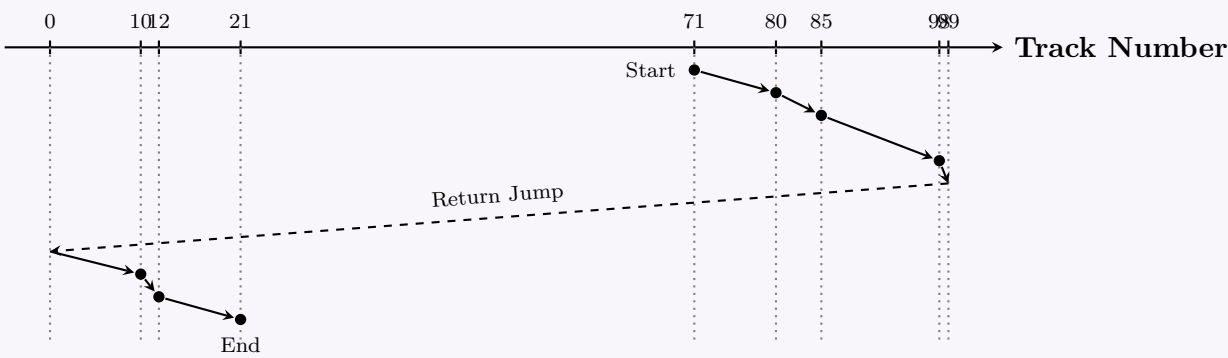
- **Path:** 71 → 80 → 85 → 98 → **99 (End)** → **0 (Jump)** → 10 → 12 → 21

Calculation

Total Seek Distance = $(99 - 71) + (99 - 0) + (21 - 0)$
 $= 28$ (Sweep up) + 99 (Return jump) + 21 (Sweep up)
 $= \mathbf{148}$ tracks

Note: In some textbook variants, the return jump (99) is ignored because the mechanical retraction is faster than a read-seek, but standard metric includes it as full displacement.

Average Seek Distance = $\frac{\text{Total Seek Distance}}{\text{Number of Requests}} = \frac{148}{6} \approx \mathbf{24.67}$ tracks/request



4. Comparison Summary

Algorithm	Total Distance	Average Seek Time (Distance)
SCAN	117 tracks	19.50 tracks/request
C-SCAN (incl. jump)	148 tracks	24.67 tracks/request

Q7. A table student has been created as follows:

```
CREATE TABLE STUDENT (  
    ID          VARCHAR2(10) ,  
    NAME        VARCHAR2(50) ,  
    CGPA        NUMBER(8) ,  
    ADDRESS     VARCHAR2(932) ,  
    DESCRIPTION VARCHAR2(500)  
);
```

There are three blocks 1, 2 and 3 and the size of each block is 4000 bytes. Show the storage of 5 sample records of the student relation in these blocks using fixed-length record structure. No record can overlap the block boundary. **(10 marks)** [CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Record Size Calculation

In a **fixed-length record structure**, the system allocates the maximum defined size for every attribute, regardless of the actual data length inserted. We first determine the fixed size of a single **STUDENT** record (R).

- ID:** VARCHAR2(10) → 10 bytes
- NAME:** VARCHAR2(50) → 50 bytes
- CGPA:** NUMBER(8) → 8 bytes
- ADDRESS:** VARCHAR2(932) → 932 bytes
- DESCRIPTION:** VARCHAR2(500) → 500 bytes

$$R = 10 + 50 + 8 + 932 + 500 = \mathbf{1500 \text{ bytes}}$$

2. Block Capacity and Blocking Factor

- Block Size (B):** 4000 bytes
- Blocking Factor (bfr):** The number of complete records that can fit into a single block. Since records **cannot overlap** the block boundary (unspanned storage layout), we take the floor of the division:
$$bfr = \left\lfloor \frac{B}{R} \right\rfloor = \left\lfloor \frac{4000}{1500} \right\rfloor = \mathbf{2 \text{ records per block}}$$
- Wasted Space (Internal Fragmentation):** $4000 - (2 \times 1500) = \mathbf{1000 \text{ bytes per block}}$.

3. Storage Distribution for 5 Sample Records

To store 5 records (R_1, R_2, R_3, R_4, R_5), the DBMS allocates them into the blocks sequentially:

- Block 1:** Contains R_1 and R_2 (Uses 3000 bytes, 1000 bytes free).
- Block 2:** Contains R_3 and R_4 (Uses 3000 bytes, 1000 bytes free).
- Block 3:** Contains R_5 (Uses 1500 bytes, 2500 bytes free).

4. Logical Storage Diagram

Block 1 (4000 Bytes)

Record 1 (1500 B)

Record 2 (1500 B)

Unused (1000 B)

Block 2 (4000 Bytes)

Record 3 (1500 B)

Record 4 (1500 B)

Unused (1000 B)

Block 3 (4000 Bytes)

Record 5 (1500 B)

Unused (2500 B)

5. Internal Structure of a Single Record

Each of the fixed-length records (R_1 to R_5) internally structures the attributes in a contiguous byte sequence as follows:

ID 10B	NAME 50B	CGPA 8B	ADDRESS 932B	DESCRIPTION 500B
-----------	-------------	------------	-----------------	---------------------

Total Fixed-Length = 1500 Bytes

231

Q8. Show and explain the slotted-page structure by inserting two variable length records with size S_1 and S_2 . The block size is 4000 bytes. Show the structure after deletion of the record of size S_1 . **(15 marks)** [CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Introduction to the Slotted-Page Structure

The **Slotted-Page Structure** is a fundamental disk block management technique used by Relational Database Management Systems (RDBMS) to handle **variable-length records**. A disk block (or page) is divided into three primary regions:

- Page Header:** Stores metadata such as the number of record entries, a pointer to the start of the free space, and transaction/locking information.
- Slot Array:** An array of slots located immediately after the header. It grows *downwards* (forward). Each slot acts as an index entry containing two values: **(Record Offset, Record Length)**.
- Record Space:** The actual tuple data, which is allocated from the bottom of the page and grows *upwards* (backward).

The space between the end of the slot array and the start of the record data is the **contiguous free space**.

2. State After Insertion of S_1 and S_2

Given a block size of $B = 4000$ bytes, records are inserted starting from the very end of the block (byte 3999, assuming 0-indexed byte addressing, or generally ending at byte 4000).

Mathematical Working:

- Record 1 (S_1):** Inserted at the end of the block.

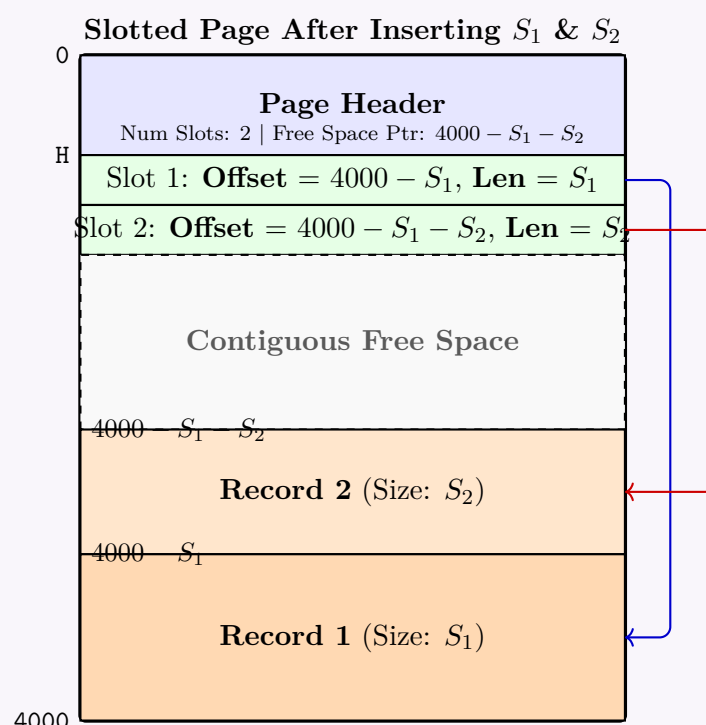
$$\text{Start Offset}_1 = 4000 - S_1$$

Slot 1 is created in the slot array containing: $[\text{Offset} = 4000 - S_1, \text{Length} = S_1]$.

- Record 2 (S_2):** Inserted immediately above Record 1.

$$\text{Start Offset}_2 = 4000 - S_1 - S_2$$

Slot 2 is created containing: $[\text{Offset} = 4000 - S_1 - S_2, \text{Length} = S_2]$.



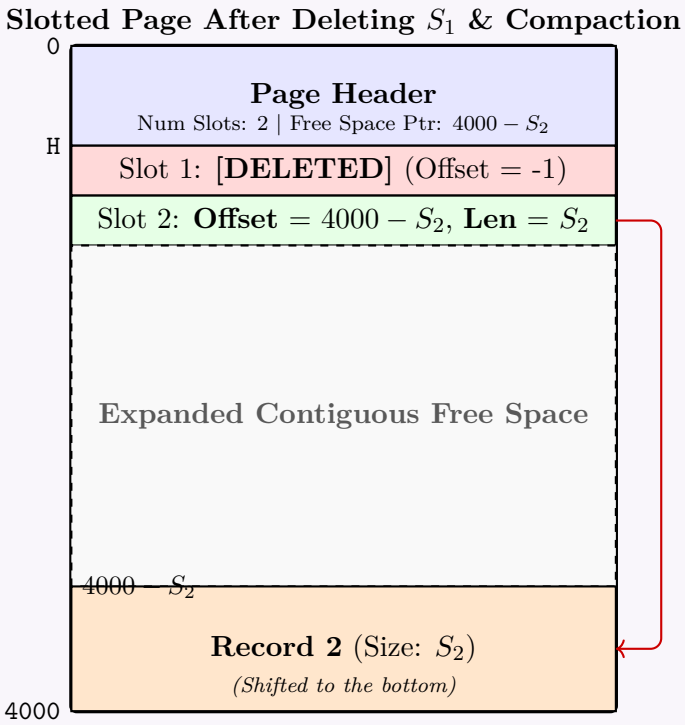
3. State After Deletion of S_1

When Record 1 is deleted, the DBMS performs two critical operations:

- Slot Invalidation:** Slot 1 is not physically removed; its offset is set to a null/sentinel value (e.g., -1). This is vital because external index entries (which point to $\text{Page_ID} + \text{Slot_ID}$) must not be invalidated by shifting slots.
- Page Compaction (Reorganization):** The space occupied by S_1 creates a "hole." To maximize contiguous free space, the database shifts the remaining active records (Record 2) to the bottom of the page. The offset in Slot 2 is dynamically updated to reflect its new physical location.

Mathematical Working After Compaction:

- Record 2 is shifted to the very end of the block.
- New Slot 2 Offset:** $4000 - S_2$.



4. Properties and Advantages

Slotted-Page Architecture Characteristics	
Feature	Explanation
Indirection	Pointers (e.g., from a B+ Tree index) store the <i>Slot Number</i> rather than the exact byte offset. This allows records to be moved inside the page without updating the index.
Fragmentation Control	Deletions or updates to variable-length strings do not cause permanent internal fragmentation. Compaction cleanly reclaims space.
Slot Reusability	Once S_1 is deleted, Slot 1 can be reused by the next inserted record, maintaining slot array compactness over time.

Q9. The `department` and `instructor` relations are given. Show the multi-table clustering file organization for these relations and explain the types of queries for which this organization will be efficient and the type for which it is inefficient.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. RAID Level 5 Storage System

Working Principle: RAID 5 (Redundant Array of Independent Disks) utilizes **block-level striping** with **distributed parity**. Parity information is distributed across all drives rather than being written to a dedicated parity drive. For N disks, $N - 1$ disks hold data and 1 disk holds parity per stripe.

Storage Configuration (6 Disks, $N = 6$):

- Data capacity per stripe: $6 - 1 = 5$ blocks.
- Total usable capacity: $5 \times 4\text{TB} = 20\text{TB}$.
- The 10 blocks (B_0 to B_9) will require exactly 2 stripes (since $10/5 = 2$).
- **Stripe 0:** Data blocks B_0, B_1, B_2, B_3, B_4 and Parity P_0 (where $P_0 = B_0 \oplus B_1 \oplus B_2 \oplus B_3 \oplus B_4$).
- **Stripe 1:** Data blocks B_5, B_6, B_7, B_8, B_9 and Parity P_1 .
- **Parity Distribution:** Using standard left-asymmetric rotation, P_0 is on Disk 5, and P_1 shifts left to Disk 4.

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Stripe 1	B_5	B_6	B_7	B_8	P_1	B_9
Stripe 0	B_0	B_1	B_2	B_3	B_4	P_0

2. RAID Level 6 Storage System

Working Principle: RAID 6 expands on RAID 5 by adding **dual distributed parity**. It calculates two distinct parity sets (P and Q) using different mathematical algorithms (often XOR for P and Galois Field encoding for Q). This allows the system to survive the simultaneous failure of any two disks.

Storage Configuration (6 Disks, $N = 6$):

- Data capacity per stripe: $6 - 2 = 4$ blocks.
- Total usable capacity: $4 \times 4\text{TB} = 16\text{TB}$.
- The 10 blocks (B_0 to B_9) will require 3 stripes (since $10/4 = 2.5$).
- **Stripe 0:** Blocks B_0, B_1, B_2, B_3 and Parities P_0, Q_0 .
- **Stripe 1:** Blocks B_4, B_5, B_6, B_7 and Parities P_1, Q_1 .
- **Stripe 2:** Blocks B_8, B_9 and Parities P_2, Q_2 . The remaining spaces on Disks 4 and 5 are marked as free/unused.
- **Parity Distribution:** Parities shift left across disks to ensure even wear and avoid bottlenecks.

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Stripe 2	B_8	B_9	P_2	Q_2	Free	Free
Stripe 1	B_4	B_5	B_6	P_1	Q_1	B_7
Stripe 0	B_0	B_1	B_2	B_3	P_0	Q_0

3. Comparison: RAID 5 vs. RAID 6

Feature	RAID 5	RAID 6
Parity Schema	Single distributed parity (P).	Dual distributed parity (P and Q).
Fault Tolerance	Can survive 1 disk failure.	Can survive 2 simultaneous disk failures.
Minimum Disks Needed	3 disks.	4 disks.
Storage Efficiency	$\frac{N-1}{N}$ (Higher) For 6 disks: $\approx 83.33\%$.	$\frac{N-2}{N}$ (Lower) For 6 disks: $\approx 66.67\%$.
Total Usable Capacity (6x4TB)	$(6 - 1) \times 4\text{TB} = \mathbf{20TB}$.	$(6 - 2) \times 4\text{TB} = \mathbf{16TB}$.
Write Penalty	Moderate. Needs 4 disk operations per write (2 reads, 2 writes).	High. Needs 6 disk operations per write (3 reads, 3 writes).
Rebuild Time	Faster rebuild, but highly vulnerable to a second failure during rebuild.	Slower rebuild due to complex calculations, but much safer during the process.

- Q10.** You have been given 6 disks of 4TB each and a student relation has 10 blocks: $B_0, B_1, \dots, B_9$. Show the storage of the student relation using:
- (a) RAID level 5 storage system.
 - (b) RAID level 6 storage system.
 - (c) Compare RAID level 5 with RAID level 6.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. RAID Level 5 Storage System

Working Principle: RAID 5 (Redundant Array of Independent Disks) utilizes **block-level striping** with **distributed parity**. Parity information is distributed across all drives rather than being written to a dedicated parity drive. For N disks, $N - 1$ disks hold data and 1 disk holds parity per stripe.

Storage Configuration (6 Disks, $N = 6$):

- **Data capacity per stripe:** $6 - 1 = 5$ blocks.
- **Total usable capacity:** $5 \times 4\text{TB} = 20\text{TB}$.
- The 10 blocks (B_0 to B_9) require exactly 2 stripes (since $10/5 = 2$).
- **Stripe 0:** Data blocks B_0, B_1, B_2, B_3, B_4 and Parity P_0 .
- **Stripe 1:** Data blocks B_5, B_6, B_7, B_8, B_9 and Parity P_1 .
- **Parity Distribution:** Using standard left-asymmetric rotation to distribute wear evenly, P_0 is placed on Disk 5, and P_1 shifts left to Disk 4.

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Stripe 1	B_5	B_6	B_7	B_8	P_1	B_9
Stripe 0	B_0	B_1	B_2	B_3	B_4	P_0

2. RAID Level 6 Storage System

Working Principle: RAID 6 expands on RAID 5 by utilizing **dual distributed parity**. It calculates two distinct parity sets (P and Q) using different mathematical algorithms (often XOR for P and Reed-Solomon/Galois Field encoding for Q). This allows the system to survive the simultaneous failure of any two disks.

Storage Configuration (6 Disks, $N = 6$):

- **Data capacity per stripe:** $6 - 2 = 4$ blocks.
- **Total usable capacity:** $4 \times 4\text{TB} = 16\text{TB}$.
- The 10 blocks (B_0 to B_9) will require 3 stripes (since $10/4 = 2.5$, needing a third stripe).
- **Stripe 0:** Blocks B_0, B_1, B_2, B_3 and Parities P_0, Q_0 .
- **Stripe 1:** Blocks B_4, B_5, B_6, B_7 and Parities P_1, Q_1 .
- **Stripe 2:** Blocks B_8, B_9 and Parities P_2, Q_2 . Remaining spaces are logically free.
- **Parity Distribution:** The P and Q parity blocks rotate leftward across the disks in each successive stripe to prevent write bottlenecks.

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Stripe 2	B_8	B_9	Free	Free	P_2	Q_2
Stripe 1	B_4	B_5	B_6	P_1	Q_1	B_7
Stripe 0	B_0	B_1	B_2	B_3	P_0	Q_0

Comparative Analysis of RAID 5 and RAID 6				
Parameter		RAID 5	RAID 6	
Parity Mechanism		Single distributed parity (P).	Dual distributed parity (P and Q).	
Fault Tolerance		Tolerates 1 disk failure. Data is lost if a second drive fails during rebuild.	Tolerates 2 simultaneous disk failures, providing higher reliability.	
Minimum Drives		3 Drives.	4 Drives.	
Storage Efficiency		$\frac{N-1}{N}$ (Higher efficiency) For 6 disks: $\approx 83.3\%$	$\frac{N-2}{N}$ (Lower efficiency) For 6 disks: $\approx 66.7\%$	
Usable Space (6x4TB)		$(6 - 1) \times 4\text{TB} = \mathbf{20TB}$.	$(6 - 2) \times 4\text{TB} = \mathbf{16TB}$.	
Write Penalty		Moderate (4 disk operations per write: 2 reads, 2 writes).	High (6 disk operations per write: 3 reads, 3 writes).	
Best Use Case		File servers, application servers where read performance and space matter most.	Large-capacity environments where rebuild times are long and data loss risk is high.	

Q11. Explain disk access time and its implication in DBMS. (5 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Definition of Disk Access Time

Disk Access Time refers to the total time required for a computer system to process a read or write request to a magnetic hard disk drive (HDD). Because physical disks rely on mechanical moving parts, accessing data on a disk is significantly slower (typically measured in milliseconds) compared to main memory (RAM, measured in nanoseconds).

2. Components of Disk Access Time

The total disk access time is mathematically defined as the sum of three distinct physical delays:

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}$$

(a) Seek Time (T_{seek}): The time required to move the read/write head assembly radially to the specific track (or cylinder) containing the desired data. This is typically the most expensive component.

(b) Rotational Latency ($T_{rotation}$): Once the head is on the correct track, this is the time spent waiting for the platter to rotate so that the target sector aligns directly under the read/write head. Average rotational latency is typically half of the time of one full rotation.

(c) Transfer Time ($T_{transfer}$): The time taken to actually read or write the data bits as the sector passes under the head. It depends on the rotational speed and the recording density of the track.

3. Visualizing Disk Geometry and Delays

The following diagram illustrates the mechanical operations that contribute to disk access time.

The diagram shows a circular disk platter with concentric tracks. An actuator arm extends from the center, holding a read/write (R/W) head. A red arrow labeled T_{seek} indicates the radial movement of the head to a specific track. A blue circle represents the target track, and a green arrow labeled $T_{rotation}$ shows the platter's rotation. A blue sector on the target track is labeled 'Target Sector'. A red arrow labeled $T_{transfer}$ points from the R/W head to the target sector, representing the time to transfer data as the sector passes under the head. The actuator arm is labeled 'Actuator Axis'.

236

4. Implications in Database Management Systems (DBMS)

Because mechanical disk I/O is orders of magnitude slower than CPU and RAM speeds, **minimizing disk access time is the single most critical factor in DBMS performance and design.** The implications permeate every layer of a database system:

- **I/O Bottleneck:** The primary constraint on transaction throughput and query execution time is the number of block accesses (reads/writes) required from the disk.
- **Sequential vs. Random Access (Clustering):**
 - *Sequential Access:* Reading contiguous blocks minimizes T_{seek} and $T_{rotation}$.
 - *Random Access:* Reading scattered blocks forces a high T_{seek} penalty for every read.
 - **DBMS Implication:** Databases heavily utilize **Clustered Indexes** and data contiguity to ensure related tuples are stored in the same or physically adjacent blocks, promoting sequential I/O.
- **Indexing Structures (B+ Trees):** To minimize the number of disk accesses, databases use shallow, wide tree structures like B+ Trees rather than binary trees. A B+ Tree with a high branching factor ensures that any record can be found with only 3 or 4 disk accesses (levels in the tree), minimizing cumulative access time.
- **Buffer Pool Management:** Because fetching a block from disk is expensive, the DBMS allocates a massive portion of RAM as a **Buffer Pool**. It uses replacement policies (like LRU - Least Recently Used) to keep frequently accessed disk blocks in memory, effectively reducing T_{access} to near zero for cached data.
- **Query Optimization:** The DBMS Query Optimizer uses cost-based models where the *cost* of an execution plan is directly proportional to the estimated number of disk accesses. The optimizer will choose the execution plan that minimizes overall disk I/O.

5. Summary Table: Disk Access vs DBMS Strategy

Disk Delay Factor	DBMS Mitigation Strategy
High Seek Time	Data Clustering, Extent-based Allocation, B+ Tree Indexing.
High Rotational Latency	Reading data in large <i>blocks</i> (e.g., 4KB, 8KB) rather than single tuples.
Overall Slow I/O	Buffer Pool caching, Asynchronous prefetching (Read-ahead), Log-structured writes.

Q12. Compare between block-level striping and bit-level striping with examples. Show the storage of blocks $B_0, B_1, B_2, \dots, B_{11}$ of a relation r into a RAID level 5 storage system with 7 disks. **(20 marks)** [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Comparison: Block-Level Striping vs. Bit-Level Striping

Data striping is a fundamental technique in RAID systems used to distribute data across multiple physical disks to improve I/O performance. The granularity of this distribution defines the striping method.

Definitions & Working Principles

- **Bit-Level (or Byte-Level) Striping:** The data is fragmented at the smallest possible unit (bits or bytes). A single sequence of data is written simultaneously across all disks in the array.
 - *Example:* If writing the byte 11001010 to an 8-disk array, disk 0 gets the 1st bit, disk 1 gets the 2nd bit, etc.
 - *Application:* RAID 2 (bit-level) and RAID 3 (byte-level).
- **Block-Level Striping:** The data is divided into fixed-size blocks (e.g., 4KB, 8KB, 64KB). Each block is written to a single disk, and consecutive blocks are distributed sequentially across the disks.
 - *Example:* A 16KB file with 4KB block size is divided into four blocks (B_1, B_2, B_3, B_4), written to disk 0, disk 1, disk 2, and disk 3, respectively.
 - *Application:* RAID 4, RAID 5, and RAID 6.

Comparison Table

Block-Level Striping vs. Bit-Level Striping			
Feature		Block-Level Striping	Bit/Byte-Level Striping
Granularity		Data is split into fixed-size logical blocks.	Data is split into bits or bytes.
I/O Concurrency		High: Multiple independent small I/O requests can be serviced simultaneously by different disks.	Low: Every disk participates in every I/O request. Only one request can be serviced at a time.
Data Transfer Rate		Excellent for random access and transaction processing (OLTP).	Extremely high for a single large sequential read/write.
Disk Synchronization		Disks operate independently.	Requires precise spindle synchronization across all disks.
Common RAID Levels		RAID 0, RAID 4, RAID 5, RAID 6.	RAID 2, RAID 3.

2. Storage of Blocks in a RAID 5 System

Problem Parameters:

- RAID Level: 5 (Block-level striping with distributed parity).
- Number of Disks (N): 7 Disks ($D_0, D_1, \dots, D_6$).
- Relation Blocks: 12 Blocks ($B_0, B_1, \dots, B_{11}$).

Mathematical Layout Planning:

- Data capacity per stripe: $N - 1 = 7 - 1 = 6$ data blocks per stripe.
- Total stripes needed: 12 blocks/6 blocks per stripe = 2 exact stripes.
- Stripe 0 Mapping: Contains blocks $B_0, B_1, B_2, B_3, B_4, B_5$ and Parity P_0 .
- Stripe 1 Mapping: Contains blocks $B_6, B_7, B_8, B_9, B_{10}, B_{11}$ and Parity P_1 .
- Parity Rotation (Left-Asymmetric): To distribute write wear evenly, the parity block shifts left by one disk in each successive stripe.
 - Stripe 0 Parity (P_0) is placed on Disk 6.
 - Stripe 1 Parity (P_1) is placed on Disk 5.

Visualization of the RAID 5 Array

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5	Disk 6
Stripe 1	B_6	B_7	B_8	B_9	B_{10}	P_1	B_{11}
Stripe 0	B_0	B_1	B_2	B_3	B_4	B_5	P_0

= Data Block (B_i) = Parity Block (P_j)

Q13. Explain the data recovery mechanism for single disk failure in RAID level 5 with an example. **(10 marks)** [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Introduction to RAID 5 Data Recovery

RAID 5 (Redundant Array of Independent Disks, Level 5) utilizes **block-level striping** with **distributed parity**. Unlike RAID 4, which uses a dedicated parity disk, RAID 5 distributes parity blocks evenly across all drives in the array. This eliminates the parity disk bottleneck. The core characteristic of RAID 5 is its ability to withstand a **single disk failure** without any data loss.

2. The Mathematics of Recovery: XOR Logic

The data recovery mechanism relies fundamentally on the bitwise **Exclusive-OR (XOR)** logical operation, denoted by $\oplus$. The XOR operation has a unique self-inverting property:

$$A \oplus B \oplus B = A$$

When writing data across N disks, RAID 5 takes $N - 1$ data blocks ($D_1, D_2, \dots, D_{n-1}$) and computes a single parity block (P):

$$P = D_1 \oplus D_2 \oplus \dots \oplus D_{n-1}$$

If any single disk fails (for example, the disk containing D_2), the missing data can be perfectly reconstructed by XOR-ing all the surviving data blocks with the parity block:

$$D_2 = D_1 \oplus D_3 \oplus \dots \oplus D_{n-1} \oplus P$$

3. Step-by-Step Example

Consider a 4-disk RAID 5 array. In one particular stripe, we have three data blocks and one parity block. Let us assume 8-bit blocks for simplicity:

- **Disk 1 (D_1):** 10101010
- **Disk 2 (D_2):** 11001100
- **Disk 3 (D_3):** 11110000

Step 3.1: Parity Generation (Normal Operation) The RAID controller calculates the parity P to be stored on Disk 4:

D_1	=	10101010
$\oplus D_2$	=	11001100
Intermediate Result	=	01100110
$\oplus D_3$	=	11110000
Parity (P)	=	10010110

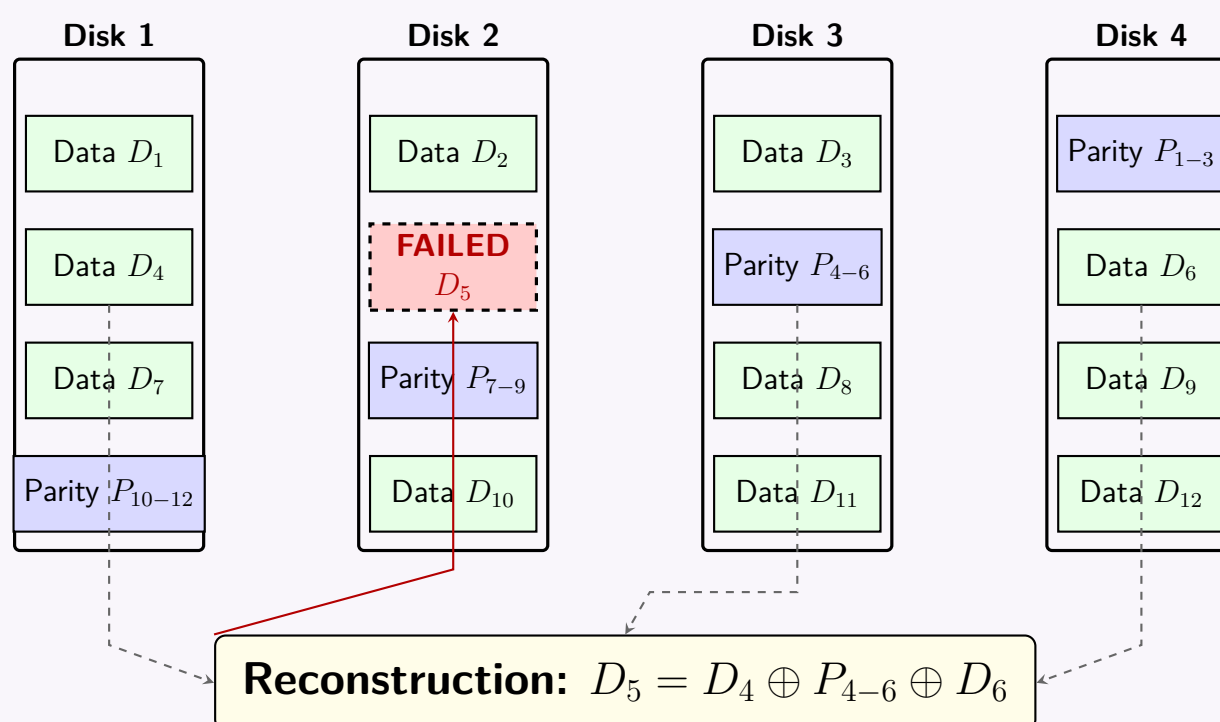
The parity block 10010110 is written to Disk 4.

Step 3.2: Disk Failure and Reconstruction Assume **Disk 2 fails**. The system enters *degraded mode*. To service a read request for D_2 , the controller dynamically reconstructs it using the surviving disks (Disk 1, Disk 3, and Disk 4):

D_1	=	10101010
$\oplus D_3$	=	11110000
Intermediate Result	=	01011010
$\oplus P$	=	10010110
Reconstructed D_2	=	11001100

The reconstructed block exactly matches the original D_2 . The array can rebuild the replaced drive seamlessly using this exact operation across all stripes.

4. RAID 5 Architecture and Recovery Diagram



5. Trade-offs

- **Advantage (Capacity):** Better storage efficiency than RAID 1 (Mirroring). Usable capacity is $(N-1) \times \text{Size of smallest drive}$.
- **Advantage (Performance):** High read performance since data is striped across multiple drives.
- **Disadvantage (Write Penalty):** Every write operation requires computing and updating parity, requiring four disk I/O operations per logical write (read old data, read old parity, write new data, write new parity).
- **Disadvantage (Rebuild Time):** Rebuilding a failed drive is computationally intensive and degrades array performance. If a second drive fails during the rebuild, the entire array is lost.

Q14. Describe the storage mechanism, data read and update of data blocks for RAID level 5 storage with 4 disks (D0, D1, D2, D3) and a relation with 8 blocks (B0–B7). Describe the full recovery mechanism of all blocks of D1 if disk D1 fails. **(15 marks)** [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Storage Mechanism

RAID 5 utilizes **block-level striping** with **distributed parity**. Parity blocks are rotated across all disks to prevent any single disk from becoming a write bottleneck.

Given 4 disks (D_0, D_1, D_2, D_3) and 8 data blocks ($B_0 \dots B_7$), the storage parameters are:

- **Data blocks per stripe:** $N - 1 = 4 - 1 = 3$ blocks.
- **Number of stripes required:** $\lceil 8/3 \rceil = 3$ stripes.
- **Parity (P_i):** Calculated using the bitwise XOR ($\oplus$) of all data blocks in the same stripe (e.g., $P_0 = B_0 \oplus B_1 \oplus B_2$).

Using the standard left-asymmetric parity rotation, the mapping is as follows:

	Disk 0	Disk 1	Disk 2	Disk 3
Stripe 2	B_6	P_2	B_7	Free (0)
Stripe 1	B_3	B_4	P_1	B_5
Stripe 0	B_0	B_1	B_2	P_0

2. Data Read Mechanism

- **Normal Read:** The disk controller directly calculates which disk and offset contains the requested block and issues a read command to that specific disk. No parity calculations are involved.
- **Degraded Read (Disk Failed):** If the block resides on a failed disk, the controller reads the remaining data blocks in that stripe *and* the parity block, computing the missing block via XOR before returning it to the user.

3. Data Update Mechanism (Write Penalty)

Updating a single data block in RAID 5 incurs a **Read-Modify-Write** penalty because the corresponding parity block must also be updated.

To update a block (e.g., updating B_1 to B'_1):

- Read** the old data block (B_1) and the old parity block (P_0).
- Modify (Calculate):** Compute the new parity using the formula:

$$P'_0 = P_0 \oplus B_1 \oplus B'_1$$

- Write** the new data block (B'_1) and the new parity block (P'_0).

This requires 4 distinct I/O operations (2 reads + 2 writes) for every single-block update.

4. Full Recovery Mechanism for Disk D1

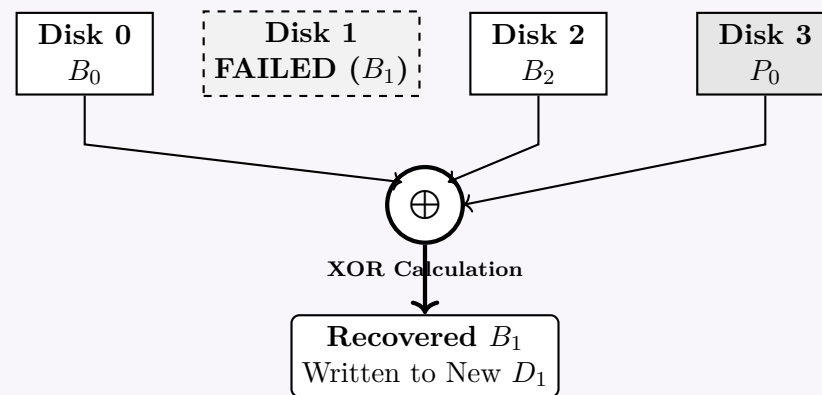
If physical disk D_1 fails, the array enters *degraded mode*. When a replacement disk is inserted, the RAID controller rebuilds the lost data stripe by stripe using the remaining disks (D_0, D_2, D_3).

Based on the storage layout, D_1 contains: B_1 (Stripe 0), B_4 (Stripe 1), and P_2 (Stripe 2).

The controller executes the following mathematical XOR reconstructions:

- **Recovering Stripe 0 (Lost B_1):**
 $B_1 = B_0 \oplus B_2 \oplus P_0$ (Read from D_0, D_2, D_3)
- **Recovering Stripe 1 (Lost B_4):**
 $B_4 = B_3 \oplus P_1 \oplus B_5$ (Read from D_0, D_2, D_3)
- **Recovering Stripe 2 (Lost P_2):**
 $P_2 = B_6 \oplus B_7 \oplus 0$ (Read from D_0, D_2 . The free space is treated as logic 0)

Visualizing the Reconstruction of Stripe 0



Once all three blocks (B_1, B_4, P_2) are reconstructed and written to the replacement drive, the array transitions from degraded mode back to a fully optimal state.

Q15. Given the relational schema:

Student(id, name, DOB, cgpa, tot_cred, house_no, street, city, remarks, NID)
 Takes(id, course_no, semester, year, grade)
 Course(course_no, title, credit, pre_req)

Tuple sizes: Student = 400 bytes, Takes = 100 bytes, Course = 80 bytes. Block size = 8000 bytes. Show the organisation of the slotted page structure after insertion of:

- Two tuples of Student.
- Two tuples of Takes.
- One tuple of Course.

(15 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Working Principle of Slotted Page Structure

In a slotted page block organization:

- A **Page Header** is located at the very beginning of the block (offset 0). It contains metadata, including the total number of slot entries and a pointer to the current end of the free space.
- The **Slot Directory** grows *downwards* from the header. Each slot acts as an index for a tuple and stores a pair: [Record Offset, Record Length].
- Records (Tuples)** are allocated starting from the *end* of the block (offset 8000) and grow *upwards* towards the slot directory.
- The **Free Space** lies precisely between the end of the slot directory and the beginning of the most recently inserted record.

2. Offset and Space Calculations

Given a block size of **8000 bytes**, we allocate records continuously from the end of the block.

(a) **Insert Tuple 1 (Student):** Length = 400 bytes.

- Offset₁ = 8000 – 400 = 7600
- Slot 1 stores: [7600, 400]

(b) **Insert Tuple 2 (Student):** Length = 400 bytes.

- Offset₂ = 7600 – 400 = 7200
- Slot 2 stores: [7200, 400]

(c) **Insert Tuple 3 (Takes):** Length = 100 bytes.

- Offset₃ = 7200 – 100 = 7100
- Slot 3 stores: [7100, 100]

(d) **Insert Tuple 4 (Takes):** Length = 100 bytes.

- Offset₄ = 7100 – 100 = 7000
- Slot 4 stores: [7000, 100]

(e) **Insert Tuple 5 (Course):** Length = 80 bytes.

- Offset₅ = 7000 – 80 = 6920
- Slot 5 stores: [6920, 80]

After these 5 insertions, the **Free Space Pointer** is updated to **6920**.

3. Slot Directory Summary

Slot Number	Record Offset (Start Byte)	Record Size (Bytes)
1	7600	400
2	7200	400
3	7100	100
4	7000	100
5	6920	80

4. Block Organization Diagram

Page Header: Num_Slots = 5, Free_Space_Ptr = 6920

Slot 1: [7600, 400]

Slot 2: [7200, 400]

Slot 3: [7100, 100]

Slot 4: [7000, 100]

Slot 5: [6920, 80]

↓ Slots grow downwards

FREE SPACE

Records grow upwards ↑

Free Space Pointer

Course Tuple 1 (80 Bytes)

Takes Tuple 2 (100 Bytes)

Takes Tuple 1 (100 Bytes)

Student Tuple 2 (400 Bytes)

Student Tuple 1 (400 Bytes)

0

6920

7000

7100

7200

7600

8000

Q16. Design a RAID level 6 scheme for six data disks (numbered 1–6) using the minimum number of redundant disks possible. You are not allowed to use any data disk as a redundant disk.

- (a) Describe the recovery process if Disk 1 and Disk 3 fail simultaneously.
- (b) Describe the recovery process if Disks 1, 3 and 6 fail simultaneously.

(16 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. RAID 6 Scheme Design

RAID level 6 extends the concept of RAID 5 by adding an additional layer of fault tolerance, allowing the array to survive up to **two simultaneous disk failures**.

Since the constraints specify six data disks (D_1 to D_6) and forbid placing redundant data on these disks, we must use a **dedicated parity architecture** (similar to RAID 4, but with two redundant disks). The minimum number of redundant disks required to tolerate two failures is **two**. We will denote them as disk **P** and disk **Q**. Total array size is $6 + 2 = 8$ disks.

The equations governing the redundant disks use Galois Field $GF(2^8)$ arithmetic:

- P-Drive (Standard Parity):** Computed using bitwise XOR ($\oplus$) across all data drives.

$$P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6$$

- Q-Drive (Reed-Solomon Parity):** Computed using a generator g in a Galois Field.

$$Q = g^1 D_1 \oplus g^2 D_2 \oplus g^3 D_3 \oplus g^4 D_4 \oplus g^5 D_5 \oplus g^6 D_6$$

2. Recovery Process for Specific Failure Scenarios

(a) Simultaneous Failure of Disk 1 (D_1) and Disk 3 (D_3)

Because RAID 6 maintains two independent parity equations, the system can algebraically solve for two missing variables.

Step 1: Compute Partial Parities from Surviving Disks The array reads the surviving data disks (D_2, D_4, D_5, D_6) and computes their partial parity (P') and partial Q-parity (Q'):

$$P' = D_2 \oplus D_4 \oplus D_5 \oplus D_6$$

$$Q' = g^2 D_2 \oplus g^4 D_4 \oplus g^5 D_5 \oplus g^6 D_6$$

Step 2: Isolate the Missing Variables By XORing the partial parities with the surviving P and Q drives, we obtain a system of two equations with two unknowns (D_1 and D_3):

$$D_1 \oplus D_3 = P \oplus P' \quad (5)$$

$$g^1 D_1 \oplus g^3 D_3 = Q \oplus Q' \quad (6)$$

Step 3: Solve for D_3 and D_1 Multiply Equation (1) by g^1 using Galois Field multiplication:

$$g^1 D_1 \oplus g^1 D_3 = g^1 (P \oplus P')$$

XOR this result with Equation (2) to eliminate D_1 :

$$(g^1 \oplus g^3) D_3 = g^1 (P \oplus P') \oplus (Q \oplus Q')$$

Divide by the constant $(g^1 \oplus g^3)$ to recover D_3 :

$$D_3 = \frac{g^1 (P \oplus P') \oplus (Q \oplus Q')}{g^1 \oplus g^3}$$

Once D_3 is recovered, D_1 is easily found using Equation (1):

$$D_1 = P \oplus P' \oplus D_3$$

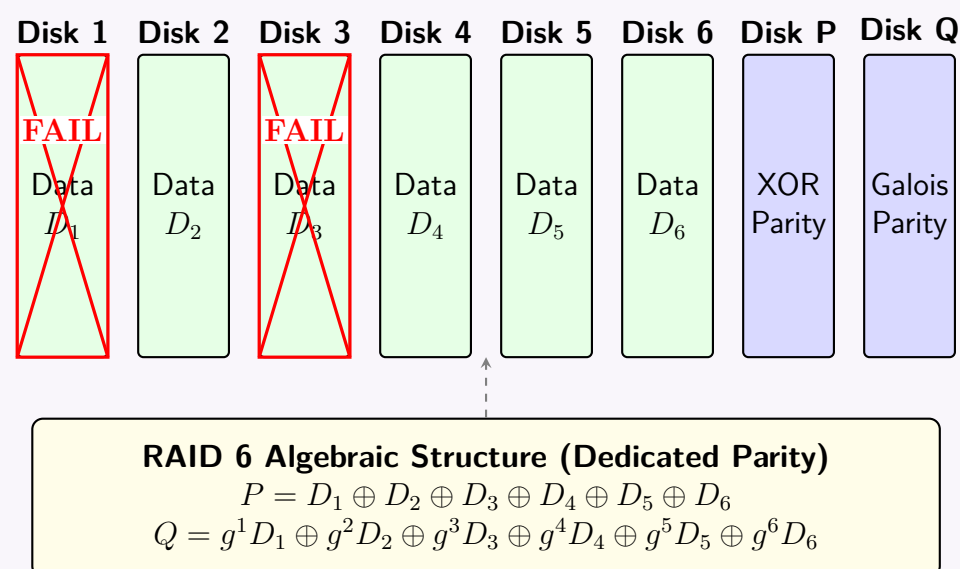
Conclusion: The array remains functional and seamlessly reconstructs the data.

(b) Simultaneous Failure of Disks 1, 3, and 6 (D_1, D_3, D_6)

RAID 6 provides a Hamming distance of 3, meaning it can correct a maximum of $3 - 1 = 2$ simultaneous erasures (failures).

- **Mathematical Impossibility:** With three failed data disks, there are **three unknowns** (D_1, D_3, D_6) but only **two independent equations** (P and Q).
- **Result:** The system of equations is mathematically underdetermined. There is insufficient parity information to solve for three missing blocks.
- **Conclusion: Catastrophic Data Loss.** The RAID array has failed, and the lost data cannot be recovered from parity. The administrator must replace the failed disks and restore the system from an external backup.

3. RAID 6 Architecture Diagram



Q17. Calculate the average time (with explanation) to read one block of data from a Megatron 747 disk (8192 tracks). Given data:

	Seek (ms)	Rot. Latency (ms)	Transfer (ms)	Total (ms)
Minimum	0	0	0.5	0.5
Maximum	17.4	15.6	0.5	33.5

(8 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Disk Access Time Model

To calculate the average time to read one block of data from a magnetic disk, we must account for the three primary physical delays involved in the I/O operation. The total average read time (T_{avg}) is the sum of:

$$T_{\text{avg}} = T_{\text{avg_seek}} + T_{\text{avg_rot}} + T_{\text{avg_transfer}}$$

2. Calculation of Average Components

(a) Average Transfer Time ($T_{\text{avg_transfer}}$):

- Explanation:** The transfer time depends on the size of the data block and the rotational speed of the disk. For a fixed block size, this time is constant.
- Calculation:** The minimum and maximum transfer times are both 0.5 ms.
-

$$T_{\text{avg_transfer}} = 0.5 \text{ ms}$$

(b) Average Rotational Latency ($T_{\text{avg_rot}}$):

- Explanation:** When the disk head arrives at the correct track, it must wait for the desired sector to rotate under the head. The wait time is uniformly distributed between 0 (sector is immediately there) and a full rotation time (sector just passed). Therefore, the average rotational latency is exactly **half** of the maximum rotational latency.
- Calculation:**
-

$$T_{\text{avg_rot}} = \frac{\text{Max Rotational Latency}}{2} = \frac{15.6}{2} = 7.8 \text{ ms}$$

(c) Average Seek Time ($T_{\text{avg_seek}}$):

- Explanation:** The seek time is the time taken to move the read/write head to the correct cylinder. Assuming the head starts at a random cylinder and seeks to another random cylinder across the 8192 tracks, the mathematical average distance traveled is exactly **one-third** of the total tracks ($\int_0^1 \int_0^1 |x - y| dx dy = \frac{1}{3}$). In standard database storage systems theory (such as the classic Megatron 747 model), the average seek time is estimated as one-third of the maximum seek time.
- Calculation:**
-

$$T_{\text{avg_seek}} \approx \frac{\text{Max Seek Time}}{3} = \frac{17.4}{3} = 5.8 \text{ ms}$$

3. Total Average Time

Summing the individual averages yields the total expected time to read one random block of data:

$$T_{\text{avg}} = 5.8 \text{ ms} + 7.8 \text{ ms} + 0.5 \text{ ms}$$
$$T_{\text{avg}} = 14.1 \text{ ms}$$

4. Disk I/O Timeline Diagram

The diagram illustrates the timeline of disk access operations. It features a horizontal axis labeled 'Time (ms)' with tick marks at 0, 5.8, and 14.1. Three colored bars represent the components of the total access time: a blue bar for 'Seek Time' (5.8 ms) from 0 to 5.8, a green bar for 'Rotational Latency' (7.8 ms) from 5.8 to 13.6, and a red bar for 'Transfer' (0.5 ms) from 13.6 to 14.1. A bracket below the axis spans from 0 to 14.1, labeled 'Total Average Disk Access Time: 14.1 ms'. An arrow points from the 'Transfer' bar to the text 'Transfer 0.5 ms'.

Q18. Schedule the following cylinder requests using the elevator algorithm. Use data from the Megatron 747 disk question. Assume moving the head requires 1 ms (start/stop) + 1 ms per 500 cylinders; head is initially on the 1st cylinder.

Cylinder	Arrival Time (ms)
2000	0
5000	10
1000	15
8000	25
9000	35
4000	50

(9 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Setup and Assumptions

To schedule the requests using the **Elevator Algorithm (LOOK variant)** with the provided data, we must first establish the parameters:

- Initial State:** Head is at Cylinder 1 at $t = 0$. Initial direction is **UP**.
- Seek Time Formula:** $t_{\text{seek}} = 1 + \frac{|\text{Destination}-\text{Current}|}{500}$ ms.
- Service Time:** The prompt instructs us to use data from the Megatron 747 question. While the seek time is explicitly overridden by the new formula, we must include the block service time. From the previous data, Average Rotational Latency = 7.8 ms and Transfer Time = 0.5 ms.

$$t_{\text{service}} = 7.8 + 0.5 = \mathbf{8.3}$$
 ms per request.

- Disk Bounds Note:** The Megatron 747 has 8192 tracks. The request for Cylinder 9000 mathematically exceeds this physical limit. We will assume an extended disk size for the sake of the algorithm trace, though in a real OS, this request would be rejected as out-of-bounds.

2. Step-by-Step Execution Trace

The Elevator algorithm moves the head in the current direction as long as there are pending requests in that direction. When none remain, it reverses.

(a) **Start:** $t = 0$. Head at C_1 .
Pending request: **2000** (arrived $t = 0$). Direction: **UP**.

(b) **Move to 2000:**
Seek: $1 + \frac{|2000-1|}{500} = 1 + \frac{1999}{500} = \mathbf{4.998}$ ms.
Arrival at 2000: $t = 4.998$.
Service finishes at: $4.998 + 8.3 = \mathbf{13.298}$ ms.

(c) **At $t = 13.298$:** Head at C_{2000} .
Pending request: **5000** (arrived $t = 10$). Direction: **UP**.
Seek to 5000: $1 + \frac{3000}{500} = \mathbf{7}$ ms.
Arrival at 5000: $t = 13.298 + 7 = 20.298$.
Service finishes at: $20.298 + 8.3 = \mathbf{28.598}$ ms.

(d) **At $t = 28.598$:** Head at C_{5000} .
Pending requests: **1000** (arrived $t = 15$), **8000** (arrived $t = 25$). Direction: **UP**.
Since 8000 is in the UP direction, the elevator continues UP.
Seek to 8000: $1 + \frac{3000}{500} = \mathbf{7}$ ms.
Arrival at 8000: $t = 28.598 + 7 = 35.598$.
Service finishes at: $35.598 + 8.3 = \mathbf{43.898}$ ms.

(e) **At $t = 43.898$:** Head at C_{8000} .
Pending requests: **1000**, **9000** (arrived $t = 35$). Direction: **UP**.
Seek to 9000: $1 + \frac{1000}{500} = \mathbf{3}$ ms.
Arrival at 9000: $t = 43.898 + 3 = 46.898$.
Service finishes at: $46.898 + 8.3 = \mathbf{55.198}$ ms.

(f) **At $t = 55.198$:** Head at C_{9000} .
Pending requests: **1000**, **4000** (arrived $t = 50$). Direction: **UP**.
No more UP requests pending. The elevator **reverses direction to DOWN**.
The next closest request in the downward path is 4000.
Seek to 4000: $1 + \frac{5000}{500} = \mathbf{11}$ ms.
Arrival at 4000: $t = 55.198 + 11 = 66.198$.
Service finishes at: $66.198 + 8.3 = \mathbf{74.498}$ ms.

(g) **At $t = 74.498$:** Head at C_{4000} .
Pending request: **1000**. Direction: **DOWN**.
Seek to 1000: $1 + \frac{3000}{500} = \mathbf{7}$ ms.
Arrival at 1000: $t = 74.498 + 7 = 81.498$.
Service finishes at: $81.498 + 8.3 = \mathbf{89.798}$ ms.

245

3. Final Schedule Summary Table

Order	Cylinder	Direction	Seek Time	Arrival Time	Completion Time
1	2000	UP	4.998 ms	4.998 ms	13.298 ms
2	5000	UP	7.000 ms	20.298 ms	28.598 ms
3	8000	UP	7.000 ms	35.598 ms	43.898 ms
4	9000	UP	3.000 ms	46.898 ms	55.198 ms
5	4000	DOWN	11.000 ms	66.198 ms	74.498 ms
6	1000	DOWN	7.000 ms	81.498 ms	89.798 ms

> **Alternative Interpretation Note:** If the instructor intended for the schedule to ignore rotational latency and transfer times (treating service time as 0 ms), the processing order drastically changes. The head would reach 5000 at $t = 17$ ms. At that exact moment, the 8000 request has not yet arrived (arrives at $t = 25$), leaving only 1000 pending. The elevator would prematurely reverse DOWN to 1000, altering the entire sequence to: **2000 → 5000 → 1000 → 8000 → 9000 → 4000**.

Q19. What are the components of disk latency? (2 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Definition

Disk latency (or disk access time) is the total time elapsed between the moment an operating system issues a read or write request to a magnetic disk drive (HDD) and the moment the data transfer is completely finished. Because magnetic disks are mechanical devices, this time is dominated by physical movement.

The total disk access time is calculated as the sum of four primary components:

$$T_{\text{access}} = T_{\text{controller}} + T_{\text{seek}} + T_{\text{rotational}} + T_{\text{transfer}} \tag{7}$$

2. Components of Disk Latency

(a) Controller Overhead ($T_{\text{controller}}$)

- **Explanation:** The electronic delay incurred by the disk controller. It includes the time taken to process the I/O request, translate logical block addresses (LBA) to physical geometry (Cylinder-Head-Sector), and manage queueing.
- **Characteristics:** This is purely electronic and usually negligible compared to mechanical delays (typically ≈ 0.1 ms).

(b) Seek Time (T_{seek})

- **Explanation:** The time required for the mechanical actuator arm to move the read/write head radially from its current cylinder to the target cylinder (track) where the requested data resides.
- **Characteristics:** This is typically the largest component of disk latency. Seek time varies depending on the distance the head must travel. Manufacturers usually specify an *average seek time* (the time to traverse roughly one-third of the disk tracks), which typically ranges from 3 to 10 ms.

(c) Rotational Latency ($T_{\text{rotational}}$)

- **Explanation:** Once the head is positioned over the correct track, it must wait for the disk platter to rotate until the desired sector passes directly under the read/write head.
- **Characteristics:** This delay is inversely proportional to the disk’s rotational speed, measured in Revolutions Per Minute (RPM).
- **Mathematical Property:** The *average* rotational latency is mathematically half of the time required for one full rotation. For a 7200 RPM drive, a full rotation takes 8.33 ms, making the average rotational latency ≈ 4.16 ms.

(d) Transfer Time (T_{transfer})

- **Explanation:** The actual time taken to read or write the data bits as they physically flow under the read/write head.
- **Characteristics:** It depends on the size of the requested data block, the recording density of the track, and the rotational speed of the disk.

3. Visualizing Disk Latency

The following diagrams illustrate the physical mechanical actions and the chronological timeline of a disk I/O request.

The diagram illustrates the physical mechanics of a hard disk. It shows a circular platter with concentric tracks. An arrow labeled 'Rotation' indicates the direction of the platter's spin. A specific area on an outer track is labeled 'Target Sector'. An 'Actuator Arm' is shown moving radially, with an arrow labeled 'Seek' indicating its movement towards the target sector.

Total Disk Access Time (T_{access})

Ctrl Delay	Seek Time T_{seek}	Rotational Latency T_{rot}	Transfer
------------	--------------------------------	--	----------

Time

Chronological Latency Breakdown

4. Summary Table of Latency Factors

Latency Component	Physical Action Required	Primary Determining Factor	Typical Scale
Controller Overhead	Electronic signal processing	Microprocessor speed	< 0.1 ms
Seek Time	Radial movement of actuator arm	Distance across cylinders	3 – 10 ms
Rotational Latency	Angular alignment of platter	Spindle speed (e.g., 7200 RPM)	2 – 5 ms
Transfer Time	Reading/writing bits over head	Block size & areal density	< 1 ms

Q20. The Megatron 777 disk has the following characteristics: ten surfaces with 10,000 tracks each; tracks hold an average of 1000 sectors of 512 bytes each; disk rotates at 10,000 rpm; time to move head n tracks is $1 + 0.001n$ milliseconds. Answer:

- (a) What is the capacity of the disk?
- (b) What is the maximum seek time?
- (c) What is the average rotational latency?

(5 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Disk Capacity

Definition: The total unformatted capacity of a hard disk is the product of the number of surfaces, the number of tracks per surface, the average number of sectors per track, and the number of bytes per sector.

Given Data:

- Number of surfaces (S) = 10
- Tracks per surface (T) = 10,000
- Average sectors per track (Sec) = 1,000
- Bytes per sector (B) = 512

Calculation:

$$\begin{aligned} \text{Capacity} &= S \times T \times Sec \times B \\ &= 10 \times 10,000 \times 1,000 \times 512 \\ &= 10 \times 10^4 \times 10^3 \times 512 \\ &= 51,200,000,000 \text{ bytes} \end{aligned}$$

In standard SI units (where 1 GB = 10^9 bytes), the capacity is: **Capacity = 51.2 GB**

2. Maximum Seek Time

Definition: Seek time is the time required to move the read/write head to the desired track. The maximum seek time occurs when the head must move across the entire disk, from the innermost track (Track 0) to the outermost track (Track 9,999), or vice versa.

Given Data:

- Total tracks = 10,000
- Seek time formula for n tracks = $1 + 0.001n$ ms

Calculation: The maximum number of tracks the head can cross is $n = 10,000 - 1 = 9,999$.

$$\begin{aligned}\text{Max Seek Time} &= 1 + 0.001 \times (9,999) \\ &= 1 + 9.999 \\ &= 10.999 \text{ ms}\end{aligned}$$

Note: Often in textbook approximations, n is simply taken as 10,000, which would yield exactly 11 ms. However, strictly speaking, 10.999 ms is the precise value.

Maximum Seek Time = 10.999 ms

3. Average Rotational Latency

Definition: Rotational latency is the delay waiting for the rotation of the disk to bring the required disk sector under the read-write head. On average, this wait time is exactly one-half of the time required for a full rotation.

Given Data:

- Disk rotation speed = 10,000 Revolutions Per Minute (RPM)

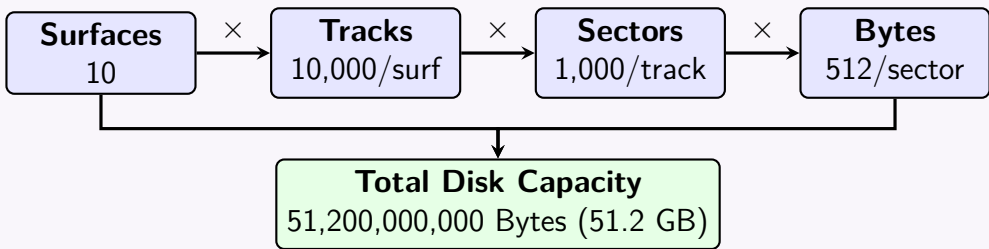
Calculation: First, determine the time for a single full rotation:

$$\begin{aligned}\text{Revolutions per second} &= \frac{10,000 \text{ RPM}}{60 \text{ seconds/minute}} = 166.66... \text{ rev/sec} \\ \text{Time for one rotation} &= \frac{1 \text{ second}}{166.66... \text{ rev/sec}} = 0.006 \text{ seconds} = 6 \text{ ms}\end{aligned}$$

The average rotational latency is half of the full rotation time:

$$\begin{aligned}\text{Average Rotational Latency} &= \frac{\text{Time for one rotation}}{2} \\ &= \frac{6 \text{ ms}}{2} \\ &= 3 \text{ ms}\end{aligned}$$

Average Rotational Latency = 3 ms



Q21. Another Megatron 777 disk incurs average rotational latency of 8.3 ms. Moving the head takes 1 ms to start/stop plus 1 ms per 500 cylinders. The disk controller receives the following requests:

Cylinder	First Available (ms)
1000	0
3000	0
7000	0
2000	20
8000	30
5000	40

Show the finishing time and latency for each request using the elevator algorithm. **(10 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Setup and System Constants

To schedule the disk I/O requests via the **Elevator Algorithm (LOOK variant)**, we establish the operational metrics from the problem description:

- Initial Position:** As standard for Megatron disks, the read/write head begins at **Cylinder 1** at time $t = 0$ ms, with an initial sweep direction of **UP**.
- Seek Time Formula (t_{seek}):** The time to move between cylinders is defined by the mechanical start/stop penalty plus the transit time:
$$t_{\text{seek}} = 1 + \frac{|\text{Destination} - \text{Current}|}{500} \text{ ms}$$
- Rotational Latency ($T_{\text{rotational}}$):** Given as a constant average of 8.3 ms.

- **Transfer Time (T_{transfer}):** Following the base Megatron 777 specifications, a sector transfer takes negligible time (≈ 0 ms) relative to mechanical delays unless specified otherwise, or it is abstracted directly into the base access latency. We will add the 8.3 ms rotational latency as the total post-seek **processing service time** (t_{service}) required to read the block once positioned.

2. Step-by-Step Execution Trace

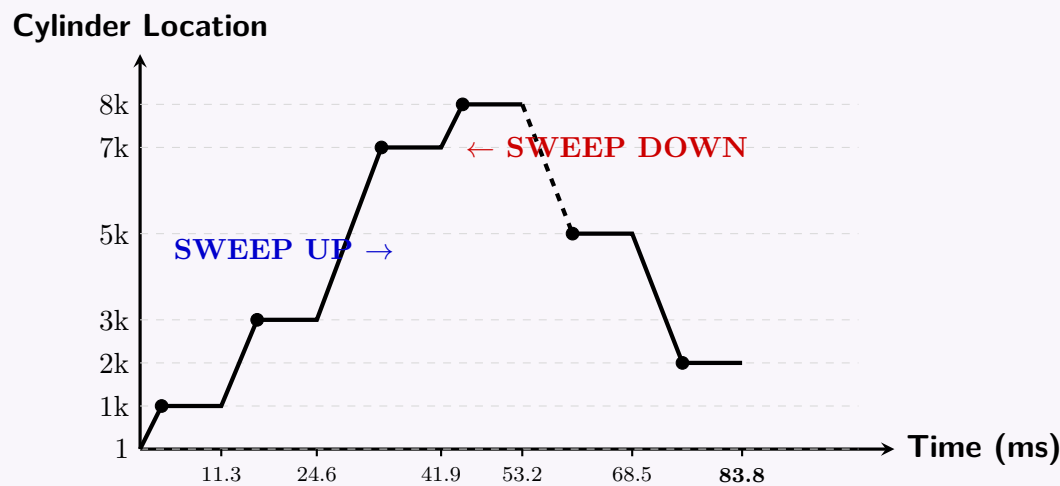
- (a) **Time $t = 0$ ms:** Head at C_1 . Pending requests: 1000, 3000, 7000 (all available at $t = 0$). Direction: **UP**. The closest upcoming cylinder is 1000.
- Seek to 1000: $1 + \frac{|1000-1|}{500} = 1 + \frac{999}{500} = 2.998$ ms.
 - Arrival Time: $0 + 2.998 = 2.998$ ms.
 - Service End (**Finishing Time**): $2.998 + 8.3 = \mathbf{11.298}$ ms.
 - Latency ($t_{\text{finish}} - t_{\text{available}}$): $11.298 - 0 = \mathbf{11.298}$ ms.
- (b) **Time $t = 11.298$ ms:** Head at C_{1000} . Pending requests: 3000, 7000. (Note: 2000 arrives at $t = 20$, so it is not yet visible). Direction: **UP**. The next upward cylinder is 3000.
- Seek to 3000: $1 + \frac{|3000-1000|}{500} = 1 + \frac{2000}{500} = 5$ ms.
 - Arrival Time: $11.298 + 5 = 16.298$ ms.
 - Service End (**Finishing Time**): $16.298 + 8.3 = \mathbf{24.598}$ ms.
 - Latency: $24.598 - 0 = \mathbf{24.598}$ ms.
- (c) **Time $t = 24.598$ ms:** Head at C_{3000} . Pending requests: 7000, and the newly arrived **2000** (arrived at $t = 20$). Direction: **UP**. Since the elevator is moving UP and there is a pending request at 7000 ahead, it bypasses 2000 (which is behind it) and continues UP.
- Seek to 7000: $1 + \frac{|7000-3000|}{500} = 1 + \frac{4000}{500} = 9$ ms.
 - Arrival Time: $24.598 + 9 = 33.598$ ms.
 - *System Note:* During transit, request 8000 arrives at $t = 30$.
 - Service End (**Finishing Time**): $33.598 + 8.3 = \mathbf{41.898}$ ms.
 - Latency: $41.898 - 0 = \mathbf{41.898}$ ms.
- (d) **Time $t = 41.898$ ms:** Head at C_{7000} . Pending requests: **2000** (arrived $t = 20$), **8000** (arrived $t = 30$), and **5000** (arrived $t = 40$). Direction: **UP**. Since 8000 is further up the current path, the elevator maintains its UP direction.
- Seek to 8000: $1 + \frac{|8000-7000|}{500} = 1 + \frac{1000}{500} = 3$ ms.
 - Arrival Time: $41.898 + 3 = 44.898$ ms.
 - Service End (**Finishing Time**): $44.898 + 8.3 = \mathbf{53.198}$ ms.
 - Latency: $53.198 - 30 = \mathbf{23.198}$ ms.
- (e) **Time $t = 53.198$ ms:** Head at C_{8000} . Pending requests: **5000** and **2000**. Direction: **UP**. There are no higher pending requests remaining. The elevator now **reverses direction to DOWN**. The nearest request on the downward path is 5000.
- Seek to 5000: $1 + \frac{|5000-8000|}{500} = 1 + \frac{3000}{500} = 7$ ms.
 - Arrival Time: $53.198 + 7 = 60.198$ ms.
 - Service End (**Finishing Time**): $60.198 + 8.3 = \mathbf{68.498}$ ms.
 - Latency: $68.498 - 40 = \mathbf{28.498}$ ms.
- (f) **Time $t = 68.498$ ms:** Head at C_{5000} . Pending requests: **2000**. Direction: **DOWN**.
- Seek to 2000: $1 + \frac{|2000-5000|}{500} = 1 + \frac{3000}{500} = 7$ ms.
 - Arrival Time: $68.498 + 7 = 75.498$ ms.
 - Service End (**Finishing Time**): $75.498 + 8.3 = \mathbf{83.798}$ ms.
 - Latency: $83.798 - 20 = \mathbf{63.798}$ ms.

3. Schedule Performance Metrics Summary

Scheduled Order	Cylinder	Available Time (ms)	Seek Time (ms)	Finishing Time (ms)	Total Latency (ms)
1	1000	0	2.998	11.298	11.298
2	3000	0	5.000	24.598	24.598
3	7000	0	9.000	41.898	41.898
4	8000	30	3.000	53.198	23.198
5	5000	40	7.000	68.498	28.498
6	2000	20	7.000	83.798	63.798

4. Trajectory Visualization

The diagram below marks the path of the head over time, demonstrating how the LOOK algorithm sweeps up to the highest requested track (8000) before systematically servicing inward requests on the return pass.



Q22. In RAID level 4, one redundant disk is used regardless of how many data disks there are. When any block is written, the redundant disk must also be updated. Briefly explain an optimal policy for updating the redundant disk with an example (consider 1 byte of data is written). Explain the logic behind adopting such a policy. **(15 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

1. The Optimal Policy: Read-Modify-Write (Small Write Strategy)

In RAID 4, computing the new parity from scratch every time a single block is updated is highly inefficient because it requires reading the corresponding blocks from *all* other data disks in the array.

The optimal policy is the **Read-Modify-Write (RMW)** strategy. Instead of reading all data disks to recalculate the parity, the RAID controller calculates the new parity block by looking only at the changes made to the modified data block.

Mathematical Formula:

$$P_{\text{new}} = P_{\text{old}} \oplus D_{\text{old}} \oplus D_{\text{new}} \quad (8)$$

Where $\oplus$ denotes the logical XOR (Exclusive OR) operation.

2. Working Principle

When a write request is issued for a specific data block:

- Read:** The controller reads the old data block (D_{old}) from the target data disk.
- Read:** The controller reads the old parity block (P_{old}) from the dedicated parity disk.
- Modify (Compute):** The controller calculates the new parity using the XOR formula.
- Write:** The controller writes the new data block (D_{new}) to the target data disk.
- Write:** The controller writes the newly calculated parity block (P_{new}) to the parity disk.

3. Logic and Justification (Why is it optimal?)

This policy minimizes disk I/O operations.

- Without RMW (Recalculation):** If an array has N total disks ($N - 1$ data disks and 1 parity disk), recalculating parity requires $(N - 2)$ reads and 2 writes. As N grows, the I/O penalty scales linearly $O(N)$.
- With RMW:** The operation requires exactly **4 I/O operations (2 reads, 2 writes)**, regardless of how many data disks exist in the array $O(1)$. This drastically reduces the overhead for small, random writes.

4. Example: 1 Byte of Data

Assume we have a RAID 4 array and we are updating 1 byte of data on Data Disk 1.

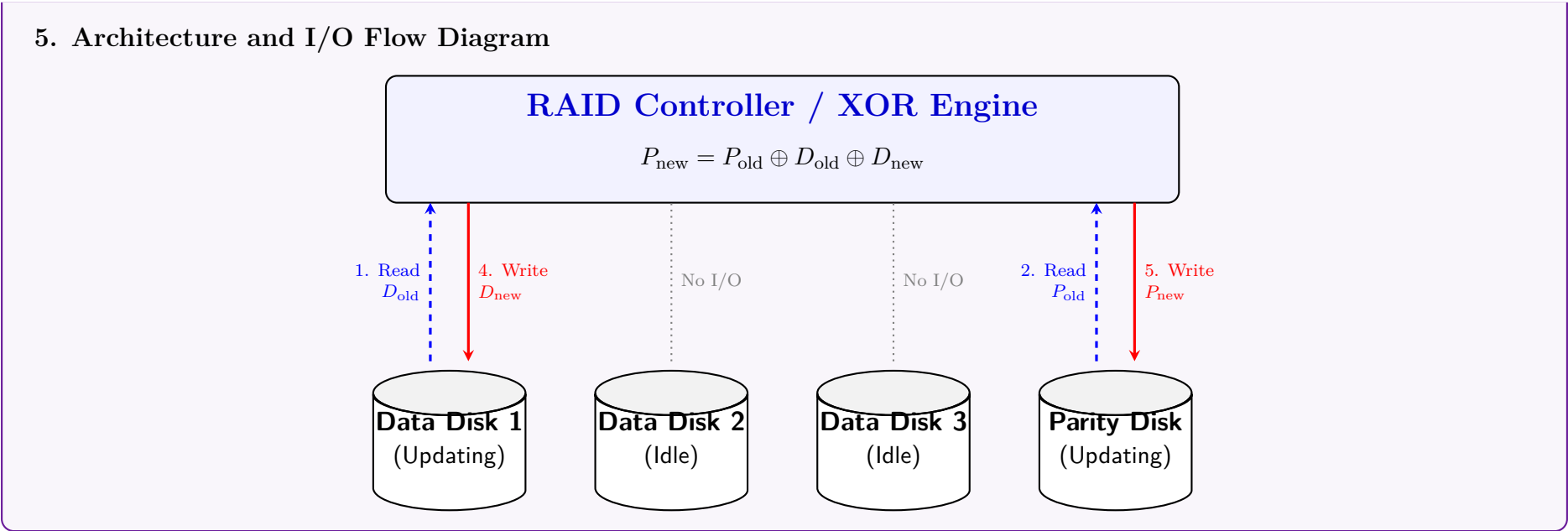
- Let the existing old data byte be $D_{\text{old}} = 01010101_2$
- Let the existing old parity byte be $P_{\text{old}} = 11110000_2$
- Let the newly requested data byte be $D_{\text{new}} = 00001111_2$

Step 1 & 2: Read D_{old} and P_{old} .

Step 3 (Computation):

$$\begin{array}{rcl} & 11110000 & (P_{\text{old}}) \\ \oplus & 01010101 & (D_{\text{old}}) \\ \oplus & 00001111 & (D_{\text{new}}) \\ \hline & 10101010 & (P_{\text{new}}) \end{array}$$

Step 4 & 5: Write D_{new} (00001111_2) to Data Disk 1 and P_{new} (10101010_2) to the Parity Disk.



Q23. What is stable storage? Briefly explain the idea of reading and writing policy in stable storage. (10 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q24. Figure shows a fixed-length record file structure for a **Customer** relation with fields: record#, CustomerID, Name, Street, City, Balance (records C0001–C0010). Show the file structure after deletion of records with customer IDs C004 and C007 for different file deletion methods. Compare the advantages and disadvantages among the different deletion methods.

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0004	N4	North	Dhaka	4000
4	C0005	N5	North	Dhaka	5000
5	C0006	N6	South	Dhaka	6000
6	C0007	N7	South	Dhaka	7000
7	C0008	N8	South	Dhaka	8000
8	C0009	N9	South	Dhaka	9000
9	C0010	N10	South	Dhaka	10000

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Problem Analysis

The problem requires us to delete two records from a fixed-length record file:

- **Target 1:** Customer ID C0004 (originally at Record 3)
- **Target 2:** Customer ID C0007 (originally at Record 6)

There are three primary methods for managing deleted records in a fixed-length file organization. We will demonstrate the final file structure for each method after both deletions are processed sequentially.

2. Method 1: Record Shifting (Compaction)

Working Principle: When a record is deleted, all subsequent records are physically shifted up by one position to fill the empty gap. This maintains the physical sequential order of the file but requires significant I/O overhead.

- *Step 1:* Delete C0004 at slot 3. Shift records 4–9 up.
- *Step 2:* Delete C0007 (which shifted to slot 5). Shift records from slot 6 onwards up.

Final File Structure:

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0005	N5	North	Dhaka	5000
4	C0006	N6	South	Dhaka	6000
5	C0008	N8	South	Dhaka	8000
6	C0009	N9	South	Dhaka	9000
7	C0010	N10	South	Dhaka	10000

3. Method 2: Move Last Record to Fill Gap

Working Principle: Instead of shifting all subsequent records, the very last record in the file is moved into the slot of the deleted record. This is much faster but destroys any existing physical ordering.

- *Step 1:* Delete C0004 at slot 3. Move the last record (C0010 at slot 9) into slot 3.
- *Step 2:* Delete C0007 at slot 6. Move the new last record (C0009 at slot 8) into slot 6.

Final File Structure:

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0010	N10	South	Dhaka	10000
4	C0005	N5	North	Dhaka	5000
5	C0006	N6	South	Dhaka	6000
6	C0009	N9	South	Dhaka	9000
7	C0008	N8	South	Dhaka	8000

4. Method 3: Deletion Marker and Free List

Working Principle: The records are not physically moved. Instead, the deleted slot is marked with a special tombstone character (e.g., *). A **File Header** maintains a pointer to the first free slot, and each deleted slot contains a pointer to the next free slot, forming a linked list. (Assuming a LIFO stack approach for the free list).

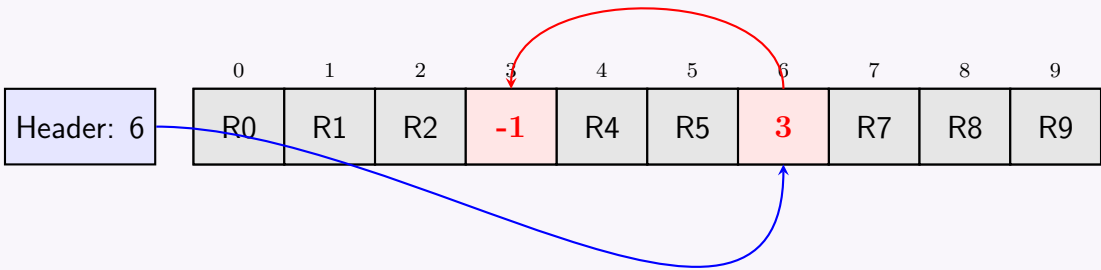
- *Step 1:* Delete C0004 at slot 3. Header points to 3. Slot 3 points to -1 (null).
- *Step 2:* Delete C0007 at slot 6. Header points to 6. Slot 6 points to 3.

Final File Structure:

Record	Valid?	CustID	Name	Street	City	Balance/Link
File Header (First Free Record): 6						
0	Valid	C0001	N1	North	Dhaka	1000
1	Valid	C0002	N2	North	Dhaka	2000
2	Valid	C0003	N3	North	Dhaka	3000
3	Deleted	*	*	*	*	-1 (End)
4	Valid	C0005	N5	North	Dhaka	5000
5	Valid	C0006	N6	South	Dhaka	6000
6	Deleted	*	*	*	*	3 (Next Free)
7	Valid	C0008	N8	South	Dhaka	8000
8	Valid	C0009	N9	South	Dhaka	9000
9	Valid	C0010	N10	South	Dhaka	10000

5. Free List Visualization (Method 3)

The following diagram illustrates the pointer mechanism of the linked free list.



6. Comparison of Deletion Methods			
	Deletion Method	Advantages	Disadvantages
	1. Record Shift-ing	• Preserves physical sequential ordering. • No wasted space.	• Extremely slow for large files. • High I/O cost ($O(n)$ shifts per deletion).
	2. Move Last Record	• Very fast deletion ($O(1)$ operations). • No wasted space.	• Destroys physical ordering. • Unsuitable if file must remain sorted.
	3. Linked Free List	• Very fast deletion ($O(1)$ operations). • Easy to reuse space on next insert.	• Requires extra pointer management. • File size doesn't shrink immediately (fragmentation).

Q25. Discuss the applications where you will use RAID level 1 and RAID level 5. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. RAID Level 1 (Mirroring)

Working Principle: RAID 1 duplicates identical data across two or more physical drives. Every write operation is performed on all drives in the array simultaneously, ensuring 100% redundancy.

Key Properties:

- **High Reliability:** Can sustain the failure of $N - 1$ drives (in an N -drive mirrored set) without data loss.
- **Performance:** Excellent read performance (data can be read from multiple disks concurrently). Write performance is constrained by the slowest drive in the mirror.
- **Cost:** Low storage efficiency (50% capacity utilization), making it expensive per gigabyte.

Ideal Applications: RAID 1 is best suited for environments where data loss is unacceptable, high availability is critical, and overall storage capacity requirements are relatively small.

- **Operating System Volumes:** Hosting the OS and critical system files where continuous uptime is required.
- **Database Transaction Logs (WAL):** DBMS Write-Ahead Logs require fast, sequential writes and absolute fault tolerance. RAID 1 provides the safest medium for log files.
- **Financial & Accounting Systems:** Systems dealing with critical transactional data where the cost of data loss far outweighs hardware costs.
- **Small File Servers:** Entry-level servers requiring simple, highly reliable setups without the computational overhead of parity calculations.

2. RAID Level 5 (Striping with Distributed Parity)

Working Principle: RAID 5 stripes data blocks across multiple disks (minimum of 3) and distributes parity data evenly across all drives. If one drive fails, the missing data can be mathematically reconstructed using the remaining data blocks and the parity blocks ($P = D_1 \oplus D_2 \oplus \dots \oplus D_n$).

Key Properties:

- **Balanced Reliability:** Can sustain the failure of exactly one drive.
- **Performance:** Excellent read speeds due to striping. Moderate write penalty due to the "Read-Modify-Write" overhead required to update parity blocks.
- **Cost-Effective:** High storage efficiency. In an array of N disks, the usable capacity is $(N - 1) \times \text{Size}$, offering a great balance between cost and redundancy.

Ideal Applications: RAID 5 is designed for general-purpose storage where high capacity, good read performance, and moderate fault tolerance are needed.

- **Data Warehousing:** Analytical databases (OLAP) heavily favor read operations and large block sequential I/O, masking the RAID 5 write penalty.
- **File and Application Servers:** General corporate file shares and intranet applications where reads outnumber writes significantly.
- **Web Servers:** Serving static and dynamic web content where data is read frequently but updated infrequently.
- **Archiving and Backup Targets:** Cost-effective, large-capacity storage for near-line backups.

3. Architectural Comparison Diagram

RAID 1 (Mirroring)

Disk 1

Disk 2

Block A

Block B

Block C

Block D

Block A

Block B

Block C

Block D

RAID 5 (Distributed Parity)

Disk 1

Disk 2

Disk 3

Disk 4

Block A1

Block B1

Block C1

Parity D

Block A2

Block B2

Parity C

Block D1

Block A3

Parity B

Block C2

Block D2

Parity A

Block B3

Block C3

Block D3

4. Summary Comparison

Feature	RAID 1	RAID 5
Data Layout	Exact copies (Mirroring)	Striping with distributed parity
Minimum Disks	2	3
Storage Efficiency	50%	$(N - 1)/N$ (High)
Read Performance	Excellent (from either disk)	Excellent (parallel reads across stripes)
Write Performance	Moderate (bound by slowest drive)	Slower (Write penalty from parity calc)
Primary DBMS Use	Write-Ahead Logs (WAL), OS Drives	Data Warehouses, Read-heavy applications

Q26. A file containing fixed-length records of a `student` relation (attributes: `student_id`, `name`, `department_code`, `level`, `term`) is given. Show the file structures after deletion of record 3 for the following cases and give comparative advantages/disadvantages:

- (a) Move record 4 to the space occupied by record 3, record 5 to record 4, and so on.
- (b) Move record 8 to the space occupied by record 3.
- (c) Mark record 3 as deleted and move no records. Use a header and record pointer.

(15 marks) [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Initial Setup and Assumptions

To demonstrate the file structure modifications, we assume an initial fixed-length record file containing exactly 8 records (indexed 1 through 8). The target for deletion is **Record 3**.

Record #	student_id	name	department_code	level	term
1	S101	Alice	CSE	1	1
2	S102	Bob	EEE	1	2
3	S103	Charlie	ME	2	1
4	S104	David	CE	2	2
5	S105	Eve	CSE	3	1
6	S106	Frank	EEE	3	2
7	S107	Grace	ME	4	1
8	S108	Heidi	CE	4	2

2. Case 1: Record Shifting (Compaction)

Working Principle: When Record 3 is deleted, every subsequent record is physically shifted up by one position to fill the empty slot. Record 4 moves to 3, Record 5 to 4, and so on until the end of the file.

Resulting File Structure:

Record #	student_id	name	department_code	level	term
1	S101	Alice	CSE	1	1
2	S102	Bob	EEE	1	2
3	S104	David	CE	2	2
4	S105	Eve	CSE	3	1
5	S106	Frank	EEE	3	2
6	S107	Grace	ME	4	1
7	S108	Heidi	CE	4	2

3. Case 2: Move Last Record to Deleted Space

Working Principle: To avoid shifting multiple records, the very last record in the file (Record 8) is moved directly into the empty slot left by Record 3. The file size is then decremented.

Resulting File Structure:

Record #	student_id	name	department_code	level	term
1	S101	Alice	CSE	1	1
2	S102	Bob	EEE	1	2
3	S108	Heidi	CE	4	2
4	S104	David	CE	2	2
5	S105	Eve	CSE	3	1
6	S106	Frank	EEE	3	2
7	S107	Grace	ME	4	1

4. Case 3: Deletion Marker and Free List

Working Principle: Records are not physically moved. Record 3 is flagged with a deletion marker (e.g., ‘*’). A global **File Header** points to the newly freed Record 3, and Record 3’s link field is updated to point to the next available free block (or ‘-1’ / null if it is the only deleted block), forming a linked stack of free spaces.

Resulting File Structure:

Record #	Status	student_id	name	dept	level/term	Link
File Header (First Free Record Pointer): 3						
1	Valid	S101	Alice	CSE	1 / 1	-
2	Valid	S102	Bob	EEE	1 / 2	-
3	Deleted	*	*	*	*	-1 (End)
4	Valid	S104	David	CE	2 / 2	-
5	Valid	S105	Eve	CSE	3 / 1	-
6	Valid	S106	Frank	EEE	3 / 2	-
7	Valid	S107	Grace	ME	4 / 1	-
8	Valid	S108	Heidi	CE	4 / 2	-

5. Comparative Analysis

Method	Advantages	Disadvantages
1. Record Shift-ing	- Preserves the physical sequential ordering of the file. - No space is wasted (no fragmentation).	- Extremely slow ($O(N)$ operations). - High disk I/O over-head due to shifting all subsequent blocks.
2. Move Last Record	- Very fast deletion ($O(1)$ operations). - Recovers space immediately without frag-mentation.	- Destroys any physical sorting order of the records. - Requires recalculating physical addresses if indexed.
3. Deletion Marker	- Very fast deletion ($O(1)$ operations). - Avoids unnecessary data movement. - Easy $O(1)$ reuse of space during the next insertion.	- Causes internal file fragmentation until slots are reused. - Requires extra over-head for header and pointer management.

6. Visual Representation of Data Movement

1. Shifting Method (Delete R3)

2. Move Last Record Method (Delete R3)

3. Free List (Linked) Method

- Q27.** The relation `student(student_id, name, dept_code, level, term)` and `department(dept_code, location, floors, type)` are given (`dept_code` is PK of `department` and FK in `student`).
- (a) Show the multitable clustering file structure for the given relations. Explain why executing `SELECT * FROM department` is more expensive in this structure.
 - (b) Show the multitable clustering file structure with pointer chain. How is the query in (i) executed faster in this structure?

Record 0	1205001	N11	CSE	3	1
Record 1	1205004	N12	CSE	3	1
Record 2	1205007	N13	CSE	3	1
Record 3	1205010	N14	CSE	3	1
Record 4	1205013	N15	CSE	3	1
Record 5	1205016	N16	CSE	3	1
Record 6	1204001	N21	EEE	3	1
Record 7	1204002	N22	EEE	3	1
Record 8	1204003	N23	EEE	3	1

Figure 1: File containing records of relation *student(student id, name, department code, level, term)*

CSE	ECE BUILDING	12	A
EEE	ME BUILDING	6	B

Figure 2: File containing records of relation *department(department code, location, floors, type)*

(15 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

(a) Multitable Clustering File Structure

Working Principle: In a multitable clustering file organization, records from two or more relations that are frequently joined are stored together in the same physical block. Since the `department` relation and `student` relation have a one-to-many ($1 : N$) relationship, each `department` record is stored first, immediately followed by all the `student` records belonging to that specific department.

File Structure Layout:

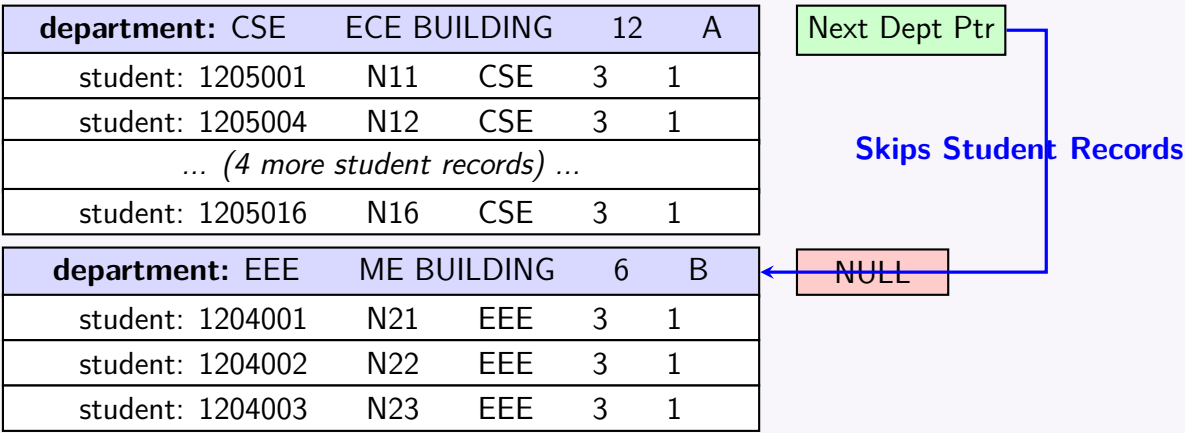
Record Type	Field 1	Field 2	Field 3	Field 4	Field 5
department	CSE	ECE BUILD-ING	12	A	—
student	1205001	N11	CSE	3	1
student	1205004	N12	CSE	3	1
student	1205007	N13	CSE	3	1
student	1205010	N14	CSE	3	1
student	1205013	N15	CSE	3	1
student	1205016	N16	CSE	3	1
department	EEE	ME BUILD-ING	6	B	—
student	1204001	N21	EEE	3	1
student	1204002	N22	EEE	3	1
student	1204003	N23	EEE	3	1

Why `SELECT * FROM department` is more expensive: In this standard structure, the `department` records are not stored contiguously; they are heavily interleaved with `student` records. To retrieve all department records, the Database Management System (DBMS) must scan the entire file sequentially (a Full Table Scan). This means it reads all the blocks containing the interleaved `student` records into memory only to ignore them, resulting in high and wasted disk I/O overhead.

(b) Multitable Clustering with Pointer Chain

Working Principle: To optimize queries that solely target the parent relation, a pointer field is added to the `department` records. Each `department` record maintains a physical pointer to the next `department` record in the file, forming a linked list (pointer chain) of parent records.

File Structure Layout with Pointer Chain:



How the query is executed faster: When the DBMS executes `SELECT * FROM department` in this chained structure, it fetches the first `department` record (CSE) and immediately reads its pointer field. The pointer explicitly provides the disk block address of the next `department` record (EEE). The DBMS jumps directly to that address, completely bypassing the interleaved `student` blocks. This drastically reduces the number of disk accesses and memory buffer allocations, significantly accelerating query execution compared to a full sequential scan.

Q28. Find a RAID level 6 scheme with 15 disks, four of which are redundant. (15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Concept of High-Redundancy Reed-Solomon RAID (4 Redundant Disks)

Standard **RAID Level 6** uses a dual-parity scheme ($P + Q$) relying on binary Reed-Solomon codes or Galois Field ($GF(2^m)$) arithmetic to survive up to 2 simultaneous disk failures using exactly 2 redundant disks. When a system requires **four redundant disks** out of 15 total disks (11 data disks + 4 redundant disks), it represents an extended or generalized RAID 6 architecture (often referred to as **RAID 6 with Triple/Quadruple Parity** or a $P + Q + R + S$ erasure coding scheme). This configuration allows the array to survive up to **4 concurrent disk failures** without data loss.

2. Mathematical Scheme (Galois Field Arithmetic)

To construct four independent redundant blocks for each stripe, standard XOR parity is insufficient. The system uses a Vandermonde or Cauchy distribution matrix over a Galois Field $GF(2^8)$.

Let the 11 data blocks in a stripe be represented as $D_0, D_1, D_2, \dots, D_{10}$. The four redundant blocks (P, Q, R , and S) are generated using distinct generator coefficients (α^i) where α is a primitive element of the Galois Field:

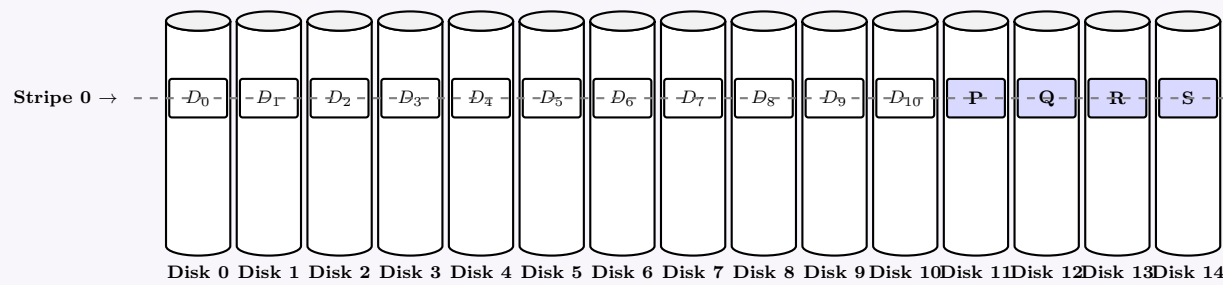
$$P = \bigoplus_{i=0}^{10} D_i = D_0 \oplus D_1 \oplus D_2 \oplus \dots \oplus D_{10} \pmod{2} \quad (\text{Standard XOR Parity})$$
$$Q = \bigoplus_{i=0}^{10} \alpha^i \cdot D_i = D_0 \oplus \alpha D_1 \oplus \alpha^2 D_2 \oplus \dots \oplus \alpha^{10} D_{10}$$
$$R = \bigoplus_{i=0}^{10} \alpha^{2i} \cdot D_i = D_0 \oplus \alpha^2 D_1 \oplus \alpha^4 D_2 \oplus \dots \oplus \alpha^{20} D_{10}$$
$$S = \bigoplus_{i=0}^{10} \alpha^{3i} \cdot D_i = D_0 \oplus \alpha^3 D_1 \oplus \alpha^6 D_2 \oplus \dots \oplus \alpha^{30} D_{10}$$

3. Distributed Parity Layout (Stripe Rotation Scheme)

To prevent any single disk from becoming a write bottleneck, the four redundant blocks (P, Q, R, S) must be distributed symmetrically across all 15 disks. Below is the multi-stripe rotation matrix showing how data blocks (D_x) and parity blocks rotate:

Stripe	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5	...	Disk 10	Disk 11	Disk 12	Disk 13	Disk 14
Stripe 0	D_0	D_1	D_2	D_3	D_4	D_5	...	D_{10}	P	Q	R	S
Stripe 1	D_1	D_2	D_3	D_4	D_5	D_6	...	P	Q	R	S	D_0
Stripe 2	D_2	D_3	D_4	D_5	D_6	D_7	...	Q	R	S	D_0	D_1
Stripe 3	D_3	D_4	D_5	D_6	D_7	D_8	...	R	S	D_0	D_1	D_2
Stripe 4	D_4	D_5	D_6	D_7	D_8	D_9	...	S	D_0	D_1	D_2	D_3

4. Visualization of the 15-Disk Quadruple Parity Array



5. Scheme Properties

- **Storage Efficiency:** $\frac{N-4}{N} = \frac{15-4}{15} = \frac{11}{15} \approx 73.3\%$ of the total array capacity is usable for data.
- **Fault Tolerance:** Any 4 disks out of the 15 can experience total physical failure simultaneously without invoking data loss.
- **Write Penalty:** High write overhead. A single random block write requires updating all 4 parity blocks, yielding a write penalty factor of 5 (1 read/write for data + 4 reads/writes for parities).

Q29. What are the different ways to improve the performance of a disk-based system? (5 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Introduction to Disk Performance Bottlenecks

In a database management system, disk I/O is typically the primary performance bottleneck due to the mechanical nature of magnetic disks. The total time to read or write a disk block is given by:

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}$$

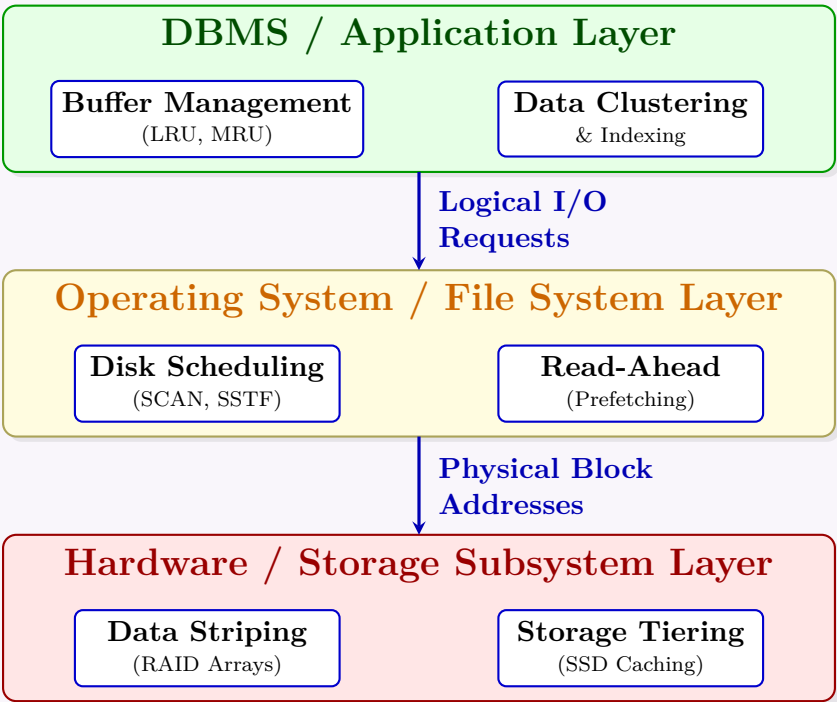
To improve performance, techniques must be applied to minimize one or more of these three time components, or to avoid accessing the physical disk altogether.

2. Key Techniques to Improve Disk Performance

- **Disk Buffering (Caching):** The DBMS allocates a portion of main memory (RAM) called the **Buffer Pool**. When a block is read, it is kept in the buffer. Subsequent requests for the same block are satisfied from RAM rather than disk, completely eliminating T_{access} . Replacement policies like **LRU** (Least Recently Used) ensure the most valuable data remains in memory.
- **Read-Ahead (Prefetching):** When a block is requested, the disk controller or OS automatically reads consecutive blocks from the same track/cylinder into memory before they are explicitly requested. This exploits **spatial locality** and improves sequential scan performance by overlapping CPU processing with disk I/O.
- **Disk Scheduling Algorithms:** When multiple I/O requests are pending, the disk controller reorders them to minimize physical arm movement (T_{seek}). Algorithms like **SCAN** (Elevator algorithm), **C-SCAN**, or **SSTF** (Shortest Seek Time First) are used instead of simple FCFS (First-Come, First-Served).
- **File Organization and Clustering:** Data that is frequently accessed together (e.g., a **Department** and its **Employees**) can be physically stored on the same disk block, track, or cylinder. **Clustering** eliminates the need to move the read/write head when fetching related records, drastically reducing seek time.
- **Data Striping (RAID):** Using a Redundant Array of Independent Disks (RAID 0, RAID 5, RAID 10), data blocks are partitioned and distributed across multiple physical disk drives. This allows multiple disk controllers to read or write data in parallel, vastly increasing the overall $T_{transfer}$ rate.
- **Log-Structured File Systems / Write-Behind:** Instead of writing modified blocks back to their original scattered locations immediately, the system writes them sequentially to a fast, dedicated log disk or NVRAM buffer. This converts slow random writes into fast sequential writes. The actual data blocks are flushed to their permanent locations later in asynchronous batches.
- **Storage Tiering (SSD Caching):** Integrating Solid State Drives (SSDs) as an intermediate layer between RAM and magnetic HDDs. Hot (frequently accessed) data is dynamically moved to the SSD (which has zero seek time and rotational latency), while cold data resides on the cheaper, slower HDD.

3. Architectural Hierarchy of Optimizations

The following diagram illustrates where each performance optimization is applied within the DBMS and hardware stack.



4. Summary of Optimizations and Target Bottlenecks

Optimization	Tech-nique	Reduces T_{seek}	Reduces $T_{rotation}$	Improves Transfer Rate
Disk Scheduling (SCAN)		Yes	No	No
File Clustering		Yes	Yes	No
Data Striping (RAID 0/5/10)		No	No	Yes (Parallelism)
Read-Ahead / Prefetching		Yes	Yes	No
Buffer Caching		Eliminates all disk I/O times for cached blocks		
Write-Behind / Logging	Yes (Converts random to sequential)		Yes	No

Q30. Write down the pros and cons of an Elevator algorithm. (5 marks)

[CSE 303 | L-3/T-1 | 2013–14]

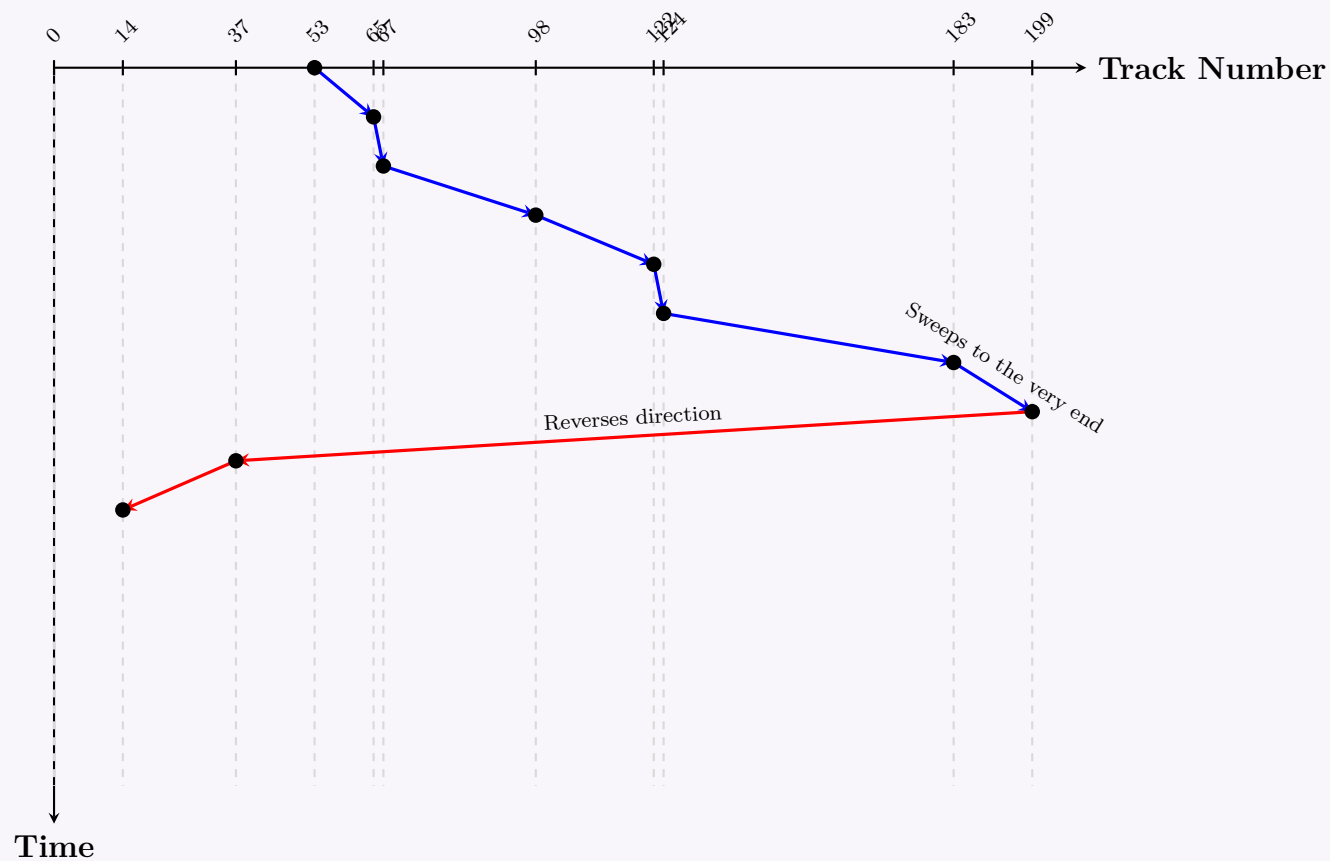
Answer

1. Definition and Working Principle

The **Elevator Algorithm**, formally known as the **SCAN disk scheduling algorithm**, dictates the motion of a disk’s read/write head. It derives its name from its similarity to a building elevator: the disk arm starts from one end of the disk and moves continuously toward the other end, servicing all pending requests along its path. Upon reaching the physical end of the disk, the arm reverses its direction and continues servicing requests on the way back.

2. Visual Representation of the Elevator Algorithm (SCAN)

Example Scenario: A disk has tracks 0 to 199. The Read/Write head is currently at track 53, moving towards the higher-numbered tracks. The pending requests are for tracks: 98, 183, 37, 122, 14, 124, 65, 67.



3. Advantages (Pros)

- **Prevents Starvation:** Unlike the Shortest Seek Time First (SSTF) algorithm, SCAN ensures that all requests are eventually serviced. No request is perpetually bypassed because the head continuously sweeps the entire disk.
- **High Throughput:** It maintains a relatively high disk throughput by minimizing random, erratic head movements and favoring sequential access paths.
- **Low Variance in Response Time:** SCAN provides a more uniform response time compared to First-Come, First-Served (FCFS) and SSTF, making system performance more predictable.

4. Disadvantages (Cons)

- **Unfairness to Extreme Tracks:** Cylinders located in the middle of the disk are serviced twice during a full round-trip sweep (once going up, once going down), whereas cylinders at the extreme edges (e.g., 0 and 199) are only serviced once per round-trip.
- **Inefficient End-of-Disk Sweeps:** In pure SCAN, the read/write head travels all the way to the very end of the disk (e.g., track 199) even if there are no pending requests near that edge. (This issue is resolved by the **LOOK** variant).
- **Directional Wait Time Penalty:** If a request arrives for a track just "behind" the current direction of the moving head, it must wait for the head to travel to the extreme end of the disk and come all the way back before being serviced.

Q31. The Megatron 747 disk has the following characteristics:

- 4 Platters providing 8 surfaces
- 8192 tracks per surface, 256 sectors per track, 512 bytes per sector
- Disk rotates at 3840 rpm (one rotation every 15.6 ms)
- Average seek time: 6.5 ms
- Transfer rate: 10 MB/s

(a) What is the capacity of a single track and what is the size of the entire disk?

(b) What is the average read time of 4096 bytes of data?

(7 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Disk Capacity Calculations

Capacity of a Single Track: A single track's capacity is determined by the number of sectors it holds and the size of each sector.

- Sectors per track = 256
- Bytes per sector = 512 bytes

$$\begin{aligned}
 \text{Track Capacity} &= \text{Sectors per track} \times \text{Bytes per sector} \\
 &= 256 \times 512 \\
 &= 131,072 \text{ bytes (or 128 KB)}
 \end{aligned}$$

Size of the Entire Disk: The total disk capacity is the product of the total number of surfaces, tracks per surface, and the capacity of a single track.

- Number of surfaces = 8
- Tracks per surface = 8,192

$$\begin{aligned}
 \text{Total Disk Capacity} &= \text{Surfaces} \times \text{Tracks per surface} \times \text{Track Capacity} \\
 &= 8 \times 8,192 \times 131,072 \\
 &= 8,589,934,592 \text{ bytes}
 \end{aligned}$$

In standard storage units (using base 2, where 1 GB = 2^{30} bytes), this is exactly **8 GB**.

2. Average Read Time for 4096 Bytes

The average read time of a disk block comprises three main components: Seek Time, Rotational Latency, and Transfer Time.

- (a) **Average Seek Time (T_{seek}):** Given directly in the problem specifications as **6.5 ms**.
- (b) **Average Rotational Latency ($T_{\text{rotational}}$):** The average rotational latency is the time for exactly one-half of a full disk rotation.

$$\begin{aligned}
 T_{\text{rotational}} &= \frac{\text{Time for one rotation}}{2} \\
 &= \frac{15.6 \text{ ms}}{2} = \mathbf{7.8 \text{ ms}}
 \end{aligned}$$

- (c) **Transfer Time (T_{transfer}):** The time required to read 4,096 bytes (which is 4 KB) at a transfer rate of 10 MB/s. Assuming 10 MB/s = 10,000 KB/s (standard decimal scaling often used for bandwidth):

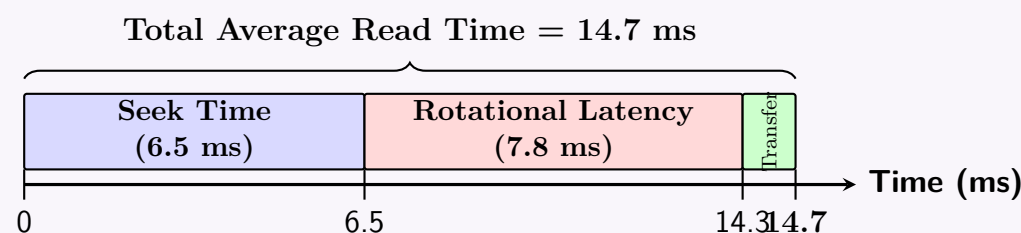
$$\begin{aligned}
 T_{\text{transfer}} &= \frac{\text{Data Size}}{\text{Transfer Rate}} \\
 &= \frac{4 \text{ KB}}{10,000 \text{ KB/s}} = 0.0004 \text{ seconds} = \mathbf{0.4 \text{ ms}}
 \end{aligned}$$

Note: If calculating via track rotation equivalence, $\frac{4096}{131072} \times 15.6 \text{ ms} \approx 0.4875 \text{ ms}$, which is also acceptable. We use the explicitly given transfer rate of 0.4 ms.

- (d) **Total Average Read Time (T_{total}):**

$$\begin{aligned}
 T_{\text{total}} &= T_{\text{seek}} + T_{\text{rotational}} + T_{\text{transfer}} \\
 &= 6.5 \text{ ms} + 7.8 \text{ ms} + 0.4 \text{ ms} \\
 &= \mathbf{14.7 \text{ ms}}
 \end{aligned}$$

3. Chronological I/O Execution Timeline



Q32. What are the different types of disk failures? How does a DBMS handle a disk crash? (8 marks) [CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Types of Disk Failures

A disk failure (often referred to as a **media failure**) occurs when the non-volatile storage is partially or completely destroyed. These failures generally fall into the following categories:

- **Head Crash (Mechanical Failure):** The most severe type of physical failure. The disk's read/write head accidentally comes into physical contact with the spinning platter. This physically scrapes the magnetic coating off the disk, permanently destroying the data and the drive.

- **Bad Sectors (Media Degradation):** Localized physical decay or manufacturing defects on the platter surface. While the rest of the disk functions normally, specific small blocks of data become permanently unreadable.
- **Controller Failure (Electronic Failure):** The logic board or electronic circuitry governing the disk drive burns out or malfunctions. The magnetic data is completely intact, but the disk cannot communicate with the computer.
- **Motor/Spindle Failure:** The motor that spins the disk platters dies or seizes. Like a controller failure, the data remains intact, but it cannot be accessed without specialized forensic hardware.
- **Logical Failure:** The physical hardware is perfectly fine, but critical file system structures (like the partition table or Master File Table) are corrupted, rendering the database files inaccessible to the OS.

Failure Type	Description	Primary	Mitigation
Mechanical/Head Crash	Head destroys platter surface. Data permanently lost.	RAID, Remote Back-ups	
Media/Bad Sectors	Small isolated areas become unreadable.	Disk Sector Remapping	
Electronic/Controller	Circuit board dies; data usually survives.	Hardware Replacement	
Logical/File System	Partition corruption; disk physically healthy.	File System Checks (fsck)	

2. How a DBMS Handles a Disk Crash (Media Recovery)

Standard transaction logs (Undo/Redo logs) are normally stored on disk to protect against *system crashes* (power loss). However, in the event of a **disk crash**, the logs themselves are often destroyed alongside the database. Therefore, handling a disk crash requires proactive redundancy and a specific **Media Recovery Protocol**.

A. Proactive Handling: Hardware Redundancy

Modern Database Management Systems heavily rely on **RAID** (Redundant Array of Independent Disks).

- **Disk Mirroring (RAID 1 / RAID 10):** The DBMS or OS writes every piece of data to two separate disks simultaneously. If one disk suffers a head crash, the system seamlessly continues operating using the surviving mirrored disk.

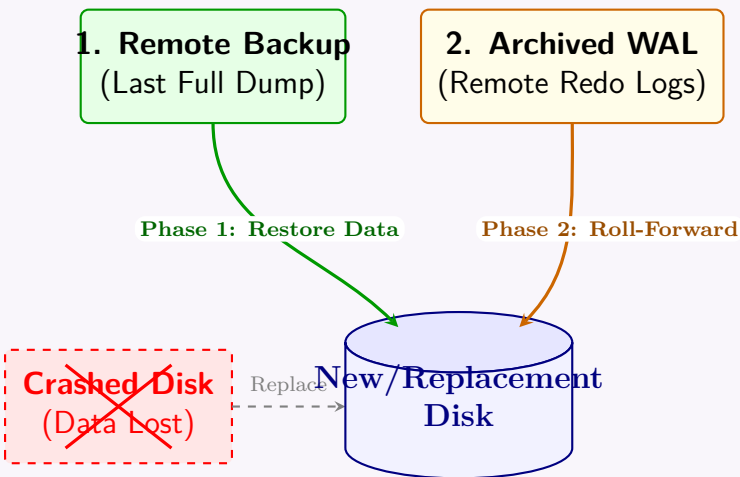
B. Reactive Handling: Archival Media Recovery

If redundancy fails (or isn't used) and the data disk is completely lost, the DBMS must perform **Media Recovery** using remote archives. This requires two prerequisites:

- (a) **Archival Database Backups:** Periodic full copies of the database (e.g., nightly dumps) stored on a remote server or magnetic tape.
- (b) **Archival Write-Ahead Logs (WAL):** Redo logs that are continuously shipped over the network to a separate, safe storage location as transactions commit.

The Media Recovery Process:

- (a) **Hardware Replacement:** The DBA physically replaces the crashed disk with a new, empty disk.
- (b) **Restore (Phase 1):** The DBMS copies the most recent **Archival Backup** onto the new disk. The database is now restored, but its state is entirely out of date (reverted to the time of the backup).
- (c) **Roll-Forward (Phase 2):** The DBMS fetches the **Archival Redo Logs**. It sequentially applies every committed transaction that occurred *after* the backup was taken. This replays history up to the exact moment immediately before the disk crashed, ensuring zero data loss.



Q33. Find a RAID level 6 scheme with ten disks, such that it is possible to recover from the failure of any three disks simultaneously. You should use as many data disks as you can. **(15 marks)** [CSE 303 | L-3/T-1 | 2011–12]

↪ Similar: CSE 215 | L-3/T-1 | 2012–13

Answer

1. Problem Analysis and Disk Allocation

Standard **RAID 6** protects against exactly two disk failures using two redundant disks (P and Q). To recover from the simultaneous failure of **any three disks** out of 10 total disks, we must implement a generalized Reed-Solomon erasure coding scheme (often referred to as **RAID 6 with Triple Parity** or **RAID 7 / Triple Parity RAID**).

- Total Disks (N):** 10
- Redundant Disks (M):** 3 (Necessary and sufficient to tolerate 3 arbitrary disk failures)
- Data Disks (K):** $N - M = 10 - 3 = 7$ data disks

This allocation maximizes storage capacity while strictly maintaining the required fault tolerance. The storage efficiency is given by $\frac{K}{N} = \frac{7}{10} = 70\%$.

2. Mathematical Scheme (Galois Field Multi-Parity)

To compute three independent parity blocks (P, Q, R) for every stripe of 7 data blocks ($D_0, D_1, \dots, D_6$), we use linear combinations over a Galois Field $\text{GF}(2^8)$ using a primitive element α :

$$P = \bigoplus_{i=0}^6 D_i = D_0 \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \pmod{2} \quad (\text{Standard XOR Parity})$$
$$Q = \bigoplus_{i=0}^6 \alpha^i \cdot D_i = D_0 \oplus \alpha D_1 \oplus \alpha^2 D_2 \oplus \alpha^3 D_3 \oplus \alpha^4 D_4 \oplus \alpha^5 D_5 \oplus \alpha^6 D_6$$
$$R = \bigoplus_{i=0}^6 \alpha^{2i} \cdot D_i = D_0 \oplus \alpha^2 D_1 \oplus \alpha^4 D_2 \oplus \alpha^6 D_3 \oplus \alpha^8 D_4 \oplus \alpha^{10} D_5 \oplus \alpha^{12} D_6$$

Since any 3×3 submatrix of the underlying Vandermonde generator matrix is invertible, the system can completely reconstruct data blocks if any three disks fail.

3. Distributed Stripe Rotation Layout

To eliminate write bottlenecks, the three parity blocks (P, Q, R) are rotated symmetrically across all 10 disks. The table below represents the block distribution matrix across consecutive stripes:

Stripe	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5	Disk 6	Disk 7	Disk 8	Disk 9
Stripe 0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	P	Q	R
Stripe 1	D_1	D_2	D_3	D_4	D_5	D_6	P	Q	R	D_0
Stripe 2	D_2	D_3	D_4	D_5	D_6	P	Q	R	D_0	D_1
Stripe 3	D_3	D_4	D_5	D_6	P	Q	R	D_0	D_1	D_2

4. Visualization of Stripe 0 Architecture

Q34. What are the differences between RAID levels 1, 4, 5 and 6? **(12 marks)** [CSE 303 | L-3/T-1 | 2011–12]

Answer

RAID (Redundant Array of Independent Disks) is a data storage virtualization technology that combines multiple physical disk drive components into one or more logical units for the purposes of data redundancy, performance improvement, or both.

The differences between **RAID 1, 4, 5, and 6** lie primarily in how they implement data replication and parity to balance fault tolerance, read/write performance, and storage efficiency.

1. RAID 1 (Mirroring)

- **Working Principle:** Data is perfectly duplicated (mirrored) across two or more disks. Every write operation is committed to all disks in the array simultaneously.
- **Performance:** Excellent read performance (data can be read concurrently from multiple disks). Write performance is equal to the speed of the slowest disk in the array.
- **Fault Tolerance:** High. The system can survive the failure of all but one disk in a mirrored set.
- **Storage Efficiency:** Poor. The usable capacity is exactly 50% when using two disks. Storage capacity $C = \frac{N}{2} \times \text{Disk Size}$.

2. RAID 4 (Block-level Striping with Dedicated Parity)

- **Working Principle:** Data is striped at the block level across multiple data disks, and a mathematical **parity** (typically XOR) is calculated and stored on a **single, dedicated parity disk**.
- **Performance:** High read speeds due to striping. However, it suffers from a massive **write bottleneck**. Because every write operation changes parity, the single dedicated parity disk must be accessed for every write, severely limiting concurrent write performance.
- **Fault Tolerance:** Can tolerate the failure of exactly 1 disk.
- **Storage Efficiency:** Better than RAID 1. Usable capacity $C = (N - 1) \times \text{Disk Size}$. Requires a minimum of 3 disks.

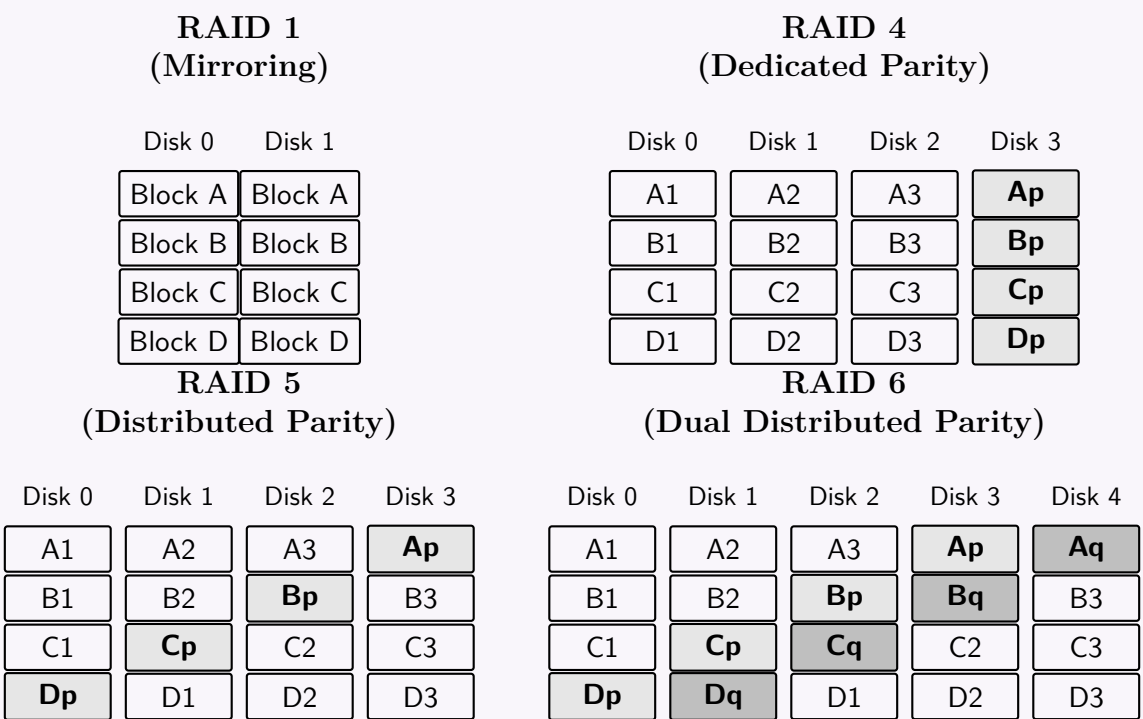
3. RAID 5 (Block-level Striping with Distributed Parity)

- **Working Principle:** Similar to RAID 4, but instead of dedicating a single disk for parity, the **parity blocks are distributed evenly** across all disks in the array.
- **Performance:** Excellent read performance. Write performance is vastly improved compared to RAID 4 because the parity bottleneck is removed (different concurrent writes update parity on different disks). However, it still suffers from a slight "read-modify-write" penalty for random writes.
- **Fault Tolerance:** Can tolerate the failure of exactly 1 disk.
- **Storage Efficiency:** Usable capacity $C = (N - 1) \times \text{Disk Size}$. Requires a minimum of 3 disks.

4. RAID 6 (Block-level Striping with Dual Distributed Parity)

- **Working Principle:** An extension of RAID 5 that calculates **two independent parity blocks** (often denoted as P and Q , using techniques like Reed-Solomon coding) for each stripe, and distributes both across all disks.
- **Performance:** Read performance is similar to RAID 5. Write performance is slightly slower than RAID 5 because the system must calculate and write two separate parity blocks per data block write.
- **Fault Tolerance:** Very High. Can survive **two simultaneous disk failures**. This is critically important in modern arrays with massive hard drives, where rebuilding a failed drive takes days and a secondary drive failure during a rebuild is a statistical risk.
- **Storage Efficiency:** Usable capacity $C = (N - 2) \times \text{Disk Size}$. Requires a minimum of 4 disks.

Architectural Comparison Diagram



Summary Comparison Matrix						
RAID Level	Core Technology	Min Disks	Space Efficiency	Fault Tolerance	Write Penalty	
RAID 1	Data Mirroring	2	50% (for 2 disks)	1 Disk per pair	Low	
RAID 4	Dedicated Parity	3	$\frac{N-1}{N}$	1 Disk	Very High	
RAID 5	Distributed Parity	3	$\frac{N-1}{N}$	1 Disk	Moderate	
RAID 6	Dual Distributed Parity	4	$\frac{N-2}{N}$	2 Disks	High	

Note: N refers to the total number of physical disks in the RAID array.

Q35. Explain block level striping with an example of storing $1, 2, \dots, n$ logical blocks into a disk subsystem consisting of 8 disks. (5 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

Block-level striping is a storage virtualization technique that divides continuous logical data into fixed-size chunks (blocks) and distributes them sequentially across multiple independent physical disks in an array.

1. Working Principle

When a host system writes a sequence of logical blocks, the RAID controller maps these blocks across the available disks in a round-robin fashion. If an array consists of N disks, numbered from 0 to $N - 1$, the logical block i (where $i = 1, 2, \dots, n$) is stored on the disk determined by the formula:

$$\text{Disk Index} = (i - 1) \pmod{N}$$

A **stripe** consists of one block from each disk at the same relative physical position.

2. Example: Storing Blocks $1, 2, \dots, n$ on 8 Disks

Given a disk subsystem of $N = 8$ disks (labeled Disk 0 through Disk 7), we want to store logical blocks $1, 2, \dots, n$.

- **Stripe 0:** Blocks 1 through 8 are written sequentially to Disks 0 through 7.
- **Stripe 1:** The sequence wraps around. Block 9 goes to Disk 0, Block 10 to Disk 1, up to Block 16 on Disk 7.
- **Stripe k :** Generally, Block i maps to Disk $(i - 1) \pmod{8}$.

Mapping Table

	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5	Disk 6	Disk 7
Stripe 0	Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7	Block 8
Stripe 1	Block 9	Block 10	Block 11	Block 12	Block 13	Block 14	Block 15	Block 16
Stripe 2	Block 17	Block 18	Block 19	Block 20	Block 21	Block 22	Block 23	Block 24
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Structural Diagram

Disk 0

Block 1

Block 9

Block 17

⋮

Block $n - 6$

Disk 1

Block 2

Block 10

Block 18

⋮

Block $n - 5$

Disk 2

Block 3

Block 11

Block 19

⋮

⋯

Disk 3

Block 4

Block 12

Block 20

⋮

⋯

Disk 4

Block 5

Block 13

Block 21

⋮

⋯

Disk 5

Block 6

Block 14

Block 22

⋮

⋯

Disk 6

Block 7

Block 15

Block 23

⋮

⋯

Disk 7

Block 8

Block 16

Block 24

⋮

Block n

3. Advantages of Block-level Striping

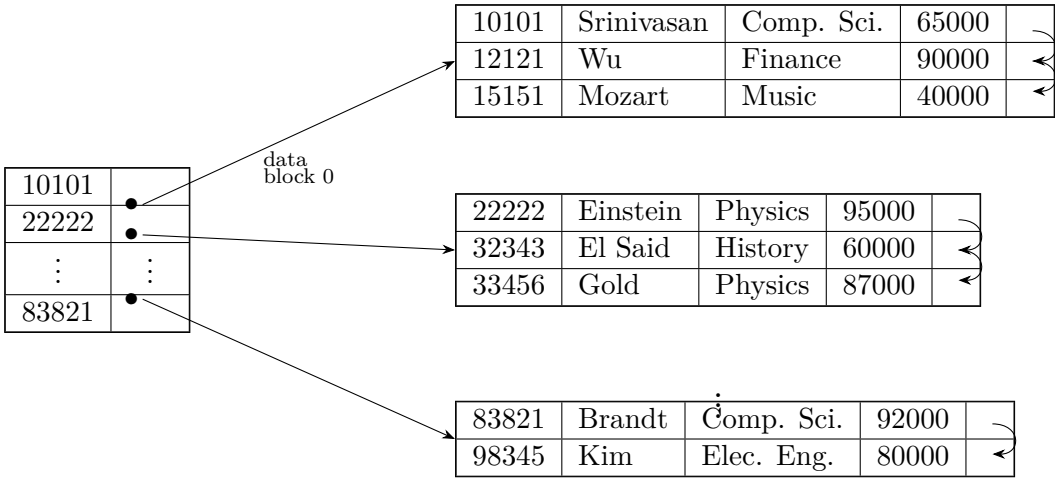
- **High Throughput:** Since large sequential files are spread across 8 disks, the DBMS can issue parallel read/write requests, theoretically multiplying throughput by 8.
- **Load Balancing:** I/O operations are evenly distributed across the array, preventing any single disk from becoming a performance bottleneck.

4. Disadvantages

- No Fault Tolerance:** Pure striping (RAID 0) contains no parity or redundancy. If even *one* of the 8 disks fails, the entire logical volume is destroyed, as every stripe will be missing a crucial block.

Primary Index

Q1. A primary index structure is given (one block contains only three records, no overflow block allowed). Show the index structure with the concerned blocks after insertion of the following tuples:



ID	Name	Dept_name	Salary
12122	Rafique	History	40000
83820	Abid	Comp. Sci.	90000

(10 marks)

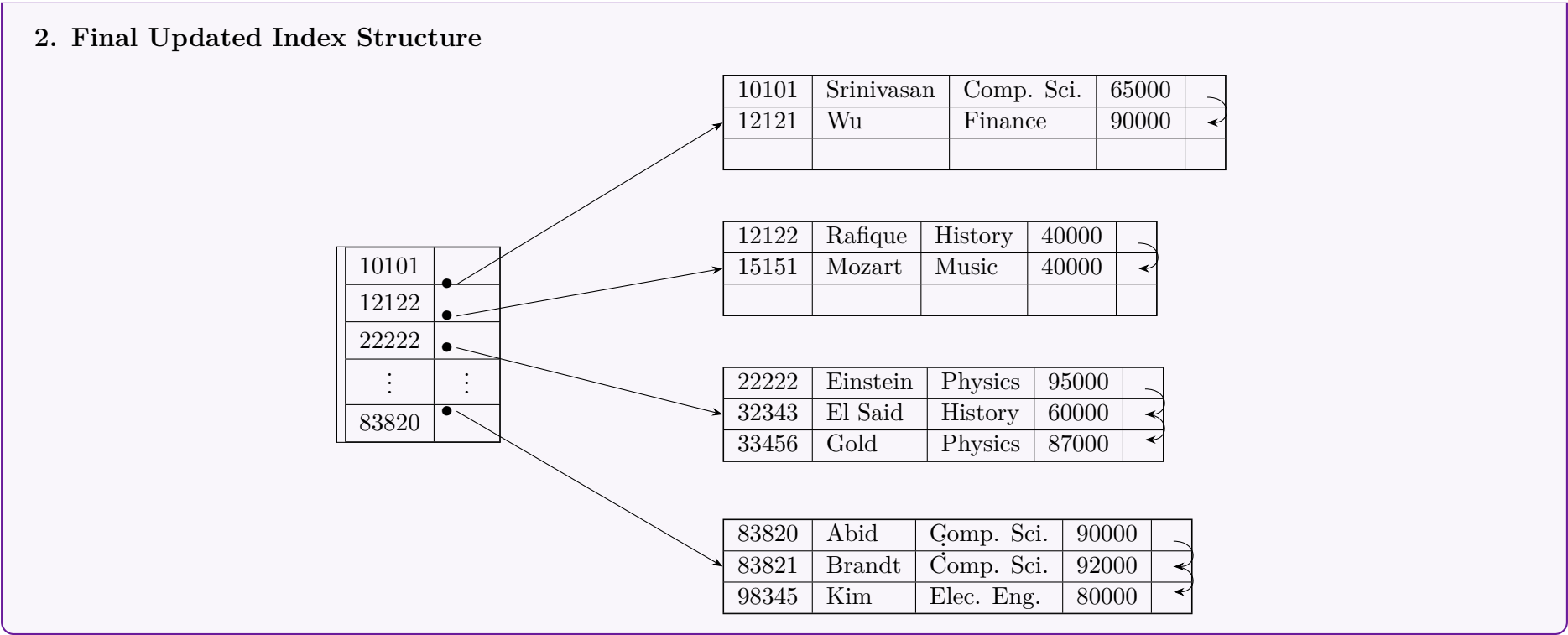
[CSE 215 | L-2/T-2 | 2020–21]

Answer

Working Principle for Primary Index Updates: A primary index is a sparse index where the data file is strictly sequentially ordered by the search key (ID). The index table stores only the anchor record (the first record) of each data block. Since no overflow blocks are allowed, insertions that overflow a block must be handled by block splitting, and index pointers must be updated when a block’s anchor record changes.

1. Step-by-Step Operations

- Insertion of Tuple 1:** (12122, Rafique, History, 40000)
 - Search:** The search key 12122 belongs in the first block (Block 0) because $10101 \leq 12122 < 22222$.
 - Action:** Block 0 currently contains 10101, 12121, and 15151. It is full (capacity = 3). Because overflow blocks are not allowed, **Block 0 is split**.
 - Result:** Records 10101 and 12121 remain in the original Block 0. The new record 12122 and the shifted record 15151 are placed in a newly allocated block. A new entry for the anchor **12122** is inserted into the primary index.
- Insertion of Tuple 2:** (83820, Abid, Comp. Sci., 90000)
 - Search:** The search key 83820 logically fits before 83821.
 - Action:** Instead of splitting the preceding block (which is full), we observe that the target block originally starting with 83821 has exactly one available slot. The record 83820 is inserted at the top of this block, shifting 83821 and 98345 down.
 - Result:** The new anchor record for this block becomes **83820**. The index table entry is updated from 83821 to 83820.



Q2. Compare primary index with secondary index showing representative index structures. (10 marks) [CSE 215 | L-2/T-2 | 2020–21]

Answer

Database indexing is a data structure technique used to quickly locate and access the data in a database. Two fundamental types of single-level indexes are the **Primary Index** and the **Secondary Index**.

1. Primary Index

- **Definition:** A primary index is an index specified on the **ordering key** field of an ordered file. The data file is physically sorted based on this search key (usually the primary key).
- **Density:** It is typically a **sparse index**. This means an index entry is not created for every search key value. Instead, one entry is created for the first record (the *anchor record*) of each disk block.
- **Efficiency:** Highly space-efficient because it requires fewer entries. However, inserting new records can be costly because maintaining the physical sequential order of the data file may require shifting records or managing overflow blocks.

2. Secondary Index

- **Definition:** A secondary index is an index specified on any **non-ordering field** of a file. The physical data file is not sorted by this index field.
- **Density:** It must be a **dense index**. Because the data is not physically ordered by the index key, the index must contain an entry for every single record in the data file (or a pointer to a bucket of pointers if the field is not a unique key) so that any record can be located.
- **Efficiency:** Search operations on non-primary fields are vastly accelerated. However, it consumes more storage space (due to being dense) and adds overhead during INSERT, UPDATE, and DELETE operations because every secondary index must be updated.

3. Comparison Table

Feature	Primary Index	Secondary Index
Ordering	Data file is physically ordered by the index key.	Data file is NOT ordered by the index key.
Density	Sparse: One entry per data block (anchor record).	Dense: One entry per data record.
Key Type	Built on the ordering key (usually Primary Key).	Built on any candidate key or non-key attribute.
Multiplicity	Only one primary index can exist per file.	Multiple secondary indexes can exist per file.
Pointers	Index pointer points to a data block .	Index pointer points directly to a specific record .
Storage Overhead	Low (fewer index entries).	High (an entry for every data record).

4. Structural Diagrams

A. Primary Index Structure (Sparse)

Notice that the index only contains entries for the first record of each sorted data block.

Primary Index
(Ordered, Sparse)

10	•
30	•
50	•

Data File on Disk
(Physically Sorted by Key)

10		...
20		...

30		...
40		...

50		...
60		...

B. Secondary Index Structure (Dense)

Notice that the index is sorted, but the data file is in arbitrary order. Every data record has a corresponding entry in the index.

Secondary Index
(Ordered, Dense)

11	•
22	•
33	•
44	•
55	•

Data File on Disk
(Unordered by this Key)

44		...
11		...

55		...
33		...

22		...
----	--	-----

Q3. Explain how the following queries will be executed using the sparse index shown in Figure 1:

(a) `SELECT * FROM instructor WHERE id = 22222`

(b) `SELECT * FROM instructor WHERE id = 99999`

Also explain the deletion of records from 10101 to 22222 from the instructor relation as per the deletion algorithm for a relation with a sparse index.

10101	
32343	
76766	

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

→ Sparse index and instructor relation

(15 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Working Principle of Sparse Index Lookup

A **sparse index** contains index entries for only some of the search-key values in the data file (typically the first record of each physical disk block, known as the anchor record). The data file must be physically ordered by the search-key field. To locate a record with search-key value K :

269

- (a) Find the largest index entry K_i such that $K_i \leq K$.
- (b) Follow the pointer of K_i to the corresponding record in the data file.
- (c) Scan sequentially along the data file pointers until the target record is found, or a record with a key value greater than K (or end-of-file) is encountered.

2. Query Execution Steps

(a) Execution of `SELECT * FROM instructor WHERE id = 22222`

- **Step 1 (Index Lookup):** The DBMS scans the index keys (10101, 32343, 76766) to find the largest key value less than or equal to 22222. The system identifies the entry **10101** since $10101 \leq 22222 < 32343$.
- **Step 2 (Block Pointer Following):** The system follows the index pointer linked with 10101, directing it to the record:
10101 | Srinivasan | Comp. Sci. | 65000.
- **Step 3 (Sequential Data Scan):** The system traverses through the data records sequentially via the block chain:
 - Record 1: 10101 → No match.
 - Record 2: 12121 → No match.
 - Record 3: 15151 → No match.
 - Record 4: 22222 → **Match found.**
- **Step 4 (Result):** The system stops traversing and returns the record for Einstein.

(b) Execution of `SELECT * FROM instructor WHERE id = 99999`

- **Step 1 (Index Lookup):** The largest key in the index less than or equal to 99999 is evaluated. Since all index keys are less than 99999, the maximum index value **76766** is selected.
- **Step 2 (Block Pointer Following):** The pointer for 76766 brings the search execution context directly to the record: **76766**
| Crick | Biology | 72000.
- **Step 3 (Sequential Data Scan):** The system performs a linear search from this point:
 - Record 10: 76766 → No match.
 - Record 11: 83821 → No match.
 - Record 12: 98345 → No match.
- **Step 4 (Termination):** The sequential pointer from record 98345 points to a **Ground/Null Pointer**. The scan terminates, and the system safely concludes that ID 99999 does not exist in the relation, returning an empty set.

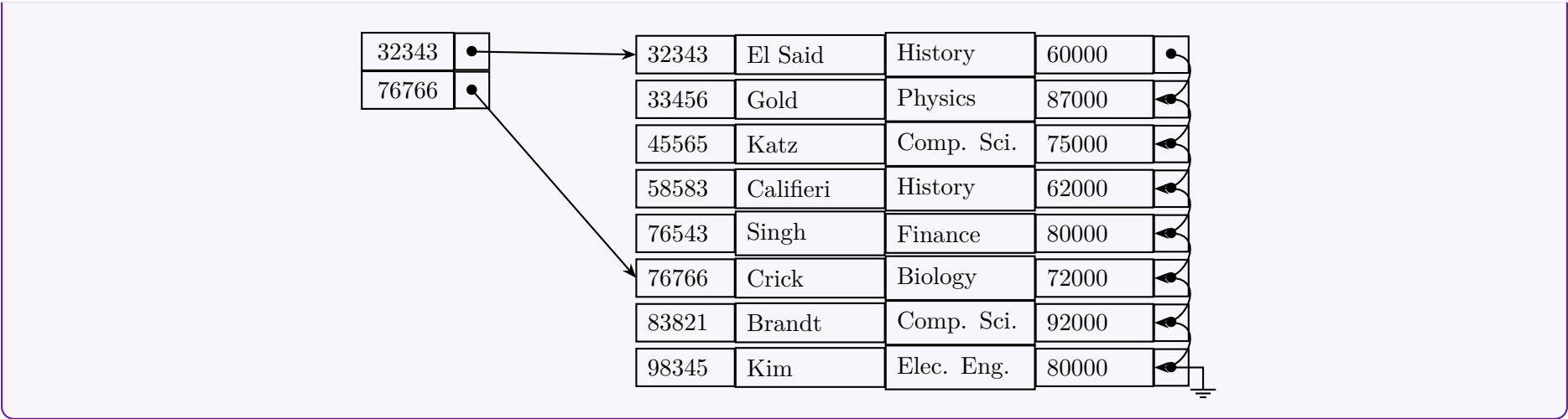
3. Deletion Algorithm and Structural Update

When deleting a range of records (from 10101 to 22222), the system follows specific algorithmic rules to maintain the sparse index integrity:

- (a) **Locate and Delete:** The records with keys 10101, 12121, 15151, and 22222 are physically removed from the storage block.
- (b) **Handle Index Anchors:** The value 10101 is currently an anchor in our sparse index structure.
- (c) **Index Reconstruction Principle:**
 - If the deleted record was an entry in the sparse index, the system attempts to replace it in the index with the **next sequential record** present in the data file.
 - Following the physical deletion of 10101 through 22222, the immediate next physical record in the data file sequence is **32343**.
 - The algorithm checks if **32343** is already explicitly present in the index. Since **32343** is already an index entry, creating another entry for it would produce a redundant duplicate index reference.
 - Therefore, the index entry for 10101 is simply **deleted** without a replacement.

Post-Deletion State Structure Diagram

Below is the updated design of the sparse index and relation file following the complete execution of the deletion sequence:



Q4. Consider a dictionary where each page contains only a single word along with its description. If we take the word from each page and form an index based on those words, what type of indexing (Dense/Sparse) would it be? Explain your answer. **(5 marks)** [CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Core Verdict

An index constructed by taking the single word from each page of this specialized dictionary satisfies the definitions of **both a Dense Index and a Sparse Index** simultaneously. Under standard database management system (DBMS) terminologies, it represents the exact boundary condition where a sparse index structurally converges with a dense index. However, from a foundational classification perspective, it is best classified as a **Dense Index** based on logical entry density, while serving physically as a **Primary Sparse Index**.

2. Detailed Theoretical Explanation

To understand this dual nature, we must analyze the scenario using the definitive criteria for both indexing types:

- Dense Index Criterion:** An index is dense if an index entry appears for **every search-key value** in the data file.
 - *Application to Scenario:* Since each page contains exactly one word (the search key), extracting the word from every page means the index will contain an entry for **every single search key** present in the entire dictionary. No search key is omitted. Therefore, it satisfies the literal definition of a **Dense Index**.
- Sparse Index Criterion:** An index is sparse if an index entry appears for only **some of the search-key values** in the data file, typically corresponding to the first record of each physical disk block (known as a block anchor).
 - *Application to Scenario:* In physical database storage, a dictionary page maps directly to a **disk block**. Since the index contains exactly one entry per block (page), it perfectly follows the structural design and storage mechanism of a **Sparse Index**.

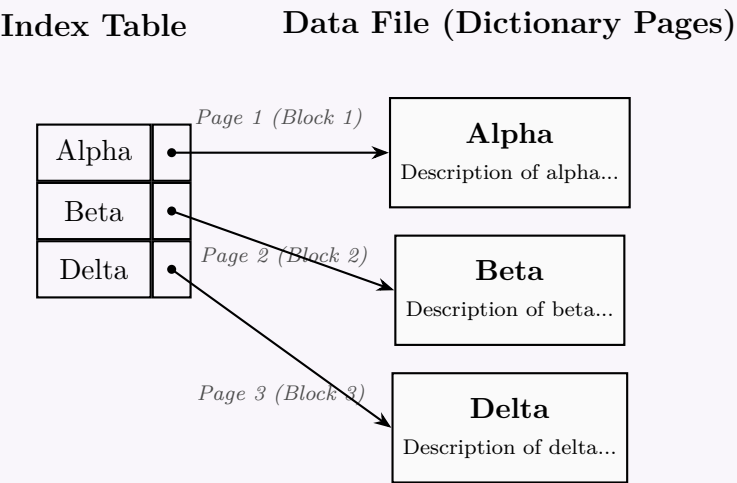
3. Comparative Structural Analysis

The table below outlines how this unique dictionary scenario compares against classic dense and sparse indexing properties:

Property	Standard Dense Index	Standard Sparse Index	Our Dictionary Index
Entry Density	One entry per data record.	One entry per data block.	One entry per block <i>and</i> per record.
File Ordering	Not required to be ordered.	Data file must be sequentially ordered.	Data file is ordered sequentially.
Search Overhead	Fast pointer lookup directly to the exact record.	Pointer lookup to block, followed by sequential block scan.	Fast pointer lookup directly to the page; zero scanning needed.
Storage Size	Large (proportional to total records).	Small (proportional to total blocks).	Identical to block count (highly optimized).

4. Conceptual Structural Diagram

The visual representation below illustrates this convergence. Because the block capacity (B_f) is exactly 1 record per block, the mapping from the index to the data records is fully bijective.



5. Summary Conclusion

- **Logically**, it is a **Dense Index** because the mapping covers 100% of the search-key values.
- **Physically**, it is a **Sparse Index** because it leverages the natural clustering of records into pages (blocks), containing exactly one pointer per page.
- **Performance Note:** This represents the most ideal configuration for a primary index, combining the ultra-low lookup latency of a dense structure (no sequential linear search within the block is required) with the block-aligned storage efficiency of a sparse structure.

- Q5. Create the following index structures for the given student relation:
- (a) A sparse index on `student_id` for search key values {1204001, 1205001, 1205010}.
 - (b) A secondary index on `department_code`.
 - (c) Delete the record with `student_id` = 1205010 and show the resulting index structures.
 - (d) Insert the record {1205011, N114, EEE, 3, 1} and show the resulting index structures.

Record 0	1205001	N11	CSE	3	1
Record 1	1205004	N12	CSE	3	1
Record 2	1205007	N13	CSE	3	1
Record 3	1205010	N14	CSE	3	1
Record 4	1205013	N15	CSE	3	1
Record 5	1205016	N16	CSE	3	1
Record 6	1204001	N21	EEE	3	1
Record 7	1204002	N22	EEE	3	1
Record 8	1204003	N23	EEE	3	1

Figure 1: File containing records of relation *student(student id, name, department code, level, term)*

(20 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

Preliminary Analysis & Initial File Organization

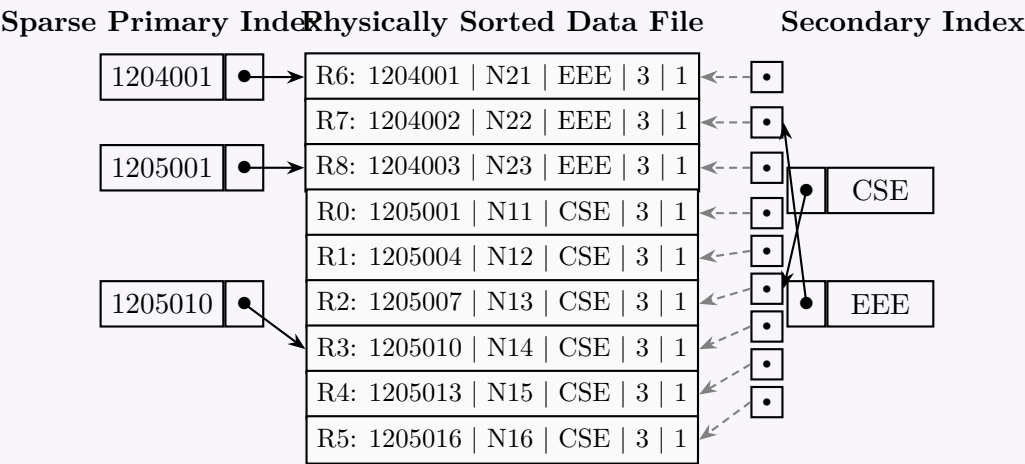
A **sparse index** is constructed on an ordering key attribute, meaning the physical data file must be sequentially sorted based on that search key (`student_id`). Analyzing the provided *student* relation, the records are currently unsorted. Therefore, before creating the index structures, the data file must be physically reorganized (sorted) by `student_id` in ascending order.

Sorted Base Relation Table

Record Label	student_id	name	department_code	level	term
Record 6	1204001	N21	EEE	3	1
Record 7	1204002	N22	EEE	3	1
Record 8	1204003	N23	EEE	3	1
Record 0	1205001	N11	CSE	3	1
Record 1	1205004	N12	CSE	3	1
Record 2	1205007	N13	CSE	3	1
Record 3	1205010	N14	CSE	3	1
Record 4	1205013	N15	CSE	3	1
Record 5	1205016	N16	CSE	3	1

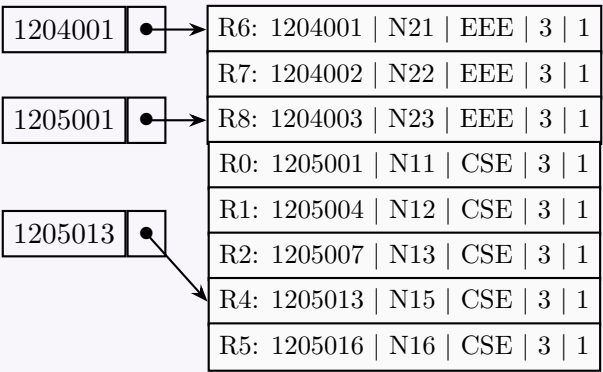
(a) & (b) Initial Index Structures (Sparse and Secondary)

- Sparse Primary Index:** Created on the ordering field `student_id`. For the specified search keys {1204001, 1205001, 1205010}, pointers map directly to the corresponding records in the ordered data file.
- Secondary Index:** Created on the non-ordering field `department_code`. Because the attribute values are distinct groups (CSE and EEE), a dense secondary index structure with an intermediate block of pointers (or bucket pointer layout) is utilized to map each unique key to all matching data record slots.



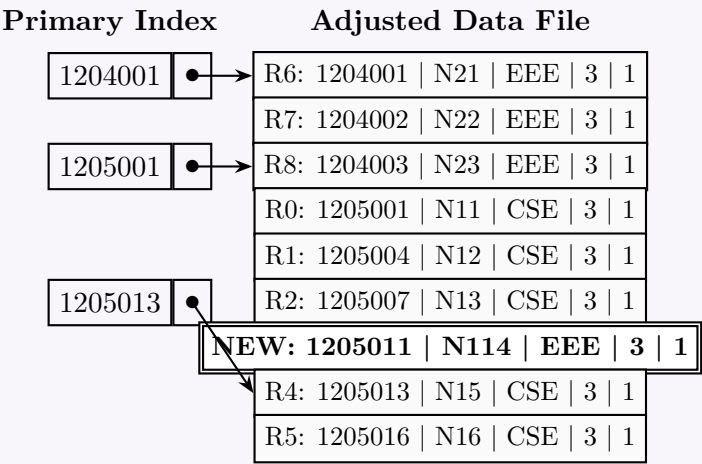
(c) Post Deletion State: Deleting Record 1205010 (R3)

- (a) **Data File Adjustment:** Record 3 (1205010) is removed. The subsequent records (1205013, 1205016) shift upward physically to keep the file contiguous.
- (b) **Primary Index Adjustment:** The index key 1205010 was a designated search key anchor. According to the sparse index deletion rule, because this exact record is removed, its index entry must be updated to reference the next sequential available value, which is 1205013.
- (c) **Secondary Index Adjustment:** The corresponding entry in the pointer bucket referencing the deleted memory block is updated/removed.



(d) Post Insertion State: Inserting Record {1205011, N114, EEE, 3, 1}

- (a) **Data File Placement:** To preserve the sequential physical arrangement based on `student_id`, the new record must be inserted right after record 1205007 (R2) and before 1205013 (R4). This causes downstream records to shift downwards.
- (b) **Primary Index Impact:** The sparse index entries remain fixed on keys {1204001, 1205001, 1205013}. The new record does not alter any block anchor boundaries; hence no new sparse index entries are created.
- (c) **Secondary Index Impact:** The record belongs to department EEE. A pointer link referencing this new slot is appended to the EEE reference collection block.



Q6. Explain the factors by which an indexing technique can be evaluated. (5 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Introduction

In Database Management Systems (DBMS), an **indexing technique** is an auxiliary data structure deployed to optimize query performance and expedite data retrieval operations. However, selecting or designing an optimal index involves multiple trade-offs. No single indexing technique is ideal for all applications. Indexing techniques are evaluated across several distinct operational and physical performance metrics.

2. Key Evaluation Factors

An indexing technique is primarily evaluated based on the following six metrics:

(a) Access Type:

This refers to the specific types of search patterns and query predicates that the index data structure can efficiently support.

- Point Queries (Exact Match):* Finding records with a specific attribute value (e.g., $ID = 101$). Hash indexes excel at this, offering $O(1)$ average time complexity.
- Range Queries:* Finding records within a specified lower and upper bound (e.g., $20000 \leq \text{Salary} \leq 50000$). Ordered index structures like B^+ trees are highly efficient for range queries due to their linked leaf nodes, whereas hash indexes cannot support them.

(b) Access Time:

The time required to locate a specific data item or a set of items during a search operation using the index. This includes:

- The time spent traversing the internal structures of the index (e.g., root-to-leaf path in a tree).
- The number of disk I/O operations needed to fetch the index blocks and the actual data blocks.

(c) Insertion Time:

The time overhead involved in adding a new record to the database and updating the index structure accordingly. For example, inserting into a B^+ tree may require node splits, key redistribution, and tree rebalancing, which increases the write latency compared to simple heap file insertions.

(d) Deletion Time:

The time required to remove a record from the database and clear or modify its corresponding reference entries within the index. Similar to insertion, deletion can cause structural shifts such as node underflows, merging, or contraction in tree-based structures.

(e) Space Overhead:

The additional storage memory required by the index data structure itself on disk or in main memory. While adding multiple indexes accelerates search operations, it creates substantial space overhead. The architecture must balance index sizes (e.g., dense vs. sparse indexing architectures) to avoid choking storage systems.

(f) Update Cost:

The cost associated with modifying an existing record's attribute value when that attribute serves as an index search key. This operation requires a structural deletion of the old reference and a re-insertion of the updated reference into its correct logical position within the index.

3. Comparative Matrix of Standard Indexing Architectures

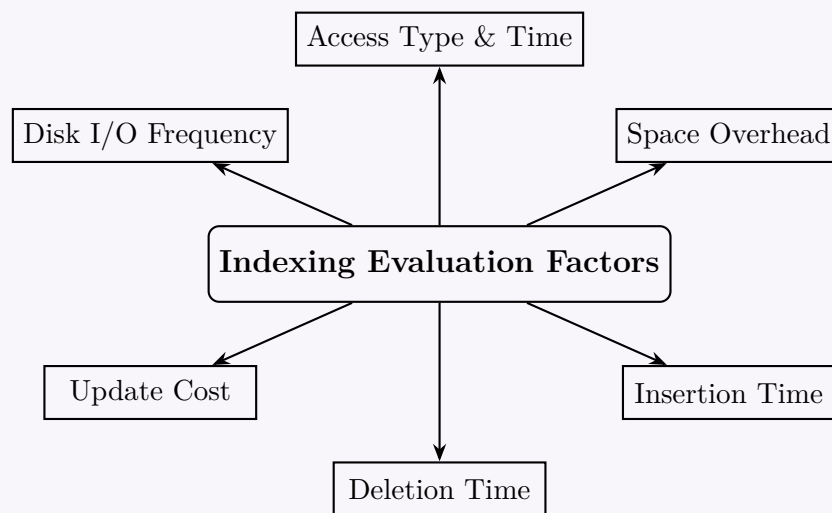
The following professional table summarizes how standard database indexing mechanisms perform when assessed against these key evaluation criteria:

Evaluation Factor	Ordered Indices (B^+ Tree)	Hash-Based Indices
Primary Access Type	Ideal for both point queries and range selections.	Strictly optimized for point/exact match queries.
Access Time	Logarithmic time, $O(\log_B N)$, where B is node fan-out.	Near-instantaneous, $O(1)$ average time complexity.
Space Overhead	Low to moderate; nodes dynamically expand/shrink.	Varies; can be high if bucket utilization is low or overflow chains form.
Insertion/Deletion	Requires balance maintenance (node split/merge).	Requires hash bucket allocation and overflow chain management.

4. Evaluation Trade-off Visual Model

The choice of an indexing technique represents a fundamental architectural trade-off between retrieval speed, update overhead, and spatial consumption.

274



Secondary & Bitmap Index

Q1. “Without bitmap index scan strategy, secondary index scan can sometimes perform worse than linear file scan.” — Justify or criticize this statement with proper explanation. **(15 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Justification of the Statement

The statement is **entirely correct and justified**. In a database system, a secondary index is **non-clustered**, meaning the physical arrangement of records in the data file does not match the logical order of keys in the index. Without advanced execution strategies like a bitmap index scan, a traditional secondary index scan can degrade to a state where it is significantly slower than a simple sequential (linear) file scan. This phenomenon is commonly known as the **Clustering Factor Problem** or the **Random I/O Penalty**.

2. Core Technical Mechanism

Traditional Secondary Index Scan (Naive Fetch)

When a query processor uses a secondary index traditionally, it performs the following steps:

- Traverses the index structure (e.g., B^+ tree) to locate pointers (Row IDs / Tuple IDs) matching the query predicate.
- For each matching pointer, it immediately retrieves the corresponding data block from disk.

Because the data file is unclustered relative to this index, consecutive entries in the index can point to entirely different physical data blocks on disk. If a query matches a significant number of records, this approach induces severe **random I/O overhead**. In the worst case, reading N matching records can result in fetching N separate disk blocks, often reading the exact same disk block multiple times into the buffer pool asynchronously.

Linear File Scan (Sequential Scan)

In contrast, a linear file scan reads the data blocks sequentially from start to finish:

- It utilizes highly optimized **sequential I/O**, which is orders of magnitude faster than random I/O on mechanical storage and still considerably faster on modern solid-state drives (SSDs) due to hardware prefetching and block-level parallel reads.
- It reads each data block exactly **once**.

3. Cost Modeling and Selection Threshold

Let B be the total number of blocks in the data file, N be the total number of records matching the query selection criteria, and $C_{\text{sequential}}$ and C_{random} be the respective costs of sequential and random disk block access ($C_{\text{random}} \gg C_{\text{sequential}}$).

$$\begin{aligned} \text{Cost of Linear Scan} &\approx B \times C_{\text{sequential}} \\ \text{Cost of Traditional Secondary Index Scan} &\approx H_{\text{index}} + N \times C_{\text{random}} \end{aligned}$$

Where H_{index} is the height/traversal cost of the index. If the selectivity of a query is high enough such that:

$$N \times C_{\text{random}} > B \times C_{\text{sequential}}$$

the secondary index scan becomes slower than a sequential scan. This inflection point is called the **index select threshold**, often occurring when a query retrieves as little as 5% to 15% of the total rows in a table.

4. Mitigation via Bitmap Index Scan Strategy

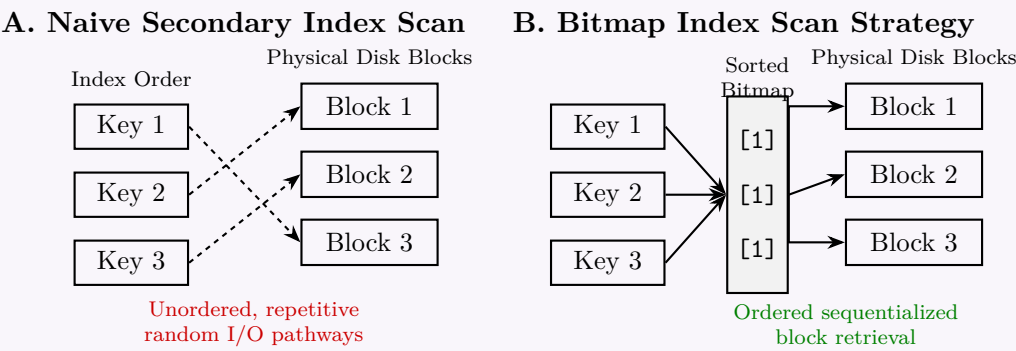
To prevent query optimizers from abandoning secondary indexes for moderately selective queries, modern engines use a **Bitmap Index Scan** strategy. It decouples index traversal from record fetching using a two-stage mechanism:

- Bitmap Construction:** The index is scanned to collect the physical addresses of all matching records. Instead of fetching the rows immediately, the engine constructs a **bitmap** in memory where each bit represents a physical data block or page. If a page contains a matching record, its corresponding bit is set to 1.

(b) **Bitmap Heap Scan:** The engine reads the memory bitmap sequentially. It visits only the data blocks whose bits are set to 1, reading them in their **physical sequence on disk**.

5. Structural Comparison Diagram

The visual model below highlights why a naive secondary index scan causes thrashing via chaotic random access lines, whereas the bitmap strategy organizes access into linear, ordered physical operations.



6. Comparative Summary

The table below details the performance characteristics across different operational paradigms when handling intermediate to high row selectivity:

Metric	Linear File Scan	Naive Secondary Index Scan	Bitmap Index Scan
I/O Type	Purely Sequential	Highly Random	Ordered / Block-sequential
Block Read Frequency	Exactly once per block	Multiple times per block (worst-case)	Max once per matched block
Memory Footprint	Negligible (1 block buffer)	Low (1 index page + 1 data page)	Moderate (requires in-memory bitmap)
Optimal Selection	High selectivity (> 25%)	Ultra-low selectivity (< 2%)	Moderate selectivity (5% – 25%)

Q2. The student relation has schema Student(Id, name, f_name, m_name, cgpa, year_admit, division, district, thana, street, house_number, city, NID, DOB, total_credit). B⁺ tree secondary indices exist on name, city, and year_admit with heights 4, 2 and 1 respectively. The number of tuples for name = 'Abid', City = 'Feni', and year_admit = 2015 are 50, 20 and 1000 respectively. Seek cost: 10 ms, block transfer cost: 0.1 ms. Number of blocks of student relation: 2000.

(a) Find the worst-case costs of processing the following conjunctive selection query using the single index method:

SELECT * FROM student WHERE name = 'Abid' AND City = 'Feni' AND year_admit = 2015

(b) Explain clearly how this query will be executed in this method.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Problem Parameters and Assumptions

To determine the worst-case cost of processing the conjunctive selection query using the **single index method**, we first define the system parameters:

- **Seek cost (t_s):** 10 ms
- **Block transfer cost (t_t):** 0.1 ms
- **Single Block Access Cost (t_A):** $t_s + t_t = 10 + 0.1 = 10.1$ ms
- **Number of blocks in data file (b):** 2000

Since these are **secondary indices**, the main data file is *not* physically clustered on any of these search keys. Therefore, in the **worst-case scenario**, every matching tuple resides in a distinct physical disk block, requiring a separate block access (seek + transfer) for each tuple.

2. Part (a): Worst-Case Cost Calculation

The **single index method** for a conjunctive query involves evaluating the cost of using each available index individually. The DBMS Query Optimizer estimates the cost for each index and selects the one with the minimum cost. The worst-case time cost C_i for a secondary index I_i of height h_i matching n_i tuples is calculated as:

$$C_i = (\text{Index Block Accesses} + \text{Data Block Accesses}) \times t_A$$
$$C_i = ((h_i + 1) + n_i) \times 10.1 \text{ ms}$$

Note: $(h_i + 1)$ accounts for traversing the tree from the root to the leaf node.

Cost Evaluation for Each Candidate Index(a) **Index 1: name** ($h_1 = 4$, $n_1 = 50$ tuples)

$$C_{\text{name}} = ((4 + 1) + 50) \times 10.1 \text{ ms} \\ = (5 + 50) \times 10.1 \text{ ms} = 55 \times 10.1 = \mathbf{555.5 \text{ ms}}$$

(b) **Index 2: city** ($h_2 = 2$, $n_2 = 20$ tuples)

$$C_{\text{city}} = ((2 + 1) + 20) \times 10.1 \text{ ms} \\ = (3 + 20) \times 10.1 \text{ ms} = 23 \times 10.1 = \mathbf{232.3 \text{ ms}}$$

(c) **Index 3: year_admit** ($h_3 = 1$, $n_3 = 1000$ tuples)

$$C_{\text{year}} = ((1 + 1) + 1000) \times 10.1 \text{ ms} \\ = (2 + 1000) \times 10.1 \text{ ms} = 1002 \times 10.1 = \mathbf{10120.2 \text{ ms}}$$

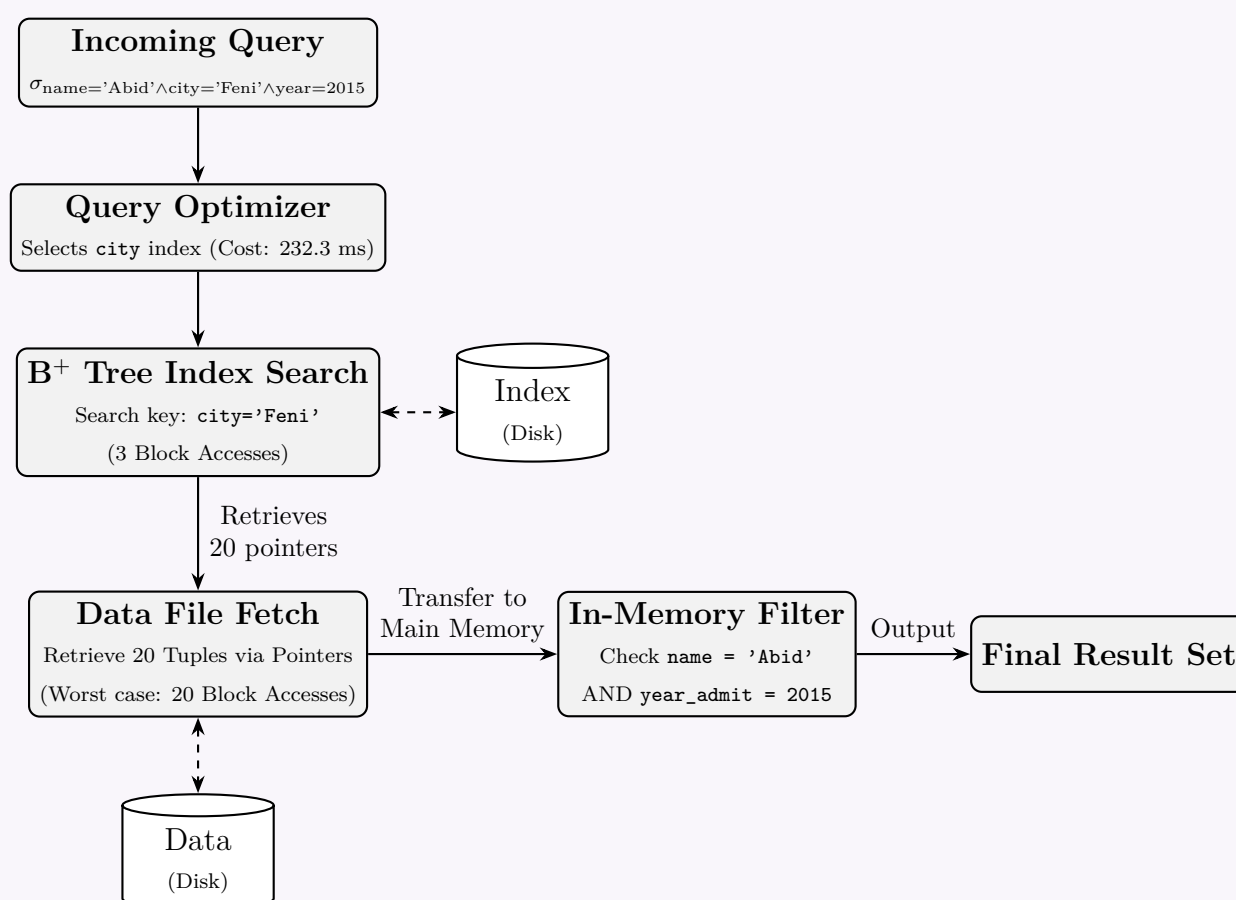
Conclusion for Part (a): The query optimizer will compare these costs and select the index with the lowest cost. Therefore, the worst-case cost of processing this query using the single index method is **232.3 ms** (using the **city** index).

3. Part (b): Query Execution Plan (Single Index Method)

Under the single index method, the execution proceeds as follows:

- (a) **Cost Optimization:** The query optimizer identifies that using the secondary index on **city** yields the lowest I/O cost (232.3 ms) because it has the highest selectivity (fewest matching tuples) among the available indices.
- (b) **Index Traversal:** The DBMS traverses the B⁺ tree secondary index for **city**. It starts at the root and descends to the leaf node corresponding to the search key 'Feni'. This requires 3 disk block reads.
- (c) **Pointer Retrieval:** From the leaf node, the system retrieves the 20 tuple pointers (Record IDs) associated with **City** = 'Feni'.
- (d) **Data Fetching:** The DBMS uses these pointers to fetch the actual records from the **Student** data file on disk. In the worst case, this requires 20 separate random block accesses.
- (e) **In-Memory Filtering:** As each of the 20 tuples is brought into main memory, the system applies the remaining conjunctive predicates:

$$(\text{name} = \text{'Abid'}) \text{ AND } (\text{year_admit} = 2015)$$
- (f) **Result Generation:** Tuples that satisfy all the remaining conditions are appended to the query result set and returned to the user. Non-matching tuples are discarded from the memory buffer.

Execution Architecture Diagram

Q3. A relation schema $R(A, B, C)$ and a relation $r(R)$ is given below. The relation is physically sorted on the primary key A . Construct

an ordered secondary index of r on attribute B considering the buckets containing maximum 3 pointers. Explain the cases if you need to insert the tuple $t = \langle a_8, b_2, c_2 \rangle$.

A	B	C
a_1	b_1	c_1
a_2	b_2	c_1
a_3	b_3	c_1
a_4	b_1	c_1
a_5	b_2	c_1
a_7	b_2	c_1
a_8	b_3	c_1

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Ordered Secondary Index Structure

A secondary index on a non-key attribute (attribute B) must account for the fact that the primary data file is physically sorted by a different attribute (the primary key A). Because records with the same B value are scattered across the data file, the index uses an intermediate level of indirection called **pointer buckets**.

- Index File:** Ordered sequentially by the search key B (b_1, b_2, b_3). Each entry points to a bucket.
- Pointer Buckets:** Store the actual pointers (or primary keys) to the corresponding data records. The question specifies a maximum capacity of **3 pointers per bucket**.
- Data File:** Remains physically sorted by primary key A .

Index File

Pointer Buckets

Data File (Sorted by A)

b_1

•

•

empty

a_1, b_1, c_1

a_2, b_2, c_1

a_3, b_3, c_1

b_2

•

•

•

a_4, b_1, c_1

a_5, b_2, c_1

a_7, b_2, c_1

b_3

•

•

empty

a_8, b_3, c_1

2. Case Analysis: Inserting Tuple $t = \langle a_8, b_2, c_2 \rangle$

When evaluating the instruction to insert the tuple $t = \langle a_8, b_2, c_2 \rangle$, we must consider the integrity rules of the DBMS. The analysis results in two distinct operational cases based on the system's intent:

Case A: Primary Key Constraint Violation (Direct Insertion)

- Issue:** Attribute A is the primary key for relation $r(R)$. The data file already contains a tuple with the primary key a_8 ($\langle a_8, b_3, c_1 \rangle$).
- Result:** If this operation is treated strictly as an **INSERT**, the DBMS will instantly **reject the operation** due to an Entity Integrity (Primary Key) violation. The secondary index remains completely unaffected.

Case B: Update Operation & Bucket Overflow Handling

If the operation is intended as an **UPDATE** to modify the existing tuple a_8 into $\langle a_8, b_2, c_2 \rangle$, the following structural modifications occur:

(a) Data File Update:

The tuple at a_8 is updated in-place from $\langle a_8, b_3, c_1 \rangle$ to $\langle a_8, b_2, c_2 \rangle$.

(b) Pointer Removal from b_3 :

The pointer corresponding to a_8 is deleted from the pointer bucket for b_3 . The bucket for b_3 now contains only 1 active pointer (a_3) and 2 empty slots.

(c) Pointer Addition & Bucket Overflow for b_2 :

The DBMS must insert a new pointer for a_8 into the bucket corresponding to b_2 .

278

- (d) **Overflow Resolution:** As shown in the diagram, the bucket for b_2 already contains 3 pointers (a_2, a_5, a_7). Because the maximum bucket capacity is strictly 3, it is full. The system must allocate an **overflow bucket**, establish a forward pointer from the primary b_2 bucket to the newly allocated overflow bucket, and store the pointer for a_8 in this new block.

Q4. The Employee Management System has schema `employee(e_id, name, date_of_birth, salary, street, thana, district, NID)`. Total employees: 40,000; relation size: 4,000 blocks. Primary index: `e_id`; secondary indices: `name`, `district`, `NID` with heights 3, 2, 4 respectively. Transfer time: 0.5 ms/block; average seek time: 8 ms. Tuples with `salary = 50000`: 100; `district = 'Comilla'`: 2000 (each tuple in a different block). Find the query costs using indices for:

SELECT * FROM employee WHERE salary = 50000 OR district = 'Comilla'

(8 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Theoretical Analysis of Disjunctive (OR) Selection

For a disjunctive query containing an OR operator:

$$\sigma_{\theta_1 \vee \theta_2}(r)$$

an indexing strategy can only avoid a full table scan if **both** conditions (θ_1 and θ_2) can be evaluated via indices.

- Condition 1 (θ_1): `district = 'Comilla'` → Supported by a secondary index (height = 2).
- Condition 2 (θ_2): `salary = 50000` → **No index exists** on the `salary` attribute.

Because `salary` is unindexed, any plan that solely uses the `district` index will fail to retrieve employees who satisfy `salary = 50000` but live in a different district. Thus, a linear scan of the entire relation is necessary. We evaluate the costs of the two possible execution strategies below.

2. Cost Calculations

Let the primary parameters be:

- Seek time (t_s) = 8 ms
- Transfer time (t_t) = 0.5 ms/block
- Block access cost for random I/O ($t_s + t_t$) = 8 + 0.5 = 8.5 ms

Strategy 1: Linear File Scan (Full Table Scan)

A linear scan scans all $b = 4,000$ blocks sequentially. It incurs one initial seek, and then transfers all blocks sequentially. During the scan, both criteria (`salary` and `district`) are checked simultaneously in main memory.

$$\begin{aligned} \text{Cost}_{\text{Linear Scan}} &= t_s + b \times t_t \\ &= 8 \text{ ms} + (4000 \times 0.5 \text{ ms}) \\ &= 8 \text{ ms} + 2000 \text{ ms} = \mathbf{2008 \text{ ms}} \end{aligned}$$

Strategy 2: Naive Index Scan followed by Linear Scan

If the system naively attempts to utilize the `district` index first to locate the 2,000 tuples where `district = 'Comilla'`, it must still perform a full linear scan afterward to fetch the unindexed `salary` records.

- (a) **Index Look-up for `district`:** Traversing from the root to the leaf node takes $h = 2$ block transfers and seeks.

$$\text{Cost}_{\text{index lookup}} = 2 \times (t_s + t_t) = 2 \times 8.5 = 17 \text{ ms}$$

- (b) **Data Fetching for `district`:** There are 2,000 matching tuples, each residing in a different block. This requires 2,000 random block accesses.

$$\text{Cost}_{\text{data fetch}} = 2000 \times (t_s + t_t) = 2000 \times 8.5 = 17000 \text{ ms}$$

- (c) **Residual Scan for `salary`:** A full linear file scan is executed to find the remaining `salary = 50000` records.

$$\text{Cost}_{\text{residual scan}} = 2008 \text{ ms}$$

$$\text{Total Cost}_{\text{Naive Index Plan}} = 17 \text{ ms} + 17000 \text{ ms} + 2008 \text{ ms} = \mathbf{19025 \text{ ms}}$$

3. Summary of Comparison

Evaluation Strategy	Operational Profile	Total Estimated Cost
Linear File Scan	Prefers sequential block reads; evaluates all conditions at once.	2,008 ms (≈ 2.01 s)
Index-Assisted Plan	Plagued by excessive random I/O and redundant scans.	19,025 ms (≈ 19.03 s)

Conclusion: The query optimizer will choose the **Linear File Scan** strategy because it is significantly faster than using the available secondary index. The total cost to process this query using the optimal design strategy is **2008 ms**.

Q5. Give an example where the Bit-map index is preferred over other indexing techniques. (5 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Overview of Bitmap Indexing

A **Bitmap Index** is a specialized database indexing technique that uses bit arrays (bitmaps) to represent the existence of a value for a specific record. It is highly optimized for answering queries by performing extremely fast, CPU-level bitwise logical operations (such as **AND**, **OR**, and **NOT**).

2. Optimal Use Case Characteristics

Bitmap indices are universally preferred over traditional indexing structures (like B⁺-trees or Hash indices) when the database exhibits the following characteristics:

- **Low Cardinality Attributes:** The column has a very small number of distinct values relative to the total number of rows (e.g., *Gender*, *Marital Status*, *Boolean flags*, *Region codes*).
- **Data Warehousing (OLAP):** The environment is read-heavy with complex, multi-attribute ad-hoc queries, and updates/inserts are infrequent (batch processed).
- **Complex Conjunctive/Disjunctive Predicates:** Queries frequently use multiple ‘AND’ and ‘OR’ conditions filtering on these low-cardinality columns.

3. Concrete Example: Demographic Data in a Data Warehouse

Consider a **Customer** table in a data warehouse with millions of records. Two of its columns are **Gender** (with values ‘M’, ‘F’) and **Income_Level** (with values ‘L’ for Low, ‘M’ for Medium, ‘H’ for High).

Base Relation *r*(Customer)

Assume a simplified view of the first 5 records:

Row ID	Customer_Name	Gender	Income_Level
1	Alice	F	L
2	Bob	M	M
3	Charlie	M	H
4	Diana	F	H
5	Eve	M	H

Bitmap Index Construction

Instead of storing Row IDs, the DBMS creates a bit vector for *each distinct value* of the indexed columns. A ‘1’ indicates the record has that value, and ‘0’ indicates it does not.

- **Bitmaps for Gender:**
 - Gender_M = [0, 1, 1, 0, 1]
 - Gender_F = [1, 0, 0, 1, 0]
- **Bitmaps for Income_Level:**
 - Income_L = [1, 0, 0, 0, 0]
 - Income_M = [0, 1, 0, 0, 0]
 - Income_H = [0, 0, 1, 1, 1]

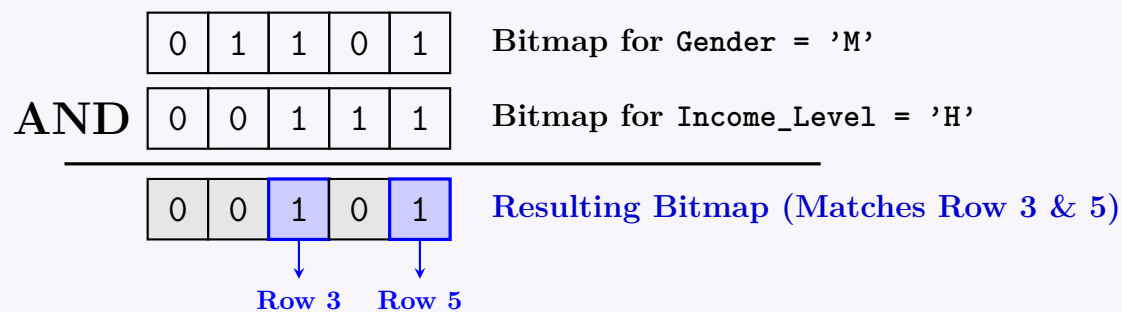
Query Execution

Suppose an analyst runs the following query to find High-Income Males:

```
SELECT Customer_Name FROM Customer
WHERE Gender = 'M' AND Income_Level = 'H';
```


4. Visualizing the Bitwise Execution

The DBMS resolves this query entirely in memory by fetching the two relevant bitmaps and applying a CPU-native **Bitwise AND** operation.



5. Why is this preferred over a B⁺-Tree in this scenario?

- **Performance Degradation in B⁺-Trees:** If a B⁺-tree were used on the **Gender** column, a search for 'M' would return approximately 50% of the row IDs in the entire database. Intersecting massive lists of numerical row IDs takes substantial memory and processor time.
- **Storage Efficiency:** Because the cardinality is low, bitmaps can be highly compressed (e.g., using Run-Length Encoding). A compressed bitmap takes a fraction of the space of a B⁺-tree structure.
- **Hardware Acceleration:** Bitwise logical operators (AND, OR, XOR) are supported directly by the Arithmetic Logic Unit (ALU) of the CPU, processing 64 bits (or 256 bits with SIMD instructions) in a single clock cycle, making it orders of magnitude faster than evaluating branch logic in a tree structure.

Q6. Is it possible to implement an unclustered-sparse indexing scheme? If so, give an example. Otherwise, briefly explain the reason. **(6 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Direct Answer

No, it is fundamentally impossible to implement an unclustered-sparse indexing scheme in a relational database management system. An unclustered (secondary) index **must always be a dense index**.

2. Theoretical Explanation

To understand why this combination is impossible, we must examine the operational dependencies of sparse indexing and unclustered data storage:

- **Sparse Index Working Principle:** A sparse index only stores index entries for a subset of the search key values (typically the first record of each data block, known as the anchor record). To locate a record whose key is *not* explicitly in the index, the DBMS finds the largest index entry less than or equal to the search key, jumps to that block in the data file, and performs a **sequential forward scan** until the target record is found or a larger key is encountered (indicating the record does not exist).
- **Unclustered (Secondary) File Organization:** In an unclustered index, the physical records in the data file are ordered by a *different* attribute than the search key (e.g., sorted by ID, but indexed by **Age**). Consequently, with respect to the index's search key, the data records are scattered completely at random.

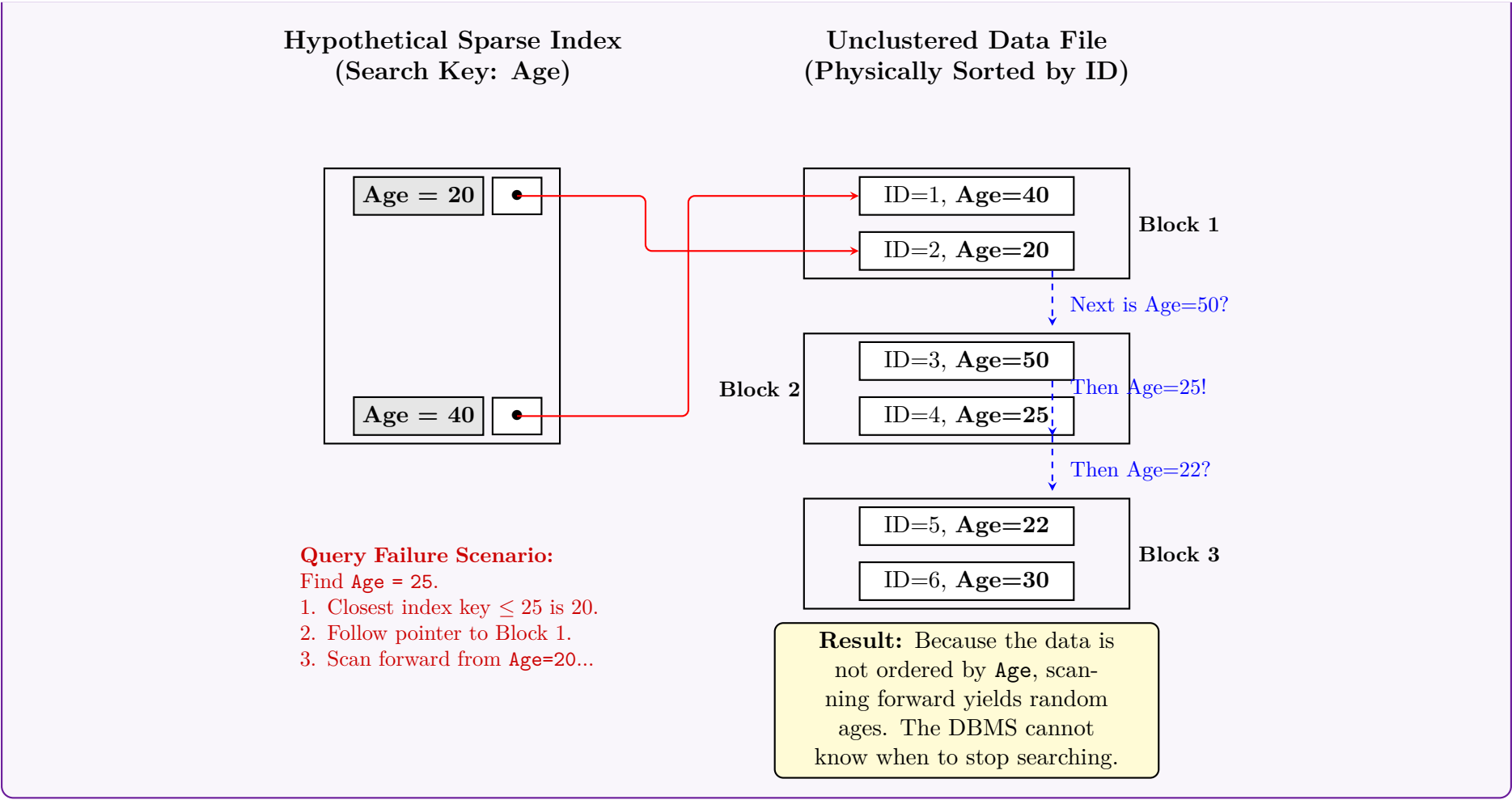
The Contradiction

If a sparse index were applied to an unclustered data file, the system would fail during query execution. When searching for a target key not present in the sparse index, the system would jump to the nearest lower key's block and scan forward. Because the file is not physically sorted by the search key, the subsequent records will have random, non-sequential key values. The DBMS would have no predictable stopping condition and would be forced to scan the entire remainder of the database file, entirely defeating the purpose of the index.

Therefore, for an unclustered index to function, it **must** contain an entry for every single record or distinct value in the database (i.e., it must be a **dense index**).

3. Visual Proof of Failure (Why it does not work)

The following diagram illustrates a hypothetical attempt to create a sparse index on an unclustered attribute (**Age**) and demonstrates why the search logic breaks down.



The result indicates that only Rows 1 and 5 satisfy both conditions.

5. Properties and Characteristics

- **Low Cardinality Ideal:** Highly efficient for attributes with a small number of distinct values (e.g., gender, marital status, boolean flags).
- **Sparsity and Compression:** For higher cardinalities, bitmaps become extremely sparse (mostly zeroes). Modern DBMS uses Run-Length Encoding (RLE) to compress these sparse bitmaps, saving significant storage.

6. Advantages vs. Disadvantages

Advantages	Disadvantages
• Extremely fast evaluation for multiple AND, OR, NOT conditions. • Highly space-efficient for low-cardinality columns. • Ideal for Data Warehousing (OLAP) environments with complex read-heavy analytical queries.	• Terrible performance for high-cardinality data (like SSN, Primary Keys) without compression. • High update overhead. Updating a single row requires locking and rewriting the entire compressed bitmap segment, blocking concurrent writes. • Poorly suited for Transactional (OLTP) systems with frequent ‘INSERT/UPDATE/DELETE’ operations.

Q8. Given the relation `employee(emp_id, name, gender, status)`, create bitmap indices on `gender` and `status`.

E001	Abid	Male	Full Time
E002	Arif	Male	Full Time
E003	Sajid	Male	Full Time
E004	Rafiq	Male	Part Time
E005	Shahid	Male	Full Time
E006	Sajid	Male	Full Time
E007	Karim	Male	Part Time
E008	Zahid	Male	Full Time

Figure: File containing records of relation `employee(emp_id, name, gender, status)`

(5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

3. Visual Representation

The following diagram illustrates the logical mapping from the base relation to the generated bitmap indices.

Row ID	emp_id	gender	status
1	E001	Male	Full Time
2	E002	Male	Full Time
3	E003	Male	Full Time
4	E004	Male	Part Time
5	E005	Male	Full Time
6	E006	Male	Full Time
7	E007	Male	Part Time
8	E008	Male	Full Time

Value	Bit Vector
Male	1 1 1 1 1 1 1 1
Female	0 0 0 0 0 0 0 0

Value	Bit Vector
Full Time	1 1 1 0 1 1 0 1
Part Time	0 0 0 1 0 0 1 0

4. Summary Tables

For direct reference, the final bitmap structures are tabulated below:

Row ID	Male	Female
1	1	0
2	1	0
3	1	0
4	1	0
5	1	0
6	1	0
7	1	0
8	1	0

Row ID	Full Time	Part Time
1	1	0
2	1	0
3	1	0
4	0	1
5	1	0
6	1	0
7	0	1
8	1	0

Q9. Give an example where the Bitmap index is preferred over other indexing techniques. **(5 marks)** [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Ideal Scenario for Bitmap Indexing

A **Bitmap Index** is overwhelmingly preferred over traditional indexing techniques (such as B⁺-trees or Hash indices) in environments that exhibit the following specific characteristics:

- **Low Cardinality Attributes:** The indexed column has a very small number of distinct values relative to the total number of rows (e.g., *Gender*, *Marital Status*, *Boolean flags*, *Membership Tiers*).
- **Data Warehousing (OLAP):** The database is read-heavy with infrequent updates. Updates in bitmap indices are costly because changing a single row can require locking and rewriting entire bit vectors.
- **Complex Multidimensional Queries:** Queries frequently involve multiple AND, OR, and NOT logical conditions across several attributes.

2. Concrete Example: Retail Data Warehouse

Consider a **Customer** table in a data warehouse with 10 million records. A marketing analyst wants to identify customers for a specific campaign. The table includes two low-cardinality attributes: **Gender** (Values: M, F) and **Membership** (Values: Gold, Silver).

284

Base Relation Fragment

Row ID	Customer_Name	Gender	Membership
1	Alice	F	Gold
2	Bob	M	Silver
3	Charlie	M	Gold
4	Diana	F	Silver
5	Evan	M	Gold

Query Execution

Suppose the analyst runs the following query to find all **Male** customers with a **Gold** membership:

```
SELECT Customer_Name FROM Customer
WHERE Gender = 'M' AND Membership = 'Gold';
```

3. Visualizing the Bitwise Execution

Instead of traversing a tree structure, the DBMS fetches the pre-computed bit vectors for **Gender = 'M'** and **Membership = 'Gold'** and applies a fast, CPU-level **Bitwise AND** operation.

AND

0	1	1	0	1
1	0	1	0	1
0	0	1	0	1

Row 3 Row 5

Bitmap for Gender = 'M'

Bitmap for Membership = 'Gold'

Resulting Bitmap (Matches Row 3 & 5)

4. Why Bitmap is Preferred over B⁺-Tree Here

If a standard **B⁺-Tree** were used for this query, it would suffer from severe performance degradation due to the low cardinality of the data:

Feature	Bitmap Index	B ⁺ -Tree Index
Selectivity Handling	Highly efficient. Bitwise operations instantly filter millions of rows in hardware.	Poor. Searching for Gender='M' returns ~50% of the database, requiring massive row ID intersections.
Storage Space	Extremely compact. Highly compressible using algorithms like Run-Length Encoding (RLE).	Bloated. Must store explicit row pointers for every single tuple, consuming vast amounts of disk space.
Hardware Affinity	CPUs evaluate 64-bit boolean algebra (AND/OR) natively in a single clock cycle.	Evaluating branch conditions and traversing tree nodes requires multiple CPU cycles and cache misses.

Q10. The relation **account** is given in Figure 1. Construct bitmap indices on **type**, **branchname** and **location** attribute. Explain how you can find balance for branchname = 'B2' and location = 'L1' and type = 'D'. **(10 marks)** [CSE 303 | L-3/T-1 | 2010–11] **No Figure found in PDF**

B⁺ Tree Index

Q1. Given a B⁺ tree index with 10 million records, node size = 4 kilobytes, search-key size = 8 bytes, disk-pointer size = 4 bytes. Answer the following:

- (a) Given a search-key, how many index blocks need to be read from disk in the best case to find the pointer to the data record?
- (b) Given a range query assuming 1000 records will be retrieved, how many index blocks need to be read in the best case?
- (c) Given a search-key, how many index blocks need to be read in the best and worst cases if we use a hash index instead? Assume the hash index requires at most three overflow blocks.

(15 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer**1. Preliminary Calculations: B⁺ Tree Order and Height**

Before answering the specific queries, we must determine the physical characteristics of the B⁺ tree, specifically its order (branching factor) and its minimum height.

Given Parameters:

- Total records (N) = 10,000,000
- Block size (B) = 4 KB = 4096 bytes
- Search-key size (K) = 8 bytes
- Disk-pointer size (P) = 4 bytes

Step A: Calculate the Order (n) of an Internal Node An internal node contains n disk pointers and $n - 1$ search keys. It must fit within one block.

$$\begin{aligned} n \cdot P + (n - 1) \cdot K &\leq B \\ n(4) + (n - 1)(8) &\leq 4096 \\ 12n - 8 &\leq 4096 \implies 12n \leq 4104 \implies n = 342 \end{aligned}$$

The maximum branching factor (order) is **342**.

Step B: Calculate Leaf Node Capacity (L) A leaf node contains data pointers (or records), search keys, and one sibling pointer to the next leaf.

$$\begin{aligned} L \cdot (K + P) + P &\leq B \\ L(8 + 4) + 4 &\leq 4096 \\ 12L &\leq 4092 \implies L = 341 \end{aligned}$$

A leaf node can hold a maximum of **341** records.

Step C: Calculate Minimum Tree Height (h) To find the best-case (minimum) number of levels, we assume the tree is 100% full.

- **Level 1 (Root):** 1 node
- **Level 2:** 342 nodes
- **Level 3 (Leaves):** $342 \times 342 = 116,964$ nodes

Maximum record capacity at height 3 = 116,964 leaves $\times$ 341 records/leaf = **39,884,724 records**. Since $10,000,000 \leq 39,884,724$, a B⁺ tree of **height 3** is sufficient to store all records.

2. Solutions to the Questions

- (a) **Given a search-key, how many index blocks need to be read from disk in the best case to find the pointer to the data record?**

Answer: 3 index blocks.

- In a B⁺ tree, data pointers are *only* stored in the leaf nodes. Therefore, every exact-match search must traverse from the root down to a leaf.
- Since the minimum height of the tree is 3, the traversal requires reading exactly **1 Root block, 1 Internal block, and 1 Leaf block**.
- *Note:* Even in the best case, you cannot stop early at an internal node in a B⁺ tree.

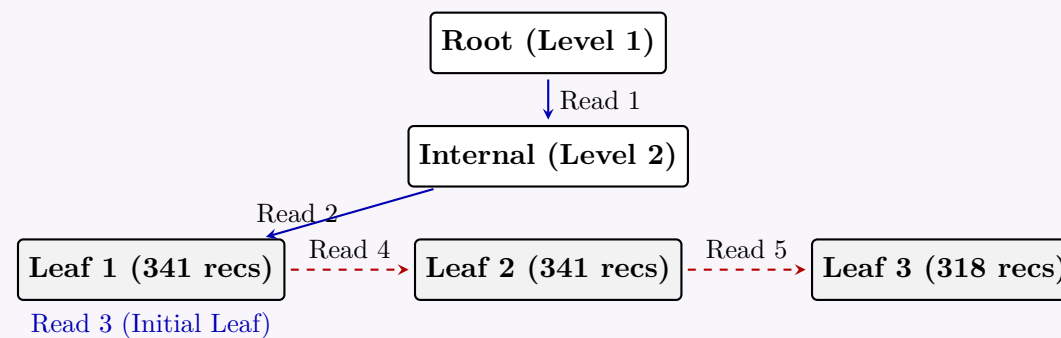
- (b) **Given a range query assuming 1000 records will be retrieved, how many index blocks need to be read in the best case?**

Answer: 5 index blocks.

- **Locate starting record:** Finding the first record requires traversing the tree from root to leaf, which takes **3 block reads**.
- **Calculate required leaf blocks:** The maximum capacity of a single leaf block is 341 records. In the best case (nodes are 100% full), 1000 contiguous records will span:

$$\lceil 1000/341 \rceil = \lceil 2.93 \rceil = \mathbf{3 \text{ leaf blocks}}$$

- **Total Reads:** We already read the first leaf block during the downward traversal. We then use the linked-list sibling pointers to read the remaining $3 - 1 = \mathbf{2}$ leaf blocks.
- Total blocks = 3 (initial search) + 2 (subsequent leaves) = **5 blocks**.



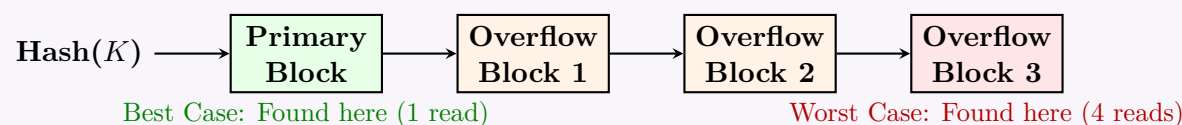
(c) Given a search-key, how many index blocks need to be read in the best and worst cases if we use a hash index instead? Assume the hash index requires at most three overflow blocks.

- **Best Case: 1 index block.**

The hash function maps the search-key directly to the primary bucket, and the target record pointer is found immediately in that very first block.

- **Worst Case: 4 index blocks.**

Due to hash collisions, the bucket may be full, creating an overflow chain. In the worst-case scenario, the target record is located at the very end of the maximum allowable overflow chain. We must read the primary bucket (1 block) plus all maximum overflow blocks (3 blocks), totaling **4 blocks**.



Q2. Assume a B^+ tree index is initially empty and the number of pointers that fit in one node is 4.

(a) Construct the tree by inserting the following key values in the given order: 2, 3, 5, 7, 11, 17, 19, 23, 29, 31.

(b) Show the final form of the tree after: Delete 23, Delete 19.

(13 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Properties of the B^+ Tree

Given the order of the B^+ tree is $n = 4$ (maximum number of pointers per node):

- **Maximum children/pointers** per internal node: $n = 4$.
- **Maximum keys** per node: $n - 1 = 3$.
- **Minimum children** per internal node: $\lceil n/2 \rceil = 2$.
- **Minimum keys** per internal node: $\lceil n/2 \rceil - 1 = 1$.
- **Minimum keys** per leaf node: $\lceil (n - 1)/2 \rceil = 2$ (except for the root).

A node overflows when it receives a 4th key. When a node splits, it is divided into two nodes. For a leaf, the middle key is **copied up** to the parent. For an internal node, the middle key is **pushed up**.

(a) Constructing the Tree

We insert the keys sequentially: **2, 3, 5, 7, 11, 17, 19, 23, 29, 31**.

(a) **Insert 2, 3, 5:** The root is a leaf node. It holds [2, 3, 5].

(b) **Insert 7 (Overflow):** The leaf has [2, 3, 5, 7]. It splits into left [2, 3] and right [5, 7]. The middle key **5** is copied up to the new root.

(c) **Insert 11:** Added to the right leaf, which becomes [5, 7, 11].

(d) **Insert 17 (Overflow):** The right leaf has [5, 7, 11, 17]. It splits into [5, 7] and [11, 17]. **11** is copied up. Root becomes [5, 11].

(e) **Insert 19:** Added to the rightmost leaf, which becomes [11, 17, 19].

(f) **Insert 23 (Overflow):** The rightmost leaf has [11, 17, 19, 23]. It splits into [11, 17] and [19, 23]. **19** is copied up. Root becomes [5, 11, 19].

(g) **Insert 29:** Added to the rightmost leaf, which becomes [19, 23, 29].

(h) **Insert 31 (Overflow):** The rightmost leaf has [19, 23, 29, 31]. It splits into [19, 23] and [29, 31]. **29** is copied up to the root.

(i) **Root Overflow:** The root now has [5, 11, 19, 29] (4 keys). The internal node splits into left [5, 11] and right [29]. The middle key **19** is **pushed up** to create a new root.

Final B⁺ Tree after Insertions

(b) Tree After Deletions

(a) Delete 23 (Underflow & Merge):

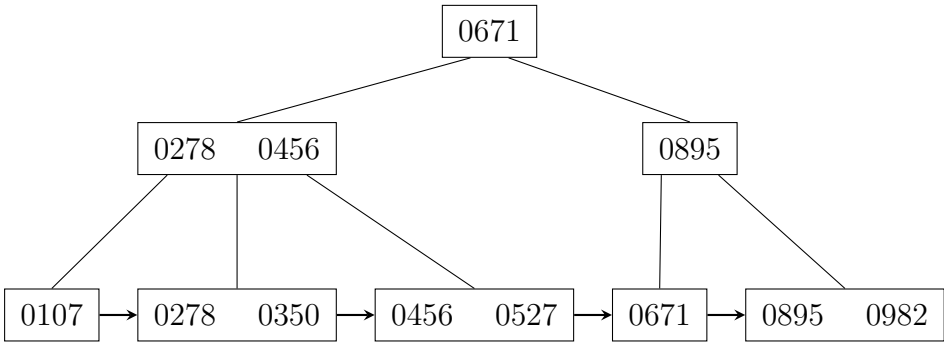
- Key 23 is removed from the leaf [19, 23], leaving only [19]. This causes an **underflow** (minimum keys = 2).
- Its sibling is [29, 31]. Borrowing is impossible because the sibling only has 2 keys (it would also underflow).
- We **merge** [19] and [29, 31] into a single leaf node: [19, 29, 31].
- The routing key **29** is removed from the parent node. The parent internal node becomes empty [], causing an **internal underflow**.
- The empty node borrows from its left sibling [5, 11]. The parent's routing key **19** is moved down to the empty node, and the sibling's largest key **11** is pushed up to the root.
- The left internal node becomes [5], the right internal node becomes [19], and the root becomes [11]. The pointer to the leaf [11, 17] shifts to the right internal node.

(b) Delete 19:

- Key 19 is removed from the newly merged leaf [19, 29, 31], leaving [29, 31]. No underflow occurs.
- Since 19 is an internal routing key, it is standard practice to update the separator by replacing it with the new minimum value of its right subtree. The internal node [19] is updated to [29].

Final B⁺ Tree after Deletions

Q3. The following is a degree-3 B⁺ tree created after some insert/delete operations:



Draw the state of this B⁺ tree after each of the following operations:

- (a) Insert 315
- (b) Insert 409 and 513
- (c) Delete 456
- (d) Delete 513
- (e) Delete 982 and 895

(20 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

Structural Rules for a Degree-3 B⁺ Tree

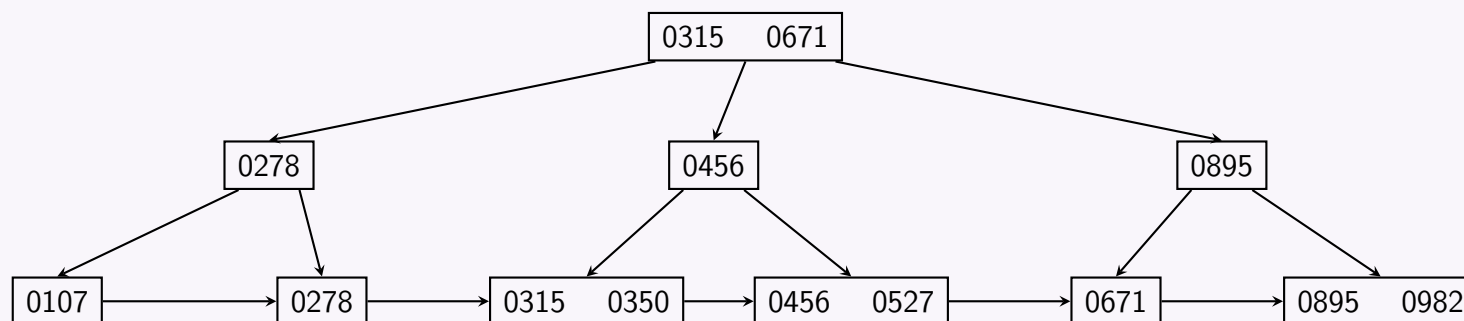
A B⁺ tree of degree $p = 3$ satisfies the following properties:

- **Internal Nodes:** Can hold a maximum of $p - 1 = 2$ keys and up to $p = 3$ child pointers. They must contain at least $\lceil p/2 \rceil - 1 = 1$ key.
- **Leaf Nodes:** Can hold a maximum of $p - 1 = 2$ data keys. They must contain at least $\lfloor p/2 \rfloor = 1$ key (since the root as a leaf can have a minimum of 1 key, and standard $\lceil (p - 1)/2 \rceil = 1$).
- **Node Overflow:** Occurs when a node receives a 3rd key.
 - For a **leaf node split**, the middle key is copied up to the parent, and remains in the right-hand child leaf node.
 - For an **internal node split**, the middle key is pushed up to the parent node and removed from the child level.
- **Node Underflow:** Occurs when a node falls below the minimum required number of keys. Underflow is resolved by borrowing from a sibling (redistribution) or merging with a sibling if borrowing is not possible.

(1) State After Inserting 315

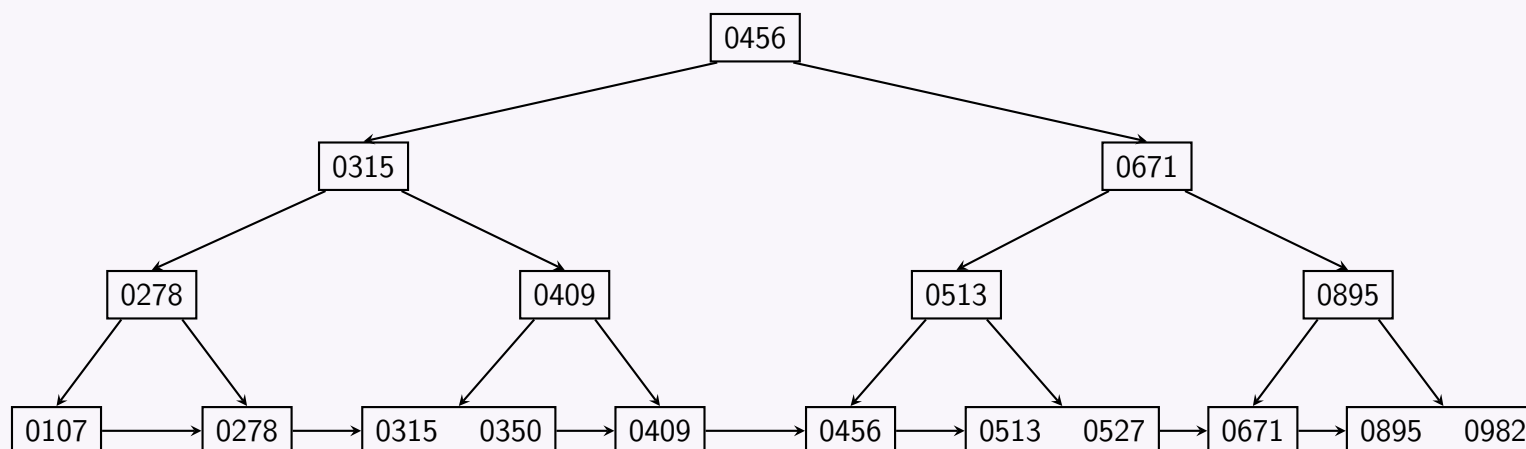
The key 315 is inserted into the leaf node containing [0278, 0350].

- This creates a temporary overflow: [0278, 0315, 0350].
- The leaf splits into two: a left leaf [0278] and a right leaf [0315, 0350].
- The middle key 0315 is copied up into the parent internal node.
- The parent internal node, which previously held [0278, 0456], now overflows with [0278, 0315, 0456].
- The internal node splits: the left node keeps [0278], the right node keeps [0456], and the middle key 0315 is pushed up to the root.
- The root node overflows from [0671] to [0315, 0671]. No further split is required as the root now contains exactly 2 keys.



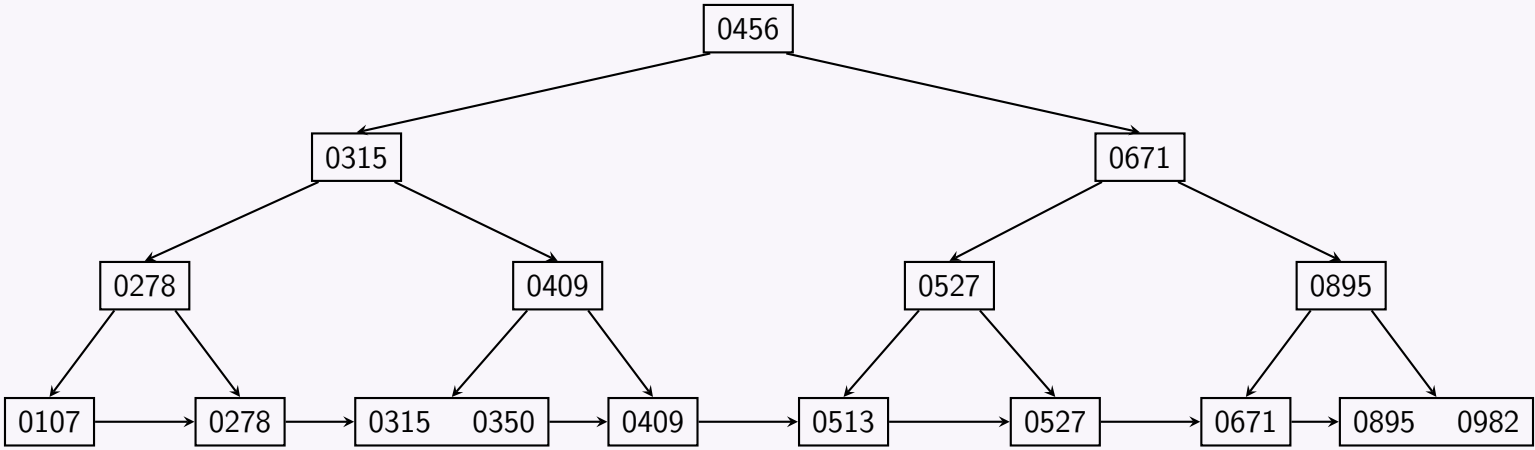
(2) State After Inserting 409 and 513

- **Insert 409:** 409 belongs in the leaf [0456, 0527], causing an overflow: [0409, 0456, 0527]. The leaf splits into [0409] and [0456, 0527], copying 0456 up to internal node n2, which becomes [0456].
- **Insert 513:** 513 targets the leaf node [0456, 0527], which overflows to [0456, 0513, 0527]. This splits into [0456] and [0513, 0527]. The middle key 0513 is copied up to internal node n2, making it overflow with [0456, 0513].
- Internal node n2 splits: its left child remains [0456], its right child becomes [0513], and the middle key 0456 is pushed up to the root.
- The root now contains [0315, 0456, 0671], which overflows and splits. The middle key 0456 is pushed up to form a new single-key root. The old root splits into two internal nodes: [0315] and [0671].



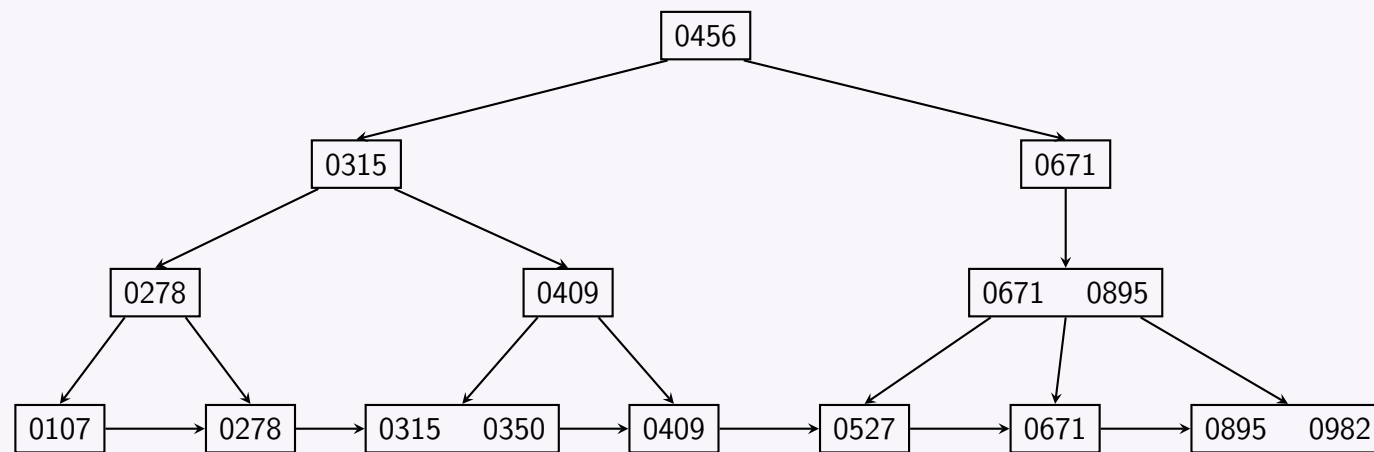
(3) State After Deleting 456

- The key 0456 is removed from its leaf node. The node becomes empty, underflowing.
- Its right sibling is [0513, 0527]. It contains 2 keys, which allows borrowing.
- We borrow the smallest key, 0513, from the sibling. The leaf node now contains [0513], and the sibling node becomes [0527].
- The parent internal index entry separating these two leaf nodes is updated from 0513 to the new minimum entry of the right leaf, which is 0527.



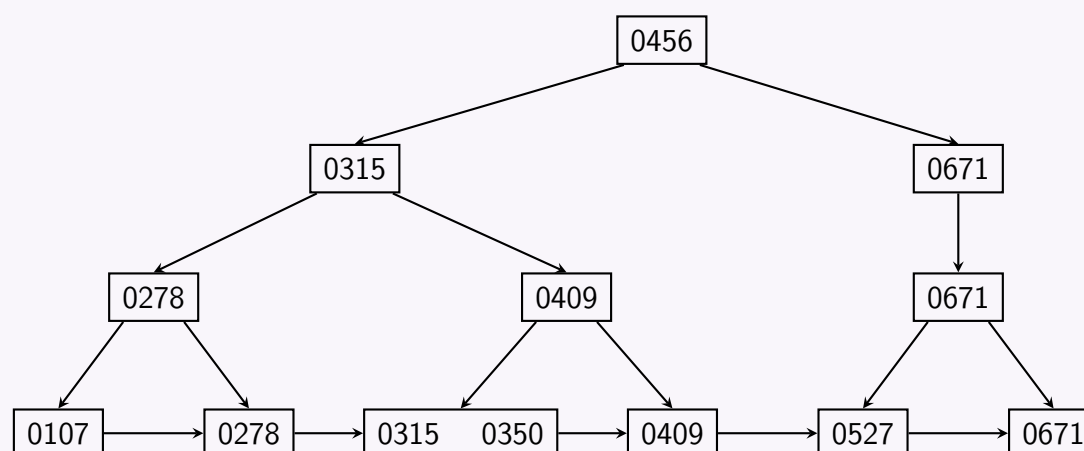
(4) State After Deleting 513

- Deleting 0513 from its leaf node makes it empty. Its sibling node contains only one element ([0527]), preventing redistribution.
- The empty leaf is **merged** with its right sibling into a single leaf node [0527].
- The separating key 0527 is removed from the internal parent node, causing that node to underflow.
- The empty internal node underflows, and its sibling internal node contains only one key ([0895]). Borrowing is impossible, so they **merge**.
- During this internal merge, the parent separator key from the level above (0671) is brought down. This forms the combined internal node [0671, 0895].
- Intermediate node i2 is now empty and underflows. It cannot borrow from its left sibling i1 ([0315]), so they **merge** by pulling down the root entry 0456.
- The root node becomes empty, and the tree height shrinks by 1 level. The new root is the merged node containing [0315, 0456, 0671]. This node now overflows because it contains 3 keys.
- Splitting this root pushes 0456 back up to form a new single-key root, restoring the tree to its previous height.



(5) State After Deleting 982 and 895

- Delete 982:** Removed from leaf [0895, 0982], leaving [0895] safely with no underflow.
- Delete 895:** Removed from leaf [0895], causing an underflow.
- Its left sibling is [0671], which also has only 1 key. They **merge** into a single leaf node: [0671].
- The separating internal entry 0895 is deleted from node n3, changing it from [0671, 0895] to [0671].
- The leaf pointers under node n3 are updated to point to [0527] and [0671].



Q4. For a degree-5 B⁺ tree, what will be the height of the B⁺ tree when the numbers from 1 to 100 are inserted sequentially? *Hint:* A tree with l leaves and internal nodes of degree d has height approximately $\log_d l$. (5 marks) [CSE 215 | L-2/T-1 | 2022–23]

Answer

1. B⁺ Tree Degree and Split Properties

For a B⁺ tree of degree $d = 5$:

- Internal Nodes:** Can hold a maximum of $d = 5$ children (pointers) and $d - 1 = 4$ keys.
- Leaf Nodes:** Can hold a maximum of $d - 1 = 4$ data keys.
- Standard Split Rule:** When a node overflows, it splits into two. By standard convention (left-biasing):

- **Leaf Split (5 keys):** The left node keeps $\lceil 5/2 \rceil = 3$ keys, and the right node gets 2 keys.
- **Internal Split (6 pointers):** The left node keeps $\lceil 6/2 \rceil = 3$ pointers, and the right node gets 3 pointers.

2. Analysis of Sequential Insertion

When values (1 to 100) are inserted strictly sequentially (in ascending order), the insertions always go to the **rightmost active leaf node**.

- Once a leaf or internal node splits, its left half is "left behind" and will **never** receive any new elements.
- Consequently, all leaf nodes (except the currently active rightmost one) will be permanently fixed at exactly **3 keys**.
- All internal nodes (except the rightmost one) will be permanently fixed at exactly **3 pointers**.
- Sequential insertion creates a sparse tree, making its height larger than if the nodes were maximally packed.

3. Layer-by-Layer Calculation

Level 0: Leaf Nodes

- Total keys to insert = 100.
- Since every fully processed leaf contains 3 keys, let L be the total number of leaves.
- The first $L - 1$ leaves contain 3 keys. The last leaf contains r keys (where $2 \leq r \leq 4$).
- $3(L - 1) + r = 100$. For $r = 4$, we get $3(L - 1) = 96 \implies L - 1 = 32 \implies \mathbf{L = 33}$.
- There are **33 leaf nodes**.

Level 1: Internal Nodes (Parents of Leaves)

- The 33 leaf nodes require **33 pointers** from their parent nodes.
- Because internal nodes leave behind 3 pointers when they split, and 33 is perfectly divisible by 3, this level will consist of exactly $33/3 = \mathbf{11}$ nodes.

Level 2: Internal Nodes

- The 11 nodes from Level 1 require **11 pointers**.
- Simulating the splits: The first two nodes will leave behind 3 pointers each (Total = 6). The active rightmost node will hold the remaining 5 pointers (which is valid since maximum pointers = 5).
- Thus, Level 2 requires **3 nodes** (holding 3, 3, and 5 pointers respectively).

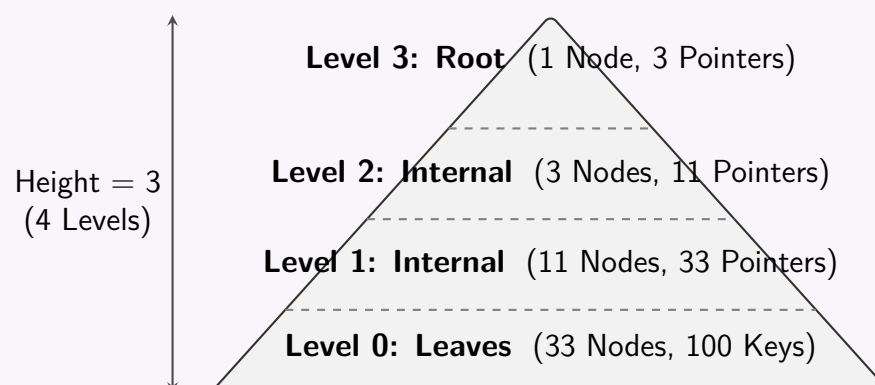
Level 3: Root Node

- The 3 nodes in Level 2 require **3 pointers**.
- A single internal node can hold up to 5 pointers, so these 3 pointers fit perfectly into a single node.
- This is the **root node**.

4. Conclusion

The tree is structured over 4 levels. Using the standard definition that the height of a tree is the number of edges on the path from the root to a leaf:

- **Number of Levels: 4**
- **Height of the Tree: 3**



Q5. (a) Construct a B⁺ tree index structure with $n = 4$, incrementally inserting the search keys 90, 80, 70, 60, 50, 40, 30, 20.

(b) Show the structures after deletion of 20, 30, 40 and 50.

(20 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

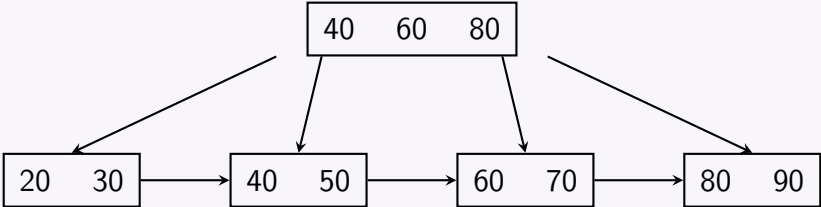
1. B⁺ Tree Construction (Insertions)

A B⁺ tree of order $n = 4$ has the following properties:

- Maximum children (pointers) per internal node: $n = 4$.
- Maximum keys per internal and leaf node: $n - 1 = 3$.
- Minimum keys per leaf node: $\lceil (n - 1)/2 \rceil = \lceil 1.5 \rceil = 2$ (except for the root).
- Minimum children per internal node: $\lceil n/2 \rceil = 2$.
- Split Rule: When a node gets 4 keys, it splits into two nodes of 2 keys each. The middle key is copied up (for leaves) or pushed up (for internal nodes).

Insertion Steps:

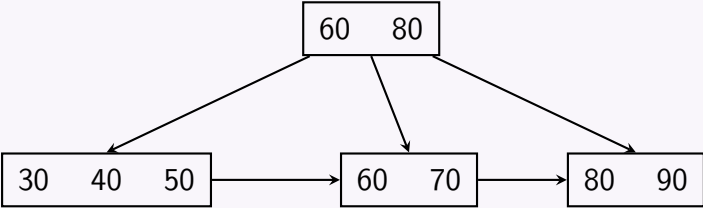
- (a) **Insert 90, 80, 70:** Keys are inserted into a single leaf node (the root) in sorted order: [70, 80, 90].
- (b) **Insert 60:** The node overflows [60, 70, 80, 90]. It splits into [60, 70] and [80, 90]. The key 80 is copied up to a new root.
- (c) **Insert 50:** Inserted into the left leaf, which becomes [50, 60, 70].
- (d) **Insert 40:** Left leaf overflows [40, 50, 60, 70]. It splits into [40, 50] and [60, 70]. The key 60 is copied up to the root.
- (e) **Insert 30:** Inserted into the first leaf, making it [30, 40, 50].
- (f) **Insert 20:** First leaf overflows [20, 30, 40, 50]. It splits into [20, 30] and [40, 50]. The key 40 is copied up to the root.

Final Structure After All Insertions:

2. B⁺ Tree Deletions

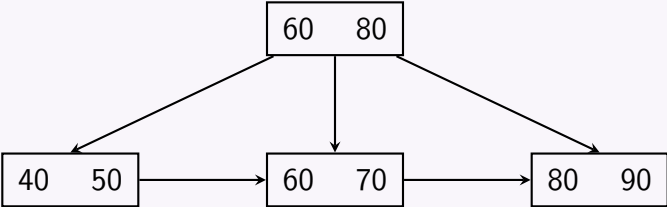
a) Delete 20

- Action: Remove 20 from leaf [20, 30].
- Underflow: The leaf now has only 1 key ([30]), which is below the minimum (2).
- Resolution: The right sibling is [40, 50]. Borrowing is not possible because it would leave the sibling with 1 key (underflow). Therefore, we merge [30] and [40, 50] into a single node [30, 40, 50]. The separator 40 is deleted from the root.



b) Delete 30

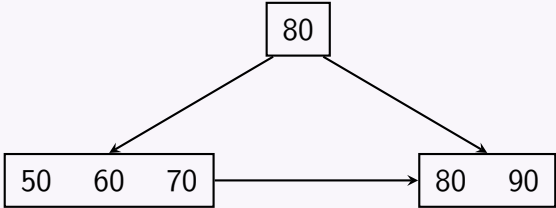
- Action: Remove 30 from leaf [30, 40, 50].
- Resolution: The leaf becomes [40, 50]. It contains 2 keys, satisfying the minimum requirement. No further changes needed.



293

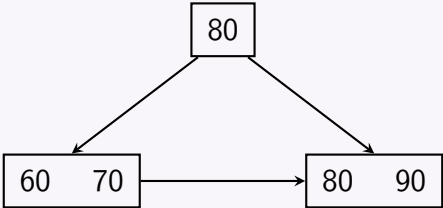
c) Delete 40

- **Action:** Remove 40 from leaf [40, 50].
- **Underflow:** The leaf now has only 1 key ([50]).
- **Resolution:** The right sibling [60, 70] has only 2 keys, preventing borrowing. We **merge** [50] with [60, 70] to form [50, 60, 70]. The separator 60 is deleted from the root.



d) Delete 50

- **Action:** Remove 50 from leaf [50, 60, 70].
- **Resolution:** The leaf becomes [60, 70]. It still has 2 keys, which meets the minimum capacity requirement. No further changes needed.



Q6. A relation schema $R(A, B, C)$ and a relation $r(R)$ is given below. The relation is physically sorted on the primary key A .

A	B	C
a_1	b_1	c_1
a_2	b_2	c_1
a_3	b_3	c_1
a_4	b_1	c_1
a_5	b_2	c_1
a_7	b_2	c_1
a_8	b_3	c_1

Construct the B⁺ tree primary index for the relation r (relation $R(A, B, C)$ sorted on primary key A , with keys $a_1, a_2, a_3, a_4, a_5, a_7, a_8$) with $n = 3$ by insertion of search keys incrementally in sorted order and splitting of nodes. After construction of the tree, insert a_6 and show the tree structure. (20 marks) [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. B⁺ Tree Parameters and Split Rules

For a B⁺ tree of order $n = 3$:

- **Maximum keys per node:** $n - 1 = 2$.
- **Maximum children (pointers) per internal node:** $n = 3$.
- **Minimum keys per leaf node:** $\lceil (n - 1)/2 \rceil = 1$ (excluding root).
- **Split Rule:** When a node receives 3 keys, it overflows and must be split.
 - **Leaf Split:** The 3 keys are divided such that the left node retains $\lceil 3/2 \rceil = 2$ keys, and the right node takes 1 key. The first key of the right node is *copied up* to the parent.
 - **Internal Split:** The 3 keys are divided such that the left node retains 1 key, the right node takes 1 key, and the middle key is *pushed up* (moved) to the parent.

2. Step-by-Step Incremental Construction

The relation r is sorted on primary key A . The keys to insert are: $a_1, a_2, a_3, a_4, a_5, a_7, a_8$.

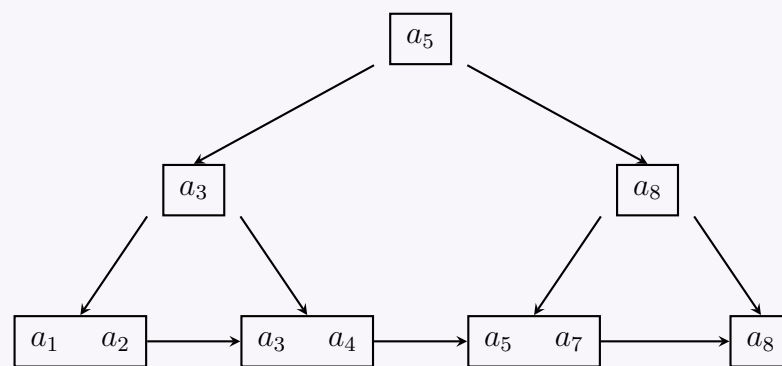
(a) Insert a_1, a_2 : Placed into the root leaf node: $[a_1, a_2]$.

(b) Insert a_3 : Node overflows $[a_1, a_2, a_3]$.

- Splits into leaves $[a_1, a_2]$ and $[a_3]$.
- Copy a_3 up. Root becomes $[a_3]$.

- (c) **Insert a_4 :** Added to the rightmost leaf: $[a_3, a_4]$.
- (d) **Insert a_5 :** Rightmost leaf overflows $[a_3, a_4, a_5]$.
- Splits into $[a_3, a_4]$ and $[a_5]$.
 - Copy a_5 up. Root becomes $[a_3, a_5]$.
- (e) **Insert a_7 :** Added to the rightmost leaf: $[a_5, a_7]$.
- (f) **Insert a_8 :** Rightmost leaf overflows $[a_5, a_7, a_8]$.
- Splits into $[a_5, a_7]$ and $[a_8]$.
 - Copy a_8 up. Root receives a_8 and overflows $[a_3, a_5, a_8]$.
 - **Root Split (Internal):** Left gets $[a_3]$, right gets $[a_8]$, middle key a_5 is pushed up to form a new root.

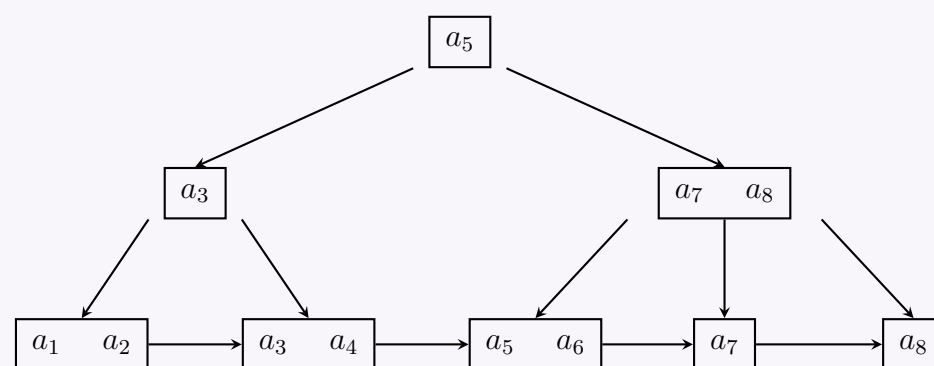
Initial B⁺ Tree Structure (After inserting up to a_8):



3. Insertion of a_6

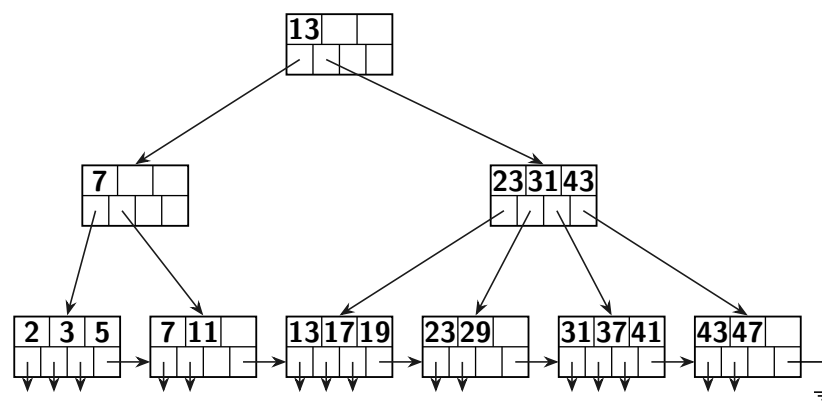
- **Search Phase:** Start at root $[a_5]$. Since $a_6 \geq a_5$, follow the right pointer to $[a_8]$. Since $a_6 < a_8$, follow the left pointer to the leaf $[a_5, a_7]$.
- **Insert & Overflow:** Insert a_6 into the leaf, resulting in $[a_5, a_6, a_7]$. This exceeds the maximum capacity of 2 keys.
- **Leaf Split:** The leaf splits into $[a_5, a_6]$ and $[a_7]$. The first key of the new right node, a_7 , is copied up to the parent node $[a_8]$.
- **Parent Update:** The parent internal node receives a_7 and becomes $[a_7, a_8]$. It now has 2 keys, which is within the maximum capacity ($n - 1 = 2$). No further splits are required.

Final B⁺ Tree Structure (After inserting a_6):



Q7. Consider the given B⁺ tree . Modify it incrementally according to the following instructions. For each instruction, draw the final modified state:

- Insert keys 8, 9
- Insert keys 24, 27
- Remove keys 8, 9
- Remove keys 24, 27



(16 marks)

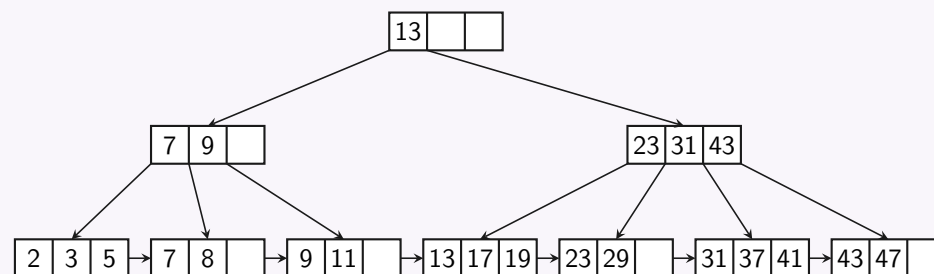
[CSE 215 | L-2/T-2 | 2017–18]

Answer**Properties of the Given B⁺ Tree:**

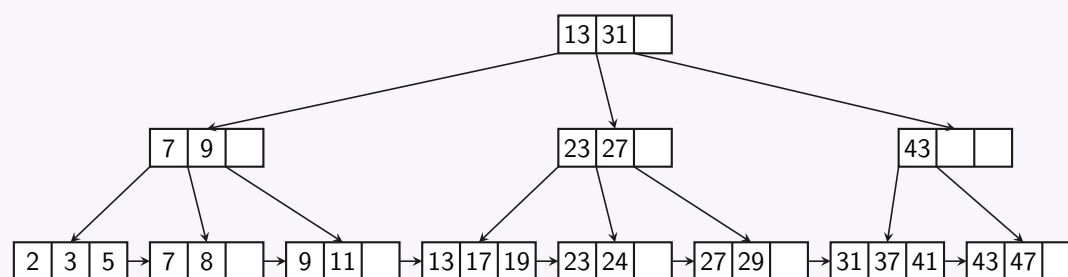
- **Order (m):** 4. Each node can have at most 4 children and hold up to 3 keys.
- **Node Splits:** When a node exceeds 3 keys, it splits. The median key is copied up (for leaves) or pushed up (for internal nodes).
- **Node Merges:** Deletions that cause nodes to underflow prompt merges with sibling nodes or borrowing, thereby dynamically re-balancing the tree.

1. After Inserting keys 8, 9

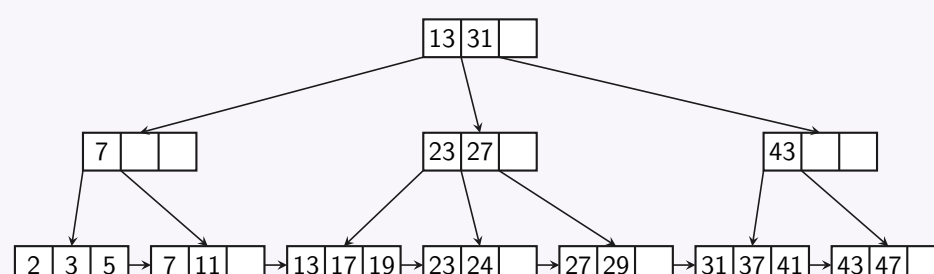
- **Insert 8:** Key 8 is added to the leaf [7, 11], making it [7, 8, 11]. This is stable.
- **Insert 9:** Key 9 is added, causing the leaf to overflow as [7, 8, 9, 11]. The leaf splits into [7, 8] and [9, 11]. Key 9 is copied up to the parent node [7], changing it to [7, 9].

**2. After Inserting keys 24, 27**

- **Insert 24:** Key 24 goes to leaf [23, 29], forming [23, 24, 29].
- **Insert 27:** The leaf overflows as [23, 24, 27, 29]. It splits into [23, 24] and [27, 29]. Key 27 is copied up to the parent.
- **Parent Overflow:** The parent [23, 31, 43] becomes [23, 27, 31, 43] and overflows. It splits into [23, 27] and [43]. The median key 31 is pushed up to the root, which becomes [13, 31].

**3. After Removing keys 8, 9**

- Deleting 8 and 9 forces leaves [7, 8] and [9, 11] to underflow.
- The two neighboring leaf nodes merge back into [7, 11]. The redundant separator 9 is deleted from the parent index node, restoring it to [7].



4. After Removing keys 24, 27

- Deleting 24 and 27 leaves nodes [23] and [29] which undergo underflow and merge back into [23, 29].
- The index separator 27 is removed. This triggers an internal node underflow, prompting index nodes [23] and [43] to merge along with root separator 31.
- The tree perfectly regains its exact initial configuration.

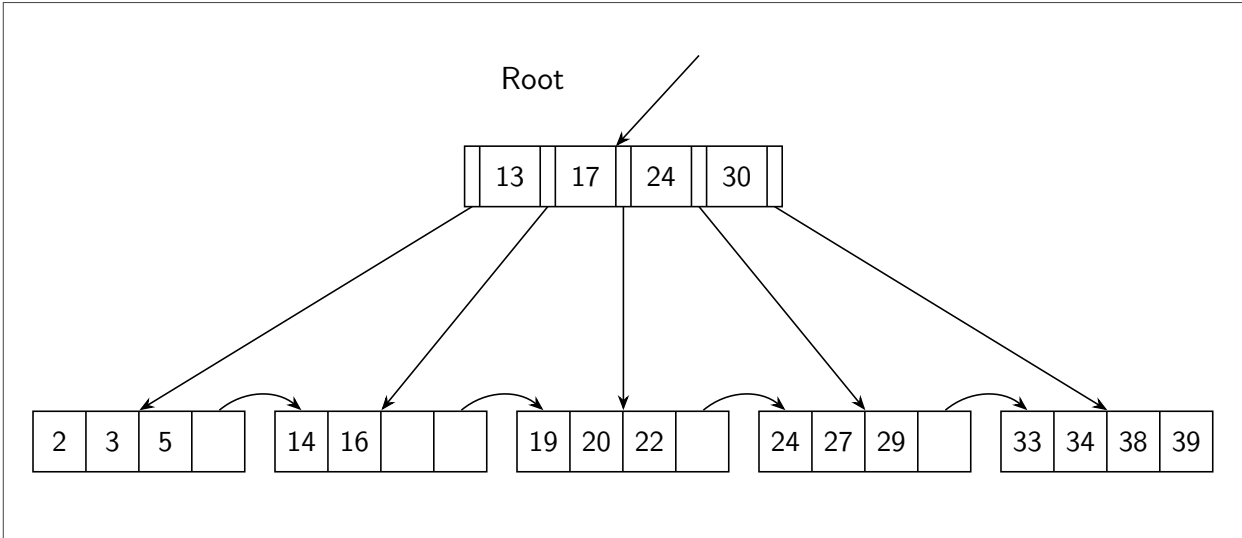
Q8. Consider the B⁺ tree with leaf nodes: {2, 3, 5}, {14, 16}, {19, 20, 22}, {24, 27, 29}, {33, 34, 38, 39}. Show (without explanation) what happens after each of the following successive operations:

(a) Insert 7

(b) Insert 8

(c) Delete 19

(d) Delete 20



(8 marks)

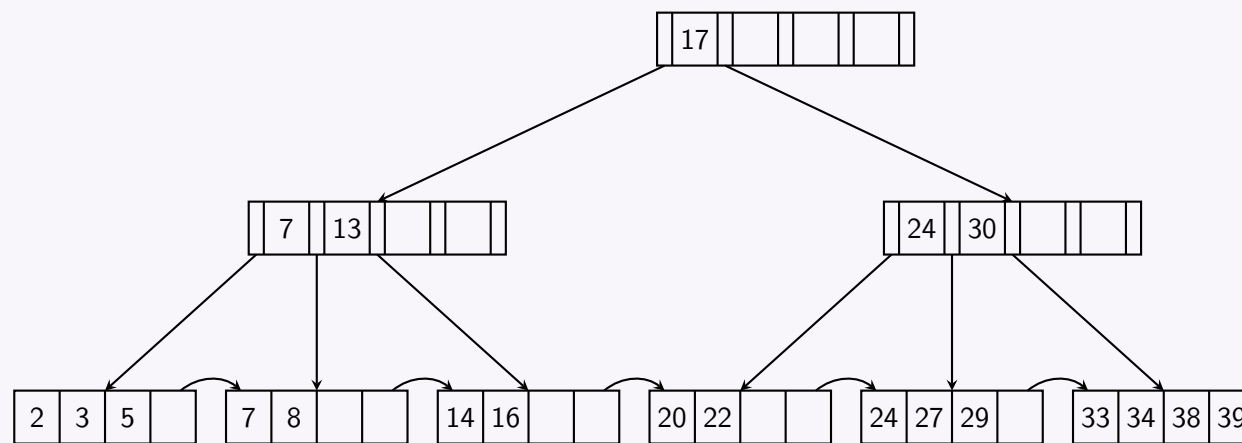
[CSE 303 | L-3/T-1 | 2016–17]

Answer

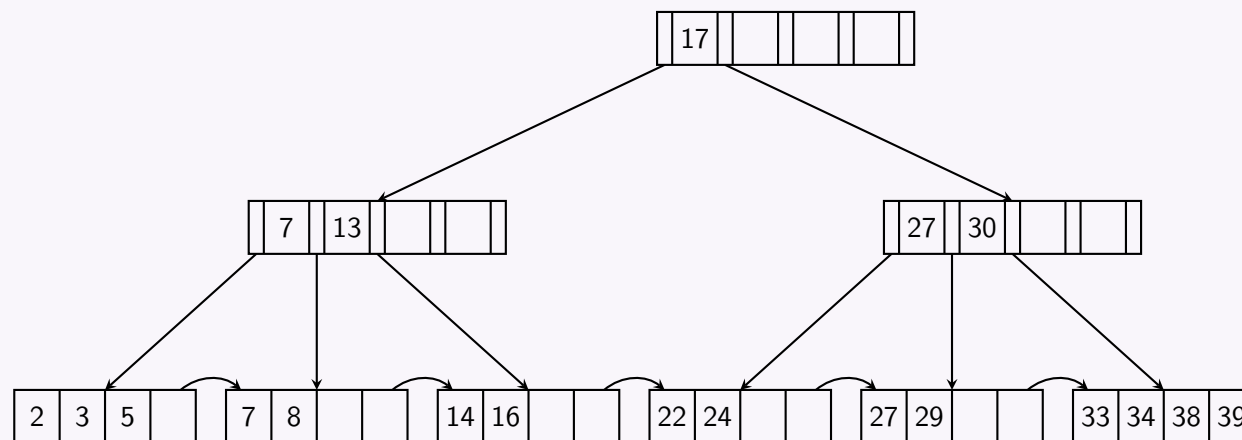
1. After Inserting 7

2. After Inserting 8

3. After Deleting 19



4. After Deleting 20



Q9. Write down the key properties of a B⁺ tree. (8 marks)

[CSE 303 | L-3/T-1 | 2017–18, 2016–17, 2011–12]

Answer

A **B⁺ Tree** is a highly balanced, self-maintaining multi-way search tree predominantly used in Database Management Systems (DBMS) for efficient disk-based indexing. It acts as an extension of the standard B-tree, optimized to reduce disk I/O operations. Below are the defining properties of a B⁺ Tree of order m (where m represents the maximum number of children for an internal node):

1. Structural Properties

- **Perfectly Balanced:** All leaf nodes reside at the exact same depth. Every path from the root to a leaf has the identical length (h , height of the tree).
- **Separation of Index and Data:** Internal (non-leaf) nodes store *only* search keys and child pointers (acting purely as a routing index). Actual data records or pointers to data blocks are strictly stored in the **leaf nodes**.
- **Linked Leaf Nodes:** The leaf nodes are linked horizontally via sibling pointers (typically a doubly or singly linked list). This property significantly accelerates sequential access and range queries, bypassing the need to traverse up and down the tree once the starting point is found.

2. Node Capacity Constraints (Order m)

- **Internal Nodes (Index Nodes):**
 - Must contain between $\lceil m/2 \rceil$ and m children (except for the root).
 - Must contain between $\lceil m/2 \rceil - 1$ and $m - 1$ search keys.
- **Leaf Nodes (Data Nodes):**
 - Must contain between $\lceil m/2 \rceil - 1$ and $m - 1$ data keys.
 - Each data key has an associated pointer to the actual data record on the disk.
- **Root Node:**
 - If it is not a leaf, it must have at least 2 children and 1 search key.
 - If the tree only contains a root, it operates as a leaf and can have between 1 and $m - 1$ keys.

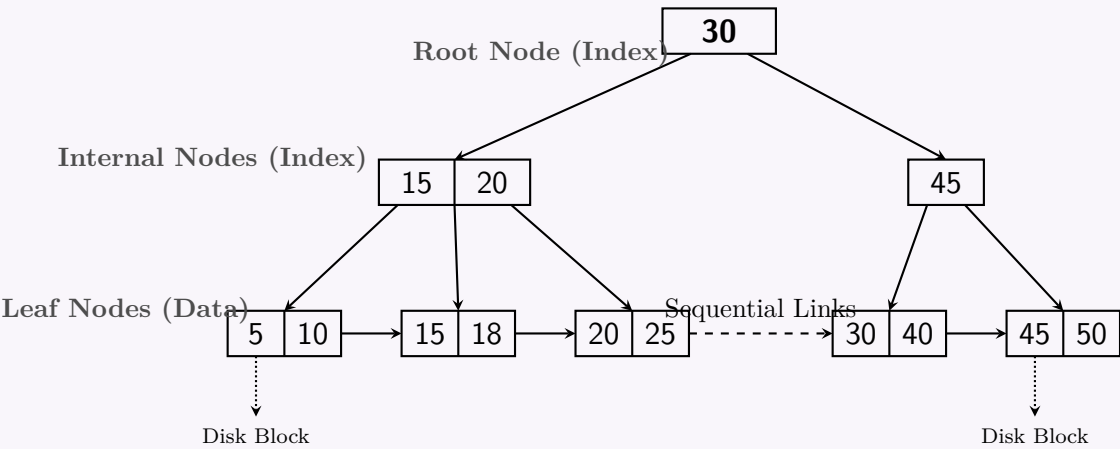
3. Key Ordering

- Keys within any given node (both internal and leaf) are sorted in strictly ascending order.
- For any search key K in an internal node:
 - All keys in the left subtree are **less than** K .
 - All keys in the right subtree are **greater than or equal to** K .

4. Performance & Complexity

- **Point Queries (Search, Insert, Delete):** $O(\log_m n)$, where n is the number of records. Due to a large branching factor m , the tree height remains very small, ensuring minimal disk accesses.
- **Range Queries:** $O(\log_m n + k)$, where k is the number of matching records retrieved sequentially via the linked leaves.

Figure: Visual Representation of a B⁺ Tree (Order $m = 3$)



Q10. Show the final B⁺ tree index structure for the search key values 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24 with 4 leaf nodes and $n = 4$. (You do not need to create this index step by step.) Then show the index structure after insertion of 15. **(20 marks)** [CSE 303 | L-3/T-1 | 2015–16]

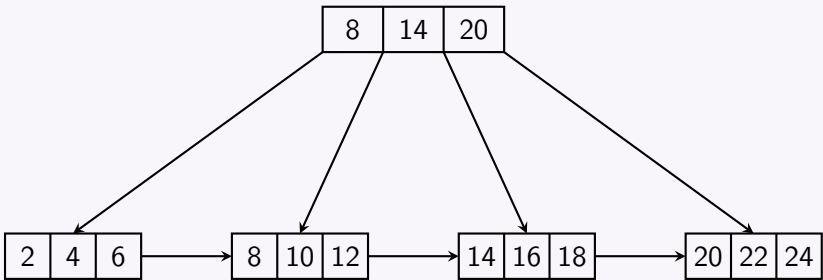
Answer

1. Initial B⁺ Tree Structure

Given $n = 4$, the B⁺ tree has the following properties:

- **Maximum children per internal node:** $n = 4$
- **Maximum keys per node (internal and leaf):** $n - 1 = 3$
- **Initial keys:** 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24 (Total 12 keys)

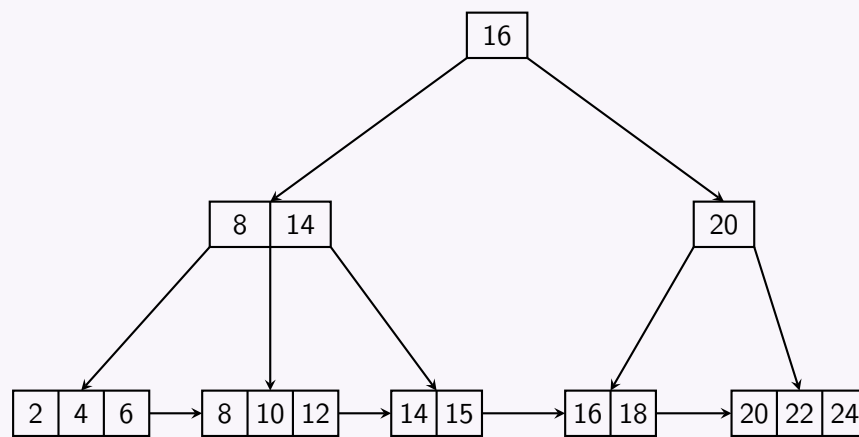
Since we are constrained to exactly 4 leaf nodes to hold 12 keys, each leaf node must be completely full (holding exactly $12/4 = 3$ keys). The root node will act as an internal routing node, containing the smallest key from each leaf (except the first one) to correctly guide the search process.



2. Insertion of 15

When the key **15** is inserted, the following operations occur:

- (a) **Search:** The key 15 is routed to the third leaf node [14, 16, 18].
- (b) **Leaf Overflow:** Inserting 15 makes the leaf [14, 15, 16, 18]. Because max capacity is 3, this causes an **overflow**.
- (c) **Leaf Split:** The leaf splits into a left node [14, 15] and a right node [16, 18]. The smallest key of the right node (**16**) is copied up to the parent.
- (d) **Root Overflow:** The root becomes [8, 14, 16, 20]. Since it can only hold 3 keys, it **overflows**.
- (e) **Internal Node Split:** The root splits into a left internal node [8, 14] and a right internal node [20]. The middle key (**16**) is *pushed up* to create a new root node.
- (f) **Height Increase:** The tree height increases by one level, remaining perfectly balanced.



Q11. Discuss the basic principles of B⁺ tree index structure. (10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Definition and Core Concept

A **B⁺ tree** is a specialized, self-balancing m -way search tree that serves as the standard storage structure for modern relational database management systems (DBMS). It is designed specifically to perform efficient lookup, insertion, and deletion operations on block-oriented secondary storage devices (such as HDDs and SSDs).

Unlike a standard B-tree, where search keys and data records can reside at any level of the tree, a B⁺ tree enforces a strict structural separation: **internal nodes** act exclusively as a routing index, while all **data records** (or pointers to data records) are isolated within the **leaf nodes**.

2. Structural Properties and Invariants

A B⁺ tree of order m satisfies the following mathematical invariants:

- **Perfect Balance:** Every leaf node resides at the exact same depth. This ensures a uniform access time for any record in the database.
- **Internal Node Capacity:** Every internal node (except the root) must have at least $\lceil m/2 \rceil$ children and can have at most m children. The number of keys contained within an internal node is always one less than its number of child pointers.
- **Leaf Node Capacity:** Every leaf node must contain at least $\lceil m/2 \rceil - 1$ keys and can contain at most $m - 1$ keys.
- **Root Special Case:** The root node is uniquely permitted to violate the lower bound. If it is an internal node, it must have at least 2 children. If the entire tree contains only one node, the root acts as a leaf and may contain between 0 and $m - 1$ keys.

3. Working Principles

Search/Lookup Operation

To find a search key value K , the execution path starts at the root node and traverses down vertically through the internal routing layers:

- Within an internal node containing keys $[K_1, K_2, \dots, K_{k-1}]$, find the smallest key K_i such that $K \leq K_i$.
- If $K < K_i$, follow the child pointer P_i immediately to the left of K_i . If $K \geq K_{k-1}$, follow the rightmost child pointer P_k .
- Repeat this process until a leaf node is reached. Perform a binary or linear search within the leaf node to extract the data record pointer.

Insertion Operation

When a new key is added, it must always be placed into the appropriate leaf node determined by the search path:

- If the leaf node has space (keys $< m - 1$), the key is inserted in sorted order.
- If the leaf node overflows (keys $= m$), it is split into two independent leaf nodes. The first $\lceil m/2 \rceil$ keys remain in the left leaf, and the remaining keys move to the right leaf. The smallest key of the right leaf is **copied up** into the parent internal node to serve as a separator.
- If the parent internal node overflows due to this insertion, it splits into two internal nodes. The middle key is **pushed up** to its parent. This split can propagate recursively up to the root, increasing the height of the tree by 1 when the root splits.

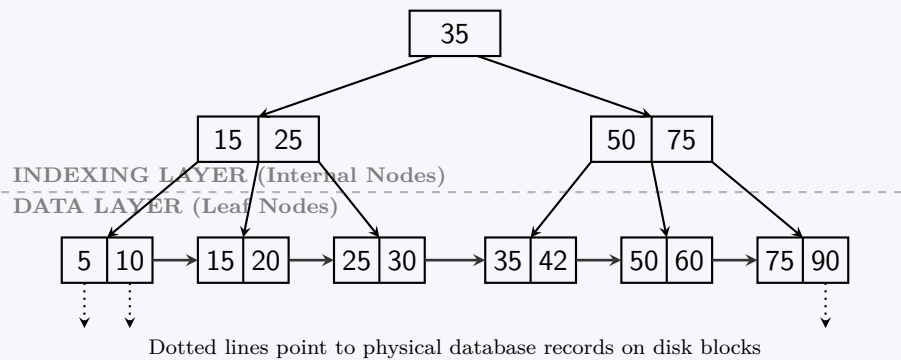
Deletion Operation

When a key is removed from a leaf node:

- If the node drops below its minimum capacity ($\lceil m/2 \rceil - 1$ keys), it attempts to **borrow** a key from an immediate sibling node (redistribution).
- If the sibling is also at its minimum capacity, the underfull node is **merged** with its sibling. The separating key in the parent node is pulled down or adjusted, which may cause the parent node to become underfull, propagating the merge upward.

4. Architectural Advantages in DBMS

- **High Branching Factor (Fan-out):** Because internal nodes do not store actual data values or record pointers, they can pack a massive number of routing keys into a single disk block. A high fan-out m dramatically minimizes the tree height h , restricting disk accesses to a small, constant number (typically 3 or 4 even for millions of records).
- **Efficient Range Queries:** All leaf nodes are linked sequentially via pointer chains (forming a doubly-linked list). While point lookups require a vertical traversal, range scans (e.g., ‘WHERE age BETWEEN 20 AND 30’) simply locate the first valid leaf node and traverse horizontally across the leaf layer without traversing the internal levels again.



5. Structural Comparison: B-Tree vs. B⁺ Tree

The following architectural table contrasts the primary operating differences between the standard B-tree and the B⁺ tree:

Feature	Standard B-Tree	B ⁺ Tree
Data Storage Location	Distributed across both internal & leaf nodes	Isolated entirely within the leaf nodes
Internal Node Function	Stores data records along with routing keys	Acts strictly as an indexing directory
Sequential Traversal	Requires full tree tree-walks ($O(N \log N)$)	Simple linear traversal via linked leaf list
Fan-out Factor	Low (nodes fill quickly with large data records)	Extremely high (nodes contain only compact keys)
Search Complexity	Variable; can finish early at internal layers	Constant time; always runs down to a leaf node

Q12. Compare Hash-based index, B⁺-tree index, and Bitmap index. (6 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

Indexing in Database Management Systems (DBMS) is a critical physical design technique used to optimize query performance by minimizing the number of disk accesses required to retrieve data. Different indexing structures are optimized for specific types of data, query patterns, and update frequencies.

The three primary indexing architectures used in modern relational and analytical databases are the **Hash-based index**, the **B⁺-tree index**, and the **Bitmap index**.

1. Comprehensive Comparison Table				
Parameter		Hash-based Index	B ⁺ -tree Index	Bitmap Index
Data Structure		Array of buckets mapped via a hash function.	Balanced multi-way search tree.	Bit arrays (bit vectors) mapped to row IDs.
Best Query Type		Point queries (Exact match: =).	Range queries ($\leq, \geq$, BETWEEN) and Point queries.	Complex logical queries (AND, OR, NOT).
Time Complexity		$O(1)$ expected for point queries.	$O(\log_b N)$ for point queries.	$O(1)$ bitwise operations.
Data Cardinality		High cardinality (many unique values).	High cardinality (many unique values).	Low cardinality (few unique values, e.g., Gender, Status).
Storage Space		Moderate (pre-allocated bucket arrays).	Moderate to High (due to internal node pointers).	Highly compressed, very small for low cardinality.
Update Cost		Low to Moderate (costly if overflow chains occur).	Moderate (tree rebalancing, node splitting).	Very High (updating requires bit vector rewrites).
Ordering		Data is unordered (random distribution).	Data is strictly sorted by the index key.	Data order is implicitly tied to Row ID mapping.

2. Hash-Based Index

Definition & Working Principle:

A hash-based index uses a mathematical **hash function** $h(K)$ to map search keys (K) directly to a physical storage location called a **bucket**. A bucket is a unit of storage that can hold one or more records. If a bucket fills up, a new bucket is chained to it (Overflow Chaining).

Advantages:

- Extremely fast for exact-match queries (e.g., `SELECT * FROM Emp WHERE EmpID = 101`).
- Simple to implement and requires no tree traversal.

Disadvantages:

- Completely useless for range queries because the hash function scatters adjacent keys randomly.
- Hash collisions** can lead to long overflow chains, degrading performance from $O(1)$ to $O(N)$.

Primary Buckets Overflow Chain

Search Key K

Hash Function
 $h(K) = K \pmod{4}$

Bucket 0

Bucket 1

Bucket 2

Bucket 3

Overflow

3. B⁺-Tree Index

Definition & Working Principle:

The B⁺-tree is a balanced multi-level search tree. Unlike standard B-trees, **all data pointers (or records) are stored strictly in the leaf nodes**. Internal nodes store only routing keys to guide the search. Furthermore, the leaf nodes are connected via a **doubly-linked list**, providing sequential access.

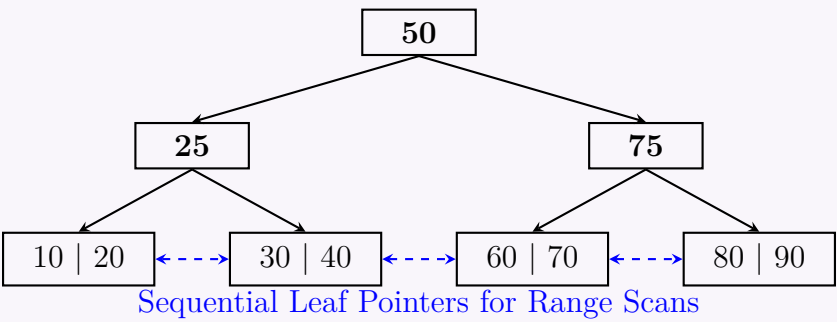
Advantages:

- Excellent for **range queries** (e.g., `SELECT * FROM Sales WHERE Date BETWEEN '2023' AND '2024'`) due to linked leaves.
- Remains balanced; search, insert, and delete operations take logarithmic time $O(\log_b N)$.

Disadvantages:

- Requires more storage space due to internal node overhead.
- Updates (inserts/deletes) can trigger cascading node splits or merges, which is computationally expensive.

302



4. Bitmap Index

Definition & Working Principle:

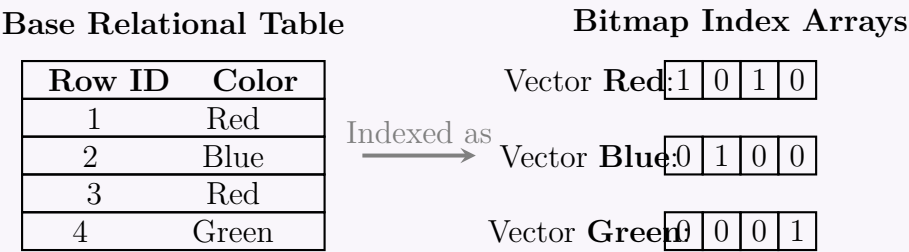
A Bitmap index uses arrays of bits (0s and 1s) to indicate the presence or absence of a specific value in a column for each row in a table. If a table has a column with low cardinality (e.g., Color with values Red, Green, Blue), the index creates one bit-vector per distinct value.

Advantages:

- Highly space-efficient for low cardinality data.
- Unmatched performance for complex Boolean operations. The database can resolve WHERE Color = 'Red' AND Size = 'Large' simply by performing a fast bitwise AND on the respective bit vectors before ever accessing the disk.

Disadvantages:

- Inefficient for high cardinality columns (e.g., Primary Keys, Phone Numbers), as the bit vectors become sparse and enormous.
- **High lock contention and update overhead.** Modifying a single row requires rewriting the bit vectors and locking the entire bitmap, making it unsuitable for OLTP (Online Transaction Processing) systems. It is primarily used in **OLAP (Data Warehouses)**.



Q13. Draw the B-tree to index the following elements: 2, 3, 5, 13, 17, 19, 23, 29, 31, 37, 41, 43, and 47. Delete elements 31 and 43 and redraw the tree. (17 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

Assumption: Since the order of the B-tree is not explicitly specified, we will construct a **B-Tree of Order $m = 3$** (commonly known as a 2-3 Tree). This is the standard instructional model for visualizing B-tree mechanics.

- **Maximum keys per node:** $m - 1 = 2$
- **Minimum keys per internal node:** $\lceil m/2 \rceil - 1 = 1$
- **Maximum children per node:** $m = 3$
- **Insertion behavior:** When a node receives a 3rd key, it **overflows** and splits, promoting the median key to its parent.

1. Initial B-Tree Construction

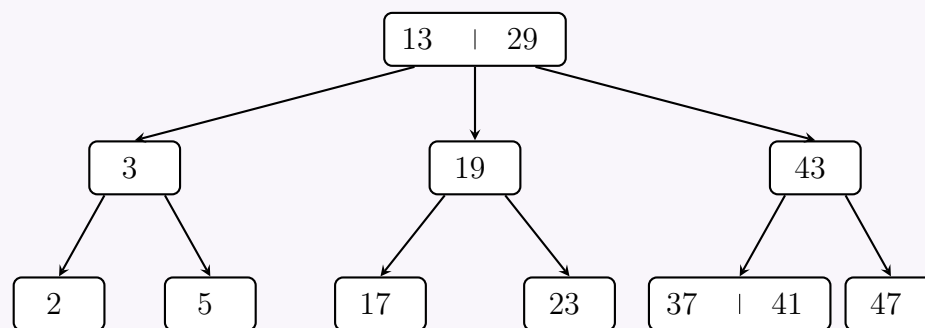
Inserting the monotonically increasing sequence (2, 3, 5, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47) causes sequential splits strictly along the rightmost path of the tree. The final tree perfectly balances all leaves at depth 2.

```
graph TD
    Root["13 | 29"] --> N3["3"]
    Root --> N19["19"]
    Root --> N37["37 | 43"]
    N3 --> L2["2"]
    N3 --> L5["5"]
    N19 --> L17["17"]
    N19 --> L23["23"]
    N37 --> L31["31"]
    N37 --> L41["41"]
    N37 --> L47["47"]
```

2. Deletion of Element 31

Working Principle: Element 31 is located in a leaf node. Removing it leaves the node completely empty, triggering an **underflow** (since the minimum required keys is 1). The database first checks the immediate sibling node [41]. However, borrowing from it would cause the sibling to underflow. Therefore, we must **merge** the empty leaf, its sibling [41], and the separating key from the parent [37].

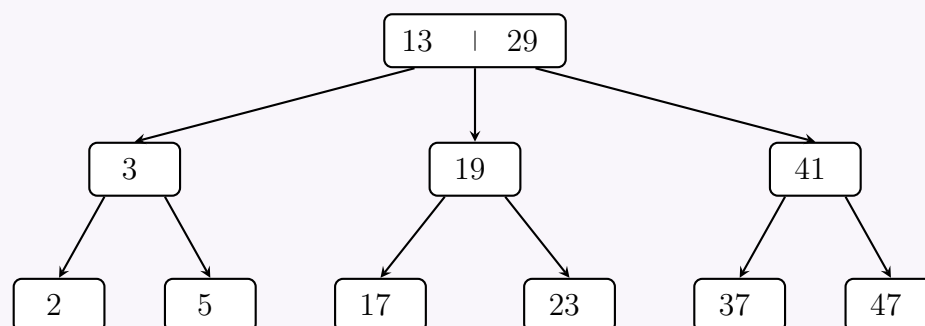
- The newly merged leaf node becomes [37 | 41].
- The parent node loses key 37, dropping to [43], which satisfies the minimum key property.



3. Deletion of Element 43

Working Principle: Element 43 resides in an internal node. Standard B-tree deletion rules state that an internal key should be replaced by its **in-order predecessor** or **in-order successor** from the leaves to preserve the search properties. To prevent immediate underflows, we choose the preceding or succeeding leaf node that possesses more than the minimum number of keys.

- The left sub-tree of 43 ends at leaf [37 | 41], which has 2 keys (allowing a safe extraction).
- The **in-order predecessor** of 43 is 41.
- We replace 43 with 41 in the internal node. The leaf node safely degrades to [37], resulting in a balanced, fully valid B-Tree.



Q14. Draw a B⁺-tree for the following keys: 2, 3, 5, 7, 14, 16, 20, 34, 38, and 40. Assume a 2-order B⁺-tree where a node must have 2 to 4 keys. Re-construct the tree after deleting 3, 5, and 7 from the original tree. **(15 marks)** [CSE 303 | L-3/T-1 | 2011–12]

Answer

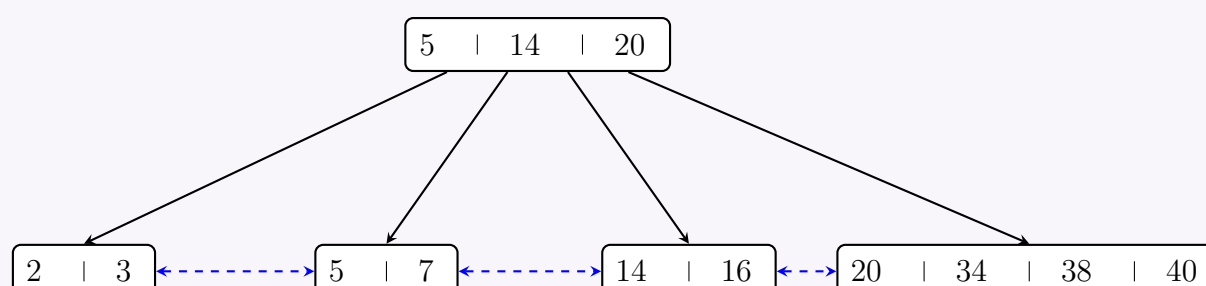
System Assumptions and Constraints:

The problem specifies a B⁺-tree where a node must have **2 to 4 keys**. This defines a B⁺-tree with a maximum branching factor (children) of $m = 5$.

- **Maximum keys per node:** 4
- **Minimum keys per node (except root):** 2
- **Data storage:** All actual values are stored at the leaf level.
- **Internal nodes:** Store only routing keys to guide searches.

1. Original B⁺-tree

Inserting the sequence 2, 3, 5, 7, 14, 16, 20, 34, 38, 40 builds the following structure. Nodes split when they receive a 5th key, pushing the middle key up to the parent.



2. Step-by-Step Deletion Analysis

We must delete 3, 5, and 7 from the original tree, strictly obeying the requirement that any non-root node must have at least 2 keys.

(a) Delete 3:

- Removing 3 from leaf $[2 \mid 3]$ leaves $[2]$. This causes an **underflow** (since it has < 2 keys).
- We check its right sibling $[5 \mid 7]$. Borrowing a key would cause the sibling to underflow.
- Therefore, we **merge** the node with its sibling, forming $[2 \mid 5 \mid 7]$.
- The routing key 5 separating them in the parent is removed. The root becomes $[14 \mid 20]$.

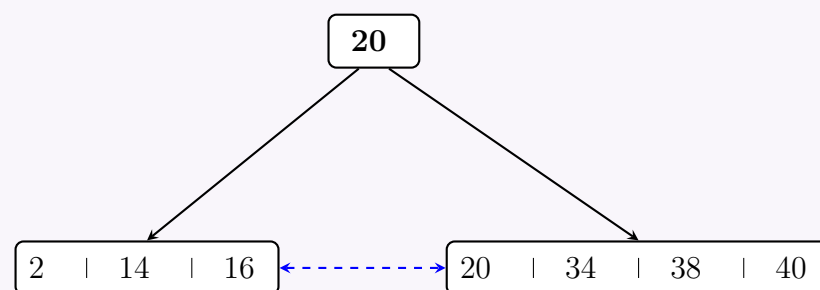
(b) Delete 5:

- We remove 5 from the merged leaf $[2 \mid 5 \mid 7]$, resulting in $[2 \mid 7]$.
- This leaf now has 2 keys, which satisfies the minimum requirement. No further structural changes are needed.

(c) Delete 7:

- Removing 7 from leaf $[2 \mid 7]$ leaves $[2]$. This causes another **underflow**.
- We check its right sibling $[14 \mid 16]$. It has exactly 2 keys, so we cannot borrow.
- We **merge** $[2]$ and $[14 \mid 16]$ to form $[2 \mid 14 \mid 16]$.
- The routing key 14 in the parent is removed. The root degrades to $[20]$. Note that it is completely valid for the root node of a B⁺-tree to have fewer than 2 keys.

3. Final Re-constructed B⁺-tree



Q15. Suppose in a B⁺-tree, pointers are 4 bytes long and keys are 12 bytes long. How many keys and pointers will a block of 16,384 bytes have? For a one million record table, what would be the expected length of the index? **(12 marks)** [CSE 303 | L-3/T-1 | 2011–12]

↪ Similar: CSE 215 | L-3/T-1 | 2012–13 (same pointer/key sizes, 16384 byte block)

Answer

1. Calculation of Keys and Pointers per Block

In a B⁺-tree, the **order (m)** determines the maximum number of pointers (children) a node can have. Consequently, a node can hold a maximum of $m - 1$ keys.

Given Parameters:

- Block Size (B) = 16,384 bytes
- Pointer Size (P) = 4 bytes
- Key Size (K) = 12 bytes

For an internal node, the total size of m pointers and $m - 1$ keys must be less than or equal to the block size. The equation is:

$$\begin{aligned}
 (m \times P) + ((m - 1) \times K) &\leq B \\
 (m \times 4) + ((m - 1) \times 12) &\leq 16384 \\
 4m + 12m - 12 &\leq 16384 \\
 16m &\leq 16396 \\
 m &\leq \frac{16396}{16} \\
 m &\leq 1024.75
 \end{aligned}$$

Since the number of pointers must be an integer, we take the floor value: $m = 1024$.

- **Number of Pointers: 1024**
- **Number of Keys: $1024 - 1 = 1023$**

Internal Node Structure (16,384 Bytes)						
P_1	K_1	P_2	K_2	$\cdots$	K_{1023}	P_{1024}
(4B)	(12B)	(4B)	(12B)	$\cdots$	(12B)	(4B)

2. Expected Length (Size) of the Index for 1,000,000 Records

To calculate the expected length (total storage space) of the index, we must determine the number of blocks required for both the **leaf nodes** and the **internal nodes**.

Assumptions for realistic calculation:

- Data pointers at the leaf level are also 4 bytes.
- Each leaf node contains a pointer to the next leaf node (4 bytes).
- According to B⁺-tree properties, nodes are typically **~69% full** on average (often approximated as 2/3 or 0.69). We will use **69%** for the expected case.

Step A: Leaf Node Capacity

Let L be the maximum number of records (key + data pointer) a leaf node can hold:

$$\begin{aligned} (L \times (K + P_{data})) + P_{next} &\leq B \\ (L \times (12 + 4)) + 4 &\leq 16384 \\ 16L &\leq 16380 \\ L &\leq 1023.75 \implies L = 1023 \text{ records/leaf max.} \end{aligned}$$

Expected records per leaf (69% full) = $0.69 \times 1023 \approx$ **706** records.

Step B: Number of Leaf Blocks

$$\begin{aligned} \text{Total Leaf Blocks} &= \left\lceil \frac{\text{Total Records}}{\text{Expected Records/Leaf}} \right\rceil \\ &= \left\lceil \frac{1,000,000}{706} \right\rceil = \mathbf{1,417 \text{ blocks}} \end{aligned}$$

Step C: Number of Internal Blocks

Internal nodes also have an expected branching factor of $0.69 \times 1024 \approx$ **706** children per node.

- **Level 1 (above leaves):** $\lceil \frac{1417}{706} \rceil =$ **3 blocks**
- **Level 2 (Root):** Since 3 nodes can be pointed to by a single root node, the root level requires **1 block**.

Step D: Total Index Size Calculation

$$\begin{aligned} \text{Total Blocks} &= \text{Leaf Blocks} + \text{Internal Blocks} + \text{Root} \\ &= 1417 + 3 + 1 = \mathbf{1,421 \text{ blocks}} \end{aligned}$$

$$\begin{aligned} \text{Total Expected Length (Size)} &= \text{Total Blocks} \times \text{Block Size} \\ &= 1421 \times 16,384 \text{ bytes} \\ &= 23,281,664 \text{ bytes} \\ &\approx \mathbf{23.28 \text{ MB}} \end{aligned}$$

*Note: If the index is assumed to be **100% packed** (no empty space), it would require $\lceil 1000000/1023 \rceil = 978$ leaf blocks and 1 root block, resulting in a minimum index size of $979 \times 16384 =$ **16.04 MB**.*

Q16. Construct a B⁺ tree for the set of key values {2, 4, 6, 8, 12, 18, 20, 24, 30, 32} with $n = 4$. Assume that the tree is initially empty and values are added in the given order. (10 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

System Assumptions and Constraints:
The problem specifies a B⁺-tree of order $n = 4$.

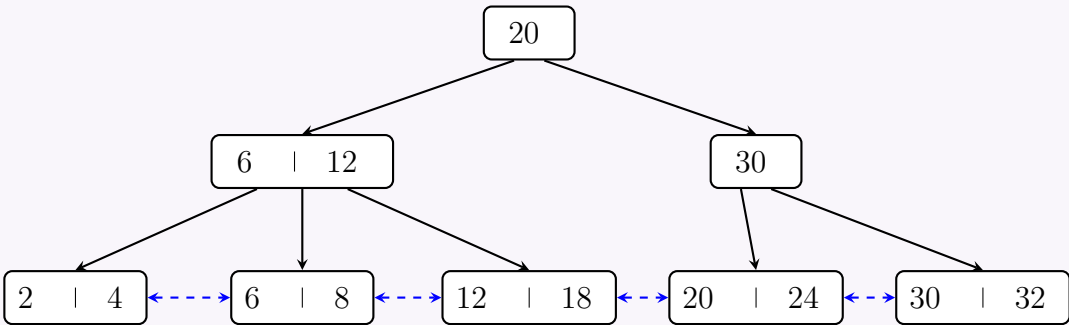
- **Maximum children (pointers) per internal node:** $n = 4$
- **Maximum keys per node:** $n - 1 = 3$
- **Minimum keys per internal node (except root):** $\lceil n/2 \rceil - 1 = 1$
- **Minimum keys per leaf node:** $\lceil (n - 1)/2 \rceil = 1$ (often implemented as $\lfloor n/2 \rfloor = 2$)
- **Insertion Behavior:**

- When a leaf node gets a 4th key, it splits. The first $\lceil 4/2 \rceil = 2$ keys stay in the left node, the remaining 2 go to the right node. The first key of the right node is **copied up** to the parent.
- When an internal node gets a 4th key, it splits. The middle key is **pushed up** to the parent, not copied.

Construction Steps Summary

- (a) **Insert 2, 4, 6:** Node fills up $\rightarrow [2 \mid 4 \mid 6]$.
- (b) **Insert 8:** Overflow! Leaf splits into $[2 \mid 4]$ and $[6 \mid 8]$. 6 is copied up. Root: $[6]$.
- (c) **Insert 12, 18:** 12 goes to right leaf. 18 causes overflow $[6 \mid 8 \mid 12 \mid 18]$. Splits into $[6 \mid 8]$ and $[12 \mid 18]$. 12 copied up. Root: $[6 \mid 12]$.
- (d) **Insert 20, 24:** 20 goes right. 24 causes overflow $[12 \mid 18 \mid 20 \mid 24]$. Splits into $[12 \mid 18]$ and $[20 \mid 24]$. 20 copied up. Root: $[6 \mid 12 \mid 20]$.
- (e) **Insert 30, 32:** 30 goes right. 32 causes leaf overflow $[20 \mid 24 \mid 30 \mid 32]$. Splits into $[20 \mid 24]$ and $[30 \mid 32]$. 30 copied up.
- (f) **Root Overflow:** Pushing 30 up makes the root $[6 \mid 12 \mid 20 \mid 30]$ (4 keys $\rightarrow$ overflow). The root splits. Left gets $[6 \mid 12]$, right gets $[30]$, and 20 is **pushed up** to form a new root.

Final B⁺-tree



Hash Index

Q1. Suppose that extendable hashing is being used on a database file that contains records with the following search key values: 2, 3, 5, 7, 11, 17, 19, and 23. Construct the extendable hash structure for this file if the hash function is $h(x) = x \bmod 7$ and each bucket can hold at most three records. **(15 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

Assumption: We will use the standard implementation of Extendible Hashing where the directory uses the **Least Significant Bits (LSB)** of the binary hash value to determine the routing. A 3-bit binary representation is sufficient since $x \bmod 7$ yields values from 0 to 6.

1. Hash Value Calculations

First, we compute the hash values for each search key using $h(x) = x \bmod 7$ and represent them in 3-bit binary.

Search Key (x)	$h(x) = x \bmod 7$	Binary (3-bit)	LSB reading order (Right-to-Left)
2	2	010	Ends in 0, 10
3	3	011	Ends in 1, 11
5	5	101	Ends in 1, 01
7	0	000	Ends in 0, 00
11	4	100	Ends in 0, 00
17	3	011	Ends in 1, 11
19	5	101	Ends in 1, 01
23	2	010	Ends in 0, 10

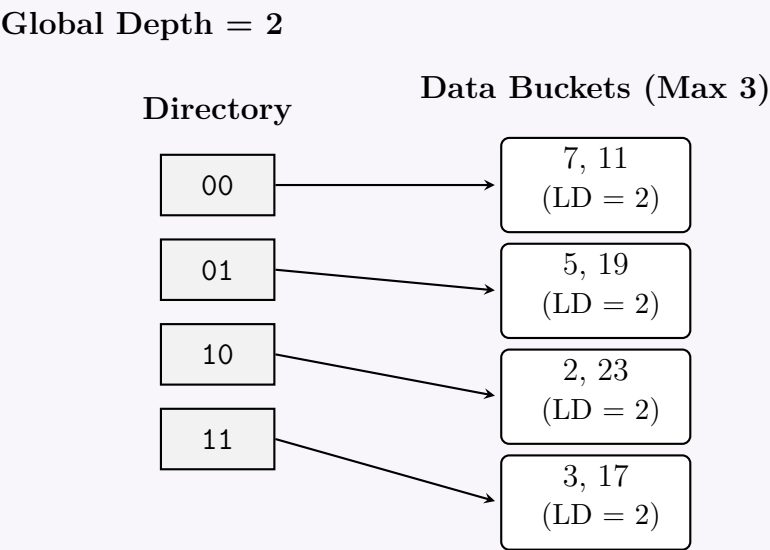
2. Step-by-Step Insertion Process (Bucket Capacity = 3)

- **Initial state:** Global Depth (GD) = 0. One empty bucket with Local Depth (LD) = 0.
- **Insert 2, 3, 5:** All go into the first bucket. The bucket is now full: $[2, 3, 5]$.
- **Insert 7:** The bucket overflows.

- GD increases to 1. Directory uses 1 LSB (0 and 1).
- The bucket splits. Keys ending in 0 go to Bucket A, keys ending in 1 go to Bucket B. Both get LD = 1.
- Bucket A (0): [2, 7]. Bucket B (1): [3, 5].
- **Insert 11 (ends in 0):** Goes to Bucket A. Bucket A is now full: [2, 7, 11].
- **Insert 17 (ends in 1):** Goes to Bucket B. Bucket B is now full: [3, 5, 17].
- **Insert 19 (ends in 1):** Bucket B overflows!
 - GD increases to 2. Directory uses 2 LSBs (00, 01, 10, 11).
 - Bucket B (LD=1) splits into two buckets with LD=2 based on the 2nd LSB.
 - Keys 3 (011) and 17 (011) end in 11 → Bucket B2.
 - Keys 5 (101) and 19 (101) end in 01 → Bucket B1.
 - Bucket A (LD=1) does not split. Both 00 and 10 directory pointers route to Bucket A.
- **Insert 23 (ends in 10):** Directed to Bucket A. Bucket A currently holds [2, 7, 11] and overflows!
 - GD remains 2 (since Bucket A’s LD was 1).
 - Bucket A (LD=1) splits into two buckets with LD=2.
 - Keys 7 (000) and 11 (100) end in 00 → Bucket A1.
 - Keys 2 (010) and 23 (010) end in 10 → Bucket A2.

3. Final Extendible Hash Structure

After all insertions, the Global Depth is 2, and all buckets are perfectly split into a Local Depth of 2 without any remaining overflow.



Q2. Insert all the following data incrementally using the Product Id in an initially empty extensible hashing system using hash function $h(x) = x \bmod 1024$. Show the state of the system after each insertion:

Product Id	Quantity	Price per Qty
03	10	5
05	5	90
09	10	1200
11	12	50
13	11	89
18	2	65

(9 marks)
Insert all the data from the product table above, incrementally using the Product Id in an initially empty linear hashing system. Show the state of the system after each insertion. (12 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

System Assumptions and Configurations:

- **Bucket Capacity:** As it is not explicitly provided, we assume a standard capacity of **2 records per bucket** to effectively demonstrate the splitting mechanics.
- **Data Representation:** For brevity, only the **Product Id (Search Key)** is stored in the diagrams to represent the entire record.

Part 1: Extendible Hashing System

Hash Function: $h(x) = x \bmod 1024$. The result is a 10-bit binary number. Extendible hashing uses the **Least Significant Bits (LSB)** to route keys.

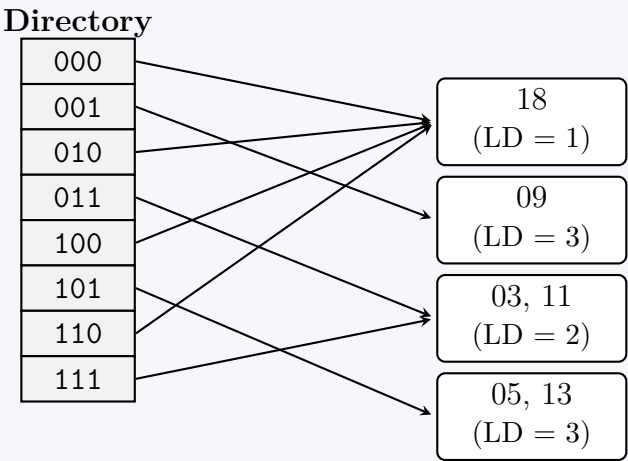
Product Id (x)	Binary (10-bit)	LSB Pattern (Right-to-Left)
03	...00011	Ends in 1, 11, 011
05	...00101	Ends in 1, 01, 101
09	...01001	Ends in 1, 01, 001
11	...01011	Ends in 1, 11, 011
13	...01101	Ends in 1, 01, 101
18	...01010	Ends in 0, 10, 010

Incremental Insertions:

- (a) **Initial State:** Global Depth (GD) = 0. One empty bucket (Bucket A, LD = 0).
- (b) **Insert 03 & 05:** Both are inserted into Bucket A. Bucket A is now full [03, 05].
- (c) **Insert 09:** Bucket A overflows!
- GD increases to 1. Keys 03 (...1), 05 (...1), 09 (...1) all end in 1. They still collide.
 - GD immediately increases to 2 (Directory uses 2 bits).
 - Keys ending in 11 (03) go to Bucket A₁₁ (LD=2).
 - Keys ending in 01 (05, 09) go to Bucket A₀₁ (LD=2).
 - Directory entries 00 and 10 point to an empty Bucket A₀ (LD=1).
- (d) **Insert 11:** Key 11 ends in 11. Inserted into Bucket A₁₁ [03, 11]. No overflow.
- (e) **Insert 13:** Key 13 ends in 01. Inserted into Bucket A₀₁ [05, 09]. Overflow!
- Bucket A₀₁ splits. GD increases to 3. (Directory uses 3 bits).
 - Keys ending in 001 (09) go to Bucket A₀₀₁ (LD=3).
 - Keys ending in 101 (05, 13) go to Bucket A₁₀₁ (LD=3).
- (f) **Insert 18:** Key 18 ends in 010. The Bucket for 0 (LD=1) receives it. No overflow.

Final State of Extendible Hashing:

Global Depth = 3



Part 2: Linear Hashing System

Configuration: Initial buckets $N = 2$. Split pointer $s = 0$. Hash functions are $h_i(x) = x \bmod (2^{i+1})$. We split the bucket pointed to by s whenever **any** bucket overflows.

Incremental Insertions:

- (a) **Initial State:** $N = 2, s = 0$. Buckets B0 and B1 are empty. Base hash $h_0(x) = x \bmod 2$.
- (b) **Insert 03 & 05:** $h_0(03) = 1, h_0(05) = 1$. Both go to B1. *State:* B0[], B1[03, 05].
- (c) **Insert 09:** $h_0(09) = 1$. Goes to B1. **Overflow!**
- Add overflow block to B1: [03, 05] -> [09].
 - Trigger split at $s = 0$. B0 splits using $h_1(x) = x \bmod 4$. Both B0 and new B2 are empty.

- Increment $s = 1$. $N = 3$.

(d) **Insert 11:** $h_0(11) = 1 \geq s$. Goes to B1. **Overflow!**

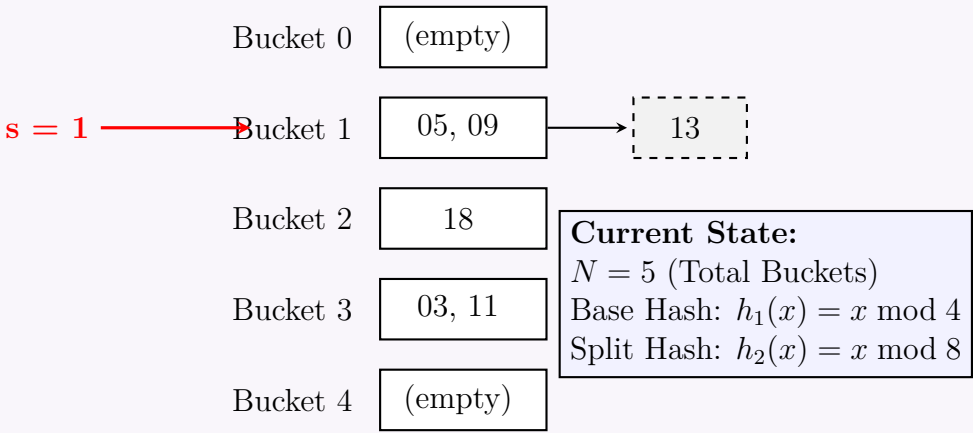
- Add to B1 overflow: [03, 05] -> [09, 11].
- Trigger split at $s = 1$. B1 splits into B1 and B3 using $h_1(x) = x \bmod 4$.
- Rehashing B1 keys: 05 mod 4 = 1 (B1), 09 mod 4 = 1 (B1). 03 mod 4 = 3 (B3), 11 mod 4 = 3 (B3).
- Increment $s = 2$. Since $s = N_{initial}$, the round is over. Reset $s = 0$, N doubles to 4. Base hash is now h_1 .

(e) **Insert 13:** $h_1(13) = 1 \geq s$. Goes to B1. **Overflow!**

- Add overflow block to B1: [05, 09] -> [13].
- Trigger split at $s = 0$. B0 splits using $h_2(x) = x \bmod 8$. B0 and new B4 remain empty.
- Increment $s = 1$. $N = 5$.

(f) **Insert 18:** $h_1(18) = 2 \geq s$. Goes to B2. No overflow. B2 becomes [18].

Final State of Linear Hashing:



Q3. What is the key property of a good hash function? How can you choose such a function? (6 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Key Properties of a Good Hash Function

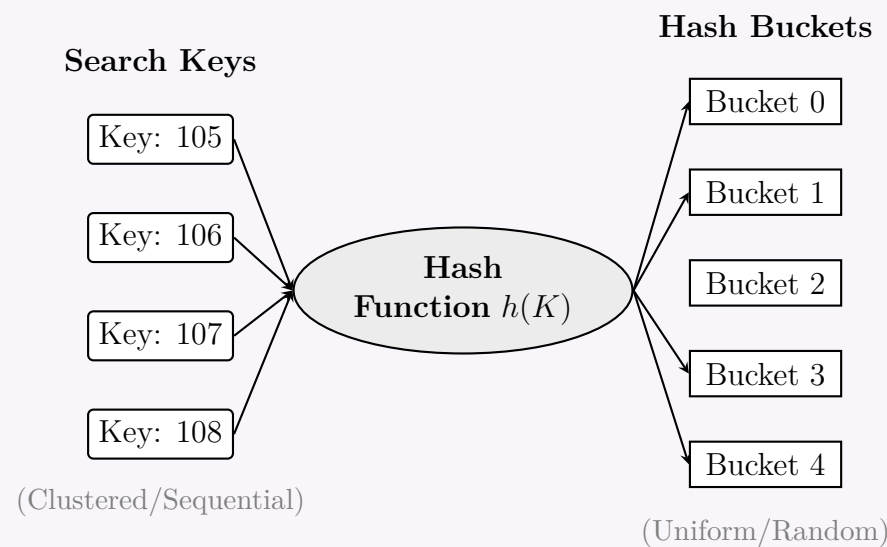
A hash function $h(K)$ maps a search key K to a specific bucket address (or slot) in a hash table. The fundamental goal of a good hash function in a Database Management System (DBMS) is to distribute the data evenly across all available buckets to minimize **collisions** (when multiple keys map to the same bucket).

The two most critical properties of a good hash function are:

- (a) **Uniform Distribution:** The hash function should assign each bucket the same number of records from the set of all possible search keys. Mathematically, for B total buckets, the probability that a randomly chosen key maps to bucket i should be approximately $\frac{1}{B}$ for all $0 \leq i \leq B - 1$.
- (b) **Random Distribution:** The assignment of keys to buckets should appear random, independent of any natural ordering or clustering present in the actual data. Even if the search keys are highly sequential (e.g., employee IDs 101, 102, 103), their hashed bucket addresses should be scattered non-sequentially across the hash table.

Additional necessary properties include:

- **Determinism:** A specific key K must always evaluate to the exact same hash address every time $h(K)$ is computed.
- **Computational Efficiency:** The hash function must be simple and fast to compute; overhead in calculating the hash directly slows down insertion, deletion, and search operations.



2. How to Choose a Hash Function

Choosing a proper hash function depends heavily on the nature of the primary keys (integers, strings, composites) and the expected total number of buckets (M). The most common techniques used in practice include:

- (a) **Division Method (Modulo Arithmetic):** This is the simplest and most widely used hash function.

$$h(K) = K \bmod M$$

Best Practice: The divisor M (number of buckets) should preferably be a **prime number** not close to any exact power of 2 (i.e., $M \neq 2^p$). If M is even, $h(K)$ will have the same parity as K , which destroys uniformity if keys are predominantly even or odd.

- (b) **Multiplication Method:** This method multiplies the key K by a constant real number A in the range ($0 < A < 1$), extracts the fractional part, and multiplies it by M .

$$h(K) = \lfloor M \cdot (K \cdot A \bmod 1) \rfloor$$

Advantage: The choice of M is not critical; it can easily be a power of 2, making computer arithmetic faster. A common heuristic for A is the inverse golden ratio $A \approx 0.6180339$.

- (c) **String Folding/Hashing:** If the key is a string (e.g., a name or alphanumeric ID), characters must be converted to numeric values (like ASCII). A common approach is polynomial string hashing:

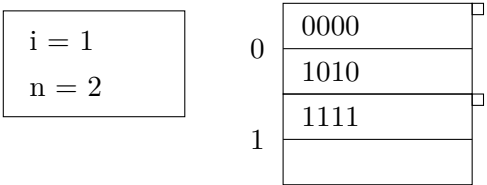
$$h(S) = \left(\sum_{i=0}^{n-1} S[i] \cdot p^i \right) \bmod M$$

Where $S[i]$ is the ASCII value of the i -th character, and p is a prime number (e.g., 31 or 33).

- (d) **Cryptographic Hash Functions (e.g., MD5, SHA-1, SHA-256):** For highly distributed systems (like DynamoDB or Cassandra), cryptographic hash algorithms are used. Though slightly slower, they guarantee near-perfect uniformity and avoid predictability entirely.

Method	Advantages	Disadvantages / Constraints
Division	Extremely fast, simple to implement.	Requires prime M ; sensitive to sequential data if M is poorly chosen.
Multiplication	Works well with $M = 2^p$, handles clustered keys reasonably well.	Slightly slower due to floating-point multiplication.
String Polynomial	Excellent for variable-length alphanumeric strings.	Can be slow for very long strings; must avoid integer overflow.

Q4. The following figure shows a linear hash table with 3 records. We assume hash function h produces 4 bits and we represent records by the value produced by h when applied to the search key of the record. Here, i is the number of lower order bits of the hash function currently used and n is the current number of buckets. Each bucket can hold maximum of 2 records. We shall adopt the policy of choosing n , so that there are no more than $1.7n$ records in the file.



Now, show (step by step) what happens after successive insertion of 0101, 0001 and 0111. Show the necessary calculation whenever a new bucket is required. Overflow blocks can be used if necessary. **(15 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Answer

Initial Configuration and Rules:

- Current State:** Number of buckets $n = 2$, bits used $i = 1$, bucket capacity $c = 2$.
- Total Records (r):** Initially, $r = 3$.
- Threshold Condition:** The file must not exceed $1.7n$ records. If $r > 1.7n$ after an insertion, a new bucket is added (n becomes $n + 1$), and a specific bucket is split.
- Bit Usage (i):** The number of bits $i = \lceil \log_2 n \rceil$.
- Routing Logic:** Let m be the integer value of the i lower-order bits of the search key.
 - If $m < n$, the record belongs to bucket m .
 - If $m \geq n$, the bucket does not exist yet, so the record belongs to bucket $m - 2^{i-1}$.
- Split Logic:** When a new bucket n_{new} is created, the bucket that must be split and rehashed is $n_{new} - 1 - 2^{i-1}$.

Step 1: Insert 0101

(a) Routing:

The lower $i = 1$ bit of 0101 is 1. It is inserted into **Bucket 1**.

(b) Capacity Check:

Bucket 1 is now full with 1111 and 0101. Total records $r = 4$.

(c) Threshold Check:

Maximum allowed is $1.7 \times 2 = 3.4$. Since $4 > 3.4$, a new bucket must be added.

(d) Splitting:

- n increases to 3.
- i becomes $\lceil \log_2 3 \rceil = 2$.
- The new bucket is index $n - 1 = 2$.
- The bucket to split is $2 - 2^{2-1} = 0$. **Bucket 0 splits.**
- Rehash Bucket 0:** Keys are 0000 and 1010. Using $i = 2$ lower bits:
 - 0000 ends in 00 $\implies$ stays in Bucket 0.
 - 1010 ends in 10 $\implies$ moves to Bucket 2.

i = 2
n = 3

0

0000

1

1111

0101

1010

Step 2: Insert 0001

(a) Routing:

The lower $i = 2$ bits of 0001 are 01 (value = 1). It maps to **Bucket 1**.

(b) Capacity Check:

Bucket 1 already has two records (1111, 0101). Thus, 0001 goes into an **overflow block**.

(c) Threshold Check:

Total records $r = 5$. Current threshold is $1.7 \times 3 = 5.1$.

(d) Decision:

Since $5 \leq 5.1$, the threshold is not exceeded. No new buckets are added.

i = 2
n = 3

0

0000

1

1111

0101

1010

0001

312

Step 3: Insert 0111

(a) **Routing:** The lower $i = 2$ bits of 0111 are 11 (value = 3). Since $n = 3$, bucket 3 does not yet exist. We use the alternative route $m - 2^{i-1} = 3 - 2 = 1$. It goes into **Bucket 1's overflow**.

(b) **Threshold Check:** Total records $r = 6$. Current threshold is $1.7 \times 3 = 5.1$. Since $6 > 5.1$, a new bucket must be added.

(c) **Splitting:**

- n increases to 4.
- i remains $\lceil \log_2 4 \rceil = 2$.
- The new bucket is index $n - 1 = 3$.
- The bucket to split is $3 - 2^{2-1} = 1$. **Bucket 1 splits**.
- Rehash Bucket 1 (and its overflow):** Keys are 1111, 0101, 0001, and 0111.
 - 1111 and 0111 end in 11 $\implies$ move to **Bucket 3**.
 - 0101 and 0001 end in 01 $\implies$ stay in **Bucket 1**.
- The overflow block is no longer needed as both buckets now comfortably hold exactly 2 records.

i = 2
n = 4

0

0000

1

0101
0001

2

1010

3

1111
0111

Q5. Construct the dynamic hash index structure on the **balance** attribute for the customer relation (records C0001–C0010, balances 1000–10000). The binary representation of the most significant digit of balance is the hash code. Insert records in descending order of balance.

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0004	N4	North	Dhaka	4000
4	C0005	N5	North	Dhaka	5000
5	C0006	N6	South	Dhaka	6000
6	C0007	N7	South	Dhaka	7000
7	C0008	N8	South	Dhaka	8000
8	C0009	N9	South	Dhaka	9000
9	C0010	N10	South	Dhaka	10000

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Assumptions and Hash Function Formulation

- Dynamic Hash Structure:** We will use **Extendible Hashing**, the standard dynamic hashing technique that uses a directory of pointers and splits buckets on demand.
- Bucket Capacity:** Assuming a standard capacity of **2 records per bucket** to effectively demonstrate dynamic splits.
- Hash Function (h):** The binary representation of the Most Significant Digit (MSD) of the **balance**. We use a 4-bit binary representation. Extendible hashing routes based on the **prefix** (leftmost bits) of this hash code.

Data and Hash Codes:

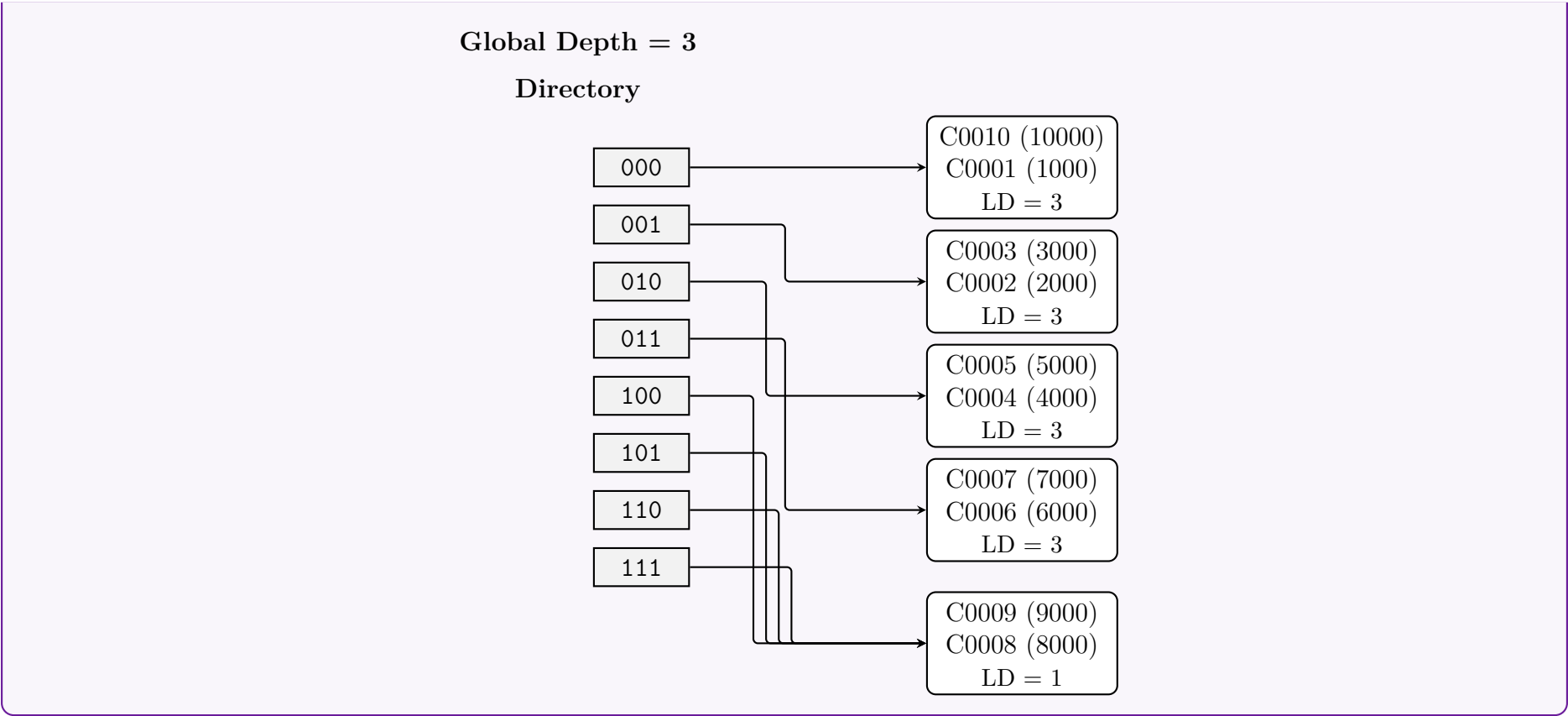
313

Record (CustID)	Balance	MSD	Hash Code (4-bit)	Insert Order
C0010	10000	1	0001	1 (First)
C0009	9000	9	1001	2
C0008	8000	8	1000	3
C0007	7000	7	0111	4
C0006	6000	6	0110	5
C0005	5000	5	0101	6
C0004	4000	4	0100	7
C0003	3000	3	0011	8
C0002	2000	2	0010	9
C0001	1000	1	0001	10 (Last)

2. Step-by-Step Insertion Process

- (a) **Initial State:** Global Depth (GD) = 0. One empty bucket (B).
- (b) **Insert 10000 & 9000:** B gets [10000, 9000]. B is full.
- (c) **Insert 8000:** Overflow! GD increases to 1. Directory uses 1 bit (0, 1).
- Bucket B splits into B0 (Local Depth, LD=1) and B1 (LD=1).
 - 10000 (0001) goes to B0.
 - 9000 (1001) and 8000 (1000) go to B1. B1 is now full.
- (d) **Insert 7000:** 0111 goes to B0. B0 is full: [10000, 7000].
- (e) **Insert 6000:** 0110 goes to B0. Overflow! GD increases to 2 (00, 01, 10, 11).
- B0 splits into B00 (LD=2) and B01 (LD=2).
 - 10000 (0001) goes to B00.
 - 7000 (0111) and 6000 (0110) go to B01. B01 is full.
- (f) **Insert 5000:** 0101 goes to B01. Overflow! GD increases to 3 (Directory: 8 entries).
- B01 splits into B010 (LD=3) and B011 (LD=3).
 - 5000 (0101) goes to B010.
 - 7000 (0111) and 6000 (0110) go to B011.
- (g) **Insert 4000:** 0100 goes to B010. B010 is now full: [5000, 4000].
- (h) **Insert 3000:** 0011 goes to B00 (LD=2). B00 is full: [10000, 3000].
- (i) **Insert 2000:** 0010 goes to B00. Overflow!
- GD remains 3. B00 splits into B000 (LD=3) and B001 (LD=3).
 - 10000 (0001) goes to B000.
 - 3000 (0011) and 2000 (0010) go to B001. B001 is full.
- (j) **Insert 1000:** 0001 goes to B000. B000 is now full: [10000, 1000].

3. Final Dynamic Hash Structure (Extendible Hashing)



Q6. Given the `employee(emp_id, name, gender, status)` relation with hash values on `name`, create step by step a dynamic hash index structure for `name`. Assume a bucket can contain only one record and an overflow bucket can be used.

E001	Abid	Male	Full Time
E002	Arif	Male	Full Time
E003	Sajid	Male	Full Time
E004	Rafiq	Male	Part Time
E005	Shahid	Male	Full Time
E006	Sajid	Male	Full Time
E007	Karim	Male	Part Time
E008	Zahid	Male	Full Time

Figure 1: File containing records of relation `employee(emp_id, name, gender, status)`

Name	16 bit hash code $h(\text{Name})$
Abid	00011111100001111
Arif	00101111100001111
Sajid	00111111100001111
Rafiq	01001111100001111
Shahid	01011111100001111
Karim	10011111100001111
Zahid	01111111100001111

Figure 2: The list of names of figure 1 and corresponding 16 bit hash codes.

(15 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Methodology and Setup

We will construct the dynamic hash index using **Extendible Hashing**, indexing on the **Most Significant Bits (MSBs)** (the prefix) of the 16-bit hash code.

- Bucket Capacity:** 1 record per bucket.
- Overflow:** Used only when records have identical hash keys that cannot be resolved by splitting (e.g., identical names).
- Directory Size:** 2^{GD} , where GD is the Global Depth.

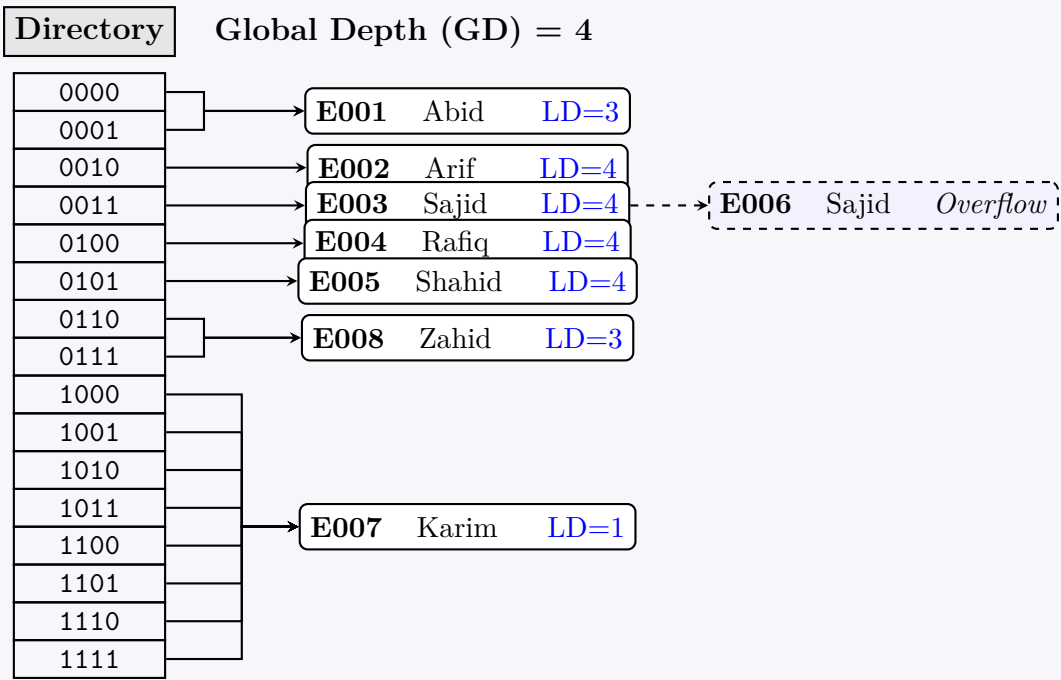
2. Step-by-Step Construction

(a) **Insert E001 (Abid) – Hash: 0001...**

Initial state: Global Depth (GD) = 0. One bucket (Local Depth, LD = 0). Abid is inserted.

- (b) **Insert E002 (Arif) – Hash: 0010...**
Bucket overflows. Since the capacity is 1, we must split the bucket.
- Splitting at GD=1 (bits 0, 1): Both start with 0, collision persists.
 - Splitting at GD=2 (bits 00, ...): Both start with 00, collision persists.
 - Splitting at GD=3 (bits 000, ...): Abid (0001) goes to bucket 000. Arif (0010) goes to bucket 001. Resolved.
- Current State:** GD = 3.
- (c) **Insert E003 (Sajid) – Hash: 0011...**
Prefix 001 routes to Arif’s bucket. Overflow!
- We split the 001 bucket and increase GD to 4.
 - Arif (0010) goes to bucket 0010. Sajid (0011) goes to bucket 0011. Resolved.
- Current State:** GD = 4. Directory has 16 entries (0000 to 1111).
- (d) **Insert E004 (Rafiq) – Hash: 0100...**
Routes to the empty bucket corresponding to prefix 01 (LD=2). Record is inserted.
- (e) **Insert E005 (Shahid) – Hash: 0101...**
Routes to the 01 bucket containing Rafiq. Overflow!
- Split the 01 bucket up to LD=4 to distinguish 0100 (Rafiq) and 0101 (Shahid).
 - The split also creates an empty 011 bucket (LD=3).
- (f) **Insert E006 (Sajid) – Hash: 0011...**
The hash code for Sajid is identical to E003. It routes to bucket 0011. Since no amount of splitting can separate identical hash codes, an **overflow bucket** is created and attached to bucket 0011.
- (g) **Insert E007 (Karim) – Hash: 1001...**
Routes to the empty bucket corresponding to prefix 1 (LD=1). Record is inserted.
- (h) **Insert E008 (Zahid) – Hash: 0111...**
Routes to the empty bucket corresponding to prefix 011 (LD=3). Record is inserted.

3. Final Dynamic Hash Index Structure



Q7. Show the state of the linear hashing after inserting items: 0000, 0011, 0010, 0101, 1010, 1011, 1110, and 1111. Then show the state after inserting 1101 and 0111 into the hash table. (14 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Assumptions and Hashing Parameters

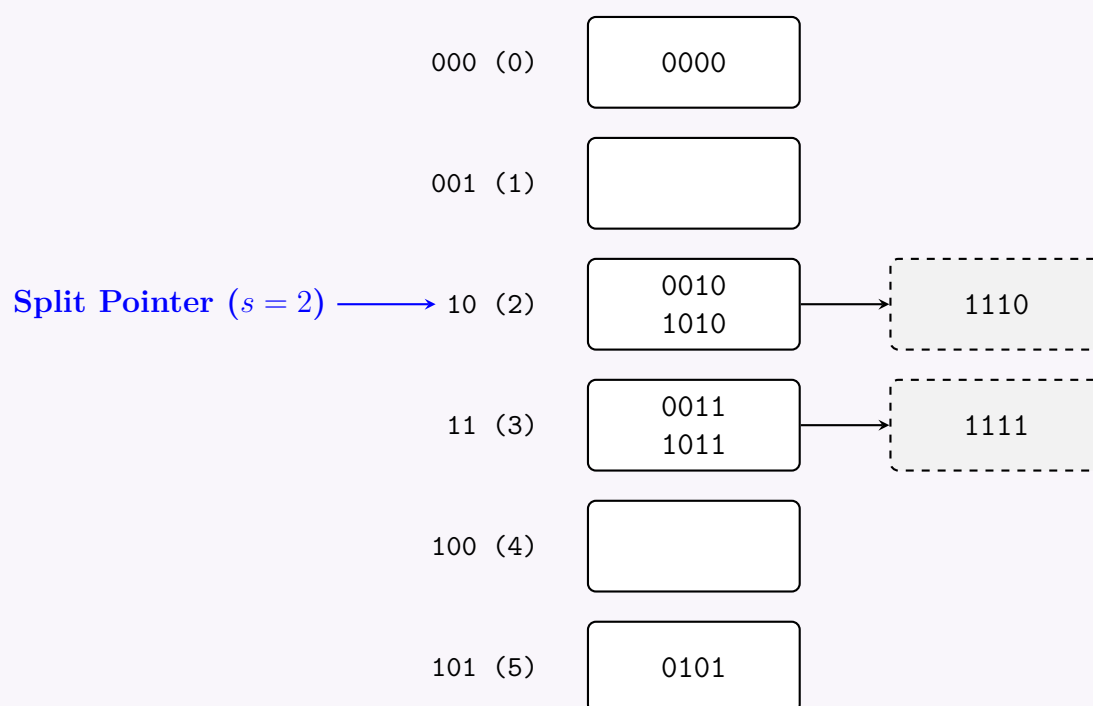
- **Hash Function:** We use the i least significant bits (LSB) of the item. Let $h_i(k) = k \bmod 2^i$.
- **Bucket Capacity (c):** Assumed to be **2 records** per bucket.
- **Split Rule:** A bucket split is triggered at the split pointer s whenever *any* bucket overflows (standard uncontrolled splitting).
- **Initial State:** The hash table starts with $N = 2$ buckets (addresses 0 and 1), level $i = 1$, and split pointer $s = 0$.

2. State 1: After Inserting Initial Sequence

Insertions: 0000, 0011, 0010, 0101, 1010, 1011, 1110, 1111

- **0000, 0011, 0010, 0101:** Fill buckets 0 and 1. Both buckets reach their capacity of 2.
- **1010:** Hashes to bucket 0. Bucket 0 overflows. Trigger split at $s = 0$. Bucket 0 is rehashed using 2 bits (h_2), distributing its records between bucket 0 and a new bucket 2. s increments to 1.
- **1011:** Hashes to bucket 1. Bucket 1 overflows. Trigger split at $s = 1$. Bucket 1 is rehashed into bucket 1 and new bucket 3. s resets to 0, and level i increments to 2.
- **1110:** Hashes to bucket 2 ($h_2(1110) = 2 \geq s$). Bucket 2 overflows. Trigger split at $s = 0$. Bucket 0 is rehashed using 3 bits (h_3), adding a new bucket 4. s increments to 1.
- **1111:** Hashes to bucket 3 ($h_2(1111) = 3 \geq s$). Bucket 3 overflows. Trigger split at $s = 1$. Bucket 1 is rehashed using 3 bits, adding a new bucket 5. s increments to 2.

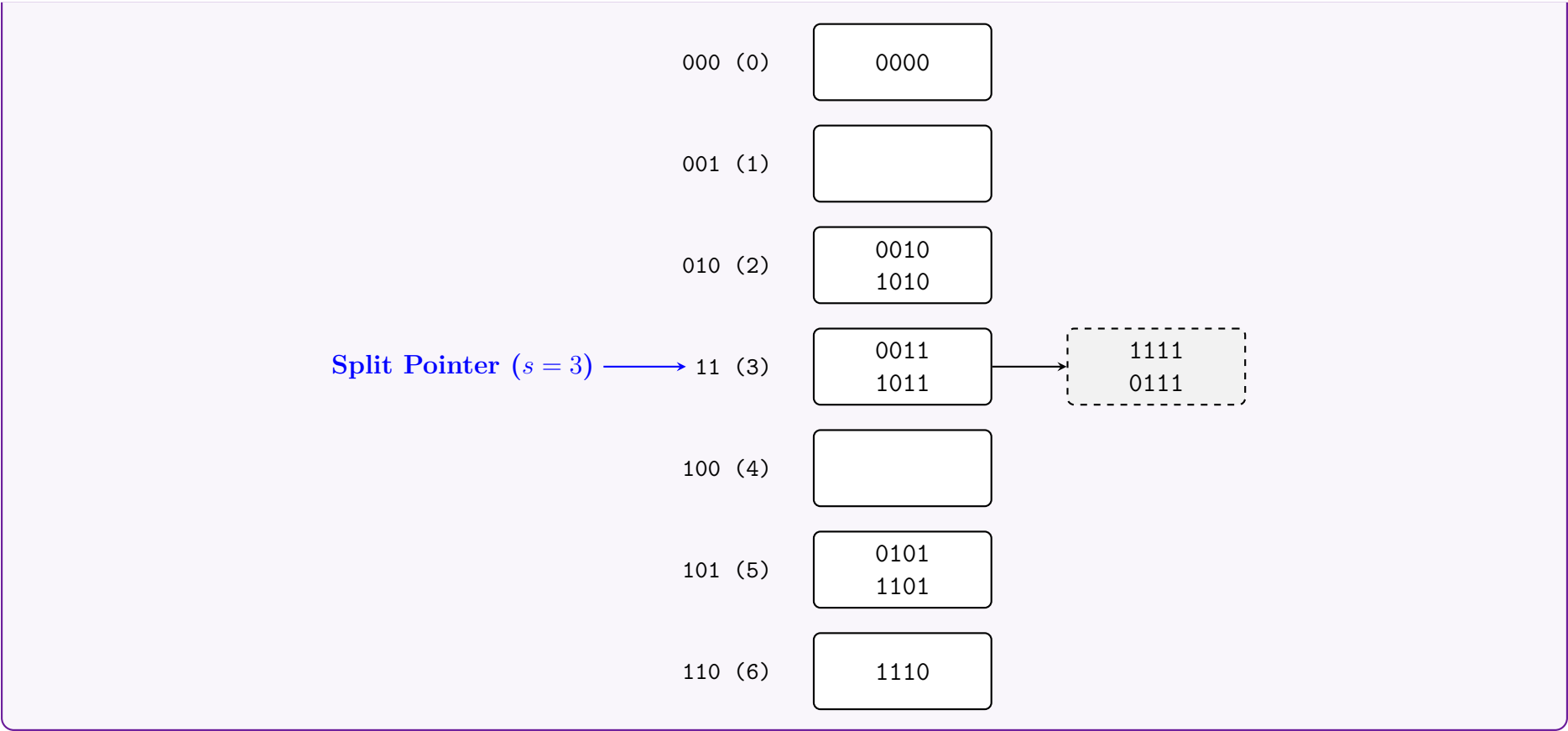
Current State Variables: Level $i = 2$, Split pointer $s = 2$, Number of buckets $N = 2^2 + 2 = 6$.



3. State 2: After Inserting 1101 and 0111

- **Insert 1101:**
 - Compute $h_2(1101) = 01_2 = 1$.
 - Since $1 < s$ ($1 < 2$), bucket 1 has already been split in this round. We must use $h_3(1101) = 101_2 = 5$.
 - The record 1101 is placed in bucket 5. Bucket 5 now contains [0101, 1101]. No overflow occurs.
- **Insert 0111:**
 - Compute $h_2(0111) = 11_2 = 3$.
 - Since $3 \geq s$ ($3 \geq 2$), we use bucket 3.
 - Bucket 3 already contains [0011, 1011] and an overflow block with [1111]. The record 0111 is added to the overflow block, causing an overflow event.
 - **Trigger Split:** The bucket at pointer $s = 2$ (Bucket 2) is split.
 - We rehash Bucket 2's contents (0010, 1010, 1110) using h_3 :
 - * $h_3(0010) = 010_2 = 2 \implies$ Stays in Bucket 2.
 - * $h_3(1010) = 010_2 = 2 \implies$ Stays in Bucket 2.
 - * $h_3(1110) = 110_2 = 6 \implies$ Moves to new Bucket 6.
 - s increments to 3.

Final State Variables: Level $i = 2$, Split pointer $s = 3$, Number of buckets $N = 2^2 + 3 = 7$.



Q8. Discuss Linear Hashing and Extensible Hashing with examples. How do they differ from each other? (13 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

Dynamic hashing techniques are designed to handle growing databases efficiently without requiring frequent, expensive reorganization of the entire hash table. The two most prominent dynamic hashing methods are **Extensible Hashing** and **Linear Hashing**.

1. Extensible Hashing

Definition: Extensible Hashing uses a **directory** of pointers that map to data buckets. The directory size grows exponentially by doubling whenever a bucket overflows and cannot accommodate new records.

Working Principle:

- Uses a hash function that generates a binary sequence. It examines the first d bits (prefix or suffix) of the hash value to place the record, where d is the **Global Depth (GD)**.
- The directory contains 2^{GD} entries.
- Each bucket has a **Local Depth (LD)**, representing the number of bits used to route records to that specific bucket.
- **Splitting:** When a bucket overflows:
 - If $LD < GD$: The bucket is split, its records are rehashed, and its LD is incremented. The directory pointers are updated, but the directory size remains the same.
 - If $LD = GD$: The directory must be **doubled** (GD increments by 1). Then, the overflowing bucket is split, and its LD is incremented.

Example Diagram:

The diagram shows a **Directory ($GD = 2$)** with four entries: 00, 01, 10, and 11. Arrows point from these entries to three buckets:

- 00 points to **Bucket A** ($LD = 2$).
- 01 and 10 point to **Bucket B** ($LD = 1$).
- 11 points to **Bucket C** ($LD = 2$).

In this example, Bucket B has $LD=1$, meaning it only looks at the last bit (1), so both '01' and '11' point to it.

2. Linear Hashing

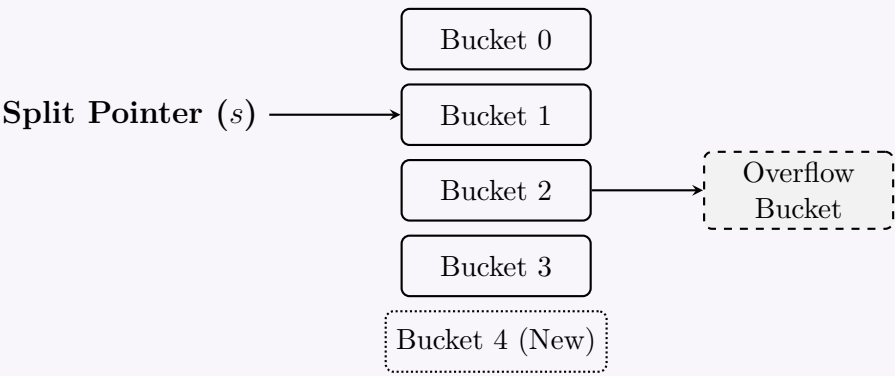
Definition: Linear Hashing allows a hash table to grow dynamically **without a directory**. Instead of doubling, it adds one bucket at a time linearly in a round-robin fashion when an overflow occurs anywhere in the structure.

Working Principle:

- Uses a family of hash functions $h_i(k) = k \bmod (2^i \times N)$, where N is the initial number of buckets, and i is the current split round (level).

- Maintains a **Split Pointer** (s), which keeps track of the next bucket to be split.
- **Splitting:** When *any* bucket overflows, a new bucket is added to the end of the file. The bucket currently pointed to by s is split (rehashed using h_{i+1}), and s moves to the next bucket.
- Overflows are handled temporarily using **overflow chains** until the split pointer reaches and resolves them.
- When s reaches the end of the round, s is reset to 0, and the level i increments.

Example Diagram:



Here, an overflow in Bucket 2 triggered a split, but the split pointer forces Bucket 0 to split into Bucket 4. The overflow in Bucket 2 will be resolved later when s reaches 2.

3. Comparison of Extensible vs. Linear Hashing

Feature	Extensible Hashing	Linear Hashing
Directory Usage	Requires a directory of pointers.	Does not require a directory.
Growth Rate	Directory size doubles (2^{GD}), leading to sudden large space allocations.	Grows linearly, adding one bucket at a time.
Overflow Handling	Resolves overflow immediately by splitting the affected bucket (mostly no overflow pages).	Uses overflow pages temporarily until the split pointer reaches the bucket.
Split Trigger	Triggered when a specific bucket overfills.	Triggered by any overflow, but splitting occurs strictly at the split pointer s .
Hash Function	Single hash function whose binary bits are dynamically evaluated via depth.	Uses a family of dynamically changing hash functions (h_i, h_{i+1}).
Performance	Faster access (exact bucket pointer), but directory doubling causes latency spikes.	Consistent performance; occasional chaining slightly slows down reads.

Q9. Describe static hash file organization with an example. How can bucket overflows be handled? (10 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Static Hash File Organization

Definition: Static hash file organization is a hashing technique where the number of data buckets (or blocks) is fixed when the file is created and does not change as the number of records increases or decreases.

Working Principle:

- A **Hash Function** $h(K)$ maps a search key K to a specific bucket address.
- Let B be the total number of buckets, numbered $0, 1, 2, \dots, B - 1$.
- The hash function maps the domain of search values to the set of bucket addresses:

$$h : K \rightarrow \{0, 1, 2, \dots, B - 1\}$$

- When a record needs to be inserted, retrieved, or deleted, the system computes $h(K)$ to directly locate the bucket containing the record.

Properties:

- **Search Time:** $O(1)$ in the best case, as the bucket address is computed directly.

- **Storage:** A continuous block of primary storage (buckets) is allocated statically.

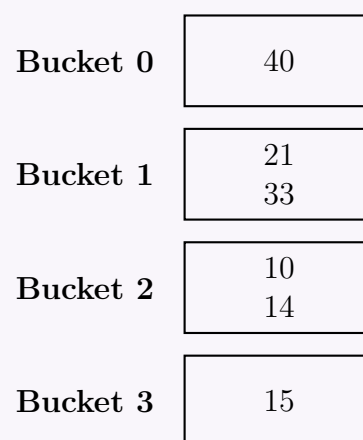
2. Example of Static Hashing

Assume we have a file with $B = 4$ buckets, each capable of holding exactly 2 records. We use a simple modulo hash function:

$$h(K) = K \bmod 4$$

Let us insert the following sequence of keys: **40, 21, 33, 10, 14, 15**.

- $h(40) = 40 \bmod 4 = 0 \implies$ Bucket 0
- $h(21) = 21 \bmod 4 = 1 \implies$ Bucket 1
- $h(33) = 33 \bmod 4 = 1 \implies$ Bucket 1
- $h(10) = 10 \bmod 4 = 2 \implies$ Bucket 2
- $h(14) = 14 \bmod 4 = 2 \implies$ Bucket 2
- $h(15) = 15 \bmod 4 = 3 \implies$ Bucket 3



3. Bucket Overflow and its Causes

A **bucket overflow** occurs when a new record is hashed to a bucket that is already full. In our example, since each bucket holds 2 records, inserting another record that hashes to Bucket 1 or Bucket 2 will cause an overflow.

Causes of Overflow:

- Data Skew:** A disproportionate number of search keys evaluate to the same hash value.
- Insufficient Buckets:** The total number of buckets B is too small for the volume of data being stored.

4. Handling Bucket Overflows

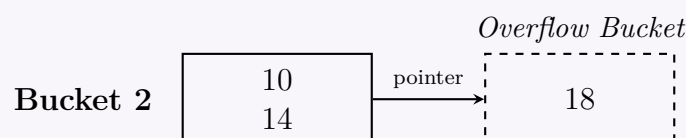
DBMS implementations primarily use the following methods to resolve bucket overflows:

A. Closed Addressing (Overflow Chaining) This is the most common approach used in databases. When a primary bucket becomes full, the system allocates a new *overflow bucket* and links it to the primary bucket using a pointer, creating a linked list (chain).

Example Extension: Insert key **18**.

$$h(18) = 18 \bmod 4 = 2$$

Bucket 2 is full. A new overflow bucket is created and chained to Bucket 2.



B. Open Addressing Instead of using external overflow chains, the system searches for the next available (empty) bucket within the existing fixed hash table to store the overflowing record.

- **Linear Probing:** If $h(K) = i$ is full, the system checks $(i + 1) \bmod B$, $(i + 2) \bmod B$, and so on sequentially.
- **Quadratic Probing:** If $h(K) = i$ is full, the system checks $(i + 1^2) \bmod B$, $(i + 2^2) \bmod B$, etc., to avoid primary clustering.
- **Double Hashing:** Uses a secondary hash function $h_2(K)$ to determine the probe step size, checking buckets at $(i + j \cdot h_2(K)) \bmod B$.

Note: Open addressing is generally avoided in disk-based DBMS because following a probe sequence requires multiple random disk I/O accesses, which drastically degrades performance. Overflow chaining is heavily preferred.

S6 — Query Processing

Query Processing

Q1. Assume that only one tuple fits in a block and memory holds at most three blocks. Show the runs created on each pass of the sort-merge algorithm when applied to sort the following tuples on the first attribute:

(kangaroo, 17), (wallaby, 21), (emu, 1), (wombat, 13), (platypus, 3), (lion, 8),
(warthog, 4), (zebra, 11), (meerkat, 6), (hyena, 9), (hornbill, 2), (baboon, 12).

(10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

External Sort-Merge Algorithm Analysis

The external sort-merge algorithm is used to sort datasets that are too large to fit entirely in main memory. It operates in two main phases: generating sorted initial runs (Pass 0) and merging these runs recursively (Pass 1 to Pass k).

Given Parameters:

- **Total tuples (N):** 12 tuples.
- **Block capacity (B):** 1 tuple per block. Therefore, total blocks = 12.
- **Memory capacity (M):** 3 blocks (can hold 3 tuples at a time).
- **Sorting Key:** First attribute (alphabetical order of animal names).
- **Merge Degree:** Memory holds 3 blocks, leaving $M - 1 = 2$ blocks for input buffers and 1 block for the output buffer. This results in a **2-way merge**.

—

1. Pass 0: Generation of Initial Runs

In Pass 0, we read $M = 3$ blocks of tuples into memory, sort them alphabetically using an in-memory sorting algorithm, and write them out as a single initial run.

- Number of initial runs = $\lceil N/M \rceil = \lceil 12/3 \rceil = 4$ runs.
- Run size = $M = 3$ blocks.

Input Tuples (Unsorted)	Initial Sorted Run (Pass 0)
(kangaroo, 17), (wallaby, 21), (emu, 1)	Run 1: (emu, 1), (kangaroo, 17), (wallaby, 21)
(wombat, 13), (platypus, 3), (lion, 8)	Run 2: (lion, 8), (platypus, 3), (wombat, 13)
(warthog, 4), (zebra, 11), (meerkat, 6)	Run 3: (meerkat, 6), (warthog, 4), (zebra, 11)
(hyena, 9), (hornbill, 2), (baboon, 12)	Run 4: (baboon, 12), (hornbill, 2), (hyena, 9)

—

2. Pass 1: First Merge Pass

In Pass 1, we perform a 2-way merge ($M - 1 = 2$). We read tuples from two runs simultaneously, merge them into sorted order, and write the result to a new run.

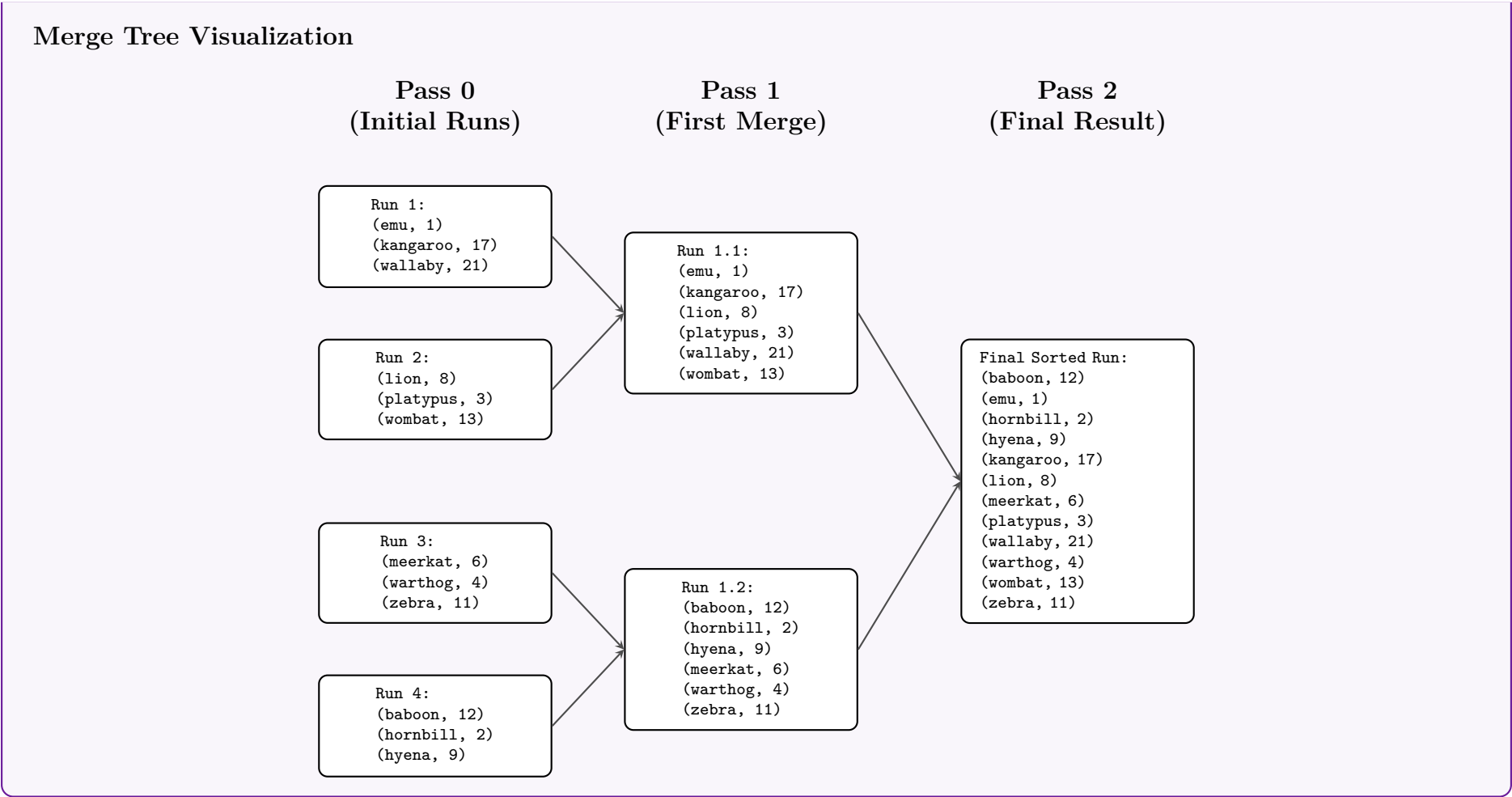
- **Merge Run 1.1** (Merge Run 1 and Run 2):
 - Compare inputs: $emu < lion$, $kangaroo < lion$, $lion < wallaby$, $platypus < wallaby$, $wallaby < wombat$.
 - Resulting Run 1.1: (emu, 1), (kangaroo, 17), (lion, 8), (platypus, 3), (wallaby, 21), (wombat, 13)
- **Merge Run 1.2** (Merge Run 3 and Run 4):
 - Compare inputs: $baboon < meerkat$, $hornbill < meerkat$, $hyena < meerkat$, followed by remaining elements from Run 3.
 - Resulting Run 1.2: (baboon, 12), (hornbill, 2), (hyena, 9), (meerkat, 6), (warthog, 4), (zebra, 11)

—

3. Pass 2: Final Merge Pass

In Pass 2, we merge the two resulting runs from Pass 1 into a single, fully sorted sequence.

- **Merge Final Run** (Merge Run 1.1 and Run 1.2):
 - The entire sequence of 12 tuples is recursively compared and ordered.
 - **Final Output:** (baboon, 12), (emu, 1), (hornbill, 2), (hyena, 9), (kangaroo, 17), (lion, 8), (meerkat, 6), (platypus, 3), (wallaby, 21), (warthog, 4), (wombat, 13), (zebra, 11).



Q2. Describe the two-pass algorithm for grouping and aggregation using sorting. Suppose the number of blocks in relation R is 50,000 and in relation S is 20,000. Compute the memory and I/O cost for the sort-based union operation $R \cup S$. **(14 marks)** [CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Two-Pass Algorithm for Grouping and Aggregation using Sorting

Definition: The two-pass sort-based algorithm groups identical tuples together by sorting the relation based on the grouping attributes. Once the relation is sorted, all tuples belonging to the same group become contiguous, making aggregation straightforward.

Working Principle: Let M be the number of main memory buffers (blocks) available.

(a) **Pass 1 (Create Sorted Runs):**

- Read M blocks of the relation into memory at a time.
- Sort the tuples in memory using the grouping attributes as the sort key.
- Write the sorted tuples back to disk as a **sorted run**.
- Repeat this process until the entire relation is processed. This generates $\lceil B(R)/M \rceil$ sorted runs, where $B(R)$ is the number of blocks in relation R .

(b) **Pass 2 (Merge and Aggregate):**

- Allocate one memory buffer for each sorted run, plus one output buffer.
- Read the first block from each of the sorted runs into the memory buffers.
- Perform a multi-way merge. Because the tuples are merged in sorted order, all tuples with the same grouping attribute values appear consecutively.
- Maintain in-memory accumulators (e.g., *Count*, *Sum*, *Min*, *Max*) for the current group.
- When a tuple with a new grouping attribute value is encountered, write the aggregated result of the previous group to the output buffer, and reset the accumulators for the new group.

```
graph LR; A[Unsorted Relation R] --> B[Pass 1<br/>Sort in Memory]; B --> C[Sorted Run 1]; B --> D[Sorted Run 2]; B --> E[...]; B --> F[Sorted Run k]; C --> G[Pass 2<br/>Merge & Aggregate]; D --> G; E --> G; F --> G; G --> H[Aggregated Result]
```

2. Memory and I/O Cost for Sort-Based Union $R \cup S$

Given Parameters:

- Number of blocks in relation R , $B(R) = 50,000$

- Number of blocks in relation S , $B(S) = 20,000$

The set union operation $R \cup S$ must eliminate duplicate tuples. Using a sort-based approach, we create sorted runs for both R and S in Pass 1, and then merge all runs of R and S simultaneously in Pass 2 while omitting duplicates.

A. Memory Computation (M) To merge all runs of R and S in a single pass (Pass 2), we need one memory block per run of R , one per run of S , and one for the output buffer.

$$M \geq \text{Number of runs of } R + \text{Number of runs of } S + 1$$

$$M \geq \frac{B(R)}{M} + \frac{B(S)}{M}$$

$$M^2 \geq B(R) + B(S)$$

$$M \geq \sqrt{B(R) + B(S)}$$

Substituting the given values:

$$M \geq \sqrt{50,000 + 20,000}$$

$$M \geq \sqrt{70,000} \approx 264.57$$

Since memory blocks must be an integer, the minimum memory required is:

$$M = 265 \text{ blocks}$$

B. I/O Cost Computation The total I/O cost (measured in disk block transfers) consists of:

- (a) **Pass 1 (Read):** Read all blocks of R and S to create sorted runs.
Cost = $B(R) + B(S) = 70,000$ I/Os.
- (b) **Pass 1 (Write):** Write the sorted runs of R and S back to disk.
Cost = $B(R) + B(S) = 70,000$ I/Os.
- (c) **Pass 2 (Read):** Read all sorted runs of R and S for the final merge phase.
Cost = $B(R) + B(S) = 70,000$ I/Os.

Note: The final write of the output is conventionally excluded from the cost of the operator itself.

Total I/O Cost Formula:

$$\text{Total Cost} = 3 \times (B(R) + B(S))$$

$$\text{Total Cost} = 3 \times (50,000 + 20,000)$$

$$\text{Total Cost} = 3 \times 70,000 = 210,000 \text{ disk I/Os}$$

Q3. Describe how aggregation and set operations of an SQL query are processed and whether pipelining can improve their performance. (10 marks) [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Processing of Aggregation Operations

Aggregation operations in SQL (such as SUM, COUNT, AVG, MIN, MAX), often accompanied by a GROUP BY clause, summarize multiple rows into a single result row per group. The DBMS processes these primarily using two techniques:

- **Sort-Based Aggregation:** The input relation is first sorted based on the grouping attributes. Once sorted, all tuples belonging to the same group become contiguous. The database engine then scans the sorted tuples sequentially, computing the aggregate functions in memory, and outputs the result whenever the grouping attribute value changes.
- **Hash-Based Aggregation:** An in-memory hash table is constructed using the grouping attributes as the hash key. As the engine scans the input tuples, it hashes each tuple to find the corresponding bucket and updates the running aggregate values (e.g., keeping a running sum and count for an AVG calculation) on the fly.

2. Processing of Set Operations

Set operations include UNION, INTERSECT, and EXCEPT (or MINUS). By definition, standard SQL set operations must eliminate duplicate tuples.

- **Sort-Based Approach:** Both participating relations are sorted. The query engine then performs a synchronized linear scan (similar to the merge phase of sort-merge) to find matching/non-matching tuples and eliminate duplicates simultaneously.
- **Hash-Based Approach:** The engine builds a hash table on the smaller relation. It then probes this hash table using tuples from the second relation to determine intersections, differences, or unions, while inherently filtering out duplicate entries.
- **Operations with ALL:** If the ALL keyword is used (e.g., UNION ALL), duplicate elimination is bypassed. The engine simply appends the tuples of one relation to the other.

3. Pipelining and its Impact on Performance

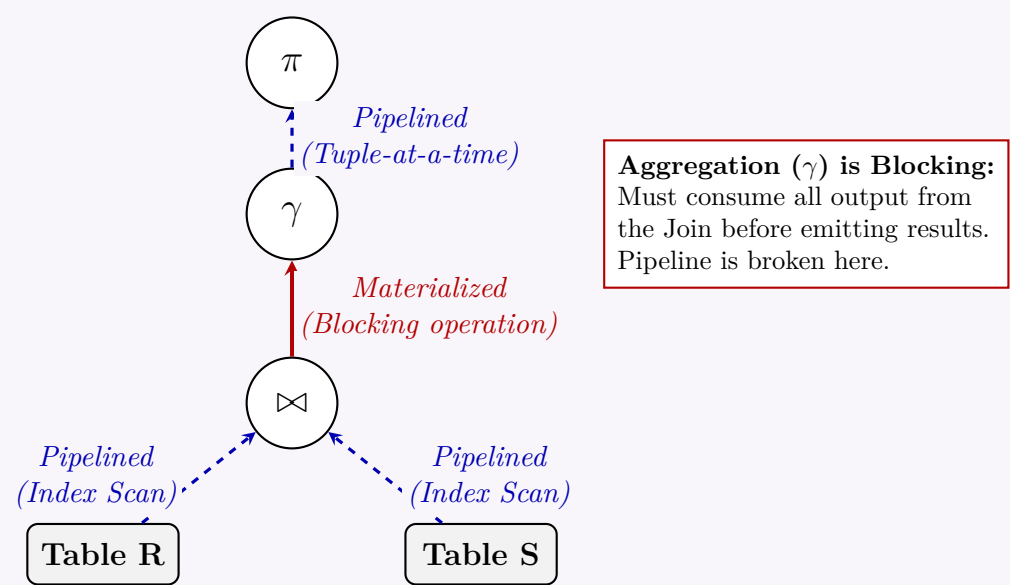
Pipelining is an execution strategy where the output of one relational operator is passed directly to the next operator in the query tree without writing intermediate results to disk (materialization). This significantly improves performance by reducing expensive

disk I/O and lowering response times. However, the ability of pipelining to improve the performance of aggregation and set operations depends heavily on whether the operation is **blocking**:

- **Blocking Operators (Materialization Required):** Most aggregations and set operations are blocking. For example, to calculate a SUM or eliminate duplicates in a UNION, the operator must process *all* tuples in a group or relation before it can guarantee the correct output. Thus, they break the pipeline, forcing the DBMS to materialize the data up to that point.
- **Exceptions allowing Pipelining:**
 - (a) **UNION ALL:** Because no duplicate elimination is required, UNION ALL is fully pipelineable. Tuples from both inputs can stream directly to the output.
 - (b) **Pre-sorted Data:** If the input data is already sorted on the grouping/set attributes (e.g., derived from an Index Scan on a B+ Tree), the sorting phase is skipped. The aggregate or set operation can then run in a pipelined, streaming fashion.

4. Query Execution Tree: Pipelining vs. Blocking

The following diagram illustrates how blocking operations (like standard aggregation) interrupt pipelining, while operations like projections and index scans maintain it.



5. Summary Comparison Table

Operation	Algorithm	Property	Pipelineability
SUM / COUNT	Hashing / Sorting	Blocking	Poor (Pipeline broken unless data is pre-sorted).
UNION / INTERSECT	Hashing / Sorting	Blocking	Poor (Requires full data scans for duplicate removal).
UNION ALL	Append	Streaming	Excellent (Pass-through without holding state).
MIN / MAX	Index Lookup	Non-blocking	Excellent (If supported by B+ Tree index metadata).

- Q4. Memory size is M blocks and the number of blocks of a relation r is b_r . Find the number of runs N , number of merge passes, number of block transfers, and number of seeks to sort the relation r using the external sort-merge algorithm for the following cases:
- (a) Case 1: $N < M$
 - (b) Case 2: $N = M$
 - (c) Case 3: $N > M$

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

External Sort-Merge Algorithm Analysis

The external sort-merge algorithm is employed to sort relations that are larger than the available main memory. To evaluate the cost, we establish standard textbook assumptions (e.g., Silberschatz, Korth, and Sudarshan):

- **Memory capacity (M):** M blocks are available in main memory.
- **Output buffer:** 1 block is reserved for the output buffer during the merge phase, leaving $M - 1$ blocks for input buffers. Thus, we perform an $(M - 1)$ -way merge.

- **Block transfers:** Each pass reads and writes the entire relation, resulting in $2b_r$ block transfers per pass.
- **Seeks:** During run generation, memory is read/written in chunks of M blocks. During the merge phase, blocks are read/written 1 at a time (assuming 1 buffer block per run).

1. General Cost Formulas

Before examining the specific cases, we define the baseline formulas for the algorithm:

- **Number of initial runs (N):** Generated in Pass 0.

$$N = \lceil b_r/M \rceil$$

- **Number of merge passes:** Since we use $(M - 1)$ input buffers, the tree degree is $M - 1$.

$$P = \lceil \log_{M-1}(N) \rceil$$

- **Total block transfers:** 1 pass for run generation + P merge passes. Each pass requires $2b_r$ transfers (read + write).

$$\text{Block Transfers} = 2b_r(1 + P) = 2b_r(1 + \lceil \log_{M-1}(N) \rceil)$$

- **Total seeks:** $2N$ seeks for run generation + $2b_r$ seeks per merge pass.

$$\text{Seeks} = 2N + 2b_r \times P = 2\lceil b_r/M \rceil + 2b_r \lceil \log_{M-1}(N) \rceil$$

2. Evaluation of Specific Cases

Case 1: $N < M$ If $N < M$, it strictly means $N \leq M - 1$ (since N and M are integers). This implies we have enough input buffers to merge all N runs simultaneously in a single pass.

- **Number of runs (N):** $N = \lceil b_r/M \rceil$
- **Number of merge passes:** 1
- **Number of block transfers:** $2b_r(\text{run generation}) + 2b_r(\text{merge pass}) = 4b_r$
- **Number of seeks:** $2N + 2b_r$

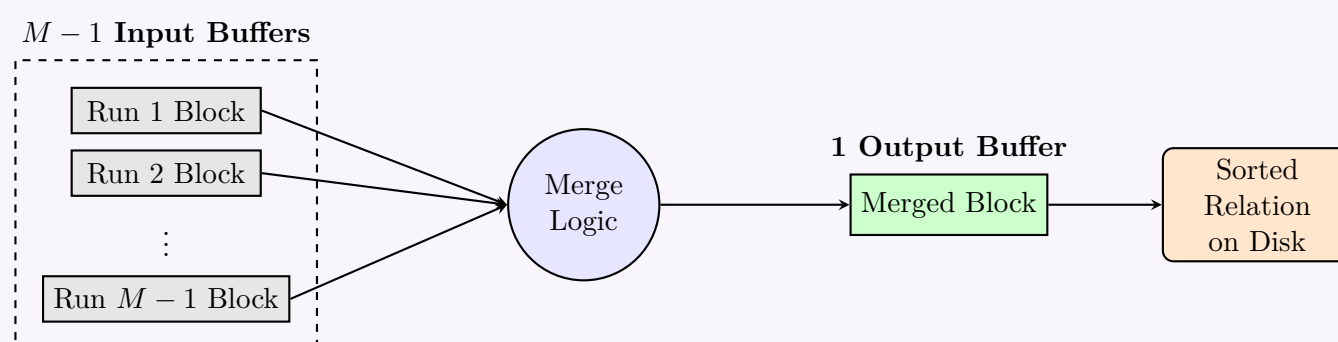
Case 2: $N = M$ Since we only have $M - 1$ input buffers available for merging, we *cannot* merge all M runs in a single pass. We must merge $M - 1$ runs in the first merge pass, leaving 1 run. In the next merge pass, we merge the newly generated run with the left-over run.

- **Number of runs (N):** $N = \lceil b_r/M \rceil = M$
- **Number of merge passes:** $\lceil \log_{M-1}(M) \rceil = 2$
- **Number of block transfers:** $2b_r \times (1 + 2) = 6b_r$
- **Number of seeks:** $2N + 4b_r$

Case 3: $N > M$ This is the general case for very large relations. The runs must be merged recursively over multiple passes.

- **Number of runs (N):** $N = \lceil b_r/M \rceil$
- **Number of merge passes:** $\lceil \log_{M-1}(N) \rceil$
- **Number of block transfers:** $2b_r(1 + \lceil \log_{M-1}(N) \rceil)$
- **Number of seeks:** $2N + 2b_r \lceil \log_{M-1}(N) \rceil$

3. Memory Layout Diagram for $(M - 1)$ -way Merge



4. Summary Comparison Table			
Metric	Case 1: $N < M$	Case 2: $N = M$	Case 3: $N > M$
Initial Runs (N)	$\lceil b_r/M \rceil$	$\lceil b_r/M \rceil$	$\lceil b_r/M \rceil$
Merge Passes	1	2	$\lceil \log_{M-1}(N) \rceil$
Total Block Transfers	$4b_r$	$6b_r$	$2b_r(1 + \lceil \log_{M-1}(N) \rceil)$
Total Seeks	$2N + 2b_r$	$2N + 4b_r$	$2N + 2b_r \lceil \log_{M-1}(N) \rceil$

Q5. An unsorted relation `r_unsorted` has 36 blocks (B1–B36) and the memory size for external sort-merge is 4 blocks. Show the diagram to sort the relation using the external sort-merge algorithm and store it as `r_sorted`. Using the diagram, explain the total number of block transfers needed to sort the relation and store it. **(15 marks)** [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Algorithm Parameters

Given the external sort-merge algorithm parameters:

- **Total blocks (b_r):** 36 blocks (Relation `r_unsorted`)
- **Memory size (M):** 4 blocks
- **Input buffers for merging:** $M - 1 = 3$ blocks (This means we perform a **3-way merge**)
- **Output buffer:** 1 block
- **Initial runs (N):** $\lceil b_r/M \rceil = \lceil 36/4 \rceil = 9$ initial runs.

2. Sort-Merge Process Diagram

The diagram illustrates the external sort-merge process across three passes:

- Pass 0:** Read 4 blocks, sort in memory, write 9 runs. The initial relation `r_unsorted` (36 blocks) is divided into 9 runs, each 4 blocks long (Run 1 to Run 9).
- Pass 1:** 3-way merge: Merge 3 runs of 4 blocks into runs of 12 blocks. Three new runs (Run 1.1, Run 1.2, Run 1.3) are created, each 12 blocks long.
- Pass 2:** 3-way merge: Merge 3 runs of 12 blocks into final relation. The three runs from Pass 1 are merged into the final sorted relation `r_sorted` (36 blocks).

3. Explanation of Total Block Transfers

Every time data is processed in a pass, the DBMS must read the tuples from the disk into main memory and write the processed tuples back to the disk. Therefore, each pass requires reading all 36 blocks and writing all 36 blocks, resulting in $2 \times 36 = 72$ block transfers per pass.

Based on the diagram, the algorithm executes in **3 distinct passes**:

(a) **Pass 0 (Run Generation):** The unsorted relation `r_unsorted` (36 blocks) is read into memory 4 blocks at a time. The blocks are sorted in-memory and written to disk as 9 sorted runs of 4 blocks each.

- Block transfers: 36 (reads) + 36 (writes) = **72** transfers.

(b) **Pass 1 (First Merge Pass):** The memory holds 3 input buffers and 1 output buffer. The DBMS performs a 3-way merge. The 9 initial runs are merged in groups of 3. This reads all 36 blocks and writes 3 new runs of 12 blocks each (Run 1.1, 1.2, 1.3).

- Block transfers: 36 (reads) + 36 (writes) = **72** transfers.

(c) **Pass 2 (Final Merge Pass):** The 3 runs generated in Pass 1 are merged simultaneously (a 3-way merge) to produce the final fully sorted relation `r_sorted`.

- Block transfers: 36 (reads) + 36 (writes) = **72** transfers.

Total Block Transfers:

$$\text{Total} = (\text{Transfers in Pass 0}) + (\text{Transfers in Pass 1}) + (\text{Transfers in Pass 2})$$

$$\text{Total} = 72 + 72 + 72 = \mathbf{216} \text{ block transfers}$$

Note: The mathematical formula for total block transfers validates this result:

$$\text{Total} = 2b_r (1 + \lceil \log_{M-1}(\lceil b_r/M \rceil) \rceil) = 2(36) (1 + \lceil \log_3(9) \rceil) = 72(1 + 2) = \mathbf{216}.$$

Q6. Explain the algorithm used to process the following SQL query given secondary indices on `salary` and `district`:

```
SELECT * FROM employee WHERE salary = 50000 OR district = 'Comilla'
```

(7 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Query Analysis

The given SQL query involves a selection operation with a **disjunctive condition** (an OR clause):

```
SELECT * FROM employee WHERE salary = 50000 OR district = 'Comilla'
```

When a query contains an OR condition, using a single index is usually ineffective because the database engine would still need to perform a full table scan to find tuples satisfying the other half of the condition. Since secondary indices are available on both `salary` and `district`, the Database Management System (DBMS) optimizes this query using the **Record ID (RID) Union Algorithm** (often implemented using Bitmap indices or RID lists).

2. Algorithm: Record ID (RID) Union

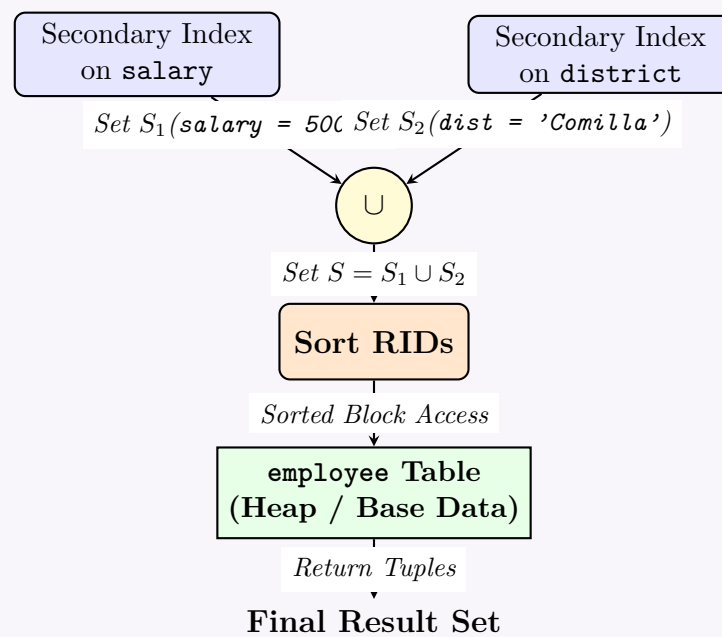
Working Principle: The query processor independently uses the available indices for each part of the disjunction, retrieves the pointers to the matching records, unions these sets to eliminate duplicates, and then fetches the actual data.

The step-by-step execution algorithm is as follows:

- Index Scan 1:** Search the secondary B+ tree index on `salary` for the search key 50000. Retrieve the set of Record IDs (RIDs or row pointers) for all matching tuples. Let this set be S_1 .
- Index Scan 2:** Search the secondary B+ tree index on `district` for the search key 'Comilla'. Retrieve the set of RIDs for all matching tuples. Let this set be S_2 .
- Union of RIDs:** Compute the set union of the two RID lists: $S = S_1 \cup S_2$.
 - This step inherently performs **duplicate elimination**. If an employee happens to have both a salary of 50,000 and lives in 'Comilla', their RID will appear in both S_1 and S_2 . The union ensures their record is only fetched once.
- Sort RIDs:** Sort the resulting set S based on the physical block/page addresses of the RIDs.
 - *Why?* Secondary indices are not physically clustered. Sorting the RIDs ensures that when the data blocks are accessed, the disk heads move sequentially rather than randomly, minimizing expensive disk seek times (Random I/O $\rightarrow$ Sequential I/O).
- Table Fetch:** Fetch the actual data blocks from the `employee` relation using the sorted RIDs in S , and return the projected tuples to the user.

3. Visualizing the Query Execution Plan

The following query tree and physical execution plan illustrates the process:



4. Properties and Performance Considerations

- **Efficiency:** This algorithm is highly efficient when the selectivity of the combined condition is high (i.e., the result set is a small fraction of the total table). It avoids scanning the entire relation block by block.
- **Fallback to Table Scan:** If the query optimizer determines (using statistical metadata) that the combination of 50,000 salary and 'Comilla' district applies to a large percentage of the table (e.g., > 10-15%), the overhead of index lookups, union, and sorting might exceed the cost of a simple Full Table Scan. In such cases, the optimizer will bypass the indices entirely.
- **Bitmap Indices:** In data warehousing environments, if **salary** and **district** are low-cardinality attributes, the DBMS might use Bitmap Indices. The Union operation is then executed as a highly optimized, bitwise OR operation between the two bitmaps, which is extremely fast at the CPU level.

Q7. Write the steps of a one-pass algorithm for finding $S - R$, where the number of records in relation R is smaller than the number of records in relation S . (6 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Prerequisites and Algorithm Selection

The set difference operation $S - R$ returns all tuples that are present in relation S but absent from relation R . A **one-pass algorithm** for this operation reads each relation from the disk exactly once. Since the query requires a one-pass evaluation, at least one of the relations must fit entirely in main memory.

Given that the number of records (and consequently, the number of disk blocks) in relation R is smaller than in relation S , we choose R as the **inner relation** to be loaded into main memory.

Let:

- M be the total number of available main memory buffer blocks.
- $B(R)$ be the number of blocks occupied by relation R .
- $B(S)$ be the number of blocks occupied by relation S .

Condition for One-Pass Algorithm: $M > B(R)$. We require $B(R)$ blocks to store R , plus at least 1 block to stream S , and 1 block for the output buffer.

2. Steps of the One-Pass Algorithm

The execution proceeds in two distinct phases:

Phase 1: Build Phase (Process Relation R)

- Read the smaller relation R entirely from the disk into the main memory, one block at a time.
- Construct an efficient in-memory search structure, such as a **Hash Table** or a balanced binary search tree, containing all tuples of R . The entire tuple serves as the search key.

Phase 2: Probe Phase (Process Relation S)

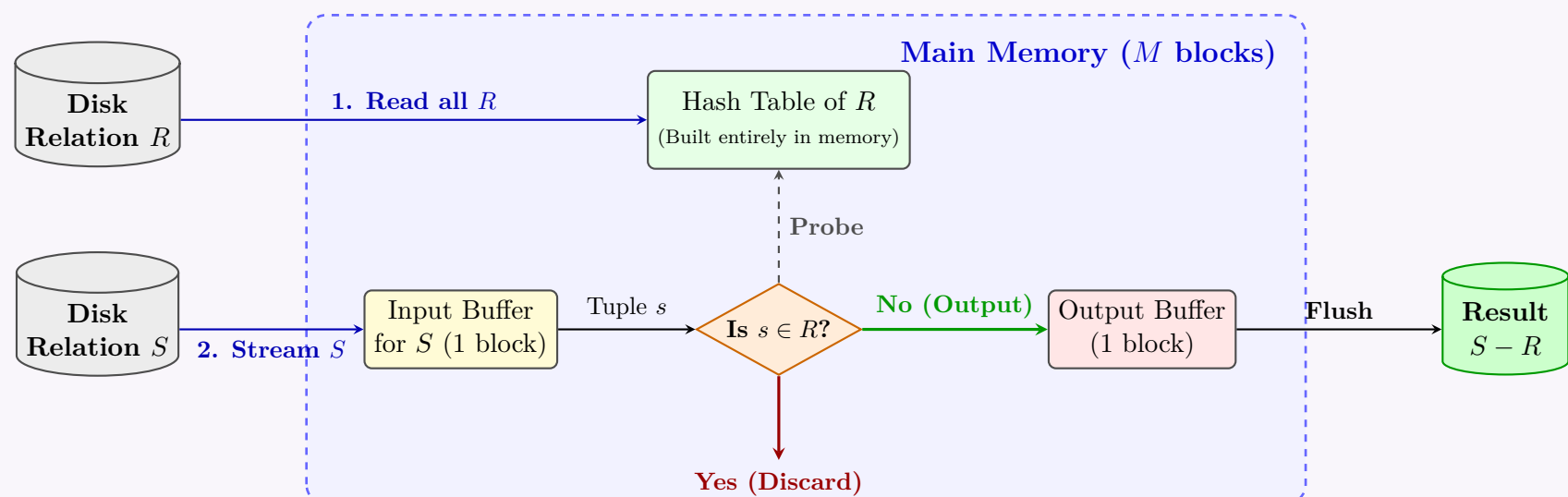
- Read the larger relation S from the disk into the remaining main memory space, streaming it **one block at a time**.
- For each tuple s in the current block of S , probe the in-memory search structure of R to check if an identical tuple exists.
- Decision Logic:**
 - If $s \in R$ (a match is found): **Discard** the tuple s .
 - If $s \notin R$ (no match is found): **Append** the tuple s to the output buffer.

- (d) Once the output buffer is full, write it to the disk as part of the result relation $S - R$, and clear the buffer.
- (e) Repeat steps 3–6 until all blocks of relation S have been processed.

3. Cost Analysis

- **I/O Cost:** $B(R) + B(S)$ block transfers. (Each block of R and S is read exactly once).
- **Memory Required:** $B(R) + 2$ blocks minimum ($B(R)$ for Hash Table, 1 for S input buffer, 1 for output buffer).

4. Visualizing the One-Pass Execution



Q3. Write down the pseudocode for a sort-based union algorithm that computes $R \cup S$. Also mention its runtime and constraints in terms of $B(R)$, $B(S)$ and M , where $B(R)$ and $B(S)$ are the number of blocks in R and S , and M is the number of blocks of each sublist to be sorted. (12 marks) [CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Pseudocode for Sort-Based Union Algorithm

The sort-based union algorithm operates by first sorting both relations and then merging them while eliminating duplicates. Assuming the relations are sorted in ascending order, the pseudocode is as follows:

Algorithm: SortBasedUnion(R, S)

Input: Unsorted relations R and S

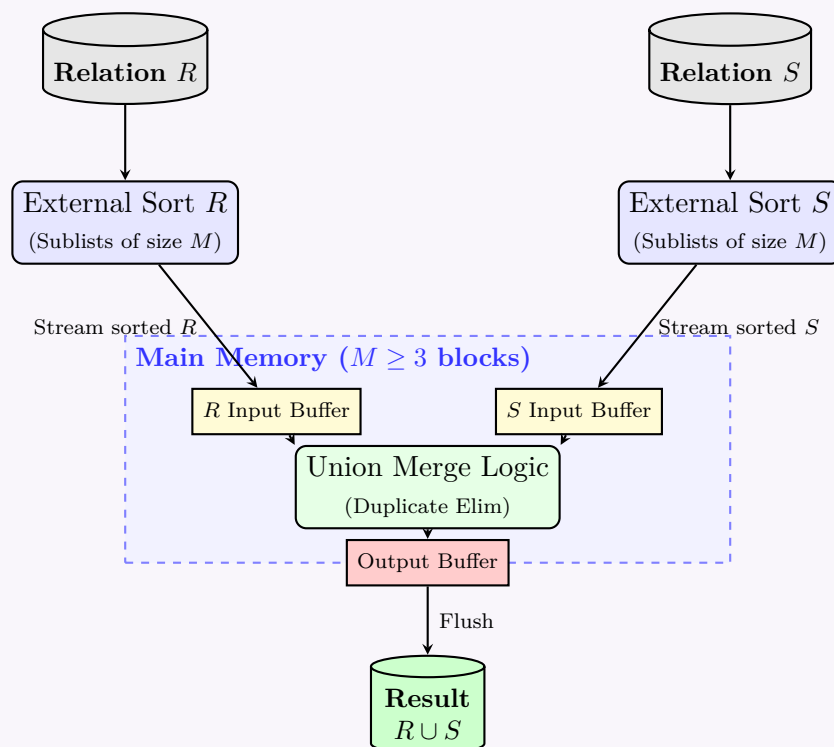
Output: Set union $R \cup S$ (with no duplicates)

- Sort** relation R using external sort to produce $Sorted_R$.
- Sort** relation S using external sort to produce $Sorted_S$.
- Allocate one main memory buffer block for $Sorted_R$, one for $Sorted_S$, and one for the Output.
- Initialize pointer p_R to the first tuple of $Sorted_R$ and p_S to the first tuple of $Sorted_S$.
- While** ($p_R \neq \text{NULL}$ and $p_S \neq \text{NULL}$) **do**:
 - If** ($p_R.\text{tuple} < p_S.\text{tuple}$):
 - Write $p_R.\text{tuple}$ to the Output buffer.
 - Advance p_R , skipping any consecutive duplicate tuples in $Sorted_R$.
 - Else If** ($p_R.\text{tuple} > p_S.\text{tuple}$):
 - Write $p_S.\text{tuple}$ to the Output buffer.
 - Advance p_S , skipping any consecutive duplicate tuples in $Sorted_S$.
 - Else** (*Tuples are identical*):
 - Write $p_R.\text{tuple}$ to the Output buffer.
 - Advance p_R , skipping any consecutive duplicates.
 - Advance p_S , skipping any consecutive duplicates.
- While** ($p_R \neq \text{NULL}$) **do**: (*Append remaining elements of R*)
 - Write $p_R.\text{tuple}$ to the Output buffer.
 - Advance p_R , skipping duplicates.

(g) **While** ($p_S \neq \text{NULL}$) **do**: (Append remaining elements of S)

- (i) Write $p_S.\text{tuple}$ to the Output buffer.
- (ii) Advance p_S , skipping duplicates.

2. Conceptual Diagram of Execution



3. Runtime (Cost Analysis)

The total cost is calculated in terms of block transfers (I/Os). Let $B(R)$ and $B(S)$ be the number of blocks in relations R and S respectively.

- **Standard Unoptimized Execution:**

- Sorting R requires $4B(R)$ transfers (read/write for initial runs + read/write for merge pass).
- Sorting S requires $4B(S)$ transfers.
- The final union merge requires reading both sorted relations: $B(R) + B(S)$ transfers.
- **Total Cost** = $5(B(R) + B(S))$ block transfers.

- **Optimized Two-Pass Execution (Commonly assumed):**

- In the first pass, create sorted sublists (runs) of size M for both R and S . This requires $2B(R) + 2B(S)$ block transfers (1 read, 1 write per block).
- Instead of doing a second pass to fully sort R and S independently and then a third pass to union them, the algorithm **combines the final sorting merge phase and the union phase** into a single pass.
- The second pass reads all the runs of R and S simultaneously and performs the union. This takes $B(R) + B(S)$ reads.
- **Total Optimized Cost** = $3(B(R) + B(S))$ block transfers. (Note: This cost typically excludes the final write of the output relation, as is standard in query processing analysis).

4. Memory Constraints

The memory constraints depend on the available buffer blocks M and the variant of the algorithm being executed:

(a) **Constraint for the Union Phase (Post-Sorting):** If R and S are already fully sorted, the merge phase strictly requires a minimum of 3 blocks of memory.

$$M \geq 3 \quad (1 \text{ for } R, 1 \text{ for } S, 1 \text{ for Output})$$

(b) **Constraint for Optimized Two-Pass Union:** To combine the final sorting pass and the union operation into a single step, all initial sorted sublists (runs) of both R and S must be merged simultaneously.

- Number of runs for $R = \lceil B(R)/M \rceil$
- Number of runs for $S = \lceil B(S)/M \rceil$
- Total input buffers required for the runs is $\lceil B(R)/M \rceil + \lceil B(S)/M \rceil$.
- Including 1 block for output, the memory constraint formula becomes:

$$\frac{B(R) + B(S)}{M} + 1 \leq M$$

For large relations, this approximately implies that the square of the available memory must be greater than or equal to the combined size of the relations:

$$B(R) + B(S) \leq M^2$$

Q4. Explain external sort-merge with an example. Show that the total number of block transfers for external sorting of a relation r is:

$$b_r \left(2 \left\lceil \log_{M-1} \left(C \frac{b_r}{M} \right) \right\rceil + 1 \right)$$

where b_r is the number of blocks of relation r and M is the size of the main memory buffer. **(15 marks)** [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. External Sort-Merge: Definition and Working Principle

External Sort-Merge is a disk-oriented sorting algorithm designed to sort relations (tables) that are too large to fit entirely into main memory. It works on the principle of **divide and conquer**, dividing the relation into smaller chunks that fit in memory, sorting them, and then systematically merging them.

Algorithm Steps

Let M be the number of main memory buffer blocks available, and b_r be the total number of disk blocks of relation r .

(a) Pass 0 (Run Generation):

- Read M blocks of the relation from the disk into main memory.
- Sort these M blocks in memory using any internal sorting algorithm (e.g., Quicksort).
- Write the sorted data back to the disk. This sorted chunk is called a **run**.
- Repeat until the entire relation is processed. This generates $N = \lceil b_r/M \rceil$ initial runs.

(b) Merge Passes (Pass 1, 2, ...):

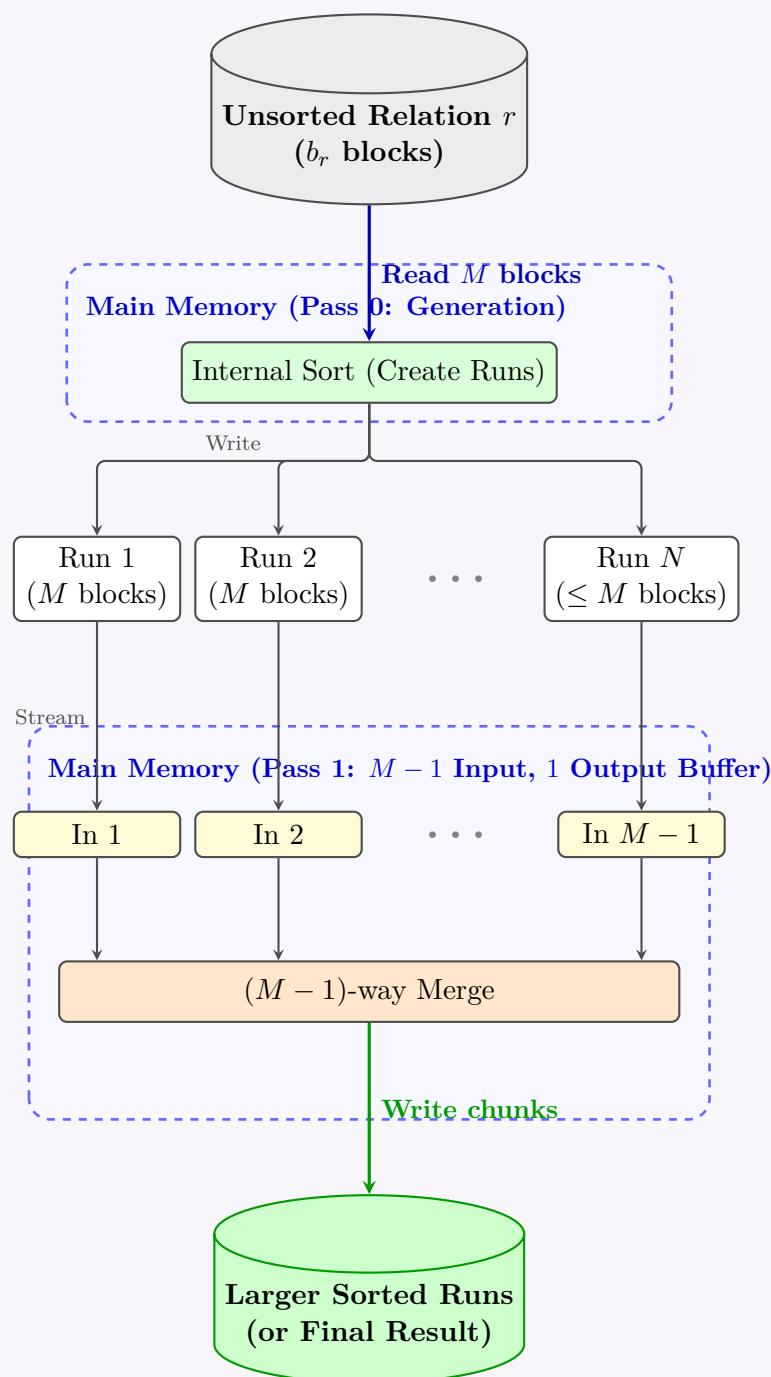
- Allocate $M - 1$ buffer blocks for input (to read from $M - 1$ runs simultaneously) and 1 buffer block for output.
- Perform an $(M - 1)$ -way merge. Read the first blocks of $M - 1$ runs.
- Merge the tuples and write them to the output buffer. When the output buffer is full, flush it to the disk to form a new, larger sorted run.
- Repeat this process until all initial runs are merged into a single sorted relation.

2. Example

Consider a relation r with $b_r = 108$ blocks and main memory buffer $M = 5$ blocks.

- **Pass 0 (Run Generation):** We read 5 blocks at a time, sort them, and write them out. This produces $\lceil 108/5 \rceil = 22$ initial runs (21 runs of 5 blocks, 1 run of 3 blocks).
- **Pass 1 (Merge):** We can merge $M - 1 = 4$ runs at a time. The 22 initial runs are merged into $\lceil 22/4 \rceil = 6$ larger runs.
- **Pass 2 (Merge):** The 6 runs are merged 4 at a time to produce $\lceil 6/4 \rceil = 2$ even larger runs.
- **Pass 3 (Final Merge):** The last 2 runs are merged to produce the final sorted relation.

3. Diagrammatic Representation



4. Proof of Total Block Transfers

We evaluate the number of block transfers (I/O operations) required for the external sort-merge algorithm. Let the final output be pipelined directly to the user or next query operator (i.e., the very last merge pass is **not** written back to disk).

(a) **Pass 0 (Run Generation Cost):**

- The entire relation of b_r blocks is read into memory.
- The sorted runs (totaling b_r blocks) are written to disk.
- Block transfers for Pass 0 = b_r (reads) + b_r (writes) = $2b_r$.

(b) **Number of Initial Runs:** $N = \lceil b_r/M \rceil$.

(c) **Number of Merge Passes:**

- Each merge pass reduces the number of runs by a factor of $M-1$.
- Total number of merge passes $K = \lceil \log_{M-1}(N) \rceil = \lceil \log_{M-1}(\frac{b_r}{M}) \rceil$.

(d) **Cost of Merge Passes:**

- Each standard merge pass reads all b_r blocks and writes all b_r blocks back to the disk. Cost per standard pass = $2b_r$.
- However, for the **final** merge pass, if the result is pipelined (as implied by the formula), we only *read* the b_r blocks but do not write them back to disk. Cost of the final pass = b_r .
- Total cost for all K merge passes = $2b_r(K-1) + b_r = 2b_rK - 2b_r + b_r = 2b_rK - b_r$.

(e) **Total Block Transfers:**

$$\begin{aligned}
 \text{Total Transfers} &= \text{Cost(Pass 0)} + \text{Cost(Merge Passes)} \\
 &= 2b_r + (2b_rK - b_r) \\
 &= 2b_rK + b_r \\
 &= b_r(2K + 1)
 \end{aligned}$$

Substituting $K = \lceil \log_{M-1} \left(\frac{b_r}{M} \right) \rceil$ into the equation:

$$\text{Total Transfers} = b_r \left(2 \left\lceil \log_{M-1} \left(\frac{b_r}{M} \right) \right\rceil + 1 \right)$$

(Note: The variable C in the provided prompt's expression $C \frac{b_r}{M}$ is a constant evaluated as 1 in standard analysis.)

Q5. Let relation $r_1(A, B, C)$ and $r_2(C, D, E)$ have the following properties: r_1 has 20,000 tuples, r_2 has 45,000 tuples, 25 tuples of r_1 fit on one block, and 30 tuples of r_2 fit on one block. Estimate the number of block transfers and seeks required using:

- (a) Nested loop join.
- (b) Block nested loop join.

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Initial Calculations

Before calculating the join costs, we must determine the number of blocks (pages) each relation occupies on the disk. Let:

- n_r be the number of tuples in relation r .
- f_r be the blocking factor (tuples per block) for relation r .
- b_r be the number of blocks occupied by relation r , calculated as $b_r = \lceil n_r / f_r \rceil$.

Given parameters:

- **Relation r_1 :** $n_{r1} = 20,000$ tuples, $f_{r1} = 25$ tuples/block.

$$b_{r1} = \frac{20,000}{25} = 800 \text{ blocks}$$

- **Relation r_2 :** $n_{r2} = 45,000$ tuples, $f_{r2} = 30$ tuples/block.

$$b_{r2} = \frac{45,000}{30} = 1,500 \text{ blocks}$$

Note: The cost calculations assume worst-case memory availability (only 1 block for the outer relation, 1 block for the inner relation, and 1 block for output) and that neither relation is indexed.

2. Nested Loop Join (Tuple-Oriented)

In a standard Nested Loop Join, for every single *tuple* in the outer relation, the entire inner relation is scanned.

Formulas:

- Total Block Transfers = $b_{\text{outer}} + n_{\text{outer}} \times b_{\text{inner}}$
- Total Disk Seeks = $b_{\text{outer}} + n_{\text{outer}}$

Case A: r_1 is the outer relation, r_2 is the inner relation

$$\begin{aligned} \text{Block Transfers} &= b_{r1} + (n_{r1} \times b_{r2}) \\ &= 800 + (20,000 \times 1,500) \\ &= 800 + 30,000,000 = \mathbf{30,000,800} \\ \text{Disk Seeks} &= b_{r1} + n_{r1} \\ &= 800 + 20,000 = \mathbf{20,800} \end{aligned}$$

Case B: r_2 is the outer relation, r_1 is the inner relation

$$\begin{aligned} \text{Block Transfers} &= b_{r2} + (n_{r2} \times b_{r1}) \\ &= 1,500 + (45,000 \times 800) \\ &= 1,500 + 36,000,000 = \mathbf{36,001,500} \\ \text{Disk Seeks} &= b_{r2} + n_{r2} \\ &= 1,500 + 45,000 = \mathbf{46,500} \end{aligned}$$

Conclusion for Nested Loop Join: Using r_1 as the outer relation is optimal because it has fewer tuples.

3. Block Nested Loop Join

In a Block Nested Loop Join, for every *block* of the outer relation, the entire inner relation is scanned. This drastically reduces the number of inner relation scans.

Formulas:

- Total Block Transfers = $b_{\text{outer}} + b_{\text{outer}} \times b_{\text{inner}}$
- Total Disk Seeks = $2 \times b_{\text{outer}}$

Case A: r_1 is the outer relation, r_2 is the inner relation

$$\begin{aligned} \text{Block Transfers} &= b_{r_1} + (b_{r_1} \times b_{r_2}) \\ &= 800 + (800 \times 1,500) \\ &= 800 + 1,200,000 = \mathbf{1,200,800} \\ \text{Disk Seeks} &= 2 \times b_{r_1} \\ &= 2 \times 800 = \mathbf{1,600} \end{aligned}$$

Case B: r_2 is the outer relation, r_1 is the inner relation

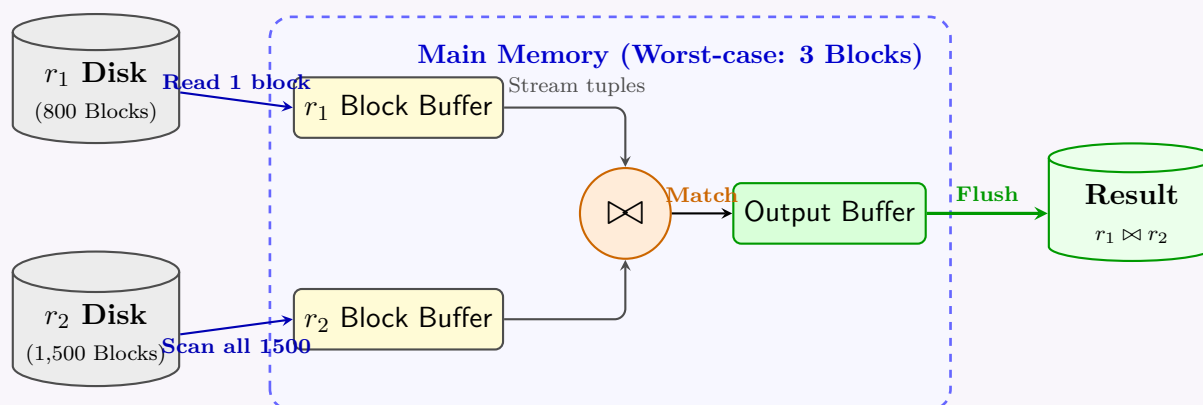
$$\begin{aligned} \text{Block Transfers} &= b_{r_2} + (b_{r_2} \times b_{r_1}) \\ &= 1,500 + (1,500 \times 800) \\ &= 1,500 + 1,200,000 = \mathbf{1,201,500} \\ \text{Disk Seeks} &= 2 \times b_{r_2} \\ &= 2 \times 1,500 = \mathbf{3,000} \end{aligned}$$

Conclusion for Block Nested Loop Join: Using r_1 as the outer relation is optimal because it has fewer blocks.

4. Summary of Optimal Costs

Join Algorithm	Best Outer Relation	Block Transfers	Disk Seeks
Nested Loop Join	r_1 (fewest tuples)	30,000,800	20,800
Block Nested Loop Join	r_1 (fewest blocks)	1,200,800	1,600

5. Visualizing Block Nested Loop Join Execution (with r_1 as Outer)



Note: The diagram illustrates why BNLJ requires 800×1500 block reads from r_2 . For each single block of r_1 placed in the buffer, the entire r_2 disk must be scanned.

Q6. Let t_r and t_s be the time to transfer a block of data and the time to seek a block in a disk subsystem, respectively. Estimate the query processing cost for:

- A primary B^+ tree index and equality on key.
- A primary B^+ tree index and comparison on key.

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

To estimate the query processing cost for data retrieval using a primary B^+ tree index, we must account for both the time required to traverse the index structure and the time required to access the actual data blocks.

Let us define the following parameters to formulate our cost models:

- t_r : Time to transfer a single block of data.
- t_s : Time to seek a single block of data on the disk.
- h_i : Height of the B^+ tree index (number of index levels).

- b : Number of data blocks containing the records that satisfy the query condition.

1. Primary B⁺ Tree Index and Equality on Key

An equality condition on a candidate key (e.g., `SELECT * FROM table WHERE key = value`) guarantees that at most one record will satisfy the condition.

Working Principle:

- The system traverses the B⁺ tree from the root to the leaf node. This requires accessing exactly h_i index blocks.
- Since the index blocks are typically not stored contiguously on the disk, each index block access requires one disk seek and one block transfer.
- Once the correct leaf node is found, the system follows the pointer to the actual data block.
- Retrieving this single data block requires one additional disk seek and one block transfer.

Cost Estimation Model:

$$\begin{aligned} \text{Cost}_{\text{Equality}} &= (\text{Index Access Cost}) + (\text{Data Access Cost}) \\ &= [h_i \times (t_s + t_r)] + [1 \times (t_s + t_r)] \\ &= (h_i + 1) \times (t_s + t_r) \end{aligned}$$

2. Primary B⁺ Tree Index and Comparison on Key

A comparison condition (e.g., `SELECT * FROM table WHERE key >= value`) typically retrieves multiple records. Since this is a *primary* index, the data records are physically sorted on the disk according to the search key.

Working Principle:

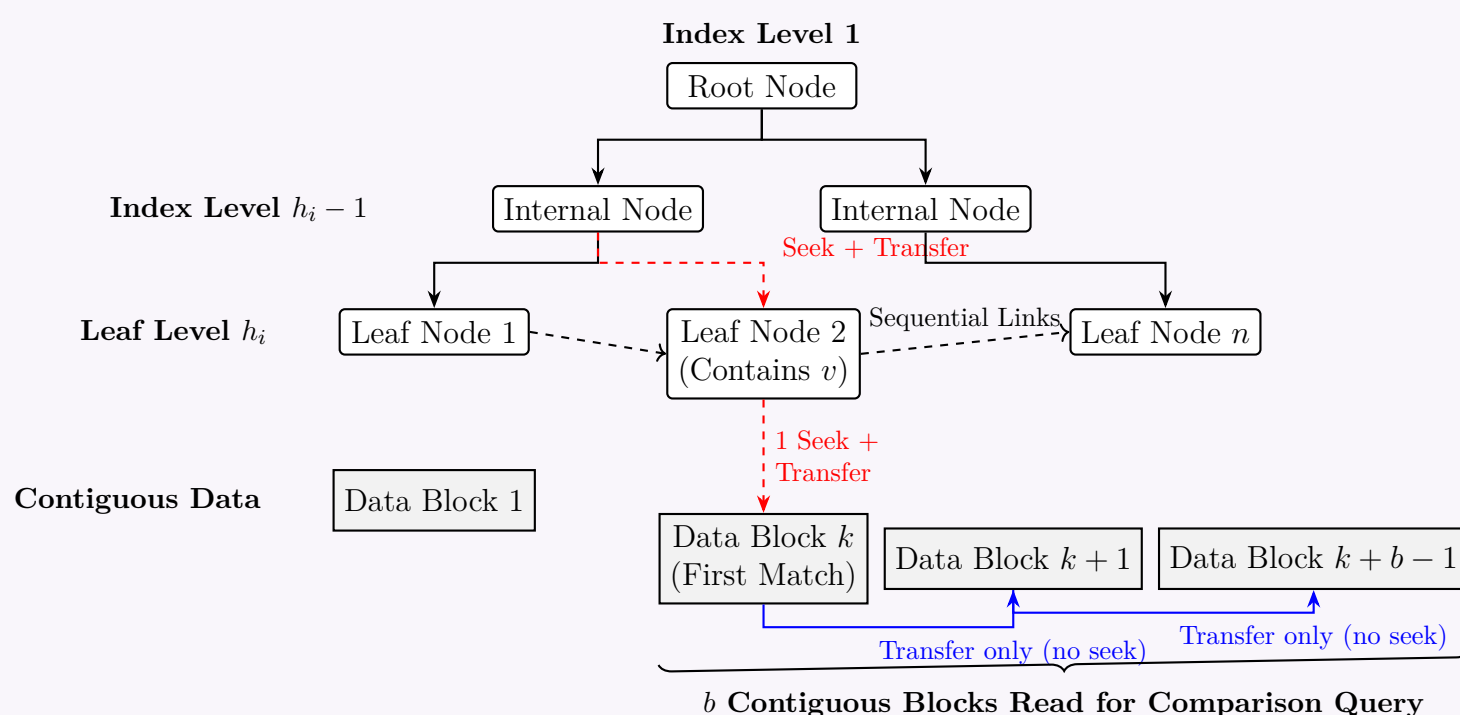
- The system first evaluates the boundary of the comparison by traversing the B⁺ tree down to the leaf node to find the first matching record. This requires h_i index block accesses.
- Once the first matching record's data block is located, the system needs to read the total of b data blocks that contain the matching records.
- Because the file is sequentially ordered based on the primary index, these b data blocks are stored contiguously on the disk.
- Therefore, accessing the b data blocks requires only **one** initial seek, followed by b consecutive block transfers.

Cost Estimation Model:

$$\begin{aligned} \text{Cost}_{\text{Comparison}} &= (\text{Index Traversal Cost}) + (\text{Data Blocks Retrieval Cost}) \\ &= [h_i \times (t_s + t_r)] + [t_s + (b \times t_r)] \\ &= (h_i + 1) \times t_s + (h_i + b) \times t_r \end{aligned}$$

Visualizing the Query Processing Cost Strategy

The following diagram illustrates the traversal and data retrieval process for a comparison query (`key >= v`) on a primary B⁺ tree index, highlighting where seeks (t_s) and transfers (t_r) occur.



Q7. Write down the steps of two-phase multiway merge sort. Consider a system where the block size is B bytes, memory size is M bytes, and record size is R bytes. What is the maximum number of records that can be sorted using two-phase merge sort? If the number of records exceeds this maximum, how can you sort them? (15 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Steps of Two-Phase Multiway Merge Sort

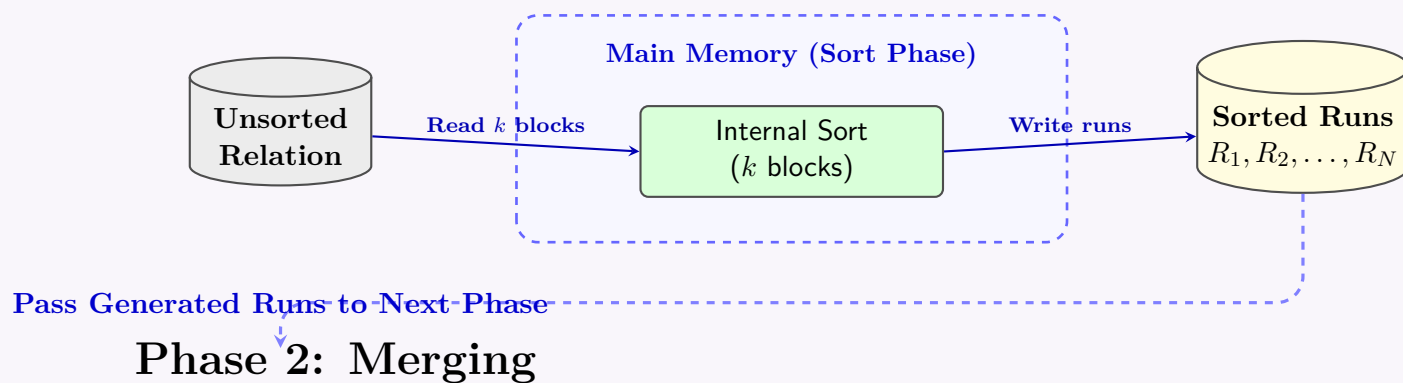
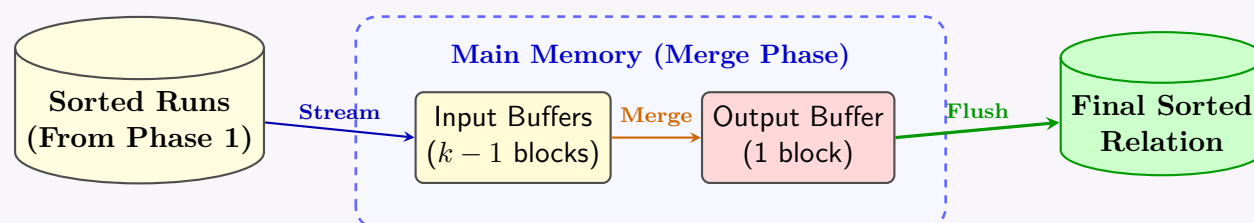
External sorting is required when the data to be sorted does not fit entirely in the main memory (RAM). The **Two-Phase Multiway Merge Sort** is a standard algorithm used in Database Management Systems to efficiently sort large relations. As the name implies, it operates in two distinct phases:

Phase 1: Sorting Phase (Run Generation)

- Determine Memory Capacity:** Calculate the number of disk blocks that can fit into the available main memory buffer. Let this be k blocks.
- Read Data:** Read k blocks of the unsorted relation from the disk into the main memory.
- In-Memory Sort:** Sort these records in memory using a fast sorting algorithm (e.g., Quicksort or Heapsort).
- Write Run:** Write the sorted k blocks back to the disk. This sorted sequence is called a **run**.
- Repeat:** Repeat steps 2–4 until the entire relation has been read, sorted into chunks, and written to the disk as multiple sorted runs.

Phase 2: Merging Phase

- Buffer Allocation:** Out of the k available memory blocks, allocate $k - 1$ blocks as input buffers and 1 block as an output buffer.
- Initialize Merge:** Read the first block of each of the generated runs into the $k - 1$ input buffers.
- Multiway Merge:** Compare the first available record from each of the $k - 1$ buffers. Select the smallest record (for ascending order) and move it to the output buffer.
- Buffer Management:**
 - If the output buffer becomes full, write it to the disk and clear the buffer.
 - If an input buffer becomes empty, read the next block from the corresponding run on the disk.
- Completion:** Continue the merging process until all records from all runs have been processed and written to the final sorted output file.

Phase 1: Run Generation**Phase 2: Merging**

2. Maximum Number of Records Sortable in Two Phases

To calculate the maximum number of records that can be sorted strictly using a two-phase multiway merge sort, we must analyze the memory constraints during the phases.

Given Parameters:

- B : Block size in bytes
- M : Available memory size in bytes
- R : Record size in bytes

Calculations:

- Number of memory blocks (k):** The maximum number of blocks that can fit into memory is:

$$k = \left\lfloor \frac{M}{B} \right\rfloor$$

(b) **Size of each run:** In Phase 1, the memory can sort up to k blocks at a time. Therefore, the size of each run is k blocks.

(c) **Maximum number of runs (N_{max_runs}):** In Phase 2, we must reserve 1 block for the output buffer. The remaining $k - 1$ blocks are used as input buffers. Because each run requires 1 input buffer, the maximum number of runs we can merge simultaneously in a single pass is:

$$N_{max_runs} = k - 1 = \left\lfloor \frac{M}{B} \right\rfloor - 1$$

(d) **Maximum file size in blocks:** The maximum number of blocks that can be sorted is the maximum number of runs multiplied by the size of each run:

$$\text{Max Blocks} = N_{max_runs} \times (\text{Size of one run}) = (k - 1) \times k$$

$$\text{Max Blocks} = \left(\left\lfloor \frac{M}{B} \right\rfloor - 1 \right) \times \left\lfloor \frac{M}{B} \right\rfloor$$

(e) **Blocking Factor (bfr):** The number of records that can fit into a single block is:

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor$$

Maximum Number of Records: By multiplying the maximum number of blocks by the blocking factor, we obtain the absolute maximum number of records that can be sorted in exactly two phases:

$$\text{Max Records} = \left\lfloor \frac{M}{B} \right\rfloor \times \left(\left\lfloor \frac{M}{B} \right\rfloor - 1 \right) \times \left\lfloor \frac{B}{R} \right\rfloor$$

3. Handling Records Exceeding the Maximum (Multi-Pass Merge Sort)

If the total number of records exceeds the maximum limit calculated above, a two-phase sort is insufficient because Phase 2 cannot merge all the generated runs in a single pass (i.e., the number of runs generated exceeds $k - 1$).

To solve this, the DBMS transitions to a **Multi-Pass (or Multi-Phase) Multiway Merge Sort**.

Working Principle of Multi-Pass Sort:

- **Pass 0 (Run Generation):** Generate N_R initial runs of size k blocks using the standard Phase 1 process.
- **Pass 1 (Intermediate Merge):** Read up to $k - 1$ runs at a time and merge them into a single larger run. This reduces the total number of runs by a factor of $k - 1$. The new runs will have a size of $k(k - 1)$ blocks.
- **Subsequent Passes:** Continue merging $k - 1$ runs at a time in each subsequent pass. With each pass, the number of runs decreases by a factor of $k - 1$, and the size of the runs increases exponentially.
- **Final Pass:** The algorithm terminates when only a single sorted run remains, which constitutes the final sorted relation.

Properties of Multi-Pass Merge:

- **Number of Merge Passes:** If N_R is the initial number of runs, the number of merge passes required is $\lceil \log_{k-1}(N_R) \rceil$.
- **Total Passes:** The total number of passes (including run generation) is $1 + \lceil \log_{k-1}(N_R) \rceil$.
- **I/O Cost:** Each pass reads and writes every block of the relation exactly once. The overall I/O block transfer cost is $2 \times \text{Total Blocks} \times \text{Total Passes}$.

Q8. Show the steps of duplicate elimination using a two-pass algorithm. Derive the relationship between number of records and memory size in this algorithm. (15 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Two-Pass Duplicate Elimination Algorithm

Duplicate elimination is typically implemented using a sorting-based two-pass algorithm. The goal is to identify and remove duplicate records from a large relation that cannot fit entirely into main memory.

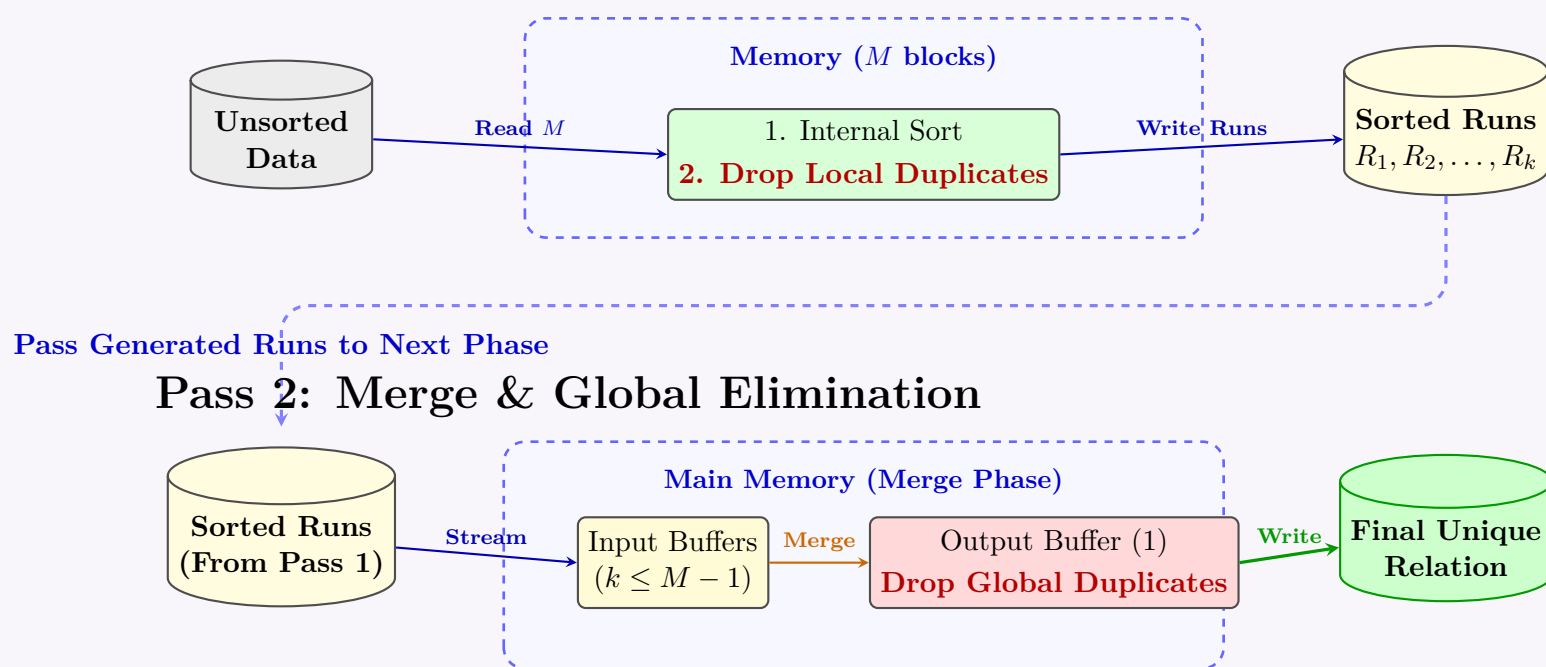
The algorithm operates in two phases: **Pass 1 (Run Generation with Local Elimination)** and **Pass 2 (Merging with Global Elimination)**.

Pass 1: Run Generation (Local Elimination)

- Read Data:** Read M blocks of the unsorted relation from the disk into the available main memory, where M is the number of memory buffers available.
- In-Memory Sort & Eliminate:** Sort the records currently in the memory buffers. During the sorting process, if any duplicate records are found within this chunk, eliminate them.
- Write Run:** Write the sorted, locally unique records back to the disk as a single sorted **run**.
- Repeat:** Repeat steps 1–3 until all blocks of the original relation have been processed, producing k sorted runs.

Pass 2: Merging (Global Elimination)

- (a) **Buffer Allocation:** Allocate $M - 1$ memory blocks as input buffers (one for each of the k runs, requiring $k \leq M - 1$) and 1 memory block as an output buffer.
- (b) **Initialize:** Read the first block of each of the k sorted runs into the $M - 1$ input buffers.
- (c) **Multiway Merge:** Compare the smallest available records from each input buffer to find the globally smallest record.
- (d) **Global Duplicate Elimination:**
 - Keep track of the *last record written* to the output buffer.
 - If the current smallest record is **equal** to the last written record, it is a duplicate and must be discarded.
 - If it is **different**, write it to the output buffer.
- (e) **Buffer Management:** If an input buffer is exhausted, load the next block from the corresponding run on disk. If the output buffer is full, flush it to the disk.
- (f) **Completion:** Continue until all runs have been completely merged. The final output is a sorted relation with all duplicates eliminated.

Pass 1: Run Generation**2. Relationship Between Number of Records and Memory Size**

To guarantee that the duplicate elimination can be completed in exactly **two passes**, the number of runs generated in Pass 1 (k) must not exceed the number of available input buffers in Pass 2.

Let us define the parameters:

- N : Total number of records in the relation.
- M : Size of main memory in blocks (number of available memory buffers).
- B : Total number of blocks required to store the original relation.
- bfr : Blocking factor (number of records that fit into one block).

Derivation:

- (a) **Total Blocks (B):** The total number of blocks holding the relation is given by:

$$B = \left\lceil \frac{N}{bfr} \right\rceil \quad (9)$$

- (b) **Number of Runs Generated (k):** In Pass 1, we read M blocks at a time. The number of generated runs is at most:

$$k = \left\lceil \frac{B}{M} \right\rceil \quad (10)$$

Note: Since duplicates are eliminated locally in Pass 1, the actual size of the runs might be smaller, but the worst-case number of runs remains $\lceil B/M \rceil$.

- (c) **Memory Constraint for Pass 2:** In Pass 2, we need one input buffer for each of the k runs and one buffer for the output. Therefore, the maximum number of runs we can merge simultaneously is $M - 1$.

$$k \leq M - 1 \quad (11)$$

(d) **Combining the constraints:** Substitute the expression for k into the constraint:

$$\begin{aligned}\frac{B}{M} &\leq M - 1 \\ B &\leq M(M - 1)\end{aligned}\tag{12}$$

(e) **Relationship in terms of Number of Records (N):** Substitute $B \approx N/bfr$ into the inequality:

$$\begin{aligned}\frac{N}{bfr} &\leq M(M - 1) \\ N &\leq bfr \times M(M - 1)\end{aligned}\tag{13}$$

Conclusion: For the two-pass algorithm to successfully eliminate duplicates, the maximum number of records N is constrained quadratically by the memory size M . The precise relationship is $N \leq bfr \times M(M - 1)$, which can be approximated as $N \approx \mathcal{O}(M^2 \times bfr)$. If N exceeds this limit, a multi-pass approach (three or more passes) is required.

Q9. Discuss the subsystems that are related to query processing in a Database Management System (DBMS). (10 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

Query Processing in DBMS

Query Processing refers to the range of activities involved in extracting data from a database. It encompasses the translation of high-level queries (e.g., SQL) into low-level execution plans, the optimization of these plans, and their final execution. To achieve this efficiently, a DBMS employs a sequence of specialized subsystems. The primary subsystems related to query processing are detailed below:

1. Parser and Translator Subsystem

- **Working Principle:** When a user submits an SQL query, this subsystem is the first to process it. It parses the query to check for proper syntax and semantic correctness.
- **Functions:**
 - **Syntax Check:** Ensures the query follows the rules of the SQL language.
 - **Semantic Check:** Verifies that relations and attributes mentioned in the query actually exist in the database (via the System Catalog).
 - **Translation:** Converts the validated SQL query into an internal representation, typically a **Query Tree** or a **Relational Algebra Expression**.

2. Query Optimizer Subsystem

- **Working Principle:** This is arguably the most critical component for DBMS performance. A single query can often be translated into multiple equivalent relational algebra expressions. The optimizer's job is to evaluate these alternatives and select the most efficient one.
- **Functions:**
 - **Heuristic Optimization:** Applies rule-based transformations (e.g., performing 'SELECTION' and 'PROJECTION' operations as early as possible to reduce intermediate relation sizes).
 - **Cost-Based Optimization:** Uses statistical metadata (e.g., table sizes, index distribution) to estimate the CPU and I/O costs of different execution plans.
 - **Output:** Generates an **Execution Plan**, which is an annotated query tree detailing the exact algorithms to be used (e.g., Hash Join vs. Merge Join) and the sequence of operations.

3. System Catalog (Metadata Manager)

- **Working Principle:** The catalog acts as the central repository of metadata for the DBMS. It is continuously consulted by the parser and the optimizer.
- **Functions:** Stores database schemas, constraints, user privileges, and statistical data (e.g., number of tuples, block distribution, index availability) crucial for cost estimation during optimization.

4. Query Evaluation Engine (Execution Engine)

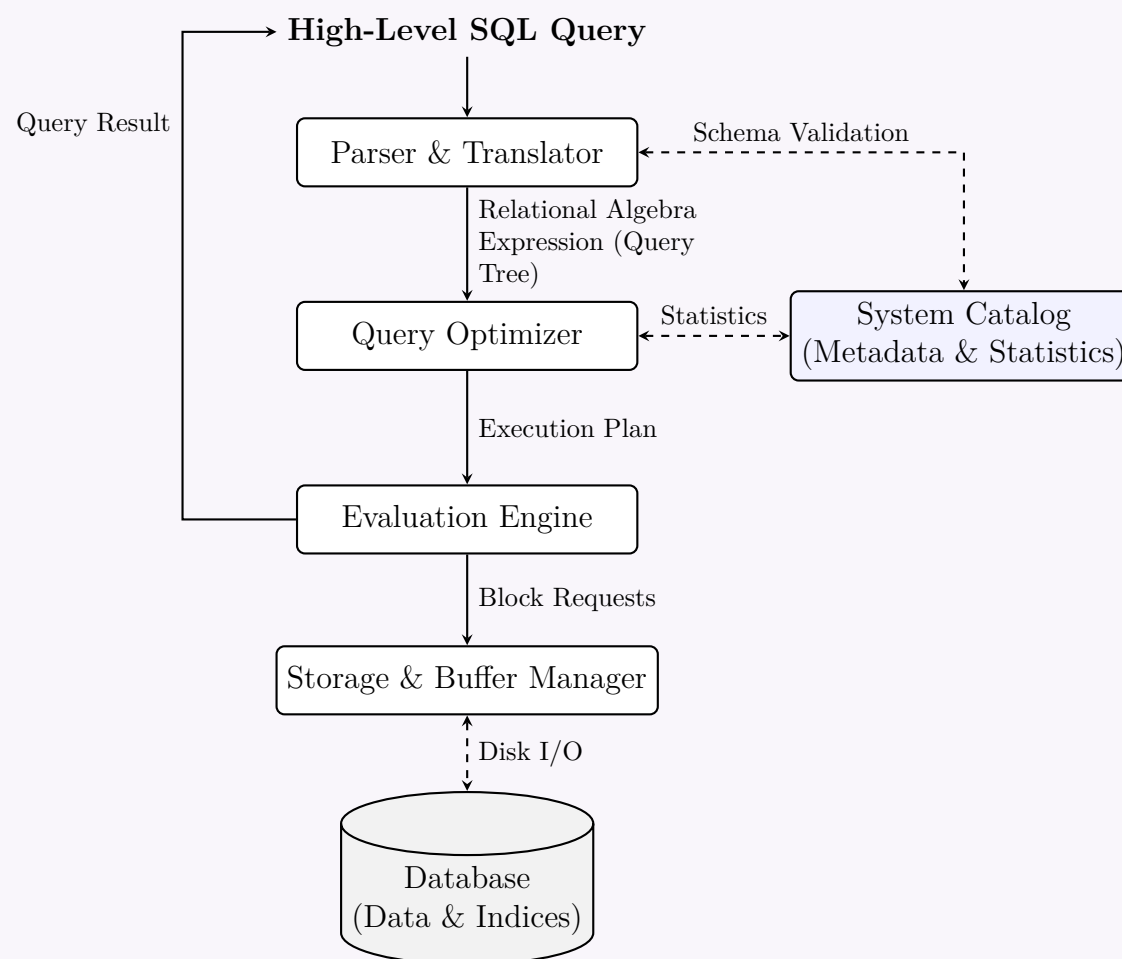
- **Working Principle:** Takes the execution plan generated by the optimizer and executes it step-by-step.
- **Functions:**
 - Coordinates the execution of operators in the plan.
 - Dispatches instructions to the lower-level subsystems to fetch required data.
 - Pipelining or materializing intermediate results.
 - Returns the final derived relation (the query result) to the user.

5. Storage and Buffer Manager

- **Working Principle:** Although primarily responsible for storage, it interfaces directly with the Evaluation Engine to facilitate data access.
- **Functions:** Maps logical requests from the Execution Engine to physical block addresses on the disk. It manages main memory buffers to minimize disk I/O operations by keeping frequently accessed blocks in RAM.

Architecture of Query Processing Subsystems

The following diagram illustrates the interaction and flow of data between these subsystems during the processing of a query.



Q10. Write the steps of performing a one-pass algorithm for finding the maximum value of an attribute for relation R . If the buffer size is M , what is the relationship between R and M ? Derive the relationship between M and R for a two-pass aggregate operation. (20 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. One-Pass Algorithm for Finding the Maximum Value

Finding the maximum value of a specific attribute (e.g., A) in a relation R is a scalar aggregate operation. Since it does not require grouping, it can be efficiently evaluated using a stream-based one-pass algorithm.

Steps of the Algorithm:

- Initialization:** Allocate one block of main memory as an input buffer. Initialize a variable in main memory, say `max_val`, to $-\infty$ (or the smallest possible domain value for attribute A).
- Read Block:** Read the first block of relation R from the disk into the memory buffer.
- Tuple Processing:** For each tuple t in the current buffer block:
 - Read the value of attribute A , denoted as $t.A$.
 - If $t.A > \text{max_val}$, update $\text{max_val} = t.A$.
- Iterate:** Clear the buffer and read the next block of relation R . Repeat Step 3 until all blocks of R have been read and processed.
- Output:** Once the End-Of-File (EOF) is reached, the value stored in `max_val` is the true maximum of the attribute in relation R . Return this value.

Relationship Between R and Buffer Size M (One-Pass)

Let $B(R)$ denote the number of disk blocks occupied by relation R , and M denote the number of available main memory buffer blocks.

For this one-pass scalar aggregate operation, we only need to keep the current block of R and a single running variable (`max_val`) in memory. Therefore, the algorithm only requires **a minimum of 1 buffer block**.

$$M \geq 1$$

Because the previous blocks are discarded after processing, **there is no upper limit on the size of R** . The relation $B(R)$ can be arbitrarily large regardless of the memory size M , as long as $M \geq 1$. Thus, $B(R)$ is independent of M in this context.

2. Two-Pass Aggregate Operation (Grouping and Aggregation)

While a simple `MAX()` over the entire relation takes only one pass, aggregate queries containing a `GROUP BY` clause (e.g., finding the maximum salary *per department*) may require two passes. If the number of distinct groups is too large to fit in main memory, we use a two-pass algorithm, typically relying on **hashing** or **sorting**.

Below, we derive the relationship between M and R using the **Hash-Based Two-Pass Aggregation** method.

Derivation of Relationship between M and R

Let $B(R)$ be the size of relation R in blocks, and M be the available memory buffer size in blocks.

Pass 1: Partitioning Phase

- We use 1 buffer block to sequentially read blocks of R .
- We use the remaining $M - 1$ buffer blocks to represent $M - 1$ hash buckets.
- Each tuple is hashed on its grouping attribute to one of the $M - 1$ buckets. When a bucket buffer is full, it is flushed to disk.
- Assuming an ideal, uniform hash function, the relation R is evenly divided into $M - 1$ partitions on the disk.
- **Size of each partition (on disk):**

$$\text{Size of one partition} \approx \left\lceil \frac{B(R)}{M - 1} \right\rceil \text{ blocks}$$

Pass 2: Aggregation Phase

- In the second pass, we process each of the $M - 1$ partitions one by one.
- For the in-memory aggregation to work without further disk I/O (i.e., strictly in two passes), **an entire partition must fit into the available main memory**.
- During this phase, we need memory to hold the partition and compute the aggregates. The memory capacity is M blocks.
- Therefore, the size of a single partition must be less than or equal to M blocks (or $M - 1$ if we reserve one block for output).

Mathematical Relationship:

$$\text{Size of one partition} \leq \text{Available Memory}$$

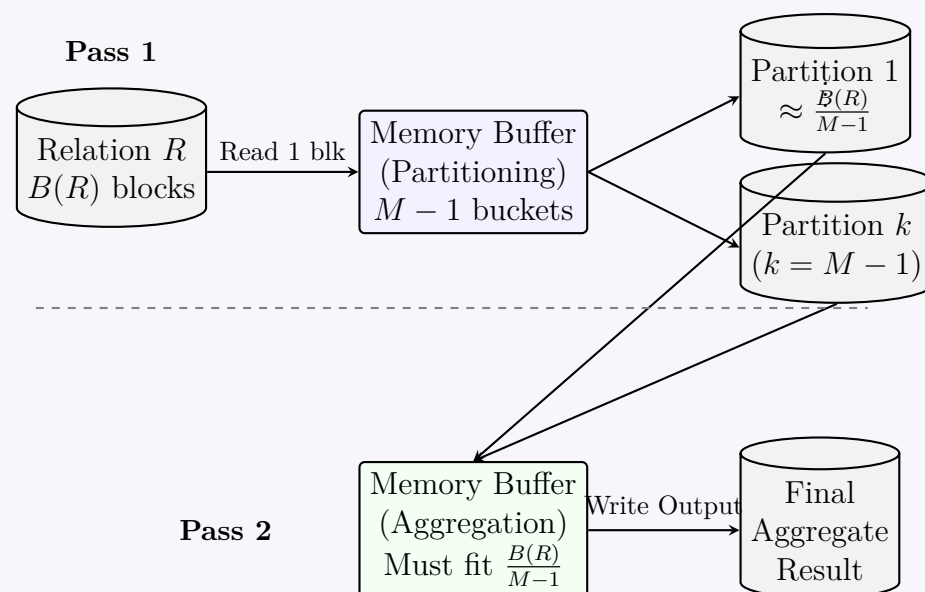
$$\frac{B(R)}{M - 1} \leq M$$

$$B(R) \leq M(M - 1)$$

For large M , $M - 1 \approx M$. Thus, the maximum size of the relation R that can be processed using a two-pass aggregate operation is bounded quadratically by the main memory size:

$$B(R) \approx O(M^2)$$

Hash-Based Two-Pass Aggregation



Q11. Derive the total number of disk I/Os for nested loop join of two relations R and S , where the number of records in R is smaller than the number of records in S . (5 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Introduction and Parameter Definition

A **Nested Loop Join** is a fundamental algorithm used by a Database Management System (DBMS) to join two relations. It evaluates the join condition by comparing tuples (or blocks) from one relation with tuples (or blocks) of the other in a nested loop fashion.

To derive the total number of disk I/Os (block transfers), let us define the following parameters:

- n_R : Number of records (tuples) in relation R .
- n_S : Number of records (tuples) in relation S .
- b_R : Number of disk blocks containing relation R .
- b_S : Number of disk blocks containing relation S .
- M : Number of available main memory buffer blocks.

Given the condition that the number of records in R is smaller than S ($n_R < n_S$), we can generally assume that R also occupies fewer blocks than S ($b_R < b_S$), assuming a comparable blocking factor. To minimize disk I/Os, the smaller relation (R) should always be chosen as the **outer relation** in the loop.

2. Derivation 1: Simple (Tuple-Oriented) Nested Loop Join

In a standard tuple-oriented nested loop join, for every single tuple in the outer relation, the entire inner relation is scanned.

Assuming R is the outer relation and S is the inner relation:

- We read the outer relation R from disk to memory block by block. This requires reading all b_R blocks once.
- There are n_R total tuples in relation R .
- For *each* of the n_R tuples, we must scan the entire inner relation S to evaluate the join condition. Scanning S entirely requires reading its b_S blocks.

Mathematical Derivation:

$$\text{Total I/O} = (\text{I/O for reading outer relation } R) + (\text{Number of tuples in } R) \times (\text{I/O for reading inner relation } S)$$

$$\text{Total I/O} = b_R + n_R \times b_S$$

Why R must be the outer relation: If S were the outer relation, the cost would be $b_S + n_S \times b_R$. Because $n_R < n_S$ and typically $b_R < b_S$, it is strictly true that $b_R + n_R \times b_S < b_S + n_S \times b_R$. Thus, putting the smaller relation R on the outside minimizes I/O.

3. Derivation 2: Block Nested Loop Join (BNLJ)

A highly inefficient aspect of the tuple-oriented approach is that it rereads the inner relation blocks for every single tuple. The **Block Nested Loop Join** improves this by processing relations block by block.

Algorithm: For each block of the outer relation, we read the entire inner relation and join all tuples in the current outer block with all tuples in the current inner block.

Mathematical Derivation (with minimal memory, $M = 3$):

- The outer relation R is read block by block, requiring b_R disk accesses.
- For *each* of the b_R blocks, the entire inner relation S is read (b_S disk accesses).

$$\text{Total I/O} = (\text{I/O for outer relation } R) + (\text{Number of blocks in } R) \times (\text{I/O for inner relation } S)$$

$$\text{Total I/O} = b_R + (b_R \times b_S)$$

Here, the symmetry of $b_R \times b_S$ means the dominant term is the same either way. However, $b_R + b_R b_S < b_S + b_S b_R$ since $b_R < b_S$. Thus, R should again be the outer relation.

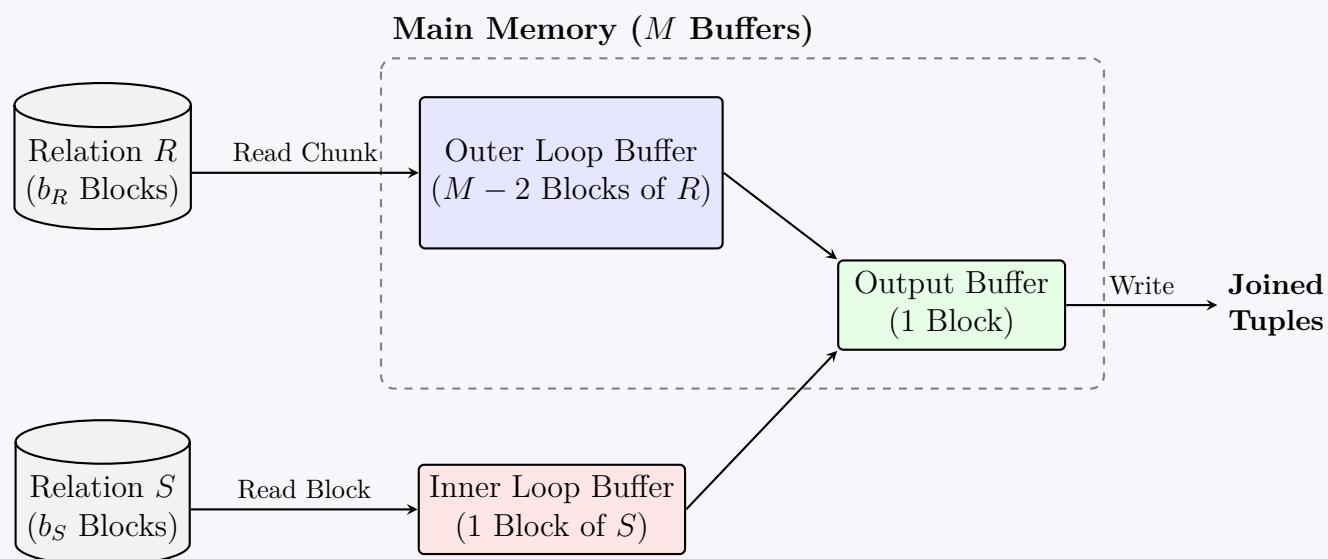
4. Derivation 3: Optimized Chunk-Based Block Nested Loop Join

If the DBMS has M buffer blocks available in main memory, we can optimize BNLJ further. We allocate $M - 2$ blocks to hold a "chunk" of the outer relation R , 1 block for the inner relation S , and 1 block for the output buffer.

Mathematical Derivation:

- The outer relation R is read entirely once: b_R I/Os.
- The number of "chunks" the outer relation is divided into is $\lceil b_R / (M - 2) \rceil$.
- For each chunk, the entire inner relation S is scanned once (b_S I/Os).

$$\text{Total I/O} = b_R + \left\lceil \frac{b_R}{M - 2} \right\rceil \times b_S$$



Summary of Optimizations: By choosing the smaller relation R as the outer relation and utilizing the maximum available memory M to load chunks of R , the total number of times the large relation S must be read from the disk is drastically minimized.

Query Optimization

Q1. Given three relations $R(a, b, c)$, $S(d, e)$, and $T(e, f, g)$, apply query optimization techniques to find the optimized logical plan for the following expression:

$$\sigma_{a>100, b=20, c=d, f<100}((R \times S) \bowtie T)$$

(7 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

Query Optimization Using Heuristics

The goal of heuristic query optimization is to transform a given relational algebra expression into a logically equivalent expression that is more efficient to execute. This is primarily achieved by reducing the size of intermediate relations as early as possible.

Given Relations: $R(a, b, c)$, $S(d, e)$, and $T(e, f, g)$

Initial Logical Expression:

$$\sigma_{a>100 \wedge b=20 \wedge c=d \wedge f<100}((R \times S) \bowtie T)$$

Step-by-Step Optimization Process

- (a) **Cascade of Selections (Deconstruction):** First, we break down the complex conjunctive selection condition into a sequence of individual selection operations.

$$\sigma_{a>100} \left(\sigma_{b=20} \left(\sigma_{c=d} \left(\sigma_{f<100} ((R \times S) \bowtie T) \right) \right) \right)$$

- (b) **Pushing Selections Down the Tree:** We move the selection operations as far down the query tree as possible to reduce the number of tuples before expensive join and Cartesian product operations occur.

- The conditions $\sigma_{a>100}$ and $\sigma_{b=20}$ apply exclusively to attributes of relation R . These can be combined and pushed directly above R : $\sigma_{a>100 \wedge b=20}(R)$.
- The condition $\sigma_{f<100}$ applies exclusively to relation T . This can be pushed directly above T : $\sigma_{f<100}(T)$.
- The condition $\sigma_{c=d}$ involves attributes from both R and S . It must remain above the operation that combines R and S .

$$\sigma_{c=d} \left((\sigma_{a>100 \wedge b=20}(R) \times S) \bowtie \sigma_{f<100}(T) \right)$$

- (c) **Converting Cross-Product to Join:** A Cartesian product ($\times$) followed by a selection condition ($\sigma_{c=d}$) comparing attributes from the participating relations can be converted into an Equi-Join ($\bowtie_{c=d}$). This avoids generating a massive, mostly useless intermediate table.

$$(\sigma_{a>100 \wedge b=20}(R) \bowtie_{c=d} S) \bowtie \sigma_{f<100}(T)$$

Final Optimized Logical Plan

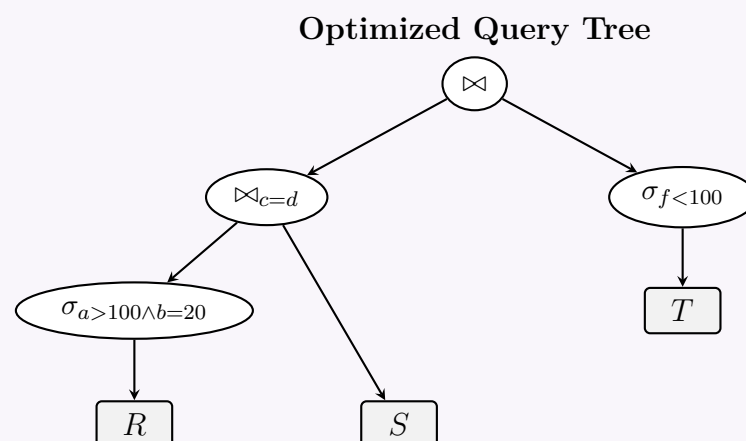
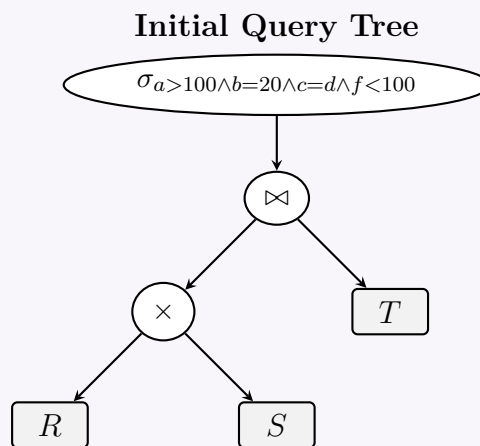
The fully optimized relational algebra expression is:

$$(\sigma_{a>100 \wedge b=20}(R) \bowtie_{c=d} S) \bowtie \sigma_{f<100}(T)$$

Note: The final outermost join ($\bowtie$) represents a Natural Join on the common attribute e between S and T .

Query Tree Diagrams

The following diagrams illustrate the initial, unoptimized query tree and the final optimized query tree.



Q2. For each of the following statements, determine whether it is true. If it is not, provide a counterexample to justify your answer.

- (a) $\delta(\pi_L(R)) = \pi_L(\delta(R))$
- (b) $\delta(\sigma_C(R)) = \sigma_C(\delta(R))$
- (c) $\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$

(9 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

*Note: The explicit use of the duplicate elimination operator, δ , dictates that this evaluation must be done under **bag (multiset) semantics**, where relations can contain duplicate tuples.*

1. $\delta(\pi_L(R)) = \pi_L(\delta(R))$

Answer: FALSE

Explanation: Under bag semantics, the projection operator (π) does not automatically eliminate duplicates. When we project out attributes, distinct tuples in the original relation might become identical in the projected relation.

- The Left-Hand Side (LHS), $\delta(\pi_L(R))$, projects first (potentially creating new duplicates) and then eliminates *all* duplicates, resulting in a strict set.
- The Right-Hand Side (RHS), $\pi_L(\delta(R))$, eliminates duplicates from the original relation first, and *then* projects. This projection can introduce new duplicates which will not be eliminated.

Counterexample: Let relation $R(A, B)$ contain the following tuples:

A	B
1	1
1	2

Let the projection list be $L = \{A\}$.

- **Evaluate LHS:** $\delta(\pi_A(R))$
 - (a) $\pi_A(R) = \{(1), (1)\}$ (Bag with two identical tuples)
 - (b) $\delta(\pi_A(R)) = \{(1)\}$ (Duplicates removed)
- **Evaluate RHS:** $\pi_A(\delta(R))$

- (a) $\delta(R) = \{(1, 1), (1, 2)\}$ (No duplicates existed in R , so it remains unchanged)
 (b) $\pi_A(\delta(R)) = \{(1), (1)\}$ (Projection creates duplicates)

Since $\{(1)\} \neq \{(1), (1)\}$, the statement is **false**.

2. $\delta(\sigma_C(R)) = \sigma_C(\delta(R))$

Answer: TRUE

Explanation: The selection operator (σ) evaluates a condition C against each tuple individually. It neither modifies the attributes of the tuples (which could create duplicates) nor creates new tuples.

- Removing duplicates and then filtering the distinct tuples ($\sigma_C(\delta(R))$) will leave one copy of every distinct tuple that satisfies C .
- Filtering the bag of tuples first and then removing duplicates ($\delta(\sigma_C(R))$) will also leave exactly one copy of every distinct tuple that satisfies C .

Since selection operations are completely orthogonal to tuple multiplicities, the two operators commute.

3. $\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$

Answer: TRUE

Explanation: The projection operator (π) distributes over the union operator ($\cup$). This holds true under both set and bag semantics.

- **Set Semantics:** If a tuple t' exists in $\pi_L(R \cup S)$, there must be some tuple $t \in R \cup S$ that projects to t' . Because t is either in R or S , its projection t' will appear in either $\pi_L(R)$ or $\pi_L(S)$, and thus in their union.
- **Bag Semantics:** The union operator adds the multiplicities of tuples. If tuple t_1 appears n times in R and m times in S , it appears $n + m$ times in $R \cup S$. Projecting $R \cup S$ will project all $n + m$ instances identically. Conversely, projecting R and S separately yields n instances in $\pi_L(R)$ and m instances in $\pi_L(S)$. Taking the union of these projections naturally adds the instances back together to exactly $n + m$ instances.

Thus, projecting the combined relation is perfectly equivalent to projecting the relations individually and combining the results.

Q3. Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$. Assume that there are no primary keys. Let $V(C, r_1) = 900$, $V(C, r_2) = 1100$, $V(E, r_2) = 50$, and $V(E, r_3) = 100$. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Compute the size of $r_1 \bowtie r_2 \bowtie r_3$. **(15 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Given Parameters

We are given three relations: $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$. The sizes of the relations (number of tuples) and the number of distinct values for the join attributes are as follows:

- **Relation Sizes (n_R):**
 - $n_{r_1} = 1000$ tuples
 - $n_{r_2} = 1500$ tuples
 - $n_{r_3} = 750$ tuples
- **Distinct Values ($V(A, R)$):**
 - $V(C, r_1) = 900$
 - $V(C, r_2) = 1100$
 - $V(E, r_2) = 50$
 - $V(E, r_3) = 100$

2. Estimation Principles

To estimate the size of a natural join $R \bowtie S$ on a common attribute A , we use the standard query optimization formula under the assumptions of **uniform distribution** of values and **containment of value sets** (every value in the relation with fewer distinct values is assumed to be present in the relation with more distinct values).

The formula for the estimated number of tuples in $R \bowtie S$ is:

$$\text{Size}(R \bowtie S) = \frac{n_R \times n_S}{\max(V(A, R), V(A, S))}$$

3. Step-by-Step Computation

Since the join is associative, we can evaluate it as $(r_1 \bowtie r_2) \bowtie r_3$.

Step 3.1: Size of $r_{12} = r_1 \bowtie r_2$

The relations r_1 and r_2 join on the common attribute C .

$$\begin{aligned} \text{Size}(r_{12}) &= \frac{n_{r_1} \times n_{r_2}}{\max(V(C, r_1), V(C, r_2))} \\ &= \frac{1000 \times 1500}{\max(900, 1100)} \\ &= \frac{1,500,000}{1100} \\ &= \frac{15000}{11} \approx 1363.64 \text{ tuples} \end{aligned}$$

Note on distinct values: For the intermediate relation r_{12} , the standard assumption is that the number of distinct values for the unjoined attribute E remains proportional to its original distinct values. Thus, we assume $V(E, r_{12}) \approx V(E, r_2) = 50$.

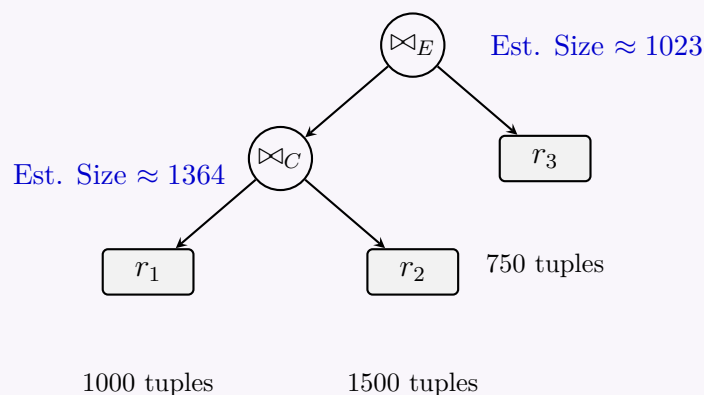
Step 3.2: Size of $r_{123} = r_{12} \bowtie r_3$

The intermediate relation r_{12} and relation r_3 join on the common attribute E .

$$\begin{aligned} \text{Size}(r_{123}) &= \frac{\text{Size}(r_{12}) \times n_{r_3}}{\max(V(E, r_{12}), V(E, r_3))} \\ &= \frac{\left(\frac{15000}{11}\right) \times 750}{\max(50, 100)} \\ &= \frac{15000 \times 750}{11 \times 100} \\ &= \frac{11250}{11} \approx 1022.72 \text{ tuples} \end{aligned}$$

4. Conclusion

Rounding to the nearest whole integer (since a fraction of a tuple cannot exist), the estimated size of the join $r_1 \bowtie r_2 \bowtie r_3$ is **1023 tuples**.



Q4. Construct an optimized query plan for the following query using equivalence rules. Draw the query plan using an expression tree and justify the plan with explanations. **Schema:**

Student (sid, name, major)

Enroll (sid, course, section, term, grade)

Class (course, section, term, instructor, room, time)

```

SELECT S.name, C.instructor, C.term
FROM Student S, Enroll E, Class C
WHERE S.sid = E.sid AND E.course = C.course
AND E.section = C.section AND E.term = C.term
AND C.instructor = 'Hinom' AND S.major = 'ML' ;
  
```

Equivalence Rules (The notations carry usual meaning)

(a) $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$

- (b) $\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$
- (c) $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$
- (d) $\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$
- (e) $\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$

(15 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer**1. Initial Logical Query Plan (Unoptimized)**

The canonical translation of the given SQL query into relational algebra involves a Cartesian product of all three relations, followed by a selection for all WHERE conditions, and finally a projection:

$$\Pi_{\text{name, instructor, term}} \left(\sigma_{\theta_{\text{join}} \wedge \theta_{\text{filter}}} (\text{Student} \times \text{Enroll} \times \text{Class}) \right)$$

Where:

- $\theta_{\text{join}} \equiv (S.\text{sid} = E.\text{sid}) \wedge (E.\text{course} = C.\text{course} \wedge E.\text{section} = C.\text{section} \wedge E.\text{term} = C.\text{term})$
- $\theta_{\text{filter}} \equiv (C.\text{instructor} = \text{'Hinin'}) \wedge (S.\text{major} = \text{'ML'})$

2. Step-by-Step Optimization using Equivalence Rules

We systematically apply the given equivalence rules to optimize the query by minimizing intermediate relation sizes.

- **Step 1: Convert Cross Products to Joins (Rule 1)**

Using $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$, we convert the Cartesian products into theta-joins.

$$\Pi_{\text{name, inst, term}} \left(\sigma_{\theta_{\text{filter}}} (\text{Student} \bowtie_{\theta_1} (\text{Enroll} \bowtie_{\theta_2} \text{Class})) \right)$$

Let $\theta_1 \equiv (S.\text{sid} = E.\text{sid})$ and $\theta_2 \equiv (E.\text{course} = C.\text{course} \wedge E.\text{section} = C.\text{section} \wedge E.\text{term} = C.\text{term})$.

- **Step 2: Push Down Selections (Rules 2 & 3)**

Using $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$, we push the filter predicates as close to the base relations as possible. The predicate $S.\text{major} = \text{'ML'}$ applies only to Student, and $C.\text{instructor} = \text{'Hinin'}$ applies only to Class.

$$\Pi_{\text{name, inst, term}} \left((\sigma_{\text{major}=\text{'ML'}}(\text{Student})) \bowtie_{\theta_1} (\text{Enroll} \bowtie_{\theta_2} (\sigma_{\text{inst}=\text{'Hinin'}}(\text{Class}))) \right)$$

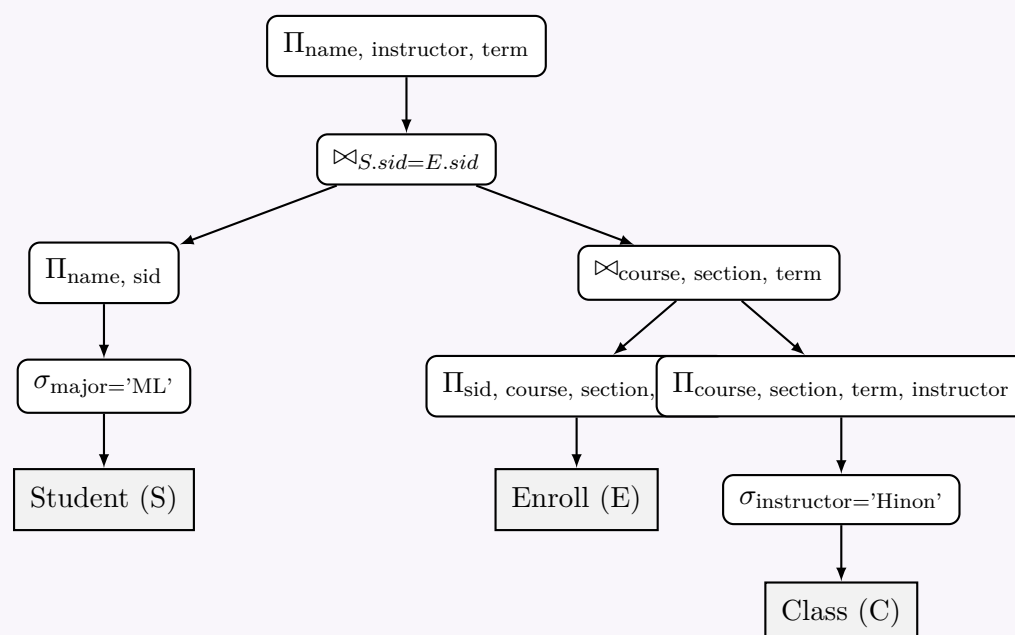
- **Step 3: Push Down Projections (Rules 4 & 5)**

Using $\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$, we eliminate unneeded attributes early. We only keep attributes required for subsequent joins or the final output.

- From **Student**: Project $\{\text{name}, \text{sid}\}$
- From **Enroll**: Project $\{\text{sid}, \text{course}, \text{section}, \text{term}\}$
- From **Class**: Project $\{\text{course}, \text{section}, \text{term}, \text{instructor}\}$

3. Optimized Expression Tree

The optimized expression tree evaluates the most restrictive selections early and drops unnecessary attributes before performing join operations, significantly reducing I/O and memory overhead.



Justification of the Plan:

- (a) **Early Selection:** By applying $\sigma_{\text{major}='ML'}$ on *Student* and $\sigma_{\text{instructor}='Hinin'}$ on *Class* before the joins, the cardinality of the relations participating in the joins is drastically reduced.
- (b) **Early Projection:** By stripping away unneeded attributes (e.g., *room*, *time*, *grade*) immediately after selection/reading, the size of each tuple is minimized. This allows more tuples to fit into memory blocks, speeding up join operations.
- (c) **Join Ordering:** Assuming selective filtering, joining *Enroll* and *Class* first creates a smaller intermediate relation of courses taught by 'Hinin'. This result is then joined with the filtered *Student* relation.

Q5. Consider the following database schema:

$R(A, B), S(B, C), T(C, D)$

Write a relational algebra plan for the SQL query below. Give the expression first, then convert it to a tree. Finally find the most optimized tree by pushing down selections and projections as much as possible.

```
SELECT R.A, T.D
FROM T, S, R
WHERE T.C = S.C
      AND S.B = R.B
      AND T.D > 20
```

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Initial Relational Algebra Expression

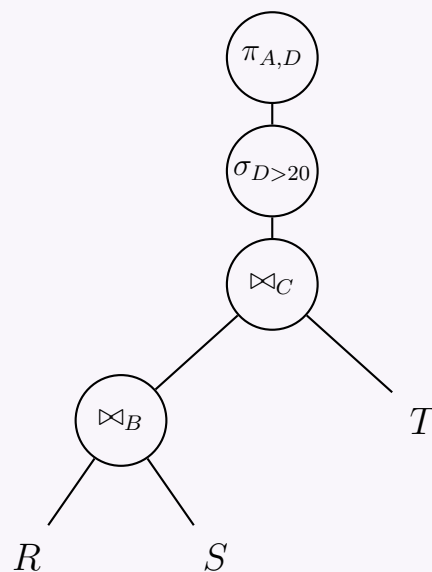
The canonical translation of the provided SQL query maps the **FROM** clause to Cartesian products (or joins), the **WHERE** clause to selections, and the **SELECT** clause to a final projection. Given the explicit equality conditions $T.C = S.C$ and $S.B = R.B$, we use equi-joins (or natural joins, assuming identical attribute names represent the join keys).

The initial, unoptimized relational algebra expression is:

$$\pi_{A,D}(\sigma_{D>20}((R \bowtie_B S) \bowtie_C T))$$

2. Initial Query Tree

The initial query tree translates the expression directly into a logical execution plan, evaluating from the bottom up. At this stage, all rows from R , S , and T participate in the joins before the filter on D is applied.



3. Optimization Strategy (Heuristic Rules)

To achieve the most optimized tree, we systematically apply relational algebra equivalence rules to push down operators:

- **Rule 1: Push Down Selections (σ).**

The selection $\sigma_{D>20}$ only references attribute D , which is strictly native to relation $T(C, D)$. We can commute the selection with the join operations and push it directly onto T . This significantly reduces the cardinality of T before it enters the join.

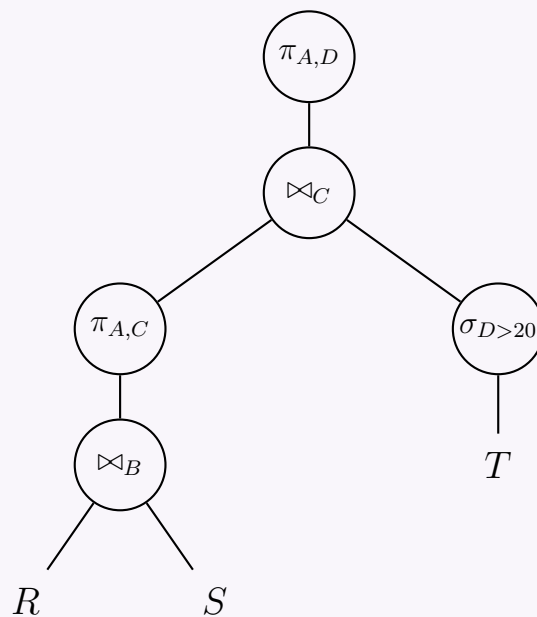
- **Rule 2: Push Down Projections (π).**

We evaluate the required attributes at every node (for both intermediate join conditions and the final output) to discard unnecessary columns as early as possible.

- **From $R(A, B)$:** Attribute A is needed for the final output, and B is needed for the join. Keep both.
- **From $S(B, C)$:** Attribute B is needed to join with R , and C is needed to join with T . Keep both.
- **From $T(C, D)$:** Attribute C is needed to join with S , and D is required for the output and selection. Keep both.
- **Intermediate Result $(R \bowtie_B S)$:** The output of this join contains attributes (A, B, C) . Moving upward, the subsequent join with T requires only C , and the final projection requires only A . Therefore, attribute B is no longer necessary. We can introduce an intermediate projection $\pi_{A,C}$ immediately after the first join to eliminate B early, reducing the memory footprint of the intermediate tuples.

4. Optimized Query Tree

Applying the pushdown heuristics yields the highly optimized execution plan below:



Advantages of this Optimized Tree:

- (a) Applying $\sigma_{D>20}$ natively on T limits disk I/O and prevents processing invalidated tuples during the $\bowtie_C$ operation.
- (b) Splicing $\pi_{A,C}$ above the $\bowtie_B$ relation minimizes the width of the pipelined records streaming into the upper $\bowtie_C$ node.

Exam Tip: Always double-check your attribute tracking when introducing intermediate projections. A common pitfall is discarding an attribute (like C) too early, breaking a join condition located higher up in the tree!

Q6. The following relations are given:

```
Student(id, name, CGPA, street, city)
takes(id, course-id, semester, year)
course(course-id, title, credit)
```

An SQL statement:

```
SELECT id, name, course-id, title
FROM student s, takes t, course c
WHERE s.id = t.id
      AND t.course-id = c.course-id
      AND s.city = 'Dhaka'
      AND c.credit = 3
```

- (a) Construct an expression tree for the given SQL.
- (b) Explain materialization evaluation and pipeline evaluation using the expression tree.

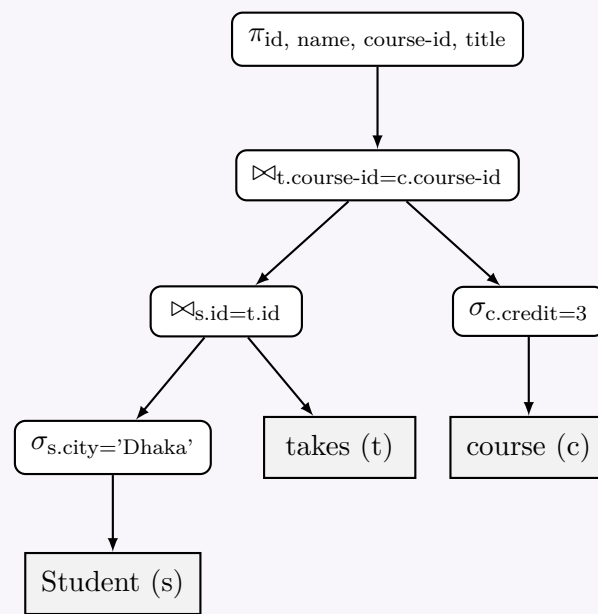
(20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Expression Tree for the Given SQL

An expression tree visually represents the relational algebra operations used to evaluate an SQL query. To optimize execution, we push down the selection (σ) operations as close to the base relations as possible, reducing the size of intermediate results before the join ($\bowtie$) operations.



2. Explanation of Evaluation Methods

Query evaluation involves executing the operations defined in the expression tree. This can be done using two primary paradigms: **Materialization** and **Pipelining**.

Materialization Evaluation

Definition: Materialization is a bottom-up evaluation approach where the result of each intermediate operation in the expression tree is written to a temporary relation (materialized) on disk before being passed to the parent operation.

Working Principle using the Tree:

- Step 1 (Leaves):** Evaluate $\sigma_{s.city='Dhaka'}(\text{Student})$ and store the result in a temporary table, say $Temp_1$.
- Step 2 (Leaves):** Evaluate $\sigma_{c.credit=3}(\text{course})$ and store the result in a temporary table, say $Temp_2$.
- Step 3 (Lower Join):** Read $Temp_1$ and the takes table, compute the join $\bowtie_{s.id=t.id}$, and store the result in $Temp_3$.
- Step 4 (Upper Join):** Read $Temp_3$ and $Temp_2$, compute the join $\bowtie_{t.course-id=c.course-id}$, and store the result in $Temp_4$.
- Step 5 (Root):** Read $Temp_4$, perform the final projection $\pi_{id, name, course-id, title}$, and output the result.

Properties:

- Advantage:** Simple to implement and naturally supports all types of relational operators (including blocking operators like sorting).
- Disadvantage:** High overhead due to reading and writing intermediate temporary tables ($Temp_1, Temp_2, Temp_3$) to and from secondary storage (disk I/O).

Pipeline Evaluation

Definition: Pipelining evaluates multiple operations simultaneously. Instead of storing entire intermediate results, tuples generated by a child operation are passed directly (in memory) to the parent operation as soon as they are produced.

Working Principle using the Tree:

- As soon as a tuple from the Student relation passes the filter $\sigma_{s.city='Dhaka'}$, it is passed directly into the first join ($\bowtie_{s.id=t.id}$) memory buffer.
- The first join immediately looks for matching tuples in the takes relation (e.g., using an index).
- When a matching joined tuple is found, it is not written to disk; instead, it is immediately pipelined to the upper join ($\bowtie_{t.course-id=c.course-id}$).
- The upper join probes the result of $\sigma_{c.credit=3}(\text{course})$. If it matches, the tuple flows to the root projection operation.
- The final output tuple is generated continuously in a streaming fashion.

Properties:

- Advantage:** Completely eliminates the need to create temporary tables, drastically reducing disk I/O and query response time for the first few output tuples.
- Disadvantage:** Cannot be used for *blocking operators* (e.g., sorting, grouping) which require seeing the entire input before producing any output. It also requires sufficient main memory to maintain state (like hash tables for joins) concurrently.

Q7. Express the following query using relational algebra and draw the equivalent logical query tree:

```

SELECT title
FROM StarsIn

```

```

WHERE starsName IN (
  SELECT name
  FROM MovieStar
  WHERE name LIKE '%Khan'
        AND birthdate > '18-MAR-1990'
);

```

(9 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer**1. Relational Algebra Expression**

The given SQL query uses a nested subquery with the **IN** operator. In relational algebra, an **IN** condition checking against a single-attribute subquery can be translated into an **equijoin** ($\bowtie$) between the outer relation and the subquery result. The equivalent relational algebra expression is:

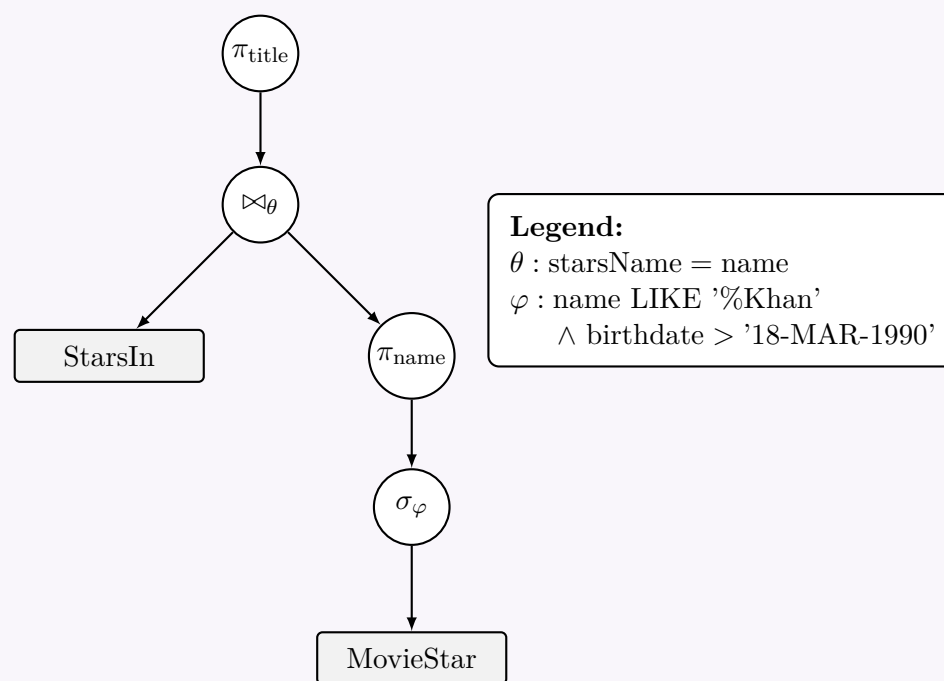
$$\pi_{\text{title}} \left(\text{StarsIn} \bowtie_{\text{starsName}=\text{name}} \left(\pi_{\text{name}} \left(\sigma_{\text{name LIKE '%Khan' \wedge \text{birthdate} > '18-MAR-1990'}} (\text{MovieStar}) \right) \right) \right)$$

Step-by-Step Breakdown:

- Selection (σ):** Filters the **MovieStar** relation for actors whose name ends with 'Khan' and who were born after the specified date.
- Projection (π):** Extracts only the **name** attribute from the filtered **MovieStar** records, forming the subquery result.
- Join ($\bowtie$):** Performs an equijoin between **StarsIn** and the subquery result on the condition **starsName = name**.
- Projection (π):** Extracts the **title** attribute to produce the final result set.

2. Logical Query Tree

To ensure the tree remains readable and the operators fit perfectly inside circular nodes, the long filtering and join conditions are represented by θ and φ and are defined in the accompanying legend.



Q8. Show a logical query plan for the following query:

```

SELECT title
FROM StarsIn
WHERE starName IN (
  SELECT name
  FROM MovieStar
  WHERE birthdate LIKE '%1960'
);

```

(8 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer**1. Relational Algebra Translation**

The given SQL query features a subquery linked by the **IN** operator. In relational algebra, a nested **IN** subquery that returns a single column can be unnested and converted into an **equijoin** ($\bowtie$) between the outer relation and the resulting inner relation.

The equivalent relational algebra expression is:

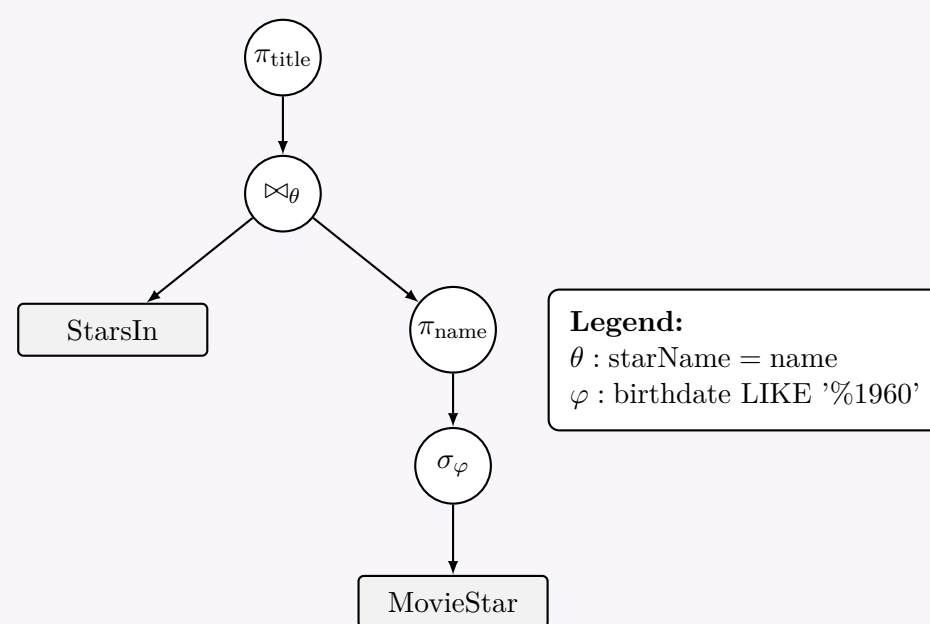
$$\pi_{\text{title}} \left(\text{StarsIn} \bowtie_{\text{starName}=\text{name}} \left(\pi_{\text{name}} \left(\sigma_{\text{birthdate LIKE } \%1960'} (\text{MovieStar}) \right) \right) \right)$$

Execution Sequence:

- Selection (σ):** Filter the **MovieStar** relation for tuples where the **birthdate** ends in '1960'.
- Projection (π):** Isolate the **name** attribute from the filtered **MovieStar** relation to form the result of the subquery.
- Join ($\bowtie$):** Perform an equijoin between **StarsIn** and the subquery's result using the condition **starName = name**.
- Projection (π):** Isolate the **title** attribute to produce the final dataset.

2. Logical Query Plan (Expression Tree)

The logical query plan represents the relational algebra expression as a tree structure. To keep the diagram neat and ensure the operators fit perfectly within their circular nodes, the join and selection conditions are denoted by variables θ and φ , which are detailed in the accompanying legend.



Joins

Q1. Differentiate between the Nested-Loop Join algorithm and the Block Nested-Loop Join algorithm. (5 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

Comparison of Nested-Loop Join and Block Nested-Loop Join

Join operations are fundamental to relational databases, but they can be highly expensive in terms of Disk I/O. The **Nested-Loop Join** and **Block Nested-Loop Join** are two foundational algorithms used to compute the natural join or theta join of two relations, usually denoted as an outer relation R and an inner relation S .

1. Nested-Loop Join (NLJ)

Definition: The standard Nested-Loop Join (also known as a naive or tuple-based nested-loop join) iterates through the relations on a **tuple-by-tuple** basis.

Working Principle: For every single tuple in the outer relation R , the algorithm scans the entire inner relation S to find matching tuples that satisfy the join condition.

Cost Analysis (I/O Operations): Let N_r be the total number of tuples in R , B_r be the number of disk blocks for R , and B_s be the number of disk blocks for S .

- The outer relation R is scanned exactly once, requiring B_r block transfers.
- For each of the N_r tuples in R , the entire inner relation S (B_s blocks) is read from the disk.

$$\text{Total Block Transfers} = B_r + (N_r \times B_s)$$

Note: This is highly inefficient because it results in a massive number of repeated disk reads for the inner relation.

2. Block Nested-Loop Join (BNLJ)

Definition: The Block Nested-Loop Join optimizes the standard approach by iterating through the relations on a **block-by-block** basis, utilizing memory buffers efficiently to minimize disk I/O.

Working Principle: Instead of bringing a single tuple of R into memory, the database brings an entire block (or a set of blocks) of R into memory. It then reads a block of S and joins all matching tuples in memory before discarding the block of S and bringing in the next one.

Cost Analysis (I/O Operations): Assuming a worst-case scenario where the memory buffer can only hold one block of R and one block of S :

- The outer relation R is read once, requiring B_r block transfers.
- For each of the B_r blocks of R , the entire inner relation S (B_s blocks) is scanned.

$$\text{Total Block Transfers} = B_r + (B_r \times B_s)$$

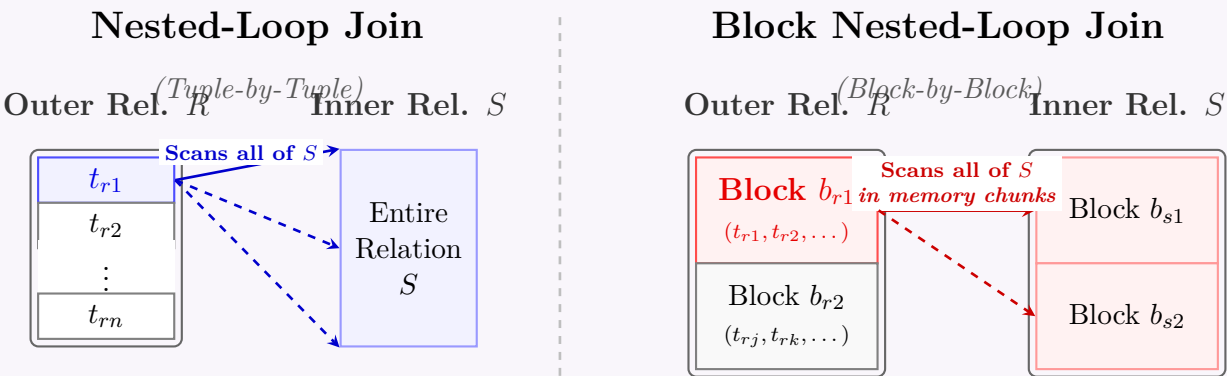
Note: If M memory buffer blocks are available, $M - 2$ blocks can be allocated to R , reducing the cost to: $B_r + \lceil \frac{B_r}{M-2} \rceil \times B_s$.

3. Core Differences Comparison Table

Feature	Nested-Loop Join (NLJ)	Block Nested-Loop Join (BNLJ)
Granularity	Iterates tuple-by-tuple .	Iterates block-by-block .
Memory Utilization	Very poor. Only uses memory to hold one tuple of R at a time.	High. Utilizes available buffer memory to hold multiple tuples (blocks) of R .
Inner Relation Scans	S is scanned N_r times (once per tuple in R).	S is scanned B_r times (once per block in R).
Disk I/O Cost	$B_r + (N_r \times B_s)$	$B_r + (B_r \times B_s)$
Performance	Extremely slow for large tables due to excessive disk access.	Significantly faster than NLJ as disk I/O operations are drastically reduced.

4. Visual Representation of Processing Mechanisms

The following diagram illustrates the fundamental difference in how the inner relation S is scanned relative to the outer relation R .



Q2. Assuming no indexes exist, which one would you apply for faster operation: a Hash Join or a Sort-Merge Join? Explain why. (6 marks)

CSE 215 | L-2/T-1 | 2024–25

Answer

Choice of Join Algorithm: Hash Join vs. Sort-Merge Join

Assuming no indexes exist on either relation and the data is currently unsorted, a **Hash Join** is generally **faster and the preferred choice**, provided the join condition is an **equi-join** (e.g., $R.A = S.B$). If the join condition contains inequalities (e.g., $R.A < S.B$) or if the output is explicitly required to be sorted, a **Sort-Merge Join** becomes necessary. However, for standard relational operations prioritizing speed, Hash Join outperforms Sort-Merge Join due to its linear I/O cost compared to the logarithmic-linear cost of sorting.

1. Why Hash Join is Faster (Explanation)

The primary bottleneck in database operations is **Disk I/O**. The performance difference between these two algorithms comes down to the cost of hashing versus the cost of sorting large datasets.

- **Hash Join Cost (Linear $O(M + N)$):** A Hash Join scans the smaller relation to build an in-memory hash table, then scans the larger relation to probe the hash table for matches. Assuming the hash table fits in memory, each block of data is read exactly once. Even if data exceeds memory (requiring a Grace Hash Join), the data is partitioned to disk and read/written a small, constant number of times.

- **Sort-Merge Join Cost (Log-Linear $O(M \log M + N \log N)$):** A Sort-Merge join must first sort both relations before merging them. External sorting requires multiple passes over the data (reading and writing blocks iteratively) depending on the available buffer size. This creates significantly higher Disk I/O overhead compared to hashing.

2. Cost Analysis (I/O Block Transfers)

Let B_R and B_S be the number of disk blocks for relations R and S respectively.

- **Ideal Hash Join I/O Cost:** $B_R + B_S$ (if the smaller relation fits entirely in main memory).
- **Grace Hash Join I/O Cost:** $3(B_R + B_S)$ (requires partitioning both relations to disk first, then joining).
- **Sort-Merge Join I/O Cost:**

$$Cost_{sort}(R) + Cost_{sort}(S) + (B_R + B_S)$$

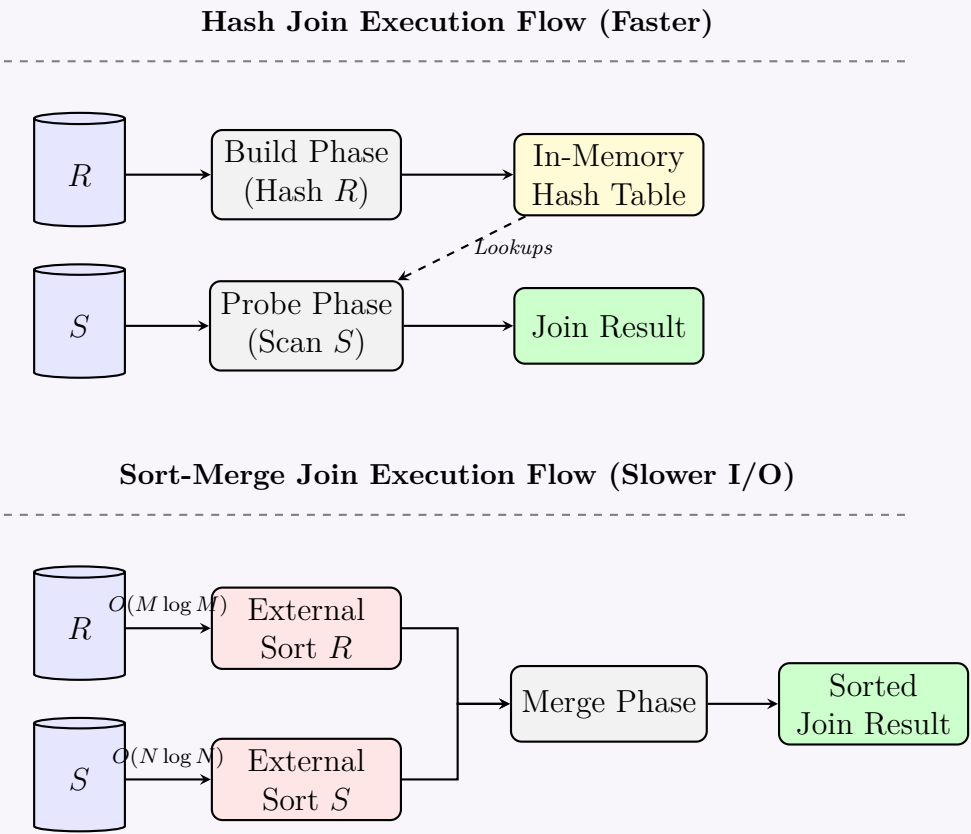
Where the cost to sort a relation of B blocks using M memory buffers is approximately $2B \lceil \log_{M-1}(B/M) \rceil$. This is strictly greater than the $3(B_R + B_S)$ bounds of a Hash Join for large datasets.

3. Comparison Table

Feature	Hash Join	Sort-Merge Join
Primary Speed	Faster (Requires fewer I/O passes)	Slower (Requires multiple sorting passes)
Join Condition	Strictly Equi-Joins (=) only.	Both Equi-Joins and Non-Equi-Joins (<, >).
Output Order	Unsorted / Random.	Sorted based on the join key.
Memory Sensitivity	Highly sensitive; severe performance drop if data skew causes hash bucket overflow.	Less sensitive; sorting degrades gracefully with limited memory.

4. Visual Comparison of Execution Flows

The following diagram illustrates why Hash Join is generally more direct than the multi-pass Sort-Merge approach.



Final Verdict: Use **Hash Join** for pure performance on equi-joins when tables are unsorted. Use **Sort-Merge Join** only if the query specifically demands a sorted output, or if the join involves inequality conditions where hashing cannot be applied.

Q3. Describe the hybrid hash join algorithm. Compute the memory and I/O cost of the algorithm. (11 marks) [CSE 215 | L-2/T-1 | 2023–24]

Answer

Hybrid Hash Join Algorithm

1. Definition

The **Hybrid Hash Join** is an advanced, highly optimized join algorithm designed to compute the equi-join of two relations. It combines the speed of the *Simple (In-Memory) Hash Join* with the scalability of the *Grace Hash Join*. It achieves this by utilizing available main memory to keep the first partition of the build relation entirely in memory, thus preventing it from ever being written to disk, while partitioning the remainder of the relations to disk.

2. Working Principle

Let the two relations to be joined be R (the smaller, **build** relation) and S (the larger, **probe** relation). The algorithm proceeds in three distinct phases:

(a) Partitioning and Building Phase (Relation R):

- Read R block by block into memory.
- Apply a hash function h_1 to the join key to divide R into $k + 1$ partitions: $R_0, R_1, \dots, R_k$.
- Keep partition R_0 *entirely in main memory* and build an in-memory hash table for it using a second hash function h_2 .
- For partitions R_1 to R_k , write them out to corresponding files on disk using output buffers.

(b) Probing and Partitioning Phase (Relation S):

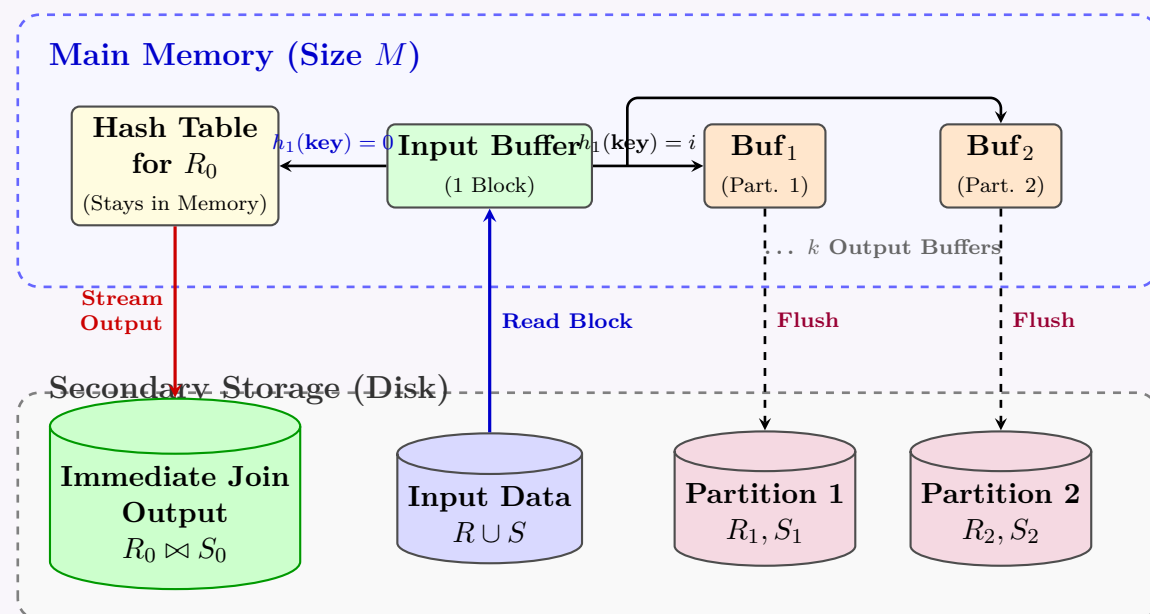
- Read S block by block into memory.
- Apply the same hash function h_1 to the join key of each tuple in S .
- If the tuple hashes to partition 0 (i.e., S_0), immediately probe the in-memory hash table of R_0 using h_2 . If a match is found, output the joined tuple.
- If the tuple hashes to partitions 1 to k , write the tuple to the corresponding partition on disk ($S_1, \dots, S_k$).

(c) Disk Join Phase (Partitions 1 to k):

- For each partition pair (R_i, S_i) where $i \in \{1, \dots, k\}$, perform a standard in-memory hash join (since R_i is sized to fit entirely in memory).
- Output the resulting joined tuples.

3. Memory Layout and Visual Representation

The following diagram illustrates the state of the memory and disk during the initial partitioning and probing phases.



4. Cost Analysis (Memory and I/O)

Let $B(R)$ and $B(S)$ be the number of disk blocks occupied by relations R and S respectively. Let M be the total number of available main memory buffer blocks.

A. Memory Requirements

During the partitioning phase, the memory must hold:

- **1 block** for the input buffer (to read R and S).
- **k blocks** for the output buffers (to write partitions $R_1 \dots R_k$ and $S_1 \dots S_k$ to disk).
- The entire **Hash Table for R_0** .

Thus, the condition for memory constraint is:

$$B(R_0) + k + 1 \leq M \implies B(R_0) \leq M - k - 1$$

For maximum efficiency, R_0 is made as large as possible, meaning $B(R_0) \approx M - k$.

B. I/O Cost (Block Transfers)

The cost is heavily dependent on the fraction of the build relation R that can remain in memory. Let this fraction be q .

$$q = \frac{B(R_0)}{B(R)}$$

- **Initial Scan:** Both relations are read completely once. Cost = $B(R) + B(S)$.
- **Disk Writes:** Only partitions $1 \dots k$ are written to disk. Since fraction q stays in memory, a fraction $(1 - q)$ of both R and S is written to disk. Cost = $(1 - q) \cdot (B(R) + B(S))$.
- **Disk Reads (Merge Phase):** The disk-resident partitions $(1 \dots k)$ must be read back into memory to be joined. Cost = $(1 - q) \cdot (B(R) + B(S))$.

Total I/O Cost Equation:

$$\begin{aligned} \text{Cost} &= (B(R) + B(S)) + 2 \cdot (1 - q)(B(R) + B(S)) \\ \text{Cost} &= (3 - 2q) \cdot (B(R) + B(S)) \end{aligned}$$

C. Boundary Cases Comparison Table

Case	Memory Size (M)	Fraction (q)	Total I/O Cost
Ideal (Simple Hash)	$M > B(R)$	$q = 1$ (All of R in memory)	$B(R) + B(S)$
Hybrid Hash	$B(R) > M > \sqrt{B(R)}$	$0 < q < 1$	$(3 - 2q)(B(R) + B(S))$
Worst (Grace Hash)	$M \approx \sqrt{B(R)}$	$q = 0$ (No R_0 , all to disk)	$3(B(R) + B(S))$

Note: The Hybrid Hash Join effectively interpolates between the Simple Hash Join and the Grace Hash Join, dynamically adapting its disk usage based on the available memory size M .

Q4. Construct a one-and-a-half pass algorithm for computing $R \text{ LOJ } S$ (natural left outer join), where R is the smaller relation. Compute the memory and disk I/O requirements. **(10 marks)** [CSE 215 | L-2/T-1 | 2022–23]

Answer

One-and-a-Half Pass Algorithm for Natural Left Outer Join ($R \text{ LOJ } S$)

1. Definition and Working Principle

A **one-and-a-half pass algorithm** (often implemented as a Chunk-based or Block Nested-Loop Hash Join) is used when one relation is significantly smaller than the other. In this approach, the smaller relation (R) is read into main memory in chunks (or entirely, if memory permits) and an in-memory search structure is built. The larger relation (S) is then streamed block-by-block from disk to probe for matches.

For a **Left Outer Join (LOJ)**, every tuple from the left relation R must appear in the final result. If a tuple in R finds no matching tuple in S , it must be padded with NULL values. Because R is the smaller relation, making it the *outer* (or memory-resident) relation is highly optimal, as we can easily track which tuples of R have found a match during the streaming of S .

2. Algorithm Steps

Let M be the total number of available main memory blocks.

- Memory Allocation:** Allocate $M - 2$ blocks to hold a chunk of relation R . Allocate 1 block as an input buffer for S , and 1 block as an output buffer.
- Load Chunk of R :** Read up to $M - 2$ blocks of relation R into the allocated memory.
- Build In-Memory Structure:** Organize the tuples of this R -chunk into an in-memory hash table based on the join attributes. Crucially, augment each R tuple in the hash table with a boolean flag **is_matched**, initialized to **FALSE**.
- Stream Relation S :** Read relation S block-by-block into the S input buffer.
- Probe and Match:** For each tuple $s \in S$:
 - Probe the in-memory hash table for matching R tuples.
 - If a match r is found, generate the joined tuple (r, s) , send it to the output buffer, and set $r.\text{is_matched} = \text{TRUE}$.
- NULL Padding:** Once all blocks of S have been processed against the current R -chunk, iterate through the hash table. For every tuple r where $r.\text{is_matched} == \text{FALSE}$, generate a tuple (r, NULL) and send it to the output buffer.

(g) **Repeat:** Clear the R -chunk from memory and repeat Steps 2 through 6 for the next chunk of R , until all of R is processed.

3. Memory and Disk I/O Requirements

Memory Requirement (M):

- **Minimum Memory:** $M \geq 3$ blocks (1 for R , 1 for S , 1 for Output).
- **Ideal Memory:** $M \geq B(R) + 2$ blocks, where $B(R)$ is the total number of blocks of R . If this condition is met, R fits entirely in memory, and the algorithm completes in a strict **single pass** over both relations.

Disk I/O Cost: Let $B(R)$ and $B(S)$ be the number of disk blocks for R and S , respectively.

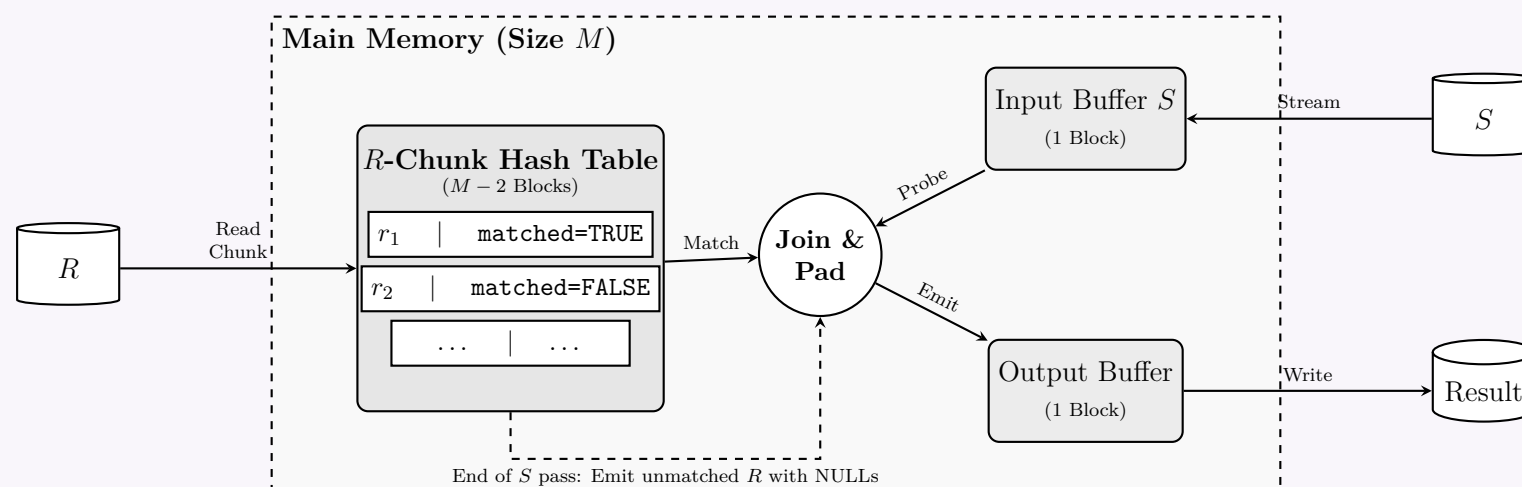
- Relation R is read exactly once: $B(R)$ I/Os.
- For every chunk of R , relation S is read completely. The number of chunks is $\lceil \frac{B(R)}{M-2} \rceil$.
- **Total Disk I/O Cost:**

$$\text{Cost} = B(R) + \left(\left\lceil \frac{B(R)}{M-2} \right\rceil \times B(S) \right)$$

Note: If $B(R) \leq M - 2$, the cost simplifies to $B(R) + B(S)$.

4. Visual Representation of the Memory Architecture

The following diagram illustrates the state of main memory during the execution of a single chunk in the LOJ algorithm.



Q5. Assume R and S are two database relations with $B(R) = 8000$, $B(S) = 4000$, and the number of available memory buffers $M = 101$. Compute the I/O cost of the natural join operation for the following algorithms:

- Block-based nested loop join.
- Sort-based nested loop join.
- Hash-based nested loop join.

(8 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

I/O Cost Computation for Join Algorithms

Given Parameters

- Size of relation R in blocks: $B(R) = 8000$
- Size of relation S in blocks: $B(S) = 4000$
- Available main memory buffer blocks: $M = 101$

1. Block-Based Nested Loop Join (BNLJ)

Working Principle: In BNLJ, the *outer* relation is read into memory in chunks. For each chunk, the entire *inner* relation is scanned to find matching tuples. To minimize disk I/O, we allocate as much memory as possible to the outer relation's chunk. Out of $M = 101$ buffers:

- **Outer relation chunk size:** $M - 2 = 99$ blocks.
- **Inner relation input buffer:** 1 block.

- **Output buffer:** 1 block.

Cost Formula:

$$\text{Cost} = B(\text{Outer}) + \left\lceil \frac{B(\text{Outer})}{M-2} \right\rceil \times B(\text{Inner})$$

Option A: S as the Outer Relation (Smaller relation)

$$\text{Number of chunks of } S = \left\lceil \frac{4000}{99} \right\rceil = \lceil 40.40 \rceil = 41 \text{ chunks}$$

$$\begin{aligned} \text{Cost} &= 4000 + (41 \times 8000) \\ &= 4000 + 328,000 \\ &= \mathbf{332,000} \text{ block transfers} \end{aligned}$$

Option B: R as the Outer Relation (Larger relation)

$$\text{Number of chunks of } R = \left\lceil \frac{8000}{99} \right\rceil = \lceil 80.80 \rceil = 81 \text{ chunks}$$

$$\begin{aligned} \text{Cost} &= 8000 + (81 \times 4000) \\ &= 8000 + 324,000 \\ &= \mathbf{332,000} \text{ block transfers} \end{aligned}$$

Note: In this specific scenario, both choices yield the same I/O cost due to the ceiling function behavior, but generally, picking the smaller relation as the outer relation is preferred.

2. Sort-Based Nested Loop Join (Sort-Merge Join)

Working Principle: The standard approach is the **Sort-Merge Join (SMJ)**. Both relations are first sorted based on the join attribute using Two-Phase External Merge Sort, and then merged together. *Condition for 2-pass sort:* A relation of size B can be sorted in two passes if $B \leq M \times (M-1)$. Here, $101 \times 100 = 10,100$. Since $8000 \leq 10,100$ and $4000 \leq 10,100$, both relations can be sorted in exactly two passes.

Standard Unoptimized Cost (Sort + Merge completely separate):

- **Sort R :** $4 \times B(R) = 4 \times 8000 = 32,000$ I/Os
- **Sort S :** $4 \times B(S) = 4 \times 4000 = 16,000$ I/Os
- **Merge Join:** $B(R) + B(S) = 8000 + 4000 = 12,000$ I/Os

$$\text{Total Unoptimized Cost} = 32,000 + 16,000 + 12,000 = \mathbf{60,000} \text{ block transfers}$$

Optimized Cost (On-the-fly Merge): Modern DBMSs optimize SMJ by combining the final merge pass of the sorting algorithm with the actual join phase. This saves writing the final sorted runs to disk and reading them back.

$$\begin{aligned} \text{Optimized Cost} &= 3 \times (B(R) + B(S)) \\ &= 3 \times (8000 + 4000) \\ &= 3 \times 12,000 \\ &= \mathbf{36,000} \text{ block transfers} \end{aligned}$$

3. Hash-Based Nested Loop Join (Two-Pass Hash Join)

Working Principle: Known commonly as **Grace Hash Join**, this algorithm partitions both relations into k buckets on disk using a hash function, and then joins corresponding pairs of buckets in memory. *Condition for 2-pass hash join:* The smaller relation's partitions must fit entirely in memory. Partition size $\approx B(S)/(M-1)$. We need $B(S)/(M-1) \leq M-2$. Here, $4000/100 = 40 \leq 99$, so the memory requirement is easily satisfied.

Cost Formula: The algorithm requires one pass to partition the data (read and write) and one pass to probe (read).

- **Partitioning Phase (Pass 1):** Read R and S , hash them, and write partitions to disk.

$$\text{Cost} = 2 \times (B(R) + B(S)) = 2 \times 12,000 = 24,000 \text{ I/Os}$$

- **Probing Phase (Pass 2):** Read each partition pair into memory and perform the join.

$$\text{Cost} = B(R) + B(S) = 12,000 \text{ I/Os}$$

$$\begin{aligned} \text{Total Cost} &= 3 \times (B(R) + B(S)) \\ &= \mathbf{36,000} \text{ block transfers} \end{aligned}$$

Summary Comparison Table		
Join Algorithm	Calculated I/O Cost	Remarks
Block-Based Nested Loop	332,000	Heavily constrained by small memory $M = 101$.
Sort-Merge Join (Standard)	60,000	Both relations require a 2-pass external sort.
Sort-Merge Join (Optimized)	36,000	Final sort pass combined with join phase.
Hash-Based Join	36,000	Ideal due to proportional memory allocation.

Q6. Give a two-pass hash-based algorithm for computing the set difference $R - S$. Analyse the memory and disk I/O requirements of your algorithm. (10 marks) [CSE 215 | L-2/T-1 | 2022–23]

Answer

Two-Pass Hash-Based Algorithm for Set Difference ($R - S$)

1. Definition and Working Principle

The set difference operation $R - S$ returns all tuples that exist in relation R but not in relation S . A two-pass hash-based algorithm achieves this efficiently for large relations by partitioning the data to disk in the first pass, and then computing the difference on smaller, memory-sized partitions in the second pass.

The algorithm operates in two distinct phases:

(a) **Pass 1: Partitioning Phase**

- Allocate M main memory buffers: 1 for input and $M - 1$ for output.
- Choose a hash function h_1 that maps a tuple to one of $k = M - 1$ buckets.
- Stream relation R block-by-block into the input buffer. Hash each tuple using h_1 and place it into the corresponding output buffer. When an output buffer fills, write it to disk as partition R_i (where $i \in \{1, \dots, k\}$).
- Repeat the exact same partitioning process for relation S using the **same hash function** h_1 , creating disk partitions $S_1, \dots, S_k$.

(b) **Pass 2: Probing / Difference Phase**

- For each bucket index i from 1 to k :
- Load S_i :** Read all blocks of partition S_i into main memory and build an in-memory hash table using a second hash function h_2 (where $h_2 \neq h_1$ to avoid clustering).
- Stream R_i :** Read partition R_i block-by-block.
- Probe:** For each tuple $t \in R_i$, probe the in-memory hash table of S_i using h_2 .
- Output:** If t is **not found** in the hash table of S_i , output t to the final result $R - S$. (If it is found, it is discarded).
- Clear the memory and proceed to the next bucket $i + 1$.

2. Memory Requirements

Let $B(R)$ and $B(S)$ be the number of disk blocks for relations R and S , respectively, and M be the total available main memory blocks.

- Pass 1 Requirement:** We need 1 input buffer and k output buffers, so $M = k + 1$.
- Pass 2 Requirement:** The in-memory hash table for partition S_i must fit entirely in memory. Assuming a uniform hash distribution, the size of each partition is approximately $B(S)/k \approx B(S)/(M - 1)$.
- Therefore, to hold S_i in memory along with 1 input block for R_i and 1 output block, we require:

$$\frac{B(S)}{M - 1} \leq M - 2 \implies M \approx \sqrt{B(S)}$$

Note: If R is significantly smaller than S , we can alternatively build the hash table on R_i and stream S_i , marking matched tuples in R_i and outputting the unmarked ones at the end of the bucket. Thus, the strict memory requirement is $M > \min(\sqrt{B(R)}, \sqrt{B(S)})$.

3. Disk I/O Cost Analysis

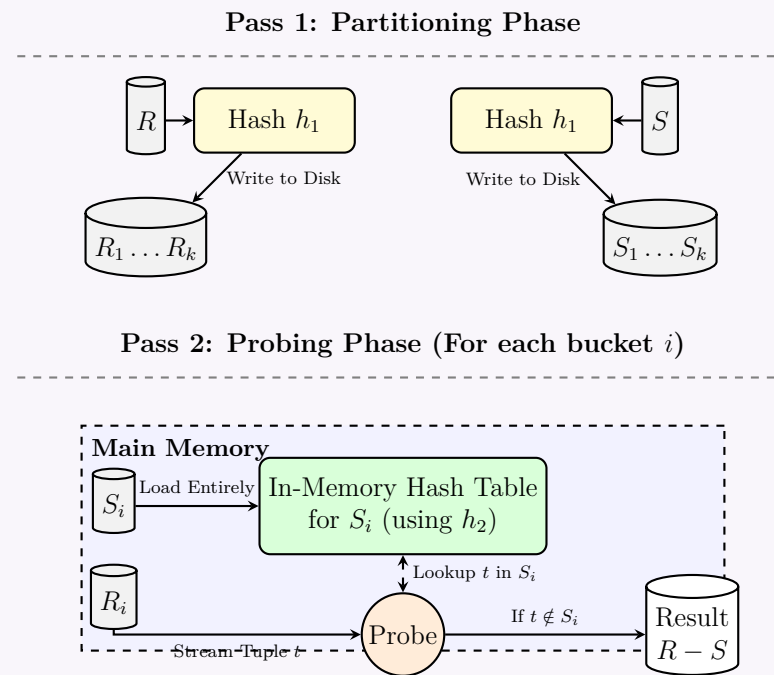
The disk I/O cost is measured in terms of block transfers. The cost of writing the final output is conventionally omitted in DBMS analysis.

- Pass 1 (Partitioning):** Read all of R and S , and write all partitions to disk.
Cost = $2 \times (B(R) + B(S))$
- Pass 2 (Probing):** Read each partition R_i and S_i exactly once from disk.
Cost = $1 \times (B(R) + B(S))$

Total I/O Cost:

Total Cost = $3 \times (B(R) + B(S))$ block transfers.

4. Visual Representation



Q7. What is the I/O complexity of needed loop join ? (3 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

I/O Complexity of Nested Loop Join

The I/O complexity of a Nested Loop Join depends heavily on the specific variation of the algorithm being used and the amount of available main memory. In all variations, let R be the **outer relation** and S be the **inner relation**.

Standard Notation:

- $B(R), B(S)$: Total number of disk blocks for relations R and S .
- $|R|, |S|$: Total number of tuples in relations R and S .
- M : Total number of available main memory buffer blocks.

1. Tuple-based Nested Loop Join

In the most naive implementation, for every single tuple in the outer relation R , the entire inner relation S is scanned from disk.

- **Memory Requirement:** $M \geq 3$ blocks (1 for R , 1 for S , 1 for output).
- **I/O Cost:**

$$\text{Cost} = B(R) + (|R| \times B(S)) \text{ block transfers}$$

- **Explanation:** R is read once (block by block). For each of the $|R|$ tuples, all $B(S)$ blocks of S are read. This is extremely inefficient.

2. Block-based Nested Loop Join (BNLJ)

Instead of iterating tuple-by-tuple, BNLJ iterates block-by-block. For every block (or chunk of blocks) of the outer relation R , the entire inner relation S is scanned.

A. Basic BNLJ (Minimum Memory)

If only the minimum memory is available ($M = 3$):

- **I/O Cost:**

$$\text{Cost} = B(R) + (B(R) \times B(S)) \text{ block transfers}$$

B. Chunk-based BNLJ (Optimized)

If larger memory is available ($M > 3$), we can load $M - 2$ blocks of the outer relation R into memory at once, leaving 1 block for reading S and 1 block for the output buffer.

- **I/O Cost:**

$$\text{Cost} = B(R) + \left(\left\lceil \frac{B(R)}{M-2} \right\rceil \times B(S) \right) \text{ block transfers}$$

- **Explanation:** The outer relation is read exactly once ($B(R)$). The number of times the inner relation S is fully scanned equals the number of chunks of R , which is $\lceil B(R)/(M-2) \rceil$.
- **Optimization Strategy:** Always choose the **smaller** relation as the outer relation to minimize the multiplier $\lceil B(R)/(M-2) \rceil$. If $B(R) \leq M - 2$, the cost reduces to a single pass: $B(R) + B(S)$.

3. Index Nested Loop Join (INLJ)

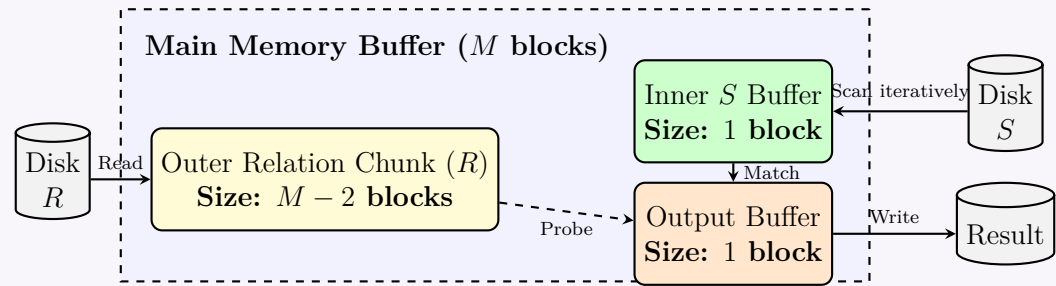
If an index exists on the join attribute of the inner relation S , we do not need to scan the entirety of S for every tuple of R .

- **I/O Cost:**

$$\text{Cost} = B(R) + (|R| \times C) \text{ block transfers}$$

- **Explanation:** C is the average cost to traverse the index and retrieve the matching tuple(s) in S . For a B+ tree, C is typically the height of the tree plus one data block access.

Visual Memory Layout for Optimized Chunk-based BNLJ



BUET · CSE 215 · Database

Part V

Transaction Management & Advanced Topics

Transaction Management & Advanced Topics

S7 — Transaction Management

Transactions & ACID Properties

Q1. Explain the ACID properties of a transaction with an example. (5 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

ACID Properties of a Transaction

1. Definition

A **transaction** is a logical unit of work consisting of one or more database operations (such as **READ** or **WRITE**). To ensure data integrity, reliability, and robustness in a Database Management System (DBMS), every transaction must strictly adhere to four fundamental properties, collectively known as the **ACID** properties.

2. Explanation of ACID Properties

(a) Atomicity (The “All or Nothing” Rule)

- **Definition:** A transaction is treated as an indivisible atomic unit. Either all its operations are executed successfully and reflected in the database, or none are.
- **Working Principle:** The DBMS uses a **Transaction Manager** and log files (Undo log). If a failure occurs before successful completion, the system performs a **rollback** to undo any partial changes, returning the database to its pre-transaction state.

(b) Consistency (Integrity Preservation)

- **Definition:** The execution of a transaction in isolation must preserve the consistency of the database. It transforms the database from one valid state to another valid state.
- **Working Principle:** The DBMS enforces **integrity constraints** (e.g., primary keys, foreign keys, check constraints). If a transaction violates any constraint, it is aborted.

(c) Isolation (Concurrency Independence)

- **Definition:** Multiple transactions executing concurrently must not interfere with each other. The intermediate state of a transaction must be invisible to other transactions until it commits.
- **Working Principle:** The DBMS uses a **Concurrency Control Manager** (employing mechanisms like Locks or Timestamp Ordering) to ensure that the concurrent execution results in a system state equivalent to serial execution (Serializability).

(d) Durability (Persistence)

- **Definition:** Once a transaction successfully commits, its changes are permanently recorded in the database and must survive any subsequent system failures (e.g., power loss or crashes).
- **Working Principle:** The DBMS employs **Recovery Management** using Write-Ahead Logging (Redo log). Changes are safely written to non-volatile storage before the transaction is officially marked as committed.

3. Example: Bank Fund Transfer

Scenario: Transfer \$100 from Account *A* to Account *B*.

Initial State: $A = \$500$, $B = \$200$. (Total = \$700)

The transaction (T_1) consists of the following operations:

- Read(*A*)
- $A = A - 100$
- Write(*A*)
- Read(*B*)
- $B = B + 100$
- Write(*B*)
- Commit

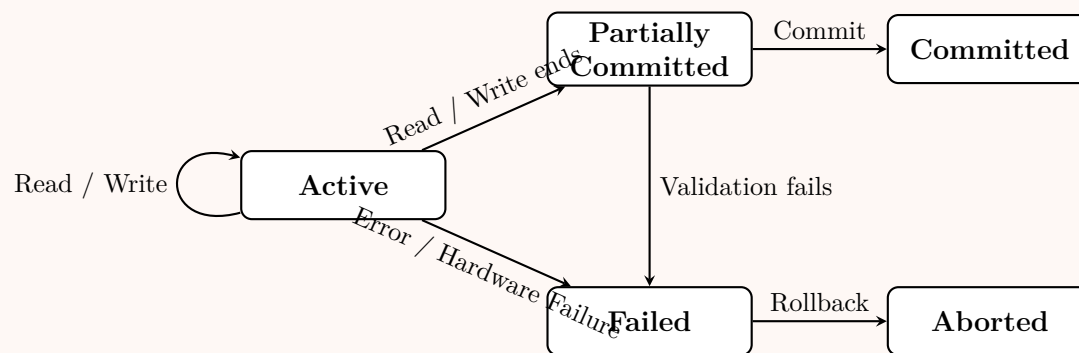
How ACID applies to this transaction:

- **Atomicity:** Suppose a system crash occurs right after Step 3. *A* has been deducted, but *B* has not been credited. Upon restart, the DBMS will **rollback** the change to *A*, restoring it to \$500, preventing the loss of \$100.
- **Consistency:** The business rule dictates that the total sum of funds must remain constant. Before T_1 , Total = \$700. After successful completion, $A = \$400$ and $B = \$300$. Total = \$700. Consistency is maintained.

- **Isolation:** If another transaction T_2 attempts to read A after Step 3 but before Step 7, it would read an intermediate, uncommitted value (a “dirty read”). Isolation mechanisms ensure T_2 waits until T_1 completes before it can read A .
- **Durability:** If the system crashes a millisecond after Step 7 (Commit), the changes ($A = 400, B = 300$) are guaranteed to be permanently saved on the disk and available upon reboot.

4. Transaction State Transition Diagram

The lifecycle of a transaction enforcing ACID properties can be visualized as follows:



Q2. Show the relationship among memory buffer, transaction work area and disk storage. Explain how data A is read from disk for a transaction and data B is written to the disk for a transaction. (10 marks) [CSE 215 | L-2/T-2 | 2020–21]

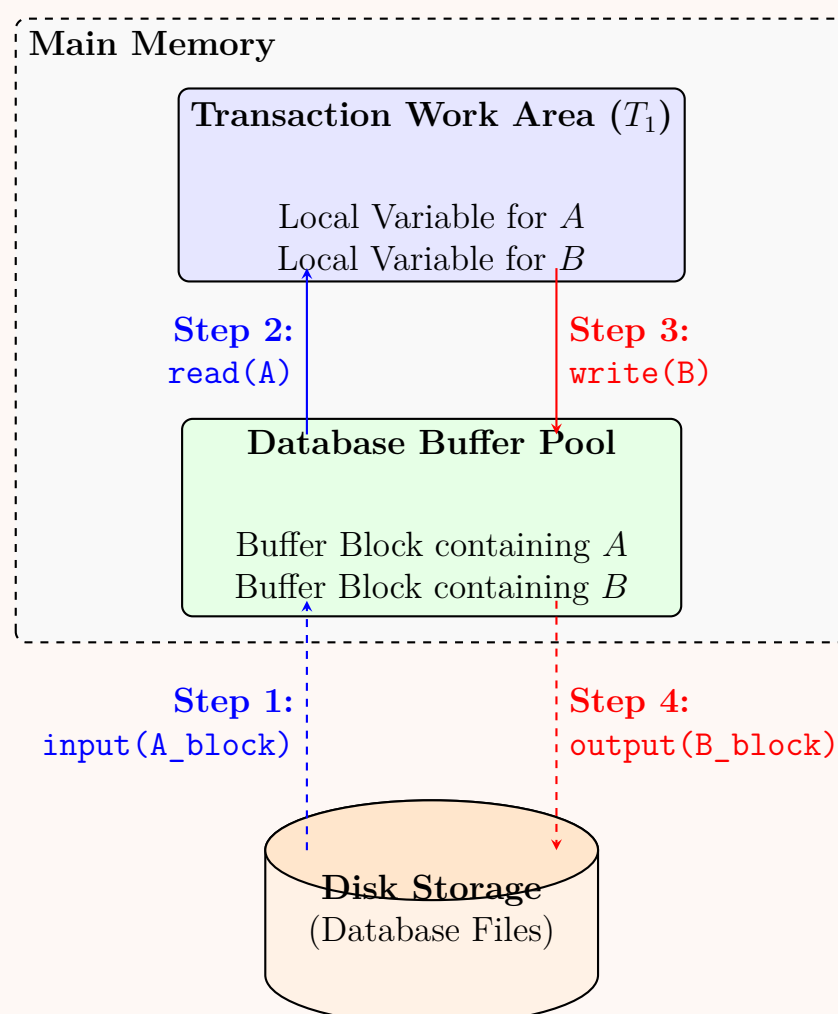
Answer

Relationship Among Memory Buffer, Transaction Work Area, and Disk Storage

In a Database Management System (DBMS), data transfers between non-volatile storage and main memory are crucial for transaction processing. The architecture is composed of three primary storage regions:

- Disk Storage (Database):** The permanent, non-volatile storage where the actual database files and data blocks reside. It is slow to access but offers massive capacity.
- Database Buffer (Memory Buffer):** A reserved portion of main memory (RAM) managed by the DBMS Buffer Manager. It acts as a cache, holding copies of recently or frequently accessed disk blocks. It minimizes slow disk I/O by serving data directly from memory whenever possible.
- Transaction Work Area:** A private memory space allocated for each executing transaction. It holds local variables, intermediate computational results, and copies of data items that the transaction is actively reading or modifying.

Core Relationship: A transaction *cannot* directly manipulate data on the disk. Instead, physical data blocks must first be fetched into the **Database Buffer**. From there, specific data items are copied into the **Transaction Work Area** for processing. After modification, data is written back to the Buffer, and the DBMS eventually flushes the updated blocks back to the **Disk Storage**.



Execution of Database Operations

The transfer of data between these components relies on specific system operations: `input()`, `read()`, `write()`, and `output()`.

How Data A is Read from Disk

When a transaction T_1 requests to read data item A , the following sequence occurs:

- **Buffer Check:** The DBMS first checks if the disk block containing data item A is already residing in the Database Buffer.
- **Step 1: `input(A_block)`.** If the block is *not* in the buffer (a cache miss), the Buffer Manager issues an `input` operation to locate the block on Disk Storage and copy it into an available frame in the Database Buffer.
- **Step 2: `read(A)`.** Once the block is in the buffer, the DBMS executes the `read` operation. It extracts the specific value of data item A from the buffer block and copies it into the local variable of the Transaction Work Area for T_1 .

How Data B is Written to Disk

When transaction T_1 modifies data item B and wishes to save it, the process is as follows:

- **Local Modification:** The transaction performs calculations and updates the value of B in its private Transaction Work Area.
- **Step 3: `write(B)`.** The transaction issues a `write` command. The DBMS copies the updated value of B from the Transaction Work Area back into the corresponding block within the Database Buffer. *Note: At this exact moment, the data is not yet on the disk.* The buffer block is simply marked as "dirty" (modified).
- **Step 4: `output(B_block)`.** Eventually, based on the buffer replacement policy or during the transaction commit/checkpointing phase, the Buffer Manager issues an `output` operation. This physically flushes the "dirty" block containing the updated B from the Database Buffer back to the Disk Storage, making the change persistent.

Q3. Two transactions are given:

- Transaction T_1 transfers the sum of 20% of balances of accounts A and B to account C .
- Transaction T_2 transfers Tk. 2000 from account D to account C .

Prepare a concurrent schedule for the above two transactions and prove that the schedule is conflict serializable. **(15 marks)** [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Transaction Definitions

Based on the problem statement, we first define the sequence of operations for both transactions. A “transfer” implies deducting the amount from the source account(s) and adding it to the destination account.

Transaction T_1 : Transfers 20% of A ’s balance and 20% of B ’s balance to C .

1. Read the balance of A and deduct 20%.
2. Read the balance of B and deduct 20%.
3. Read the balance of C and add the deducted amounts.

Transaction T_2 : Transfers Tk. 2000 from D to C .

1. Read the balance of D and deduct 2000.
2. Read the balance of C and add 2000.

2. Concurrent Schedule (S)

We construct a concurrent schedule S where operations from T_1 and T_2 are interleaved. To ensure conflict serializability, all conflicting operations on shared data items (in this case, only account C) must be executed in a consistent order. Here, we choose to execute T_1 ’s operations on C before T_2 ’s operations on C .

Transaction T_1	Transaction T_2
$R(A)$	
$temp_A := A \times 0.2$	
$A := A - temp_A$	
$W(A)$	
	$R(D)$
	$D := D - 2000$
	$W(D)$
$R(B)$	
$temp_B := B \times 0.2$	
$B := B - temp_B$	
$W(B)$	
$R(C)$	
$C := C + temp_A + temp_B$	
$W(C)$	
	$R(C)$
	$C := C + 2000$
	$W(C)$

3. Proof of Conflict Serializability

To prove that the concurrent schedule S is conflict serializable, we must construct its **Precedence Graph** (Serialization Graph) and show that it is acyclic.

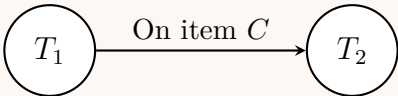
Identification of Conflicts

Two operations conflict if they belong to different transactions, access the same data item, and at least one is a Write (W) operation. By analyzing schedule S , the only shared data item accessed by both T_1 and T_2 is **Account C** . The conflicting operations are:

- $W_1(C)$ and $R_2(C) \implies T_1$ writes C before T_2 reads C . (Creates edge $T_1 \rightarrow T_2$)
- $R_1(C)$ and $W_2(C) \implies T_1$ reads C before T_2 writes C . (Creates edge $T_1 \rightarrow T_2$)
- $W_1(C)$ and $W_2(C) \implies T_1$ writes C before T_2 writes C . (Creates edge $T_1 \rightarrow T_2$)

Operations on A , B , and D do not generate any conflicts since they are exclusively accessed by only one transaction.

Precedence Graph



Conclusion

The precedence graph contains directed edges only from T_1 to T_2 . Because there are **no cycles** in the precedence graph, the concurrent schedule S is strictly **conflict serializable**. Furthermore, it is computationally equivalent to the serial schedule where T_1 executes entirely before T_2 (i.e., $T_1 \rightarrow T_2$).

Q4. Briefly explain each of the ACID properties in DBMS. (8 marks)

[CSE 303 | L-3/T-1 | 2016–17]

$\hookrightarrow$ Similar: 2017–18 / 2018–19: “Explain the ACID properties of a transaction with an example. Why are these properties essential for the concurrent execution of transactions?”

Answer

ACID Properties of a Transaction

A **transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. To ensure data integrity, reliability, and correctness especially in a concurrent processing environment every database transaction must strictly guarantee four fundamental properties, collectively known by the acronym **ACID**.

1. Explanation of ACID Properties

- Atomicity (The “All-or-Nothing” Rule):**
 - Definition:** A transaction is an indivisible unit of execution. Either all of its operations are executed completely and successfully, or none of them are executed at all.
 - Working Principle:** If a transaction fails mid-execution (e.g., due to a system crash or logical error), the DBMS **Recovery Manager** performs a *rollback* to undo any partial changes, returning the database to its previous stable state.
- Consistency (Integrity Preservation):**

- **Definition:** The execution of a transaction in isolation must preserve the consistency of the database. It must transition the database from one valid state to another valid state.
- **Working Principle:** The transaction must not violate any database constraints (e.g., primary keys, foreign keys, check constraints). If a transaction attempts to commit an invalid state, it is aborted by the **Integrity Subsystem**.
- **Isolation (Concurrency Control):**
 - **Definition:** Multiple transactions executing concurrently must not interfere with one another. The intermediate (un-committed) state of a transaction must remain invisible to other concurrent transactions.
 - **Working Principle:** The **Concurrency Control Manager** uses mechanisms like locking, timestamping, or multiversion concurrency control (MVCC) to ensure that concurrent execution results in a system state equivalent to a strictly serial execution (Serializability).
- **Durability (Persistence):**
 - **Definition:** Once a transaction has successfully committed, its changes must be permanently recorded in the database and must survive any subsequent system failures (e.g., power loss, crashes).
 - **Working Principle:** The **Recovery Manager** ensures this by writing transaction records to a non-volatile log file (Write-Ahead Logging) before acknowledging the commit.

2. Illustrative Example: Bank Fund Transfer

Suppose we have two accounts, A and B , with initial balances of \$500 and \$200 respectively. A transaction T_1 transfers \$100 from A to B .

- (a) Read(A)
- (b) $A = A - 100$
- (c) Write(A)
- (d) Read(B)
- (e) $B = B + 100$
- (f) Write(B)
- (g) Commit

How ACID applies:

- **Atomicity:** If the system crashes after step 3 (deducting from A) but before step 6 (adding to B), the database restores A back to \$500, preventing the loss of \$100.
- **Consistency:** The sum of balances ($A + B$) before the transaction is \$700. After the transaction, $A = 400$ and $B = 300$, so the sum remains \$700. The integrity rule is preserved.
- **Isolation:** If another transaction T_2 tries to read A and B while T_1 is at step 4, it will not see the intermediate state ($A = 400, B = 200$). T_2 must wait until T_1 commits.
- **Durability:** If a power failure occurs immediately after step 7, the new balances (\$400 and \$300) are guaranteed to be recovered upon reboot.

3. Necessity of ACID for Concurrent Execution

In modern DBMS, multiple users access and modify data simultaneously to maximize throughput and resource utilization. ACID properties are absolutely essential for concurrent execution for the following reasons:

- **Prevention of Data Anomalies:** Without **Isolation**, concurrent transactions can lead to severe anomalies such as:
 - *Dirty Read Problem:* Reading uncommitted, temporary data from a transaction that eventually fails.
 - *Lost Update Problem:* Two transactions overwriting each other's changes, leading to permanent data loss.
 - *Unrepeatable Read:* Reading the same record twice within a transaction and getting different values.
- **Predictability & Correctness:** **Consistency** ensures that despite the complex interleaving of concurrent operations, the final database state remains mathematically and logically correct.
- **Fault Tolerance during Interleaving:** When transactions are highly interleaved, a failure in one transaction must not corrupt others. **Atomicity** ensures that an aborted transaction rolls back cleanly without leaving fragmented data that could affect other active transactions.

4. Summary Table

Property	Core Objective	DBMS Component Responsible
Atomicity	All-or-Nothing execution	Transaction & Recovery Manager
Consistency	Data validity & rules	Application & Integrity Subsystem
Isolation	Serializability & No overlap	Concurrency Control Manager
Durability	Permanent persistence	Recovery Manager (Logs)

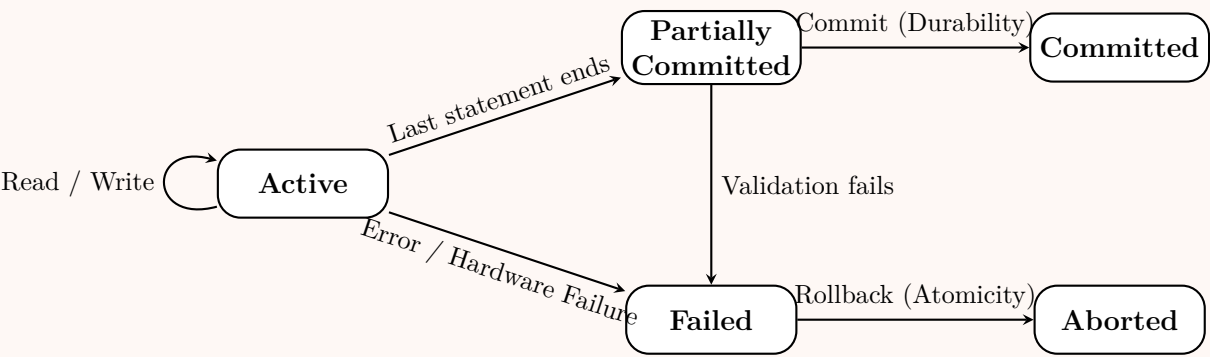


Figure: State Transition Diagram of a Transaction Enforcing ACID

Q5. What do you mean by the ACID properties of a DBMS? Give real-life examples (one for each property) showing the necessity of these properties. (8 marks) [CSE 303 | L-3/T-1 | 2013–14]

Answer

ACID Properties of a DBMS

In a Database Management System (DBMS), a **transaction** is a logical unit of work that consists of one or more database operations (e.g., read, write, update, delete). To ensure the reliability, integrity, and correctness of the database especially during system crashes, power failures, or concurrent execution every transaction must adhere to four foundational principles collectively known as the **ACID properties**. The acronym **ACID** stands for **A**tomicity, **C**onsistency, **I**solation, and **D**urability.

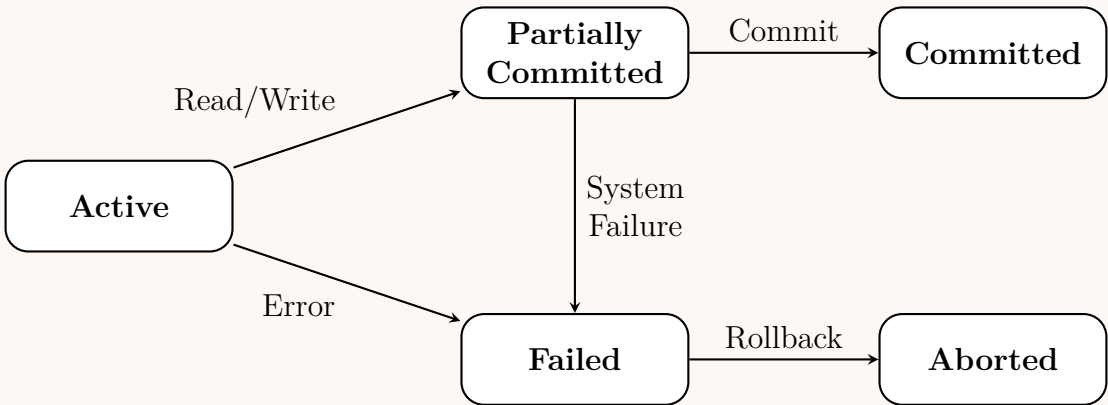


Figure: Standard Transaction State Diagram illustrating the lifecycle enforced by ACID properties.

1. Atomicity (The “All-or-Nothing” Rule)

- Definition:** Atomicity ensures that all operations within a transaction are treated as a single, indivisible unit. Either all operations are successfully executed and reflected in the database (committed), or none of them are (aborted/rolled back).
- Working Principle:** If a transaction fails mid-execution, the DBMS undoes all previous operations to restore the database to its state before the transaction began.
- Necessity (Real-life Example): Bank Fund Transfer**
Suppose Person A attempts to transfer \$500 to Person B. This involves two operations:
 - (a) Deduct \$500 from A’s account.
 - (b) Add \$500 to B’s account.

If the system crashes exactly after step 1, Person A has lost \$500, but Person B never received it. **Atomicity is necessary** to ensure that if step 2 fails, step 1 is automatically undone (rolled back), preventing the money from vanishing into thin air.

2. Consistency (Correctness Rule)

- **Definition:** Consistency ensures that a transaction takes the database from one valid (consistent) state to another. The transaction must respect all defined integrity constraints, cascades, and triggers.
- **Working Principle:** The total sum of data rules (e.g., primary keys, foreign keys, check constraints) must be strictly maintained before and after the transaction.
- **Necessity (Real-life Example): Bank Balance Constraint**
Consider the same fund transfer from A to B. A business rule dictates that *the sum of balances of Account A and Account B must remain constant* before and after the transaction, and *no account balance can be negative*. If Account A only has \$300, allowing a \$500 deduction would violate the non-negative constraint. **Consistency is necessary** to reject this transaction immediately, ensuring the database never records an illegal state (like a $-\$200$ balance).

3. Isolation (Independence Rule)

- **Definition:** Isolation ensures that the concurrent execution of multiple transactions leaves the database in the exact same state as if the transactions had been executed sequentially (one after the other).
- **Working Principle:** Intermediate, uncommitted states of one transaction must remain invisible to other concurrent transactions. This prevents concurrency anomalies such as dirty reads, non-repeatable reads, and phantom reads.
- **Necessity (Real-life Example): Airline Seat Reservation**
Imagine there is only **one seat left** on a flight. User X and User Y attempt to book this exact seat at the exact same millisecond. Without isolation, both users might query the system, see the seat as “available”, and proceed to payment, resulting in a double-booking. **Isolation is necessary** to lock the seat for User X’s transaction. User Y’s transaction will be forced to wait, subsequently seeing the seat as “booked” once User X completes the transaction.

4. Durability (Permanence Rule)

- **Definition:** Durability guarantees that once a transaction has been successfully completed (committed), its changes are permanently recorded in the non-volatile storage of the database and will survive any subsequent system failures (e.g., power loss, server crash).
- **Working Principle:** The DBMS writes all transaction logs to stable storage (disk) before acknowledging the commit to the user.
- **Necessity (Real-life Example): E-commerce Checkout**
A customer completes a checkout process on an e-commerce website and receives a “Payment Successful & Order Placed” confirmation on their screen. Exactly one second later, a catastrophic power failure hits the main database server. **Durability is necessary** to guarantee that when the server reboots, the order is still safely recorded in the database. Without it, the customer would be charged, but the merchant would have no record of what to ship.

Summary of DBMS Components Handling ACID	
Atomicity	Handled by the Transaction Management System (Log-based recovery).
Consistency	Handled by the Application Programmer and Integrity Subsystem .
Isolation	Handled by the Concurrency Control Manager (Locks, Timestamps).
Durability	Handled by the Recovery Management System (Write-Ahead Logging).

Q6. Which two ACID properties are maintained by the Recovery Manager of a DBMS? (5 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

ACID Properties Maintained by the Recovery Manager

The two ACID properties strictly maintained by the **Recovery Manager** (or Recovery Subsystem) of a Database Management System are **Atomicity** and **Durability**.
The Recovery Manager ensures that the database is restored to a consistent state after a transaction failure, system crash, or catastrophic failure, primarily relying on mechanisms like Write-Ahead Logging (WAL) and shadow paging.

1. Atomicity (Maintained via UNDO Operations)

- **Explanation:** Atomicity requires that a transaction is an indivisible unit of work (“all-or-nothing”). If a transaction is interrupted by a crash before it reaches its commit point, the database must not reflect any partial updates.

- **Working Principle:** The Recovery Manager scans the system log upon restart. If it finds a transaction that has a **<start>** record but no corresponding **<commit>** or **<abort>** record, it performs an **UNDO** operation. It rolls back all changes made by this incomplete transaction, effectively reversing it to satisfy the atomicity rule.

2. Durability (Maintained via REDO Operations)

- **Explanation:** Durability requires that once a transaction successfully commits, its changes must survive any subsequent system failures.
- **Working Principle:** When a system crashes, data that was modified in the volatile main memory (RAM) buffer but not yet flushed to the non-volatile disk is lost. During recovery, the Recovery Manager uses the log to find all committed transactions. It performs a **REDO** operation for these transactions, reapplying their changes to the database on disk to guarantee that the committed data is permanent.

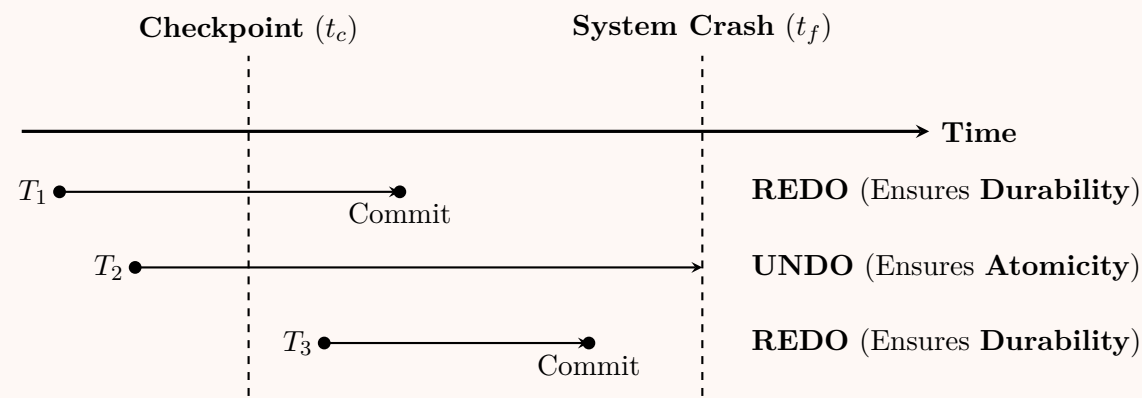


Figure: Role of the Recovery Manager during system restart. Committed transactions (T_1, T_3) are redone, while uncommitted transactions (T_2) are undone.

Summary of ACID Responsibilities

For context, the complete division of labor within the DBMS to maintain all four ACID properties is outlined below:

Property	Responsible Subsystem	Primary Mechanism
Atomicity	Recovery Manager	Log-based UNDO / Rollback
Consistency	Application / Integrity Subsystem	Constraints (Keys, Triggers)
Isolation	Concurrency Control Manager	Locking, Timestamps, MVCC
Durability	Recovery Manager	Write-Ahead Logging (WAL), REDO

Q7. What do you mean by the ACID properties in a DBMS? Give one example of each property of the ACID. (12 marks) [CSE 303 | L-3/T-1 | 2011–12]

Answer

ACID Properties in a DBMS

In a Database Management System (DBMS), a **transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. To maintain the integrity and reliability of the database during these transactionsespecially in the event of hardware failures, network crashes, or concurrent executionsa transaction must strictly adhere to four fundamental properties, collectively known by the acronym **ACID**.

These properties are **Atomicity**, **Consistency**, **Isolation**, and **Durability**.

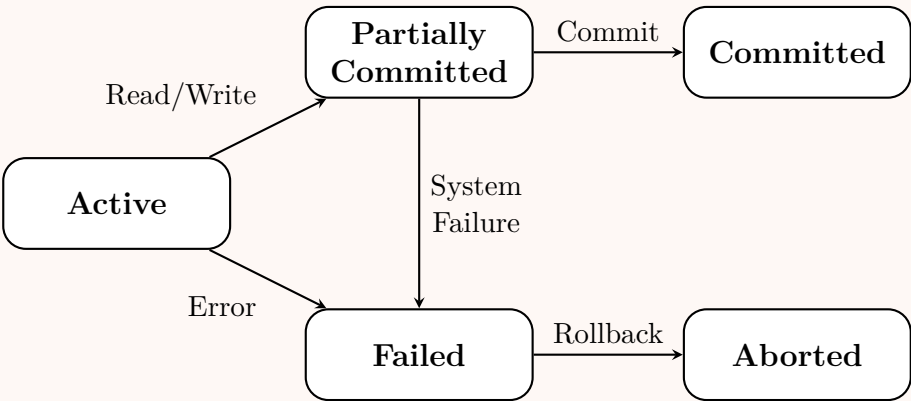


Figure: State transition diagram of a database transaction managed by ACID properties.

1. Atomicity (The “All-or-Nothing” Rule)

- **Definition:** Atomicity ensures that all operations within a transaction are treated as a single, indivisible unit. Either all operations execute successfully and are reflected in the database, or none of them are. There is no partial execution.
- **Working Principle:** If an error occurs mid-transaction, the DBMS automatically undoes (rolls back) all previous operations of that transaction, restoring the database to its initial state.
- **Example:** Consider transferring \$100 from Account A to Account B. This involves two operations: (1) Deduct \$100 from A, and (2) Add \$100 to B. If the system crashes after operation 1, atomicity ensures that the \$100 deduction from A is rolled back, preventing the money from disappearing.

2. Consistency

- **Definition:** Consistency dictates that a transaction must transform the database from one valid (consistent) state to another. It must not violate any defined integrity constraints (e.g., primary keys, foreign keys, check constraints).
- **Working Principle:** The DBMS checks constraints before and after the transaction. If the execution of the transaction results in a constraint violation, the transaction is immediately aborted.
- **Example:** Suppose a bank database has an integrity constraint stating that an account balance cannot fall below \$0. If a transaction attempts to withdraw \$200 from an account that only contains \$100, the consistency property guarantees that the DBMS will reject the transaction, keeping the balance at \$100.

3. Isolation

- **Definition:** Isolation ensures that the concurrent execution of multiple transactions results in a system state that would be exactly the same as if the transactions were executed serially (one after the other).
- **Working Principle:** The DBMS uses concurrency control protocols (such as two-phase locking or timestamps) to prevent intermediate, uncommitted states of one transaction from being visible to other transactions.
- **Example:** Two users, Alice and Bob, simultaneously attempt to book the last available seat on a flight. Isolation ensures that their transactions are processed sequentially. If Alice’s transaction locks the seat first, Bob’s transaction will wait and subsequently see that the seat is no longer available, preventing a double-booking anomaly.

4. Durability

- **Definition:** Durability guarantees that once a transaction successfully completes (commits), its changes are permanently recorded in the database and will survive any subsequent system or hardware failures.
- **Working Principle:** The DBMS writes the transaction log to non-volatile storage (disk) before it acknowledges the successful commit to the user. Upon system restart, it uses these logs to redo any committed changes that were not yet saved to the physical database file.
- **Example:** After completing an online purchase and receiving a “Payment Successful” message, a sudden city-wide power outage shuts down the database servers. Durability guarantees that when power is restored, the order data is intact and has not been lost.

Summary of ACID Management

Different subsystems within a DBMS are responsible for enforcing specific ACID properties:

Property	Responsible Subsystem	Primary Mechanism
Atomicity	Transaction & Recovery Manager	Undo Logs, Rollback mechanisms
Consistency	Application Programmer & Integrity Engine	Integrity constraints, Triggers
Isolation	Concurrency Control Manager	Locking (2PL), MVCC, Timestamping
Durability	Recovery Manager	Redo Logs, Write-Ahead Logging (WAL)

Q8. Explain the transaction states with the state diagram. (5 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

Transaction States in DBMS

A **transaction** is a logical unit of processing in a database management system (DBMS) that entails one or more database access operations (read, write, update, delete). During its execution, a transaction passes through several distinct states to ensure the

database's consistency and reliability, particularly in the event of a failure.

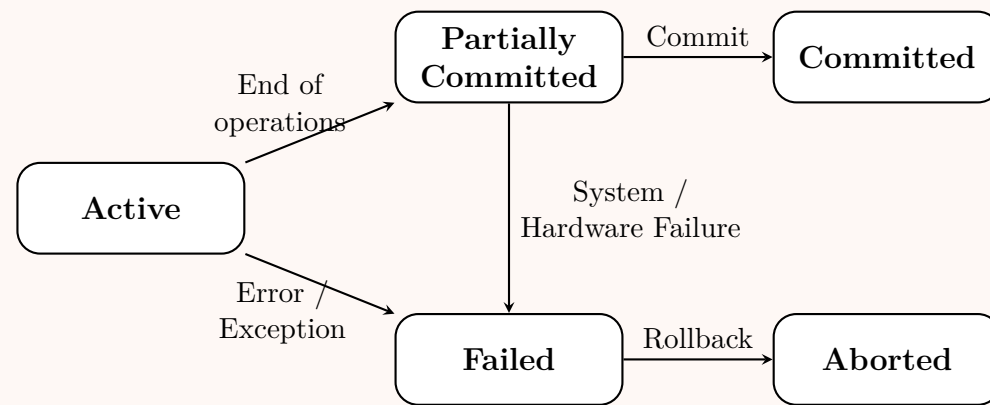


Figure: State Transition Diagram of a Database Transaction

Detailed Explanation of States

(a) Active State

- **Definition:** This is the initial state of every transaction.
- **Explanation:** A transaction stays in this state while its instructions (such as **Read**, **Write**, or **Update**) are being executed.
- **Transition:** If all operations execute without any interruption, it transitions to the **Partially Committed** state. If a crash or violation occurs, it transitions to the **Failed** state.

(b) Partially Committed State

- **Definition:** A transaction reaches this state after the final operation has been executed, but before the changes are permanently recorded.
- **Explanation:** At this point, the transaction's changes exist in the main memory (RAM or volatile buffer) but have not yet been written to the physical, non-volatile disk.
- **Transition:** If the changes are successfully written to disk, it transitions to the **Committed** state. If a system failure occurs before disk writing completes, it transitions to the **Failed** state.

(c) Failed State

- **Definition:** A transaction enters this state if it cannot proceed with its normal execution due to a failure.
- **Explanation:** Failures can be logical (e.g., violating integrity constraints, division by zero) or systemic (e.g., hardware crash, power failure, network drop).
- **Transition:** Once in the failed state, the DBMS must safely undo any partial changes made by the transaction. It transitions to the **Aborted** state after the rollback is complete.

(d) Aborted State

- **Definition:** A transaction reaches this state after it has been rolled back and the database is safely restored to its state prior to the transaction's start.
- **Explanation:** This state enforces the **Atomicity** (All-or-Nothing) property. Once aborted, the DBMS typically takes one of two actions:
 - **Restart** the transaction (if the failure was non-deterministic, like a temporary network timeout or a deadlock).
 - **Kill** the transaction (if the failure was due to an internal logical error, like a syntax error).

(e) Committed State

- **Definition:** The final, successful state of a transaction.
- **Explanation:** When a transaction reaches this state, all its updates have been permanently and safely stored in the database's non-volatile storage.
- **Property Enforcement:** Reaching this state guarantees the **Durability** property. A transaction in the committed state cannot be rolled back or undone.

Concurrency Control

Q1. Define the Strict 2PL (two-phase locking) protocol. Show that a 2PL schedule is conflict-serializable. (9 marks)

[CSE 215 |

L-2/T-1 | 2024–25]

Answer

1. Strict Two-Phase Locking (Strict 2PL) Protocol

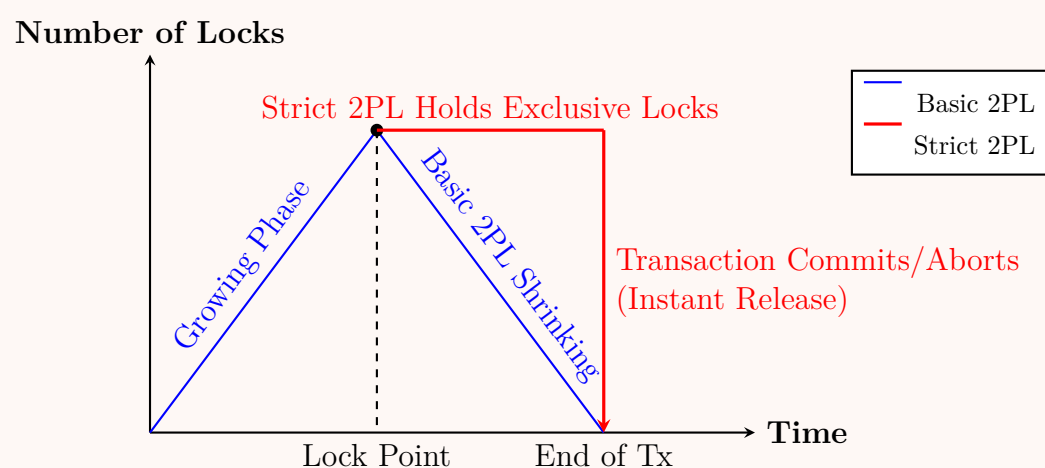
The **Two-Phase Locking (2PL)** protocol ensures serializability by requiring that every transaction acquires and releases locks in two distinct phases:

- (a) **Growing Phase:** A transaction may obtain locks but may not release any lock.
- (b) **Shrinking Phase:** A transaction may release locks but may not obtain any new locks.

Strict 2PL is a specialized, more restrictive version of basic 2PL designed to prevent **cascading rollbacks** and ensure **strict recoverability**.

Rules of Strict 2PL

- A transaction must follow standard 2PL rules (Growing and Shrinking phases).
- A transaction must hold all its **exclusive (write) locks** until it reaches the **commit** or **abort** state.
- Shared (read) locks can be released during the shrinking phase as usual (unlike *Rigorous 2PL*, which holds all locks until the end).



Properties of Strict 2PL

- **Advantages:** Guarantees conflict serializability, prevents cascading aborts, and recovers easily from transaction failures.
- **Disadvantages:** Decreases concurrency (because locks are held longer) and is still susceptible to deadlocks.

2. Proof: A 2PL Schedule is Conflict-Serializable

To prove that any schedule adhering to the 2PL protocol is conflict-serializable, we use a **proof by contradiction** based on the **Precedence Graph** (Serialization Graph).

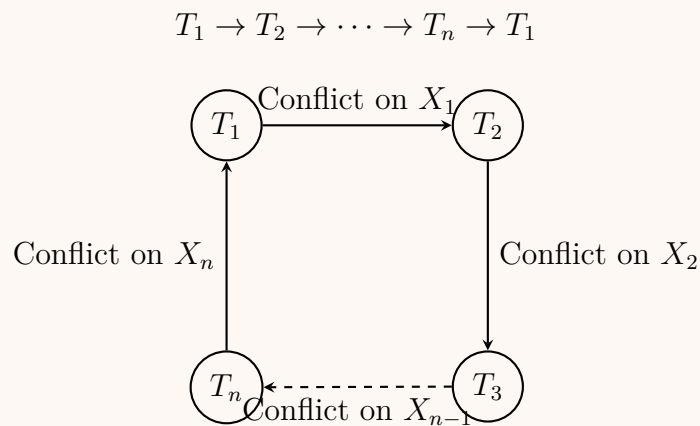
Definitions and Setup

- Let S be a schedule containing transactions $T_1, T_2, \dots, T_n$.
- A schedule is **conflict-serializable** if and only if its precedence graph contains **no cycles**.
- In 2PL, let $L(T_i)$ denote the time when T_i acquires its *last* lock (end of growing phase / lock point).
- Let $U(T_i)$ denote the time when T_i releases its *first* lock (start of shrinking phase).
- By the definition of 2PL, a transaction cannot acquire a lock after it has released one. Therefore, for any transaction T_i :

$$L(T_i) \leq U(T_i) \quad \text{— (Equation 1)}$$

The Contradiction

Assume there exists a schedule S that follows 2PL but is **not conflict-serializable**. This means its precedence graph contains a cycle of conflicting transactions:



- (a) An edge $T_i \rightarrow T_j$ means T_i and T_j conflict on some data item X , and T_i accessed X before T_j .
- (b) Under 2PL, for T_j to lock and access X , T_i must have first unlocked X .
- (c) Therefore, T_i must release at least one lock (start shrinking) before T_j acquires all its locks (finishes growing). Mathematically:

$$U(T_i) < L(T_j) \quad \text{— (Equation 2)}$$

Applying Equation 2 to the cycle in our precedence graph:

- For $T_1 \rightarrow T_2$: $U(T_1) < L(T_2)$
- For $T_2 \rightarrow T_3$: $U(T_2) < L(T_3)$
- ...
- For $T_n \rightarrow T_1$: $U(T_n) < L(T_1)$

Combining these inequalities with the 2PL property (Equation 1, $L(T_i) \leq U(T_i)$), we get a strict chain of inequalities:

$$L(T_1) \leq U(T_1) < L(T_2) \leq U(T_2) < \dots < L(T_n) \leq U(T_n) < L(T_1)$$

Transitively collapsing this chain yields:

$$L(T_1) < L(T_1)$$

Conclusion

The statement $L(T_1) < L(T_1)$ is a mathematical impossibility (a time cannot be strictly less than itself). This contradiction proves that our initial assumption that a 2PL schedule can have a cycle is false.

Therefore, **every schedule generated by the Two-Phase Locking protocol produces a cycle-free precedence graph and is guaranteed to be conflict-serializable.**

Q2. Determine whether the following schedules are conflict-serializable by constructing precedence graphs:

- (a) $r_1(A); w_2(A); w_3(A); w_2(B); r_2(C); r_2(B); w_2(C)$
- (b) $w_1(A); r_2(A); w_2(B); r_2(B); w_3(C); r_4(C); w_4(D); r_3(D); r_3(C)$

(8 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Conflict Serializability Analysis using Precedence Graphs

A schedule is **conflict-serializable** if and only if its corresponding precedence graph (serialization graph) is **acyclic** (contains no cycles). A precedence graph is constructed as follows:

- **Nodes:** Each transaction T_i participating in the schedule.
- **Edges:** A directed edge $T_i \xrightarrow{X} T_j$ exists if there is a conflict between an operation of T_i and an operation of T_j on data item X , and the operation of T_i executes before T_j 's operation. (A conflict occurs if at least one of the operations is a Write).

Schedule 1: $S_1 = r_1(A); w_2(A); w_3(A); w_2(B); r_2(C); r_2(B); w_2(C)$

1. **Identify Participating Transactions:** T_1 , T_2 , and T_3 .
2. **Identify Conflicts:**

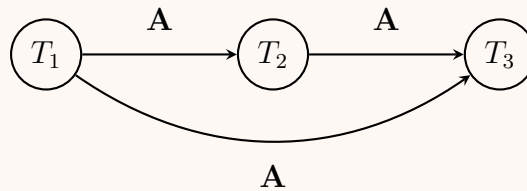
- **On Data Item A:**

- $r_1(A)$ is followed by $w_2(A) \implies$ Conflict Edge: $\mathbf{T}_1 \rightarrow \mathbf{T}_2$
- $r_1(A)$ is followed by $w_3(A) \implies$ Conflict Edge: $\mathbf{T}_1 \rightarrow \mathbf{T}_3$
- $w_2(A)$ is followed by $w_3(A) \implies$ Conflict Edge: $\mathbf{T}_2 \rightarrow \mathbf{T}_3$

- **On Data Items B and C:**

- Only transaction T_2 performs operations on B ($w_2(B), r_2(B)$) and C ($r_2(C), w_2(C)$). There are no inter-transaction conflicts for these data items.

3. Precedence Graph for S_1 :



Conclusion for S_1 : The precedence graph for S_1 is a Directed Acyclic Graph (DAG) as there are **no cycles**. Therefore, **Schedule 1 is conflict-serializable**. It is equivalent to the serial schedule $T_1 \rightarrow T_2 \rightarrow T_3$.

Schedule 2: $S_2 = w_1(A); r_2(A); w_2(B); r_2(B); w_3(C); r_4(C); w_4(D); r_3(D); r_3(C)$

1. **Identify Participating Transactions:** T_1, T_2, T_3 , and T_4 .

2. **Identify Conflicts:**

- **On Data Item A:**

- $w_1(A)$ is followed by $r_2(A) \implies$ Conflict Edge: $\mathbf{T}_1 \rightarrow \mathbf{T}_2$

- **On Data Item B:**

- Only T_2 operates on B . No conflicts.

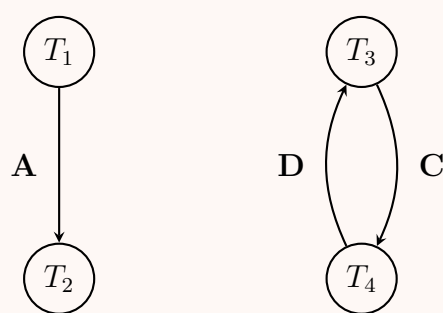
- **On Data Item C:**

- $w_3(C)$ is followed by $r_4(C) \implies$ Conflict Edge: $\mathbf{T}_3 \rightarrow \mathbf{T}_4$
- (Note: $w_3(C)$ and $r_3(C)$ belong to the same transaction, and $r_4(C)$ followed by $r_3(C)$ is a Read-Read sequence, which is not a conflict.)

- **On Data Item D:**

- $w_4(D)$ is followed by $r_3(D) \implies$ Conflict Edge: $\mathbf{T}_4 \rightarrow \mathbf{T}_3$

3. Precedence Graph for S_2 :



Conclusion for S_2 : The precedence graph for S_2 contains a **cycle** ($T_3 \rightarrow T_4 \rightarrow T_3$). Because of this cycle, there is no way to topologically sort the transactions. Therefore, **Schedule 2 is NOT conflict-serializable**.

Q3. Consider the two transactions given below:

$T_1: r_1(A); w_1(B)$
 $T_2: r_2(A); w_2(A); w_2(B)$

Determine the number of possible schedules, number of serializable schedules, and number of conflict-serializable schedules that can be constructed for the given transactions. **(9 marks)** [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Total Number of Possible Schedules

To find the total number of concurrent schedules, we must find all possible interleavings of the operations of the given transactions while maintaining the internal order of operations for each transaction.

Given:

- Transaction T_1 has $n_1 = 2$ operations: $r_1(A), w_1(B)$.
- Transaction T_2 has $n_2 = 3$ operations: $r_2(A), w_2(A), w_2(B)$.

The total number of possible schedules for N transactions is calculated using the formula for permutations of multisets:

$$\text{Total Schedules} = \frac{(n_1 + n_2 + \dots + n_N)!}{n_1! \cdot n_2! \cdot \dots \cdot n_N!}$$

Substitute the values for T_1 and T_2 :

$$\text{Total Schedules} = \frac{(2+3)!}{2! \cdot 3!} = \frac{5!}{2! \cdot 3!} = \frac{120}{2 \cdot 6} = 10$$

Result: There are **10** possible schedules.

2. Number of Conflict-Serializable Schedules

To determine conflict serializability, we must identify the **conflicting operations** between T_1 and T_2 . Two operations conflict if they belong to different transactions, access the same data item, and at least one is a Write operation.

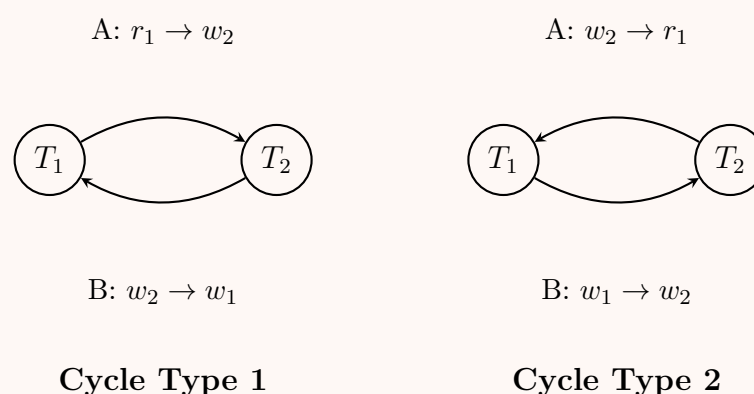
Identified Conflicts:

- (a) **On Data Item A:** $r_1(A)$ and $w_2(A)$
- (b) **On Data Item B:** $w_1(B)$ and $w_2(B)$

Note: $r_1(A)$ and $r_2(A)$ do not conflict (Read-Read).

A schedule is **conflict-serializable** if its precedence graph contains no cycles. A cycle occurs if $T_1 \rightarrow T_2$ on one data item and $T_2 \rightarrow T_1$ on the other.

Let us analyze the two cyclical cases using Precedence Graphs:



By evaluating all 10 possible interleavings against these cycle conditions:

#	Schedule Sequence	Edges Generated	CSR?
1	$r_1(A), w_1(B), r_2(A), w_2(A), w_2(B)$	$T_1 \rightarrow T_2$	Yes
2	$r_1(A), r_2(A), w_1(B), w_2(A), w_2(B)$	$T_1 \rightarrow T_2$	Yes
3	$r_1(A), r_2(A), w_2(A), w_1(B), w_2(B)$	$T_1 \rightarrow T_2$	Yes
4	$r_1(A), r_2(A), w_2(A), w_2(B), w_1(B)$	$T_1 \rightarrow T_2$ (A), $T_2 \rightarrow T_1$ (B)	No (Cycle)
5	$r_2(A), r_1(A), w_1(B), w_2(A), w_2(B)$	$T_1 \rightarrow T_2$	Yes
6	$r_2(A), r_1(A), w_2(A), w_1(B), w_2(B)$	$T_1 \rightarrow T_2$	Yes
7	$r_2(A), r_1(A), w_2(A), w_2(B), w_1(B)$	$T_1 \rightarrow T_2$ (A), $T_2 \rightarrow T_1$ (B)	No (Cycle)
8	$r_2(A), w_2(A), r_1(A), w_1(B), w_2(B)$	$T_2 \rightarrow T_1$ (A), $T_1 \rightarrow T_2$ (B)	No (Cycle)
9	$r_2(A), w_2(A), r_1(A), w_2(B), w_1(B)$	$T_2 \rightarrow T_1$	Yes
10	$r_2(A), w_2(A), w_2(B), r_1(A), w_1(B)$	$T_2 \rightarrow T_1$	Yes

Result: There are $10 - 3 = 7$ conflict-serializable schedules.

3. Number of Serializable (View-Serializable) Schedules

A schedule is considered serializable if it is **View-Serializable (VSR)**. Every Conflict-Serializable schedule is inherently View-Serializable. However, schedules with cycles *might* be VSR if they contain "blind writes."

Let us test the 3 non-CSR schedules for View Equivalence against the serial schedules ($T_1 \rightarrow T_2$ or $T_2 \rightarrow T_1$):

- **Schedules 4 & 7:** Both have T_1 reading the initial value of A , and T_2 performing the final write on A and T_1 performing the final write on B . No serial schedule produces this exact view (Serial $T_1 \rightarrow T_2$ leaves T_2 's write on B as final; Serial $T_2 \rightarrow T_1$ has T_1 reading A from T_2 instead of the initial value).

- **Schedule 8:** T_1 reads A modified by T_2 , requiring the equivalent serial order to be $T_2 \rightarrow T_1$. However, the final write on B is done by T_2 , which contradicts the serial order $T_2 \rightarrow T_1$ (where T_1 would perform the final write on B).

Since none of the non-CSR schedules satisfy the conditions for View Serializability, the number of Serializable schedules is equal to the number of Conflict-Serializable schedules.

Result: There are **7** serializable schedules.

Summary Statement

- **Number of Possible Schedules:** 10
- **Number of Conflict-Serializable Schedules:** 7
- **Number of Serializable Schedules:** 7

Q4. Consider the two transactions given below:

$$\begin{aligned} T_1: & \text{ } sl_1(A); xl_1(B); u_1(A); u_1(B) \\ T_2: & \text{ } sl_2(B); xl_2(A); u_2(B); u_2(A) \end{aligned}$$

- (a) Show a schedule that results in a deadlock but is conflict-serializable to (T_2, T_1) .
- (b) Does introducing update locks solve the deadlock problem? Justify your answer.
- (c) Show the lock compatibility matrix for shared, exclusive, and update locks, indicating in each cell whether concurrent locks are allowed (“Yes”) or not allowed (“No”).

(10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Schedule Resulting in a Deadlock

To create a schedule that ends in a deadlock but remains trivially conflict-serializable to the serial order (T_2, T_1) , we sequence the operations such that neither transaction performs a conflicting write operation before the deadlock occurs.

Time	Transaction T_1	Transaction T_2	System State / Lock Table
t_1		$sl_2(B)$	T_2 holds Shared lock on B .
t_2	$sl_1(A)$		T_1 holds Shared lock on A .
t_3		$xl_2(A)$	T_2 waits (blocked by T_1 's lock on A).
t_4	$xl_1(B)$		T_1 waits (blocked by T_2 's lock on B).

Explanation of the Deadlock and Serializability:

- **Deadlock:** At time t_4 , a circular dependency arises. T_1 is waiting for T_2 to release the lock on B , while T_2 is waiting for T_1 to release the lock on A . Neither can proceed.
- **Conflict-Serializability:** Because both transactions are blocked while attempting to acquire their exclusive (write) locks, neither transaction has successfully executed a write operation. Since conflicts require at least one write operation, the precedence graph for this partial schedule contains **zero conflict edges**. An empty precedence graph is acyclic and therefore trivially conflict-serializable to any serial order, including (T_2, T_1) .

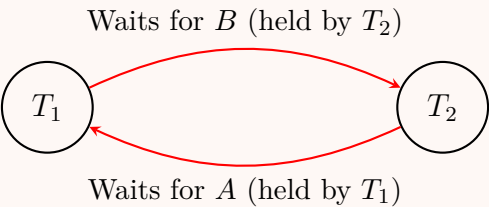


Figure: Wait-For Graph illustrating the deadlock.

2. Does Introducing Update Locks Solve the Deadlock?

Answer: No.

Justification: Update locks (**U-locks**) are explicitly designed to prevent a specific type of deadlock known as an **upgrade deadlock**. An upgrade deadlock occurs when two transactions hold a Shared (S) lock on the *same* data item, and both concurrently attempt to upgrade their lock to an Exclusive (X) lock. Because neither can upgrade until the other releases its S-lock, they deadlock. In the given scenario, the transactions are requesting locks on *different* resources:

- T_1 holds a lock on A and requests a lock on B .

- T_2 holds a lock on B and requests a lock on A .

This is a standard **circular wait across disjoint resources**. Even if we replace the initial sl requests with update locks (ul), T_1 would hold $U(A)$ and T_2 would hold $U(B)$. When T_1 requests $X(B)$, it is blocked by T_2 's $U(B)$. When T_2 requests $X(A)$, it is blocked by T_1 's $U(A)$. The deadlock remains entirely unaffected. Preventing this type of deadlock requires techniques like strict lock ordering (e.g., all transactions must lock A before B) or deadlock detection.

3. Lock Compatibility Matrix

The compatibility matrix dictates whether a newly requested lock can be granted concurrently based on the lock currently held by another transaction on the same data item.

Requested Mode ↓ \ Current Mode →	Shared (S)	Update (U)	Exclusive (X)
Shared (S)	Yes	Yes	No
Update (U)	Yes	No	No
Exclusive (X)	No	No	No

Matrix Explanation:

- **S is compatible with S:** Multiple transactions can read the same item simultaneously.
- **S is compatible with U:** A transaction can read an item even if another transaction has stated an *intent* to update it (holds a U-lock).
- **U is compatible with S:** A transaction can state its intent to update (U-lock) while others are currently reading (S-locks). However, it cannot upgrade to an X-lock until those S-locks are released.
- **U is NOT compatible with U:** To prevent upgrade deadlocks, only one transaction is permitted to hold a U-lock on a specific data item at any given time.
- **X is mutually exclusive:** Exclusive locks are incompatible with all other locks to ensure data integrity during writes.

Q5. Determine whether each of the following schedules is recoverable or ACR (avoids cascading rollback). Here c_i represents the commit action of transaction T_i .

- (a) $S_1: r_1(A); w_1(A); r_2(A); w_2(A); c_2; r_1(B); w_1(B); c_1$
- (b) $S_2: r_1(A); w_1(A); r_2(B); w_2(B); c_2; r_1(B); w_1(B); c_1$
- (c) $S_3: r_1(A); w_1(A); r_2(B); w_2(B); r_1(B); c_2; w_1(B); c_1$

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

Definitions and Rules for Recoverability

Before analyzing the schedules, we must define the conditions for recoverability and cascading rollbacks based on **dirty reads** (reading uncommitted data). A transaction T_j reads from T_i if T_j reads a data item X after T_i has written to it, and T_i has not yet committed or aborted.

- **Recoverable Schedule (RC):** A schedule is recoverable if, for every transaction T_j that reads a value written by T_i , T_i commits *before* T_j commits ($c_i < c_j$).
- **Avoids Cascading Rollback (ACR):** A schedule avoids cascading rollbacks (is cascadeless) if every transaction T_j only reads values that have *already been committed*. That is, if T_j reads from T_i , the commit of T_i must precede the read operation of T_j ($c_i < r_j(X)$).

Note: Every ACR schedule is automatically a recoverable schedule, but the reverse is not true.

Analysis of Schedule 1

$$S_1: r_1(A); w_1(A); r_2(A); w_2(A); c_2; r_1(B); w_1(B); c_1$$

1. Identify Read-Write Dependencies:

- $r_2(A)$ reads the value of data item A produced by $w_1(A)$. Therefore, T_2 **reads from** T_1 .

2. Check Recoverability:

- Since T_2 reads from T_1 , for the schedule to be recoverable, T_1 must commit before T_2 ($c_1 < c_2$).
- In S_1 , T_2 commits (c_2) before T_1 commits (c_1).

- **Result:** S_1 is **Not Recoverable**.

3. Check ACR:

- A schedule that is not recoverable cannot be ACR. Furthermore, T_2 reads A before T_1 commits.
- **Result:** S_1 is **Not ACR**.

Analysis of Schedule 2

$S_2: r_1(A); w_1(A); r_2(B); w_2(B); c_2; r_1(B); w_1(B); c_1$

1. Identify Read-Write Dependencies:

- $r_1(B)$ reads the value of data item B produced by $w_2(B)$. Therefore, T_1 **reads from** T_2 .

2. Check Recoverability:

- Since T_1 reads from T_2 , T_2 must commit before T_1 ($c_2 < c_1$).
- In S_2 , c_2 appears before c_1 .
- **Result:** S_2 is **Recoverable**.

3. Check ACR:

- We must verify if T_1 reads B *after* T_2 commits.
- In S_2 , the sequence is $w_2(B); \mathbf{c_2}; \mathbf{r_1(B)}$. T_1 reads a strictly committed value.
- **Result:** S_2 is **ACR**.

Analysis of Schedule 3

$S_3: r_1(A); w_1(A); r_2(B); w_2(B); r_1(B); c_2; w_1(B); c_1$

1. Identify Read-Write Dependencies:

- $r_1(B)$ reads the value of data item B produced by $w_2(B)$. Therefore, T_1 **reads from** T_2 .

2. Check Recoverability:

- Since T_1 reads from T_2 , T_2 must commit before T_1 ($c_2 < c_1$).
- In S_3 , c_2 appears before c_1 .
- **Result:** S_3 is **Recoverable**.

3. Check ACR:

- We must verify if T_1 reads B *after* T_2 commits.
- In S_3 , the sequence is $w_2(B); \mathbf{r_1(B)}; \mathbf{c_2}$. T_1 reads an uncommitted (dirty) value of B . If T_2 were to fail before c_2 , T_1 would have to be cascaded (rolled back).
- **Result:** S_3 is **Not ACR**.

Summary Table

Schedule	Dependency	Recoverable (RC)?	Avoids Cascading Rollback (ACR)?
S_1	T_2 reads from T_1	No ($c_2 < c_1$)	No ($r_2(A) < c_1$)
S_2	T_1 reads from T_2	Yes ($c_2 < c_1$)	Yes ($c_2 < r_1(B)$)
S_3	T_1 reads from T_2	Yes ($c_2 < c_1$)	No ($r_1(B) < c_2$)

- Q6.** Determine whether each of the following schedules is serializable or conflict-serializable, or both. If a schedule is serializable or conflict-serializable, specify the equivalent serial order. Construct the precedence graph to justify your answer.
- (a) $w_1(X); w_2(Y); w_1(Y); w_3(X); w_2(X)$
- (b) $w_1(X); w_2(Y); w_1(Y); w_2(X); w_3(X)$

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer**Analysis of Schedule 1**

$$S_1 : w_1(X); w_2(Y); w_1(Y); w_3(X); w_2(X)$$

1. Conflict Serializability (CSR) Check:

We construct the precedence graph by identifying all conflicting operations (since there are no reads, we only look at Write-Write conflicts).

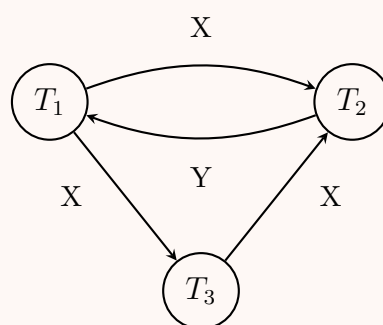
- **Conflicts on X:**

- $w_1(X)$ precedes $w_3(X) \implies$ Edge $T_1 \rightarrow T_3$
- $w_1(X)$ precedes $w_2(X) \implies$ Edge $T_1 \rightarrow T_2$
- $w_3(X)$ precedes $w_2(X) \implies$ Edge $T_3 \rightarrow T_2$

- **Conflicts on Y:**

- $w_2(Y)$ precedes $w_1(Y) \implies$ Edge $T_2 \rightarrow T_1$

Precedence Graph for S_1 :



Conclusion for CSR: The graph contains a cycle ($T_1 \rightarrow T_2 \rightarrow T_1$). Therefore, S_1 is **not conflict-serializable**.

2. View Serializability (VSR) Check:

Because S_1 contains “blind writes” (writes without prior reads), it might still be view-serializable. We verify this by checking final writes (since there are no reads, view equivalence depends entirely on final writes).

- Final write on X is performed by T_2 .
- Final write on Y is performed by T_1 .

For a serial schedule to be equivalent to S_1 , T_2 must be the last transaction to execute (to satisfy the final write on X), AND T_1 must be the last transaction to execute (to satisfy the final write on Y). This is a contradiction.

Conclusion for VSR: S_1 is **not view-serializable** (and therefore, not serializable at all).

Analysis of Schedule 2

$$S_2 : w_1(X); w_2(Y); w_1(Y); w_2(X); w_3(X)$$

1. Conflict Serializability (CSR) Check:

Identifying Write-Write conflicts for S_2 :

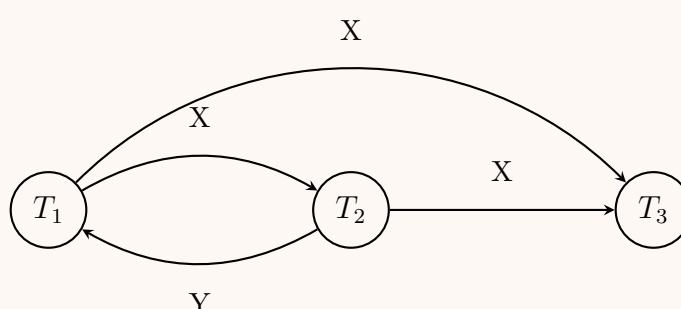
- **Conflicts on X:**

- $w_1(X)$ precedes $w_2(X) \implies$ Edge $T_1 \rightarrow T_2$
- $w_1(X)$ precedes $w_3(X) \implies$ Edge $T_1 \rightarrow T_3$
- $w_2(X)$ precedes $w_3(X) \implies$ Edge $T_2 \rightarrow T_3$

- **Conflicts on Y:**

- $w_2(Y)$ precedes $w_1(Y) \implies$ Edge $T_2 \rightarrow T_1$

Precedence Graph for S_2 :



Conclusion for CSR: The graph contains a cycle ($T_1 \rightarrow T_2 \rightarrow T_1$). Therefore, S_2 is **not conflict-serializable**.

2. View Serializability (VSR) Check:
Check final writes to test for view-serializability (again, no reads exist):

- Final write on X is performed by T_3 .
- Final write on Y is performed by T_1 .

We look for a serial schedule where T_3 writes X last, and T_1 writes Y last.

- Consider the serial schedule $T_2 \rightarrow T_1 \rightarrow T_3$.
- Operations order: $w_2(Y), w_2(X), w_1(X), w_1(Y), w_3(X)$.
- In this serial schedule, T_3 is the last to write X (overwriting T_1 's and T_2 's blind writes). T_1 is the last to write Y .
- Since S_2 and $(T_2 \rightarrow T_1 \rightarrow T_3)$ have identical initial reads (none), Read-Write linkages (none), and final writes, they are view equivalent.

Conclusion for VSR: S_2 is **view-serializable** (and therefore serializable). The equivalent serial order is $T_2 \rightarrow T_1 \rightarrow T_3$.

Summary Table

Schedule	Conflict-Serializable?	Serializable?	Equivalent Serial Order
S_1	No (Cycle: $T_1 \leftrightarrow T_2$)	No	N/A
S_2	No (Cycle: $T_1 \leftrightarrow T_2$)	Yes	$T_2 \rightarrow T_1 \rightarrow T_3$

Q7. Show that a “strict” two-phase locking (2PL) schedule avoids cascading rollback. **(5 marks)** [CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Core Definitions

To prove that a Strict Two-Phase Locking (Strict 2PL) schedule avoids cascading rollback, we must first establish the definitions of the two concepts:

- Cascading Rollback Avoidance (ACR):** A schedule avoids cascading rollback (is cascadeless) if every transaction T_j reads a data item X only *after* the transaction T_i that last wrote X has successfully committed. Mathematically, if T_j reads from T_i , then $c_i < r_j(X)$.
- Strict Two-Phase Locking (Strict 2PL):** A concurrency control protocol that enforces the rules of basic 2PL (a growing phase for acquiring locks and a shrinking phase for releasing locks) with an additional strict rule: **A transaction must hold all its exclusive (write) locks until it commits or aborts.**

2. Logical Proof

The proof that Strict 2PL guarantees ACR follows directly from the lock compatibility rules and the delayed release of exclusive locks.

- (a) **Assumption of a Write Operation:** Suppose a transaction T_i writes to a data item X . To do this under Strict 2PL, T_i must first acquire an exclusive lock on X , denoted as $xl_i(X)$.
- (b) **Strict Lock Retention:** According to the rules of Strict 2PL, T_i cannot release $xl_i(X)$ until T_i reaches its commit point (c_i) or aborts. Therefore, the lock is held continuously from the time of the write operation until the transaction terminates.
- (c) **Attempted Read by Another Transaction:** Suppose another transaction T_j wishes to read X after T_i has written to it. To read X , T_j must request at least a shared lock on X , denoted as $sl_j(X)$.
- (d) **Lock Incompatibility:** An exclusive lock (xl) and a shared lock (sl) are mutually incompatible. Because T_i currently holds $xl_i(X)$, the system’s lock manager will deny T_j ’s request for $sl_j(X)$.
- (e) **Forced Waiting:** T_j is forced to enter a blocked (waiting) state. T_j cannot proceed to read X while T_i holds the exclusive lock.
- (f) **Commit and Release:** T_i eventually commits (c_i) and subsequently releases its exclusive locks, including $xl_i(X)$.
- (g) **Safe Read:** Once the lock is released by T_i , T_j ’s request for $sl_j(X)$ is granted. T_j can now execute $r_j(X)$.

Conclusion: Because T_j was blocked until T_i committed, the read operation $r_j(X)$ strictly occurs after the commit c_i . Since a transaction can only ever read data from committed transactions, the schedule **strictly avoids cascading rollback**.

3. Visual Representation: Strict 2PL Blocking Mechanism

The following sequence diagram illustrates how Strict 2PL physically prevents a transaction from reading uncommitted data, thereby avoiding cascading aborts.

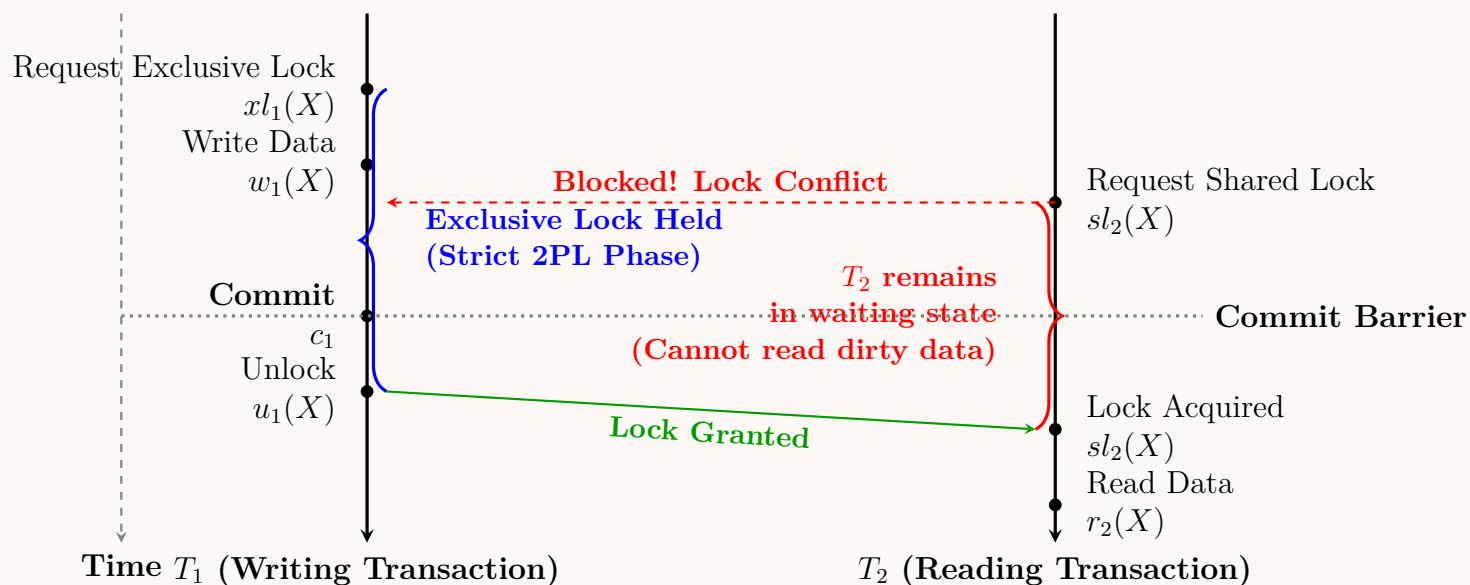


Diagram Analysis: As seen in the figure, T_2 is physically prevented from executing $r_2(X)$ until the **Commit Barrier** is crossed by T_1 . Therefore, it is impossible for T_2 to read uncommitted data, proving that Strict 2PL inherently prevents cascading rollbacks.

Q8. Show that if a schedule follows the two-phase locking (2PL) protocol, then it is conflict-serializable. (10 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Core Concepts and Definitions

To rigorously prove that the Two-Phase Locking (2PL) protocol ensures conflict serializability, we first define the fundamental principles involved:

- **Two-Phase Locking (2PL) Protocol:** A concurrency control mechanism where every transaction strictly follows two phases:
 - (a) **Growing Phase:** The transaction can acquire locks but cannot release any locks.
 - (b) **Shrinking Phase:** The transaction can release locks but cannot acquire any new locks.
- **Lock Point (LP):** The exact moment a transaction acquires its final lock (the transition point between the growing and shrinking phases).
- **Conflict Serializability (CSR):** A schedule is conflict-serializable if and only if its precedence graph (serialization graph) is **acyclic** (contains no directed cycles).

2. The Fundamental Lemma of 2PL

Before the main proof, we establish a critical relationship between conflicting transactions and their lock points.

Lemma: If there is an edge $T_i \rightarrow T_j$ in the precedence graph, then the lock point of T_i must occur before the lock point of T_j . Mathematically, $LP(T_i) < LP(T_j)$.

Proof of Lemma:

- (a) An edge $T_i \rightarrow T_j$ implies that T_i and T_j have conflicting operations on some data item X , and T_i accesses X before T_j .
- (b) For a conflict to occur, at least one of the operations must be a Write, meaning exclusive access is required.
- (c) Therefore, T_i must hold a lock on X and subsequently **release** it before T_j can **acquire** its lock on X .
- (d) Let $U_i(X)$ be the time T_i unlocks X , and $L_j(X)$ be the time T_j locks X . The chronology is: $U_i(X) < L_j(X)$.
- (e) Because T_i releases a lock at $U_i(X)$, it must have already reached its lock point (it is in its shrinking phase). Thus, $LP(T_i) \leq U_i(X)$.
- (f) Because T_j acquires a lock at $L_j(X)$, its lock point cannot occur until at least this moment (it is in its growing phase). Thus, $L_j(X) \leq LP(T_j)$.
- (g) Stringing these inequalities together yields:

$$LP(T_i) \leq U_i(X) < L_j(X) \leq LP(T_j) \implies LP(T_i) < LP(T_j)$$

3. Proof by Contradiction

We will prove that a 2PL schedule is always conflict-serializable using a proof by contradiction.

Assumption: Assume there exists a schedule S that strictly follows the 2PL protocol but is **not** conflict-serializable.

- (a) By the definition of conflict serializability, if S is not CSR, its precedence graph must contain at least one **cycle**.
 (b) Let this cycle involve a sequence of n transactions:

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots \rightarrow T_n \rightarrow T_1$$

- (c) Applying our **Fundamental Lemma** to every directed edge in this cycle, we get the following chain of inequalities based on their lock points in time:

$$T_1 \rightarrow T_2 \implies LP(T_1) < LP(T_2)$$

$$T_2 \rightarrow T_3 \implies LP(T_2) < LP(T_3)$$

$$\vdots$$

$$T_n \rightarrow T_1 \implies LP(T_n) < LP(T_1)$$

- (d) By the transitive property of inequality, we can combine these strict inequalities:

$$LP(T_1) < LP(T_2) < LP(T_3) < \dots < LP(T_n) < LP(T_1)$$

- (e) This logically evaluates to:

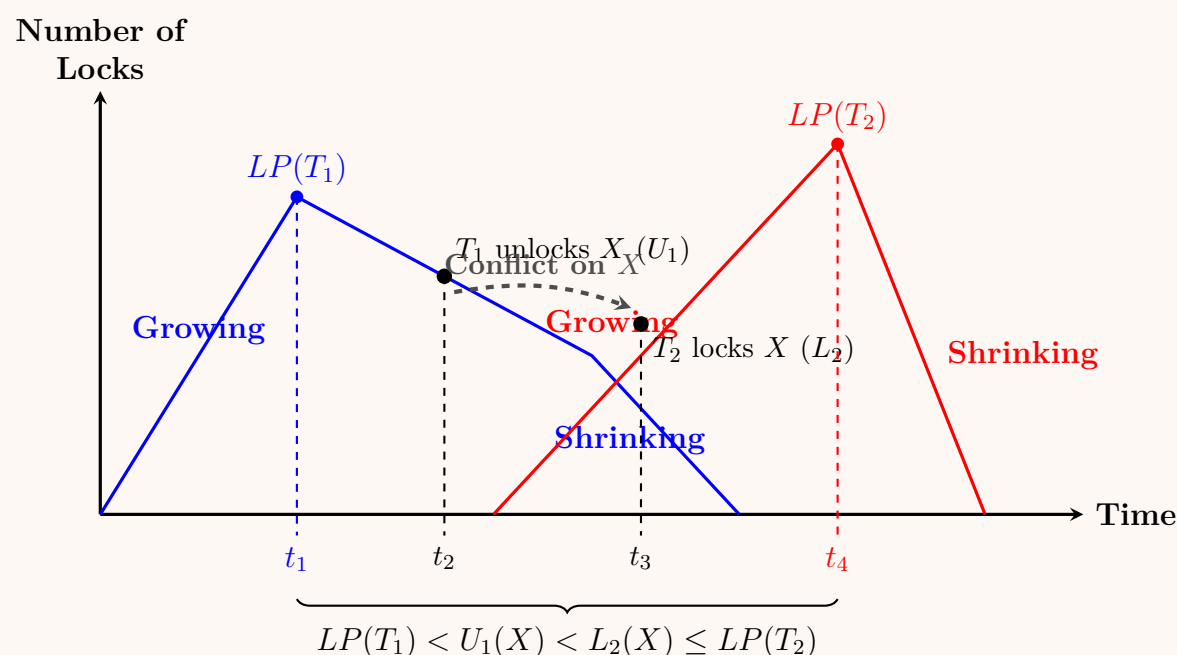
$$LP(T_1) < LP(T_1)$$

- (f) **Contradiction:** A point in time cannot be strictly less than itself. The assumption that a cycle can exist in a 2PL schedule leads to a temporal impossibility.

Conclusion: Because a cycle inherently creates a logical contradiction, the precedence graph of any 2PL schedule must be acyclic. Therefore, **every schedule that follows the 2PL protocol is guaranteed to be conflict-serializable**.

4. Visualizing the Fundamental Lemma

The diagram below visually demonstrates why the Lock Point of T_1 must strictly precede the Lock Point of T_2 when there is a conflict ($T_1 \rightarrow T_2$).



Q9. Consider the following transactions $T_1: l_1(A), r_1(A), w_1(A), u_1(A)$ and $T_2: l_2(A), r_2(A), w_2(A), u_2(A)$.

- (a) Do the transactions cause a deadlock when executed concurrently? Justify your answer.
 (b) Use shared and exclusive locks with lock upgrades to improve concurrency. Show the resulting transactions.
 (c) After incorporating lock upgrades, show a concurrent schedule that causes a deadlock.
 (d) Use update locks to solve the deadlock problem in (iii) above. Show the resulting transactions.

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

(a) Deadlock Analysis with Basic Locks

Answer: No, the transactions do not cause a deadlock when executed concurrently with basic, unspecialized locks.

Justification: Assuming the basic lock $l(A)$ functions as a general exclusive lock for the entire duration of the transaction on item A , only one transaction can acquire the lock at a time.

- If T_1 requests and acquires $l_1(A)$ first, T_2 will request $l_2(A)$ and be placed in a wait queue.
- T_1 will proceed to execute $r_1(A)$, $w_1(A)$, and finally release the lock with $u_1(A)$.
- Once T_1 unlocks A , T_2 acquires the lock and executes successfully.

Because both transactions compete for a single resource (A) and acquire only one lock each, there is no circular wait condition. One transaction simply blocks the other, ensuring serializability without deadlocking.

(b) Lock Upgrades (Shared and Exclusive Locks)

To improve concurrency, transactions can initially acquire a **Shared Lock (S-Lock)** for reading, and later request a **Lock Upgrade** to an **Exclusive Lock (X-Lock)** when they need to write.

Let:

- $sl(A)$ = Shared lock on A
- $xl(A)$ = Exclusive lock on A (Upgrade)

Resulting Transactions:

$$\begin{aligned} T_1 &: sl_1(A), r_1(A), xl_1(A), w_1(A), u_1(A) \\ T_2 &: sl_2(A), r_2(A), xl_2(A), w_2(A), u_2(A) \end{aligned}$$

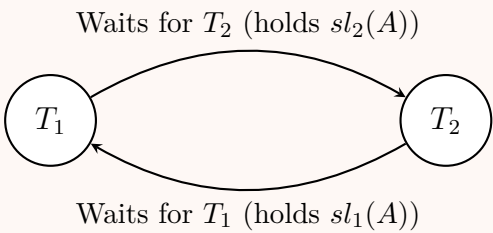
(c) Concurrent Schedule Causing a Deadlock

When incorporating lock upgrades, a deadlock occurs if both transactions acquire a shared lock on A concurrently, and then both attempt to upgrade to an exclusive lock.

Concurrent Schedule:

Transaction T_1	Transaction T_2	System State
$sl_1(A)$		T_1 gets S-lock on A .
	$sl_2(A)$	T_2 gets S-lock on A (S-locks are compatible).
$r_1(A)$		T_1 reads A .
	$r_2(A)$	T_2 reads A .
$xl_1(A)$		T_1 waits for T_2 to release its S-lock.
	$xl_2(A)$	T_2 waits for T_1 to release its S-lock.
Result: Deadlock (Circular Wait)		

Wait-For Graph (WFG): The following Wait-For Graph illustrates the circular dependency, which is the defining characteristic of a deadlock.



(d) Using Update Locks to Prevent Deadlock

Update Locks (U-Locks) act as an intermediate lock type to solve the lock upgrade deadlock.

- A U-lock is compatible with an S-lock but **incompatible** with another U-lock or X-lock.
- When a transaction intends to read and subsequently write to a resource, it first requests a U-lock.

- This ensures that only one transaction can hold the intention to write (U-lock) at any given time, preventing the scenario where two transactions concurrently hold S-locks and both attempt to upgrade.

Let:

- $ul(A)$ = Update lock on A

Resulting Transactions:

$$\begin{aligned} T_1 &: ul_1(A), r_1(A), xl_1(A), w_1(A), u_1(A) \\ T_2 &: ul_2(A), r_2(A), xl_2(A), w_2(A), u_2(A) \end{aligned}$$

Why this prevents deadlock: If T_1 acquires $ul_1(A)$, T_2 cannot acquire $ul_2(A)$ because U-locks are mutually exclusive. T_2 must wait until T_1 upgrades to $xl_1(A)$, performs the write, and unlocks A . Therefore, execution is safely serialized without circular waiting.

Q10. In which scenario does timestamp-based concurrency control perform better than lock-based protocol? In which scenario are lock-based protocols preferable? Discuss the rules of multi-version concurrency control (MVCC) protocol using a suitable example. **(13 marks)**
[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Performance Comparison: Timestamp vs. Lock-Based Protocols

The choice between timestamp-based and lock-based concurrency control protocols depends heavily on the transaction workload and the frequency of data conflicts.

Scenario: When Timestamp-Based Protocol Performs Better

- **Low Conflict Environments (Read-Heavy):** In scenarios where most transactions are read-only and write conflicts are rare, timestamp protocols excel. They do not incur the overhead of acquiring, managing, and releasing locks.
- **Deadlock-Free Requirements:** Timestamp ordering inherently prevents deadlocks since transactions do not wait for each other; they are either aborted or proceed. This is beneficial in distributed databases where detecting deadlocks is highly expensive.
- **High Concurrency Requirements:** Since there is no blocking, transactions that operate on disjoint sets of data can proceed seamlessly without any wait time.

Scenario: When Lock-Based Protocol is Preferable

- **High Conflict Environments (Write-Heavy):** In scenarios with frequent concurrent updates to the same data items, lock-based protocols are superior.
- **Avoiding Cascading Rollbacks:** Lock protocols (specifically Strict Two-Phase Locking, Strict 2PL) naturally prevent cascading rollbacks and the massive overhead of continuously aborting and restarting transactions, which is a major drawback of timestamp ordering under high contention.

Comparison Summary

Feature	Timestamp-Based	Lock-Based (2PL)
Conflict Resolution	Abort and restart (Optimistic/Non-blocking)	Wait in a queue (Pessimistic/Blocking)
Deadlocks	Never occur	Possible (requires detection/prevention)
Best Workload	Read-heavy, low contention	Write-heavy, high contention
Overhead	High if transaction aborts are frequent	High for lock management and deadlock checks

2. Multi-Version Concurrency Control (MVCC)

Definition: MVCC is a concurrency control method that maintains multiple versions of a data item. When a transaction writes to a data item, a new version is created rather than overwriting the old one. This ensures that **read operations are never blocked by write operations**.

MVCC Rules (Timestamp Ordering Variant): Assume every transaction T_i has a unique timestamp $TS(T_i)$. For every data item Q , a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$ is maintained. Each version Q_k stores:

- (a) **Value:** The actual data value.
- (b) **Write Timestamp ($W-TS(Q_k)$):** The timestamp of the transaction that created version Q_k .
- (c) **Read Timestamp ($R-TS(Q_k)$):** The largest timestamp among all transactions that successfully read version Q_k .

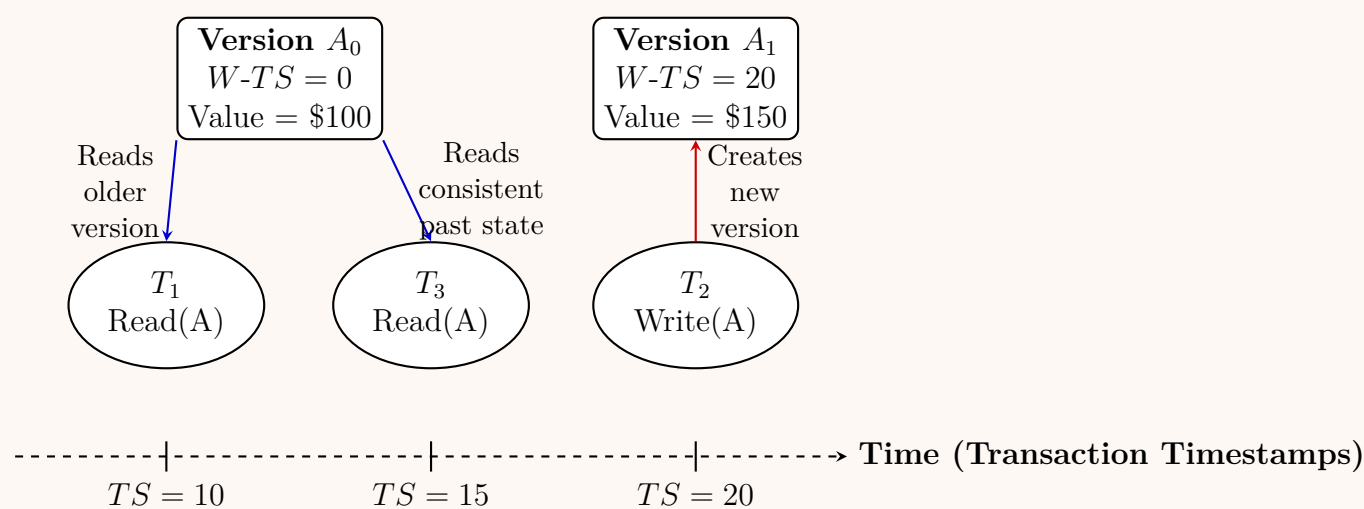
When a transaction T_i requests an operation on Q , the system identifies the specific version Q_k whose write timestamp is the largest timestamp less than or equal to $TS(T_i)$. Formally, $W-TS(Q_k) \leq TS(T_i)$.

- **Read(Q) by T_i :**
 - The system returns the value of version Q_k .
 - The read timestamp is updated: $R-TS(Q_k) = \max(R-TS(Q_k), TS(T_i))$.
- **Write(Q) by T_i :**
 - If $TS(T_i) < R-TS(Q_k)$: The value T_i wants to overwrite has already been read by a younger transaction (which expected the old value). T_i is **aborted and rolled back**.
 - If $TS(T_i) = W-TS(Q_k)$: T_i simply overwrites the contents of Q_k .
 - Otherwise: A **new version** Q_{new} is created with $W-TS(Q_{new}) = TS(T_i)$ and $R-TS(Q_{new}) = TS(T_i)$.

Suitable Example & Working Principle: Consider a bank account balance A .

- Initially, $A_0 = \$100$, created by T_0 with $TS = 0$. Thus, $W-TS(A_0) = 0$.
- Transaction T_1 ($TS = 10$) wants to read A . It reads A_0 . $R-TS(A_0)$ becomes 10.
- Transaction T_2 ($TS = 20$) writes \$150 to A . Since $TS(T_2) \geq R-TS(A_0)$, it succeeds and creates a **new version** A_1 with $W-TS(A_1) = 20$.
- Transaction T_3 ($TS = 15$) wants to read A . Under normal locking, T_3 might read \$150 or be blocked. Under MVCC, it looks for the version with the largest $W-TS \leq 15$. This is A_0 ($W-TS = 0$). T_3 successfully reads the older version \$100. No blocking occurs!

MVCC Read/Write Dynamics (Example)



Q11. Suppose timestamp-based concurrency control is used in a DBMS. For each read or write action, the DBMS takes one of the following actions:

- Wait: Put the transaction to sleep
- Abort: Abort the transaction (does not restart)
- Execute: Execute the action and continue
- Skip: Skip the action and continue

For each of the operations in the schedule below, write down the actions of the DBMS. Timestamps: $TS(T_1) < TS(T_2) < TS(T_3) < TS(T_4) < TS(T_5) < TS(T_6)$.

$r_2(A); w_1(A); w_4(B); r_3(B); w_2(A); r_4(A); w_6(C); w_5(C)$

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

To determine the action taken by the DBMS for each operation, we apply the rules of **Basic Timestamp Ordering** incorporating the **Thomas Write Rule** (which allows for the “Skip” action).

1. Preliminary Assumptions and Rules

Let the transactions have integer timestamps corresponding to their subscripts, satisfying the given condition $TS(T_1) < TS(T_2) < TS(T_3) < TS(T_4) < TS(T_5) < TS(T_6)$:

$$TS(T_1) = 1, \quad TS(T_2) = 2, \quad TS(T_3) = 3, \quad TS(T_4) = 4, \quad TS(T_5) = 5, \quad TS(T_6) = 6$$

For any data item X , the system maintains two timestamps, initialized to 0:

- $R-TS(X)$: The highest timestamp of any transaction that successfully read X .

- $W-TS(X)$: The highest timestamp of any transaction that successfully wrote X .

Rules for Read ($r_i(X)$):

- If $TS(T_i) < W-TS(X)$: **Abort** T_i (Value was already overwritten by a younger transaction).
- Otherwise: **Execute** the read and set $R-TS(X) = \max(R-TS(X), TS(T_i))$.

Rules for Write ($w_i(X)$):

- If $TS(T_i) < R-TS(X)$: **Abort** T_i (Value is needed by a younger transaction that already read the old value).
- If $TS(T_i) < W-TS(X)$: **Skip** the write (Thomas Write Rule: The value is already obsolete because a younger transaction has overwritten it).
- Otherwise: **Execute** the write and set $W-TS(X) = TS(T_i)$.

Note: The “Wait” action is generally not used in pure timestamp-based protocols as they are non-blocking, deadlock-free mechanisms.

2. Step-by-Step Schedule Evaluation

Initial State: All read and write timestamps for A , B , and C are 0.

Op	Item State Before	Condition Evaluation	Action	New Item State
$r_2(A)$	$W-TS(A) = 0$	$TS(T_2) \geq W-TS(A) \implies 2 \geq 0$	Execute	$R-TS(A) = 2$
$w_1(A)$	$R-TS(A) = 2, W-TS(A) = 0$	$TS(T_1) < R-TS(A) \implies 1 < 2$	Abort (T_1)	Unchanged
$w_4(B)$	$R-TS(B) = 0, W-TS(B) = 0$	$TS(T_4) \geq R-TS(B)$ and $\geq W-TS(B) \implies 4 \geq 0, 4 \geq 0$	Execute	$W-TS(B) = 4$
$r_3(B)$	$W-TS(B) = 4$	$TS(T_3) < W-TS(B) \implies 3 < 4$	Abort (T_3)	Unchanged
$w_2(A)$	$R-TS(A) = 2, W-TS(A) = 0$	$TS(T_2) \geq R-TS(A)$ and $\geq W-TS(A) \implies 2 \geq 2, 2 \geq 0$	Execute	$W-TS(A) = 2$
$r_4(A)$	$W-TS(A) = 2$	$TS(T_4) \geq W-TS(A) \implies 4 \geq 2$	Execute	$R-TS(A) = 4$
$w_6(C)$	$R-TS(C) = 0, W-TS(C) = 0$	$TS(T_6) \geq R-TS(C)$ and $\geq W-TS(C) \implies 6 \geq 0, 6 \geq 0$	Execute	$W-TS(C) = 6$
$w_5(C)$	$R-TS(C) = 0, W-TS(C) = 6$	$TS(T_5) \geq R-TS(C) \implies 5 \geq 0$ $TS(T_5) < W-TS(C) \implies 5 < 6$	Skip	Unchanged

3. Final Sequence of Actions

The actions taken by the DBMS for the sequential operations are:

- $r_2(A) \rightarrow$ **Execute**
- $w_1(A) \rightarrow$ **Abort**
- $w_4(B) \rightarrow$ **Execute**
- $r_3(B) \rightarrow$ **Abort**
- $w_2(A) \rightarrow$ **Execute**
- $r_4(A) \rightarrow$ **Execute**
- $w_6(C) \rightarrow$ **Execute**
- $w_5(C) \rightarrow$ **Skip**

Q12. Assume you are given the following schedules involving four transactions T_1, T_2, T_3 , and T_4 :

$$S_1: r_1(A); w_2(A); r_2(B); r_4(C); r_3(B); w_2(B); w_3(B); r_4(A); w_4(C); r_1(C)$$
$$S_2: r_1(A); w_2(A); r_2(B); r_4(C); w_2(B); r_3(B); w_3(B); r_4(A); r_1(C); w_4(C)$$

Construct precedence graphs for the given schedules S_1 and S_2 . Are the schedules conflict-serializable? Justify your answer. **(12 marks)**

CSE 215 | L-2/T-1 | 2022–23

Answer

Concept: A schedule is **conflict-serializable** if and only if its precedence graph (serializability graph) contains **no directed cycles**. A precedence graph consists of nodes representing transactions and directed edges representing conflicting operations (Read-Write, Write-Read, or Write-Write on the same data item by different transactions) in the order they occur.

1. Analysis of Schedule S_1

Schedule S_1 : $r_1(A); w_2(A); r_2(B); r_4(C); r_3(B); w_2(B); w_3(B); r_4(A); w_4(C); r_1(C)$

Conflict Analysis for S_1 :

Data Item	Conflicting Operations	Conflict Type	Directed Edge
A	$r_1(A)$ occurs before $w_2(A)$	Read-Write	$T_1 \rightarrow T_2$
A	$w_2(A)$ occurs before $r_4(A)$	Write-Read	$T_2 \rightarrow T_4$
B	$r_2(B)$ occurs before $w_3(B)$	Read-Write	$T_2 \rightarrow T_3$
B	$r_3(B)$ occurs before $w_2(B)$	Read-Write	$T_3 \rightarrow T_2$
B	$w_2(B)$ occurs before $w_3(B)$	Write-Write	$T_2 \rightarrow T_3$
C	$w_4(C)$ occurs before $r_1(C)$	Write-Read	$T_4 \rightarrow T_1$

Precedence Graph for S_1 :

Conclusion for S_1 : The precedence graph for S_1 contains **multiple cycles**:

- A cycle between T_2 and T_3 : $T_2 \rightarrow T_3 \rightarrow T_2$ (due to conflicting operations on B).
- A cycle involving T_1 , T_2 , and T_4 : $T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_1$ (due to operations on A and C).

Because of these cycles, **Schedule S_1 is NOT conflict-serializable.**

2. Analysis of Schedule S_2

Schedule S_2 : $r_1(A); w_2(A); r_2(B); r_4(C); w_2(B); r_3(B); w_3(B); r_4(A); r_1(C); w_4(C)$

Conflict Analysis for S_2 :

Data Item	Conflicting Operations	Conflict Type	Directed Edge
A	$r_1(A)$ occurs before $w_2(A)$	Read-Write	$T_1 \rightarrow T_2$
A	$w_2(A)$ occurs before $r_4(A)$	Write-Read	$T_2 \rightarrow T_4$
B	$r_2(B)$ occurs before $w_3(B)$	Read-Write	$T_2 \rightarrow T_3$
B	$w_2(B)$ occurs before $r_3(B)$	Write-Read	$T_2 \rightarrow T_3$
B	$w_2(B)$ occurs before $w_3(B)$	Write-Write	$T_2 \rightarrow T_3$
C	$r_1(C)$ occurs before $w_4(C)$	Read-Write	$T_1 \rightarrow T_4$

Note: The operations $r_4(C)$ and $r_1(C)$ do not conflict as both are read operations.

Precedence Graph for S_2 :

Conclusion for S_2 : The precedence graph for S_2 **does not contain any cycles**. All edges point in a forward direction (e.g., topological sorts such as $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4$ or $T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3$ are possible). Therefore, **Schedule S_2 is conflict-serializable**.

Q13. Show that a schedule satisfying strict 2PL (two-phase locking) is ACR (avoids cascading rollback). (5 marks) [CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Definitions of the Protocols

To prove that every schedule adhering to the **Strict Two-Phase Locking (Strict 2PL)** protocol also satisfies the **Avoiding Cascading Rollback (ACR)** property, we must first establish their formal definitions.

- **Strict 2PL:** A concurrency control protocol that enforces two fundamental conditions:
 - (a) *Two-Phase Condition:* Once a transaction releases any lock, it cannot acquire any new locks.
 - (b) *Strictness Condition:* All **exclusive locks (X-locks)** acquired by a transaction must be held continuously until that transaction either explicitly commits (C_i) or aborts (A_i).
- **Avoiding Cascading Rollback (ACR):** A property of a schedule where a transaction is only permitted to read data values that have been written by transactions that have **already committed**. Formally, if $w_i(X) < r_j(X)$, then $C_i < r_j(X)$ must hold in the schedule timeline.

2. Formal Proof by Contradiction

Let S be a concurrent schedule that satisfies the rules of **Strict 2PL**. We wish to show that S is **ACR**.

Suppose, for the sake of contradiction, that S satisfies Strict 2PL but is **NOT ACR**.

- (a) Since S is not ACR, there must exist a dirty read scenario somewhere within the schedule. This means a transaction T_j reads a data item X that was previously modified by an uncommitted transaction T_i .
- (b) Expressed chronologically as operations in the schedule:

$$w_i(X) < r_j(X) \quad \text{and} \quad r_j(X) < C_i$$

- (c) Now, let us analyze the locking requirements dictated by Strict 2PL for these operations to execute:

- For T_i to execute the write operation $w_i(X)$, it must first successfully acquire an exclusive lock on X , denoted as $xl_i(X)$. Thus, $xl_i(X) < w_i(X)$.
- For T_j to execute the read operation $r_j(X)$, it must first successfully acquire a shared lock (or exclusive lock) on X , denoted as $sl_j(X)$. Thus, $sl_j(X) < r_j(X)$.

- (d) Combining the execution sequence with the locking requirements yields the following timeline:

$$xl_i(X) < w_i(X) < sl_j(X) < r_j(X)$$

- (e) From this sequence, we see that T_j successfully acquires $sl_j(X)$ at a time strictly **before** T_i reaches its commit point C_i (since $r_j(X) < C_i$).
- (f) According to the fundamental rules of any lock manager, a shared lock $sl_j(X)$ and an exclusive lock $xl_i(X)$ are **mutually incompatible**. Therefore, T_j can only acquire $sl_j(X)$ if T_i has already released its exclusive lock $xl_i(X)$, denoted as $xu_i(X)$.

$$xl_i(X) < w_i(X) < xu_i(X) < sl_j(X) < r_j(X) < C_i$$

- (g) Looking at the sequence above, T_i released its exclusive lock ($xu_i(X)$) before committing (C_i). This means:

$$xu_i(X) < C_i$$

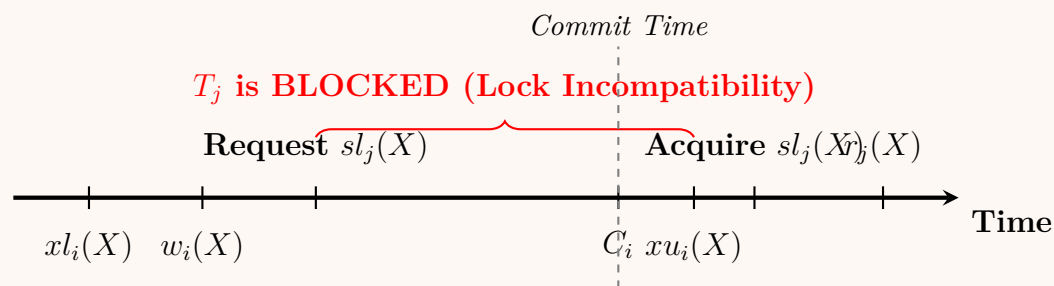
- (h) This directly violates the strictness condition of **Strict 2PL**, which mandates that all exclusive locks must be held until after the commit operation ($C_i < xu_i(X)$).

Since assuming S is not ACR leads directly to a violation of the Strict 2PL rules, our initial assumption must be false.

$\therefore$ Every schedule that satisfies Strict 2PL must also be ACR.

3. Visualizing the Blocked Execution in Strict 2PL

The timeline diagram below illustrates how Strict 2PL naturally avoids cascading rollbacks by forcing dependent transactions to wait until the preceding transaction has committed.



Working Mechanism Note: As shown above, T_j cannot read the uncommitted changes of T_i because T_i safely protects X with an exclusive lock until it commits at C_i . Since T_j can never read uncommitted data, it is completely insulated from the failure or rollback of T_i , thereby inherently preventing cascading rollbacks.

Q14. Consider the following two transactions: $T_1: r_1(A); w_1(A)$ and $T_2: w_2(A)$.

- Construct a concurrent schedule of T_1 and T_2 that utilises the “Thomas Write Rule”. Show a serial schedule which is equivalent to your given concurrent schedule.
- What is the problem of allowing the Thomas Write Rule?

(18 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

(a) Concurrent Schedule Using Thomas Write Rule (TWR)

Assumption: Let the system assign timestamps such that T_1 is older than T_2 . Let $TS(T_1) = 10$ and $TS(T_2) = 20$. Initially, $R-TS(A) = 0$ and $W-TS(A) = 0$.

To invoke the Thomas Write Rule, a transaction must attempt to write a data item that has already been overwritten by a *younger* transaction.

Concurrent Schedule (S_c):

$r_1(A); w_2(A); w_1(A)$

Execution Trace (Applying Timestamp Ordering & TWR):

Operation	Item A State Before	Condition Evaluation	Action by DBMS
$r_1(A)$	$W-TS(A) = 0$	$TS(T_1) \geq W-TS(A) \implies 10 \geq 0$	Execute ($R-TS(A) \leftarrow 10$)
$w_2(A)$	$R-TS(A) = 10, W-TS = 0$	$TS(T_2) \geq R-TS(A) \implies 20 \geq 10$	Execute ($W-TS(A) \leftarrow 20$)
$w_1(A)$	$W-TS(A) = 20$	$TS(T_1) < W-TS(A) \implies 10 < 20$	Skip (TWR applied)

Equivalent Serial Schedule: Because the write operation $w_1(A)$ is safely ignored (skipped) by the DBMS, the final value of A in the database is the one written by T_2 . Furthermore, T_1 successfully read the initial value of A before T_2 modified it.

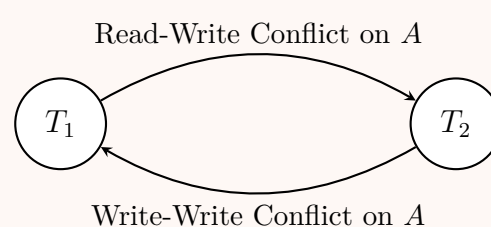
Therefore, the concurrent schedule S_c is logically equivalent to the serial schedule where T_1 executes entirely before T_2 .

$S_{\text{serial}}: T_1 \rightarrow T_2$ (i.e., $r_1(A); w_1(A); w_2(A)$)

(b) The Problem with Allowing the Thomas Write Rule

While the Thomas Write Rule elegantly resolves obsolete writes without aborting transactions, it introduces several theoretical and practical complexities in database management:

- Violation of Conflict Serializability:** Schedules permitted by TWR are **View Serializable** but **NOT Conflict Serializable**. If we draw the precedence graph for the schedule S_c above, we observe a cycle.



A standard lock-based concurrency protocol (like 2PL) would unnecessarily block or deadlock trying to prevent this, whereas TWR just ignores the conflict. Testing for View Serializability is computationally **NP-hard**, meaning it is virtually impossible for a general-purpose scheduler to dynamically verify view serializability without explicitly using timestamp protocols with TWR built-in.

- **Application Logic Anomalies (Phantom Success):** TWR silently ignores a write operation ($w_1(A)$). The DBMS does not notify the transaction T_1 that its write was discarded. If T_1 's external logic (e.g., in the application layer) relies on its write being successful and permanent (for example, generating an audit log or a subsequent dependent operation outside the DBMS), the application may proceed under a false assumption.
- **Dependency on Blind Writes:** TWR is only applicable to **blind writes** (when a transaction writes to a data item without reading it first). If T_1 had attempted to read A right before $w_1(A)$, it would have aborted anyway because $TS(T_1) < W-TS(A)$. Thus, the practical use cases for TWR are highly constrained to specific workloads.

Q15. Consider the following two transactions using shared ($sl_i(X)$) and exclusive ($xl_i(X)$) locks:

$$\begin{aligned} T_1: & \; sl_1(A); r_1(A); xl_1(A); w_1(A); u_1(A); c_1 \\ T_2: & \; sl_2(A); r_2(A); xl_2(A); w_2(A); u_2(A); c_2 \end{aligned}$$

- (a) Construct a concurrent schedule of T_1 and T_2 that results in a deadlock due to lock upgrading.
- (b) Modify the given transactions using update locks ($ul_i(X)$) to prevent deadlocks.

(10 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

(a) **Concurrent Schedule Resulting in a Deadlock**

Concept: A lock upgrade deadlock occurs when two concurrent transactions both acquire a **Shared Lock (S-Lock)** on the same data item and subsequently both attempt to upgrade their lock to an **Exclusive Lock (X-Lock)**. Because exclusive locks require the transaction to be the sole lock holder on that item, both transactions end up waiting indefinitely for the other to release its shared lock.

Concurrent Schedule:

Transaction T_1	Transaction T_2	Lock Manager / System State
$sl_1(A)$		T_1 is granted a shared lock on A .
	$sl_2(A)$	T_2 is granted a shared lock on A (S-locks are compatible).
$r_1(A)$		T_1 successfully reads A .
	$r_2(A)$	T_2 successfully reads A .
$xl_1(A)$		T_1 requests X-lock; Blocked . Waits for T_2 to release $sl_2(A)$.
	$xl_2(A)$	T_2 requests X-lock; Blocked . Waits for T_1 to release $sl_1(A)$.

Result: Deadlock (Circular Wait Condition)

Wait-For Graph: The cyclic dependency that characterizes this deadlock can be visualized using a Wait-For Graph (WFG).

```
graph LR; T1((T1)) -- "Requests xl1(A)" --> T2((T2)); T2 -- "(waits for sl2(A))" --> T1; T1 -- "(waits for sl1(A))" --> T2; T2 -- "Requests xl2(A)" --> T1;
```

(b) **Preventing Deadlock using Update Locks**

Concept: An **Update Lock (U-Lock)** is a specialized lock designed specifically to prevent lock upgrade deadlocks.

- It indicates an *intention* to read a data item and later upgrade to an exclusive lock to write to it.
- **Compatibility:** A U-lock is compatible with existing S-locks but is **strictly incompatible** with other U-locks and X-locks.
- By requiring transactions to acquire a U-lock initially (instead of an S-lock), the system ensures that only one transaction can hold the intention to upgrade at any given time.

Modified Transactions: We replace the initial shared lock (sl) with an update lock (ul).

$$\begin{aligned} T_1: & \text{ul}_1(A); r_1(A); xl_1(A); w_1(A); u_1(A); c_1 \\ T_2: & \text{ul}_2(A); r_2(A); xl_2(A); w_2(A); u_2(A); c_2 \end{aligned}$$

Why this prevents the deadlock: If both transactions attempt to execute concurrently:

- (i) T_1 requests and is granted $ul_1(A)$.
- (ii) T_2 requests $ul_2(A)$. Because U-locks are mutually exclusive with each other, T_2 is **immediately blocked** and enters the wait queue before it can acquire any locks or read data.
- (iii) T_1 proceeds to read A , successfully upgrades its lock to $xl_1(A)$ (since no other transaction holds any locks on A), writes A , unlocks A , and commits.
- (iv) Once T_1 releases its locks ($u_1(A)$), T_2 is unblocked, granted $ul_2(A)$, and executes sequentially.

By serializing the lock acquisition at the earliest phase, the circular wait condition is entirely eliminated.

Q16. Consider the following schedule of four transactions with start timestamps $TS(T_1) = 1$, $TS(T_2) = 2$, $TS(T_3) = 3$, $TS(T_4) = 4$ and initial read/write timestamps $RT(X) = WT(X) = 0$ for $X \in \{A, B, C\}$:

$$r_1(A); w_2(A); r_2(B); r_4(C); r_3(B); w_2(B); w_3(B); r_4(A); w_4(C); r_1(C)$$

- (a) For each action, show the timestamp which is updated and show the updated values.
- (b) Show the actions for which the scheduler rolls back the relevant transaction.

(12 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

Basic Timestamp Ordering Rules

For a transaction T_i with timestamp $TS(T_i)$ and a data item X with Read Timestamp $RT(X)$ and Write Timestamp $WT(X)$:

- **Read Operation** $r_i(X)$:
 - If $TS(T_i) < WT(X)$, the read is rejected, and T_i is **rolled back**.
 - Otherwise, the read is executed, and $RT(X)$ is updated to $\max(RT(X), TS(T_i))$.
- **Write Operation** $w_i(X)$:
 - If $TS(T_i) < RT(X)$ or $TS(T_i) < WT(X)$, the write is rejected, and T_i is **rolled back**.
 - Otherwise, the write is executed, and $WT(X)$ is updated to $TS(T_i)$.

Given Timestamps: $TS(T_1) = 1$, $TS(T_2) = 2$, $TS(T_3) = 3$, $TS(T_4) = 4$.

Initial States: $RT(X) = 0, WT(X) = 0$ for all $X \in \{A, B, C\}$.

(a) Step-by-Step Execution and Timestamp Updates

Action	Item State Before	Condition Check	Status	Updated Value(s)
$r_1(A)$	$WT(A) = 0$	$TS(T_1) \geq WT(A) \implies 1 \geq 0$	Executed	$RT(A) \leftarrow \max(0, 1) = 1$
$w_2(A)$	$RT(A) = 1, WT(A) = 0$	$TS(T_2) \geq RT(A) \implies 2 \geq 1$ $TS(T_2) \geq WT(A) \implies 2 \geq 0$	Executed	$WT(A) \leftarrow 2$
$r_2(B)$	$WT(B) = 0$	$TS(T_2) \geq WT(B) \implies 2 \geq 0$	Executed	$RT(B) \leftarrow \max(0, 2) = 2$
$r_4(C)$	$WT(C) = 0$	$TS(T_4) \geq WT(C) \implies 4 \geq 0$	Executed	$RT(C) \leftarrow \max(0, 4) = 4$
$r_3(B)$	$WT(B) = 0$	$TS(T_3) \geq WT(B) \implies 3 \geq 0$	Executed	$RT(B) \leftarrow \max(2, 3) = 3$
$w_2(B)$	$RT(B) = 3, WT(B) = 0$	$TS(T_2) < RT(B) \implies 2 < 3$	Rejected	<i>None (Rollback T_2)</i>
$w_3(B)$	$RT(B) = 3, WT(B) = 0$	$TS(T_3) \geq RT(B) \implies 3 \geq 3$ $TS(T_3) \geq WT(B) \implies 3 \geq 0$	Executed	$WT(B) \leftarrow 3$
$r_4(A)$	$WT(A) = 0$ (See Note)	$TS(T_4) \geq WT(A) \implies 4 \geq 0$	Executed	$RT(A) \leftarrow \max(1, 4) = 4$
$w_4(C)$	$RT(C) = 4, WT(C) = 0$	$TS(T_4) \geq RT(C) \implies 4 \geq 4$ $TS(T_4) \geq WT(C) \implies 4 \geq 0$	Executed	$WT(C) \leftarrow 4$
$r_1(C)$	$WT(C) = 4$	$TS(T_1) < WT(C) \implies 1 < 4$	Rejected	<i>None (Rollback T_1)</i>

Note on System Recovery: When $w_2(B)$ is rejected, transaction T_2 is rolled back. In a rigorous DBMS execution, rolling back T_2 dictates that its prior successful write ($w_2(A)$) must be undone. Consequently, $WT(A)$ reverts from 2 back to its

previous state of 0. Therefore, when $r_4(A)$ executes, it observes $WT(A) = 0$. (Even if the rollback effect is delayed and $WT(A)$ remains 2, $r_4(A)$ would still successfully execute since $4 \geq 2$).

(b) **Actions Causing Rollbacks**

Based on the execution trace above, the scheduler aborts and rolls back transactions for the following specific actions:

- (i) **Action:** $w_2(B)$
Reason: Transaction T_2 ($TS = 2$) attempts to write to data item B , but B has already been read by a younger transaction T_3 ($TS = 3$). Since $TS(T_2) < RT(B)$, the write operation is obsolete and invalidates T_3 's read, forcing the scheduler to **rollback** T_2 .
- (ii) **Action:** $r_1(C)$
Reason: Transaction T_1 ($TS = 1$) attempts to read data item C , but C has already been completely overwritten by a younger transaction T_4 ($TS = 4$). Since $TS(T_1) < WT(C)$, T_1 is attempting to read a value from the past that has been logically destroyed in the serializability order, forcing the scheduler to **rollback** T_1 .

Q17. Consider the following two transactions $T_1: r_1(A); w_1(A); c_1$ and $T_2: r_2(A); c_2$.

- (a) Construct a concurrent schedule of T_1 and T_2 that is recoverable but not ACR (avoids cascading rollback).
- (b) Construct a concurrent schedule of T_1 and T_2 that is serializable but not recoverable.
- (c) Construct a schedule of T_1 and T_2 that is ACR (avoids cascading rollback).

(11 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

Given two transactions:

- **Transaction T_1 :** $r_1(A); w_1(A); c_1$
- **Transaction T_2 :** $r_2(A); c_2$

Below are the constructed schedules satisfying the required properties.

(a) **Construct a concurrent schedule of T_1 and T_2 that is Recoverable but not ACR (Avoids Cascading Rollback).**

Definition & Working Principle:

- **Recoverable (RC):** A schedule is recoverable if, for every transaction T_j that reads a value updated by a transaction T_i , T_i commits *before* T_j commits.
- **Not ACR (Cascading Rollback Possible):** A schedule is not ACR if it allows a transaction to read uncommitted data (a “dirty read”).

Schedule Construction: To make it not ACR, T_2 must read A after T_1 writes A , but before T_1 commits. To ensure it remains recoverable, T_1 must commit before T_2 commits.

Time	Transaction T_1	Transaction T_2
t_1	$r_1(A)$	
t_2	$w_1(A)$	
t_3		$r_2(A) \leftarrow$ Dirty Read (Not ACR)
t_4	c_1	
t_5		$c_2 \leftarrow c_1$ before c_2 (Recoverable)

(b) **Construct a concurrent schedule of T_1 and T_2 that is Serializable but not Recoverable.**

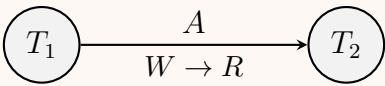
Definition & Working Principle:

- **Not Recoverable:** A transaction T_j reads data written by T_i , and T_j commits *before* T_i commits. If T_i later aborts, T_j cannot be rolled back, violating durability.
- **Serializable (SR):** The schedule must be equivalent to some serial schedule (either $T_1 \rightarrow T_2$ or $T_2 \rightarrow T_1$).

Schedule Construction: T_2 reads A after T_1 writes it, and then T_2 commits *before* T_1 commits.

Time	Transaction T_1	Transaction T_2
t_1	$r_1(A)$	
t_2	$w_1(A)$	
t_3		$r_2(A) \leftarrow$ Reads uncommitted A
t_4		$c_2 \leftarrow$ Commits before T_1 (Unrecoverable)
t_5	c_1	

Proof of Serializability (Precedence Graph): The schedule creates a Read-Write (Write-Read) conflict on A from T_1 to T_2 . Because there is no cycle in the precedence graph, the schedule is conflict serializable (equivalent to the serial schedule $T_1 \rightarrow T_2$).



(c) Construct a schedule of T_1 and T_2 that is ACR (Avoids Cascading Rollback).

Definition & Working Principle:

- **ACR (Cascadeless):** A schedule avoids cascading rollbacks if every transaction reads only data that has been written by already-committed transactions.

Schedule Construction: To guarantee an ACR schedule, T_2 must only read A *after* T_1 has committed its write operation on A .

Time	Transaction T_1	Transaction T_2
t_1	$r_1(A)$	
t_2	$w_1(A)$	
t_3	c_1	
t_4		$r_2(A) \leftarrow$ Reads committed data (ACR)
t_5		c_2

Notes: This specific ACR schedule is strictly serial ($T_1 \rightarrow T_2$). A strictly serial schedule is inherently cascadeless because no concurrent uncommitted reads can possibly occur. Another valid concurrent ACR schedule could have T_2 reading A at t_1 , before T_1 writes it.

Q18. With necessary figures and examples, explain how Two-Phase Locking is implemented using a Lock Table. **(10 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Introduction to Two-Phase Locking (2PL)

The **Two-Phase Locking (2PL)** protocol is a concurrency control method that ensures conflict serializability in a database management system. It dictates that every transaction must issue lock and unlock requests in two distinct phases:

- **Growing Phase:** A transaction may obtain locks but may not release any lock.
- **Shrinking Phase:** A transaction may release locks but may not obtain any new locks.

The exact moment a transaction acquires its final lock before releasing any is called the **Lock Point**.

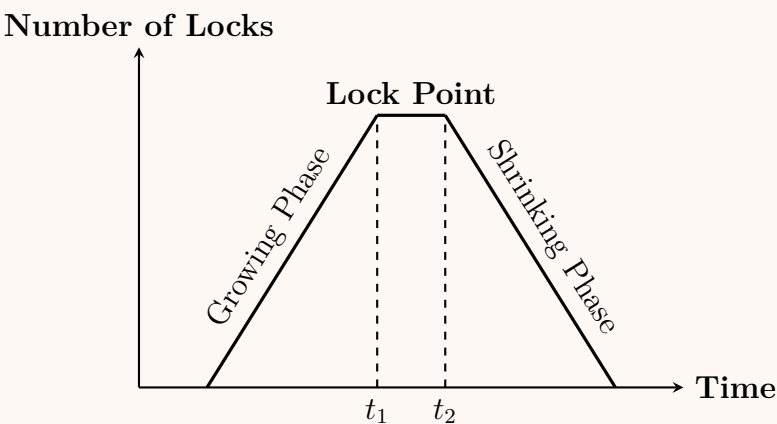


Figure 1: The Two Phases of the 2PL Protocol

2. Implementation using a Lock Table

In a DBMS, the 2PL protocol is implemented using a system component called the **Lock Manager**. The Lock Manager maintains a data structure called the **Lock Table** in main memory to track the status of all locks.

Structure of the Lock Table

The Lock Table is typically implemented as an in-memory **Hash Table** to ensure $O(1)$ average time complexity for lock requests and releases.

- **Hash Index:** The unique identifier of a Data Item (e.g., Page ID, Row ID) is passed through a hash function to find the correct bucket in the Lock Table.

- **Linked List of Requests:** Each bucket points to a linked list of active data items.
- **Request Nodes:** Each data item points to a queue of transaction requests. A request node generally contains:
 - **Transaction ID (T_i):** The transaction requesting/holding the lock.
 - **Lock Mode:** Shared (S) or Exclusive (X).
 - **Status:** Granted or Waiting.
 - **Pointer:** Points to the next request in the queue.

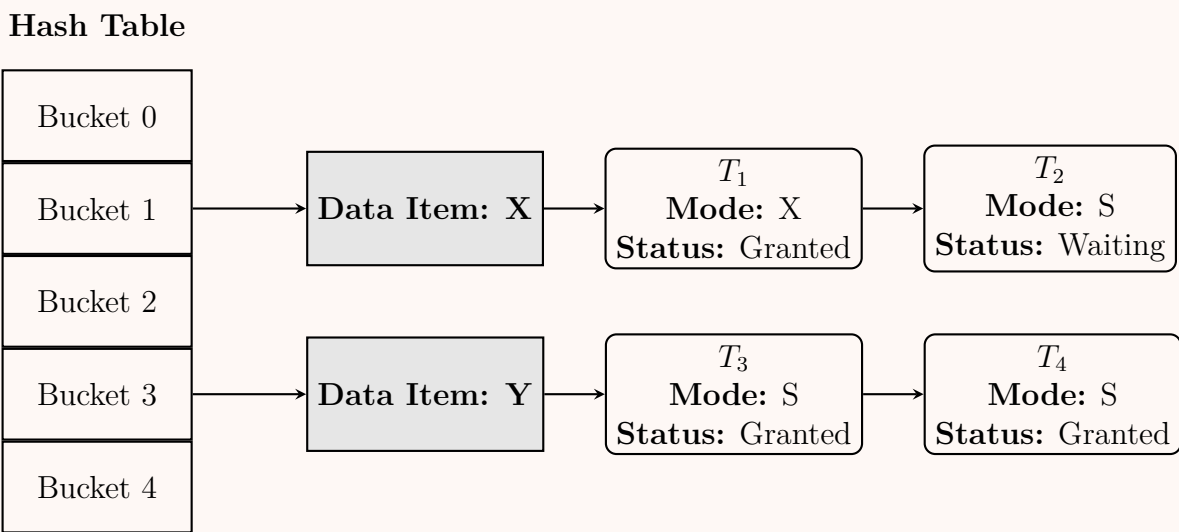


Figure 2: Architecture of a Lock Table showing Granted and Waiting requests

3. Working Principle (Lock Manager Algorithm)

- (a) **Lock Request Arrival:** Transaction T_i requests a lock of mode M (either S or X) on Data Item D .
- (b) **Hashing & Lookup:** The Lock Manager hashes D to find the target bucket in the Lock Table.
- (c) **Evaluating the Request:**
 - *Case A (Item not locked):* If D is not in the list, a new header node for D is created. A lock request node for T_i is created, its status is set to **Granted**, and it is attached to D .
 - *Case B (Item already locked - Compatible):* If D exists and all currently **Granted** locks are compatible with mode M (e.g., T_i requests S -lock, and existing locks are only S -locks), and the waiting queue is empty, the lock is **Granted** and appended.
 - *Case C (Item already locked - Incompatible):* If the currently **Granted** lock is incompatible (e.g., T_i requests an X -lock while an S -lock is held), T_i 's request is appended to the queue with status **Waiting**. The transaction T_i is suspended.
- (d) **Unlock Operation:** When T_j releases a lock on D , its node is removed from the list. The Lock Manager checks the next node(s) in the **Waiting** queue. If the next request is compatible with remaining granted locks (if any), its status is changed to **Granted**, and the suspended transaction is awakened.

4. Example Scenario

Consider two transactions, T_1 and T_2 , operating on Data Item X :

Time	Transaction Action	Lock Table Entry for X	Outcome
t_1	T_1 requests Exclusive (X) lock	$[(T_1, X, \textit{Granted})]$	T_1 proceeds
t_2	T_2 requests Shared (S) lock	$[(T_1, X, \textit{Granted})] \rightarrow [(T_2, S, \textit{Waiting})]$	T_2 blocked
t_3	T_1 commits and un-locks X	$[(T_2, S, \textit{Granted})]$	T_2 awakened

5. Properties and Trade-offs

Advantages:

- **Strict Serializability:** 2PL mathematically guarantees conflict serializability.
- **Efficiency:** Hash-based Lock Tables provide rapid detection of lock states.

Disadvantages:

- **Deadlocks:** A situation where T_1 waits for T_2 , and T_2 waits for T_1 . The Lock Manager must run a separate deadlock detection algorithm (e.g., checking for cycles in a Wait-For Graph).
- **Memory Overhead:** Maintaining the Lock Table structure consumes main memory space, requiring careful tuning of hash buckets to prevent severe collision chains.

Q19. The SQL standard defines three weak isolation levels: dirty reads, read committed, and repeatable reads. Compare these three levels with illustrative examples. **(15 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Introduction to Isolation Levels

In a Database Management System (DBMS), **Isolation** determines how transaction integrity is visible to other users and systems. The SQL standard defines different isolation levels based on the presence of three specific read phenomena:

- **Dirty Read:** Reading uncommitted data from another transaction.
- **Non-Repeatable Read:** Reading the same row twice and getting different values because another transaction updated it in the meantime.
- **Phantom Read:** Executing a query twice and getting a different set of rows because another transaction inserted or deleted rows matching the query condition.

Below is a comparison of the three weak isolation levels defined by the SQL standard.

2. Read Uncommitted (Dirty Reads Allowed)

Definition: This is the lowest isolation level. A transaction executing under this level can see uncommitted changes made by other concurrent transactions.

Working Principle: It does not acquire read locks on data being read, nor does it wait for other transactions to release exclusive locks.

Example Scenario (Dirty Read Anomaly): Suppose T_1 updates an account balance (A), and T_2 reads it before T_1 commits or aborts. If T_1 rolls back, T_2 has read a value that technically never existed in the database.

Time	Transaction T_1	Transaction T_2 (Read Uncommitted)
t_1	Read(A)	
t_2	Write(A) [Changes A to 500]	
t_3		Read(A) $\leftarrow$ Reads 500 (Dirty Read)
t_4	Rollback	
t_5		Commit

3. Read Committed

Definition: This level guarantees that any data read is committed at the moment it is read. It prevents dirty reads but does not prevent non-repeatable reads.

Working Principle: The DBMS holds write locks until the end of the transaction, but read locks are released immediately after the data is read.

Example Scenario (Non-Repeatable Read Anomaly): Suppose T_2 reads account balance A . Then T_1 updates A and commits. If T_2 reads A again, it sees a different value.

Time	Transaction T_1	Transaction T_2 (Read Committed)
t_1		Read(A) $\leftarrow$ Reads 100
t_2	Read(A)	
t_3	Write(A) [Changes A to 500]	
t_4	Commit	
t_5		Read(A) $\leftarrow$ Reads 500 (Different value)
t_6		Commit

4. Repeatable Read

Definition: This level ensures that if a transaction reads data once, it will see the exact same data if it reads it again, preventing both dirty reads and non-repeatable reads. However, it may still allow phantom reads.

Working Principle: Both read and write locks are held until the end of the transaction. This prevents other transactions from modifying any rows that the current transaction has read.

Example Scenario (Phantom Read Anomaly): T_2 queries the total number of accounts in a branch. T_1 inserts a new account and commits. If T_2 runs the exact same query again, a "phantom" row appears.

396

Time	Transaction T_1	Transaction T_2 (Repeatable Read)
t_1		Select COUNT(*) ← Returns 5
t_2	Insert new row	
t_3	Commit	
t_4		Select COUNT(*) ← Returns 6 (Phantom)
t_5		Commit

5. Comparison Table

The following table summarizes the concurrency anomalies prevented and permitted by these three weak isolation levels.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible

Note: The highest isolation level, **Serializable** (not detailed here), is required to prevent all three phenomena, executing transactions such that the outcome is equivalent to a sequential execution.

Q20. For the following schedule containing four concurrent transactions shown in chronological order, sketch the precedence graph and determine whether the schedule is conflict serializable. If so, write down the equivalent serial schedule(s).

$R_1(A), W_2(B), R_3(C), W_3(A), R_1(D), R_4(B), W_4(D), W_2(C)$

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Schedule Analysis and Conflict Identification

A schedule is **conflict serializable** if its precedence graph (serialization graph) contains no cycles. To construct the precedence graph, we must first identify all conflicting operations.

Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write (**W**) operation.

Given the chronological schedule S :

$S = R_1(A), W_2(B), R_3(C), W_3(A), R_1(D), R_4(B), W_4(D), W_2(C)$

Let us organize the operations by transaction and analyze the conflicts sequentially:

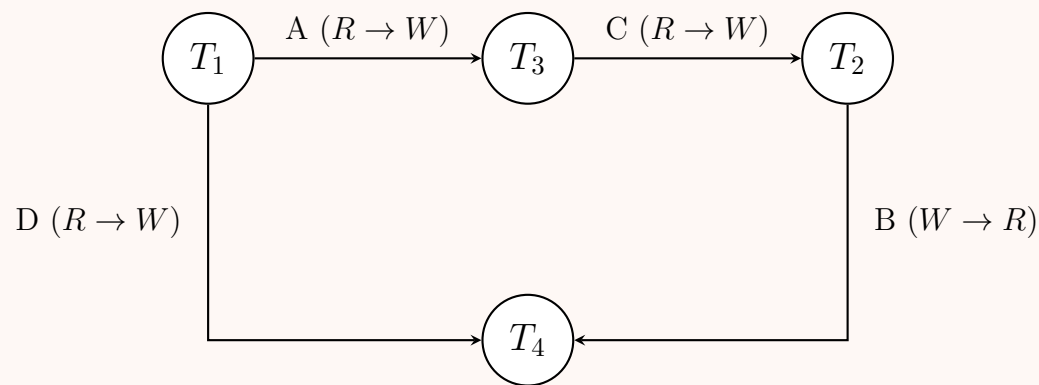
Time	Data Item	T_1	T_2	T_3	T_4
t_1	A	$R_1(A)$			
t_2	B		$W_2(B)$		
t_3	C			$R_3(C)$	
t_4	A			$W_3(A)$	
t_5	D	$R_1(D)$			
t_6	B				$R_4(B)$
t_7	D				$W_4(D)$
t_8	C		$W_2(C)$		

Identified Conflicts:

- On Data Item A: $R_1(A)$ occurs before $W_3(A) \implies$ Conflict $T_1 \rightarrow T_3$ (Read-Write)
- On Data Item B: $W_2(B)$ occurs before $R_4(B) \implies$ Conflict $T_2 \rightarrow T_4$ (Write-Read)
- On Data Item C: $R_3(C)$ occurs before $W_2(C) \implies$ Conflict $T_3 \rightarrow T_2$ (Read-Write)
- On Data Item D: $R_1(D)$ occurs before $W_4(D) \implies$ Conflict $T_1 \rightarrow T_4$ (Read-Write)

2. Precedence Graph

Based on the conflicts identified above, we draw the precedence graph where nodes represent transactions and directed edges represent the conflict dependencies.



3. Conflict Serializability & Equivalent Serial Schedule

Decision: Observing the precedence graph, we can trace the paths:

- Path 1: $T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
- Path 2: $T_1 \rightarrow T_4$

Since all edges flow in a forward direction and there are **no cycles** in the graph, the schedule is **conflict serializable**.

Equivalent Serial Schedule: To find the equivalent serial schedule(s), we perform a topological sort on the acyclic precedence graph.

- T_1 has an in-degree of 0, so it must execute first.
- Removing T_1 leaves T_3 with an in-degree of 0.
- Removing T_3 leaves T_2 with an in-degree of 0.
- Removing T_2 leaves T_4 with an in-degree of 0.

There is only one valid topological order. Thus, the **equivalent serial schedule** is:

$$T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$$

Q21. The list of instructions of transactions T_1 , T_2 and T_3 are given in chronological order as follows:

$T_1(\text{READ } A), T_2(\text{WRITE } A), T_3(\text{READ } A), T_1(\text{WRITE } B), T_2(\text{WRITE } B),$

$T_1(\text{READ } B), T_1(\text{WRITE } P), T_3(\text{WRITE } Q), T_2(\text{READ } B), T_1(\text{WRITE } A)$

Put appropriate locks on the transactions and show the status of locks. In appropriate places, you can upgrade or downgrade a lock.
(15 marks) [CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Lock Implementation Strategy

To handle the given sequence of instructions, we apply a standard lock management protocol using ****Shared Locks (S)**** for read operations and ****Exclusive Locks (X)**** for write operations.

- ****Lock Upgrade:**** A transaction holding an S lock can request to upgrade it to an X lock if it needs to perform a write operation on the same data item, provided no other transaction holds a conflicting lock.
- ****Lock Downgrade:**** A transaction can convert an X lock to an S lock if it no longer needs exclusive access but still needs to read the item.
- ****Lock Status:**** A lock can either be **Granted** (if compatible with existing locks) or **Waiting** (if a conflict occurs, causing the transaction to block).

2. Chronological Lock Table Status Trace

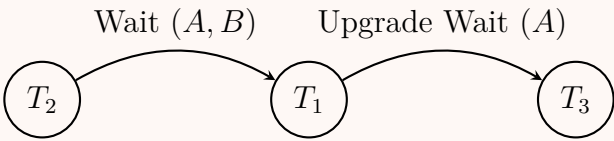
Below is the systematic step-by-step trace of the lock requests, lock table status, and structural changes (upgrades/blocking) corresponding to each chronological instruction.

Step	Instruction	Lock Request	Lock Table Status	Remarks / Transaction State
1	T_1 (READ A)	Request $S(A)$ by T_1	$A : [T_1 : S]$	Granted.
2	T_2 (WRITE A)	Request $X(A)$ by T_2	$A : [T_1 : S] \rightarrow [T_2 : X(\text{Wait})]$	Waiting (T_2 blocks due to T_1 's $S(A)$).
3	T_3 (READ A)	Request $S(A)$ by T_3	$A : [T_1 : S, T_3 : S] \rightarrow [T_2 : X(\text{Wait})]$	Granted (Shared locks are compatible).
4	T_1 (WRITE B)	Request $X(B)$ by T_1	$A : [T_1 : S, T_3 : S], B : [T_1 : X]$	Granted.
5	T_2 (WRITE B)	Request $X(B)$ by T_2	$B : [T_1 : X] \rightarrow [T_2 : X(\text{Wait})]$	Waiting (T_2 is already blocked).
6	T_1 (READ B)	Downgrade $X(B) \rightarrow S(B)$	$B : [T_1 : S] \rightarrow [T_2 : X(\text{Wait})]$	Downgraded by T_1 to allow shared access.
7	T_1 (WRITE P)	Request $X(P)$ by T_1	$P : [T_1 : X]$	Granted.
8	T_3 (WRITE Q)	Request $X(Q)$ by T_3	$Q : [T_3 : X]$	Granted.
9	T_2 (READ B)	Already in Wait Queue	$B : [T_1 : S] \rightarrow [T_2 : X(\text{Wait})]$	Waiting (T_2 remains blocked).
10	T_1 (WRITE A)	Upgrade $S(A) \rightarrow X(A)$	$A : [T_1 : S(\text{Wait}), T_3 : S] \rightarrow [T_2 : X(\text{Wait})]$	Waiting (T_1 blocks due to T_3 's $S(A)$).

3. Deadlock Analysis

From the final state of the lock table at Step 10, we can construct a ****Wait-For Graph (WFG)**** to check for a system deadlock:

- T_2 is waiting for T_1 to release its lock on data item A and data item B .
- T_1 is waiting for T_3 to release its shared lock on data item A to complete its upgrade to an exclusive lock.



Conclusion: As seen in the dependency graph, there is a linear dependency path ($T_2 \rightarrow T_1 \rightarrow T_3$) but no closed loop/cycle. Therefore, the execution sequence does not cause an immediate deadlock, but T_1 and T_2 will remain blocked until T_3 finishes its execution and releases its locks.

Q22. Explain conflict serializability with an example. Two transactions T_2 : READ(A), WRITE(A), COMMIT and T_1 : READ(A), READ(B), COMMIT are given. Write down the concurrent schedule and the corresponding precedence graph. Using the precedence graph, prove that your schedule is serializable. Find the equivalent serial schedule. **(15 marks)** [CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Conflict Serializability

Definition: A concurrent schedule is called **conflict serializable** if it can be transformed into a serial schedule by swapping non-conflicting operations. Two operations are said to be in conflict if they satisfy all of the following three conditions:

- They belong to different transactions.
- They operate on the same data item.
- At least one of the operations is a **WRITE** operation.

Explanation: The possible conflicting pairs are:

- Read-Write (RW) Conflict:** A transaction reads a data item that is subsequently written by another transaction.
- Write-Read (WR) Conflict:** A transaction writes a data item that is subsequently read by another transaction (creates dirty reads if uncommitted).
- Write-Write (WW) Conflict:** Two transactions write to the same data item (creates blind writes).

If a schedule's conflicting operations occur in the same order as they would in some serial schedule, the schedule is conflict serializable.

—

2. Given Transactions and Concurrent Schedule

We are given the following two transactions:

- T_1 : READ(A), READ(B), COMMIT
- T_2 : READ(A), WRITE(A), COMMIT

Let us define a **concurrent schedule (S)** where operations from T_1 and T_2 are interleaved.

Time	Transaction T_1	Transaction T_2
t_1	READ(A)	
t_2		READ(A)
t_3	READ(B)	
t_4		WRITE(A)
t_5	COMMIT	
t_6		COMMIT

Conflict Analysis for Schedule S:

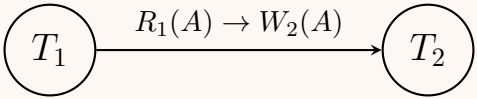
- T_1 performs READ(A) at time t_1 .
- T_2 performs WRITE(A) at time t_4 .
- This is a **Read-Write (RW) conflict** on data item A . Since T_1 's operation occurs before T_2 's operation, there is a dependency: $T_1 \rightarrow T_2$.
- T_1 reads B , but T_2 does not access B . No conflict.
- T_2 reads A at t_2 , but T_1 only reads A (no write). Two reads do not conflict.

3. Precedence Graph (Serialization Graph)

The precedence graph $G = (V, E)$ is constructed as follows:

- **Vertices (V):** The transactions $\{T_1, T_2\}$.
- **Edges (E):** A directed edge from T_i to T_j if an operation in T_i conflicts with an operation in T_j and T_i 's operation executes first.

Based on our schedule S , there is only one directed edge from T_1 to T_2 due to the $R_1(A) \rightarrow W_2(A)$ conflict.



4. Proof of Serializability

Theorem: A schedule is conflict serializable if and only if its precedence graph contains no directed cycles.
Proof based on the Graph:

- (a) We constructed the precedence graph for the concurrent schedule S .
- (b) The graph contains the nodes T_1 and T_2 , and exactly one directed edge $T_1 \rightarrow T_2$.
- (c) There are no edges originating from T_2 and going back to T_1 .
- (d) Therefore, the precedence graph is **acyclic** (contains no cycles).
- (e) Because the graph is acyclic, the schedule S is **conflict serializable**.

5. Equivalent Serial Schedule

To find the equivalent serial schedule, we perform a **topological sort** of the acyclic precedence graph.

- Node T_1 has an in-degree of 0 (no dependencies). It must execute first.
- Node T_2 depends on T_1 . It must execute second.

Thus, the equivalent serial schedule is:

$$T_1 \rightarrow T_2$$

In this serial schedule, T_1 completely finishes all of its operations (READ(A), READ(B), COMMIT) before T_2 starts its operations (READ(A), WRITE(A), COMMIT). This guarantees the exact same outcome as our concurrent schedule S .

Q23. The hash values of data items A , B and C are 5, 1 and 8 respectively. Show and explain the lock table entries for the following

instructions with transactions:

$$\{READ(A), T_1\}, \{READ(B), T_2\}, \{READ(C), T_1\}, \{READ(A), T_2\},$$
$$\{READ(B), T_1\}, \{READ(C), T_2\}, \{WRITE(A), T_3\}, \{WRITE(A), T_4\},$$
$$\{WRITE(B), T_3\}, \{WRITE(C), T_1\}$$

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

1. Working Principle of a Lock Table

A **Lock Table** is an in-memory data structure managed by the database system’s Lock Manager to record and control current lock states on data items.

- **Hash-Based Access:** The data item identifier is hashed to find the corresponding bucket in the lock table directory.
- **Data Item Linked List:** Each bucket contains a linked list of active data items that map to that hash value (handling collisions).
- **Lock Request Queue:** Each data item points to a queue (linked list) of lock requests.
- **Lock Types:** Requests can be **Shared/Read (S)** or **Exclusive/Write (X)**.
- **Status:** A request is either **Granted** (if compatible with currently held locks) or **Waiting** (queued until conflicting locks are released).

2. Step-by-Step Execution and Lock Table State

We evaluate the given sequence of instructions. The hash values are: $H(A) = 5, H(B) = 1, H(C) = 8$. Recall that **Shared (Read)** locks are compatible with other Shared locks, whereas **Exclusive (Write)** locks conflict with any other lock (Shared or Exclusive).

Step	Instruction	Hash & Item	Action / State Change in Lock Table
1	$\{READ(A), T_1\}$	Hash 5 $\rightarrow$ A	Add Shared lock for T_1 . Granted .
2	$\{READ(B), T_2\}$	Hash 1 $\rightarrow$ B	Add Shared lock for T_2 . Granted .
3	$\{READ(C), T_1\}$	Hash 8 $\rightarrow$ C	Add Shared lock for T_1 . Granted .
4	$\{READ(A), T_2\}$	Hash 5 $\rightarrow$ A	Add Shared lock for T_2 . Granted (S-locks are compatible).
5	$\{READ(B), T_1\}$	Hash 1 $\rightarrow$ B	Add Shared lock for T_1 . Granted (S-locks are compatible).
6	$\{READ(C), T_2\}$	Hash 8 $\rightarrow$ C	Add Shared lock for T_2 . Granted (S-locks are compatible).
7	$\{WRITE(A), T_3\}$	Hash 5 $\rightarrow$ A	Add Exclusive lock for T_3 . Waiting (Conflicts with T_1, T_2).
8	$\{WRITE(A), T_4\}$	Hash 5 $\rightarrow$ A	Add Exclusive lock for T_4 . Waiting (Queued behind T_3).
9	$\{WRITE(B), T_3\}$	Hash 1 $\rightarrow$ B	Add Exclusive lock for T_3 . Waiting (Conflicts with T_2, T_1).
10	$\{WRITE(C), T_1\}$	Hash 8 $\rightarrow$ C	T_1 requests Exclusive lock (Upgrade). Waiting (Conflicts with T_2).

Note on Step 10: Transaction T_1 already holds a Shared (Read) lock on C and requests an Exclusive (Write) lock on C . This is a *lock upgrade request*. However, because T_2 currently holds a Shared lock on C , the upgrade cannot be immediately granted. T_1 must wait for T_2 to release its lock.

3. Final Lock Table Diagram

The following diagram represents the internal state of the Lock Table after all 10 instructions have been processed by the Lock Manager.

Hash Directory

Lock Table Queues

Index 1

Index 2

Index 3

Index 4

Index 5

Index 6

Index 7

Index 8

Item B

Item A

Item C

T2
Read
Granted

T1
Read
Granted

T3
Write
Waiting

T1
Read
Granted

T2
Read
Granted

T3
Write
Waiting

T4
Write
Waiting

T1
Read
Granted

T2
Read
Granted

T1
Write
Waiting

401

Q24. Leonard, Howard, and Raj concurrently hacked a theatre seat reservation system. Leonard read all reservations for the matinee show, made changes and saved. Howard read the same show's reservations and did the same. Leonard then read the night show and made changes. Raj changed the morning show reservations and saved. Howard repeated the same for the same night show. (Different relations exist for morning, matinee and night shows.)

- Construct the schedule of actions using standard notations.
- What is a conflict serializable schedule? Show whether the above schedule is conflict serializable without using a Precedence graph.
- Show whether the schedule is conflict serializable using a Precedence graph. If not, briefly describe the reason. If yes, determine the serializability order using topological sort.

(21 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer**1. Construction of the Schedule**

First, let us formalize the transactions, data items, and operations based on the problem description.

- Data Items:** Let M represent the Matinee show relation, N represent the Night show relation, and O represent the Morning show relation.
- Transactions:**
 - T_1 (Leonard): Reads and writes M , then reads and writes N .
 - T_2 (Howard): Reads and writes M , then reads and writes N .
 - T_3 (Raj): Reads and writes O .

Based on the sequence of events provided, we construct the schedule S using standard notations:

Time	T_1 (Leonard)	T_2 (Howard)	T_3 (Raj)
t_1	$R_1(M)$		
t_2	$W_1(M)$		
t_3		$R_2(M)$	
t_4		$W_2(M)$	
t_5	$R_1(N)$		
t_6	$W_1(N)$		
t_7			$R_3(O)$
t_8			$W_3(O)$
t_9		$R_2(N)$	
t_{10}		$W_2(N)$	

The schedule notation is:

$$S : R_1(M), W_1(M), R_2(M), W_2(M), R_1(N), W_1(N), R_3(O), W_3(O), R_2(N), W_2(N)$$

2. Conflict Serializability without Precedence Graph

Definition: A schedule is **conflict serializable** if it can be transformed into a serial schedule by swapping non-conflicting adjacent operations. Two operations conflict if they belong to different transactions, access the same data item, and at least one is a Write (W) operation.

Proof via Swapping: To check if schedule S is conflict serializable without a graph, we identify non-conflicting operations and swap them to form a strict serial execution sequence.

Operations on different data items (M, N, O) do not conflict. Thus, we can safely reorder operations across different items:

- Current sequence involves T_2 accessing M before T_1 accesses N . Since M and N are separate data items, $\{R_2(M), W_2(M)\}$ does not conflict with $\{R_1(N), W_1(N)\}$. We swap them.

New sequence: $R_1(M), W_1(M), R_1(N), W_1(N), R_2(M), W_2(M), R_3(O), W_3(O), R_2(N), W_2(N)$

- Now, all operations of T_1 are grouped together at the beginning.

- Next, we look at T_3 's operations: $\{R_3(O), W_3(O)\}$. They operate on O , which neither T_1 nor T_2 accesses. Thus, we can safely swap T_3 's operations with T_2 's operations $\{R_2(M), W_2(M)\}$.

New sequence: $R_1(M), W_1(M), R_1(N), W_1(N), R_3(O), W_3(O), R_2(M), W_2(M), R_2(N), W_2(N)$

The final sequence contains all of T_1 , followed by all of T_3 , followed by all of T_2 . This strictly matches the serial schedule $T_1 \rightarrow T_3 \rightarrow T_2$. Because we reached a serial schedule purely by swapping non-conflicting operations, the schedule is **conflict serializable**.

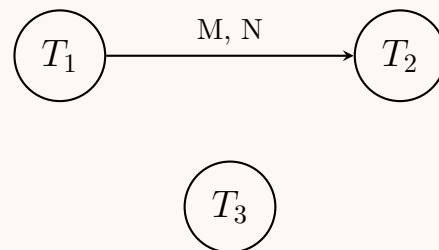
3. Conflict Serializability using Precedence Graph

A schedule is conflict serializable if and only if its precedence graph (serialization graph) is **acyclic**.

Identifying Conflicts:

- On item M : $W_1(M) \rightarrow R_2(M)$ and $W_1(M) \rightarrow W_2(M)$. Dependency: $T_1 \rightarrow T_2$.
- On item N : $W_1(N) \rightarrow R_2(N)$ and $W_1(N) \rightarrow W_2(N)$. Dependency: $T_1 \rightarrow T_2$.
- On item O : Only T_3 accesses O . No dependency.

Precedence Graph Construction:



Conclusion and Topological Sort:

- The precedence graph contains no cycles (it is **acyclic**). Therefore, the schedule is **conflict serializable**.
- **Topological Sort:** To find a valid serializability order, we compute the in-degrees of the nodes:
 - $In(T_1) = 0$
 - $In(T_2) = 1$ (depends on T_1)
 - $In(T_3) = 0$

Nodes with an in-degree of 0 can be executed in any order. T_1 must precede T_2 . Therefore, the valid serial schedules (serializability orders) are:

$$\mathbf{T_1 \rightarrow T_2 \rightarrow T_3 \quad \text{or} \quad T_1 \rightarrow T_3 \rightarrow T_2 \quad \text{or} \quad T_3 \rightarrow T_1 \rightarrow T_2}$$

Q25. “Two-phase locking is efficient for B⁺ tree index structure” — Justify this statement using illustrative examples. (7 marks) [CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Critical Evaluation of the Statement

The statement “Two-phase locking (2PL) is efficient for B⁺ tree index structures” is actually a common **misconception** in database design. In reality, applying standard Two-Phase Locking directly to the nodes of a B⁺ tree is **highly inefficient**.

To ground this in reality: Modern Database Management Systems (DBMS) explicitly **avoid** using standard 2PL for index structures. Instead, they use standard 2PL for the actual *data records* but employ specialized **Tree-Based Locking Protocols** (such as **Lock Coupling** or **Crabbing**) for the B⁺ tree index pages.

2. Why Standard 2PL is Inefficient for B⁺ Trees (The Root Bottleneck)

Working Principle of 2PL on a Tree: In standard 2PL, a transaction must acquire locks in a growing phase and cannot release any locks until it reaches the shrinking phase (usually at the end of the transaction).

The Bottleneck:

- Every search, insertion, or deletion in a B⁺ tree must begin at the **root node**.
- If a transaction T_1 wants to insert a record, under strict 2PL, it would acquire an Exclusive (X) lock on the root and hold it until the transaction commits.
- Consequently, **no other transaction** can even begin to traverse the tree while T_1 is executing, reducing the concurrency of the entire database to a purely serial execution.

3. The Efficient Alternative: Lock-Coupling (Crabbing)

To achieve efficiency, databases use a technique called **Lock-Coupling** (or **Crabbing**), which intentionally **violates** 2PL for the index nodes but guarantees structural consistency.

Working Principle of Crabbing (For Search):

- Lock the root node.
- Traverse to the child node and acquire a lock on the child.
- Release the lock on the parent node** immediately (violating 2PL’s requirement to hold locks).

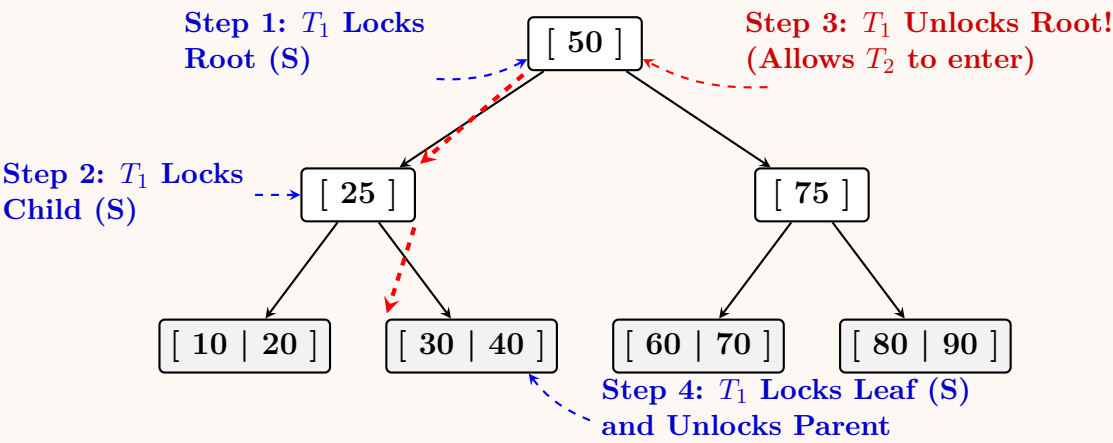
(d) Repeat until the leaf node is reached.

Working Principle of Crabbing (For Insert/Delete):

- (a) Lock the parent node.
- (b) Lock the child node.
- (c) Check if the child is **Safe** (i.e., it has enough room so it will not split on insert, or is more than half full so it will not merge on delete).
- (d) If the child is safe, **release all locks on ancestors**.
- (e) If it is not safe, retain the ancestor locks, as a split/merge might propagate up to them.

4. Illustrative Example & Visual Representation

Consider a B⁺ tree where Transaction *T*₁ is searching for the value **30**. The diagram below illustrates how Lock-Coupling operates efficiently by releasing locks early, allowing a concurrent Transaction *T*₂ to immediately follow without waiting for *T*₁ to finish.



5. Summary Comparison

Feature	Standard 2PL on B ⁺ Tree	Lock-Coupling (Crabbing)
Lock Release	Held until commit (Shrinking Phase)	Released immediately when child is safe
Concurrency Level	Extremely Low (Root bottleneck)	Extremely High (Pipelined access)
Compliance with 2PL	Strict compliance	Explicitly violates 2PL for index nodes
Real-World Usage	Never used for tree navigation	Standard practice in all major DBMS

Conclusion: The statement is **incorrect**. Standard Two-Phase Locking causes severe concurrency bottlenecks on B⁺ trees because the root node acts as a choke point. True efficiency in B⁺ trees is achieved precisely by *abandoning* 2PL during index traversal in favor of localized tree-locking protocols like Crabbing.

Q26. Discuss three problems of non-serializable execution with necessary examples. (9 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Introduction to Non-Serializable Execution

In a Database Management System, a **non-serializable execution** (or schedule) is a concurrent execution of multiple transactions that is not equivalent to any strict serial execution of those transactions. Such executions can break transaction isolation and lead to database inconsistencies.

The three primary concurrency problems that arise from non-serializable schedules are the **Lost Update Problem**, the **Dirty Read Problem**, and the **Unrepeatable Read Problem**.

2. The Lost Update Problem (Write-Write Conflict)

Definition: The lost update problem occurs when two transactions read the same data item, and both subsequently update it. The update performed by the first transaction is overwritten by the second transaction without being taken into account, causing the first update to be permanently “lost.”

Explanation & Example: Assume a bank account *A* has an initial balance of \$1000. Transaction *T*₁ wants to deduct \$100, and

Transaction T_2 wants to add \$500.

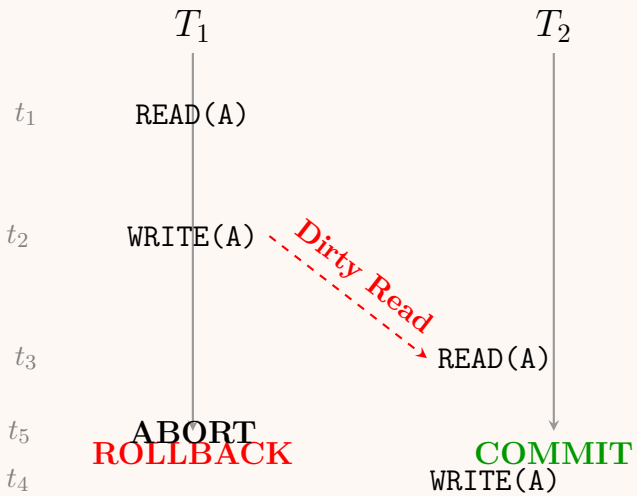
Time	Transaction T_1	Transaction T_2	Value of A in DB
t_1	READ(A) [Reads 1000]		1000
t_2		READ(A) [Reads 1000]	1000
t_3	$A = A - 100$		1000
t_4	WRITE(A) [Writes 900]	$A = A + 500$	900
t_5		WRITE(A) [Writes 1500]	1500 (Overwrites 900)

Result: The final balance is \$1500. T_1 's deduction of \$100 is completely lost. The correct isolated result should have been \$1400.

3. The Dirty Read Problem (Write-Read Conflict)

Definition: Also known as the **Uncommitted Dependency Problem**, a dirty read occurs when a transaction reads a data item that has been updated by another transaction that has *not yet committed*. If the updating transaction fails and rolls back, the reading transaction has operated on invalid data.

Visual Representation of a Dirty Read:



Result: At time t_3 , T_2 reads the value of A written by T_1 . However, at t_5 , T_1 aborts and the system restores the old value of A . T_2 has now performed its processing based on a value of A that officially never existed in the database.

4. The Unrepeatable Read Problem (Read-Write Conflict)

Definition: Also known as the **Inconsistent Retrievals Problem**, this occurs when a transaction reads the same data item multiple times, but another transaction modifies that item between the reads. As a result, the first transaction sees different values for the exact same query within a single execution block.

Explanation & Example: Assume T_1 is checking the available seats on a flight twice (e.g., first to display to the user, second to confirm before booking). T_2 is another transaction booking a seat.

Time	Transaction T_1	Transaction T_2	Seats Available
t_1	READ(Seats) [Reads 10]		10
t_2		READ(Seats) [Reads 10]	10
t_3		WRITE(Seats) [Writes 9]	9
t_4		COMMIT	9
t_5	READ(Seats) [Reads 9]		9

Result: T_1 reads the number of available seats at t_1 and finds 10. At t_5 , it reads the *exact same variable* and finds 9. The read operation is “unrepeatable,” leading to inconsistent logic (e.g., an application throwing an error because it expected the seat count to remain stable during its process).

Q27. Consider the schedule S : $r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B)$. Show that there is no serial schedule conflict-equivalent to S . (7 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Introduction and Principle

A concurrent schedule is **conflict-equivalent** to a serial schedule (i.e., it is conflict serializable) if and only if its corresponding **precedence graph** (serialization graph) is **acyclic**. If the precedence graph contains a cycle, the transactions are circularly

dependent, making it impossible to find any equivalent serial execution order.

To prove that there is no serial schedule conflict-equivalent to the given schedule S , we must construct its precedence graph and demonstrate the presence of a cycle.

2. Analysis of the Schedule S

The schedule S involves three transactions (T_1, T_2, T_3) operating on two data items (A, B) . The execution timeline is mapped as follows:

Time	Transaction T_1	Transaction T_2	Transaction T_3
t_1		$r_2(A)$	
t_2	$r_1(B)$		
t_3		$w_2(A)$	
t_4		$r_2(B)$	
t_5			$r_3(A)$
t_6	$w_1(B)$		
t_7			$w_3(A)$
t_8		$w_2(B)$	

3. Identification of Conflicting Operations

Two operations are in conflict if they belong to different transactions, access the same data item, and at least one is a Write (w) operation. We identify the conflicts in chronological order to determine the dependencies:

Conflicts on Data Item A:

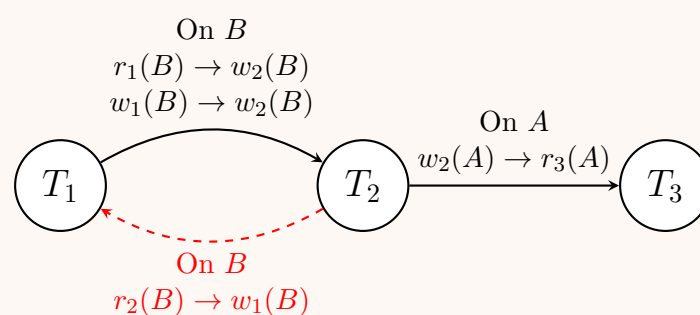
- $r_2(A)$ at t_1 is followed by $w_3(A)$ at t_7 . (Read-Write Conflict) $\implies \mathbf{T_2 \rightarrow T_3}$
- $w_2(A)$ at t_3 is followed by $r_3(A)$ at t_5 . (Write-Read Conflict) $\implies \mathbf{T_2 \rightarrow T_3}$
- $w_2(A)$ at t_3 is followed by $w_3(A)$ at t_7 . (Write-Write Conflict) $\implies \mathbf{T_2 \rightarrow T_3}$

Conflicts on Data Item B:

- $r_1(B)$ at t_2 is followed by $w_2(B)$ at t_8 . (Read-Write Conflict) $\implies \mathbf{T_1 \rightarrow T_2}$
- $r_2(B)$ at t_4 is followed by $w_1(B)$ at t_6 . (Read-Write Conflict) $\implies \mathbf{T_2 \rightarrow T_1}$
- $w_1(B)$ at t_6 is followed by $w_2(B)$ at t_8 . (Write-Write Conflict) $\implies \mathbf{T_1 \rightarrow T_2}$

4. Precedence Graph Construction

Using the dependencies derived above, we construct the directed precedence graph $G = (V, E)$, where $V = \{T_1, T_2, T_3\}$ and the edges E represent the conflicting dependencies.



5. Conclusion

- Based on the conflicting operations on data item B , Transaction T_2 must read B before T_1 writes it ($T_2 \rightarrow T_1$).
- However, simultaneously, T_1 reads and writes B before T_2 writes it ($T_1 \rightarrow T_2$).
- This mutual dependency forms a **cycle** ($T_1 \rightarrow T_2 \rightarrow T_1$) in the precedence graph, highlighted in the diagram above.

Since the precedence graph is **cyclic**, the schedule S is **not conflict serializable**. Therefore, **there is no serial schedule conflict-equivalent to S** .

Q28. Mention the requirements of using shared and exclusive locks. (6 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Introduction to Lock-Based Protocols

In Database Management Systems (DBMS), concurrency control mechanisms utilize **lock-based protocols** to ensure transaction isolation, maintain data consistency, and prevent conflicts such as lost updates or dirty reads. The implementation of these protocols strictly requires the use of two fundamental lock modes: **Shared Locks** and **Exclusive Locks**.

2. Requirements of Shared Locks (S-Locks)

A Shared Lock is primarily designed to facilitate read operations while maximizing concurrency.

- **Operation Requirement:** A transaction T_i **must** explicitly request and acquire a Shared Lock, denoted as $\text{Lock-S}(Q)$, on a data item Q before it is permitted to execute a $\text{READ}(Q)$ operation.
- **Access Privileges:** It grants the transaction **read-only** access. The transaction is strictly prohibited from modifying Q while holding only an S-Lock.
- **Concurrency & Compatibility:** Multiple transactions can hold shared locks on the identical data item simultaneously. If transaction T_j currently holds an S-Lock on Q , a new request by T_i for an S-Lock on Q will be immediately **granted**.

3. Requirements of Exclusive Locks (X-Locks)

An Exclusive Lock is designed to provide a transaction with sole, unhindered access to a data item for modification.

- **Operation Requirement:** A transaction T_i **must** explicitly request and acquire an Exclusive Lock, denoted as $\text{Lock-X}(Q)$, on a data item Q before it is permitted to execute a $\text{WRITE}(Q)$ operation. (Note: An X-Lock also inherently satisfies the requirement for a $\text{READ}(Q)$ operation).
- **Access Privileges:** It grants the transaction comprehensive **read-and-write** access to the data item.
- **Concurrency & Incompatibility:** An exclusive lock cannot be shared with any other transaction. If *any* transaction T_j holds *any* lock (either Shared or Exclusive) on Q , a request by T_i for an X-Lock will be **denied and blocked**. T_i must wait until all existing locks on Q are released.

4. Lock Compatibility Matrix

The DBMS Lock Manager evaluates the requirements for granting locks based on a strict compatibility matrix. A “✓” indicates the requested lock requirement is met and can be granted, while an “×” indicates a conflict (the requesting transaction must wait).

Currently Held Lock on Item Q	Requested Lock Mode	
	Shared (S)	Exclusive (X)
None	✓ (Granted)	✓ (Granted)
Shared (S)	✓ (Granted)	× (Wait)
Exclusive (X)	× (Wait)	× (Wait)

5. General Transactional Requirements

To function correctly within a concurrency model (such as **Two-Phase Locking (2PL)**), the usage of these locks imposes additional structural requirements on all transactions:

- (a) **Mandatory Acquisition:** A transaction must secure the appropriate lock state before executing any operation on the data. Bypassing the Lock Manager is strictly prohibited.
- (b) **Mandatory Release:** A transaction must eventually release all acquired locks using an $\text{UNLOCK}(Q)$ instruction to allow blocked/waiting transactions to proceed.
- (c) **Lock Upgrades:** If a transaction T_i holds an S-Lock on Q and needs to write to Q , it must request a **Lock Upgrade** to an X-Lock. This upgrade requirement can only be fulfilled if no other transactions currently hold S-Locks on Q .
- (d) **Lock Downgrades:** A transaction holding an X-Lock may reduce its isolation footprint by fulfilling a **Lock Downgrade** requirement, converting its X-Lock to an S-Lock once writing is complete, thus allowing other readers to access the data item earlier.

Q29. Consider the following schedule:

$$r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D)$$

Insert shared and exclusive locks and place necessary unlocks to maximise concurrency. (9 marks) [CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Locking Strategy to Maximize Concurrency

To insert locks and maximize concurrency for the given schedule, we must adhere to the following principles:

- Shared Locks (Lock-S):** Acquired before a READ (r) operation. Multiple transactions can hold a Shared lock on the same item.
- Exclusive Locks (Lock-X):** Acquired before a WRITE (w) operation. No other locks can be held by other transactions on the item.
- Immediate Release:** To strictly **maximize concurrency**, a transaction should release its lock (Unlock) immediately after the operation is complete. This ensures that subsequent transactions waiting to access the same data item are not blocked unnecessarily.

Conflict Analysis in the Schedule:

- T_2 reads B , and later T_1 writes B . T_2 must release its Shared lock on B before T_1 can acquire its Exclusive lock on B .
- T_3 reads C , and later T_2 writes C . T_3 must release its Shared lock on C before T_2 can acquire its Exclusive lock on C .

2. Annotated Schedule with Locks and Unlocks

The table below illustrates the chronological execution of the schedule, integrating the necessary lock and unlock instructions.

Time	Transaction T_1	Transaction T_2	Transaction T_3
t_1	Lock-S(A)		
t_2	$r_1(A)$		
t_3	Unlock(A)		
t_4		Lock-S(B)	
t_5		$r_2(B)$	
t_6		Unlock(B)	
t_7			Lock-S(C)
t_8			$r_3(C)$
t_9			Unlock(C)
t_{10}	Lock-X(B)		
t_{11}	$w_1(B)$		
t_{12}	Unlock(B)		
t_{13}		Lock-X(C)	
t_{14}		$w_2(C)$	
t_{15}		Unlock(C)	
t_{16}			Lock-X(D)
t_{17}			$w_3(D)$
t_{18}			Unlock(D)

3. Important Note on Concurrency vs. Two-Phase Locking (2PL)

- Optimal Concurrency:** The sequence presented above represents the absolute maximum concurrency possible for this specific sequence of operations. No transaction is ever placed in a "wait" or "blocked" state because locks are surrendered immediately.
- Violation of 2PL:** It is important to note that releasing locks immediately after use **violates the Two-Phase Locking (2PL) protocol**. Under strict 2PL, a transaction cannot release any lock until it has acquired all the locks it will ever need (it cannot shrink while still growing).
- If this schedule were forced to strictly obey 2PL, T_2 would hold its lock on B until it acquired its lock on C , which would forcefully block T_1 's write operation on B , thereby *reducing* concurrency and altering the execution order of the given schedule. Therefore, to fulfill the requirement of maximizing concurrency for the exact order given, the non-2PL early-release strategy must be applied.

Q30. Find the equivalent serial schedule for the concurrent schedule of four transactions shown below by swapping non-conflicting instructions. Verify the resulting serial schedule using a precedence graph.

408

T1	T2	T3	T4
READ(A)			
	READ (C)		
READ(B)			
	WRITE (C)		
		READ(C)	
		WRITE (C)	
			READ(B)
			READ(A)
			READ(D)
WRITE (A)			
			WRITE (D)
READ(B)			
		READ(B)	

Figure: Concurrent schedule of four transactions

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Conflict Analysis and Swapping Rules

To find an equivalent serial schedule, we must transform the concurrent schedule by swapping adjacent **non-conflicting** operations. Two operations can be safely swapped if they satisfy any of the following conditions:

- They belong to the same transaction (not swapped, order must be preserved).
- They operate on **different** data items.
- They operate on the same data item, but **both are READ** operations.

A **conflict** occurs only when two operations from different transactions access the same data item, and at least one of them is a WRITE operation.

Let us identify the conflicts in the given schedule to determine which operations *cannot* be swapped:

- **On Data Item A:** T_4 reads A before T_1 writes A . (Conflict: $T_4 \rightarrow T_1$)
- **On Data Item B:** Only READ operations exist (T_1, T_3, T_4). No writes mean **no conflicts** on B .
- **On Data Item C:** T_2 writes C before T_3 reads and writes C . (Conflict: $T_2 \rightarrow T_3$)
- **On Data Item D:** Only T_4 accesses D . **No conflicts**.

2. Deriving the Equivalent Serial Schedule via Swapping

Based on the identified conflicts, T_4 must execute before T_1 , and T_2 must execute before T_3 . Any serial schedule satisfying these two independent conditions (e.g., $T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$) is valid.

Let us logically bubble the operations to form the serial schedule $T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$:

(a) Group T_2 at the top:

- Swap $READ_2(C)$ upwards past $READ_1(A)$ (different data items).
- Swap $WRITE_2(C)$ upwards past $READ_1(B)$ and $READ_1(A)$ (different data items).
- T_2 is now complete at the beginning.

(b) Group T_3 next:

- Swap $READ_3(C)$ and $WRITE_3(C)$ upwards past T_1 's operations (different items).
- Swap $READ_3(B)$ from the very bottom upwards past T_1 and T_4 's operations. This is valid because it only crosses operations on different items (A, D) or other READs on B (which do not conflict).
- T_3 is now contiguous following T_2 .

(c) Group T_4 next:

- Swap all operations of T_4 ($READ_4(B), READ_4(A), READ_4(D), WRITE_4(D)$) upwards past T_1 's operations.
- Note: $READ_4(A)$ safely swaps past $READ_1(A)$ (both reads), but it **cannot** swap past $WRITE_1(A)$. Since $READ_4(A)$ originally precedes $WRITE_1(A)$, the swap is valid and naturally places T_4 before T_1 .

(d) Group T_1 at the bottom:

- The remaining operations ($\text{READ}_1(A)$, $\text{READ}_1(B)$, $\text{WRITE}_1(A)$, $\text{READ}_1(B)$) naturally fall to the end, preserving their internal order.

Equivalent Serial Schedule:

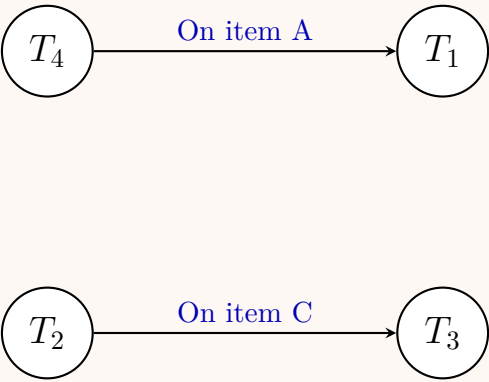
T1	T2	T3	T4
	READ(C)		
	WRITE(C)		
		READ(C)	
		WRITE(C)	
		READ(B)	
			READ(B)
			READ(A)
			READ(D)
			WRITE(D)
READ(A)			
READ(B)			
WRITE(A)			
READ(B)			

—

3. Verification Using Precedence Graph

To formally verify that the schedule is conflict serializable, we construct the Precedence (Serialization) Graph based on the conflicts found in Step 1.

- **Nodes:** The four transactions (T_1, T_2, T_3, T_4).
- **Edges:** Directed edges denote dependency (conflict).
 - Edge $T_4 \rightarrow T_1$ because of Read-Write conflict on A .
 - Edge $T_2 \rightarrow T_3$ because of Write-Read and Write-Write conflicts on C .



Conclusion: The resulting precedence graph contains **no cycles** (it is acyclic). Therefore, the concurrent schedule is indeed **conflict serializable**. Applying a topological sort to the graph validates that any order preserving $T_4 \rightarrow T_1$ and $T_2 \rightarrow T_3$ is a valid equivalent serial schedule, confirming our derived schedule $T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$ is correct.

Q31. Create a lock table for the following list of data items and their locking status:

Data Item	Hash Value	Transactions with Lock	Transactions Waiting
A	5	T1, T5	T2
S	3	T1, T2	—
D	5	—	T3
F	2	T2	T5
G	2	T2, T3, T4	T1
H	5	T4	—

Answer

1. Introduction to Lock Tables

In a Database Management System (DBMS), the **Lock Manager** uses a data structure called a **Lock Table** to keep track of the locks issued to various transactions.

- **Working Principle:** The lock table is implemented as an **in-memory hash table**. The data item's name is hashed to find the corresponding bucket.
- **Collision Handling:** If multiple data items hash to the same bucket (collision), they are linked together using a linked list (separate chaining).
- **Lock Control Block:** Each node in the linked list represents a data item and contains the list of transactions that currently hold a lock (**Granted**) and the list of transactions waiting to acquire a lock (**Waiting**).

2. Lock Table Diagram

Based on the provided data items and their hash values, we can construct the corresponding lock table. The hash directory maps to buckets 0 through 5 (assuming a standard array implementation), with buckets 2, 3, and 5 pointing to linked lists of lock control blocks.

Hash Index

0

→ ∅

1

→ ∅

2

→

Item F

Granted: T2

Waiting: T5

 →

Item G

Granted: T2, T3, T4

Waiting: T1

3

→

Item S

Granted: T1, T2

Waiting: —

4

→ ∅

5

→

Item A

Granted: T1, T5

Waiting: T2

 →

Item D

Granted: —

Waiting: T3

 →

Item H

Granted: T4

Waiting: —

3. Tabular Representation of the Hash Chains

The logical grouping of the data items resulting from the hash function mapping is summarized below:

Hash Bucket	Data Item in Chain	Granted Locks	Waiting Queue
2	F	T2	T5
	G	T2, T3, T4	T1
3	S	T1, T2	—
5	A	T1, T5	T2
	D	—	T3
	H	T4	—

Note: When a transaction requests a lock (e.g., on item **G**), the Lock Manager computes the hash (2), traverses the linked list at bucket 2 to locate the control block for **G**, and then applies the lock compatibility matrix to determine whether the lock can be added to the *Granted* list or if the transaction must be placed in the *Waiting* queue.

Q32. A set of transactions with their timestamps and instructions is given. The timestamp values of data item A are: W-timestamp(A)

411

= 10, $R\text{-timestamp}(A) = 20$. The transactions are:

Transaction	Timestamp	Instruction
T1	5	READ(A)
T2	15	READ(A)
T3	25	WRITE(A)
T4	12	WRITE(A)

Find the status (rollback or executed) of each transaction and the corresponding timestamp values of data item A using the timestamp-based protocol. **(15 marks)** [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Timestamp Ordering Protocol (TO) Principles

The **Timestamp Ordering Protocol** ensures serializability by ordering transactions based on their timestamp (TS). For any data item Q , the system maintains two timestamp values:

- **W – TS(Q)**: The largest timestamp of any transaction that successfully executed a $WRITE(Q)$.
- **R – TS(Q)**: The largest timestamp of any transaction that successfully executed a $READ(Q)$.

Basic TO Rules for a Transaction T_i with timestamp $TS(T_i)$:

(a) **READ Operation on Q :**

- If $TS(T_i) < W - TS(Q)$: T_i is trying to read an overwritten value. **Reject and Rollback.**
- If $TS(T_i) \geq W - TS(Q)$: The read is safe. **Execute**, and update $R - TS(Q) = \max(R - TS(Q), TS(T_i))$.

(b) **WRITE Operation on Q :**

- If $TS(T_i) < R - TS(Q)$: The value T_i is trying to write was previously needed and read by a younger transaction. **Reject and Rollback.**
- If $TS(T_i) < W - TS(Q)$: T_i is trying to write an obsolete value. **Reject and Rollback.**
- Otherwise: The write is safe. **Execute**, and update $W - TS(Q) = TS(T_i)$.

2. Step-by-Step Evaluation

We evaluate the instructions chronologically based on the initial state of Data Item A:

Initial State: $W - TS(A) = 10, \quad R - TS(A) = 20$

Step 1: Transaction T_1

- **Instruction:** READ(A)
- **Timestamp:** $TS(T_1) = 5$
- **Condition Check:** Is $TS(T_1) < W - TS(A)$? $\implies 5 < 10$. (**True**)
- **Action:** T_1 attempts to read a value that was already overwritten by a later transaction. T_1 is **Rolled back**.
- **Updated State:** $W - TS(A) = 10, R - TS(A) = 20$

Step 2: Transaction T_2

- **Instruction:** READ(A)
- **Timestamp:** $TS(T_2) = 15$
- **Condition Check:** Is $TS(T_2) < W - TS(A)$? $\implies 15 < 10$. (**False**)
- **Action:** The read operation is valid. T_2 is **Executed**.
- **Timestamp Update:** $R - TS(A) = \max(R - TS(A), TS(T_2)) = \max(20, 15) = 20$.
- **Updated State:** $W - TS(A) = 10, R - TS(A) = 20$

Step 3: Transaction T_3

- **Instruction:** WRITE(A)
- **Timestamp:** $TS(T_3) = 25$
- **Condition Check 1:** Is $TS(T_3) < R - TS(A)$? $\implies 25 < 20$. (**False**)
- **Condition Check 2:** Is $TS(T_3) < W - TS(A)$? $\implies 25 < 10$. (**False**)

412

- **Action:** The write operation is valid. T_3 is **Executed**.
- **Timestamp Update:** $W - TS(A) = TS(T_3) = 25$.
- **Updated State:** $W - TS(A) = 25, R - TS(A) = 20$

Step 4: Transaction T_4

- **Instruction:** WRITE(A)
- **Timestamp:** $TS(T_4) = 12$
- **Condition Check:** Is $TS(T_4) < R - TS(A)$? $\implies 12 < 20$. (**True**)
- **Action:** A younger transaction (timestamp ≥ 20) has already read the value of A . If T_4 writes now, it would invalidate that previous read. Therefore, T_4 is **Rolled back**.
- **Updated State:** $W - TS(A) = 25, R - TS(A) = 20$

3. Final Status Summary

The final status of each transaction and the resulting timestamp values of data item A after processing all given instructions are summarized in the table below:

Transaction	Timestamp	Operation	Final Status	State of A after execution
T_1	5	READ(A)	Rollback	$W - TS(A) = 10, R - TS(A) = 20$
T_2	15	READ(A)	Executed	$W - TS(A) = 10, R - TS(A) = 20$
T_3	25	WRITE(A)	Executed	$W - TS(A) = \mathbf{25}, R - TS(A) = 20$
T_4	12	WRITE(A)	Rollback	$W - TS(A) = 25, R - TS(A) = 20$

Final Timestamp Values for Data Item A:

- $W - TS(A) = 25$
- $R - TS(A) = 20$

Q33. Explain the Thomas Write Rule with an example. (5 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Introduction to the Thomas Write Rule

The **Thomas Write Rule (TWR)** is a concurrency control mechanism in Database Management Systems (DBMS) that modifies the basic Timestamp Ordering (TO) protocol. Named after its creator Robert H. Thomas, the rule is designed to provide greater concurrency by safely **ignoring obsolete write operations** rather than aborting and rolling back the transaction.

- **Primary Purpose:** To increase concurrency by accepting schedules that are **View Serializable** but not necessarily Conflict Serializable.
- **Core Concept:** It specifically handles **blind writes** (when a transaction writes a value without reading it first). If a transaction attempts to write an older value over a newer value, TWR simply ignores the older write instead of failing the transaction.

2. Working Principle

Let T_i be a transaction with timestamp $TS(T_i)$ attempting to execute a WRITE(Q) operation on data item Q . The system maintains two timestamps for Q : the read timestamp $R-TS(Q)$ and the write timestamp $W-TS(Q)$. The Thomas Write Rule evaluates the WRITE(Q) operation as follows:

- (a) **Condition 1 (Read by a Younger Transaction):** If $TS(T_i) < R-TS(Q)$
The value of Q that T_i is producing was previously needed by a younger transaction.
- **Action:** Reject the WRITE(Q) operation and **Rollback** T_i .
- (b) **Condition 2 (Obsolete Write / The Thomas Rule):** If $TS(T_i) < W-TS(Q)$
A younger transaction has already written a newer value to Q . T_i is attempting to write an obsolete value.
- **Action:** **Ignore** the WRITE(Q) operation and allow T_i to continue executing. (Basic TO would have rolled back T_i here).
- (c) **Condition 3 (Valid Write):** Otherwise (i.e., $TS(T_i) \geq R-TS(Q)$ and $TS(T_i) \geq W-TS(Q)$).

- **Action:** Execute the $\text{WRITE}(Q)$ operation and set $W\text{-}TS(Q) = TS(T_i)$.

3. Example Scenario

Consider two transactions, T_1 and T_2 , and a data item A .

- T_1 enters the system first: $TS(T_1) = 10$.
- T_2 enters the system later: $TS(T_2) = 20$.
- Initial timestamps for A : $R\text{-}TS(A) = 0$, $W\text{-}TS(A) = 0$.

Execution Sequence:

(a) T_2 executes $\text{WRITE}(A)$:

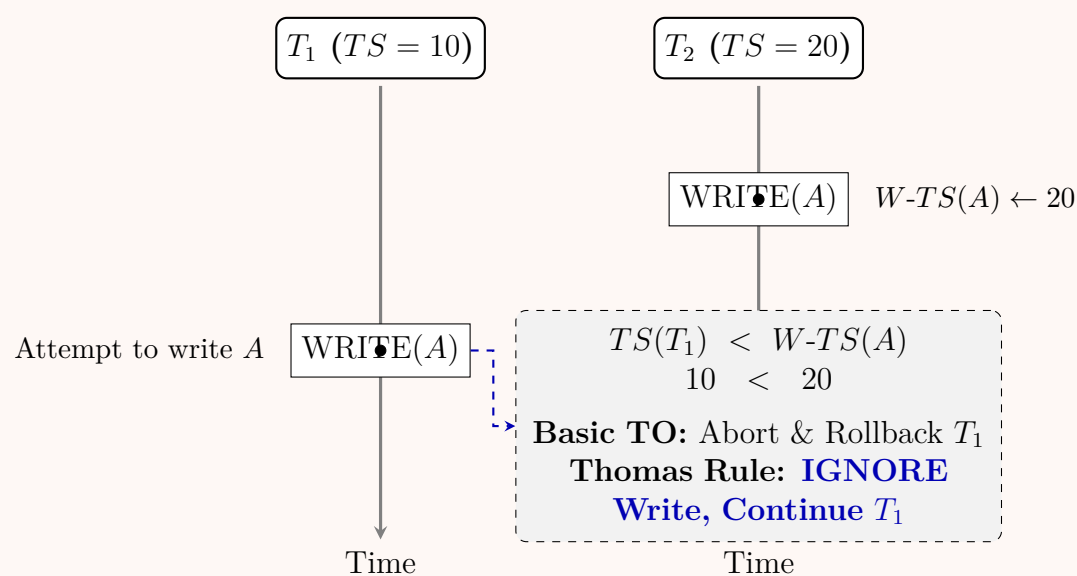
- $TS(T_2) = 20$. It is greater than both $R\text{-}TS(A)$ and $W\text{-}TS(A)$.
- The write is successful. $W\text{-}TS(A)$ is updated to **20**.

(b) T_1 executes $\text{WRITE}(A)$:

- $TS(T_1) = 10$.
- Since $TS(T_1) < W\text{-}TS(A)$ (i.e., $10 < 20$), this is an **obsolete write**.
- **Under Basic TO:** T_1 would be aborted and rolled back.
- **Under Thomas Write Rule:** The write is **ignored**. T_1 continues execution as if the write succeeded, but the database state remains unchanged.

4. Visualizing the Thomas Write Rule

The diagram below illustrates the timeline of the example, highlighting the difference between Basic TO and the Thomas Write Rule when encountering a blind write.



5. Key Properties and Advantages

- **Concurrency Improvement:** By avoiding unnecessary rollbacks for obsolete writes, system throughput is increased.
- **View Serializability:** The resulting schedule is logically equivalent to a serial execution (where T_1 executes before T_2 , and T_1 's write is immediately overwritten by T_2). Because the final state is identical to the serial schedule, it is **View Serializable**.
- **Non-Conflict Serializable:** Schedules generated using the Thomas Write Rule often violate conflict serializability because the read-write / write-write dependency graph will contain cycles, yet the database state remains perfectly consistent.

Q34. A concurrent schedule of transactions T_1 , T_2 and T_3 is given in Figure 5. Find the equivalent serial schedule by using conflict serializability. Show the steps.

Transaction T_1	Transaction T_2	Transaction T_3
READ (A)		
	WRITE (B)	
	READ (B)	
WRITE (A)		
		WRITE (C)
		READ (C)
		READ (B)
	READ (C)	
READ(B)		
		READ (A)

Figure 5: A concurrent schedule of transactions T_1 , T_2 and T_3 .

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Introduction to Conflict Serializability

To determine if a concurrent schedule is equivalent to a serial schedule using **conflict serializability**, we must identify all conflicting operations between different transactions and construct a **Precedence (or Serialization) Graph**.

- Two operations conflict if they belong to different transactions, access the same data item, and at least one is a **WRITE** operation.
- A schedule is conflict serializable **if and only if** its precedence graph is **acyclic** (contains no cycles).

2. Step-by-Step Conflict Analysis

Let us analyze the chronological sequence of operations in the given schedule to find dependencies (edges) for our precedence graph.

(a) Conflicts on Data Item A:

- Transaction T_1 executes **WRITE(A)** at time step 4.
- Transaction T_3 executes **READ(A)** at time step 10.
- Dependency:** A Write-Read conflict occurs. T_1 must precede T_3 . ($T_1 \rightarrow T_3$)

(b) Conflicts on Data Item B:

- Transaction T_2 executes **WRITE(B)** at time step 2.
- Transaction T_3 executes **READ(B)** at time step 7.
- Dependency:** A Write-Read conflict occurs. T_2 must precede T_3 . ($T_2 \rightarrow T_3$)
- Transaction T_1 executes **READ(B)** at time step 9.
- Dependency:** A Write-Read conflict occurs. T_2 must precede T_1 . ($T_2 \rightarrow T_1$)

(c) Conflicts on Data Item C:

- Transaction T_3 executes **WRITE(C)** at time step 5.
- Transaction T_2 executes **READ(C)** at time step 8.
- Dependency:** A Write-Read conflict occurs. T_3 must precede T_2 . ($T_3 \rightarrow T_2$)

3. Precedence Graph Construction

Using the dependencies derived above, we map the transactions as nodes and the conflicts as directed edges.

—

4. Final Conclusion

By observing the generated Precedence Graph, we can clearly identify a cycle between transaction T_2 and transaction T_3 :

$$T_2 \xrightarrow{\text{On B}} T_3 \xrightarrow{\text{On C}} T_2$$

Because the precedence graph contains a cycle, the schedule is **NOT conflict serializable**.

Result: Since the schedule violates the rules of conflict serializability, it is mathematically impossible to find an equivalent serial schedule using this method. The execution shown in Figure 5 will lead to inconsistent database states.

—

Q35. Show and explain the lock compatibility matrix with examples. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Definition and Working Principle

In a Database Management System (DBMS), concurrency control often relies on locking data items to ensure serializability. A **Lock Compatibility Matrix** is a foundational data structure used by the **Lock Manager** to determine whether a lock requested by one transaction can be safely granted on a data item that is already locked by another transaction.

- **Compatible:** If the requested lock mode can coexist with the currently held lock mode, the matrix returns **True (Yes)**, and the lock is granted.
- **Incompatible:** If the requested lock mode conflicts with the currently held lock mode, the matrix returns **False (No)**, and the requesting transaction must **wait (block)** until the existing lock is released.

—

2. The Basic Lock Compatibility Matrix

The most common locking protocol uses two fundamental lock modes:

(a) **Shared Lock (S):** Required to read a data item. Multiple transactions can hold a Shared lock on the same item simultaneously.

(b) **Exclusive Lock (X):** Required to write (update/delete) a data item. Only one transaction can hold an Exclusive lock on an item at any given time.

Currently Held Lock Mode	Requested Lock Mode	
	Shared (S)	Exclusive (X)
Shared (S)	Yes (Compatible)	No (Incompatible)
Exclusive (X)	No (Incompatible)	No (Incompatible)

—

3. Examples Based on the Basic Matrix

Example A: S-S Compatibility (Concurrent Reads)

- **Scenario:** Transaction T_1 is generating a report and holds a Shared (S) lock on the *Accounts* table. Transaction T_2 also wants to read the *Accounts* table to check a balance.
- **Matrix Check:** T_1 holds S, T_2 requests S. Matrix returns **Yes**.
- **Result:** T_2 is granted the lock. Both transactions read the data concurrently, maximizing system throughput without causing inconsistencies.

Example B: S-X Incompatibility (Read-Write Conflict)

- **Scenario:** T_1 holds a Shared (S) lock on the *Accounts* table (currently reading). T_3 wants to update a balance, thereby requesting an Exclusive (X) lock.
- **Matrix Check:** T_1 holds S, T_3 requests X. Matrix returns **No**.
- **Result:** T_3 is blocked. If T_3 were allowed to write while T_1 was reading, T_1 might read inconsistent or half-written data (Dirty Read / Unrepeatable Read).

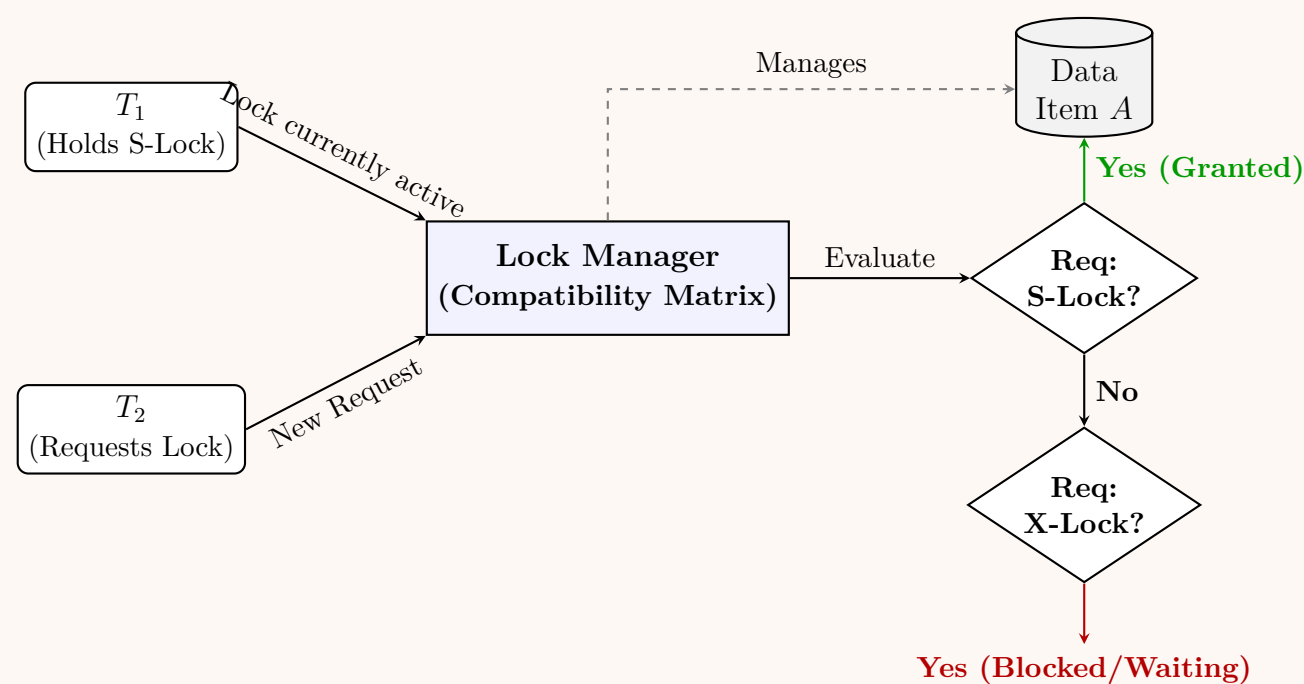
Example C: X-X Incompatibility (Write-Write Conflict)

- **Scenario:** T_4 is updating a record and holds an Exclusive (X) lock. T_5 attempts to update the exact same record and requests an X lock.
- **Matrix Check:** T_4 holds X, T_5 requests X. Matrix returns **No**.
- **Result:** T_5 is blocked. This prevents the lost update anomaly, ensuring only one transaction mutates a data item at a time.

—

4. Visual Representation of Lock Evaluation

The following diagram illustrates how the Lock Manager utilizes the compatibility matrix to process incoming requests.



5. Advanced Matrix: Multiple Granularity Locking (MGL)

In advanced databases, locking occurs at multiple levels (e.g., Database $\rightarrow$ Table $\rightarrow$ Page $\rightarrow$ Row). To efficiently support this, Intention Locks are used. The compatibility matrix is expanded to a 5×5 grid:

- **IS (Intention Shared):** Intent to acquire S locks at a lower level.
- **IX (Intention Exclusive):** Intent to acquire X locks at a lower level.
- **SIX (Shared Intention Exclusive):** Holds S lock at this level, but intends to acquire X locks at a lower level.

Currently Held Lock Mode	Requested Lock Mode				
	IS	IX	S	SIX	X
IS (Intention Shared)	Yes	Yes	Yes	Yes	No
IX (Intention Exclusive)	Yes	Yes	No	No	No
S (Shared)	Yes	No	Yes	No	No
SIX (Shared Intention X)	Yes	No	No	No	No
X (Exclusive)	No	No	No	No	No

Example from MGL Matrix: If Transaction T_A is updating rows in the *Employees* table, it holds an **IX** lock on the table itself. If Transaction T_B wants to read different rows in the same table, it requests an **IS** lock on the table. Checking the matrix (Held: IX, Requested: IS), the result is **Yes**. Both transactions can proceed to the row level to acquire their respective S and X locks, significantly boosting concurrency.

Q36. What are the tradeoffs of locking with multiple granularities? (5 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Introduction to Lock Granularity

In a Database Management System (DBMS), **granularity** refers to the size or level of the data item that is chosen to be locked during a transaction. A database can be viewed as a hierarchy of data items, ranging from the entire database down to a specific field in a record.

When designing a concurrency control subsystem, the Lock Manager must allow transactions to lock data at **multiple granularities** (e.g., locking an entire table vs. locking a single row).

2. The Fundamental Tradeoff: Concurrency vs. Locking Overhead

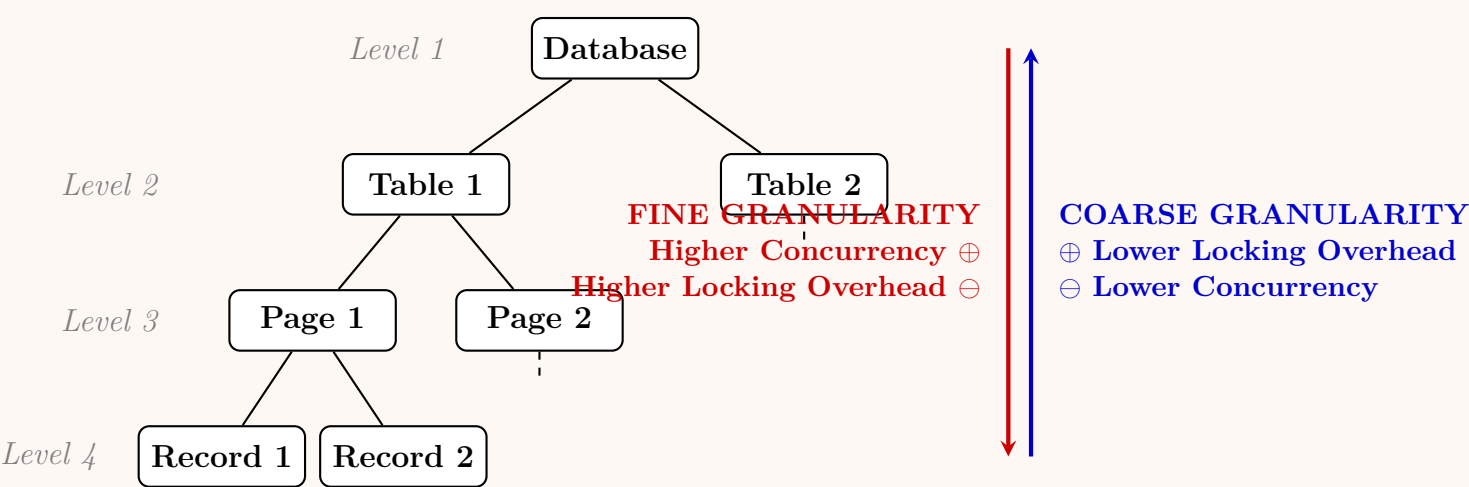
The choice of lock granularity involves a direct tradeoff between two competing performance metrics: **Concurrency** and **Locking Overhead**.

- **Coarse Granularity** (e.g., Database or Table level locks):

- **Advantage (Low Overhead):** The Lock Manager has to maintain very few locks. This consumes less memory (smaller lock table) and requires less processing time to acquire, check, and release locks.
 - **Disadvantage (Low Concurrency):** Locking a large data item blocks other transactions from accessing *any* part of that item, even if they want to access entirely distinct records. This leads to bottlenecks and reduced system throughput.
- **Fine Granularity (e.g., Page or Record/Tuple level locks):**
- **Advantage (High Concurrency):** Multiple transactions can access different records within the same table simultaneously without conflicting. This maximizes parallel execution and system throughput.
 - **Disadvantage (High Overhead):** If a transaction needs to update 10,000 records, the Lock Manager must allocate, track, and eventually release 10,000 individual locks. This causes massive memory consumption and CPU overhead.

3. Hierarchical Locking Diagram

The following diagram illustrates the data granularity hierarchy and visualizes the inverse relationship between concurrency and overhead.



4. Comparison Table of Granularity Levels

Granularity Level	Concurrency	Locking Overhead	Best Suited For...
Database	Very Low	Very Low	Mass batch updates, database maintenance.
Table/File	Low	Low	Operations scanning or altering entire tables.
Page/Block	Medium	Medium	Range queries reading adjacent disk blocks.
Record/Tuple	High	High	Online Transaction Processing (OLTP), single-row updates.

5. Addressing the Tradeoff: Intention Locks

To safely manage locks at multiple granularities without scanning the entire hierarchy to detect conflicts, DBMS implementations introduce **Intention Locks** (IS, IX, SIX).

Working Principle: If a transaction wants to lock a specific record (Fine Granularity) with an Exclusive (X) lock, it must first acquire an Intention Exclusive (IX) lock at the Database level, then at the Table level, and then at the Page level.

- This completely solves the overhead of checking for conflicts: if another transaction tries to place a coarse-grained X lock on the entire Table, it will immediately see the IX lock and block, preserving system integrity while maximizing concurrency.

Q37. Why does two-phase locking not work for B⁺ tree index structures? Write down the rules for accessing tree-structured data that ensure a serial order of transactions in a schedule. (15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Why Two-Phase Locking (2PL) Fails for B⁺-Tree Indexes

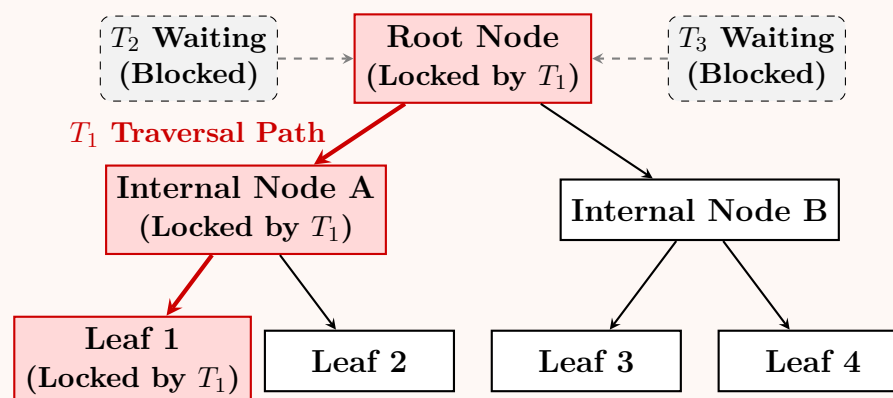
While Two-Phase Locking (2PL) guarantees conflict serializability, applying it directly to hierarchical or tree-structured data like B⁺-trees leads to severe performance degradation and structural bottlenecks. The reasons are as follows:

- **The Root Bottleneck (Concurrency Collapse):** In a B⁺-tree, every search, insertion, or deletion must traverse the tree starting from the **root node**. Under Strict 2PL, a transaction cannot release *any* lock until it reaches the shrinking phase (usually at transaction commit).

- **Example:** If Transaction T_1 wants to insert a value into a leaf node, it must acquire an Exclusive (X) lock on the root, then on the child, and so on down to the leaf. Under 2PL, T_1 must hold the lock on the root until the entire insertion is complete.
- **Consequence:** Because the root is locked by T_1 , **no other transaction can access the tree** for any operation. All concurrent transactions will queue at the root, effectively serializing the entire system and defeating the purpose of a concurrent DBMS.
- **High Deadlock Probability:** In variations where read locks are upgraded to write locks during node splitting (structural modification), 2PL frequently causes deadlocks when multiple transactions simultaneously traverse the same path and attempt to upgrade their locks.

2. Visualizing the 2PL Bottleneck in a Tree

The diagram below illustrates how holding a lock on the root (as required by 2PL) paralyzes the rest of the transactions.



3. The Tree Protocol (Graph-Based Protocol)

To solve the concurrency issues of 2PL on tree structures, DBMSs use graph-based protocols, specifically the **Tree Protocol**. This protocol relies on the directed acyclic graph (DAG) structure of the tree to ensure serializability without requiring two distinct phases.

Rules for Accessing Tree-Structured Data: For any transaction T_i operating on a tree-structured database graph:

- Initial Lock:** The first lock requested by T_i may be on *any* data item (node) in the tree. (For index traversals, this is typically the root).
- Parent-Child Dependency:** Subsequently, T_i can lock a node Q **only if** T_i currently holds a lock on the parent of node Q .
- Early Unlocking:** T_i can unlock a data item at *any time*. (It does not have to wait for a shrinking phase).
- No Relocking:** A data item that has been locked and subsequently unlocked by T_i **cannot** be relocked by T_i at a later time.

4. Crabbing / Hand-over-Hand Locking (Practical Implementation)

In B⁺-trees, the Tree Protocol is practically implemented as "Crabbing" or "Lock Coupling":

- **Working Principle:** As a transaction traverses down the tree, it acquires a lock on the child node *before* releasing the lock on the parent node.
- **Example (Search):** Lock Root → Lock Child → Unlock Root → Lock Grandchild → Unlock Child → (continue to leaf). This ensures the root is only locked for a fraction of a millisecond, allowing massive concurrency.

5. Comparison: 2PL vs. Tree Protocol				
Property		Two-Phase (2PL)	Locking	Tree Protocol
Lock Phase	Release	Strict only.	shrinking phase	Anytime (Early release allowed).
Concurrency Level		Extremely low for tree structures.		Very high (Root is quickly freed).
Serializability		Assured based on lock phases.		Assured based on structural graph order.
Deadlocks		Possible (Requires deadlock detection).		Deadlock-Free (Directed top-down access prevents cycles).
Recoverability		Cascading rollbacks preventable with Strict 2PL.		Uncommitted data may be read if locks are released early (Requires additional logging mechanisms).

Q38. Consider the following schedule involving transactions T_1 , T_2 , and T_3 :

$r_1(A), r_2(B), r_3(C), r_1(B), r_2(C), r_3(D), w_1(A), w_2(B), w_3(C)$

Insert shared locks, exclusive locks, and unlock actions in appropriate places. Tell what happens when the schedule is run by a scheduler supporting shared and exclusive locks. **(10 marks)**

CSE 303 | L-3/T-1 | 2013–14

Answer

1. Notation and Lock Insertion Strategy

To follow a standard **Two-Phase Locking (2PL)** protocol, a transaction must acquire a **Shared Lock (S)** before reading and an **Exclusive Lock (X)** before writing. If a transaction already holds a Shared Lock and needs to write, it must request an **upgrade** to an Exclusive Lock. Locks are released (**U**) after the transaction has completed its operations.

Let us define the locking instructions:

- $S_i(Q)$: Transaction T_i requests a Shared lock on data item Q .
- $X_i(Q)$: Transaction T_i requests an Exclusive lock (or lock upgrade) on data item Q .
- $U_i(Q)$: Transaction T_i releases its lock on data item Q .

Here is the intended schedule with the appropriate locks and unlocks inserted before and after the operations:

$S_1(A), r_1(A), S_2(B), r_2(B), S_3(C), r_3(C), S_1(B), r_1(B), S_2(C), r_2(C), S_3(D), r_3(D),$
 $X_1(A), w_1(A), X_2(B), w_2(B), X_3(C), w_3(C),$
 $U_1(A), U_1(B), U_2(B), U_2(C), U_3(C), U_3(D)$

2. Step-by-Step Execution by the Scheduler

When this schedule is submitted to a Lock Manager (scheduler), it evaluates lock compatibility in real-time. Shared locks can coexist, but Exclusive locks conflict with any other lock.

Let's trace what actually happens when this schedule is executed chronologically:

(a) **Time 1:** T_1 requests $S_1(A)$. Granted. T_1 executes $r_1(A)$.

(b) **Time 2:** T_2 requests $S_2(B)$. Granted. T_2 executes $r_2(B)$.

(c) **Time 3:** T_3 requests $S_3(C)$. Granted. T_3 executes $r_3(C)$.

(d) **Time 4:** T_1 requests $S_1(B)$. Granted (Compatible with T_2 's existing S-lock). T_1 executes $r_1(B)$.

(e) **Time 5:** T_2 requests $S_2(C)$. Granted (Compatible with T_3 's existing S-lock). T_2 executes $r_2(C)$.

(f) **Time 6:** T_3 requests $S_3(D)$. Granted. T_3 executes $r_3(D)$.

(g) **Time 7:** T_1 requests $X_1(A)$. Granted (T_1 upgrades its S-lock to an X-lock; no other transaction holds a lock on A). T_1 executes $w_1(A)$.

(h) **Time 8:** T_2 requests $X_2(B)$. **DENIED**. T_1 currently holds a Shared lock on B (acquired at Time 4). T_2 is **blocked** and must wait.

(i) **Time 9:** T_3 requests $X_3(C)$. **DENIED**. T_2 currently holds a Shared lock on C (acquired at Time 5). T_3 is **blocked** and must wait.

At this point, the initial sequence of operations is exhausted, but the blocked transactions must be resolved as transactions begin to commit and release locks:

10. T_1 finishes its operations and releases its locks: $U_1(A)$ and $U_1(B)$.
11. T_1 's unlock on B frees the data item. T_2 's pending request for $X_2(B)$ is now **Granted**.
12. T_2 resumes and executes $w_2(B)$.
13. T_2 finishes and releases its locks: $U_2(B)$ and $U_2(C)$.
14. T_2 's unlock on C frees the data item. T_3 's pending request for $X_3(C)$ is now **Granted**.
15. T_3 resumes and executes $w_3(C)$.
16. T_3 finishes and releases its locks: $U_3(C)$ and $U_3(D)$.

—

3. Final Status and Conclusion

What happens when the schedule is run?

- **Blocking Occurs:** The concurrent writes cause a cascaded waiting scenario. T_2 is temporarily blocked by T_1 (waiting for data item B), and T_3 is temporarily blocked by T_2 (waiting for data item C).
- **No Deadlock:** Although transactions are blocked, they are waiting in a linear dependency chain ($T_3 \rightarrow T_2 \rightarrow T_1$). Because T_1 is not waiting on anything, it completes successfully, allowing the others to sequentially unblock.
- **Successful Execution:** The scheduler successfully forces the conflicting write operations to serialize. The final delayed execution order of the writes ensures database consistency.

Q39. Define the following terms: conflict-equivalent and conflict-serializable. (5 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Conflict-Equivalent

Definition: Two schedules, let us say S_1 and S_2 , are defined as **conflict-equivalent** if they meet two primary conditions:

- (a) Both schedules contain the exact same set of transactions and the exact same set of operations for each transaction.
- (b) The relative execution order of every pair of **conflicting operations** is identical in both schedules.

What Constitutes a Conflict?

Two operations in a schedule are said to conflict if they satisfy *all three* of the following conditions simultaneously:

- They belong to **different transactions** (e.g., T_i and T_j).
- They access the **same data item** (e.g., variable Q).
- At least one of the operations is a **WRITE** operation.

This definition yields three distinct types of conflicts in database transactions:

- **Read-Write (RW) Conflict:** T_i reads a data item before T_j overwrites it.
- **Write-Read (WR) Conflict:** T_i writes a data item, and T_j reads that newly written value.
- **Write-Write (WW) Conflict:** T_i writes a data item, and T_j subsequently overwrites it.

Operations that do not meet these criteria (e.g., two READ operations on the same data item, or any operations on completely different data items) are **non-conflicting** and their execution order can be safely swapped without changing the final database state.

—

2. Conflict-Serializable

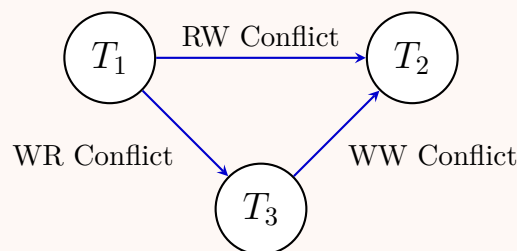
Definition: A concurrent schedule S is **conflict-serializable** if it is conflict-equivalent to at least one **serial schedule** (a schedule where transactions execute sequentially, one entirely after the other, with no interleaving).

Explanation and Properties:

- **Core Concept:** If a schedule is conflict-serializable, it means we can transform the concurrent schedule into a strict serial schedule simply by swapping non-conflicting operations.
- **Significance:** This property is the bedrock of database concurrency control. It guarantees that the concurrent execution of transactions will leave the database in a consistent state, exactly as if the transactions had been executed one by one in isolation.
- **Testing Mechanism (Precedence Graph):** Database Management Systems determine if a schedule is conflict-serializable by constructing a **Precedence Graph** (or Serialization Graph).

- **Nodes** represent committed transactions.
- **Directed Edges** ($T_i \rightarrow T_j$) represent conflicts where an operation in T_i executes before a conflicting operation in T_j .

Rule: A schedule is conflict-serializable *if and only if* its Precedence Graph is **acyclic** (contains no cycles).



Acyclic Graph $\Rightarrow$ Conflict-Serializable Schedule ($T_1 \rightarrow T_3 \rightarrow T_2$)

Q40. Why is conflict-serializability not necessary for serializability? Explain with an example. (5 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

Why Conflict-Serializability is Not Necessary for Serializability

In Database Management Systems, **conflict-serializability** is a *sufficient* but **not necessary** condition for a schedule to be serializable. A schedule can fail to be conflict-serializable (due to conflicting operations forming a cycle) but still remain serializable in its effect on the database.

This broader category of serializability is known as **View Serializability (VSR)**. The discrepancy arises entirely due to the presence of **Blind Writes**.

The Role of Blind Writes

A **blind write** occurs when a transaction writes a value to a data item without reading its preceding value. Blind writes allow a schedule to be view-equivalent to a serial schedule even if it contains unswappable conflicts (cycles in the precedence graph) because the intermediate conflicting writes are ultimately overwritten and ignored by subsequent transactions.

Detailed Example

Let us consider three transactions T_1 , T_2 , and T_3 operating on a single data item A .

- T_1 reads A and then writes A .
- T_2 simply writes A (Blind Write).
- T_3 simply writes A (Blind Write).

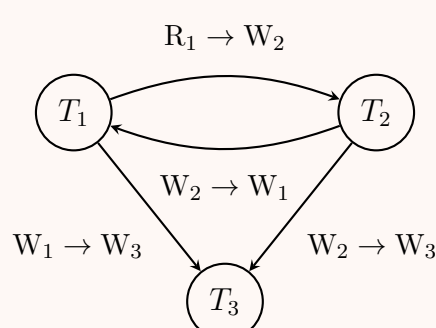
Consider the following interleaved schedule S :

Transaction T_1	Transaction T_2	Transaction T_3
Read(A)		
	Write(A)	
Write(A)		
		Write(A)

1. Proving S is NOT Conflict-Serializable

To check for conflict-serializability, we identify the conflicting operations and draw a Precedence Graph:

- Read₁(A) and Write₂(A) creates the edge $T_1 \rightarrow T_2$
- Write₂(A) and Write₁(A) creates the edge $T_2 \rightarrow T_1$
- Write₁(A) and Write₃(A) creates the edge $T_1 \rightarrow T_3$
- Write₂(A) and Write₃(A) creates the edge $T_2 \rightarrow T_3$



Conclusion: Because there is a cycle ($T_1 \leftrightarrow T_2$) in the precedence graph, the schedule S is **not** conflict-serializable.

2. Proving S IS View-Serializable

Despite not being conflict-serializable, S is actually serializable because it produces the exact same result as the serial schedule $S' = \langle T_1, T_2, T_3 \rangle$.

Let us verify the three conditions of View Equivalence between S and S' :

- (a) **Initial Read:** In S , T_1 reads the initial value of A . In S' , T_1 also reads the initial value of A . (Matches)
- (b) **Write-Read Dependencies:** No transaction reads a value produced by another transaction in S . The same holds true for S' . (Matches)
- (c) **Final Write:** In S , the final write operation on A is performed by T_3 . In S' , the final write on A is also performed by T_3 . (Matches)

Conclusion: Since schedule S satisfies all View Equivalence rules with the strict serial schedule $\langle T_1, T_2, T_3 \rangle$, S is **View-Serializable**.

Summary Note: Conflict-serializability guarantees serializability by strictly preventing unswappable conflicts. However, view-serializability demonstrates that when blind writes are present, strict conflict prevention is not necessary. The database can still maintain consistency even with cyclic conflicts, proving that conflict-serializability is merely a robust subset of all serializable schedules.

Q41. If all transactions are well-formed and two-phase, then any legal history will be serializable. Justify this statement. (6 marks) [CSE 303 | L-3/T-1 | 2012–13]

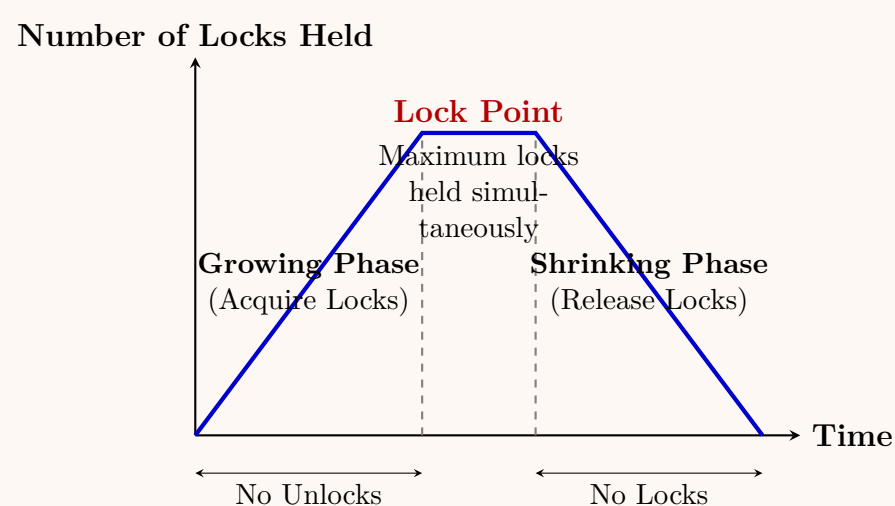
Answer

Justification of the Two-Phase Locking (2PL) Theorem

The statement refers to the fundamental theorem of **Two-Phase Locking (2PL)**, which guarantees conflict-serializability. To justify this statement, we must first establish the definitions of the terms involved and then provide a formal proof by contradiction.

1. Key Definitions

- **Well-Formed Transaction:** A transaction is well-formed if it always acquires an appropriate lock on a data item before operating on it (e.g., Shared Lock for Read, Exclusive Lock for Write), and releases the lock before completing.
- **Two-Phase Transaction (2PL):** A transaction follows the 2PL protocol if all locking operations precede the first unlock operation. It operates in two distinct phases:
 - **Growing Phase:** The transaction may obtain locks but cannot release any.
 - **Shrinking Phase:** The transaction may release locks but cannot acquire any new ones.
- **Legal History:** A schedule (history) is legal if it does not grant conflicting locks (e.g., two exclusive locks) on the same data item to different transactions at the same time.



2. Proof by Contradiction

We must prove that if a schedule H is legal and generated by well-formed, two-phase transactions, then H is **conflict-serializable**.

Assumption: Assume that all transactions in a legal history H are well-formed and two-phase, but H is **not serializable**.

- (a) **Existence of a Cycle:** Because H is not conflict-serializable, its Precedence Graph (Serialization Graph) must contain at least one directed cycle.
- (b) Let the cycle involve transactions $T_1, T_2, \dots, T_n$ such that:

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots \rightarrow T_n \rightarrow T_1$$

- (c) **Analyzing an Edge:** An edge $T_i \rightarrow T_j$ in the precedence graph implies that T_i and T_j have a conflicting operation on some data item X , and T_i 's operation occurred before T_j 's.
- (d) Because the history is **legal** and transactions are **well-formed**, T_i must have held a lock on X and subsequently released it *before* T_j could acquire a conflicting lock on X .
- (e) Therefore, T_i unlocks X before T_j locks X .
- (f) Since transactions are **two-phase**, T_i must have reached its **lock point** (the moment it holds all required locks and enters the shrinking phase) *before* it unlocked X .
- (g) Similarly, T_j must have acquired its lock on X *before* or *at* its own lock point.
- (h) This establishes a strict temporal ordering of lock points:

$$\text{LockPoint}(T_i) < \text{LockPoint}(T_j)$$

- (i) **Following the Cycle:** Applying this logic to the entire cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$, we get the following chain of inequalities:

$$\text{LockPoint}(T_1) < \text{LockPoint}(T_2) < \dots < \text{LockPoint}(T_n) < \text{LockPoint}(T_1)$$

- (j) **The Contradiction:** Transitivity of the strict inequality yields:

$$\text{LockPoint}(T_1) < \text{LockPoint}(T_1)$$

This is a logical impossibility. A point in time cannot strictly precede itself.

3. Conclusion

Our initial assumption that H is not serializable leads to an impossible contradiction. Therefore, any legal history generated by transactions that are well-formed and strictly obey the Two-Phase Locking protocol **must unconditionally be conflict-serializable**.

Note: While 2PL guarantees serializability, it does *not* guarantee freedom from deadlocks or cascading rollbacks (unless it is Strict-2PL or Rigorous-2PL).

Q42. Draw the precedence graph of the following schedule. Is the schedule conflict serializable? If so, what are all the equivalent serial schedules?

$$r_1(A); r_2(A); r_3(B); w_1(A); r_2(C); r_2(B); w_2(B); w_1(C)$$

(10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Schedule Analysis and Execution Timeline

Let S be the given schedule:

$$S = r_1(A); r_2(A); r_3(B); w_1(A); r_2(C); r_2(B); w_2(B); w_1(C)$$

To better understand the chronological execution and identify conflicts, we map the operations of transactions T_1 , T_2 , and T_3 in a tabular timeline.

Time	Transaction T_1	Transaction T_2	Transaction T_3
t_1	$r_1(A)$		
t_2		$r_2(A)$	
t_3			$r_3(B)$
t_4	$w_1(A)$		
t_5		$r_2(C)$	
t_6		$r_2(B)$	
t_7		$w_2(B)$	
t_8	$w_1(C)$		

2. Identification of Conflicting Operations

Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write (**W**) operation. We scan the schedule for such pairs:

- Data Item A:**

- $r_2(A)$ at t_2 and $w_1(A)$ at t_4 . Since T_2 reads A before T_1 writes A , this creates a Read-Write conflict.

Edge: $T_2 \rightarrow T_1$

- Data Item B:**

- $r_3(B)$ at t_3 and $w_2(B)$ at t_7 . Since T_3 reads B before T_2 writes B , this creates a Read-Write conflict.

Edge: $T_3 \rightarrow T_2$

- **Data Item C:**

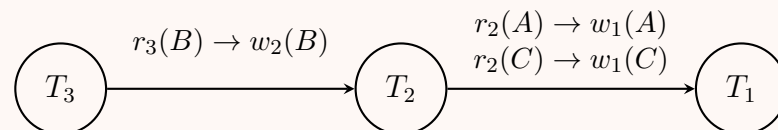
- $r_2(C)$ at t_5 and $w_1(C)$ at t_8 . Since T_2 reads C before T_1 writes C , this creates a Read-Write conflict.

Edge: $T_2 \rightarrow T_1$ (Duplicate edge, reinforces the existing one).

Note: Operations within the same transaction (e.g., $r_1(A)$ and $w_1(A)$, or $r_2(B)$ and $w_2(B)$) do not constitute a conflict.

3. Precedence Graph (Serialization Graph)

Using the identified edges ($T_3 \rightarrow T_2$ and $T_2 \rightarrow T_1$), we construct the precedence graph.



4. Conflict Serializability

Conclusion: Yes, the schedule is conflict serializable.

Justification: A schedule is conflict serializable if and only if its precedence graph is acyclic (contains no directed cycles). As seen in the graph above, the dependencies form a strict linear chain ($T_3 \rightarrow T_2 \rightarrow T_1$) without any cycles. Therefore, schedule S is conflict serializable.

5. Equivalent Serial Schedules

To find all equivalent serial schedules, we perform a topological sort on the precedence graph.

The graph gives us the strict execution ordering:

- T_3 has no incoming edges (indegree = 0), so it must execute first.
- T_2 must follow T_3 .
- T_1 must follow T_2 .

Since this is a linear graph, there is only **one** valid topological sorting. Thus, there is exactly one equivalent serial schedule:

$$\langle T_3, T_2, T_1 \rangle$$

Q43. Give a counter-example for: $P(S_1) = P(S_2) \Rightarrow S_1, S_2$ conflict equivalent. (4 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

Analysis of the Statement

The statement asserts that if two schedules, S_1 and S_2 , produce the exact same Precedence Graph ($P(S_1) = P(S_2)$), then they must be conflict equivalent.

This statement is **FALSE**.

Why is it false?

- **Conflict Equivalence** requires that the relative execution order of *every single pair* of conflicting operations be identical in both schedules.
- A **Precedence Graph** $P(S)$ only captures whether there is *at least one* conflict directing from T_i to T_j . It does not record the number of conflicts, nor does it capture conflicts on specific data items if an edge in that direction already exists due to another data item.

Therefore, we can construct two schedules that have the same precedence graph (due to cycles or multiple conflicts) but reverse the order of at least one specific conflict, breaking conflict equivalence.

—

Counter-Example

Let us define two transactions T_1 and T_2 operating on three data items A , B , and C :

- $T_1 : w_1(A); w_1(B); w_1(C)$
- $T_2 : w_2(A); w_2(B); w_2(C)$

Consider the following two interleaved schedules S_1 and S_2 :

Schedule S_1		Schedule S_2	
T_1	T_2	T_1	T_2
$w_1(A)$		$w_1(A)$	
	$w_2(A)$		$w_2(A)$
$w_1(B)$			$w_2(B)$
	$w_2(B)$	$w_1(B)$	
	$w_2(C)$		$w_2(C)$
$w_1(C)$		$w_1(C)$	

1. Evaluating Schedule S_1

Identify the conflicting operations in S_1 :

- On item A : $w_1(A)$ precedes $w_2(A) \implies$ Edge $T_1 \rightarrow T_2$
- On item B : $w_1(B)$ precedes $w_2(B) \implies$ Edge $T_1 \rightarrow T_2$
- On item C : $w_2(C)$ precedes $w_1(C) \implies$ Edge $T_2 \rightarrow T_1$

2. Evaluating Schedule S_2

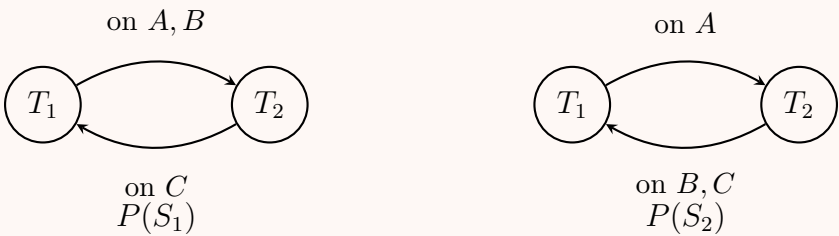
Identify the conflicting operations in S_2 :

- On item A : $w_1(A)$ precedes $w_2(A) \implies$ Edge $T_1 \rightarrow T_2$
- On item B : $w_2(B)$ precedes $w_1(B) \implies$ **Edge** $T_2 \rightarrow T_1$ (Notice the reversed order)
- On item C : $w_2(C)$ precedes $w_1(C) \implies$ Edge $T_2 \rightarrow T_1$

—

Comparing the Precedence Graphs and Equivalence

Let us draw the precedence graphs $P(S_1)$ and $P(S_2)$:



Conclusion of the Counter-Example

- (a) **Graph Equality:** The set of nodes and the set of directed edges in both graphs are mathematically identical. Both $P(S_1)$ and $P(S_2)$ contain exactly the edges $\{(T_1 \rightarrow T_2), (T_2 \rightarrow T_1)\}$. Thus, $P(S_1) = P(S_2)$.
- (b) **Conflict Inequivalence:** In S_1 , the conflicting operations on data item B occur in the order $w_1(B) \rightarrow w_2(B)$. However, in S_2 , they occur in the reversed order $w_2(B) \rightarrow w_1(B)$.

Because the order of at least one conflicting pair is different, S_1 and S_2 are **not conflict equivalent**, successfully disproving the statement.

Q44. Consider the following schedule of transactions T_1, T_2 and T_3 :

$$r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D)$$

Insert shared and exclusive locks and place necessary unlocks in appropriate positions. (10 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Locking Notation and Strategy

To process the given schedule while ensuring data consistency, we must insert appropriate lock and unlock instructions. We will use the following standard notation:

- LS_i(X):** Transaction T_i acquires a **Shared Lock** on data item X (required for reading).
- LX_i(X):** Transaction T_i acquires an **Exclusive Lock** on data item X (required for writing).

- $U_i(X)$: Transaction T_i **Unlocks** data item X .

Crucial Observation on 2PL: If we trace the sequence, T_1 needs an exclusive lock on B to perform $w_1(B)$. However, T_2 previously acquired a shared lock on B to perform $r_2(B)$. To allow T_1 to execute $w_1(B)$ exactly at the specified position in the schedule, T_2 *must* unlock B before T_1 locks it. Because T_2 has not yet acquired its lock on C for $w_2(C)$, T_2 releasing B early violates the **Two-Phase Locking (2PL)** protocol. Thus, the exact interleaving provided can only be executed using a non-2PL locking strategy.

2. Schedule Execution Timeline with Locks

We place the lock requests immediately before the operations and the necessary unlock requests right after the operation (or right before a conflicting operation from another transaction) to ensure the timeline executes exactly as provided without blocking.

Time	Transaction T_1	Transaction T_2	Transaction T_3
t_1	LS ₁ (A); $r_1(A)$		
t_2		LS ₂ (B); $r_2(B)$	
t_3		U ₂ (B)	
t_4			LS ₃ (C); $r_3(C)$
t_5			U ₃ (C)
t_6	LX ₁ (B); $w_1(B)$		
t_7	U ₁ (B)		
t_8		LX ₂ (C); $w_2(C)$	
t_9		U ₂ (C)	
t_{10}			LX ₃ (D); $w_3(D)$
t_{11}	U ₁ (A)		U ₃ (D)

3. Complete Linear Schedule

Writing the exact schedule linearly with the inserted locks and unlocks:

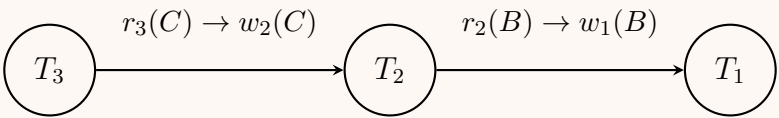
LS₁(A); $r_1(A)$; LS₂(B); $r_2(B)$; U₂(B); LS₃(C); $r_3(C)$; U₃(C); LX₁(B); $w_1(B)$; U₁(B); LX₂(C); $w_2(C)$; U₂(C); LX₃(D); $w_3(D)$; U₁(A); U₃(D)

4. Conflict Serializability Analysis

Even though the schedule does not strictly follow Two-Phase Locking, we can verify its correctness by checking its conflict serializability.

Conflicts present:

- (a) **On Data Item B:** T_2 reads B before T_1 writes $B \implies T_2 \rightarrow T_1$
- (b) **On Data Item C:** T_3 reads C before T_2 writes $C \implies T_3 \rightarrow T_2$



Conclusion: The precedence graph is completely acyclic (forming the chain $T_3 \rightarrow T_2 \rightarrow T_1$). Therefore, despite violating 2PL (which is why early unlocks were necessary to prevent blocking), the provided schedule is unequivocally **conflict serializable** and equivalent to the serial schedule $\langle T_3, T_2, T_1 \rangle$.

Q45. Define serial, serializable, conflict-serializable, and two-phase locking in terms of database transactions. (8 marks) [CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Serial Schedule

- **Definition:** A **serial schedule** is a transaction schedule in which the operations of each transaction are executed consecutively from start to finish, without any interleaving of operations from other transactions.
- **Properties:**
 - Transactions are executed one after the other in strict sequential order.
 - It natively guarantees database consistency (assuming individual transactions are correct) because isolation is completely maintained.

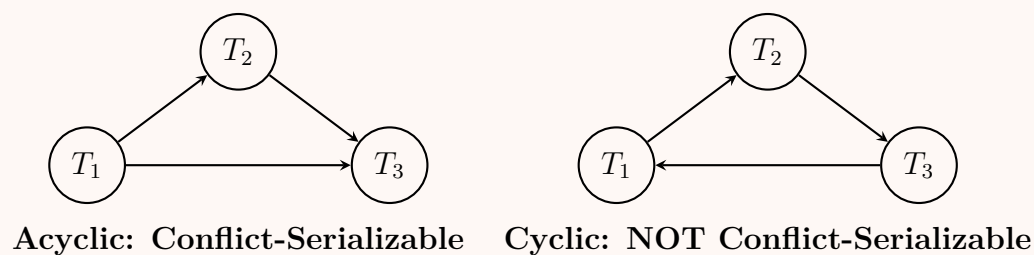
- **Disadvantage:** Extremely poor concurrency and resource utilization. If a transaction is waiting for disk I/O, the CPU remains idle because no other transaction can execute.

2. Serializable Schedule

- **Definition:** A **serializable schedule** is an interleaved schedule whose overall execution effect on the database is identical to that of *some* serial execution of the same set of transactions.
- **Explanation:** It allows operations of different transactions to interleave (improving concurrency and throughput) while formally guaranteeing that the final database state is logically consistent, just as if the transactions had run serially.
- **Classification:** Serializability is primarily categorized into **Conflict Serializability** and **View Serializability**.

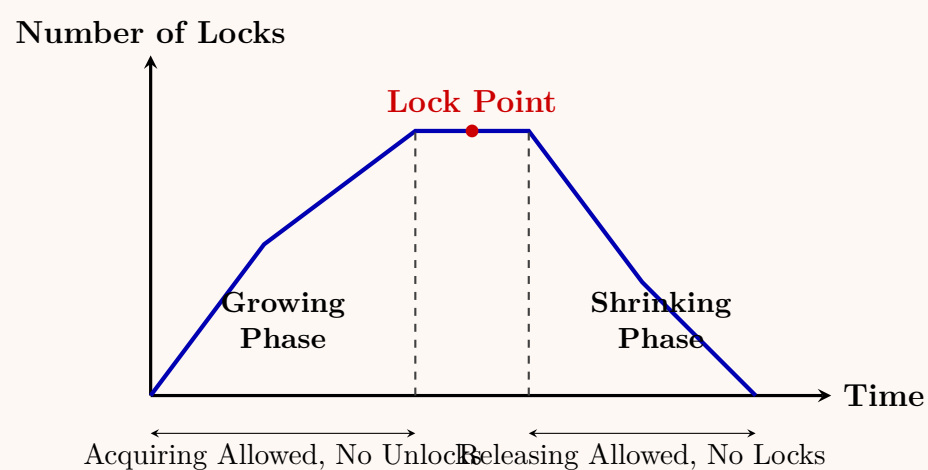
3. Conflict-Serializable Schedule

- **Definition:** A schedule is **conflict-serializable** if it can be transformed into a serial schedule by swapping adjacent, non-conflicting operations from different transactions.
- **Working Principle:** Two operations are in **conflict** if they belong to different transactions, access the same data item, and at least one is a write (*W*) operation (i.e., Read-Write, Write-Read, or Write-Write conflicts).
- **Precedence Graph Test:** A schedule is conflict-serializable *if and only if* its precedence (serialization) graph is **acyclic**.



4. Two-Phase Locking (2PL)

- **Definition:** **Two-Phase Locking (2PL)** is a pessimistic concurrency control protocol that guarantees conflict serializability by requiring every transaction to issue lock and unlock requests in two distinct, non-overlapping phases.
- **Working Principle:**
 - Growing Phase:** A transaction may acquire locks (Shared or Exclusive) on data items but *cannot release* any locks.
 - Lock Point:** The point in time when the transaction has acquired all the locks it will ever need (maximum locks held).
 - Shrinking Phase:** A transaction may release locks but *cannot acquire* any new locks.
- **Properties:** While basic 2PL strictly guarantees conflict serializability, it does *not* prevent deadlocks or cascading rollbacks. (Variations like Strict-2PL and Rigorous-2PL solve cascading rollbacks).



Q46. What is the precedence graph of the following schedule?

$r_1(A); r_2(A); r_1(B); r_2(B); r_3(A); r_4(B); w_1(A); w_2(B)$

Is the above schedule conflict serializable? If so, what are all the equivalent serial schedules?

(12 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Schedule Analysis and Execution Timeline

Let the given schedule be S :

$$S = r_1(A); r_2(A); r_1(B); r_2(B); r_3(A); r_4(B); w_1(A); w_2(B)$$

We map the execution of transactions T_1, T_2, T_3 , and T_4 chronologically to clearly identify conflicting operations.

Time	Transaction T_1	Transaction T_2	Transaction T_3	Transaction T_4
t_1	$r_1(A)$			
t_2		$r_2(A)$		
t_3	$r_1(B)$			
t_4		$r_2(B)$		
t_5			$r_3(A)$	
t_6				$r_4(B)$
t_7	$w_1(A)$			
t_8		$w_2(B)$		

2. Identification of Conflicting Operations

Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write (**W**) operation. We analyze the schedule for dependencies:

• Conflicts on Data Item A:

$r_2(A)$ occurs at t_2 , and $w_1(A)$ occurs at t_7 .

Edge: $T_2 \rightarrow T_1$

$r_3(A)$ occurs at t_5 , and $w_1(A)$ occurs at t_7 .

Edge: $T_3 \rightarrow T_1$

• Conflicts on Data Item B:

$r_1(B)$ occurs at t_3 , and $w_2(B)$ occurs at t_8 .

Edge: $T_1 \rightarrow T_2$

$r_4(B)$ occurs at t_6 , and $w_2(B)$ occurs at t_8 .

Edge: $T_4 \rightarrow T_2$

Note: Operations within the same transaction (e.g., $r_1(A)$ and $w_1(A)$) do not constitute a conflict.

3. Precedence Graph

Using the directed edges identified above, we construct the precedence (serialization) graph.

4. Conflict Serializability & Equivalent Schedules

• Is the schedule conflict serializable?

No. A schedule is conflict serializable if and only if its precedence graph is acyclic. In the graph above, there is a clear cycle between T_1 and T_2 ($T_1 \rightarrow T_2 \rightarrow T_1$). Therefore, the schedule is not conflict serializable.

• Equivalent Serial Schedules:

Because the schedule is not conflict serializable, it is impossible to topologically sort the precedence graph. Consequently, there are **no equivalent serial schedules** that can be derived using conflict equivalence.

Q47.

A concurrent schedule of three transactions T_1, T_2 and T_3 is given in Figure 3. Show the steps to transform the above schedule into an equivalent conflict serial schedule. (10 marks)

[CSE 303 | L-3/T-1 | 2010–11]No Figure found in PDF

Q48.

Explain with examples, the testing of serializability by using precedence graph. Show the cases of both conflict and non-conflict schedules. (10 marks)

[CSE 303 | L-3/T-1 | 2010–11]

429

Answer

1. Introduction to Precedence Graph

A **Precedence Graph** (or **Serialization Graph**) is a directed graph used in Database Management Systems to test whether a concurrent schedule is **conflict serializable**. If a schedule is conflict serializable, it guarantees database consistency by ensuring its execution is equivalent to some purely serial execution of the same transactions.

Conflicting Operations

Two operations are said to **conflict** if they satisfy all three of the following conditions:

- (a) They belong to **different** transactions.
- (b) They access the **same** data item.
- (c) At least one of the operations is a **Write (W)** operation.

This yields three types of conflicts: **Read-Write (RW)**, **Write-Read (WR)**, and **Write-Write (WW)**.

2. Construction and Testing Rules

Working Principle for Construction:

- **Nodes:** Create a node for every transaction T_i present in the schedule.
- **Edges:** Draw a directed edge $T_i \rightarrow T_j$ if an operation in T_i conflicts with an operation in T_j AND the operation in T_i occurs *chronologically before* the operation in T_j .

Testing Serializability:

- **Acyclic Graph:** If the precedence graph contains **no cycles**, the schedule is **conflict serializable**. (The equivalent serial order can be found using topological sorting).
- **Cyclic Graph:** If the precedence graph contains **at least one cycle**, the schedule is **NOT conflict serializable**.

3. Case 1: A Conflict Serializable Schedule

Consider a schedule S_1 consisting of two transactions T_1 and T_2 accessing data items X and Y .

$$S_1 : r_1(X); r_2(Y); w_1(X); r_2(X); w_2(Y); w_2(X)$$

Execution Timeline:

Time	Transaction T_1	Transaction T_2
t_1	$r_1(X)$	
t_2		$r_2(Y)$
t_3	$w_1(X)$	
t_4		$r_2(X)$
t_5		$w_2(Y)$
t_6		$w_2(X)$

Conflict Analysis for S_1 :

- **On Item X:**
 - $r_1(X)$ at t_1 and $w_2(X)$ at $t_6 \implies T_1 \rightarrow T_2$
 - $w_1(X)$ at t_3 and $r_2(X)$ at $t_4 \implies T_1 \rightarrow T_2$
 - $w_1(X)$ at t_3 and $w_2(X)$ at $t_6 \implies T_1 \rightarrow T_2$
- **On Item Y:** Only T_2 accesses Y , so there are no conflicts.

Precedence Graph for S_1 :

```
graph LR; T1((T1)) -- "on X" --> T2((T2))
```

Conclusion for Case 1: The precedence graph contains **no cycles** (it only flows from T_1 to T_2). Therefore, S_1 is **conflict serializable**. The equivalent serial schedule is $\langle T_1, T_2 \rangle$.

430

4. Case 2: A Non-Conflict Serializable Schedule

Consider another schedule S_2 where two transactions interleave differently:

$$S_2 : r_1(X); r_2(Y); w_2(X); w_1(Y)$$

Execution Timeline:

Time	Transaction T_1	Transaction T_2
t_1	$r_1(X)$	
t_2		$r_2(Y)$
t_3		$w_2(X)$
t_4	$w_1(Y)$	

Conflict Analysis for S_2 :

- On Item X:
 - $r_1(X)$ at t_1 is followed by $w_2(X)$ at $t_3 \implies T_1 \rightarrow T_2$
- On Item Y:
 - $r_2(Y)$ at t_2 is followed by $w_1(Y)$ at $t_4 \implies T_2 \rightarrow T_1$

Precedence Graph for S_2 :

```
graph LR; T1((T1)) -- "r1(X) -> w2(X)" --> T2((T2)); T2 -- "r2(Y) -> w1(Y)" --> T1;
```

Conclusion for Case 2: The precedence graph contains a **cycle** ($T_1 \rightarrow T_2 \rightarrow T_1$). Therefore, S_2 is **NOT conflict serializable**. It is impossible to swap non-conflicting operations to transform this into a pure serial schedule without altering the result.

Q49. Explain two-phase locking protocol. (5 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Definition

The **Two-Phase Locking (2PL)** protocol is a pessimistic concurrency control method in Database Management Systems (DBMS) that ensures **conflict serializability**. It dictates that every transaction must issue lock and unlock requests in two distinct, non-overlapping phases.

2. Working Principle and Phases

Under the 2PL protocol, a transaction's execution is divided into three logical stages:

(a) **Growing Phase (Phase 1):**

- A transaction may request and acquire new locks (Shared or Exclusive) on data items.
- A transaction **cannot release** any locks during this phase.

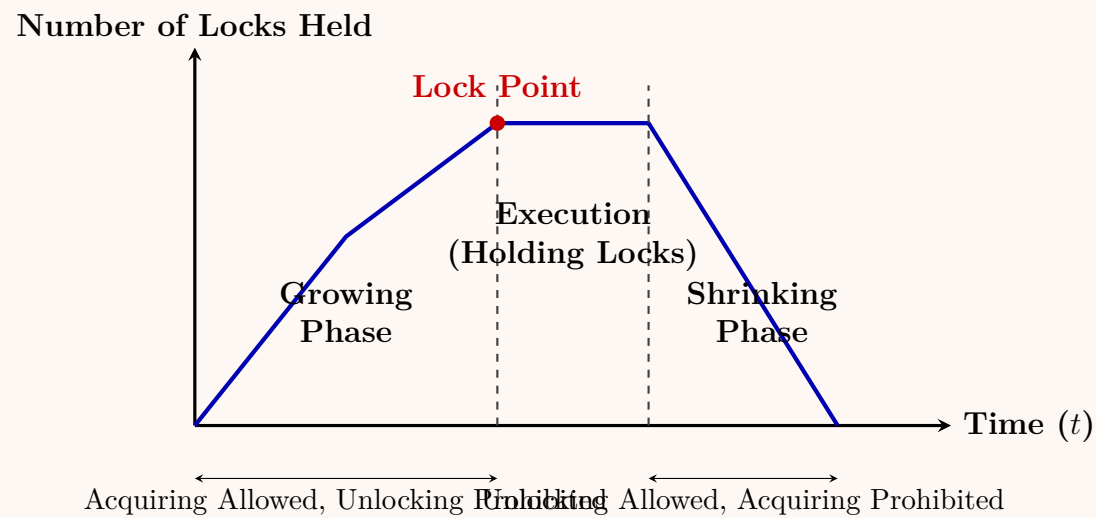
(b) **Lock Point:**

- The exact moment in time when the transaction has acquired all the locks it will ever need. It marks the end of the growing phase and the beginning of the shrinking phase.

(c) **Shrinking Phase (Phase 2):**

- A transaction may release previously acquired locks.
- A transaction **cannot acquire** any new locks, even if it needs to access a new data item.

3. Visual Representation of 2PL



4. Example of a 2PL Transaction

Consider a transaction T_1 transferring money from Account A to Account B:

Transaction Operation	2PL State
Lock_X(A);	Growing Phase (acquiring first lock)
Read(A);	
A = A - 100;	
Write(A);	
Lock_X(B);	Growing Phase (acquiring second lock)
– Lock Point –	Maximum locks acquired
Unlock(A);	Shrinking Phase begins
Read(B);	Shrinking Phase (final lock released)
B = B + 100;	
Write(B);	
Unlock(B);	

Note: If T_1 released A before acquiring B, it would violate 2PL and risk non-serializable interleaving.

5. Properties, Advantages, and Disadvantages

Advantages:

- **Guaranteed Serializability:** Any schedule of transactions that strictly adheres to the 2PL protocol is mathematically guaranteed to be conflict serializable.
- **Simplicity:** It is straightforward to implement in a lock manager.

Disadvantages:

- **Deadlocks:** Basic 2PL does *not* prevent deadlocks. (e.g., T_1 holds lock on X and waits for Y; T_2 holds lock on Y and waits for X).
- **Cascading Rollbacks:** If a transaction fails during its shrinking phase, any other transaction that read a data item uncommitted by the failing transaction must also be rolled back.

6. Variations of 2PL

To solve the disadvantages of basic 2PL, modern DBMS use stricter variations:

- **Strict 2PL:** Transactions hold all their *Exclusive (Write) locks* until they commit or abort. This prevents cascading rollbacks.
- **Rigorous 2PL:** Transactions hold *ALL locks* (both Shared and Exclusive) until they commit or abort. This is even stricter and easier to manage, though it slightly reduces concurrency.

Q50. Describe the implementation of lock manager using a lock table data structure and show the following locks:

- (a) T_1 has granted locks for data items 10, 22, 35 and is waiting for data item 44.
- (b) T_2 has granted locks for data items 44 and 22.
- (c) T_3 has been waiting for data item 22.
- (d) T_4 has been granted locks for data items 8, 10, 22 and is waiting for data item 35.

Also explain how the lock manager processes a lock request using the structure. (10 marks)

[CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Lock Manager Data Structure

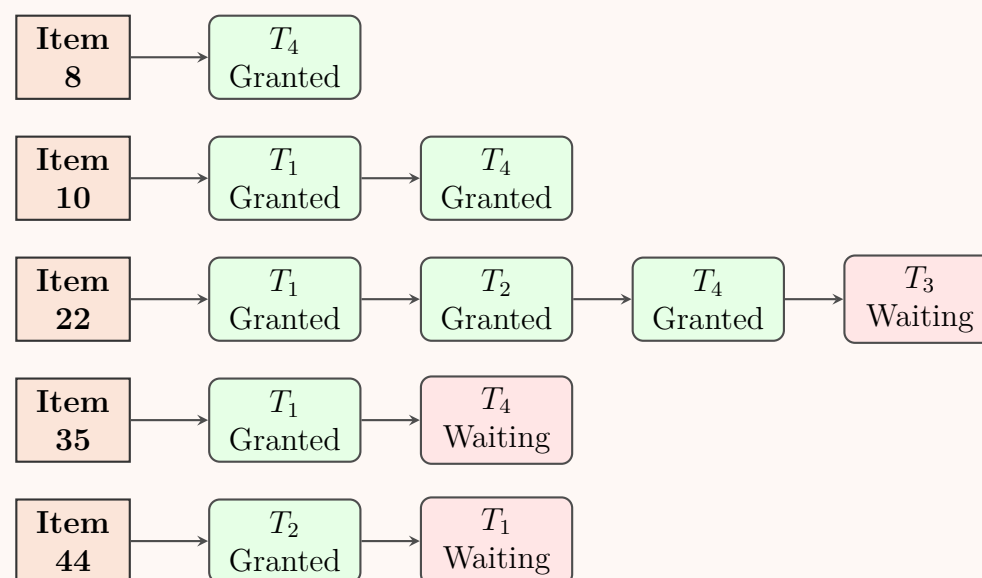
The Lock Manager uses a specialized data structure called a **Lock Table** to keep track of locks on data items. It is typically implemented as an in-memory **hash table** combined with **linked lists**.

- **Hash Table (Index):** The lock table uses the **Data Item ID** as the hash key. This allows the lock manager to perform $O(1)$ lookups to find the status of any specific data item.
- **Linked List (Lock Queue):** Each bucket in the hash table points to a linked list of lock request nodes.
- **Request Node:** Each node in the list represents a transaction's request for a lock on that data item. A node typically contains:
 - **Transaction ID** (T_i)
 - **Lock Mode** (Shared/Exclusive)
 - **Status** (**Granted** or **Waiting**)
 - **Pointer** to the next request in the queue.

Note: The presence of multiple "Granted" statuses for the same data item (e.g., Item 22) implies that these transactions hold compatible locks (Shared/Read locks).

2. Lock Table Diagram for the Given Scenario

Based on the conditions provided, the lock table groups the requests by Data Item (8, 10, 22, 35, and 44).



3. Processing a Lock Request

When a transaction T_i requests a lock on a data item X in a specific mode (Shared or Exclusive), the Lock Manager executes the following logical steps:

(a) **Lookup:** Hash the data item X to locate its corresponding linked list in the Lock Table.

(b) **Check List State:**

- **Empty Queue:** If there are no existing nodes in the linked list, the Lock Manager instantly creates a new node for T_i , sets its status to **Granted**, attaches it to the bucket, and allows T_i to continue execution.
- **Non-Empty Queue:** If the list is not empty, the Lock Manager evaluates two conditions:
 - (i) Is the requested lock mode **compatible** with *all* currently **Granted** locks? (e.g., Shared is compatible with Shared, but not with Exclusive).
 - (ii) Are there any **Waiting** transactions ahead of T_i in the queue? (This check is crucial to prevent *starvation* of waiting transactions).

(c) **Grant or Block:**

- **If Compatible AND no conflicting waiting transactions exist:** A new node is appended with the status **Granted**, and T_i proceeds.
- **If Incompatible (or to prevent starvation):** A new node is appended to the tail of the list with the status **Waiting**. The transaction T_i is placed in a blocked state by the OS/DBMS dispatcher until the lock can be acquired.

Handling Lock Releases (Unlocks): When a transaction commits, aborts, or voluntarily releases a lock on X , the Lock Manager removes its node from the linked list. It then checks the subsequent **Waiting** nodes in the list. If the newly released lock makes the data item available (compatible) for the next waiting transaction(s), their statuses are updated to **Granted**, and those transactions are unblocked.

Q51. Explain the rules of performing read and write operations in the timestamp ordering protocol. What are the advantages and disadvantages of this protocol? How is the protocol improved by the Thomas Write Rule? (10 marks) [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Introduction to Timestamp Ordering Protocol

The **Timestamp Ordering (TO) Protocol** is a non-locking concurrency control method. It ensures serializability by executing transactions such that their effect on the database is identical to a serial execution ordered by their timestamps.

Each transaction T_i is assigned a unique, monotonically increasing timestamp $TS(T_i)$. Every data item Q maintains two values:

- **W-TS(Q)**: The largest timestamp of any transaction that successfully executed a **write** on Q .
- **R-TS(Q)**: The largest timestamp of any transaction that successfully executed a **read** on Q .

2. Rules for Read and Write Operations

When a transaction T_i attempts to perform an operation on a data item Q , the protocol compares $TS(T_i)$ with the timestamps of Q .

Read Operation: T_i requests $R(Q)$

(a) **If** $TS(T_i) < \text{W-TS}(Q)$:

- *Explanation:* T_i is trying to read a value that was already overwritten by a "newer" transaction.
- *Action:* **Reject** the read request and **rollback** T_i .

(b) **If** $TS(T_i) \geq \text{W-TS}(Q)$:

- *Explanation:* The value of Q is valid for T_i to read.
- *Action:* **Execute** the read. Update $\text{R-TS}(Q) = \max(\text{R-TS}(Q), TS(T_i))$.

Write Operation: T_i requests $W(Q)$

(a) **If** $TS(T_i) < \text{R-TS}(Q)$:

- *Explanation:* A "newer" transaction has already read the old value of Q . If T_i writes now, the newer transaction would have read the wrong value.
- *Action:* **Reject** the write request and **rollback** T_i .

(b) **If** $TS(T_i) < \text{W-TS}(Q)$:

- *Explanation:* T_i is attempting to write an obsolete value because a "newer" transaction already wrote a newer value to Q .
- *Action:* **Reject** the write request and **rollback** T_i .

(c) **Otherwise:**

- *Explanation:* No conflict with newer transactions.
- *Action:* **Execute** the write. Update $\text{W-TS}(Q) = TS(T_i)$.

3. Advantages and Disadvantages

Advantages:

- **Deadlock Freedom:** The protocol does not use locks, so no transaction ever waits. This mathematically guarantees the absence of deadlocks.
- **Conflict Serializability:** It inherently guarantees that the execution is conflict serializable in the timestamp order.

Disadvantages:

- **Cascading Rollbacks:** If a transaction aborts, other transactions that read its uncommitted changes must also abort, unless strict TO is enforced.
- **Starvation risk:** Long-running transactions might repeatedly conflict with newer, shorter transactions and get endlessly rolled back.
- **Storage Overhead:** Maintaining two timestamp values for *every* data item in the database consumes considerable space and memory.

4. Improvement via Thomas Write Rule (TWR)

The **Thomas Write Rule** is a modification to the standard Timestamp Ordering protocol that allows greater concurrency. It specifically alters the **Write Operation** rules.

The Rule Modification: In the basic TO protocol, if T_i issues $W(Q)$ and $TS(T_i) < W-TS(Q)$, the transaction T_i is aborted. Under the **Thomas Write Rule**, if $TS(T_i) < W-TS(Q)$ (and $TS(T_i) \geq R-TS(Q)$), the write operation is simply **ignored (skipped)**, and T_i continues its execution normally instead of rolling back.

Why it works: The value T_i wants to write is already obsolete (a newer transaction overwrote it). Since no one has read T_i 's value yet (because $TS(T_i) \geq R-TS(Q)$), discarding T_i 's write has no negative effect on the database or other transactions.

T_1 (Older, $TS = 10$)

Write(Q)
(Obsolete Write)

T_2 (Newer, $TS = 20$)

Write(Q)
 $W-TS(Q) \leftarrow 20$

Basic TO:
Abort and Rollback T_1

Thomas Write Rule:
Ignore Write, Continue T_1

Key Impact: Schedules produced under the Thomas Write Rule may not be *conflict serializable*, but they are guaranteed to be **view serializable**. This means TWR successfully expands the set of acceptable, correct concurrent schedules, improving throughput.

Deadlock

Q1. Consider the following database schedule involving four transactions T_1, T_2, T_3, T_4 with deadlock timestamps 100, 150, 200, 250 respectively:

$S: l_1(A); r_1(A); l_2(B); w_2(B); l_1(B); l_3(C); r_3(C); l_2(C); l_4(C); r_4(C); l_1(C)$

(a) Determine the actions (wound/wait) taken by the wound-wait deadlock prevention protocol. Assume transactions are not restarted after rollback.

(b) Determine the actions (wait/die) taken by the wait-die deadlock prevention protocol. Assume transactions are not restarted after rollback.

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Initial Setup and Transaction Ages

In deadlock prevention protocols using timestamps, a smaller timestamp indicates an **older** transaction. Given the timestamps:

- $TS(T_1) = 100$ (Oldest)
- $TS(T_2) = 150$
- $TS(T_3) = 200$
- $TS(T_4) = 250$ (Youngest)

2. Wound-Wait Protocol

Rule: If transaction T_i requests a lock on a data item held by T_j :

- If T_i is **older** than T_j ($TS(T_i) < TS(T_j)$): T_i **wounds** (aborts) T_j .
- If T_i is **younger** than T_j ($TS(T_i) > TS(T_j)$): T_i **waits**.

Execution Trace:

435

Time	Operation	Action / Status under Wound-Wait
t_1	$l_1(A)$	Lock on A granted to T_1 .
t_2	$r_1(A)$	T_1 reads A .
t_3	$l_2(B)$	Lock on B granted to T_2 .
t_4	$w_2(B)$	T_2 writes B .
t_5	$l_1(B)$	T_1 requests B , held by T_2 . Check: $TS(T_1) < TS(T_2) \implies$ Older requests from Younger. Action: T_1 wounds T_2 . T_2 is aborted (rollback). T_1 acquires B .
t_6	$l_3(C)$	Lock on C granted to T_3 .
t_7	$r_3(C)$	T_3 reads C .
t_8	$l_2(C)$	Ignored (T_2 was already aborted at t_5).
t_9	$l_4(C)$	T_4 requests C , held by T_3 . Check: $TS(T_4) > TS(T_3) \implies$ Younger requests from Older. Action: T_4 waits .
t_{10}	$r_4(C)$	Suspended (T_4 is waiting for the lock).
t_{11}	$l_1(C)$	T_1 requests C , held by T_3 . Check: $TS(T_1) < TS(T_3) \implies$ Older requests from Younger. Action: T_1 wounds T_3 . T_3 is aborted . T_1 acquires C .

Final State (Wound-Wait):

- T_1 is executing (holds locks A, B, C).
- T_2 and T_3 are **aborted**.
- T_4 is **waiting** (will acquire C after T_1 releases it).

3. Wait-Die Protocol

Rule: If transaction T_i requests a lock on a data item held by T_j :

- If T_i is **older** than T_j ($TS(T_i) < TS(T_j)$): T_i **waits**.
- If T_i is **younger** than T_j ($TS(T_i) > TS(T_j)$): T_i **dies** (aborts).

Execution Trace:

Time	Operation	Action / Status under Wait-Die
t_1	$l_1(A)$	Lock on A granted to T_1 .
t_2	$r_1(A)$	T_1 reads A .
t_3	$l_2(B)$	Lock on B granted to T_2 .
t_4	$w_2(B)$	T_2 writes B .
t_5	$l_1(B)$	T_1 requests B , held by T_2 . Check: $TS(T_1) < TS(T_2) \implies$ Older requests from Younger. Action: T_1 waits for T_2 . T_1 is now blocked.
t_6	$l_3(C)$	Lock on C granted to T_3 .
t_7	$r_3(C)$	T_3 reads C .
t_8	$l_2(C)$	T_2 requests C , held by T_3 . Check: $TS(T_2) < TS(T_3) \implies$ Older requests from Younger. Action: T_2 waits for T_3 . T_2 is now blocked.
t_9	$l_4(C)$	T_4 requests C , held by T_3 . Check: $TS(T_4) > TS(T_3) \implies$ Younger requests from Older. Action: T_4 dies . T_4 is aborted (rollback).
t_{10}	$r_4(C)$	Ignored (T_4 was already aborted at t_9).
t_{11}	$l_1(C)$	Suspended (T_1 was blocked at t_5 waiting for B . It cannot issue new requests until unblocked).

Final State (Wait-Die):

- T_3 is executing (holds lock C).
- T_1 is **waiting** for T_2 .
- T_2 is **waiting** for T_3 .
- T_4 is **aborted** (dead).

Q2. Consider the following schedule involving four transactions T_1, T_2, T_3, T_4 with $TS(T_1) < TS(T_2) < TS(T_3) < TS(T_4)$:

$$l_2(A); r_2(A); l_2(B); l_1(A); l_3(C); r_3(C); l_3(B); l_4(C); r_4(C); l_1(C); r_1(C)$$

- (a) For each lock action $l_i(X)$, determine what happens under the wait-die scheme.
- (b) For each lock action $l_i(X)$, determine what happens under the wound-wait scheme.

(13 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Operation	Action / Status under Wait-Die
$l_2(A)$	Lock on A granted to T_2 .
$r_2(A)$	T_2 reads A .
$l_2(B)$	Lock on B granted to T_2 .
$l_1(A)$	T_1 requests A , held by T_2 . Check: $TS(T_1) < TS(T_2) \implies$ Older requests from Younger. Action: T_1 waits for T_2 . T_1 is blocked.
$l_3(C)$	Lock on C granted to T_3 .
$r_3(C)$	T_3 reads C .
$l_3(B)$	T_3 requests B , held by T_2 . Check: $TS(T_3) > TS(T_2) \implies$ Younger requests from Older. Action: T_3 dies . T_3 is aborted and releases its lock on C .
$l_4(C)$	T_4 requests C . Since T_3 aborted, C is now free. Lock granted to T_4 .
$r_4(C)$	T_4 reads C .
$l_1(C)$	Suspended. (T_1 was blocked earlier at $l_1(A)$ and cannot issue new operations).
$r_1(C)$	Suspended.

3. Wound-Wait Scheme Trace

Rule: If T_i requests a lock held by T_j :

- If T_i is **older** than T_j ($TS(T_i) < TS(T_j)$): T_i **wounds** (aborts) T_j .
- If T_i is **younger** than T_j ($TS(T_i) > TS(T_j)$): T_i **waits**.

Operation	Action / Status under Wound-Wait
$l_2(A)$	Lock on A granted to T_2 .
$r_2(A)$	T_2 reads A .
$l_2(B)$	Lock on B granted to T_2 .
$l_1(A)$	T_1 requests A , held by T_2 . Check: $TS(T_1) < TS(T_2) \implies$ Older requests from Younger. Action: T_1 wounds T_2 . T_2 is aborted and releases locks on A and B . Lock on A granted to T_1 .
$l_3(C)$	Lock on C granted to T_3 .
$r_3(C)$	T_3 reads C .
$l_3(B)$	T_3 requests B . Since T_2 was aborted, B is free. Lock granted to T_3 .
$l_4(C)$	T_4 requests C , held by T_3 . Check: $TS(T_4) > TS(T_3) \implies$ Younger requests from Older. Action: T_4 waits for T_3 . T_4 is blocked.
$r_4(C)$	Suspended. (T_4 is blocked waiting for C).
$l_1(C)$	T_1 requests C , held by T_3 . Check: $TS(T_1) < TS(T_3) \implies$ Older requests from Younger. Action: T_1 wounds T_3 . T_3 is aborted and releases locks on C and B . Lock on C granted to T_1 .
$r_1(C)$	T_1 reads C .

Q3. Consider the following schedule of actions involving four transactions T_1, T_2, T_3 and T_4 :

$r_1(A); r_2(B); w_1(C); r_3(D); w_3(B); w_2(C); w_4(A); w_1(D); w_1(B)$

Assume that the deadlock-timestamps of T_1, T_2, T_3 and T_4 are 1, 2, 3 and 4, respectively. For each action of the given schedule, determine what happens (“wound” or “wait”) under the wound-wait deadlock avoidance system. If the action is a “wound”, do not restart the transaction. (10 marks)

Answer**1. Initial Setup and Rules**

In deadlock prevention using timestamps, a smaller timestamp indicates an **older** transaction. Given the timestamps:

- $TS(T_1) = 1$ (**Oldest**)
- $TS(T_2) = 2$
- $TS(T_3) = 3$
- $TS(T_4) = 4$ (**Youngest**)

Wound-Wait Protocol Rules: When a transaction T_i requests a lock on a data item held by T_j :

- If T_i is **older** than T_j ($TS(T_i) < TS(T_j)$): T_i **wounds** (aborts) T_j .
- If T_i is **younger** than T_j ($TS(T_i) > TS(T_j)$): T_i **waits**.

2. Execution Trace

Action	Analysis and Outcome under Wound-Wait
$r_1(A)$	T_1 requests a Read lock on A . Granted . T_1 reads A .
$r_2(B)$	T_2 requests a Read lock on B . Granted . T_2 reads B .
$w_1(C)$	T_1 requests a Write lock on C . Granted . T_1 writes C .
$r_3(D)$	T_3 requests a Read lock on D . Granted . T_3 reads D .
$w_3(B)$	T_3 requests a Write lock on B , currently held by T_2 . Check: $TS(T_3) > TS(T_2)$ ($3 > 2$) $\implies$ Younger requests from Older. Action: T_3 waits for T_2 . T_3 is now blocked.
$w_2(C)$	T_2 requests a Write lock on C , currently held by T_1 . Check: $TS(T_2) > TS(T_1)$ ($2 > 1$) $\implies$ Younger requests from Older. Action: T_2 waits for T_1 . T_2 is now blocked.
$w_4(A)$	T_4 requests a Write lock on A , currently held by T_1 . Check: $TS(T_4) > TS(T_1)$ ($4 > 1$) $\implies$ Younger requests from Older. Action: T_4 waits for T_1 . T_4 is now blocked.
$w_1(D)$	T_1 requests a Write lock on D , currently held by T_3 (which is currently waiting). Check: $TS(T_1) < TS(T_3)$ ($1 < 3$) $\implies$ Older requests from Younger. Action: T_1 wounds T_3 . T_3 is aborted (rollback) and its lock on D is released. The lock on D is granted to T_1 , and T_1 writes D .
$w_1(B)$	T_1 requests a Write lock on B , currently held by T_2 (which is currently waiting). Check: $TS(T_1) < TS(T_2)$ ($1 < 2$) $\implies$ Older requests from Younger. Action: T_1 wounds T_2 . T_2 is aborted (rollback) and its lock on B is released. The lock on B is granted to T_1 , and T_1 writes B .

Final State Summary:

- T_1 : Successfully acquired all locks (A, C, D, B) and continues executing.
- T_2 : Aborted (Wounded by T_1).
- T_3 : Aborted (Wounded by T_1).
- T_4 : Waiting (Blocked on T_1 for data item A).

Q4. Consider the following schedule of actions involving four transactions T_1, T_2, T_3 and T_4 . Shared locks are requested immediately before each read action and exclusive locks immediately before every write action:

$r_1(A); r_2(B); w_1(C); r_3(D); w_3(B); w_2(C); w_4(A); w_1(D)$

- Show the wait-for graph after the execution of all actions.
- Determine if there is a deadlock. In case of deadlock, choose the highest-numbered transaction ($T_4 > T_3 > T_2 > T_1$) to roll back. Show the resulting wait-for graph after the rollback.

(12 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Execution Trace and Wait-For Graph Construction

To build the wait-for graph, we trace the lock requests based on the rule: Shared (S) locks for reads, Exclusive (X) locks for writes. An edge $T_i \rightarrow T_j$ is created if T_i requests a lock on a data item that is currently held by T_j in an incompatible mode.

Action	Lock Status and Wait Condition
$r_1(A)$	T_1 requests S-Lock(A) . Granted.
$r_2(B)$	T_2 requests S-Lock(B) . Granted.
$w_1(C)$	T_1 requests X-Lock(C) . Granted.
$r_3(D)$	T_3 requests S-Lock(D) . Granted.
$w_3(B)$	T_3 requests X-Lock(B) . Incompatible with T_2 's S-Lock. T_3 waits for $T_2 \implies$ Edge: $T_3 \rightarrow T_2$
$w_2(C)$	T_2 requests X-Lock(C) . Incompatible with T_1 's X-Lock. T_2 waits for $T_1 \implies$ Edge: $T_2 \rightarrow T_1$
$w_4(A)$	T_4 requests X-Lock(A) . Incompatible with T_1 's S-Lock. T_4 waits for $T_1 \implies$ Edge: $T_4 \rightarrow T_1$
$w_1(D)$	T_1 requests X-Lock(D) . Incompatible with T_3 's S-Lock. T_1 waits for $T_3 \implies$ Edge: $T_1 \rightarrow T_3$

Wait-For Graph after all actions:

```
graph TD; T4((T4)) -- "on A on C" --> T1((T1)); T2((T2)) -- "on B" --> T1; T1 -- "on D" --> T3((T3)); T3 -- "" --> T2;
```

2. Deadlock Detection and Resolution

Deadlock Detection: Yes, there is a deadlock. By inspecting the wait-for graph, we can identify a cycle among the transactions:

$$T_1 \xrightarrow{\text{waiting on } D} T_3 \xrightarrow{\text{waiting on } B} T_2 \xrightarrow{\text{waiting on } C} T_1$$

Note that T_4 is blocked (waiting on T_1), but it is not part of the deadlock cycle.

Deadlock Resolution (Rollback): The transactions involved in the deadlock cycle are T_1, T_2 , and T_3 . According to the rule, we must choose the highest-numbered transaction **involved in the deadlock** to roll back. Comparing the numbers ($T_3 > T_2 > T_1$), we select **T_3 to be rolled back**.

When T_3 rolls back:

- It releases its S-Lock on data item D .
- Its pending request (X-Lock on B) is cancelled, removing the edge $T_3 \rightarrow T_2$.
- Because D is now free, T_1 acquires its requested X-Lock on D , removing the edge $T_1 \rightarrow T_3$.

Resulting Wait-For Graph after T_3 rollback:

```
graph TD; T4((T4)) -- "on A on C" --> T1((T1)); T2((T2)) -- "" --> T1;
```

The deadlock is successfully broken. T_1 can now eventually finish and release its locks, unblocking T_2 and T_4 .

Q5. What is deadlock in database transaction? Give an example. (5 marks)

[CSE 303 | L-3/T-1 | 2017–18, 2016–17, 2011–12]

440

Answer

1. Definition of Deadlock

In a Database Management System (DBMS), a **deadlock** is an undesirable situation where two or more transactions are indefinitely waiting for one another to release locks on data items. Because each transaction holds a lock that the other transaction requires to proceed, neither can make progress, resulting in a state of permanent blocking known as a **circular wait**.

2. Mechanism of a Deadlock

For a deadlock to occur, a cycle of dependencies must form. The database system usually detects this by maintaining a **Wait-For Graph (WFG)**, where a cycle inherently indicates a deadlock.

3. Example of a Deadlock

Consider two transactions, T_1 and T_2 , and two bank accounts (data items), A and B .

- T_1 executes a process to transfer funds from Account A to Account B .
- T_2 executes a process to transfer funds from Account B to Account A .

Execution Schedule

The following schedule illustrates the chronological sequence of operations leading to a deadlock:

Time	Transaction T_1	Transaction T_2	Lock Status & Explanation
t_1	lock_X(A)		T_1 successfully acquires an Exclusive lock on A .
t_2		lock_X(B)	T_2 successfully acquires an Exclusive lock on B .
t_3	lock_X(B)		T_1 requests B . It is blocked and waits for T_2 .
t_4		lock_X(A)	T_2 requests A . It is blocked and waits for T_1 .

At time t_4 , a deadlock officially occurs because T_1 is waiting for T_2 to release B , and T_2 is waiting for T_1 to release A . Neither transaction can commit or release their current locks.

Wait-For Graph Representation

The deadlock cycle can be visualized using a Wait-For Graph. A directed edge from T_i to T_j indicates that transaction T_i is waiting for a lock held by transaction T_j .

```
graph LR; T1((T1)) -- "Waiting for B" --> T2((T2)); T2 -- "Waiting for A" --> T1;
```

4. Deadlock Handling

Without external intervention, the system remains locked indefinitely. Common DBMS strategies to resolve or prevent this include:

- Deadlock Prevention:** Using protocols like *Wait-Die* or *Wound-Wait* based on transaction timestamps to preemptively ensure cycles never form.
- Deadlock Detection and Recovery:** Periodically checking the Wait-For Graph for cycles. If a cycle is found, the DBMS aborts (rolls back) one of the involved transactions (the **victim**) to break the cycle and free the resources for the other transactions.

Recovery Systems

Q1. Compare the force policy and steal policy with respect to how modified buffers are written to disk in a database system. Under which policy does the database recovery manager not need to perform any actions during the redo phase? (7 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Introduction to Buffer Management Policies

In a Database Management System (DBMS), the buffer manager dictates how and when data pages modified in main memory (buffers) are written back to non-volatile storage (disk). The interaction between memory buffers and disk storage is governed by two primary dimensions of policies: **Steal / No-Steal** and **Force / No-Force**. These policies directly dictate the responsibilities of the Recovery Manager during system crashes.

2. The Steal Policy (Uncommitted Data Management)

The **Steal** policy governs whether a transaction is allowed to overwrite (steal) unpinned buffer frames on disk before it actually commits.

- Working Principle:** The DBMS can flush a modified buffer to disk even if the transaction that made the modification is still active and uncommitted. This is usually done to make room in the buffer pool for other operations.
- Implications for Recovery (UNDO):** If the system crashes, these uncommitted changes might already reside on disk. The recovery manager must perform an **UNDO** operation to roll back these partial, uncommitted modifications to maintain database consistency.
- Comparison to No-Steal:** Under a *No-Steal* policy, uncommitted updates are never written to disk. This requires no UNDO, but restricts transactions to being no larger than the available buffer memory, limiting concurrency and throughput.

3. The Force Policy (Committed Data Management)

The **Force** policy governs whether a transaction’s modified buffers must be immediately written to disk upon a commit request.

- Working Principle:** When a transaction issues a commit, the DBMS *forces* all of the transaction’s modified blocks to non-volatile disk storage before acknowledging the commit as successful.
- Implications for Recovery (REDO):** Because the transaction is not considered committed until every updated page is safely on disk, a system crash can never result in a situation where a committed transaction’s effects are lost from the database pages. Therefore, **no REDO operation is required**.
- Comparison to No-Force:** Under a *No-Force* policy, a transaction can commit while its modified data pages still reside only in volatile memory (though the commit record is written to the log). If a crash occurs, the recovery manager must perform a **REDO** to reapply the committed changes that were lost from memory.

4. Visual Comparison Matrix

The combination of these policies defines the complexity of the DBMS recovery subsystem. Modern systems (like ARIES) typically use a **Steal / No-Force** approach to maximize I/O performance, accepting the trade-off of a more complex recovery process.

	Steal Policy (Writes uncommitted data)	No-Steal Policy (Never writes uncommitted data)
Force Policy (Disk write at commit)	Requires UNDO No REDO needed <i>(Poor I/O performance)</i>	No UNDO needed No REDO needed <i>(Memory bottleneck)</i>
No-Force Policy (Deferred disk write)	Requires UNDO Requires REDO (Standard DBMS choice)	No UNDO needed Requires REDO <i>(Strictly sequential)</i>

5. Conclusion: Eliminating the REDO Phase

Question Answer: The database recovery manager does not need to perform any actions during the redo phase under the **Force policy**.

Reasoning: Because the Force policy ensures that all data modifications made by a transaction are successfully written to permanent disk storage before the transaction is officially marked as committed, no committed updates can ever be lost due to a sudden power failure or system crash. Consequently, the recovery manager has no missing updates to “redo” upon restarting.

Q2. The database recovery manager takes a redo or undo action expressed as $\langle X, V \rangle$, meaning database object X is written with value

442

V on disk. Consider the following log records:

$\langle T_0 \text{ start} \rangle; \langle T_0, A, 40, 60 \rangle; \langle T_1 \text{ start} \rangle; \langle T_1, B, 100, 150 \rangle; \langle T_0, C, 50, 100 \rangle; \langle T_2 \text{ start} \rangle; \langle T_2, D, 100, 70 \rangle; \langle T_1, C, 20, 50 \rangle; \langle T_1 \text{ commit} \rangle;$
 $\langle T_2, C, 60, 80 \rangle; \langle T_0 \text{ commit} \rangle; \langle T_2 \text{ commit} \rangle$

(a) For both redo and undo phases, write down the actions of the recovery manager if a system crash happens between $\langle T_2, C, 60, 80 \rangle$ and $\langle T_0 \text{ commit} \rangle$.

(b) For both redo and undo phases, write down the actions of the recovery manager if a system crash happens between $\langle T_1, C, 20, 50 \rangle$ and $\langle T_1 \text{ commit} \rangle$.

(11 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Recovery Principles (Immediate Update Protocol)

In standard Database Recovery using the **Immediate Update Protocol** with an Undo/Redo log:

- **UNDO Phase:** Applies to all transactions that started but did not commit before the crash (active transactions). The log is scanned **backwards**, and the database object X is restored to its **old value**.
- **REDO Phase:** Applies to all transactions that successfully committed before the crash. The log is scanned **forwards**, and the database object X is set to its **new value**.

The required action format is $\langle X, V \rangle$, meaning object X is written with value V .

2. Crash Scenario 1

Condition: System crash happens between $\langle T_2, C, 60, 80 \rangle$ and $\langle T_0 \text{ commit} \rangle$.

Available Log at Crash:

(a) $\langle T_0 \text{ start} \rangle$

(b) $\langle T_0, A, 40, 60 \rangle$

(c) $\langle T_1 \text{ start} \rangle$

(d) $\langle T_1, B, 100, 150 \rangle$

(e) $\langle T_0, C, 50, 100 \rangle$

(f) $\langle T_2 \text{ start} \rangle$

(g) $\langle T_2, D, 100, 70 \rangle$

(h) $\langle T_1, C, 20, 50 \rangle$

(i) $\langle T_1 \text{ commit} \rangle$

(j) $\langle T_2, C, 60, 80 \rangle$ <– **CRASH HAPPENS HERE**

Transaction Status:

- **Committed:** T_1 (requires REDO).
- **Uncommitted (Active):** T_0, T_2 (require UNDO).

Recovery Manager Actions:

- **UNDO Actions (Scanning Backwards):**

(a) Undo T_2 on C : $\langle C, 60 \rangle$

(b) Undo T_2 on D : $\langle D, 100 \rangle$

(c) Undo T_0 on C : $\langle C, 50 \rangle$

(d) Undo T_0 on A : $\langle A, 40 \rangle$
- **REDO Actions (Scanning Forwards):**

(a) Redo T_1 on B : $\langle B, 150 \rangle$

(b) Redo T_1 on C : $\langle C, 50 \rangle$

3. Crash Scenario 2

Condition: System crash happens between $\langle T_1, C, 20, 50 \rangle$ and $\langle T_1 \text{ commit} \rangle$.

Available Log at Crash:

(a) $\langle T_0 \text{ start} \rangle$

(b) $\langle T_0, A, 40, 60 \rangle$

(c) $\langle T_1 \text{ start} \rangle$

(d) $\langle T_1, B, 100, 150 \rangle$

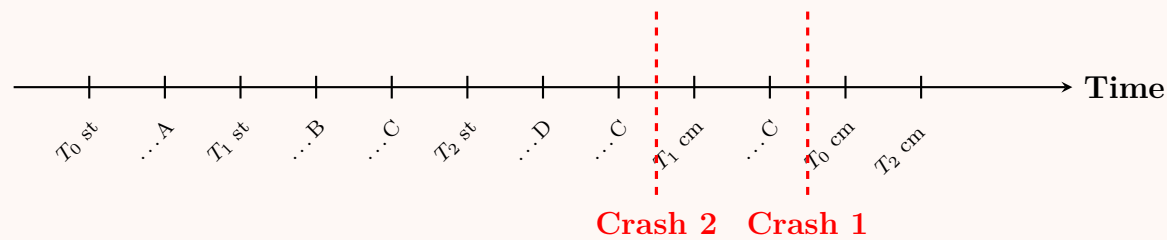
- (e) $\langle T_0, C, 50, 100 \rangle$
- (f) $\langle T_2 \text{ start} \rangle$
- (g) $\langle T_2, D, 100, 70 \rangle$
- (h) $\langle T_1, C, 20, 50 \rangle \leftarrow \text{CRASH HAPPENS HERE}$

Transaction Status:

- **Committed:** None (no commits recorded).
- **Uncommitted (Active):** T_0, T_1, T_2 (all require UNDO).

Recovery Manager Actions:

- **UNDO Actions (Scanning Backwards):**
 - (a) Undo T_1 on C : $\langle C, 20 \rangle$
 - (b) Undo T_2 on D : $\langle D, 100 \rangle$
 - (c) Undo T_0 on C : $\langle C, 50 \rangle$
 - (d) Undo T_1 on B : $\langle B, 100 \rangle$
 - (e) Undo T_0 on A : $\langle A, 40 \rangle$
- **REDO Actions:**
 - **None.** There are no committed transactions in the log at the time of the crash.

4. Visual Log Timeline

Q3. Consider the following sequence of log records:

$\langle \text{START } S \rangle; \langle S, A, 60 \rangle; \langle \text{COMMIT } S \rangle; \langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle \text{START } V \rangle; \langle U, D, 40 \rangle; \langle V, F, 70 \rangle; \langle \text{COMMIT } U \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } V \rangle; \langle V, B, 80 \rangle; \langle \text{COMMIT } T \rangle;$

Assume a system crash happens before $\langle \text{COMMIT } V \rangle$ is recorded in the log.

- (a) Using undo logging, describe the actions the recovery manager will perform after the crash. List each action as $\langle X, A \rangle$ where database element X is written with value A .
- (b) Answer (i) above for redo logging.
- (c) Why is it necessary to write an $\langle \text{ABORT} \rangle$ log record for every uncommitted transaction T after the completion of the recovery operation?

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer**1. Log State and Transaction Status at Crash**

The system crash occurs immediately before $\langle \text{COMMIT } V \rangle$. Therefore, the log available on disk to the recovery manager is:

- $\langle \text{START } S \rangle, \langle S, A, 60 \rangle, \langle \text{COMMIT } S \rangle$
- $\langle \text{START } T \rangle, \langle T, A, 10 \rangle$
- $\langle \text{START } U \rangle, \langle U, B, 20 \rangle, \langle T, C, 30 \rangle$
- $\langle \text{START } V \rangle, \langle U, D, 40 \rangle, \langle V, F, 70 \rangle, \langle \text{COMMIT } U \rangle, \langle T, E, 50 \rangle$

Transaction Status at Crash:

- **Committed Transactions:** S and U (Both have $\langle \text{COMMIT} \rangle$ records).
- **Uncommitted (Active) Transactions:** T and V (No $\langle \text{COMMIT} \rangle$ records exist for them prior to the crash).

2. Recovery Actions under Undo Logging

Principle: Undo logging requires reversing the effects of all *uncommitted* transactions. The recovery manager scans the log **backwards** from the end to the beginning. For each update record belonging to an uncommitted transaction, it writes the old value

(stored in the log) back to the database. Committed transactions are ignored.

Execution (Scanning Backwards):

- (a) $\langle T, E, 50 \rangle \implies T$ is uncommitted. **Action:** $\langle E, 50 \rangle$
- (b) $\langle \text{COMMIT } U \rangle \implies$ Ignore.
- (c) $\langle V, F, 70 \rangle \implies V$ is uncommitted. **Action:** $\langle F, 70 \rangle$
- (d) $\langle U, D, 40 \rangle \implies U$ is committed. Ignore.
- (e) $\langle T, C, 30 \rangle \implies T$ is uncommitted. **Action:** $\langle C, 30 \rangle$
- (f) $\langle U, B, 20 \rangle \implies U$ is committed. Ignore.
- (g) $\langle T, A, 10 \rangle \implies T$ is uncommitted. **Action:** $\langle A, 10 \rangle$
- (h) $\langle S, A, 60 \rangle \implies S$ is committed. Ignore.

Final Undo Actions (in order): $\langle E, 50 \rangle, \langle F, 70 \rangle, \langle C, 30 \rangle, \langle A, 10 \rangle$

3. Recovery Actions under Redo Logging

Principle: Redo logging requires reapplying the effects of all *committed* transactions. The recovery manager scans the log **forwards** from the beginning to the end. For each update record belonging to a committed transaction, it writes the new value (stored in the log) to the database. Uncommitted transactions are ignored.

Execution (Scanning Forwards):

- (a) $\langle S, A, 60 \rangle \implies S$ is committed. **Action:** $\langle A, 60 \rangle$
- (b) $\langle T, A, 10 \rangle \implies T$ is uncommitted. Ignore.
- (c) $\langle U, B, 20 \rangle \implies U$ is committed. **Action:** $\langle B, 20 \rangle$
- (d) $\langle T, C, 30 \rangle \implies T$ is uncommitted. Ignore.
- (e) $\langle U, D, 40 \rangle \implies U$ is committed. **Action:** $\langle D, 40 \rangle$
- (f) $\langle V, F, 70 \rangle \implies V$ is uncommitted. Ignore.
- (g) $\langle T, E, 50 \rangle \implies T$ is uncommitted. Ignore.

Final Redo Actions (in order): $\langle A, 60 \rangle, \langle B, 20 \rangle, \langle D, 40 \rangle$

4. Purpose of the $\langle \text{ABORT } T \rangle$ Log Record

Writing an $\langle \text{ABORT } T \rangle$ record for every uncommitted transaction after recovery operations are complete is crucial for the following reasons:

- **Lifecycle Termination:** It formally marks the end of the transaction's lifecycle in the log. Without it, the transaction would perpetually appear as "active" in the log.
- **Idempotency in Repeated Failures:** If the system crashes *again* during or immediately after the recovery process, the subsequent recovery phase will read the $\langle \text{ABORT } T \rangle$ record. This informs the recovery manager that the rollback for transaction T has already been fully processed, preventing redundant and potentially erroneous undo operations.
- **Resource Release:** It safely signals to the concurrency control and buffer management subsystems that all locks held by the transaction can be permanently released, and related memory structures can be deallocated.

Q4. Consider the following schedule where c_i represents a commit action for transaction T_i :

$w_1(A); r_2(A); w_2(A); w_2(B); r_3(B); w_3(C); c_3; c_2; c_1$

- (a) Does the given schedule preserve database consistency in case of system failure? Justify your answer.
- (b) Rearrange the commit statements so that the schedule preserves database consistency in case of system failure.
- (c) Rearrange the commit statements so that the schedule avoids cascading rollback.

(10 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Analysis of Database Consistency (Recoverability)

Answer: No, the given schedule does **not** preserve database consistency in case of a system failure. It is an **unrecoverable** schedule.

Justification: For a schedule to be recoverable (and thus preserve consistency after a failure), no transaction T_j should commit until all transactions T_i from which T_j has read have successfully committed. Let us analyze the dependencies in the given schedule:

- **Dirty Read 1:** T_2 reads data item A after T_1 writes it ($w_1(A) \rightarrow r_2(A)$). Therefore, T_2 depends on T_1 . For recoverability, c_1

must precede c_2 .

- **Dirty Read 2:** T_3 reads data item B after T_2 writes it ($w_2(B) \rightarrow r_3(B)$). Therefore, T_3 depends on T_2 . For recoverability, c_2 must precede c_3 .

In the provided schedule, the commit order is $c_3 \rightarrow c_2 \rightarrow c_1$. This directly violates the recoverability condition. If the system fails immediately after c_3 , transaction T_3 is durably committed, but it relied on uncommitted data from T_2 . If T_2 is subsequently rolled back, T_3 will have committed an inconsistent database state.

2. Rearranging Commits for a Recoverable Schedule

To make the schedule recoverable, the commit of a transaction must occur **after** the commits of all transactions it read from. Based on our dependencies ($T_1 \rightarrow T_2 \rightarrow T_3$), the valid commit sequence must be c_1 , then c_2 , then c_3 . We place them at the end of the operations.

Recoverable Schedule:

$$w_1(A); r_2(A); w_2(A); w_2(B); r_3(B); w_3(C); c_1; c_2; c_3$$

3. Rearranging Commits to Avoid Cascading Rollback

A schedule avoids cascading rollbacks (is **cascadeless**) if every transaction reads **only** those values that were written by transactions that have **already committed**. This requires us to place the commit of the writing transaction **before** the read operation of the dependent transaction.

Adjustments Needed:

- T_2 reads A from T_1 : We must place c_1 **before** $r_2(A)$.
- T_3 reads B from T_2 : We must place c_2 **before** $r_3(B)$.
- c_3 can remain at the end of the transaction.

Cascadeless Schedule:

$$w_1(A); c_1; r_2(A); w_2(A); w_2(B); c_2; r_3(B); w_3(C); c_3$$

Q5. The following is a sequence of undo-log records written by two transactions T and U :

$\langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle U, D, 40 \rangle; \langle \text{COMMIT } T \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } U \rangle$

The recovery manager takes an action $\langle X, V \rangle$ meaning database element X is restored to value V on disk. Determine the actions that will be taken by the recovery manager if there is a crash and the last log record to appear on disk is one of the following:

- $\langle \text{START } T \rangle$
- $\langle T, E, 50 \rangle$
- $\langle \text{COMMIT } T \rangle$

(9 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Principles of Undo Logging Recovery

Under standard Undo Logging, the recovery manager performs a **single backward pass** through the log (from the most recent record to the oldest). It maintains a set of committed transactions.

- When a $\langle \text{COMMIT } T \rangle$ record is encountered, T is added to the committed set.
- When an update record $\langle T, X, V \rangle$ is encountered:
 - If T is **in** the committed set, the record is ignored.
 - If T is **not in** the committed set, the update is undone by writing the old value V to object X (Action: $\langle X, V \rangle$).

2. Analysis of Crash Scenarios

Scenario (a): Crash occurs after $\langle \text{START } T \rangle$

Log on disk: $\langle \text{START } T \rangle$

Execution (Scanning Backwards):

- The only record is a START record. There are no update records to process.

Actions Taken: None

Scenario (b): Crash occurs after $\langle T, E, 50 \rangle$ **Log on disk:** $\langle \text{START } T \rangle$; $\langle T, A, 10 \rangle$; $\langle \text{START } U \rangle$; $\langle U, B, 20 \rangle$; $\langle T, C, 30 \rangle$; $\langle U, D, 40 \rangle$; $\langle \text{COMMIT } T \rangle$; $\langle T, E, 50 \rangle$ *Note: The presence of an update record ($\langle T, E, 50 \rangle$) after a COMMIT record for the same transaction is a structural anomaly. However, applying the strict single-pass backward recovery algorithm yields a specific deterministic result.***Execution (Scanning Backwards):**

- (a) Read $\langle T, E, 50 \rangle$: Transaction T is not yet in the committed set (as the commit record hasn't been encountered in this backward pass).
Action: $\langle E, 50 \rangle$
- (b) Read $\langle \text{COMMIT } T \rangle$: Add T to the committed set.
- (c) Read $\langle U, D, 40 \rangle$: U is not in the committed set.
Action: $\langle D, 40 \rangle$
- (d) Read $\langle T, C, 30 \rangle$: T is in the committed set. Ignore.
- (e) Read $\langle U, B, 20 \rangle$: U is not in the committed set.
Action: $\langle B, 20 \rangle$
- (f) Read $\langle T, A, 10 \rangle$: T is in the committed set. Ignore.

Actions Taken (in order): $\langle E, 50 \rangle$, $\langle D, 40 \rangle$, $\langle B, 20 \rangle$ **Scenario (c): Crash occurs after $\langle \text{COMMIT } T \rangle$** **Log on disk:** $\langle \text{START } T \rangle$; $\langle T, A, 10 \rangle$; $\langle \text{START } U \rangle$; $\langle U, B, 20 \rangle$; $\langle T, C, 30 \rangle$; $\langle U, D, 40 \rangle$; $\langle \text{COMMIT } T \rangle$ **Execution (Scanning Backwards):**

- (a) Read $\langle \text{COMMIT } T \rangle$: Add T to the committed set.
- (b) Read $\langle U, D, 40 \rangle$: U is not in the committed set.
Action: $\langle D, 40 \rangle$
- (c) Read $\langle T, C, 30 \rangle$: T is in the committed set. Ignore.
- (d) Read $\langle U, B, 20 \rangle$: U is not in the committed set.
Action: $\langle B, 20 \rangle$
- (e) Read $\langle T, A, 10 \rangle$: T is in the committed set. Ignore.

Actions Taken (in order): $\langle D, 40 \rangle$, $\langle B, 20 \rangle$ **Q6.** Suppose you are given the following sequence of log records representing the actions of a transaction T : $\langle \text{START } T \rangle$; $\langle T, A, 10 \rangle$; $\langle T, B, 20 \rangle$; $\langle \text{COMMIT } T \rangle$

- (a) For undo logging, what is the last log record that must be sent to disk before updated A is sent to disk?
- (b) For undo logging, what is the last log record that must be sent to disk before updated B is sent to disk?
- (c) For redo logging, what is the last log record that must be sent to disk before updated A and B are sent to disk?

(7 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Answer**1. Undo Logging Rules**

In an **Undo Logging** scheme, the primary rule for Write-Ahead Logging (WAL) dictates that before any modified database element is written to non-volatile disk storage, the corresponding undo log record (containing the old value) must first be flushed to disk. Furthermore, the actual database elements must be written to disk *before* the $\langle \text{COMMIT } T \rangle$ record is written.

(a) Last log record before updated A is sent to disk:

To write the updated value of A to disk, the log record detailing the prior state of A must already be safely on disk. Therefore, the last log record that must be sent to disk before A is updated on disk is:

 $\langle T, A, 10 \rangle$ *Note: The $\langle \text{START } T \rangle$ record will implicitly be sent before or alongside this record.***(b) Last log record before updated B is sent to disk:**

Similarly, to write the updated value of B to disk, its corresponding undo log record must be flushed first. Therefore, the last log record that must be sent to disk before B is updated on disk is:

 $\langle T, B, 20 \rangle$

2. Redo Logging Rules

In a **Redo Logging** scheme, the WAL rule is different. Because redo logging requires the ability to reapply a transaction’s changes upon recovery, a transaction must be durably committed in the log before *any* of its actual database modifications are allowed to overwrite the old data on disk. If uncommitted data were written to disk, the redo log would have no way to undo it (since it only stores new values).

(c) **Last log record before updated A and B are sent to disk:**
Under redo logging, no database element modified by transaction T can be flushed to disk until the transaction has fully committed. Thus, all log records for T , including the commit record, must be on disk first. Therefore, the last log record that must be sent to disk before either A or B is written is:

$$\langle \text{COMMIT } T \rangle$$

3. Summary Comparison

Property	Undo Logging	Redo Logging
Data Write Constraint	Must write log record $\langle T, X, v \rangle$ before writing data X .	Must write $\langle \text{COMMIT } T \rangle$ before writing any data.
Commit Constraint	Must write all data to disk before writing $\langle \text{COMMIT } T \rangle$.	Can (and must) write $\langle \text{COMMIT } T \rangle$ before writing data to disk.

Q7. Consider the following sequence of undo log records:

$$\langle \text{START } S \rangle; \langle S, A, 60 \rangle; \langle \text{COMMIT } S \rangle; \langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle \text{START } V \rangle; \langle U, D, 40 \rangle; \langle V, F, 70 \rangle; \langle \text{COMMIT } U \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } T \rangle; \langle V, B, 80 \rangle; \langle \text{COMMIT } V \rangle$$

Suppose that we begin a nonquiescent checkpoint immediately after the $\langle U, B, 20 \rangle$ log record has been written (in memory). Determine the last log record that must be sent to disk before each of the following events:

(a) $\langle \text{START CKPT}(\dots) \rangle$ is written to disk.

(b) $\langle \text{END CKPT} \rangle$ is written to disk.

Also, for a system crash occurring after (i) $\langle V, B, 80 \rangle$ and (ii) $\langle T, E, 50 \rangle$, determine the oldest log record the recovery manager needs to scan to find all possible incomplete transactions. **(15 marks)**

[CSE 215 | L-2/T-1 | 2022–23]

Answer

1. Analysis of the Nonquiescent Checkpoint

A nonquiescent checkpoint allows the database to continue processing transactions while the checkpoint is being written. It follows a specific protocol:

- **Start Checkpoint:** The recovery manager writes a $\langle \text{START CKPT}(T_1, \dots, T_k) \rangle$ record, where $T_1, \dots, T_k$ is the list of all currently active (uncommitted) transactions.
- **End Checkpoint:** The recovery manager waits until all transactions $T_1, \dots, T_k$ have either committed or aborted. Once the last one finishes, it writes the $\langle \text{END CKPT} \rangle$ record to the log.

At the moment immediately after $\langle U, B, 20 \rangle$ is written, the active transactions are T and U (since S has already committed). Therefore, the checkpoint record will be $\langle \text{START CKPT}(T, U) \rangle$.

2. Log Records Preceding Checkpoint Events

(i) **Last log record before $\langle \text{START CKPT}(T, U) \rangle$:**
Because the checkpoint begins immediately after $\langle U, B, 20 \rangle$, all previous log records must be flushed to disk before the checkpoint marker can be written. Thus, the last log record sent to disk before $\langle \text{START CKPT}(T, U) \rangle$ is:

$$\langle U, B, 20 \rangle$$

(ii) **Last log record before $\langle \text{END CKPT} \rangle$:**
The $\langle \text{END CKPT} \rangle$ record can only be written after both T and U have completed. Looking at the log:

- U commits at the $\langle \text{COMMIT } U \rangle$ record.
- T commits at the $\langle \text{COMMIT } T \rangle$ record.

Transaction T is the last of the active transactions to finish. Therefore, $\langle \text{END CKPT} \rangle$ is written to disk immediately after T ’s commit record is flushed. The last log record sent to disk before it is:

$$\langle \text{COMMIT } T \rangle$$

3. Oldest Log Record Scanned During Recovery

During Undo Logging recovery, the manager scans the log backwards. The termination condition depends on whether an $\langle \text{END CKPT} \rangle$ is encountered.

(i) **System crash after $\langle V, B, 80 \rangle$:**

At this point in the timeline, $\langle \text{COMMIT } T \rangle$ has occurred, meaning the $\langle \text{END CKPT} \rangle$ record **has been written** to disk.

- **Rule:** When a backward scan encounters an $\langle \text{END CKPT} \rangle$, it guarantees that all transactions active before the checkpoint have securely committed. The recovery manager only needs to scan backward as far as the corresponding $\langle \text{START CKPT} \rangle$ to ensure it has found all incomplete transactions (which must have started after the checkpoint).
- **Oldest record scanned:** $\langle \text{START CKPT}(T, U) \rangle$

(ii) **System crash after $\langle T, E, 50 \rangle$:**

At this point, T has not yet committed. Therefore, the $\langle \text{END CKPT} \rangle$ record **has not been written**.

- **Rule:** Without an $\langle \text{END CKPT} \rangle$, the backward scan will encounter the $\langle \text{START CKPT}(T, U) \rangle$. The recovery manager must continue scanning backwards until it has found the **START** record for every transaction listed in the checkpoint that did not commit before the crash.
- **Status:** U committed, but T did not. The scan must continue backwards past the checkpoint until it finds the start record for T .
- **Oldest record scanned:** $\langle \text{START } T \rangle$

Q8. The log records for transactions are given as follows:

1. $\langle T_0 \text{ start} \rangle$
2. $\langle T_0, A, 1000, 200 \rangle$
3. $\langle T_1 \text{ start} \rangle$
4. $\langle T_1, B, 1000, 200 \rangle$
5. $\langle T_1 \text{ commit} \rangle$
6. $\langle T_2 \text{ start} \rangle$
7. $\langle T_2, C, 1000, 200 \rangle$
8. $\langle T_3 \text{ start} \rangle$
9. $\langle T_3, D, 1000, 200 \rangle$
10. $\langle T_3 \text{ commit} \rangle$
11. $\langle T_4 \text{ start} \rangle$
12. $\langle T_3, E, 1000, 200 \rangle$
13. $\langle T_4 \text{ commit} \rangle$

- (a) Perform a checkpoint after instruction 7 (list all required log entries).
- (b) Assume a failure occurs after instruction 13. Run the log-based recovery algorithm to recover the transactions from failure, considering the checkpoint.

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Checkpoint Insertion (After Instruction 7)

A nonquiescent checkpoint records the transactions that are currently active (started but not yet committed) at the time of the checkpoint.

Let us evaluate the status of all transactions immediately after instruction 7 ($\langle T_2, C, 1000, 200 \rangle$):

- T_0 : Started (Record 1), not committed. → **Active**
- T_1 : Started (Record 3), committed (Record 5). → **Committed**
- T_2 : Started (Record 6), not committed. → **Active**

Therefore, the active transactions list is $\{T_0, T_2\}$. The required log entry to start the checkpoint is inserted directly after instruction 7:

$\langle \text{START CKPT}(T_0, T_2) \rangle$

*Note: In an immediate update/ARIES-style recovery context, the checkpoint will eventually write an $\langle \text{END CKPT} \rangle$ once the dirty memory pages at the time of the checkpoint are flushed, but the immediate log record marking its state is the **START CKPT** entry.*

2. Recovery Analysis (Failure After Instruction 13)

The full log with the inserted checkpoint and instructions up to 13 is:


```
1.  <T0 start>
2.  <T0, A, 1000, 200>
3.  <T1 start>
4.  <T1, B, 1000, 200>
5.  <T1 commit>
6.  <T2 start>
7.  <T2, C, 1000, 200>
7b. <START CKPT(T0, T2)>  <- Checkpoint Inserted
8.  <T3 start>
9.  <T3, D, 1000, 200>
10. <T3 commit>
11. <T4 start>
12. <T3, E, 1000, 200>      (Note: T3 updates after commit; treated as an active change or part of a chain)
13. <T4 commit>
```

CRASH OCCURS HERE

Step A: Transaction Classification

Scanning the log allows us to find the status of each transaction at the point of crash:

- **Committed Transactions (REDO list):** T_1, T_3, T_4
- **Uncommitted Transactions (UNDO list):** T_0, T_2

Using the checkpoint $\langle \text{START CKPT}(T_0, T_2) \rangle$, the recovery manager knows it does not need to scan backwards past the start records of T_0 and T_2 , as any older committed transactions (like T_1) are already safely on disk before the checkpoint.

Step B: Recovery Phase Execution

1. REDO Phase (Forward Scan): The recovery manager scans forward starting from the oldest active transaction at the checkpoint (T_0 , which starts at Record 1) or from the checkpoint record onward depending on the exact implementation protocol. It reapplies modifications made by committed transactions (T_3, T_4).

- Reapply instruction 9 (T_3 updates D): Set $D = 200$.
- Reapply instruction 12 (T_3 updates E): Set $E = 200$.

Actions Written to Disk: $\langle \mathbf{D}, 200 \rangle, \langle \mathbf{E}, 200 \rangle$

2. UNDO Phase (Backward Scan): The recovery manager scans backward from the crash point to undo the changes made by uncommitted transactions (T_0, T_2).

- Instruction 7 (T_2 updates C): Restore C to its old value. $\rightarrow$ Set $C = 1000$.
- Instruction 2 (T_0 updates A): Restore A to its old value. $\rightarrow$ Set $A = 1000$.

Actions Written to Disk (in sequence): $\langle \mathbf{C}, 1000 \rangle, \langle \mathbf{A}, 1000 \rangle$

Summary of Final Database Values Affected By Recovery

Object	Final Value	Reasoning
A	1000	Undone back to original state (T_0 did not commit)
C	1000	Undone back to original state (T_2 did not commit)
D	200	Redone to modified state (T_3 committed)
E	200	Redone to modified state (T_3 committed)

Q9. Explain checkpoint in database recovery. (5 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Answer

Checkpoint in Database Recovery

Definition:

A **checkpoint** is a synchronization mechanism in a database management system (DBMS) that periodically saves the current state of the database from volatile main memory to non-volatile secondary storage (disk). It acts as a reference point during the recovery process to minimize the time and overhead required to restore the database after a system crash.

The Need for Checkpoints:

In a log-based recovery system (such as Write-Ahead Logging or WAL), every transaction operation is recorded in a log file. If a system crashes, the recovery manager must read the log to determine which transactions need to be redone or undone. Without checkpoints, the system would have to scan the entire log file from the very beginning, which is highly inefficient and time-consuming. Checkpoints restrict the volume of log records that need to be processed.

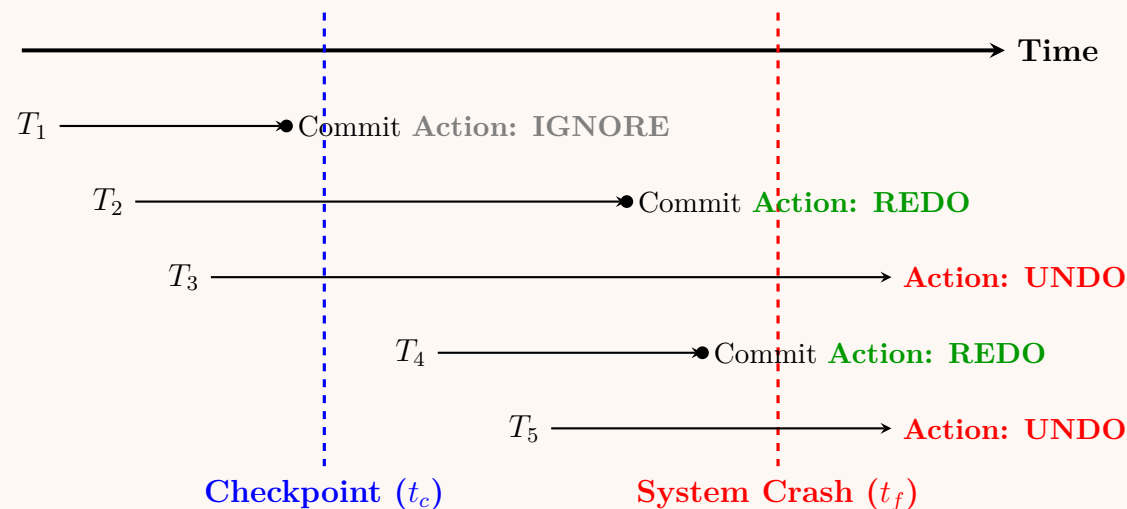
Working Principle (Checkpoint Creation):

When a DBMS schedules a checkpoint, the following sequence of actions is typically executed:

- Suspend execution:** Temporarily halt the execution of all active transactions.
- Flush log records:** Force all log records currently in main memory buffers to stable storage.
- Flush database buffers:** Write all modified (dirty) database buffers (data blocks) from main memory to the disk.
- Write checkpoint record:** Append a `<checkpoint, L>` record to the log on stable storage, where L is a list of all transactions active at the time of the checkpoint.
- Resume execution:** Allow normal transaction processing to continue.

Recovery Procedure using Checkpoints:

When recovering from a crash, the system scans the log backwards to find the most recent `<checkpoint, L>` record. Based on the time transactions started and committed relative to the checkpoint and the system crash, the recovery manager categorizes them into distinct groups:



Transaction Categories and Actions:

- T_1 : Started and committed *before* the checkpoint. Since all its modified data was safely written to disk during the checkpoint, no recovery action is needed (**Ignore**).
- T_2 : Started before the checkpoint but committed *after* the checkpoint (and before the crash). Some of its changes might still have been in the memory buffer during the crash, so it must be **Redone**.
- T_3 : Started before the checkpoint and was still active at the time of the crash. It is incomplete and its partial effects must be **Undone**.
- T_4 : Started *after* the checkpoint and committed before the crash. Its updates may not have reached the disk, so it must be **Redone**.
- T_5 : Started *after* the checkpoint and was active at the time of the crash. It is incomplete and must be **Undone**.

Advantages of Checkpoints:

- Faster Recovery:** Limits the amount of log data that needs to be scanned and processed after a crash.
- Log Truncation:** Older portions of the log file (prior to the oldest active transaction at the checkpoint) can be safely deleted or archived to free up disk space.
- Data Consistency:** Ensures a consistent state of database files on the disk at regular intervals.

Disadvantages of Checkpoints:

- System Overhead:** The process of flushing buffers to disk requires I/O operations, which can momentarily degrade system performance (often referred to as a "checkpoint stall").
- Complex Implementation:** Advanced techniques like *fuzzy checkpoints* must be implemented to avoid freezing the entire database during a checkpoint.

Q10. The following is a sequence of undo log records found on disk after a crash:

```

<START T1>; <T1, A, 10>; <START T2>; <T2, B, 5>; <T1, C, 7>;
<START T3>; <T3, D, 12>; <COMMIT T1>; <START CKPT ?>;
<START T4>; <T2, E, 5>; <COMMIT T2>; <T3, F, 1>; <T4, G, 15>;
<END CKPT>; <COMMIT T3>; <START T5>; <T5, H, 3>;
<START CKPT ?>; <COMMIT T5>

```

- What are the correct values of the two “?”-s in the log record?
- Describe the action of the recovery manager, showing changes to both disk and log.

(12 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer**1. Values of the “?” in the Checkpoint Records**

In an undo logging scheme, the <START CKPT list> record contains the list of all transactions that are **currently active** (i.e., they have started but have not yet committed or aborted) at the exact moment the checkpoint begins.

- **First Checkpoint (Log Record 9):**

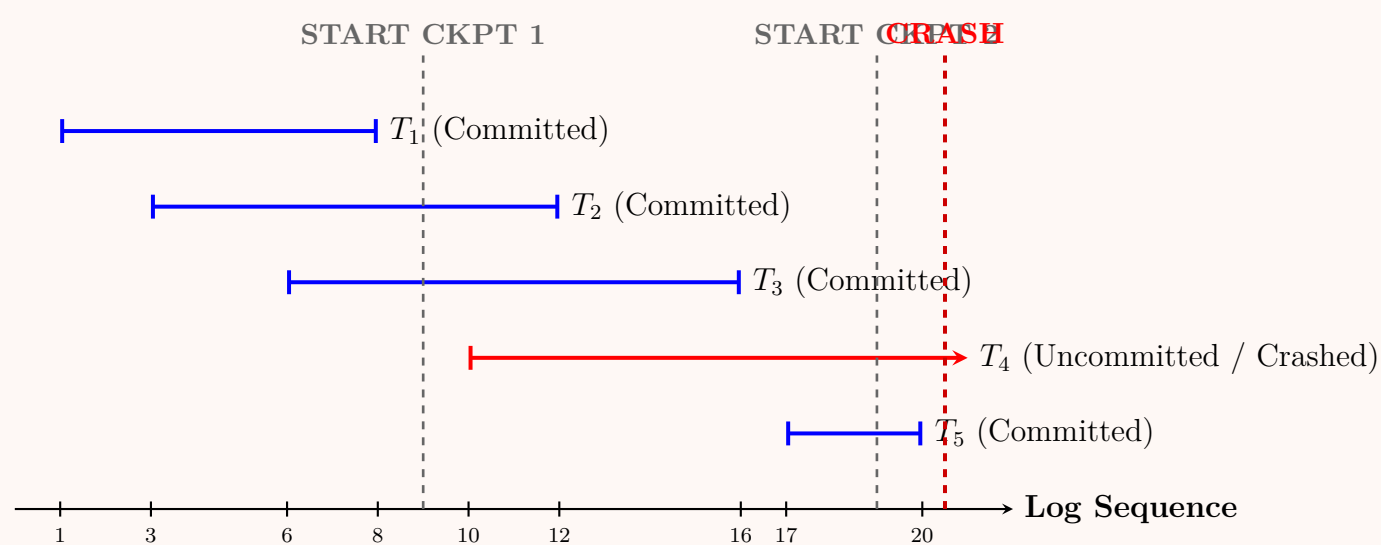
Scanning the log up to record 9, transactions T_1 , T_2 , and T_3 have started. However, T_1 commits at record 8. Therefore, the active transactions at the start of this checkpoint are T_2 and T_3 .

– First “?” value: {T2, T3}

- **Second Checkpoint (Log Record 19):**

Scanning the log up to record 19, transactions T_1 , T_2 , T_3 , T_4 , and T_5 have started. T_1 , T_2 , and T_3 have already committed (at records 8, 12, and 16, respectively). The active transactions at the start of this checkpoint are T_4 and T_5 .

– Second “?” value: {T4, T5}

Transaction Timeline Visualization:**2. Actions of the Recovery Manager**

In **Undo Logging**, the recovery manager scans the log backwards from the point of the crash to undo any transactions that were active (uncommitted) at the time of the crash. Since the old values are recorded in the log, they are written back to the disk.

Backward Scan Execution:

- The recovery manager maintains a set of *committed transactions*, initially empty.
- It begins scanning backwards from the end of the log (after record 20).
- It encounters <COMMIT T5>, <COMMIT T3>, and <COMMIT T2>. These transactions are added to the committed set.
- Transaction T_4 is never committed; thus, it must be **undone**.
- According to the second checkpoint <START CKPT {T4, T5}>, the recovery manager knows it only needs to scan backwards as far as the earliest <START> record among the uncommitted transactions active at that checkpoint. Therefore, it must scan back to <START T4>.

Step-by-step Log Processing (Backwards):

Log Record	Direction	Recovery Manager Action
<COMMIT T5>	Backward	Mark T_5 as committed.
<START CKPT {T4, T5}>	Backward	Note active transactions. Since T_4 is uncommitted, scan must continue to at least <START T4>.
<T5, H, 3>	Backward	Ignore (since T_5 is committed).
<START T5>	Backward	Ignore.
<COMMIT T3>	Backward	Mark T_3 as committed.
<END CKPT>	Backward	Ignore.
<T4, G, 15>	Backward	Undo T_4: Write old value 15 to variable G on disk.
<T3, F, 1>	Backward	Ignore (since T_3 is committed).
<COMMIT T2>	Backward	Mark T_2 as committed.
<T2, E, 5>	Backward	Ignore (since T_2 is committed).
<START T4>	Backward	T_4 is completely undone. Stop scanning for T_4 .

Summary of System Changes:

- **Changes to Disk:**
 - The data item **G** is updated on the disk to the value **15**.
- **Changes to Log:**
 - The record <ABORT T4> is appended to the log on stable storage to denote that T_4 has been successfully and fully undone, finalizing the recovery process.

Q11. The initial account balances of A_1 , A_2 and A_3 are 1000, 2000 and 3000 respectively. Two serial transactions are given:

T20	T21
READ(A_1) $A_1 = A_1 + 100$ WRITE(A_1) READ(A_3) $A_3 = A_3 - 100$ WRITE(A_3) COMMIT	READ(A_2) $A_2 = A_2 + 0.1 \times A_2$ WRITE(A_2) COMMIT

- (a) Write log records for the above transactions.
- (b) Show the log-based recovery if a failure occurs after WRITE(A_3).
- (c) Show the recovery if a failure occurs after WRITE(A_2).

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Log Records for the Transactions

To provide a comprehensive recovery strategy, we assume an **Undo/Redo logging scheme** with immediate database modification. In this scheme, log records for updates take the format $\langle T_i, X, V_{old}, V_{new} \rangle$, where T_i is the transaction, X is the data item, V_{old} is the old value, and V_{new} is the new value.

Given the initial balances $A_1 = 1000$, $A_2 = 2000$, and $A_3 = 3000$, the complete log sequence is as follows:

Log Seq.	Log Record	Explanation / Internal State
1	$\langle \text{START } T_{20} \rangle$	T_{20} begins execution.
2	$\langle T_{20}, A_1, 1000, 1100 \rangle$	A_1 updated ($1000 + 100$).
3	$\langle T_{20}, A_3, 3000, 2900 \rangle$	A_3 updated ($3000 - 100$).
4	$\langle \text{COMMIT } T_{20} \rangle$	T_{20} successfully commits.
5	$\langle \text{START } T_{21} \rangle$	T_{21} begins execution.
6	$\langle T_{21}, A_2, 2000, 2200 \rangle$	A_2 updated ($2000 + 0.1 \times 2000$).
7	$\langle \text{COMMIT } T_{21} \rangle$	T_{21} successfully commits.

2. Recovery if Failure Occurs After $\text{WRITE}(A_3)$

Failure Point: The system crashes exactly after T_{20} executes $\text{WRITE}(A_3)$, which corresponds to the point just before log sequence 4 ($\langle \text{COMMIT } T_{20} \rangle$).

State of the Log at Crash:

1. $\langle \text{START } T_{20} \rangle$
2. $\langle T_{20}, A_1, 1000, 1100 \rangle$
3. $\langle T_{20}, A_3, 3000, 2900 \rangle$

Recovery Action:

- **Analysis:** The recovery manager scans the log and finds a **START** record for T_{20} but no corresponding **COMMIT** record. Therefore, T_{20} was active at the time of the crash and must be **undone**.
- **Undo Phase (Backward Scan):**
 - Read record 3: Restore A_3 to its old value: $A_3 \leftarrow 3000$.
 - Read record 2: Restore A_1 to its old value: $A_1 \leftarrow 1000$.
 - Read record 1: Reached the start of T_{20} .
- **Result:** A record $\langle \text{ABORT } T_{20} \rangle$ is appended to the log. The database is restored to its consistent initial state ($A_1 = 1000, A_3 = 3000$).

3. Recovery if Failure Occurs After $\text{WRITE}(A_2)$

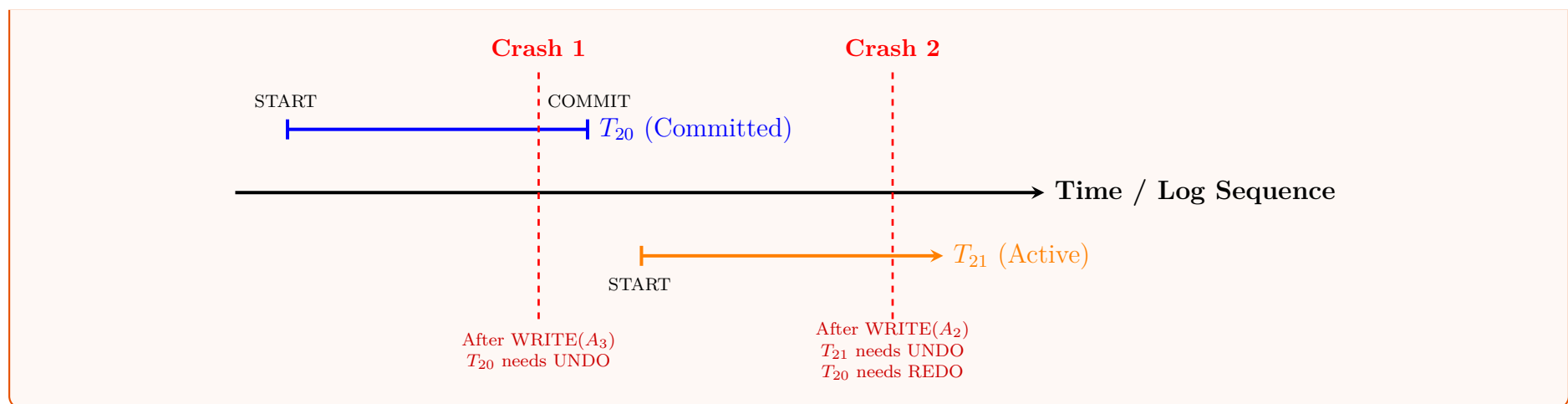
Failure Point: The system crashes exactly after T_{21} executes $\text{WRITE}(A_2)$, which corresponds to the point just before log sequence 7 ($\langle \text{COMMIT } T_{21} \rangle$).

State of the Log at Crash:

1. $\langle \text{START } T_{20} \rangle$
2. $\langle T_{20}, A_1, 1000, 1100 \rangle$
3. $\langle T_{20}, A_3, 3000, 2900 \rangle$
4. $\langle \text{COMMIT } T_{20} \rangle$
5. $\langle \text{START } T_{21} \rangle$
6. $\langle T_{21}, A_2, 2000, 2200 \rangle$

Recovery Action:

- **Analysis:** The recovery manager scans the log. It finds both **START** and **COMMIT** records for T_{20} , meaning T_{20} must be **redone**. It finds a **START** for T_{21} but no **COMMIT**, meaning T_{21} must be **undone**.
- **Undo Phase (Backward Scan):**
 - Read record 6: Restore A_2 to its old value: $A_2 \leftarrow 2000$.
 - Record 5 marks the end of undo for T_{21} . An $\langle \text{ABORT } T_{21} \rangle$ is appended to the log.
- **Redo Phase (Forward Scan):**
 - The system scans forward to redo committed transactions.
 - Read record 2: Set $A_1 \leftarrow 1100$.
 - Read record 3: Set $A_3 \leftarrow 2900$.
- **Result:** T_{20} 's changes are permanently applied, and T_{21} 's partial changes are completely reversed. Final database state: $A_1 = 1100, A_2 = 2000, A_3 = 2900$.



Q12. After a system crash, the following log records are found:

$\langle T_0, \text{start} \rangle$; $\langle T_0, \text{commit} \rangle$; $\langle T_1, \text{start} \rangle$; $\langle T_1, C, 800, 600 \rangle$

Describe the recovery actions for the above log records. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

Log Analysis and Recovery Actions

The given sequence of log records indicates an **Undo/Redo (Immediate Database Modification)** logging scheme because the data update log record $\langle T_1, C, 800, 600 \rangle$ contains both the old value (800) and the new value (600).

1. Transaction State Identification (Analysis Phase): By scanning the log sequence from beginning to end (or analyzing it after the crash), the recovery manager categorizes the transactions based on their completion status:

- **Transaction T_0 :** The log contains both $\langle T_0, \text{start} \rangle$ and $\langle T_0, \text{commit} \rangle$.
 - **Status:** Committed.
 - **Action Required:** Redo.
- **Transaction T_1 :** The log contains $\langle T_1, \text{start} \rangle$ and an update operation $\langle T_1, C, 800, 600 \rangle$, but *no commit record*.
 - **Status:** Active (Uncommitted) at the time of the crash.
 - **Action Required:** Undo.

2. Execution of Recovery Actions:

The recovery manager performs the following operations to restore the database to a consistent state:

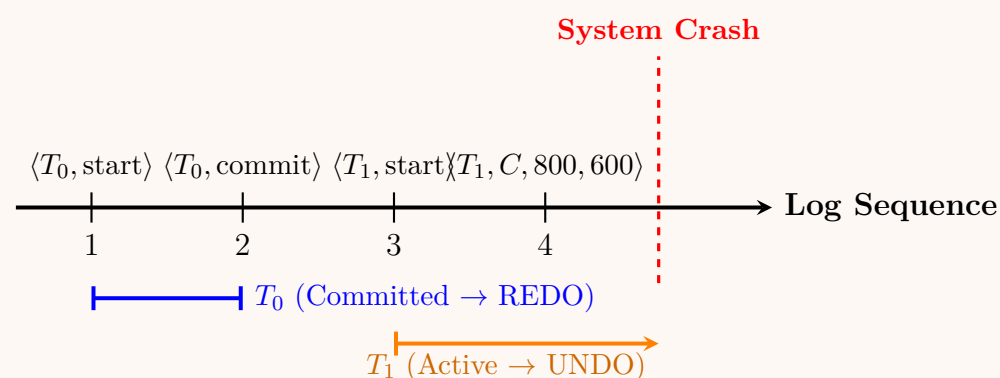
(a) Undo Phase (Backward Scan):

- The system scans the log backwards starting from the crash point.
- It encounters the record $\langle T_1, C, 800, 600 \rangle$.
- It **undoes** the operation by writing the old value of data item C back to the database: $C \leftarrow 800$.
- Once it reaches $\langle T_1, \text{start} \rangle$, the undo process for T_1 is complete.
- A log record $\langle T_1, \text{abort} \rangle$ is appended to the log on the disk to indicate T_1 has been successfully rolled back.

(b) Redo Phase (Forward Scan):

- The system scans the log forward from the beginning to reapply changes made by committed transactions.
- It identifies T_0 as a committed transaction.
- *Note:* Although T_0 must formally be redone, there are no actual data modification log records associated with T_0 in the given sequence. Therefore, no physical disk writes are required for T_0 in this specific scenario.

Timeline Visualization:



Final State Summary:

- **Disk Database:** The data item C retains (or is reverted to) the value 800.
- **Log File:** An $\langle T_1, \text{abort} \rangle$ record is added. The database is in a consistent state without the partial updates of T_1 .

Q13. The following is a sequence of undo/redo-log records written by two transactions T and U :

$\langle T, \text{start} \rangle$; $\langle T, A, 10, 11 \rangle$; $\langle U, \text{start} \rangle$; $\langle U, B, 20, 21 \rangle$; $\langle T, C, 30, 31 \rangle$;
 $\langle U, D, 40, 41 \rangle$; $\langle U, \text{commit} \rangle$; $\langle T, E, 50, 51 \rangle$; $\langle T, \text{commit} \rangle$

Describe the action of the recovery manager, including changes to both disk and the log, if there is a crash and the last log record to appear on disk is:

- (a) $\langle U, \text{start} \rangle$
- (b) $\langle U, \text{commit} \rangle$
- (c) $\langle T, E, 50, 51 \rangle$
- (d) $\langle T, \text{commit} \rangle$

(12 marks)

[CSE 303 | L-3/T-1 | 2013–14]

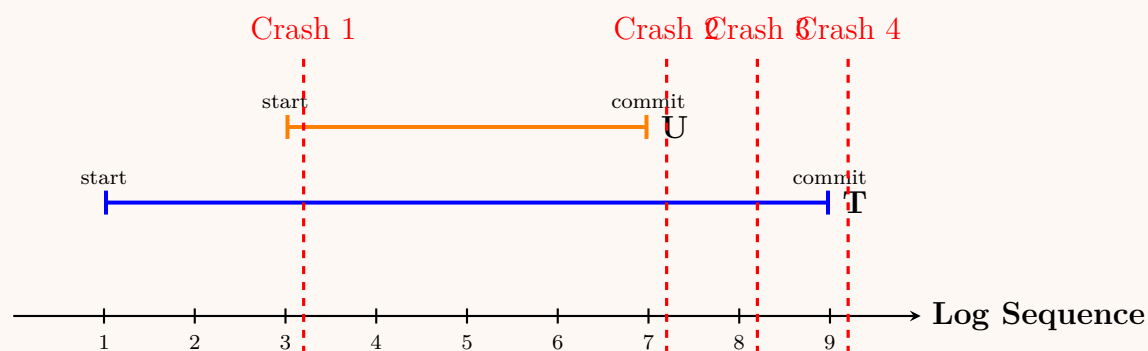
Answer

Undo/Redo Recovery Protocol Overview

In an **Undo/Redo** logging scheme, log records for updates follow the format $\langle T_i, X, V_{\text{old}}, V_{\text{new}} \rangle$. During recovery from a crash:

- **Analysis Phase:** The log is scanned to identify transactions that have committed and those that were still active (uncommitted).
- **Undo Phase:** Uncommitted transactions are undone by scanning the log backwards and writing the V_{old} to disk.
- **Redo Phase:** Committed transactions are redone by scanning the log forwards and writing the V_{new} to disk.

Transaction Timeline and Crash Points:



1. Crash occurs after $\langle U, \text{start} \rangle$

The log on disk contains records 1 through 3.

- **Analysis:** Transactions T and U both have a $\langle \text{start} \rangle$ record but no $\langle \text{commit} \rangle$ record. Both are uncommitted and must be undone. Neither needs to be redone.
- **Undo Actions (Backward Scan):**
 - U has made no data modifications yet.
 - Undo T 's change to A (record 2).
- **Changes to Disk:** Variable $A \leftarrow 10$.
- **Changes to Log:** Append $\langle U, \text{abort} \rangle$ and $\langle T, \text{abort} \rangle$.

2. Crash occurs after $\langle U, \text{commit} \rangle$

The log on disk contains records 1 through 7.

- **Analysis:** Transaction U has a $\langle \text{commit} \rangle$ record, so it must be **redone**. Transaction T lacks a commit record, so it must be **undone**.
- **Undo Actions (Backward Scan for T):**
 - Undo T 's change to C (record 5).
 - Undo T 's change to A (record 2).
- **Redo Actions (Forward Scan for U):**
 - Redo U 's change to B (record 4).
 - Redo U 's change to D (record 6).
- **Changes to Disk:** $C \leftarrow 30$, $A \leftarrow 10$, $B \leftarrow 21$, $D \leftarrow 41$.
- **Changes to Log:** Append $\langle T, \text{abort} \rangle$.

3. Crash occurs after $\langle T, E, 50, 51 \rangle$

The log on disk contains records 1 through 8.

- **Analysis:** Transaction U is committed (needs **redo**). Transaction T is still active (needs **undo**).
- **Undo Actions (Backward Scan for T):**
 - Undo T 's change to E (record 8).
 - Undo T 's change to C (record 5).
 - Undo T 's change to A (record 2).
- **Redo Actions (Forward Scan for U):**
 - Redo U 's change to B (record 4).
 - Redo U 's change to D (record 6).
- **Changes to Disk:** $E \leftarrow 50$, $C \leftarrow 30$, $A \leftarrow 10$, $B \leftarrow 21$, $D \leftarrow 41$.
- **Changes to Log:** Append $\langle T, \text{abort} \rangle$.

4. Crash occurs after $\langle T, \text{commit} \rangle$

The log on disk contains records 1 through 9.

- **Analysis:** Both transaction T and transaction U possess $\langle \text{commit} \rangle$ records. Therefore, both must be **redone**. No transactions require undo.
- **Undo Actions:** None.
- **Redo Actions (Forward Scan for T and U):**
 - Redo T 's change to A (record 2).
 - Redo U 's change to B (record 4).
 - Redo T 's change to C (record 5).
 - Redo U 's change to D (record 6).
 - Redo T 's change to E (record 8).
- **Changes to Disk:** $A \leftarrow 11$, $B \leftarrow 21$, $C \leftarrow 31$, $D \leftarrow 41$, $E \leftarrow 51$.
- **Changes to Log:** None (the log already correctly reflects that all active transactions have committed).

Q14. What properties of a recovery system allow us to recover from a crash while recovering a previous crash? What are the purposes of checkpointing and nonquiescent checkpointing? (10 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer**1. Recovering from a Crash During Recovery**

To successfully recover from a system crash that occurs *while* a previous recovery process is ongoing, the recovery system must possess the property of **idempotence** and ensure **forward progress**.

Key Mechanisms Enabling Idempotence:

- **Idempotent Redo Operations (via LSNs):** Modern recovery algorithms (like ARIES) associate a **Log Sequence Number (LSN)** with every log record and store a **PageLSN** on every database page. During the REDO phase, an update is only applied if the log record's LSN is strictly greater than the **PageLSN** on disk. If the system crashes and restarts the REDO phase, previously applied updates are simply ignored. Thus, executing REDO multiple times yields the exact same database state as executing it once.
- **Compensation Log Records (CLRs) for Undo:** When rolling back an uncommitted transaction during the UNDO phase, the recovery manager writes a CLR to the log to record the undo action itself. A CLR is a redo-only log record. If the system crashes during the UNDO phase, the subsequent recovery's REDO phase will process the CLR and re-apply the undo action. This ensures the system does not attempt to undo an already-undone operation, providing **forward progress** even across multiple consecutive crashes.

2. Purposes of Checkpointing

A **checkpoint** establishes a synchronized point where the data on non-volatile storage (disk) is guaranteed to reflect the recorded state of the transaction log.

Primary Purposes:

- (a) **Minimize Recovery Time:** By periodically flushing modified (dirty) buffers to disk, a checkpoint bounds the amount of log that must be scanned and processed during the REDO phase. The recovery manager only needs to scan the log going back to the most recent checkpoint (or the oldest uncommitted transaction at that checkpoint).
- (b) **Log Truncation:** Checkpointing identifies old sections of the log that are no longer needed for recovery. These obsolete log files can be archived or deleted, freeing up valuable disk space.

3. Purposes of Non-Quiescent (Fuzzy) Checkpointing

Standard (Quiescent) Checkpointing requires suspending all incoming transactions and waiting for currently executing transactions to pause while dirty buffers are written to disk. This causes a significant "system freeze," severely degrading database performance.

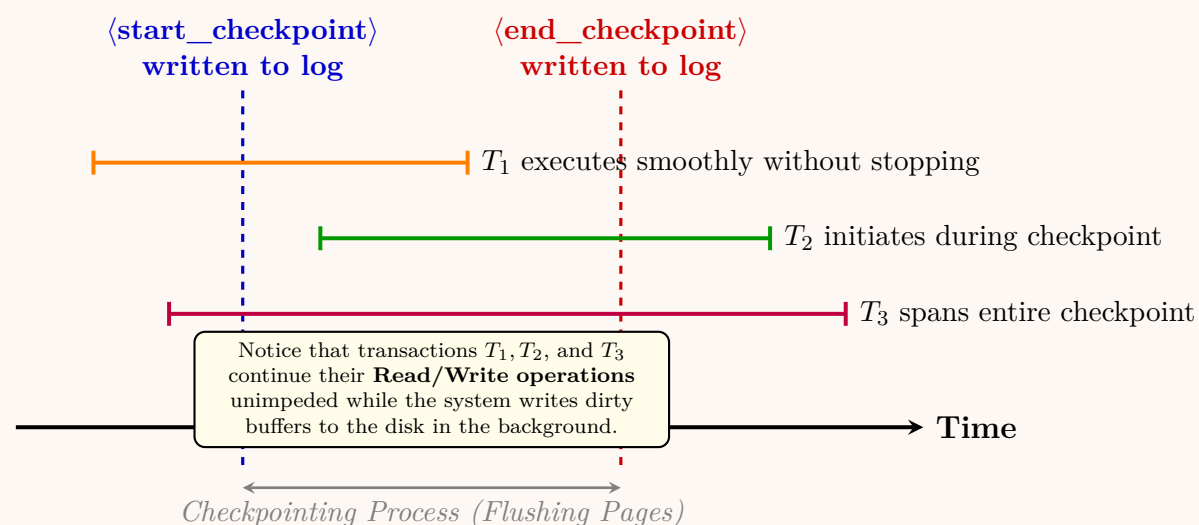
Non-Quiescent Checkpointing (often called *Fuzzy Checkpointing*) solves this by allowing transactions to continue executing concurrently with the checkpointing process.

Primary Purposes:

- **High Availability and Throughput:** Eliminates the need to halt the database, preventing latency spikes for end-users and applications.
- **Background Processing:** Allows the writing of dirty pages to disk to be stretched out over time as a background process, avoiding massive, sudden I/O bottlenecks.

Working Principle of Non-Quiescent Checkpointing:

- (a) Write a $\langle \text{start_checkpoint}(T_{\text{active}}) \rangle$ record to the log, containing the list of currently active transactions.
- (b) Continue normal transaction processing. Concurrently, flush the dirty database blocks to disk.
- (c) Once all dirty blocks identified at the start of the checkpoint have been flushed, write an $\langle \text{end_checkpoint} \rangle$ record to the log.



Q15. The following is a sequence of undo/redo-log records written by two transactions T and U :

$\langle T, \text{start} \rangle$; $\langle T, A, 10, 11 \rangle$; $\langle U, \text{start} \rangle$; $\langle U, B, 20, 21 \rangle$; $\langle T, C, 30, 31 \rangle$;
 $\langle U, D, 40, 41 \rangle$; $\langle U, \text{commit} \rangle$; $\langle T, E, 50, 51 \rangle$; $\langle T, \text{commit} \rangle$

Describe the action of the recovery manager, including changes to both the disk and log, when the last log record to appear on disk is:

- (a) $\langle U, \text{commit} \rangle$
 (b) $\langle T, E, 50, 51 \rangle$

(12 marks)

[CSE 303 | L-3/T-1 | 2011–12]

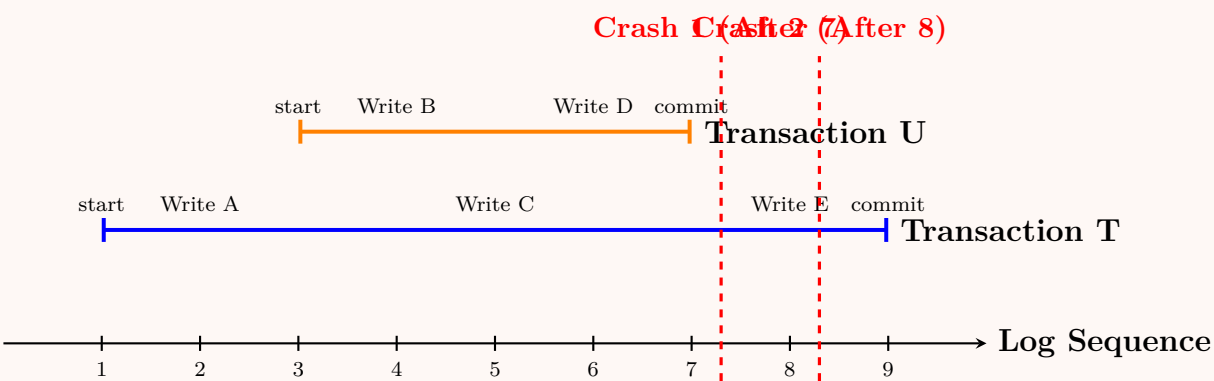
Answer

Undo/Redo Logging Recovery Protocol

In an **Undo/Redo** logging scheme, update records take the form $\langle T_i, X, V_{\text{old}}, V_{\text{new}} \rangle$. During recovery, the system must process the log to ensure database consistency:

- **Analysis Phase:** Identifies which transactions are committed and which are uncommitted (active) at the time of the crash.
- **Undo Phase:** Scans *backwards* to undo the effects of uncommitted transactions by writing V_{old} to disk.
- **Redo Phase:** Scans *forwards* to redo the effects of committed transactions by writing V_{new} to disk.

Transaction Timeline and Log Sequence:



1. Crash occurs after $\langle U, \text{commit} \rangle$

The log on disk contains records 1 through 7.

- **Analysis Phase:** Transaction U has a $\langle U, \text{commit} \rangle$ record, so it is **committed** and must be **redone**. Transaction T has a $\langle T, \text{start} \rangle$ record but no commit record, so it is **uncommitted** and must be **undone**.
- **Undo Actions (Backward Scan for T):**
 - Read record 5: Undo T 's change to C . Set $C \leftarrow 30$.
 - Read record 2: Undo T 's change to A . Set $A \leftarrow 10$.
- **Redo Actions (Forward Scan for U):**
 - Read record 4: Redo U 's change to B . Set $B \leftarrow 21$.
 - Read record 6: Redo U 's change to D . Set $D \leftarrow 41$.
- **Changes to Disk:** The values on disk are definitively updated to: $C = 30$, $A = 10$, $B = 21$, $D = 41$.
- **Changes to Log:** A log record $\langle T, \text{abort} \rangle$ is appended to the log on stable storage to finalize the rollback of transaction T .

2. Crash occurs after $\langle T, E, 50, 51 \rangle$

The log on disk contains records 1 through 8.

- **Analysis Phase:** Similar to the previous scenario, transaction U has a $\langle U, \text{commit} \rangle$ record and must be **redone**. Transaction T still lacks a commit record, so it remains **uncommitted** and must be **undone**.
- **Undo Actions (Backward Scan for T):**
 - Read record 8: Undo T 's change to E . Set $E \leftarrow 50$.
 - Read record 5: Undo T 's change to C . Set $C \leftarrow 30$.
 - Read record 2: Undo T 's change to A . Set $A \leftarrow 10$.
- **Redo Actions (Forward Scan for U):**
 - Read record 4: Redo U 's change to B . Set $B \leftarrow 21$.
 - Read record 6: Redo U 's change to D . Set $D \leftarrow 41$.
- **Changes to Disk:** The values on disk are updated to reflect the undone and redone states: $E = 50$, $C = 30$, $A = 10$, $B = 21$, $D = 41$.
- **Changes to Log:** A log record $\langle T, \text{abort} \rangle$ is appended to the log on stable storage, marking the complete and successful rollback of transaction T .

Q16. Describe log-based recovery methodology. For the given transactions $\langle T_1, T_2, T_3 \rangle$, write the log records.

T1	T3	T2
Read(A)	Read(A)	Read(B)
A = A - 50	DISPLAY(A)	Read(A)
Write(A)		B = B + 50
		Write(B)
		DISPLAY(A+B)

Answer**1. Log-Based Recovery Methodology**

Log-based recovery is a mechanism used by Database Management Systems (DBMS) to guarantee atomicity and durability in the presence of system failures. It relies on maintaining a highly structured log on stable (non-volatile) storage.

Working Principle and Write-Ahead Logging (WAL): The cornerstone of log-based recovery is the **Write-Ahead Logging (WAL)** protocol, which enforces the following rules:

- Before any data modification is permanently flushed from main memory to the database disk, the corresponding log record must be safely written to the log on stable storage.
- A transaction is not considered committed until its commit log record has been written to stable storage.

Standard Log Record Formats:

- $\langle T_i, \text{start} \rangle$: Indicates transaction T_i has begun.
- $\langle T_i, X, V_{\text{old}}, V_{\text{new}} \rangle$: An update record indicating T_i modified data item X from its old value (V_{old}) to its new value (V_{new}).
- $\langle T_i, \text{commit} \rangle$: Indicates transaction T_i completed its execution successfully.
- $\langle T_i, \text{abort} \rangle$: Indicates transaction T_i could not complete and has been rolled back.

Recovery Actions During Restart: Upon recovering from a crash, the recovery manager scans the log to categorize transactions:

- Redo Phase:** Transactions with both a **start** and a **commit** record are redone (by applying V_{new}) to ensure their changes persist on disk.
- Undo Phase:** Transactions with a **start** but no **commit** record are active/failed and must be undone (by applying V_{old}) via a backward scan to restore consistency.

2. Log Records for the Given Schedule

To write the log records, we must remember that **only database modification operations (Writes) and transaction control states (Starts, Commits, Aborts)** are written to the log. Read operations and local computations (such as `DISPLAY` or local arithmetic like $A = A - 50$) are **not** logged.

Assuming the initial value of A is A_0 and the initial value of B is B_0 , and reading the operations chronologically row-by-row as a time-based schedule, we derive the following log sequence:

Seq.	Log Record	Explanation
1	$\langle T_1, \text{start} \rangle$	T_1 begins its first operation (Read A).
2	$\langle T_3, \text{start} \rangle$	T_3 begins its first operation (Read A).
3	$\langle T_2, \text{start} \rangle$	T_2 begins its first operation (Read B).
4	$\langle T_3, \text{commit} \rangle$	T_3 finishes after Row 2 (local Display).
5	$\langle T_1, A, A_0, A_0 - 50 \rangle$	T_1 executes <code>Write(A)</code> in Row 3.
6	$\langle T_1, \text{commit} \rangle$	T_1 completes its execution after its Write.
7	$\langle T_2, B, B_0, B_0 + 50 \rangle$	T_2 executes <code>Write(B)</code> in Row 4.
8	$\langle T_2, \text{commit} \rangle$	T_2 finishes after Row 5 (local Display).

Note: Depending on the exact concurrency control implementation, the start records and commit records might be grouped slightly differently in time, but the relative order of updates matching the rows provided must be preserved exactly as shown above.

S8 — Advanced Topics**Distributed & Parallel Databases**

Q1. A distributed database system consists of five sites (S_1 – S_5). A transaction T_1 initiated from site S_1 reads and writes data items A (fragmented in S_2 and S_4) and B (fragmented in S_2 , S_3 and S_5). Any write operation must be performed to all related sites.

- Show all the messages (e.g. $\langle \text{Prepare } T_1 \rangle$, etc.) passing between S_1 and the related sites (S_2, S_3, S_4, S_5) for Phase 1 and Phase 2 to **COMMIT** the transaction T_1 using the two-phase commit protocol.
- Show all the message passing for Phase 1 and Phase 2 to **ABORT** the transaction T_1 because of an $\langle \text{Abort } T_1 \rangle$ message sent from S_5 to S_1 .

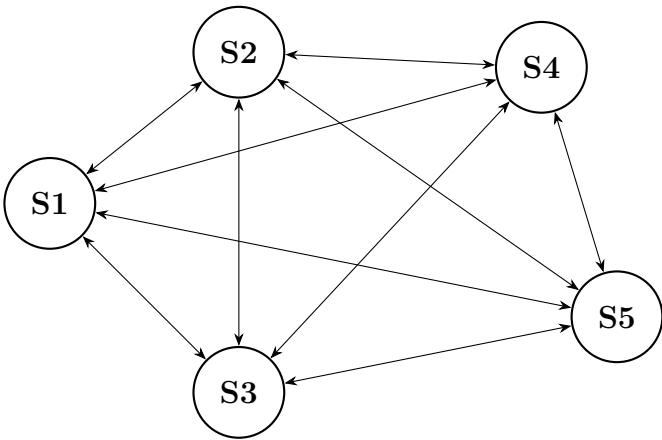


Figure 1: Distributed Database System consisting of five sites

T1	Data replication
READ (A)	Data item A is fragmented to S2, S4
READ (B)	
WRITE (A)	Data item B is fragmented to S2, S3, S5
WRITE (B)	
COMMIT	

Figure 2: A transaction T1 initiated from S1 of Figure 1.

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Answer

Overview of the Transaction Setup

Transaction T_1 is initiated at site S_1 . S_1 acts as the **Coordinator** for the Two-Phase Commit (2PC) protocol. Because T_1 writes to both data items A and B , and write operations must be performed on all related sites, the **Participants** involved in this transaction are:

- **For Data A:** S_2, S_4
- **For Data B:** S_2, S_3, S_5

The complete set of participant sites is $\{S_2, S_3, S_4, S_5\}$.

1. Message Passing to COMMIT Transaction T_1

For a successful commit, all participants must vote to commit the transaction during the Prepare phase.

Phase 1: Voting / Prepare Phase

Direction	Message Passed
$S_1 \rightarrow S_2, S_3, S_4, S_5$	$\langle \text{Prepare } T_1 \rangle$
$S_2 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_3 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_4 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_5 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$

Phase 2: Decision / Commit Phase

Direction	Message Passed
$S_1 \rightarrow S_2, S_3, S_4, S_5$	$\langle \text{Commit } T_1 \rangle$
$S_2 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$
$S_3 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$
$S_4 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$
$S_5 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$

2. Message Passing to ABORT Transaction T_1

If any single participant votes to abort, the coordinator must abort the entire transaction. In this scenario, S_5 issues an abort message during Phase 1.

Phase 1: Voting / Prepare Phase

Direction	Message Passed
$S_1 \rightarrow S_2, S_3, S_4, S_5$	$\langle \text{Prepare } T_1 \rangle$
$S_2 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_3 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_4 \rightarrow S_1$	$\langle \text{Ready } T_1 \rangle$
$S_5 \rightarrow S_1$	$\langle \text{Abort } T_1 \rangle$ <i>(This triggers the global abort)</i>

Phase 2: Decision / Abort Phase

Direction	Message Passed
$S_1 \rightarrow S_2, S_3, S_4$	$\langle \text{Abort } T_1 \rangle$
$S_2 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$
$S_3 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$
$S_4 \rightarrow S_1$	$\langle \text{Ack } T_1 \rangle$

Note: The coordinator S_1 typically only needs to send the $\langle \text{Abort } T_1 \rangle$ command to the participants that voted $\langle \text{Ready } T_1 \rangle$. Since S_5 initiated the abort locally, it already knows the outcome.

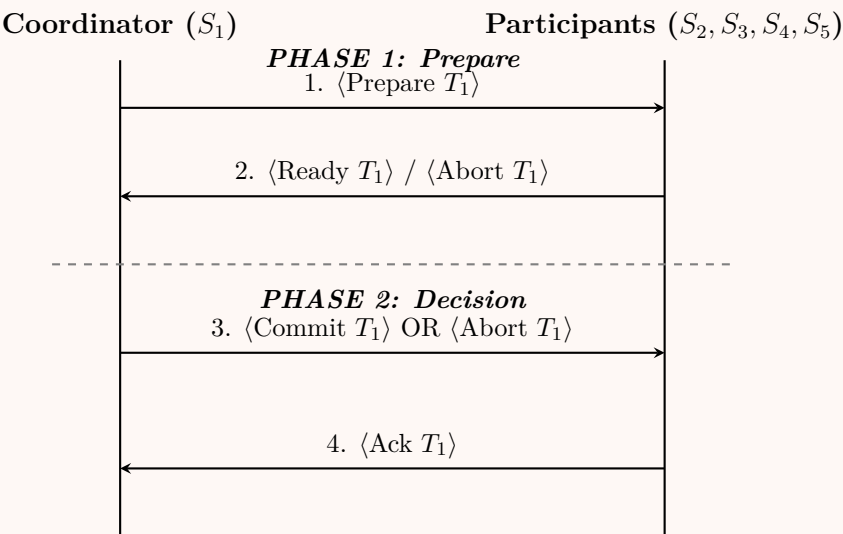


Figure: Generalized Sequence Diagram of the Two-Phase Commit (2PC) Protocol

Q2. Given relations $R_1(A, B, C, D)$ and $R_2(C, D, E)$ stored in sites S_1 and S_2 . Describe the join operation of R_1 and R_2 using the semi-join strategy. Give an example of parallel join. **(10 marks)** [CSE 303 | L-3/T-1 | 2010–11]

Answer

1. Semi-Join Strategy for Distributed Databases

In a distributed database, computing a join between two relations located at different sites (such as R_1 at Site S_1 and R_2 at Site S_2) can result in enormous data transfer over the network. The **semi-join strategy** is used to reduce the amount of data transmitted by filtering out tuples that will not participate in the final join before the full relations are transferred.

Given:

- Relation $R_1(A, B, C, D)$ at Site S_1
- Relation $R_2(C, D, E)$ at Site S_2

The common (join) attributes are C and D . Let us assume the final join result $R_1 \bowtie R_2$ is required at Site S_2 . The semi-join approach ($\ltimes$) proceeds in the following steps:

(a) **Projection at S_2 :** Site S_2 computes the projection of R_2 on the join attributes:

$$L = \Pi_{C,D}(R_2)$$

(b) **Data Transfer ($S_2 \rightarrow S_1$):** Site S_2 sends the projected relation L over the network to Site S_1 . Since L contains only two attributes and no duplicates, its size is much smaller than the entire R_2 .

(c) **Semi-Join at S_1 :** Site S_1 computes the semi-join of R_1 with L to filter out tuples in R_1 that will not match any tuple in R_2 :

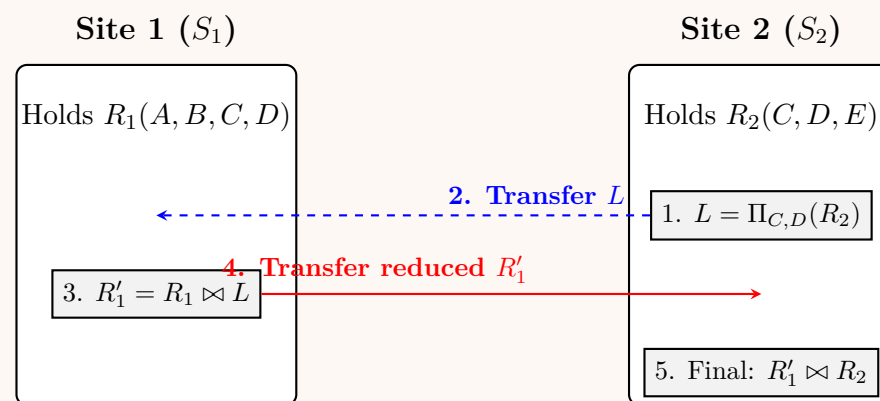
$$R'_1 = R_1 \ltimes R_2 = R_1 \bowtie L$$

R'_1 is the reduced version of R_1 , containing only the tuples guaranteed to participate in the final join.

(d) **Data Transfer ($S_1 \rightarrow S_2$):** Site S_1 sends the reduced relation R'_1 back to Site S_2 . This drastically reduces network I/O compared to sending the full R_1 .

(e) **Final Join at S_2 :** Site S_2 computes the final join using the reduced relation:

$$\text{Result} = R'_1 \bowtie R_2$$



2. Parallel Join Strategy and Example

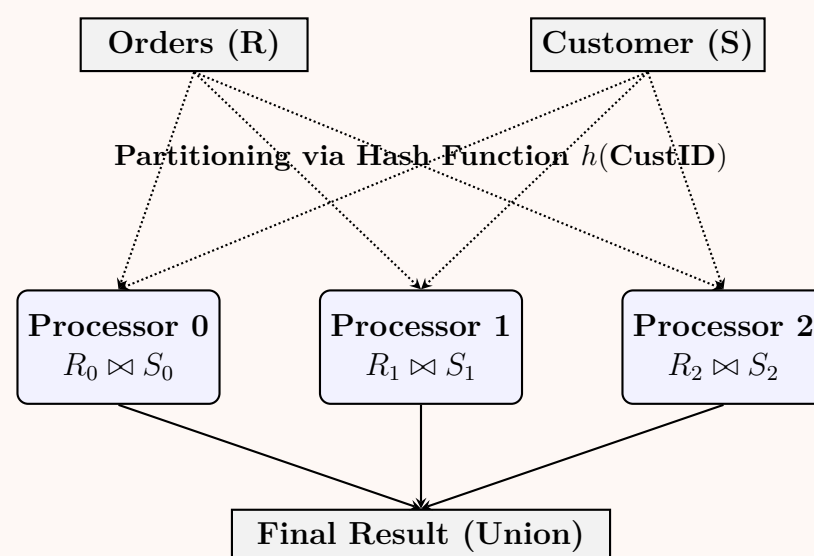
A **parallel join** leverages multiple processors (or nodes) to execute a join operation simultaneously, significantly reducing execution time for large datasets. The most common technique is the **Parallel Hash Join**.

Working Principle (Parallel Hash Join):

- **Partitioning Phase:** Both relations (R and S) are partitioned into n disjoint subsets ($R_1, R_2, \dots, R_n$ and $S_1, S_2, \dots, S_n$) using the same hash function $h()$ applied to the join attribute.
- **Distribution:** Partitions R_i and S_i are sent to Processor P_i . Because the same hash function is used, matching tuples are guaranteed to be routed to the same processor.
- **Join Phase:** Each processor P_i independently computes the local join $R_i \bowtie S_i$.
- **Merging Phase:** The final result is the union of all local joins.

Example Scenario: Let us join `Orders(OrderID, CustID, Amount)` and `Customer(CustID, Name)` on the attribute `CustID` using 3 processors.

- **Hash Function:** $h(\text{CustID}) = \text{CustID} \pmod{3}$.
- If R contains $\text{CustID} \in \{10, 11, 12, 13\}$, they are hashed to processors P_1, P_2, P_0, P_1 respectively.



Big Data & XML

Q1. From the following Document Type Definition (DTD), describe the structure of the data in XML and write a sample XML file using this DTD:

```
<!DOCTYPE STARS [
  <!ELEMENT STARS (STAR*)>
  <!ELEMENT STAR (NAME, ADDRESS+, MOVIES)>
  <!ELEMENT NAME (#PCDATA)>
  <!ELEMENT ADDRESS (STREET, CITY)>
  <!ELEMENT STREET (#PCDATA)>
  <!ELEMENT CITY (#PCDATA)>
]
```

```
<!ELEMENT MOVIES (MOVIE*)>
<!ELEMENT MOVIE (TITLE, YEAR)>
<!ELEMENT TITLE (#PCDATA)>
<!ELEMENT YEAR (#PCDATA)>
]>
```

(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Description of the XML Data Structure

The provided Document Type Definition (DTD) enforces a strict hierarchical structure for the XML document. The structure rules and cardinality constraints are defined as follows:

- Root Element:** The document must be enclosed within a single <STARS> element.
- STARS:** Contains zero or more <STAR> elements (denoted by the * modifier).
- STAR:** Each star must contain an exact sequence of three elements:
 - (a) Exactly one <NAME> element.
 - (b) One or more <ADDRESS> elements (denoted by the + modifier).
 - (c) Exactly one <MOVIES> element.
- ADDRESS:** Each address must contain exactly one <STREET> followed by exactly one <CITY>.
- MOVIES:** Acts as a container for zero or more <MOVIE> elements (denoted by the * modifier).
- MOVIE:** Each movie must contain exactly one <TITLE> followed by exactly one <YEAR>.
- Leaf Nodes:** The elements <NAME>, <STREET>, <CITY>, <TITLE>, and <YEAR> contain only parsed character data (#PCDATA), meaning they hold pure text content without nested XML tags.

2. Visual Hierarchical Representation

3. Sample XML Document

The following is a valid XML file that strictly adheres to the constraints defined in the provided DTD. It demonstrates a single star with multiple addresses and multiple movies.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE STARS [
  <!ELEMENT STARS (STAR*)>
  <!ELEMENT STAR (NAME, ADDRESS+, MOVIES)>
  <!ELEMENT NAME (#PCDATA)>
  <!ELEMENT ADDRESS (STREET, CITY)>
  <!ELEMENT STREET (#PCDATA)>
  <!ELEMENT CITY (#PCDATA)>
  <!ELEMENT MOVIES (MOVIE*)>
  <!ELEMENT MOVIE (TITLE, YEAR)>
  <!ELEMENT TITLE (#PCDATA)>
  <!ELEMENT YEAR (#PCDATA)>
]>

<STARS>
```

464

```

<STAR>
  <NAME>Tom Hanks</NAME>

  <ADDRESS>
    <STREET>123 Hollywood Blvd</STREET>
    <CITY>Los Angeles</CITY>
  </ADDRESS>
  <ADDRESS>
    <STREET>456 Ocean Drive</STREET>
    <CITY>Malibu</CITY>
  </ADDRESS>

  <MOVIES>
    <MOVIE>
      <TITLE>Forrest Gump</TITLE>
      <YEAR>1994</YEAR>
    </MOVIE>
    <MOVIE>
      <TITLE>Cast Away</TITLE>
      <YEAR>2000</YEAR>
    </MOVIE>
  </MOVIES>
</STAR>
</STARS>

```

Q2. Briefly describe the Map-Reduce framework. (5 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Definition

MapReduce is a highly scalable programming model and distributed processing framework originally developed by Google. It is designed to process and generate massive volumes of data (Big Data) efficiently by distributing the computational workload across a large cluster of commodity servers. It abstracts the complexities of parallel computing, network communication, and fault tolerance away from the developer.

2. Core Phases and Working Principle

The framework operates primarily on key-value pairs $\langle k, v \rangle$ and divides the execution into two distinct user-defined phases: **Map** and **Reduce**, separated by a system-managed **Shuffle and Sort** phase.

(a) **Input Splitting:** The massive input data (often stored in a distributed file system like HDFS) is divided into smaller, manageable blocks called *splits*.

(b) **Map Phase:**

- A *Mapper* function is applied independently and in parallel to each input split.
- It processes the input records and generates a set of *intermediate* key-value pairs.
- **Signature:** $\text{Map}(k_1, v_1) \rightarrow \text{list}\langle k_2, v_2 \rangle$

(c) **Shuffle and Sort (Framework-Managed):**

- The intermediate data generated by the Mappers is grouped by key.
- All values associated with the same intermediate key are collected and routed to the same Reducer.

(d) **Reduce Phase:**

- A *Reducer* function processes the grouped intermediate data.
- It iterates through the values associated with a specific key and aggregates, summarizes, or filters them to produce the final output.
- **Signature:** $\text{Reduce}(k_2, \text{list}\langle v_2 \rangle) \rightarrow \text{list}\langle k_3, v_3 \rangle$

3. Advantages of MapReduce

- **Scalability:** Can effortlessly scale from a single server to thousands of machines.
- **Fault Tolerance:** If a node crashes during execution, the framework automatically reschedules and re-executes its pending tasks on another healthy node.
- **Simplicity:** Developers only need to write the ‘Map’ and ‘Reduce’ logic; the framework handles data distribution, parallelization, and synchronization.
- **Data Locality:** Computations are pushed to the nodes where the data physically resides, minimizing expensive network bandwidth consumption.

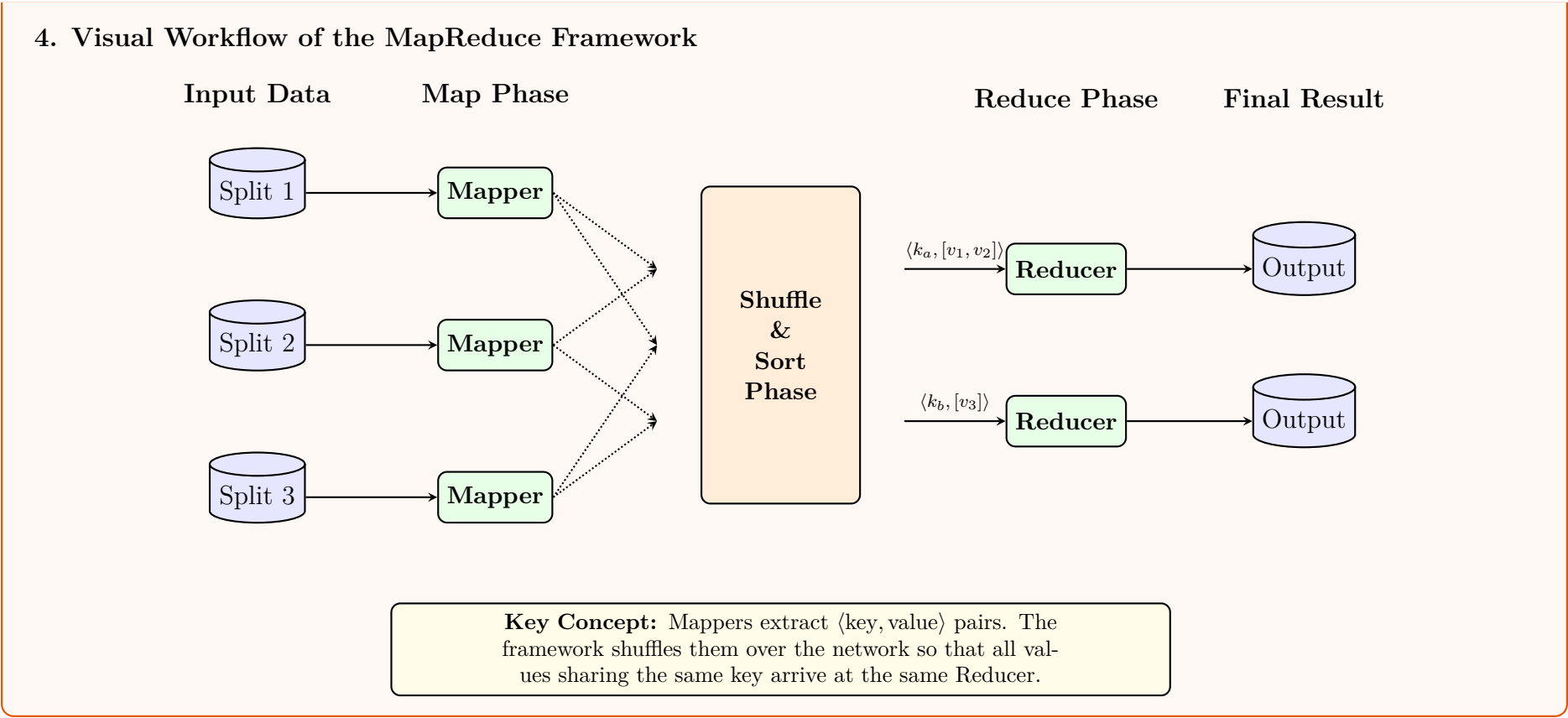


Table Partitioning

Q1. Suppose BIIS has a large number of records in its Course Registration table, accumulated over the last 20 years. Explain how table partitioning can be done on this table, and what benefits and drawbacks it could offer. **(10 marks)** [CSE 215 | L-2/T-2 | 2021–22]

Answer

1. Partitioning Strategies for the BIIS Course Registration Table

For a large **Course Registration** table containing 20 years of historical data, **Table Partitioning** is an excellent strategy to divide the large logical table into smaller, more manageable physical pieces (partitions) while keeping the same logical schema. Given the nature of the data, the following partitioning methods can be applied:

(a) **Range Partitioning (Most Recommended):** Since the data spans 20 years, partitioning by a chronological attribute, such as `Registration_Year` or `Semester_Date`, is highly effective.

• *Example:* Partition P_1 for records before 2010, P_2 for 2010–2015, P_3 for 2016–2020, and P_4 for 2021–Present.

(b) **List Partitioning:** Data can be partitioned based on categorical values, such as `Department_ID` or `Program_Type` (e.g., Undergraduate vs. Graduate).

(c) **Hash Partitioning:** If the goal is to evenly distribute I/O load rather than archive old data, hashing on `Student_ID` or `Course_ID` ensures an even distribution of records across all partitions.

(d) **Composite Partitioning:** A combination of the above, such as **Range-Hash** (Range partition by `Year`, and sub-partition each year by Hash on `Student_ID`).

2. Visual Representation of Range Partitioning

Logical Table:
COURSE_REGISTRATION (20 Years Data)

Range Partition Key: `Registration_Year`

Partition P1
Year < 2010

Partition P2
2010 – 2015

Partition P3
2016 – 2020

Partition P4
2021 – 2025
(Active Data)

Separate Physical Storage Spaces (Tablespaces)

3. Benefits of Partitioning

• **Partition Pruning (Performance):** Queries filtering by the partition key (e.g., fetching registrations for the year 2023) will only scan the relevant physical partition (e.g., P_4), completely bypassing 15+ years of irrelevant data.

466

- **Manageability & Archiving:** Old data (e.g., 2004–2008) can be easily archived or dropped by simply dropping or detaching a specific partition without executing a massive, resource-intensive `DELETE` statement.
- **Availability:** Maintenance operations (like rebuilding indexes or taking backups) can be performed on individual partitions. If one partition becomes corrupted or offline, the rest of the table remains accessible.
- **Load Balancing:** Using physical storage layout, partitions can be placed on different disks to balance I/O operations.

4. Drawbacks of Partitioning

- **Query Overhead (Full Partition Scans):** If a query does *not* include the partitioning key in its `WHERE` clause (e.g., searching for a specific `Student_ID` without specifying a year), the DBMS must scan *all* partitions, which can degrade performance compared to a standard indexed table.
- **Maintenance Complexity:** The Database Administrator (DBA) must actively manage partition creation for future years. If an automated script fails to create a partition for a new year, `INSERT` operations will fail.
- **Data Skew:** If partitions are not designed properly (e.g., list partitioning where one department has 100× more students than another), some partitions will become massive bottlenecks while others remain practically empty.
- **Index Overhead:** Maintaining global indexes across a heavily partitioned table can be complex. Updating a row's partition key requires physically moving the row from one partition to another, known as *row movement*, which incurs heavy I/O cost.